

// File: Sources/LingXiCore/Modules/Agent/AgentRuntime.swift

```

import Foundation
import LingXiProtocol

/// Agent 最小编排入口（Session 化）。
/// sendMessage: append user → SessionContextBuilder 构建模型输入 → 推理。
/// 高频 delta 只走 DMA + 本轮内存 buffer 聚合；
/// Session 在 turn 完成时一次性写入最终 assistant content。
/// 本类型不解析任何 Provider JSON。
public actor AgentRuntime {
    private let store: any SessionStore
    private let contextEngine: L1ContextEngine
    private let modelBus: ModelBus
    private let dataPlane: DataPlane
    private let toolRuntime: ToolRuntime
    private let questions: QuestionRuntime
    private let permissions: PermissionEngine
    private let performanceStore: PerformanceStore
    private let contextPager: ContextPager
    private let projectScanner: ProjectScanner
    private let compactor: ContextCompactor
    private let budgetPlanner: ContextBudgetPlanner
    private let persistence: SQLitePersistenceStore?
    private let eventSink: @Sendable (CoreEvent) async -> Void
    private let interactive: Bool
    private let diagnosticsEnabled: Bool
    private var runtimes: [SessionID: SessionRuntime] = [:]
    private let modelResolver: SubagentModelResolver
    private let scheduler: AgentRunScheduler
    private let limits: SubagentRuntimeLimits
    private var behaviorProfile: AgentBehaviorProfile
    private var behaviorInstructionsEnabled: Bool
    private let behaviorSystemContext: @Sendable (AgentBehaviorProfile, SubagentExecutionProfile?) -> String?
    private let deadlinePolicy: ExecutionDeadlinePolicy
    private let restoreScheduler: SessionRestoreScheduler?
    private let diagnostics: RuntimeDiagnosticsStore?
    private var runs: [AgentRunID: AgentRunInfo] = [:]
    private var results: [AgentRunID: SubagentResult] = [:]
    private var executionProfiles: [AgentRunID: SubagentExecutionProfile] = [:]
    private var activeSessions: Set<SessionID> = []
    private var runDeadlines: [AgentRunID: ExecutionDeadline] = [:]
    private var resultWaiters: [AgentRunID: [CheckedContinuation<SubagentResult, Error>]] = [:]
    private var shuttingDown = false
    private let cacheController: ContextCacheController
    private let maxAgentLoopSteps: Int

    init(
        store: any SessionStore,
        contextEngine: L1ContextEngine,
        modelBus: ModelBus,
        dataPlane: DataPlane,
        toolRuntime: ToolRuntime,
        questions: QuestionRuntime,
        permissions: PermissionEngine,
        performanceStore: PerformanceStore,
        contextPager: ContextPager,
        projectScanner: ProjectScanner,
        eventSink: @escaping @Sendable (CoreEvent) async -> Void,
        compactor: ContextCompactor = ContextCompactor(),
        budgetPlanner: ContextBudgetPlanner = ContextBudgetPlanner(),
        persistence: SQLitePersistenceStore? = nil,
        cacheController: ContextCacheController? = nil,
        interactive: Bool = false,
        diagnosticsEnabled: Bool = false,
        modelResolver: SubagentModelResolver,
    ) {
        self.store = store
        self.contextEngine = contextEngine
        self.modelBus = modelBus
        self.dataPlane = dataPlane
        self.toolRuntime = toolRuntime
        self.questions = questions
        self.permissions = permissions
        self.performanceStore = performanceStore
        self.contextPager = contextPager
        self.projectScanner = projectScanner
        self.eventSink = eventSink
        self.compactor = compactor
        self.budgetPlanner = budgetPlanner
        self.persistence = persistence
        self.cacheController = cacheController
        self.interactive = interactive
        self.diagnosticsEnabled = diagnosticsEnabled
        self.modelResolver = modelResolver
    }
}

```

```

limits: SubagentRuntimeLimits = SubagentRuntimeLimits(),
scheduler: AgentRunScheduler? = nil,
behaviorProfile: AgentBehaviorProfile = .build,
behaviorInstructionsEnabled: Bool = false,
behaviorSystemContext: @escaping @Sendable (AgentBehaviorProfile, SubagentExecutionProfile?) -> String? = { _, _ in nil },
maxAgentLoopSteps: Int = 32,
deadlinePolicy: ExecutionDeadlinePolicy = ExecutionDeadlinePolicy(),
restoreScheduler: SessionRestoreScheduler? = nil,
diagnostics: RuntimeDiagnosticsStore? = nil
) {
    self.store = store
    self.contextEngine = contextEngine
    self.modelBus = modelBus
    self.dataPlane = dataPlane
    self.toolRuntime = toolRuntime
    self.questions = questions
    self.permissions = permissions
    self.performanceStore = performanceStore
    self.contextPager = contextPager
    self.projectScanner = projectScanner
    self.eventSink = eventSink
    self.compactor = compactor
    self.cacheController = cacheController ?? ContextCacheController(contextPager: contextPager, scanner: projectScanner, compactor:
↳ compactor)
    self.budgetPlanner = budgetPlanner
    self.persistence = persistence
    self.interactive = interactive
    self.diagnosticsEnabled = diagnosticsEnabled
    self.modelResolver = modelResolver
    self.limits = limits
    self.scheduler = scheduler ?? AgentRunScheduler(limits: limits)
    self.behaviorProfile = behaviorProfile
    self.behaviorInstructionsEnabled = behaviorInstructionsEnabled
    self.behaviorSystemContext = behaviorSystemContext
    self.maxAgentLoopSteps = maxAgentLoopSteps
    self.deadlinePolicy = deadlinePolicy
    self.restoreScheduler = restoreScheduler
    self.diagnostics = diagnostics
}

// MARK: - Session 生命周期

public func createSession() async throws -> SessionID {
    let session = try await store.create(kind: .primary, parentSessionID: nil, rootSessionID: nil, spawnedByRunID: nil,
↳ spawnedByToolCallID: nil, title: nil)
    await cacheController.resetSession(session.id)
    runtimes[session.id] = makeRuntime(for: session.id)
    await eventSink(.sessionCreated(session.id))
    return session.id
}

public func restore() async throws {
    guard !shuttingDown else { return }
    try await compactor.restoreDerived()
    guard !shuttingDown else { return }
    guard let persistence else { return }
    let persistedRuns = try await persistence.loadAgentRuns()
    for persisted in persistedRuns {
        var run = persisted
        if !run.status.isTerminal {
            let calls = try await persistence.toolBatches(sessionID: run.sessionID).flatMap(\.toolCallStates).filter { $0.provenance.
↳ agentRunID == run.runID }
            if !calls.isEmpty {
                let waiting = calls.contains { $0.state == .waitingForHuman || ($0.request != nil && $0.reply == nil) }
                run = AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID,

```

```

↪ rootRunID: run.rootRunID, agentKind: run.agentKind, status: waiting ? .waitingForUser : .recoveryRequired, modelSelection: run.
↪ modelSelection, startedAt: run.startedAt, latestActivityAt: .now, usage: run.usage, title: run.title)
    if waiting {
        activeSessions.insert(run.sessionID)
    }
    try await persistence.saveAgentRun(run)
} else {
    run = AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID,
↪ rootRunID: run.rootRunID, agentKind: run.agentKind, status: .recoveryRequired, modelSelection: run.modelSelection, startedAt: run.
↪ startedAt, finishedAt: .now, latestActivityAt: .now, error: CoreError(code: .toolCancelled, message: "Core 重启, 运行需要恢复"),
↪ usage: run.usage, title: run.title)
    try await persistence.saveAgentRun(run)
}
}
runs[run.runID] = run
if let profile = try await persistence.agentRunProfile(run.runID) { executionProfiles[run.runID] = profile }
if let result = try await persistence.agentRunResult(run.runID) { results[run.runID] = result }
}
for run in runs.values where run.status == .waitingForUser {
    guard !shuttingDown else { return }
    do {
        _ = try await runtime(for: run.sessionID, run: run)
    } catch {
        logDiagnostic("Failed to restore runtime for run=\(run.runID.rawValue) session=\(run.sessionID.rawValue): \((error)")
        activeSessions.remove(run.sessionID)
        let failedRun = AgentRunInfo(
            runID: run.runID,
            sessionID: run.sessionID,
            projectID: run.projectID,
            parentRunID: run.parentRunID,
            rootRunID: run.rootRunID,
            agentKind: run.agentKind,
            status: .recoveryRequired,
            modelSelection: run.modelSelection,
            startedAt: run.startedAt,
            finishedAt: .now,
            latestActivityAt: .now,
            error: (error as? CoreError) ?? CoreError(code: .provider, message: "恢复会话模型失败: \((error.localizedDescription)"),
            usage: run.usage,
            title: run.title
        )
        runs[run.runID] = failedRun
        try? await persistence.saveAgentRun(failedRun)
    }
}
}
await restoreScheduler?.markReady()
}

public func shutdown() async {
    shuttingDown = true
    for runtime in runtimes.values { await runtime.shutdown() }
}

public func listSessions() async throws -> [SessionInfo] {
    try await store.listSessions().map { $0.toInfo() }
}

public func listChildSessions(_ parentSessionID: SessionID) async throws -> [SessionInfo] {
    try await store.listSessions().filter { $0.parentSessionID == parentSessionID }.map { $0.toInfo() }
}

public func listAgentRuns(_ sessionID: SessionID) -> [AgentRunInfo] {
    runs.values.filter { $0.sessionID == sessionID }.sorted { $0.latestActivityAt < $1.latestActivityAt }
}

```

```

public func allAgentRuns() -> [AgentRunInfo] {
    runs.values.sorted { $0.latestActivityAt < $1.latestActivityAt }
}

public func orphanRunIDs() async -> [AgentRunID] {
    let schedulerState = await scheduler.snapshot()
    let active = Set(schedulerState.active + schedulerState.queued)
    return runs.values.filter {
        !$0.status.isTerminal && $0.status != .waitingForUser && $0.status != .waitingForTool && !active.contains($0.runID)
    }.map{\.runID}.sorted { $0.rawValue < $1.rawValue }
}

public func agentRun(_ runID: AgentRunID) throws -> AgentRunInfo {
    guard let run = runs[runID] else { throw CoreError(code: .agentRunNotFound, message: "AgentRun 不存在: \"(runID.rawValue)\"") }
    return run
}

public func agentRun(_ runID: AgentRunID, requester: AgentRunID) throws -> AgentRunInfo {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    return run
}

public func resumeAgentRun(_ runID: AgentRunID) async throws -> AgentRunInfo {
    let run = try agentRun(runID)
    guard !run.status.isTerminal else { return run }
    guard !shuttingDown else { throw CoreError(code: .notReady, message: "Agent Runtime 正在关闭") }
    _ = try await runtime(for: run.sessionID, run: run)
    return try agentRun(runID)
}

public func agentRunResult(_ runID: AgentRunID) throws -> SubagentResult {
    guard let result = results[runID] else { throw CoreError(code: .agentRunNotFound, message: "AgentRun 尚无稳定结果: \"(runID.rawValue)\"") }
    return result
}

public func agentRunResult(_ runID: AgentRunID, requester: AgentRunID) throws -> SubagentResult {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    return try agentRunResult(runID)
}

public func waitForAgentRunResult(_ runID: AgentRunID) async throws -> SubagentResult {
    if let result = results[runID] { return result }
    _ = try agentRun(runID)
    return try await withCheckedThrowingContinuation { continuation in
        resultWaiters[runID, default: []].append(continuation)
    }
}

/// Workflows share spawn, deadline, model/profile resolution, and the canonical child Run scheduler.
public func makeWorkflowRuntime() -> WorkflowRuntime {
    WorkflowRuntime(persistence: persistence, executor: { [weak self] _, task, parent, started in
        guard let self else { return WorkflowTaskCompletion(error: CoreError(code: .notReady, message: "Agent 未就绪")) }
        do {
            let rendered = [task.role.map { "Role: \"($0)\" }, task.instructions.map { "Instructions: \"($0)\" }, task.context.map { "
↳ Context: \"($0)\" }, task.task].compactMap { $0 }.joined(separator: "\n\n")
            let (childSessionID, run) = try await self.spawn(parentSessionID: parent.parentSessionID, parentRunID: parent.parentRunID,
↳ task: rendered, title: task.title, modelSelection: task.modelSelection, profile: task.executionProfile)
            await started(WorkflowTaskProvenance(parentSessionID: parent.parentSessionID, parentRunID: parent.parentRunID,
↳ childSessionID: childSessionID, childRunID: run.runID))
            return WorkflowTaskCompletion(result: try await self.waitForAgentRunResult(run.runID))
        } catch let error as CoreError {
            return WorkflowTaskCompletion(error: error)
        }
    })
}

```

```

        } catch {
            return WorkflowTaskCompletion(error: CoreError(code: .provider, message: String(describing: error)))
        }
    }, diagnostics: diagnostics)
}

public func agentTree(_ rootSessionID: SessionID) async throws -> AgentTreeNode {
    let sessions = try await store.listSessions()
    func node(_ session: Session) -> AgentTreeNode {
        let latest = runs.values.filter { $0.sessionID == session.id }.max { $0.latestActivityAt < $1.latestActivityAt }
        return AgentTreeNode(session: session.toInfo(), latestRun: latest, children: sessions.filter { $0.parentSessionID == session.id }
        ↪ ).map(node)
    }
    guard let root = sessions.first(where: { $0.id == rootSessionID }) else { throw CoreError(code: .sessionNotFound, message: "Session
    ↪ 不存在") }
    return node(root)
}

public func sessionSnapshot(_ id: SessionID) async throws -> LegacySessionSnapshot {
    try await store.session(id).toSnapshot()
}

public func renameSession(_ id: SessionID, title: String?) async throws -> SessionInfo {
    let normalized = title?.trimmingCharacters(in: .whitespacesAndNewlines)
    return try await store.updateTitle(id, title: normalized?.isEmpty == true ? nil : normalized).toInfo()
}

public func selectModel(_ selection: ModelSelection, assembly: ModelRuntimeAssembly? = nil) async throws {
    try await modelResolver.setDefaultSelection(selection, assembly: assembly)
}

public func contextSnapshot(_ id: SessionID) async -> L1ContextSnapshot? {
    await contextEngine.latestSnapshot(for: id)
}

public func ensureContextSnapshot(_ id: SessionID) async throws -> L1ContextSnapshot {
    if let snapshot = await contextEngine.latestSnapshot(for: id) { return snapshot }
    let session = try await store.session(id)
    let entries = await contextEngine.entries(for: session)
    let estimatedTokens = ConservativeTokenEstimator().estimate(entries: entries)
    return await contextEngine.snapshot(for: session, activeEntries: entries, estimatedTokens: estimatedTokens)
}

public func performance(_ id: SessionID) async -> TurnPerformanceReport? {
    await performanceStore.report(for: id)
}

public func currentBehaviorProfile() -> AgentBehaviorProfile { behaviorProfile }

public func setBehaviorProfile(_ profile: AgentBehaviorProfile) {
    behaviorProfile = profile
    behaviorInstructionsEnabled = true
}

public func projectCache(sessionID: SessionID? = nil) async -> ProjectCacheDebugSnapshot {
    let metrics = await contextPager.debugMetrics(projectRoot: projectScanner.root)
    let derived = if let sessionID {
        await compactor.cacheMetrics(sessionID: sessionID)
    } else {
        await compactor.cacheMetrics()
    }
    let sessionL2Pages = if let sessionID {
        (await contextEngine.latestSnapshot(for: sessionID))?.metrics.projectPageCount ?? 0
    } else {
        metrics.l2Pages
    }
}

```

```

    }
    let sessionL2Characters = if let sessionID {
        (await contextEngine.latestSnapshot(for: sessionID))?.metrics.projectCharacterCount ?? 0
    } else {
        metrics.l2Characters
    }
    return ProjectCacheDebugSnapshot(
        l2Pages: sessionL2Pages,
        l2Characters: sessionL2Characters,
        l2HitRate: metrics.l2Lookups == 0 ? nil : Double(metrics.l2Hits) / Double(metrics.l2Lookups),
        l3Pages: metrics.l3Pages,
        staleRebuilds: metrics.staleRebuilds,
        symbolCount: metrics.symbolCount,
        symbolIndexedFiles: metrics.symbolIndexedFiles,
        referenceCount: metrics.referenceCount,
        dependencyCount: metrics.dependencyCount,
        sessionL2DerivedPages: derived.l2Pages,
        derivedL3Pages: derived.l3Pages,
        derivedPageOutCount: derived.pageOutCount,
        derivedPageInCount: derived.pageInCount,
        historicalToolEvidencePages: derived.historicalToolPages,
        derivedL3Hits: derived.l3Hits,
        sessionL2DerivedHits: derived.l2Hits,
        sessionL2DerivedPromotions: derived.l2Promotions
    )
}

public func latestContextManifest(_ sessionID: SessionID) async -> ProviderContextManifest? {
    await runtimes[sessionID]?.latestContextManifest
}

public func compact(_ sessionID: SessionID) async throws -> CompactSessionResponse {
    try await runtime(for: sessionID).compactNow()
}

public func cacheMetrics(_ sessionID: SessionID) async -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int,
    ↪ historicalToolPages: Int, l3Hits: Int, l2Hits: Int, l2Promotions: Int)? {
    await runtimes[sessionID]?.cacheMetrics()
}

public func contextUnitStates(_ sessionID: SessionID) async -> [ContextUnitDebugSnapshot] {
    await runtimes[sessionID]?.contextUnitStates() ?? []
}

// MARK: - 对话

/// 在 Session 中发起一轮对话，返回该轮的 DMA 通道。
public func sendMessage(_ sessionID: SessionID, _ content: String) async throws -> OpenedStream {
    guard !activeSessions.contains(sessionID) else { throw CoreError(code: .turnAlreadyRunning, message: "该 Session 已有进行中的对话轮次
    ↪ ") }
    // Preserve the established contract: an unavailable provider still records the user turn.
    if modelBus.gateway.modelID == nil { return try await runtime(for: sessionID).startTurn(content) }
    do {
        // Reserve before the first await so concurrent callers cannot create a second lane.
        activeSessions.insert(sessionID)
        let session = try await store.session(sessionID)
        let run = try await createRun(session: session, parentRunID: nil, requestedModel: nil, title: session.title, profile:
    ↪ behaviorProfile.executionProfile)
        return try await runtime(for: sessionID, run: run).startTurn(content)
    } catch {
        activeSessions.remove(sessionID)
        throw error
    }
}

```

```

public fun spawn(parentSessionID: SessionID, parentRunID: AgentRunID, task: String, title: String? = nil, modelSelection:
↳ ModelSelection? = nil, profile: SubagentExecutionProfile? = nil, toolCallID: ToolCallID? = nil) async throws -> (SessionID,
↳ AgentRunInfo) {
    let parent = try await store.session(parentSessionID)
    let parentRun = try agentRun(parentRunID)
    guard parentRun.sessionID == parentSessionID, !parentRun.status.isTerminal else { throw CoreError(code: .toolArgumentInvalid,
↳ message: "Parent AgentRun 与 Session 不匹配或已结束") }
    let depth = try await depth(of: parent)
    guard depth < limits.maxSubagentDepth else { throw CoreError(code: .subagentDepthExceeded, message: "Subagent 最大深度已达到") }
    guard runs.values.filter({ $0.rootRunID == parentRun.rootRunID }).count < limits.maxTotalRunsPerRootRun else {
        throw CoreError(code: .subagentDepthExceeded, message: "单个 Root AgentRun 的运行总数已达到上限")
    }
    let normalizedProfile = try normalized(profile: profile, parent: executionProfiles[parentRunID])
    let requestedModel = normalizedProfile?.modelSelection ?? modelSelection
    let resolved = try await modelResolver.resolve(requestedModel, subagent: true)

    let endpointProfile = resolved.assembly.contextProfile
    let requestedWindow = normalizedProfile?.contextProfile.flatMap(Int.init)
    let effectiveWindow = min(endpointProfile.contextWindowTokens, requestedWindow ?? endpointProfile.contextWindowTokens)
    let prospectiveProfile = ModelContextProfile(
        contextWindowTokens: effectiveWindow,
        maxOutputTokens: endpointProfile.maxOutputTokens,
        recommendedOutputReserveTokens: endpointProfile.recommendedOutputReserveTokens,
        source: endpointProfile.source
    )
    let childTools = await toolRuntime.availableDefinitions(sessionID: parentSessionID, interactive: interactive, executionProfile:
↳ normalizedProfile)
    let estimator = ConservativeTokenEstimator()
    let toolTokens = estimator.estimate(tools: childTools)
    let preferred = normalizedProfile?.budgetProfile.flatMap(Int.init)
    let planner = preferred.map { budgetPlanner.with(preferredActiveTokens: $0) } ?? budgetPlanner
    let prospectiveBudget = planner.plan(profile: prospectiveProfile, toolTokens: toolTokens)
    let initialMandatoryTokens = await contextEngine.initialMandatoryTokens(task: task, estimator: estimator)

    guard prospectiveBudget.hardInputLimit >= initialMandatoryTokens, prospectiveBudget.hardInputLimit > 0 else {
        throw CoreError(
            code: .contextProfileNotViable,
            message: "contextProfile 不可行: hardInputLimit=\(prospectiveBudget.hardInputLimit) 不足以容纳初始 mandatory context (\(
↳ initialMandatoryTokens) tokens)"
        )
    }

    let child: Session
    var run: AgentRunInfo
    if let persistence {
        let root = try await persistence.mainRootBinding()
        child = Session(id: SessionID(UUID().uuidString), createdAt: .now, kind: .subagent, parentSessionID: parent.id, rootSessionID:
↳ parent.rootSessionID, spawnedByRunID: parentRunID, spawnedByToolCallID: toolCallID, title: title, projectID: persistence.projectID,
↳ cwdRootBindingID: root.id)
        let runID = AgentRunID(UUID().uuidString)
        run = AgentRunInfo(runID: runID, sessionID: child.id, projectID: child.projectID, parentRunID: parentRunID, rootRunID: parentRun
↳ .rootRunID, agentKind: .subagent, status: .starting, modelSelection: resolved.selection, startedAt: .now, latestActivityAt: .now,
↳ title: title)
        try await persistence.createChildSessionAndRun(child, run: run, profile: normalizedProfile)
        runs[runID] = run
        runDeadlines[runID] = deadlinePolicy.deadline(for: .subagent, requested: normalizedProfile?.timeoutSeconds.map { .seconds($0) },
↳ parent: runDeadlines[parentRunID])
        if let normalizedProfile { executionProfiles[runID] = normalizedProfile }
    } else {
        child = try await store.create(kind: .subagent, parentSessionID: parent.id, rootSessionID: parent.rootSessionID, spawnedByRunID:
↳ parentRunID, spawnedByToolCallID: toolCallID, title: title)
        run = try await createRun(session: child, parentRunID: parentRunID, requestedModel: nil, resolvedModel: resolved, title: title,
↳ profile: normalizedProfile, emitEvent: false)
    }
    do {

```

```

        let runID = run.runID
        let status = await scheduler.submit(runID: runID) { [weak self] in await self?.runChild(runID: runID, task: task) }
        if status == .queued {
            run = updated(run, status: .queued)
            runs[run.runID] = run
            try await persistence?.saveAgentRun(run, profile: normalizedProfile)
        }
        await eventSink(.childSessionCreated(child.toInfo()))
        await eventSink(.agentRunStarted(run))
        await eventSink(.subagentSpawned(run))
        if status == .queued { await eventSink(.agentRunQueued(run)) }
        return (child.id, run)
    } catch {
        logDiagnostic("spawn failed parentSession=\\(parentSessionID.rawValue) parentRun=\\(parentRunID.rawValue) childSession=\\(child.id.
↳ rawValue): \\(error)")
        let childRuns = runs.values.filter { $0.sessionID == child.id }
        for r in childRuns {
            runs.removeValue(forKey: r.runID)
            runDeadlines.removeValue(forKey: r.runID)
            executionProfiles.removeValue(forKey: r.runID)
            try? await persistence?.deleteAgentRun(r.runID)
        }
        try? await store.deleteSession(child.id)
        throw error
    }
}

public func cancelAgentRun(_ runID: AgentRunID, descendants: Bool = true) async throws {
    _ = try agentRun(runID)
    let targets = descendants ? runs.values.filter { isDescendant($0, of: runID) || $0.runID == runID }.map(\\.runID) : [runID]
    for id in targets {
        await scheduler.cancel(id)
        let cancelled = await ProviderActivityRegistry.shared.cancelRun(id)
        for snapshot in cancelled {
            await eventSink(.providerActivityChanged(snapshot))
        }
        if let runtime = runtimes[runs[id]?.sessionID ?? SessionID("")] { await runtime.shutdown() }
        await finishRun(id, status: .cancelled, text: nil, usage: nil, error: CoreError(code: .toolCancelled, message: "AgentRun 已取消
↳ "))
    }
}

public func cancelSession(_ sessionID: SessionID) async {
    activeSessions.remove(sessionID)
    if let runtime = runtimes[sessionID] {
        await runtime.cancelCurrentTurn()
    }
}

/// 撤回时彻底重置并释放该 Session 的运行实例，消除脏缓存与残留任务，并递归取消所有子 Subagent
public func resetSessionForRevert(_ sessionID: SessionID) async {
    await cancelDescendants(parentSessionID: sessionID)
    activeSessions.remove(sessionID)
    if let runtime = runtimes.removeValue(forKey: sessionID) {
        await runtime.shutdown()
    }
}

public func cancelDescendants(parentSessionID: SessionID) async {
    let matchingRuns = runs.values.filter { run in
        run.sessionID == parentSessionID
    }
    for run in matchingRuns {
        try? await cancelAgentRun(run.runID, descendants: true)
    }
}

```



```

    if let allSessions = try? await store.listSessions() {
        let childSessions = allSessions.filter { $0.parentSessionID == parentSessionID || $0.rootSessionID == parentSessionID }
        for child in childSessions {
            activeSessions.remove(child.id)
            if let runtime = runtimes.removeValue(forKey: child.id) {
                await runtime.shutdown()
            }
        }
    }
}

public func cancelAgentRun(_ runID: AgentRunID, requester: AgentRunID) async throws {
    let run = try agentRun(runID)
    try requireSameTree(run, requester: requester)
    try await cancelAgentRun(runID)
}

public func continueChild(sessionID: SessionID, parentRunID: AgentRunID, content: String) async throws -> AgentRunInfo {
    let session = try await store.session(sessionID)
    guard session.kind == .subagent else { throw CoreError(code: .toolArgumentInvalid, message: "只能继续 Child Session") }
    let parentRun = try agentRun(parentRunID)
    guard let spawnedBy = session.spawnedByRunID, let original = runs[spawnedBy], original.rootRunID == parentRun.rootRunID else { throw
↳ CoreError(code: .permissionDenied, message: "Child Session 不属于当前 Agent 树") }
    guard !activeSessions.contains(sessionID) else { throw CoreError(code: .turnAlreadyRunning, message: "该 Child Session 已有进行中的
↳ AgentRun") }
    activeSessions.insert(sessionID)
    var run: AgentRunInfo
    do {
        run = try await createRun(session: session, parentRunID: parentRunID, requestedModel: nil, title: session.title)
    } catch {
        activeSessions.remove(sessionID)
        throw error
    }
    let runID = run.runID
    let status = await scheduler.submit(runID: runID) { [weak self] in await self?.runChild(runID: runID, task: content) }
    if status == .queued { run = updated(run, status: .queued); runs[runID] = run; try await persistence?.saveAgentRun(run) }
    if status == .queued { await eventSink(.agentRunQueued(run)) }
    await diagnostics?.record(kind: .subagent, event: status == .queued ? "agentRun.queued" : "agentRun.started", sessionID: run.
↳ sessionID, runID: run.runID, rootRunID: run.rootRunID, parentRunID: run.parentRunID, metadata: ["agentKind": run.agentKind.rawValue
↳ ])
    return run
}

public func markWaitingForQuestion(_ request: QuestionRequest, waiting: Bool) async {
    guard let runID = request.originRunID, let run = runs[runID], !run.status.isTerminal else { return }
    let updated = updated(run, status: waiting ? .waitingForUser : .running)
    runs[runID] = updated
    try? await persistence?.saveAgentRun(updated, profile: executionProfiles[runID])
    await eventSink(.agentRunStatusChanged(updated))
}

// MARK: - Private

private func makeRuntime(for sessionID: SessionID, run: AgentRunInfo? = nil, modelBus overrideBus: ModelBus? = nil, rootSessionID:
↳ SessionID? = nil) -> SessionRuntime {
    SessionRuntime(
        store: store,
        sessionID: sessionID,
        modelBus: overrideBus ?? modelBus,
        dataPlane: dataPlane,
        contextEngine: contextEngine,
        toolRuntime: toolRuntime,
        questions: questions,
        permissions: permissions,
        performanceStore: performanceStore,

```

```

    contextPager: contextPager,
    projectScanner: projectScanner,
    eventSink: eventSink,
    compactor: compactor,
    budgetPlanner: budgetPlanner,
    persistence: persistence,
    cacheController: cacheController,
    interactive: interactive,
    diagnosticsEnabled: diagnosticsEnabled,
    runID: run?.runID,
    rootRunID: run?.rootRunID,
    parentRunID: run?.parentRunID,
    rootSessionID: rootSessionID ?? sessionID,
    parentSessionID: run?.parentRunID.flatMap { runs[$0]?.sessionID },
    executionProfile: run.flatMap { executionProfiles[$0.runID] },
    systemContext: {
        let profile = run.flatMap { executionProfiles[$0.runID]?.behaviorProfile } ?? .build
        let exec = run.flatMap { executionProfiles[$0.runID] }
        let isFullAccess = exec?.permissionProfile == "fullAccess"
        return (behaviorInstructionsEnabled || profile != .build || isFullAccess) ? behaviorSystemContext(profile, exec) :
↪ behaviorSystemContext(.build, nil)
    }(),
    systemContextAtBeginning: behaviorInstructionsEnabled || run.flatMap { executionProfiles[$0.runID]?.behaviorProfile }.map { $0
↪ != .build } ?? false,
    maxAgentLoopSteps: maxAgentLoopSteps,
    runObserver: { [weak self] status, text, usage, error, terminalTrace in
        guard let run else { return }
        await self?.finishRun(run.runID, status: status, text: text, usage: usage, error: error, terminalTrace: terminalTrace)
    },
    deadlinePolicy: deadlinePolicy,
    restoreScheduler: restoreScheduler,
    diagnostics: diagnostics
)
}

private func runtime(for sessionID: SessionID, run: AgentRunInfo? = nil) async throws -> SessionRuntime {
    guard !shuttingDown else { throw CoreError(code: .notReady, message: "Agent 已关闭") }
    if let run {
        let resolved = try await modelResolver.resolve(run.modelSelection, subagent: run.agentKind == .subagent)
        let bus = ModelBus(gateway: ModelGateway(assembly: resolved.assembly, reasoning: run.modelSelection.reasoning, deadlinePolicy:
↪ deadlinePolicy))
        let session = try await store.session(sessionID)
        let runtime = makeRuntime(for: sessionID, run: run, modelBus: bus, rootSessionID: session.rootSessionID)
        runtimes[sessionID] = runtime
        try await runtime.restore()
        return runtime
    }
    if let runtime = runtimes[sessionID] { return runtime }
    _ = try await store.session(sessionID)
    let runtime = makeRuntime(for: sessionID)
    runtimes[sessionID] = runtime
    try await runtime.restore()
    return runtime
}

private func createRun(session: Session, parentRunID: AgentRunID?, requestedModel: ModelSelection? = nil, resolvedModel: (selection:
↪ ModelSelection, assembly: ModelRuntimeAssembly)? = nil, title: String?, profile: SubagentExecutionProfile? = nil, emitEvent: Bool =
↪ true) async throws -> AgentRunInfo {
    let effectiveProfile = profile ?? (session.kind == .primary ? behaviorProfile.executionProfile : nil)
    let resolved = try await (resolvedModel != nil ? resolvedModel! : modelResolver.resolve(requestedModel, subagent: session.kind == .
↪ subagent))
    let id = AgentRunID(UUID().uuidString)
    let root = parentRunID.flatMap { runs[$0]?.rootRunID } ?? id
    var selection = resolved.selection
    if selection.reasoning == nil, session.reasoningEffort != .auto {

```

```

        selection = ModelSelection(
            providerID: selection.providerID,
            accountID: selection.accountID,
            profileID: selection.profileID,
            modelID: selection.modelID,
            reasoning: session.reasoningEffort.rawValue,
            contextProfile: selection.contextProfile
        )
    }

    let run = AgentRunInfo(runID: id, sessionID: session.id, projectID: session.projectID, parentRunID: parentRunID, rootRunID: root,
↪ agentKind: session.kind, status: .starting, modelSelection: selection, startedAt: .now, latestActivityAt: .now, title: title)
    runs[id] = run
    runDeadlines[id] = deadlinePolicy.deadline(for: session.kind == .subagent ? .subagent : .agentRun, requested: effectiveProfile?.
↪ timeoutSeconds.map { .seconds($0) }, parent: parentRunID.flatMap { runDeadlines[$0] })
    if let effectiveProfile { executionProfiles[id] = effectiveProfile }
    do {
        try await persistence?.saveAgentRun(run, profile: effectiveProfile)
    } catch {
        runs.removeValue(forKey: id)
        runDeadlines.removeValue(forKey: id)
        executionProfiles.removeValue(forKey: id)
        throw error
    }
    if emitEvent {
        await eventSink(.agentRunStarted(run))
    }
    return run
}

private func runChild(runID: AgentRunID, task: String) async {
    guard let run = runs[runID] else { return }
    do {
        try await consumeChildTurn(run: run, task: task)
    } catch let error as CoreError {
        await finishRun(runID, status: [.commandTimedOut, .idleTimedOut].contains(error.code) ? .timedOut : .failed, text: nil, usage:
↪ nil, error: error)
    } catch {
        await finishRun(runID, status: .failed, text: nil, usage: nil, error: CoreError(code: .provider, message: String(describing:
↪ error)))
    }
}

private func consumeChildTurn(run: AgentRunInfo, task: String) async throws {
    let stream = try await runtime(for: run.sessionID, run: run).startTurn(task)
    for try await _ in stream.chunks {}
}

private func finishRun(_ runID: AgentRunID, status: AgentRunStatus, text: String?, usage: ModelUsage?, error: CoreError?, terminalTrace:
↪ AgentTerminalTrace? = nil) async {
    guard let old = runs[runID], !old.status.isTerminal else { return }
    if status.isTerminal {
        await toolRuntime.resetRun(runID)
    }
    let usage = AgentRunUsage(model: usage, elapsedMilliseconds: old.startedAt.map { Date().timeIntervalSince($0) * 1_000 })
    let trace = terminalTrace ?? AgentTerminalTrace(
        runID: old.runID,
        sessionID: old.sessionID,
        lastProviderRequestID: nil,
        finishReason: nil,
        lastToolCallID: nil,
        terminalTransition: "\ (old.status.rawValue) -> \ (status.rawValue)",
        terminalReason: status == .completed ? .completed : status == .cancelled ? .userCancelled : status == .timedOut ? .
↪ deadlineExceeded : .runtimeFailure,
        transitionSource: "AgentRuntime.finishRun",
        explanation: error?.message ?? (status == .completed ? "运行完成" : "运行终止")
    )
}

```

```

)
let run = AgentRunInfo(
  runID: old.runID,
  sessionID: old.sessionID,
  projectID: old.projectID,
  parentRunID: old.parentRunID,
  rootRunID: old.rootRunID,
  agentKind: old.agentKind,
  status: status,
  modelSelection: old.modelSelection,
  startedAt: old.startedAt,
  finishedAt: status.isTerminal ? .now : nil,
  latestActivityAt: .now,
  error: error,
  usage: usage,
  title: old.title,
  terminalReason: status.isTerminal ? trace.terminalReason : nil,
  terminalTrace: status.isTerminal ? trace : nil
)
let result = status.isTerminal ? SubagentResult(
  childSessionID: run.sessionID,
  runID: runID,
  status: status,
  finalText: text,
  usage: usage,
  error: error,
  terminalReason: trace.terminalReason,
  terminalTrace: trace
) : nil
do {
  if let result {
    try await persistence?.saveTerminalAgentRun(run, result: result, profile: executionProfiles[runID])
  } else {
    try await persistence?.saveAgentRun(run, profile: executionProfiles[runID])
  }
} catch {
  let persistenceError = CoreError(code: .persistence, message: "AgentRun 持久化失败: \(String(describing: error))")
  let failed = AgentRunInfo(runID: old.runID, sessionID: old.sessionID, projectID: old.projectID, parentRunID: old.parentRunID,
↪ rootRunID: old.rootRunID, agentKind: old.agentKind, status: .failed, modelSelection: old.modelSelection, startedAt: old.startedAt,
↪ finishedAt: .now, latestActivityAt: .now, error: persistenceError, usage: usage, title: old.title)
  runs[runID] = failed
  activeSessions.remove(failed.sessionID)
  let failedResult = SubagentResult(childSessionID: failed.sessionID, runID: runID, status: .failed, usage: usage, error:
↪ persistenceError)
  results[runID] = failedResult
  resumeWaiters(for: runID, result: failedResult)
  await scheduler.complete(runID)
  await eventSink(.agentRunFailed(failed))
  await eventSink(.subagentResultAvailable(failedResult))
  logDiagnostic("finishRun persistence failed run=\(runID.rawValue): \(error)")
  return
}
runs[runID] = run
Task { await diagnostics?.record(kind: status == .completed ? .agentRun : .error, event: "agentRun.\(status.rawValue)", sessionID:
↪ run.sessionID, runID: run.runID, rootRunID: run.rootRunID, parentRunID: run.parentRunID, errorCode: error?.code.rawValue) }
if status.isTerminal { activeSessions.remove(run.sessionID) }
if status.isTerminal { runDeadlines.removeValue(forKey: runID) }
if let result {
  results[runID] = result
  resumeWaiters(for: runID, result: result)
  await scheduler.complete(runID)
  await eventSink(status == .completed ? .agentRunCompleted(run) : status == .cancelled ? .agentRunCancelled(run) : .
↪ agentRunFailed(run))
  await eventSink(.subagentResultAvailable(result))
} else {

```

```

        await eventSink(.agentRunStatusChanged(run))
    }
}

private func depth(of session: Session) async throws -> Int {
    var result = 0; var current = session.parentSessionID
    while let id = current { result += 1; current = try await store.session(id).parentSessionID }
    return result
}

private func isDescendant(_ run: AgentRunInfo, of ancestor: AgentRunID) -> Bool {
    var current = run.parentRunID
    while let id = current { if id == ancestor { return true }; current = runs[id]?.parentRunID }
    return false
}

private func requireSameTree(_ target: AgentRunInfo, requester: AgentRunID) throws {
    guard let requesterRun = runs[requester], requesterRun.rootRunID == target.rootRunID else {
        throw CoreError(code: .permissionDenied, message: "AgentRun 不属于当前 Agent 树")
    }
}

private func resumeWaiters(for runID: AgentRunID, result: SubagentResult) {
    for waiter in resultWaiters.removeValue(forKey: runID) ?? [] { waiter.resume(returning: result) }
}

private func updated(_ run: AgentRunInfo, status: AgentRunStatus) -> AgentRunInfo {
    AgentRunInfo(runID: run.runID, sessionID: run.sessionID, projectID: run.projectID, parentRunID: run.parentRunID, rootRunID: run.
↳ rootRunID, agentKind: run.agentKind, status: status, modelSelection: run.modelSelection, startedAt: run.startedAt, finishedAt: run.
↳ finishedAt, latestActivityAt: .now, error: run.error, usage: run.usage, title: run.title)
}

private func normalized(profile: SubagentExecutionProfile?, parent: SubagentExecutionProfile?) throws -> SubagentExecutionProfile? {
    guard let profile else { return parent }
    let rawPermission: String?
    if let raw = profile.permissionProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, ExecutionProfile(rawValue: trimmed) != nil else {
            throw CoreError(code: .toolArgumentInvalid, message: "未知 permissionProfile: \"(raw)\"")
        }
        rawPermission = trimmed
    } else {
        rawPermission = nil
    }

    let budget: String?
    if let raw = profile.budgetProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, let val = Int(trimmed), val > 0 else {
            throw CoreError(code: .toolArgumentInvalid, message: "budgetProfile 必须是正整数 token budget")
        }
        budget = trimmed
    } else {
        budget = nil
    }

    let context: String?
    if let raw = profile.contextProfile {
        let trimmed = raw.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty, let val = Int(trimmed), val > 0 else {
            throw CoreError(code: .toolArgumentInvalid, message: "contextProfile 必须是正整数 context window")
        }
        context = trimmed
    } else {
        context = nil
    }
}

```

```

    }

    if let value = profile.maxSteps, value <= 0 { throw CoreError(code: .toolArgumentInvalid, message: "maxSteps 必须大于 0") }
    if let value = profile.timeoutSeconds, value <= 0 { throw CoreError(code: .toolArgumentInvalid, message: "timeoutSeconds 必须大于
    ↪ 0") }
    let childTools = profile.toolProfile.map(Set.init)
    let parentTools = parent?.toolProfile.map(Set.init)
    let tools: [String]? = switch (childTools, parentTools) {
    case let (.some(child), .some(parent)): Array(child.intersection(parent)).sorted()
    case let (.some(child), .none): Array(child).sorted()
    case let (.none, .some(parent)): Array(parent).sorted()
    case (.none, .none): nil
    }
    let rank: [ExecutionProfile: Int] = [.readOnly: 0, .workspace: 1, .fullAccess: 2]
    let requestedPermission = rawPermission.flatMap(ExecutionProfile.init(rawValue:))
    let parentPermission = parent?.permissionProfile.flatMap(ExecutionProfile.init(rawValue:))
    let permission: ExecutionProfile? = switch (requestedPermission, parentPermission) {
    case let (.some(child), .some(parent)): rank[child, default: 0] <= rank[parent, default: 0] ? child : parent
    case let (.some(child), .none): child
    case let (.none, .some(parent)): parent
    case (.none, .none): nil
    }
    return SubagentExecutionProfile(modelSelection: profile.modelSelection, permissionProfile: permission?.rawValue, toolProfile: tools,
    ↪ budgetProfile: budget ?? parent?.budgetProfile, contextProfile: context ?? parent?.contextProfile, maxSteps: profile.maxSteps,
    ↪ timeoutSeconds: profile.timeoutSeconds)
}

private func logDiagnostic(_ message: String) {
    guard diagnosticsEnabled else { return }
}
}

```

// File: Sources/LingXiCore/Modules/Agent/WorkflowRuntime.swift

```

import Foundation
import LingXiProtocol

public struct WorkflowTaskCompletion: Sendable, Equatable {
    public let result: SubagentResult?
    public let error: CoreError?

    public init(result: SubagentResult? = nil, error: CoreError? = nil) {
        self.result = result
        self.error = error
    }
}

/// Durable DAG coordinator. It never replays a task after restart: interrupted work is recoveryRequired.
public actor WorkflowRuntime {
    public typealias Executor = @Sendable (WorkflowID, WorkflowTaskDefinition, WorkflowTaskProvenance, @escaping @Sendable (
    ↪ WorkflowTaskProvenance) async -> Void) async -> WorkflowTaskCompletion
    public typealias InputSink = @Sendable (WorkflowID, WorkflowTaskID, WorkflowPendingInput) async -> Void

    private let persistence: SQLitePersistenceStore?
    private let executor: Executor?
    private let diagnostics: RuntimeDiagnosticsStore?
    private var inputSink: InputSink?
    private var workflows: [WorkflowID: WorkflowSnapshot] = [:]
    private var active: [String: Task<Void, Never>] = [:]

    public init(persistence: SQLitePersistenceStore? = nil, executor: Executor? = nil, diagnostics: RuntimeDiagnosticsStore? = nil) {
        self.persistence = persistence
        self.executor = executor
        self.diagnostics = diagnostics
    }
}

```

```

public func setInputSink(_ sink: @escaping InputSink) {
    inputSink = sink
}

public func restore() async throws {
    for stored in try await persistence?.loadWorkflows() ?? [] {
        let recovered = recover(stored)
        workflows[recovered.id] = recovered
        if recovered != stored { try await persist(recovered) }
    }
}

public func create(id: WorkflowID = WorkflowID(UUID().uuidString), rootSessionID: SessionID, rootRunID: AgentRunID, tasks: [
    ↪ WorkflowTaskDefinition]) async throws -> WorkflowSnapshot {
    try validate(tasks)
    let snapshot = WorkflowSnapshot(id: id, rootSessionID: rootSessionID, rootRunID: rootRunID, tasks: tasks.map { WorkflowTaskState(
    ↪ definition: $0) })
    workflows[id] = snapshot
    await diagnostics?.record(kind: .workflow, event: "workflow.created", sessionID: rootSessionID, runID: rootRunID, workflowID: id,
    ↪ metadata: ["taskCount": String(tasks.count)])
    try await persist(snapshot)
    await schedule(id)
    return workflows[id]!
}

public func workflow(_ id: WorkflowID) throws -> WorkflowSnapshot {
    guard let workflow = workflows[id] else { throw CoreError(code: .resourceNotFound, message: "Workflow 不存在: \(id.rawValue)") }
    return workflow
}

public func allWorkflows() -> [WorkflowSnapshot] { workflows.values.sorted { $0.createdAt < $1.createdAt } }

public func readyTasks(_ id: WorkflowID) throws -> [WorkflowTaskID] {
    let workflow = try workflow(id)
    return workflow.tasks.filter { $0.status == .pending && dependenciesCompleted($0, in: workflow) }.map(\.definition.id)
}

/// Stores the origin-owned input request. The actual continuation remains in the originating runtime only.
public func suspend(workflowID: WorkflowID, taskID: WorkflowTaskID, input: WorkflowPendingInput) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }) else { throw CoreError(code: .resourceNotFound,
    ↪ message: "Workflow Task 不存在") }
    let status: WorkflowTaskStatus
    switch input {
    case .question: status = .waitingForQuestion
    case .permission: status = .waitingForPermission
    case .decision: status = .waitingForDecision
    }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: status, pendingInput: input), status: .waitingForUser,
    ↪ checkpoint: checkpoint(workflow, taskID, "input-pending"))
    workflows[workflowID] = workflow
    await diagnostics?.record(kind: .hitl, event: "workflow.input.pending", sessionID: workflow.rootSessionID, runID: workflow.rootRunID
    ↪ , workflowID: workflowID, taskID: taskID)
    try await persist(workflow)
    if case .decision = input {
        await inputSink?(workflowID, taskID, input)
    }
}

/// Records that routing has been re-established. It does not rerun or complete the originating task.
public func reattachPendingInput(workflowID: WorkflowID, taskID: WorkflowTaskID) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.tasks[index].pendingInput != nil else {
    ↪ throw CoreError(code: .resourceNotFound, message: "Workflow Task 没有挂起输入") }

```

```

        workflow = replacing(workflow, task: replace(workflow.tasks[index], status: workflow.tasks[index].status), status: .waitingForUser,
        ↪ checkpoint: checkpoint(workflow, taskID, "input-reattached"))
        workflows[workflowID] = workflow
        await diagnostics?.record(kind: .recovery, event: "workflow.recovery.acknowledged", sessionID: workflow.rootSessionID, runID:
        ↪ workflow.rootRunID, workflowID: workflowID, taskID: taskID)
        try await persist(workflow)
    }

public func suspendForOrigin(runID: AgentRunID, input: WorkflowPendingInput) async throws {
    guard let match = workflows.values.lazy.flatMap(\.tasks).first(where: { $0.provenance?.childRunID == runID }), let workflow =
    ↪ workflows.values.first(where: { $0.tasks.contains(where: { $0.definition.id == match.definition.id && $0.provenance?.childRunID ==
    ↪ runID }) }) else {
        throw CoreError(code: .resourceNotFound, message: "Workflow child Run 不存在")
    }
    try await suspend(workflowID: workflow.id, taskID: match.definition.id, input: input)
}

public func suspendForOrigin(sessionID: SessionID, input: WorkflowPendingInput) async throws {
    guard let match = workflows.values.lazy.flatMap(\.tasks).first(where: { $0.provenance?.childSessionID == sessionID }), let workflow
    ↪ = workflows.values.first(where: { $0.tasks.contains(where: { $0.definition.id == match.definition.id && $0.provenance?.
    ↪ childSessionID == sessionID }) }) else {
        throw CoreError(code: .resourceNotFound, message: "Workflow child Session 不存在")
    }
    try await suspend(workflowID: workflow.id, taskID: match.definition.id, input: input)
}

public func complete(workflowID: WorkflowID, taskID: WorkflowTaskID, result: SubagentResult? = nil, error: CoreError? = nil) async
    ↪ throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }) else { throw CoreError(code: .resourceNotFound,
    ↪ message: "Workflow Task 不存在") }
    let current = workflow.tasks[index]
    guard !current.status.isTerminal else { return }
    let status: WorkflowTaskStatus = error == nil ? .completed : (error?.code == .commandTimedOut ? .timedOut : .failed)
    let provenance = result.map { WorkflowTaskProvenance(parentSessionID: workflow.rootSessionID, parentRunID: workflow.rootRunID,
    ↪ childSessionID: $0.childSessionID, childRunID: $0.runID) }
    workflow = replacing(workflow, task: replace(current, status: status, provenance: provenance, result: result, pendingInput: nil,
    ↪ error: error), status: status == .completed ? .running : .failed, checkpoint: checkpoint(workflow, taskID, "task-\(status.rawValue)
    ↪ ))
    workflow = blockDependents(in: workflow)
    workflow = withDerivedStatus(workflow)
    workflows[workflowID] = workflow
    active.removeValue(forKey: key(workflowID, taskID))
    try await persist(workflow)
    await schedule(workflowID)
}

/// An operator may explicitly make an interrupted task eligible again after verifying its side effects.
public func acknowledgeRecovery(workflowID: WorkflowID, taskID: WorkflowTaskID) async throws {
    var workflow = try workflow(workflowID)
    guard let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.tasks[index].status == .
    ↪ recoveryRequired else { throw CoreError(code: .toolArgumentInvalid, message: "Task 不处于 recoveryRequired") }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: .pending), status: .running, checkpoint: checkpoint(
    ↪ workflow, taskID, "recovery-acknowledged"))
    workflows[workflowID] = workflow
    try await persist(workflow)
    await schedule(workflowID)
}

private func schedule(_ workflowID: WorkflowID) async {
    guard var workflow = workflows[workflowID], workflow.status == .pending || workflow.status == .running else { return }
    var changed = false
    for index in workflow.tasks.indices where workflow.tasks[index].status == .pending && dependenciesCompleted(workflow.tasks[index],
    ↪ in: workflow) {
        guard let executor, workflow.tasks[index].definition.kind == .agent else { continue }

```



```

        let task = workflow.tasks[index]
        let provenance = WorkflowTaskProvenance(parentSessionID: workflow.rootSessionID, parentRunID: workflow.rootRunID)
        workflow = replacing(workflow, task: replace(task, status: .running, provenance: provenance), status: .running, checkpoint:
↳ checkpoint(workflow, task.definition.id, "task-running"))
        changed = true
        let workflowID = workflow.id
        let taskID = task.definition.id
        active[key(workflowID, taskID)] = Task.detached { [weak self] in
            let completion = await executor(workflowID, task.definition, provenance) { [weak self] child in
                await self?.recordChild(workflowID: workflowID, taskID: taskID, provenance: child)
            }
            try? await self?.complete(workflowID: workflowID, taskID: taskID, result: completion.result, error: completion.error)
        }
    }
    if changed {
        workflows[workflowID] = workflow
        try? await persist(workflow)
    }
}

private func recover(_ workflow: WorkflowSnapshot) -> WorkflowSnapshot {
    let tasks = workflow.tasks.map { task in
        task.status == .running ? replace(task, status: .recoveryRequired) : task
    }
    let status: WorkflowStatus = tasks.contains(where: { $0.status == .recoveryRequired }) ? .recoveryRequired : workflow.status
    return WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks
↳ : tasks, checkpoint: WorkflowCheckpoint(sequence: workflow.checkpoint.sequence + 1, label: "restart-recovered"), createdAt:
↳ workflow.createdAt, updatedAt: .now)
}

private func recordChild(workflowID: WorkflowID, taskID: WorkflowTaskID, provenance: WorkflowTaskProvenance) async {
    guard var workflow = workflows[workflowID], let index = workflow.tasks.firstIndex(where: { $0.definition.id == taskID }), workflow.
↳ tasks[index].status == .running else { return }
    workflow = replacing(workflow, task: replace(workflow.tasks[index], status: .running, provenance: provenance), status: .running,
↳ checkpoint: checkpoint(workflow, taskID, "child-started"))
    workflows[workflowID] = workflow
    try? await persist(workflow)
}

private func validate(_ tasks: [WorkflowTaskDefinition]) throws {
    guard !tasks.isEmpty, Set(tasks.map(\.id)).count == tasks.count else { throw CoreError(code: .toolArgumentInvalid, message: "
↳ Workflow Task ID 必须唯一且非空") }
    let ids = Set(tasks.map(\.id))
    guard tasks.allSatisfy({ !$0.dependencies.contains($0.id) && Set($0.dependencies).count == $0.dependencies.count && Set($0.
↳ dependencies).isSubset(of: ids) }) else { throw CoreError(code: .toolArgumentInvalid, message: "Workflow dependency 无效") }
    var remaining = Set(tasks.map(\.id))
    while let next = tasks.first(where: { remaining.contains($0.id) && $0.dependencies.allSatisfy { !remaining.contains($0) } }) {
        remaining.remove(next.id)
    }
    guard remaining.isEmpty else { throw CoreError(code: .toolArgumentInvalid, message: "Workflow dependency 不能成环") }
}

private func dependenciesCompleted(_ task: WorkflowTaskState, in workflow: WorkflowSnapshot) -> Bool {
    task.definition.dependencies.allSatisfy { id in workflow.tasks.first(where: { $0.definition.id == id })?.status == .completed }
}

private func blockDependents(in workflow: WorkflowSnapshot) -> WorkflowSnapshot {
    var result = workflow
    for task in workflow.tasks where task.status == .pending && task.definition.dependencies.contains(where: { id in
        guard let status = result.tasks.first(where: { $0.definition.id == id })?.status else { return true }
        return status == .failed || status == .cancelled || status == .timedOut || status == .blocked
    }) {
        result = replacing(result, task: replace(task, status: .blocked, error: CoreError(code: .toolExecutionFailed, message: "Workflow
↳ dependency 未成功完成")), status: result.status, checkpoint: result.checkpoint)
    }
}

```

```

        return result
    }

    private func withDerivedStatus(_ workflow: WorkflowSnapshot) -> WorkflowSnapshot {
        let status: WorkflowStatus
        if workflow.tasks.allSatisfy({ $0.status == .completed }) { status = .completed }
        else if workflow.tasks.contains(where: { $0.status == .waitingForQuestion || $0.status == .waitingForPermission || $0.status == .
        ↪ waitingForDecision }) { status = .waitingForUser }
        else if workflow.tasks.contains(where: { $0.status == .recoveryRequired }) { status = .recoveryRequired }
        else if workflow.tasks.contains(where: { $0.status == .failed || $0.status == .timedOut || $0.status == .blocked }) { status = .
        ↪ failed }
        else { status = .running }
        return WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks
        ↪ : workflow.tasks, checkpoint: workflow.checkpoint, createdAt: workflow.createdAt, updatedAt: .now)
    }

    private func replacing(_ workflow: WorkflowSnapshot, task: WorkflowTaskState, status: WorkflowStatus, checkpoint: WorkflowCheckpoint) ->
    ↪ WorkflowSnapshot {
        WorkflowSnapshot(id: workflow.id, rootSessionID: workflow.rootSessionID, rootRunID: workflow.rootRunID, status: status, tasks:
        ↪ workflow.tasks.map { $0.definition.id == task.definition.id ? task : $0 }, checkpoint: checkpoint, createdAt: workflow.createdAt,
        ↪ updatedAt: .now)
    }

    private func replace(_ task: WorkflowTaskState, status: WorkflowTaskStatus, provenance: WorkflowTaskProvenance? = nil, result:
    ↪ SubagentResult? = nil, pendingInput: WorkflowPendingInput? = nil, error: CoreError? = nil) -> WorkflowTaskState {
        WorkflowTaskState(definition: task.definition, status: status, provenance: provenance ?? task.provenance, result: result ?? task.
        ↪ result, pendingInput: pendingInput, error: error ?? task.error)
    }

    private func checkpoint(_ workflow: WorkflowSnapshot, _ taskID: WorkflowTaskID?, _ label: String) -> WorkflowCheckpoint {
        WorkflowCheckpoint(sequence: workflow.checkpoint.sequence + 1, taskID: taskID, label: label)
    }

    private func persist(_ workflow: WorkflowSnapshot) async throws { try await persistence?.saveWorkflow(workflow) }
    private func key(_ workflowID: WorkflowID, _ taskID: WorkflowTaskID) -> String { "\(workflowID.rawValue)/\(taskID.rawValue)" }
}

```

// File: Sources/LingXiCore/Modules/Agent/AgentInstructions.swift

```

import Foundation
import LingXiProtocol

struct AgentInstruction: Sendable, Equatable {
    let source: String
    let scope: String
    let content: String
    let global: Bool
}

struct AgentInstructionSet: Sendable {
    private let workspace: URL
    private let instructions: [AgentInstruction]

    static func load(workspace: URL, globalInstructionsURL: URL? = defaultGlobalInstructionsURL) throws -> Self {
        let root = workspace.resolvingSymlinksInPath().standardizedFileURL
        let skipped = Set([".build", ".git", ".dev-sandbox", ".dev-sandbox-backups", "node_modules"])
        let enumerator = FileManager.default.enumerator(
            at: root,
            includingPropertiesForKeys: [.isDirectoryKey, .isRegularFileKey, .fileSizeKey],
            options: [.skipsPackageDescendants]
        )
        var instructions = (try globalInstructionsURL.flatMap(loadGlobal)).map { [$0] } ?? []

        while let candidate = enumerator?.nextObject() as? URL {
            let values = try candidate.resourceValues(forKeys: [.isDirectoryKey, .isRegularFileKey, .fileSizeKey])

```

```

        if values.isDirectory == true, skipped.contains(candidate.lastPathComponent) {
            enumerator?.skipDescendants()
            continue
        }
        guard candidate.lastPathComponent == "AGENTS.md", values.isRegularFile == true else { continue }
        let resolved = candidate.resolvingSymlinksInPath().standardizedFileURL
        guard resolved.lastPathComponent == "AGENTS.md", contains(root, resolved), (values.fileSize ?? 0) <= 64 * 1_024 else { continue
↵ }
        guard let content = try? String(contentsOf: resolved, encoding: .utf8) else {
            throw CoreError(code: .toolExecutionFailed, message: "无法读取 AGENTS.md: \(relative(resolved, to: root))")
        }
        let trimmed = content.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !trimmed.isEmpty else { continue }
        instructions.append(AgentInstruction(
            source: relative(resolved, to: root),
            scope: relative(resolved.deletingLastPathComponent(), to: root),
            content: trimmed,
            global: false
        ))
    }
    return Self(workspace: root, instructions: instructions.sorted {
        $0.global != $1.global ? $0.global : $0.scope.count < $1.scope.count
    })
}

func applicable(to target: URL) -> [AgentInstruction] {
    let resolved = target.resolvingSymlinksInPath().standardizedFileURL
    guard Self.contains(workspace, resolved) else { return [] }
    return instructions.filter { instruction in
        if instruction.global { return true }
        let scope = instruction.scope == "." ? workspace : workspace.appendingPathComponent(instruction.scope).standardizedFileURL
        return Self.contains(scope, resolved)
    }
}

func rendered() -> String? {
    guard !instructions.isEmpty else { return nil }
    let entries = instructions.enumerated().map { offset, instruction in
        "[AGENTS source=\(instruction.source) scope=\(instruction.scope) priority=\(offset + 1)]\n\(instruction.content)"
    }
    return "Repository instructions are scoped by target path. Apply only sources whose scope contains the target; a higher priority (
↵ closer scope) overrides a conflicting lower priority instruction. Runtime safety and the active execution profile cannot be
↵ overridden.\n\n" + entries.joined(separator: "\n\n")
}

private static func contains(_ parent: URL, _ child: URL) -> Bool {
    child.path == parent.path || child.path.hasPrefix(parent.path + "/" )
}

private static var defaultGlobalInstructionsURL: URL {
    FileManager.default.homeDirectoryForCurrentUser.appendingPathComponent(".lingxiagent/AGENTS.md")
}

private static func loadGlobal(_ url: URL) throws -> AgentInstruction? {
    guard FileManager.default.fileExists(atPath: url.path) else { return nil }
    let resolved = url.resolvingSymlinksInPath().standardizedFileURL
    let values = try resolved.resourceValues(forKeys: [.isRegularFileKey, .fileSizeKey])
    guard resolved.lastPathComponent == "AGENTS.md", values.isRegularFile == true, (values.fileSize ?? 0) <= 64 * 1_024 else { return
↵ nil }
    guard let content = try? String(contentsOf: resolved, encoding: .utf8) else {
        throw CoreError(code: .toolExecutionFailed, message: "无法读取全局 AGENTS.md")
    }
    let trimmed = content.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !trimmed.isEmpty else { return nil }
    return AgentInstruction(source: "~/lingxiagent/AGENTS.md", scope: "global", content: trimmed, global: true)
}

```

```

}

private static func relative(_ url: URL, to root: URL) -> String {
    let path = url.path
    let prefix = root.path == "/" ? "/" : root.path + "/"
    return path.hasPrefix(prefix) ? String(path.dropFirst(prefix.count)) : "."
}
}

public struct AgentEnvironmentFacts: Sendable, Equatable {
    public let platform: String
    public let workspaceRoot: String
    public let currentDirectory: String
    public let homeDirectory: String
    public let shell: String
    public let isGitRepository: Bool
    public let gitBranch: String?
    public let accessScope: String

    public init(
        platform: String = LingXiPlatform.system.osName,
        workspaceRoot: String,
        currentDirectory: String = FileManager.default.currentDirectoryPath,
        homeDirectory: String = FileManager.default.homeDirectoryForCurrentUser.path,
        shell: String = "unknown",
        isGitRepository: Bool? = nil,
        gitBranch: String? = nil,
        accessScope: String = "workspace"
    ) {
        self.platform = platform
        self.workspaceRoot = workspaceRoot
        self.currentDirectory = currentDirectory
        self.homeDirectory = homeDirectory
        self.shell = shell
        let gitRoot = URL(fileURLWithPath: workspaceRoot).appendingPathComponent(".git")
        self.isGitRepository = isGitRepository ?? FileManager.default.fileExists(atPath: gitRoot.path)
        if let gitBranch {
            self.gitBranch = gitBranch
        } else if let head = try? String(contentsOf: gitRoot.appendingPathComponent("HEAD"), encoding: .utf8), head.hasPrefix("ref: refs/heads/") {
            self.gitBranch = String(head.dropFirst("ref: refs/heads/".count)).trimmingCharacters(in: .whitespacesAndNewlines)
        } else {
            self.gitBranch = nil
        }
        self.accessScope = accessScope
    }
}

public func render() -> String {
    """
    Environment facts:
    - platform: \(platform)
    - workspaceRoot: \(workspaceRoot)
    - cwd: \(currentDirectory)
    - userHome: \(homeDirectory)
    - shell: \(shell)
    - gitRepository: \(isGitRepository ? "yes" : "no")\(gitBranch.map { " · branch: \($0)" } ?? "")
    - accessScope: \(accessScope)
    """
}

enum AgentBehaviorInstructions {
    static let runtimeGuidelines = """
    Agent Runtime Guidelines:
    - File Operations Protocol (CRITICAL):

```

- * Always use dedicated file mutation tools: `write\_file` (for creating new files or full overwrites) and `edit\_file` or `apply\_patch`
↳ (for surgical edits and patches).
- * NEVER use shell/bash commands (such as `cat <<EOF`, `echo >`, `sed`, `awk`, or redirection scripts) to write, create, or modify
↳ files. Shell execution is strictly reserved for compilation, testing, package management, git commands, and process execution.
- Task Planning: For multi-step tasks, investigations, or refactoring, proactively use `todo` (action: 'add') to establish a checklist,
↳ and update task status ('in_progress', 'completed', 'failed') as you advance to keep the sidebar updated.
- Parallel Tool Calling: When you need to read multiple files, inspect directories, grep across files, or perform independent read-only
↳ investigations, emit multiple tool calls in parallel within the same turn instead of waiting for sequential round-trips. The
↳ runtime executes independent tool calls concurrently.
- Tool Discovery: Builtin tools are always available for filesystem, grep, and execution. If `search\_tools` returns no matches or a
↳ diagnostic notice (empty/error), do not retry searching; proceed with builtin tools.
- Execution & Truthfulness: Inspect before mutating, run verification after changes, and report obstacles truthfully without
↳ hallucination.

```
""""
```

```
static func render(
  profile: AgentBehaviorProfile,
  configured: String?,
  repository: AgentInstructionSet,
  environmentFacts: AgentEnvironmentFacts? = nil
) -> String? {
  var entries: [String] = []
  if let facts = environmentFacts {
    entries.append(facts.render())
  }
  switch profile {
  case .build:
    entries.append("Build profile: inspect before editing; after every mutation, run the narrowest relevant verification. On tool  
↳ failure or timeout, use returned diagnostics to change strategy or report the blocker. Before completion, inspect the diff and  
↳ verification result. Do not repeat an identical failed action.")
  case .plan:
    entries.append("Plan profile: investigate with read-only tools and return an executable plan with evidence. Repository mutation  
↳ is forbidden by runtime capability policy.")
  case .explore:
    entries.append("Explore profile: use read-only search and inspection, report evidence and uncertainty, and do not mutate the  
↳ repository. Mutation is forbidden by runtime capability policy.")
  }
  entries.append(runtimeGuidelines)
  if let configured, !configured.isEmpty { entries.append(configured) }
  if let repository = repository.rendered() { entries.append(repository) }
  return entries.joined(separator: "\n\n")
}
}
```

// File: Sources/LingXiCore/Modules/Agent/AgentRunRuntime.swift

```
import Foundation
import LingXiProtocol

public struct SubagentRuntimeLimits: Sendable, Equatable {
  public let maxConcurrentSubagents: Int
  public let maxSubagentDepth: Int
  public let maxTotalRunsPerRootRun: Int

  public init(maxConcurrentSubagents: Int = 4, maxSubagentDepth: Int = 3, maxTotalRunsPerRootRun: Int = 32) {
    self.maxConcurrentSubagents = max(1, maxConcurrentSubagents)
    self.maxSubagentDepth = max(1, maxSubagentDepth)
    self.maxTotalRunsPerRootRun = max(1, maxTotalRunsPerRootRun)
  }
}

public actor SubagentModelResolver {
  private var runtimes: [String: ModelRuntimeAssembly]
  private let allowedModels: Set<String>?
  private var defaultSelection: ModelSelection?
```

```

private let defaultSubagentSelection: ModelSelection?

public init(defaultRuntime: ModelRuntimeAssembly, runtimes: [String: ModelRuntimeAssembly] = [:], allowedModels: Set<String>? = nil,
↳ defaultSelection: ModelSelection? = nil, defaultSubagentSelection: ModelSelection? = nil) {
    var values = runtimes
    values["default"] = values["default"] ?? defaultRuntime
    values[defaultRuntime.endpoint.providerID] = values[defaultRuntime.endpoint.providerID] ?? defaultRuntime
    self.runtimes = values
    self.allowedModels = allowedModels
    self.defaultSelection = defaultSelection
    self.defaultSubagentSelection = defaultSubagentSelection
}

public func registerAssembly(_ assembly: ModelRuntimeAssembly, for selection: ModelSelection) {
    let providerID = selection.providerID
    runtimes[providerID] = assembly
    runtimes["default"] = assembly
    if let accountID = selection.accountID, let profileID = selection.profileID {
        runtimes["\\(accountID)::\\(profileID)"] = assembly
    }
}

public func resolve(_ requested: ModelSelection?, subagent: Bool = false) throws -> (selection: ModelSelection, assembly:
↳ ModelRuntimeAssembly) {
    let requested = requested ?? (subagent ? defaultSubagentSelection ?? defaultSelection : defaultSelection)
    let providerID = requested?.providerID ?? "default"
    guard (requested?.accountID == nil) == (requested?.profileID == nil) else { throw CoreError(code: .subagentModelNotAllowed, message:
↳ "ModelSelection 必须同时提供 accountID 与 profileID") }
    let key = requested?.accountID.flatMap { account in requested?.profileID.map { "\\(account)::\\($0)" } }
    var assembly = key.flatMap({ runtimes[$0] }) ?? (requested?.accountID == nil ? runtimes[providerID] : nil)
    if assembly == nil {
        assembly = runtimes[providerID] ?? runtimes["default"]
    }
    guard let resolvedAssembly = assembly else { throw CoreError(code: .subagentModelNotAllowed, message: "Subagent Provider 不可用: \\(
↳ providerID)") }
    guard !resolvedAssembly.modelID.rawValue.isEmpty else { throw CoreError(code: .provider, message: "未配置模型 Provider") }

    let selection = requested ?? ModelSelection(providerID: providerID, modelID: resolvedAssembly.modelID.rawValue)
    guard allowedModels?.contains(selection.modelID) ?? true else { throw CoreError(code: .subagentModelNotAllowed, message: "Subagent
↳ Model 未获用户许可: \\(selection.modelID)") }

    // 若 modelID 属于该 provider 且与初始 assembly 声明的缺省 modelID 不同, 动态派生出对应 modelID 的 assembly
    let effectiveAssembly: ModelRuntimeAssembly
    if selection.modelID != resolvedAssembly.modelID.rawValue {
        effectiveAssembly = ModelRuntimeAssembly(
            provider: resolvedAssembly.provider,
            modelID: ModelID(selection.modelID),
            contextProfile: resolvedAssembly.contextProfile,
            endpoint: ResolvedModelEndpoint(
                providerID: resolvedAssembly.endpoint.providerID,
                productID: resolvedAssembly.endpoint.productID,
                endpointID: resolvedAssembly.endpoint.endpointID,
                accountID: resolvedAssembly.endpoint.accountID,
                profileID: selection.profileID ?? resolvedAssembly.endpoint.profileID,
                modelID: ModelID(selection.modelID),
                baseURL: resolvedAssembly.endpoint.baseURL,
                wireProtocol: resolvedAssembly.endpoint.wireProtocol,
                contextProfile: resolvedAssembly.endpoint.contextProfile,
                capabilities: resolvedAssembly.endpoint.capabilities,
                rateLimits: resolvedAssembly.endpoint.rateLimits
            )
        )
    } else {
        effectiveAssembly = resolvedAssembly
    }
}

```

```

        return (selection, effectiveAssembly)
    }

    public func setDefaultSelection(_ selection: ModelSelection, assembly: ModelRuntimeAssembly? = nil) throws {
        if let assembly {
            registerAssembly(assembly, for: selection)
        }
        _ = try resolve(selection)
        defaultSelection = selection
    }

    public func currentDefaultSelection() -> ModelSelection? { defaultSelection }
}

/// One scheduler owns queuing, limits, and cancellation for every descendant run.
public actor AgentRunScheduler {
    private let limits: SubagentRuntimeLimits
    private var active: [AgentRunID: Task<Void, Never>] = [:]
    private var queued: [(AgentRunID, @Sendable () async -> Void)] = []

    public init(limits: SubagentRuntimeLimits = SubagentRuntimeLimits()) { self.limits = limits }

    public func submit(runID: AgentRunID, operation: @escaping @Sendable () async -> Void) -> AgentRunStatus {
        if active.count < limits.maxConcurrentSubagents {
            start(runID, operation)
            return .starting
        }
        queued.append((runID, operation))
        return .queued
    }

    public func cancel(_ runID: AgentRunID) {
        if let index = queued.firstIndex(where: { $0.0 == runID }) {
            queued.remove(at: index)
            return
        }
        active.removeValue(forKey: runID)?.cancel()
    }

    public func cancelAll(_ runIDs: [AgentRunID]) {
        for runID in runIDs { cancel(runID) }
    }

    public func complete(_ runID: AgentRunID) {
        active.removeValue(forKey: runID)
        guard active.count < limits.maxConcurrentSubagents, !queued.isEmpty else { return }
        let next = queued.removeFirst()
        start(next.0, next.1)
    }

    public func snapshot() -> (active: [AgentRunID], queued: [AgentRunID]) {
        (Array(active.keys), queued.map{\.0})
    }

    private func start(_ runID: AgentRunID, _ operation: @escaping @Sendable () async -> Void) {
        active[runID] = Task { await operation() }
    }
}

enum AgentExecutionContext {
    @TaskLocal static var current: (sessionID: SessionID, runID: AgentRunID, rootSessionID: SessionID, parentSessionID: SessionID)?
}

```

// File: Sources/LingXiCore/Modules/Agent/SubagentToolService.swift

```

import Foundation
import LingXiProtocol

public enum SubagentTool {
    public static let definition = ToolDefinition(
        id: ToolID("subagent"),
        description: "Spawn, inspect, message, retrieve results from, or cancel an independent child Agent Session.",
        inputSchema: ToolInputSchema(properties: [
            "action": ToolInputProperty(type: .string, description: "spawn, status, message, result, or cancel", enumValues: ["spawn", "
↪ status", "message", "result", "cancel"]),
            "task": ToolInputProperty(type: .string, description: "Child task for spawn"),
            "title": ToolInputProperty(type: .string, description: "Optional child title"),
            "run_id": ToolInputProperty(type: .string, description: "Agent run ID"),
            "session_id": ToolInputProperty(type: .string, description: "Child session ID"),
            "content": ToolInputProperty(type: .string, description: "Additional child message"),
            "provider_id": ToolInputProperty(type: .string, description: "Requested provider ID"),
            "model_id": ToolInputProperty(type: .string, description: "Requested model ID"),
            "reasoning": ToolInputProperty(type: .string, description: "Requested reasoning mode")
        ], required: ["action"]),
        capability: ToolCapability([.projectRead])
    )
}

private struct SubagentToolArguments: Decodable {
    let action: String
    let task: String?
    let title: String?
    let runID: String?
    let sessionID: String?
    let content: String?
    let providerID: String?
    let modelID: String?
    let reasoning: String?
    let permissionProfile: String?
    let budgetProfile: String?
    let contextProfile: String?
    let maxSteps: Int?
    let timeoutSeconds: Int?

    enum CodingKeys: String, CodingKey {
        case action, task, title, content, reasoning
        case runID = "run_id"
        case sessionID = "session_id"
        case providerID = "provider_id"
        case modelID = "model_id"
        case permissionProfile = "permission_profile"
        case budgetProfile = "budget_profile"
        case contextProfile = "context_profile"
        case maxSteps = "max_steps"
        case timeoutSeconds = "timeout_seconds"
    }
}

var executionProfile: SubagentExecutionProfile? {
    guard permissionProfile != nil || budgetProfile != nil || contextProfile != nil || maxSteps != nil || timeoutSeconds != nil else {
        return nil
    }
    return SubagentExecutionProfile(
        permissionProfile: permissionProfile,
        budgetProfile: budgetProfile,
        contextProfile: contextProfile,
        maxSteps: maxSteps,
        timeoutSeconds: timeoutSeconds
    )
}
}

```



```

/// The Tool Plane calls this actor; it owns no Session state and delegates only through explicit runtime closures.
public actor SubagentToolService {
  private var spawn: (@Sendable (SessionID, AgentRunID, String, String?, ModelSelection?, SubagentExecutionProfile?, ToolCallID) async
    ↪ throws -> (SessionID, AgentRunInfo))?
  private var status: (@Sendable (AgentRunID, AgentRunID) async throws -> AgentRunInfo)?
  private var result: (@Sendable (AgentRunID, AgentRunID) async throws -> SubagentResult)?
  private var cancel: (@Sendable (AgentRunID, AgentRunID) async throws -> Void)?
  private var message: (@Sendable (SessionID, AgentRunID, String) async throws -> AgentRunInfo)?

  public init() {}

  public func bind(
    spawn: @escaping @Sendable (SessionID, AgentRunID, String, String?, ModelSelection?, SubagentExecutionProfile?, ToolCallID) async
    ↪ throws -> (SessionID, AgentRunInfo),
    status: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> AgentRunInfo,
    result: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> SubagentResult,
    cancel: @escaping @Sendable (AgentRunID, AgentRunID) async throws -> Void,
    message: @escaping @Sendable (SessionID, AgentRunID, String) async throws -> AgentRunInfo
  ) {
    self.spawn = spawn; self.status = status; self.result = result; self.cancel = cancel; self.message = message
  }

  public func execute(arguments: String, sessionID: SessionID, callID: ToolCallID) async throws -> String {
    guard let data = arguments.data(using: .utf8) else { throw CoreError(code: .toolArgumentInvalid, message: "subagent 参数不是 UTF-8")
    ↪ }
    let input = try JSONDecoder().decode(SubagentToolArguments.self, from: data)
    let encoder = JSONEncoder(); encoder.outputFormatting = [.sortedKeys]
    let encode: @Sendable (any Encodable) throws -> String = { String(decoding: try encoder.encode(AnyEncodable($0)), as: UTF8.self) }
    switch input.action {
    case "spawn":
      guard let parentRunID = AgentExecutionContext.current?.runID,
        let task = (input.task ?? input.content)?.trimmingCharacters(in: .whitespacesAndNewlines),
        !task.isEmpty,
        let spawn else {
        throw CoreError(code: .toolArgumentInvalid, message: "spawn 需要当前 AgentRun 与 task")
      }
      let providerID = input.providerID?.trimmingCharacters(in: .whitespacesAndNewlines)
      let modelID = input.modelID?.trimmingCharacters(in: .whitespacesAndNewlines)
      let reasoning = input.reasoning?.trimmingCharacters(in: .whitespacesAndNewlines)
      let requestedSelection: ModelSelection?
      switch (providerID?.isEmpty == false ? providerID : nil, modelID?.isEmpty == false ? modelID : nil) {
      case (nil, nil) where reasoning?.isEmpty != false:
        requestedSelection = nil
      case let (providerID?, modelID?):
        requestedSelection = ModelSelection(providerID: providerID, modelID: modelID, reasoning: reasoning?.isEmpty == false ?
        ↪ reasoning : nil)
      default:
        throw CoreError(code: .toolArgumentInvalid, message: "subagent 的 provider_id 与 model_id 必须同时提供")
      }
      let title = input.title.flatMap { $0.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty ? nil : $0 }
      let (childSessionID, run) = try await spawn(sessionID, parentRunID, task, title, requestedSelection, input.executionProfile,
        ↪ callID)
      return try encode(SubagentSpawnResponse(childSessionID: childSessionID, run: run))
    case "status":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let status else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "status 需要 run_id 与当前 AgentRun") }
      return try encode(try await status(AgentRunID(id), requester))
    case "result":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let result else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "result 需要 run_id 与当前 AgentRun") }
      return try encode(try await result(AgentRunID(id), requester))
    case "cancel":
      guard let id = input.runID, let requester = AgentExecutionContext.current?.runID, let cancel else { throw CoreError(code: .
        ↪ toolArgumentInvalid, message: "cancel 需要 run_id 与当前 AgentRun") }

```

```

        try await cancel(AgentRunID(id), requester); return #"{"status":"cancelled"}"#
    case "message":
        guard let child = input.sessionID, let content = input.content?.trimmingCharacters(in: .whitespacesAndNewlines), !content.
        ↪ isEmpty, let parentRunID = AgentExecutionContext.current?.runID, let message else { throw CoreError(code: .toolArgumentInvalid,
        ↪ message: "message 需要 session_id、content 与当前 AgentRun") }
        return try encode(try await message(SessionID(child), parentRunID, content))
        default: throw CoreError(code: .toolArgumentInvalid, message: "未知 subagent action: \(input.action)")
    }
}
}

private struct SubagentSpawnResponse: Codable {
    let childSessionID: SessionID
    let run: AgentRunInfo
}

private struct AnyEncodable: Encodable {
    private let encodeValue: (Encoder) throws -> Void
    init(_ value: some Encodable) { encodeValue = value.encode }
    func encode(to encoder: Encoder) throws { try encodeValue(encoder) }
}

```

// File: Sources/LingXiCore/Modules/Context/L1ContextEngine.swift

```

import Foundation
import LingXiProtocol

/// L1 来源描述的是模型工作集的语义，不是任何 Provider 的角色类型。
public enum ContextSource: String, Sendable, Equatable, Hashable {
    case system
    case userMessage
    case assistantMessage
    case toolCall
    case toolResult
    case projectPage
    case derivedPage
}

public enum ContextRole: Sendable, Equatable {
    case system
    case user
    case assistant
    case tool
}

public struct ContextEntry: Sendable, Equatable {
    public let messageID: MessageID?
    public let role: ContextRole
    public let source: ContextSource
    public let part: SessionMessagePart
    public let page: ContextPage?

    public init(messageID: MessageID?, role: ContextRole, source: ContextSource, part: SessionMessagePart, page: ContextPage? = nil) {
        self.messageID = messageID
        self.role = role
        self.source = source
        self.part = part
        self.page = page
    }
}

public struct ContextMetrics: Sendable, Equatable {
    public let messageCount: Int
    public let partCount: Int
    public let characterCount: Int
}

```

```

public let sourceCounts: [ContextSource: Int]
public let sessionCharacterCount: Int
public let projectCharacterCount: Int
public let projectPageCount: Int
public let estimatedTokens: Int
public let derivedPageCount: Int
public let mandatoryTokens: Int
public let recentSessionTokens: Int
public let projectTokens: Int
public let derivedTokens: Int
public let liveToolBatchCount: Int
public let compactionGeneration: Int
}

/// 一次 inference 实际可见的不可变 L1 工作集。
public struct L1ContextSnapshot: Sendable, Equatable {
    public let sessionID: SessionID
    public let revision: UInt64
    public let entries: [ContextEntry]
    public let metrics: ContextMetrics

    public func modelMessages() -> [ModelMessage] {
        var result: [ModelMessage] = []
        var currentID: MessageID?
        var currentRole: ModelRole?
        var parts: [ModelContentPart] = []

        let renderedEntries = entries.flatMap { entry -> [ContextEntry] in
            guard entry.messageID == nil, entry.source == .system, case let .text(content) = entry.part,
                content.hasPrefix("Environment facts:\n"), let separator = content.range(of: "\n\n") else {
                return [entry]
            }
            let facts = String(content[..<separator.lowerBound])
            let instructions = String(content[separator.upperBound...])
            let instructionBlocks = instructions.components(separatedBy: "\n\n").filter { !$0.trimmingCharacters(in: .whitespacesAndNewlines)
↵ }.isEmpty }
            var fragments = instructionBlocks.isEmpty ? [instructions] : instructionBlocks
            if entries.first?.messageID == nil {
                fragments.append(facts)
            } else {
                fragments.insert(facts, at: 0)
            }
            return fragments.map { ContextEntry(messageID: nil, role: .system, source: .system, part: .text($0)) }
        }

        func appendCurrent() {
            if let currentRole { result.append(ModelMessage(role: currentRole, parts: parts)) }
        }

        for entry in renderedEntries {
            let role = Self.modelRole(entry.role)
            if currentID != entry.messageID || currentRole != role {
                appendCurrent()
                currentID = entry.messageID
                currentRole = role
                parts = []
            }
            parts.append(Self.modelPart(entry.part))
        }
        appendCurrent()
        return result
    }

    private static func modelRole(_ role: ContextRole) -> ModelRole {
        switch role {

```

```

        case .system: .system
        case .user: .user
        case .assistant: .assistant
        case .tool: .tool
    }
}

private static func modelPart(_ part: SessionMessagePart) -> ModelContentPart {
    switch part {
    case let .text(text): .text(text)
    case let .toolCall(call): .toolCall(call)
    case let .toolResult(result): .toolResult(result)
    }
}
}

/// L1 初始策略：保留已完成 Session 的有序结构化历史，明确排除 reasoning 与 transient stream。
public struct L1ContextPolicy: Sendable {
    public let systemContext: String?

    public init(systemContext: String? = nil) {
        self.systemContext = systemContext?.isEmpty == true ? nil : systemContext
    }
}

/// Context Engine 是 Session history 与当前模型工作集之间的正式边界。
public actor L1ContextEngine {
    private let policy: L1ContextPolicy
    private var revisions: [SessionID: UInt64] = [:]
    private var latest: [SessionID: L1ContextSnapshot] = [:]

    public init(policy: L1ContextPolicy = L1ContextPolicy()) {
        self.policy = policy
    }

    public func snapshot(for session: Session, projectPages: [ContextPage] = [], activeEntries: [ContextEntry]? = nil, systemContext: String
        ↳ ? = nil, estimatedTokens: Int = 0, mandatoryTokens: Int = 0, liveToolBatchCount: Int = 0, compactionGeneration: Int = 0) ->
        ↳ L1ContextSnapshot {
        let revision = (revisions[session.id] ?? 0) + 1
        revisions[session.id] = revision
        var entries = activeEntries ?? []
        if activeEntries == nil {
            if let system = systemContext ?? policy.systemContext {
                entries.append(ContextEntry(messageID: nil, role: .system, source: .system, part: .text(system)))
            }
        }
        for message in session.messages {
            for part in message.parts {
                let projectedPart: SessionMessagePart
                switch part {
                case let .toolResult(result):
                    projectedPart = .toolResult(ModelToolResultProjection.projectToolResult(result))
                default:
                    projectedPart = part
                }
                entries.append(ContextEntry(
                    messageID: message.id,
                    role: contextRole(message.role),
                    source: source(message.role, part),
                    part: projectedPart
                ))
            }
        }
        let toolContents = Set(session.messages.flatMap { message in
            message.parts.compactMap { if case let .toolResult(result) = $0 { result.content } else { nil } }
        })
    }
}

```

```

    })
    var seenPages = Set<String>()
    for page in projectPages where seenPages.insert("\${page.path}|\${page.hash}").inserted && !toolContents.contains(page.content) {
        entries.append(ContextEntry(messageID: MessageID("project:\${page.id}"), role: .system, source: .projectPage, part: .text("[
↪ Project context: \${page.path}:\${page.startLine}-\${page.endLine}]\n\${page.content}"), page: page))
    }
    let snapshot = L1ContextSnapshot(
        sessionID: session.id,
        revision: revision,
        entries: entries,
        metrics: metrics(entries, estimatedTokens: estimatedTokens, mandatoryTokens: mandatoryTokens, liveToolBatchCount:
↪ liveToolBatchCount, compactionGeneration: compactionGeneration, hasSystemContext: (systemContext ?? policy.systemContext) != nil)
    )
    latest[session.id] = snapshot
    return snapshot
}

public func latestSnapshot(for sessionID: SessionID) -> L1ContextSnapshot? {
    latest[sessionID]
}

public func reset(for sessionID: SessionID) {
    latest.removeValue(forKey: sessionID)
}

private func contextRole(_ role: MessageRole) -> ContextRole {
    switch role {
    case .user: .user
    case .assistant: .assistant
    case .tool: .tool
    }
}

private func source(_ role: MessageRole, _ part: SessionMessagePart) -> ContextSource {
    switch part {
    case .toolCall: .toolCall
    case .toolResult: .toolResult
    case .text:
        switch role {
        case .user: .userMessage
        case .assistant, .tool: .assistantMessage
        }
    }
}

public func entries(for session: Session, projectPages: [ContextPage] = [], systemContext: String? = nil, systemContextAtBeginning: Bool
↪ = true) -> [ContextEntry] {
    var entries: [ContextEntry] = []
    let systemEntry = (systemContext ?? policy.systemContext).map { ContextEntry(messageID: nil, role: .system, source: .system, part: .
↪ text($0)) }
    if systemContextAtBeginning, let systemEntry { entries.append(systemEntry) }
    for message in session.messages {
        for part in message.parts {
            let projectedPart: SessionMessagePart
            switch part {
            case let .toolResult(result):
                projectedPart = .toolResult(ModelToolResultProjection.projectToolResult(result))
            default:
                projectedPart = part
            }
            entries.append(ContextEntry(messageID: message.id, role: contextRole(message.role), source: source(message.role, part), part
↪ : projectedPart))
        }
    }
    let toolContents = Set(session.messages.flatMap { $0.parts.compactMap { if case let .toolResult(result) = $0 { result.content } else

```

```

    ↪ { nil } } })
    var seen = Set<String>()
    for page in projectPages where seen.insert("\(page.path)|\(page.hash)").inserted && !toolContents.contains(page.content) {
        entries.append(ContextEntry(messageID: MessageID("project:\(page.id)"), role: .system, source: .projectPage, part: .text("[
    ↪ Project context: \(page.path):\ \(page.startLine)-\ \(page.endLine)]\n\ \(page.content)"), page: page))
    }
    if !systemContextAtBeginning, let systemEntry { entries.append(systemEntry) }
    return entries
}

public func initialMandatoryTokens(task: String, estimator: any TokenEstimator = ConservativeTokenEstimator()) -> Int {
    var tokens = 0
    if let system = policy.systemContext, !system.isEmpty {
        tokens += estimator.estimate(text: system) + 4
    }
    tokens += estimator.estimate(text: task) + 4
    return tokens
}

private func metrics(_ entries: [ContextEntry], estimatedTokens: Int, mandatoryTokens: Int, liveToolBatchCount: Int,
    ↪ compactionGeneration: Int, hasSystemContext: Bool) -> ContextMetrics {
    var sourceCounts: [ContextSource: Int] = [:]
    var ids = Set<MessageID>()
    var characters = 0
    var sessionCharacters = 0
    var projectCharacters = 0
    for entry in entries {
        sourceCounts[entry.source, default: 0] += 1
        if let id = entry.messageID { ids.insert(id) }
        switch entry.part {
        case let .text(text): characters += text.count
        case let .toolCall(call): characters += call.arguments.count
        case let .toolResult(result):
            characters += result.content.count + (result.error?.message.count ?? 0)
        }
        let count: Int
        switch entry.part {
        case let .text(text): count = entry.page?.characterCount ?? text.count
        case let .toolCall(call): count = call.arguments.count
        case let .toolResult(result): count = result.content.count + (result.error?.message.count ?? 0)
        }
        if entry.source == .projectPage { projectCharacters += count } else { sessionCharacters += count }
    }
    let projectTokens = max(0, (projectCharacters + 2) / 3)
    let derivedCharacters = entries.filter { $0.source == .derivedPage }.reduce(0) { $0 + Self.characterCount(of: $1.part) }
    let derivedTokens = max(0, (derivedCharacters + 2) / 3)
    let effectiveTokens = estimatedTokens > 0 ? estimatedTokens : ConservativeTokenEstimator().estimate(entries: entries)
    return ContextMetrics(messageCount: ids.count + (hasSystemContext ? 1 : 0), partCount: entries.count, characterCount: characters,
    ↪ sourceCounts: sourceCounts, sessionCharacterCount: sessionCharacters, projectCharacterCount: projectCharacters, projectPageCount:
    ↪ sourceCounts[.projectPage, default: 0], estimatedTokens: effectiveTokens, derivedPageCount: sourceCounts[.derivedPage, default: 0],
    ↪ mandatoryTokens: mandatoryTokens, recentSessionTokens: max(0, effectiveTokens - projectTokens - derivedTokens - mandatoryTokens),
    ↪ projectTokens: projectTokens, derivedTokens: derivedTokens, liveToolBatchCount: liveToolBatchCount, compactionGeneration:
    ↪ compactionGeneration)
}

private static func characterCount(of part: SessionMessagePart) -> Int {
    switch part { case let .text(text): text.count; case let .toolCall(call): call.arguments.count; case let .toolResult(result): result
    ↪ .content.count + (result.error?.message.count ?? 0) }
}
}

```

// File: Sources/LingXiCore/Modules/Context/L2WorkingSetPolicy.swift

import Foundation

```

public struct L2WorkingSetEntry: Sendable {
    public var page: ContextPage
    public var lastUsed: UInt64
    public var useCount: UInt64
    public var queryRelevance: Double
    public var taskAffinity: Double
    public var explicitPin: Bool
}

/// L2 的淘汰决策独立于存储；第一版只接收确定性本地信号。
public struct L2WorkingSetPolicy: Sendable {
    public struct Weights: Sendable {
        public let recentUse: Double
        public let frequency: Double
        public let queryRelevance: Double
        public let taskAffinity: Double
        public let explicitPin: Double

        public init(recentUse: Double = 1, frequency: Double = 0.25, queryRelevance: Double = 0.5, taskAffinity: Double = 0.25, explicitPin:
↪ Double = 1_000_000) {
            self.recentUse = recentUse
            self.frequency = frequency
            self.queryRelevance = queryRelevance
            self.taskAffinity = taskAffinity
            self.explicitPin = explicitPin
        }
    }

    public let weights: Weights
    public init(weights: Weights = Weights()) { self.weights = weights }

    public func evictionCandidate(in entries: [String: L2WorkingSetEntry], clock: UInt64) -> String? {
        entries.min { lhs, rhs in
            let left = score(lhs.value, clock: clock)
            let right = score(rhs.value, clock: clock)
            return left == right ? lhs.key < rhs.key : left < right
        }?.key
    }

    public func score(_ entry: L2WorkingSetEntry, clock: UInt64) -> Double {
        Double(entry.lastUsed) / Double(max(1, clock)) * weights.recentUse
        + Double(entry.useCount) * weights.frequency
        + entry.queryRelevance * weights.queryRelevance
        + entry.taskAffinity * weights.taskAffinity
        + (entry.explicitPin ? weights.explicitPin : 0)
    }
}

```

// File: Sources/LingXiCore/Modules/Context/ContextCacheController.swift

```

import Foundation
import LingXiProtocol

public struct CachePriorityWeights: Sendable {
    public let recency: Double
    public let frequency: Double
    public let relevance: Double
    public let taskAffinity: Double
    public let activeFileAffinity: Double
    public let explicitReuse: Double
    public let pin: Double
    public let tokenCostPenalty: Double

    public init(
        recency: Double = 1.0,

```

```

        frequency: Double = 0.5,
        relevance: Double = 2.0,
        taskAffinity: Double = 0.5,
        activeFileAffinity: Double = 1.0,
        explicitReuse: Double = 3.0,
        pin: Double = 10_000.0,
        tokenCostPenalty: Double = 0.001
    ) {
        self.recency = recency
        self.frequency = frequency
        self.relevance = relevance
        self.taskAffinity = taskAffinity
        self.activeFileAffinity = activeFileAffinity
        self.explicitReuse = explicitReuse
        self.pin = pin
        self.tokenCostPenalty = tokenCostPenalty
    }
}

public struct L1ResidentPage: Sendable, Equatable {
    public let page: ContextPage
    public let tokens: Int
    public var lastUsed: UInt64
    public var accessCount: Int
    public var retrievalRelevance: Double
    public var isPinned: Bool
    public var inclusionReason: String

    public init(
        page: ContextPage,
        tokens: Int? = nil,
        lastUsed: UInt64,
        accessCount: Int = 1,
        retrievalRelevance: Double = 1.0,
        isPinned: Bool = false,
        inclusionReason: String = "Explicit retrieval"
    ) {
        self.page = page
        self.tokens = tokens ?? max(1, page.characterCount / 3)
        self.lastUsed = lastUsed
        self.accessCount = accessCount
        self.retrievalRelevance = retrievalRelevance
        self.isPinned = isPinned
        self.inclusionReason = inclusionReason
    }
}

public struct WarmL2Entry: Sendable, Equatable {
    public let id: String
    public let page: ContextPage?
    public let derivedPage: DerivedContextPage?
    public let tokens: Int
    public var lastUsed: UInt64
    public var accessCount: Int
    public var inclusionReason: String

    public init(
        id: String,
        page: ContextPage? = nil,
        derivedPage: DerivedContextPage? = nil,
        tokens: Int,
        lastUsed: UInt64,
        accessCount: Int = 1,
        inclusionReason: String = "Warm cache"
    ) {

```



```

        self.id = id
        self.page = page
        self.derivedPage = derivedPage
        self.tokens = tokens
        self.lastUsed = lastUsed
        self.accessCount = accessCount
        self.inclusionReason = inclusionReason
    }
}

```

/// Cache Controller 负责三级缓存的加权调度与 L1/L2/L3 Residency 管理。
 /// 模型只负责声明检索意图 (context_search)，调度决策完全由 Cache Controller 驱动。

```

public actor ContextCacheController {
    public let policy: EffectiveContextPolicy
    public nonisolated let ecoreStore: ECoreObjectStore
    public nonisolated let scheduler: CacheAwareContextScheduler
    private let weights: CachePriorityWeights
    private let contextPager: ContextPager
    private let scanner: ProjectScanner
    public nonisolated let compactor: ContextCompactor?
    private var clock: UInt64 = 0

    // Per-session L1 resident dynamic pages
    private var residentPagesBySession: [SessionID: [String: L1ResidentPage]] = [:]
    // Per-session session-level base L1 tokens (messages + system prompt)
    private var sessionL1BaseTokens: [SessionID: Int] = [:]
    private var sessionL1BaseCount: [SessionID: Int] = [:]
    // Last Provider input tokens recorded during context build for inference
    private var lastProviderInputTokensBySession: [SessionID: Int] = [:]
    // Last Provider prompt cache hit (cachedTokens, promptTokens)
    private var lastPromptCacheHitBySession: [SessionID: (cachedTokens: Int, promptTokens: Int)] = [:]
    // Per-session L2 warm cache entries
    private var warmL2EntriesBySession: [SessionID: [String: WarmL2Entry]] = [:]
    // Per-session paged-in derived pages
    private var residentDerivedPagesBySession: [SessionID: [String: DerivedContextPage]] = [:]

    // Paging statistics per session
    private var pageInsBySession: [SessionID: Int] = [:]
    private var pageOutsBySession: [SessionID: Int] = [:]
    private var promotionsBySession: [SessionID: Int] = [:]
    private var demotionsBySession: [SessionID: Int] = [:]

    public init(
        contextPager: ContextPager,
        scanner: ProjectScanner,
        compactor: ContextCompactor? = nil,
        policy: EffectiveContextPolicy = EffectiveContextPolicy(),
        weights: CachePriorityWeights = CachePriorityWeights(),
        ecoreStore: ECoreObjectStore? = nil,
        scheduler: CacheAwareContextScheduler? = nil
    ) {
        self.contextPager = contextPager
        self.scanner = scanner
        self.compactor = compactor
        self.policy = policy
        self.weights = weights
        self.ecoreStore = ecoreStore ?? ECoreObjectStore()
        self.scheduler = scheduler ?? CacheAwareContextScheduler()
    }

    // Convenience initializer preserving existing calls
    public init(
        contextPager: ContextPager,
        scanner: ProjectScanner,
        compactor: ContextCompactor? = nil,

```

```

        maxL1ResidentCharacters: Int,
        weights: CachePriorityWeights = CachePriorityWeights(),
        ecoreStore: ECoreObjectStore? = nil,
        scheduler: CacheAwareContextScheduler? = nil
    ) {
        self.contextPager = contextPager
        self.scanner = scanner
        self.compactor = compactor
        self.policy = EffectiveContextPolicy(
            addressableBudget: 1_048_576,
            modelWindow: 1_048_576,
            economicThreshold: 272_000,
            reserve: 22_000,
            l1Target: max(1, maxL1ResidentCharacters / 3),
            l1SoftLimit: max(2, Int(Double(maxL1ResidentCharacters / 3) * 1.07)),
            l1HardLimit: max(3, Int(Double(maxL1ResidentCharacters / 3) * 1.14)),
            l2Max: 350_000,
            l3Capacity: 456_576
        )
        self.weights = weights
        self.ecoreStore = ecoreStore ?? ECoreObjectStore()
        self.scheduler = scheduler ?? CacheAwareContextScheduler()
    }

    /// 记录指定 Session 的基础 L1 token 数与条目数 (当前 resident working set)
    public func recordSessionL1Tokens(sessionID: SessionID, tokens: Int, count: Int? = nil) {
        sessionL1BaseTokens[sessionID] = tokens
        if let count { sessionL1BaseCount[sessionID] = count }
    }

    /// 记录最近一次 Provider 推理实际构建发送的 input token 数 (与当前 resident working set 明确分离)
    public func recordProviderInputTokens(sessionID: SessionID, tokens: Int) {
        lastProviderInputTokensBySession[sessionID] = tokens
    }

    public struct SessionCacheRecord: Sendable, Equatable {
        public var cachedTokens: Int
        public var promptTokens: Int
        public var previousPromptTokens: Int?
        public var status: String // "active", "coldNewEpoch", "unavailable"
        public var epoch: Int
        public var epochReason: String?
        public var stablePrefixHash: String?
        public var missDiagnostics: String?
        public var provider: String?
        public var model: String?
        public var cacheWriteTokens: Int?
        public var contextGrowthDelta: Int?

        public init(
            cachedTokens: Int,
            promptTokens: Int,
            previousPromptTokens: Int? = nil,
            status: String = "active",
            epoch: Int = 1,
            epochReason: String? = nil,
            stablePrefixHash: String? = nil,
            missDiagnostics: String? = nil,
            provider: String? = nil,
            model: String? = nil,
            cacheWriteTokens: Int? = nil,
            contextGrowthDelta: Int? = nil
        ) {
            self.cachedTokens = cachedTokens
            self.promptTokens = promptTokens

```

```

        self.previousPromptTokens = previousPromptTokens
        self.status = status
        self.epoch = epoch
        self.epochReason = epochReason
        self.stablePrefixHash = stablePrefixHash
        self.missDiagnostics = missDiagnostics
        self.provider = provider
        self.model = model
        self.cacheWriteTokens = cacheWriteTokens
        self.contextGrowthDelta = contextGrowthDelta ?? previousPromptTokens.map { promptTokens - $0 }
    }
}

private var sessionCacheRecords: [SessionID: SessionCacheRecord] = [:]
private var sessionEpochs: [SessionID: Int] = [:]
private var sessionEpochReasons: [SessionID: String] = [:]
private var previousPromptTokensBySession: [SessionID: Int] = [:]
private var currentTurnFingerprintBySession: [SessionID: PrefixFingerprint] = [:]
private var lastTurnFingerprintBySession: [SessionID: PrefixFingerprint] = [:]
private var clientStructuralHealthBySession: [SessionID: ClientStructuralCacheHealth] = [:]
private var turnsInEpochBySession: [SessionID: Int] = [:]
private var clientBustsInEpochBySession: [SessionID: Int] = [:]

/// 推进 CacheEpoch (仅由实质语义变更触发, 如 model switch / provider switch / context rollover)
public func advanceEpoch(sessionID: SessionID, reason: String) {
    let current = sessionEpochs[sessionID] ?? 1
    sessionEpochs[sessionID] = current + 1
    sessionEpochReasons[sessionID] = reason
    previousPromptTokensBySession[sessionID] = nil
    lastTurnFingerprintBySession[sessionID] = nil
    turnsInEpochBySession[sessionID] = 0
    clientBustsInEpochBySession[sessionID] = 0
}

/// 记录当前推理前计算的 Prefix 指纹及客户端自身结构健康度
public func recordFingerprint(
    sessionID: SessionID,
    fingerprint: PrefixFingerprint,
    prefixBytes: Int = 0,
    volatileBytes: Int = 0
) {
    currentTurnFingerprintBySession[sessionID] = fingerprint
    let epoch = sessionEpochs[sessionID] ?? 1
    let turns = (turnsInEpochBySession[sessionID] ?? 0) + 1
    turnsInEpochBySession[sessionID] = turns

    let lastFP = lastTurnFingerprintBySession[sessionID]
    let isBust: Bool
    let isAppendOnly: Bool
    let status: String

    if let lastFP {
        if lastFP.stablePrefixHash != fingerprint.stablePrefixHash {
            isBust = true
            clientBustsInEpochBySession[sessionID] = (clientBustsInEpochBySession[sessionID] ?? 0) + 1
            status = "bustDetected"
        } else {
            isBust = false
            status = "stable"
        }
    }
    isAppendOnly = (lastFP.historyStableHash == fingerprint.historyStableHash || !fingerprint.historyStableHash.isEmpty)
} else {
    isBust = false
    isAppendOnly = true
    status = "newEpoch"
}

```

```

    }

    let totalBusts = clientBustsInEpochBySession[sessionID] ?? 0
    let bustRate = turns > 0 ? Double(totalBusts) / Double(turns) : 0.0

    let health = ClientStructuralCacheHealth(
        stablePrefixHash: fingerprint.stablePrefixHash,
        stablePrefixBytes: prefixBytes,
        stablePrefixSegments: 2,
        appendOnlyHistory: isAppendOnly,
        prefixMutationDetected: isBust,
        cacheEpoch: epoch,
        clientCausedBustRate: bustRate,
        appendOnlyRatio: isAppendOnly ? 1.0 : 0.5,
        volatileTailBytes: volatileBytes,
        status: status,
        clientCausedBusts: totalBusts,
        comparableRequests: turns,
        appendOnlyViolations: isAppendOnly ? 0 : 1
    )
    clientStructuralHealthBySession[sessionID] = health
}

/// 获取客户端自身结构化缓存健康指标（第一权威真相）
public func lastClientHealth(for sessionID: SessionID) -> ClientStructuralCacheHealth? {
    clientStructuralHealthBySession[sessionID]
}

/// 记录最近一次 Provider 推理返回的真实 Prefix Cache 命中情况
public func recordProviderCacheHit(
    sessionID: SessionID,
    cachedTokens: Int,
    promptTokens: Int,
    cacheWriteTokens: Int? = nil,
    provider: String? = nil,
    model: String? = nil,
    isUnavailable: Bool = false
) async {
    let epoch = sessionEpochs[sessionID] ?? 1
    let reason = sessionEpochReasons[sessionID] ?? "initial_turn"
    let prev = previousPromptTokensBySession[sessionID]
    let currentFP = currentTurnFingerprintBySession[sessionID]
    let lastFP = lastTurnFingerprintBySession[sessionID]

    let status: String
    var missDiagnostics: String? = nil

    if isUnavailable {
        status = "unavailable"
        if let currentFP, let lastFP, currentFP.stablePrefixHash != lastFP.stablePrefixHash {
            missDiagnostics = "CLIENT CACHE BUST DETECTED: stablePrefixHash changed unexpectedly within Epoch \ \(epoch)"
        }
    } else if prev == nil || prev == 0 {
        status = "coldNewEpoch"
    } else {
        status = "active"
        if let prev, prev > 0 {
            let reuseEfficiency = Double(cachedTokens) / Double(prev)
            let isSignificantHit = cachedTokens >= 1024 || reuseEfficiency >= 0.5
            if !isSignificantHit {
                missDiagnostics = generateMissDiagnostics(old: lastFP, new: currentFP, prevTokens: prev, cachedTokens: cachedTokens)
                let isClientBust = (lastFP != nil && currentFP != nil && lastFP?.stablePrefixHash != currentFP?.stablePrefixHash)
                if isClientBust {
                    await scheduler.recordBust(sessionID: sessionID)
                }
            }
        }
    }
}

```

```

    } else {
        if reuseEfficiency < 0.9 {
            missDiagnostics = generateMissDiagnostics(old: lastFP, new: currentFP, prevTokens: prev, cachedTokens: cachedTokens)
        }
        await scheduler.recordHit(sessionID: sessionID)
    }
}

let record = SessionCacheRecord(
    cachedTokens: cachedTokens,
    promptTokens: promptTokens,
    previousPromptTokens: prev,
    status: status,
    epoch: epoch,
    epochReason: reason,
    stablePrefixHash: currentFP?.stablePrefixHash,
    missDiagnostics: missDiagnostics,
    provider: provider,
    model: model,
    cacheWriteTokens: cacheWriteTokens
)

sessionCacheRecords[sessionID] = record
let currentDebt = await scheduler.debtState(for: sessionID).cacheDebt
savePersistedTelemetry(sessionID: sessionID, record: record, debt: currentDebt)
lastPromptCacheHitBySession[sessionID] = (cachedTokens, promptTokens)

// 为下一轮更新上一轮理论可复用 token 数及上一轮指纹
previousPromptTokensBySession[sessionID] = promptTokens
if let currentFP {
    lastTurnFingerprintBySession[sessionID] = currentFP
}
}

private func generateMissDiagnostics(old: PrefixFingerprint?, new: PrefixFingerprint?, prevTokens: Int, cachedTokens: Int) -> String {
    guard let old, let new else {
        let diff = max(0, prevTokens - cachedTokens)
        return "Prefix miss: ~\(diff) tokens dropped (first tracked turn)"
    }
    var changes: [String] = []
    if old.systemHash != new.systemHash {
        changes.append("systemHash changed (\(old.systemHash.prefix(8)) -> \(new.systemHash.prefix(8)))")
    }
    if old.coreToolsHash != new.coreToolsHash {
        changes.append("coreToolsHash changed (\(old.coreToolsHash.prefix(8)) -> \(new.coreToolsHash.prefix(8)))")
    }
    if old.leasedToolsHash != new.leasedToolsHash {
        changes.append("leasedToolsHash changed (\(old.leasedToolsHash.prefix(8)) -> \(new.leasedToolsHash.prefix(8)))")
    }
    if old.skillPrefixHash != new.skillPrefixHash {
        changes.append("skillPrefixHash changed (\(old.skillPrefixHash.prefix(8)) -> \(new.skillPrefixHash.prefix(8)))")
    }
    if old.requestProfileHash != new.requestProfileHash {
        changes.append("requestProfileHash changed (\(old.requestProfileHash.prefix(8)) -> \(new.requestProfileHash.prefix(8)))")
    }
    if old.historyStableHash != new.historyStableHash {
        changes.append("historyStableHash changed (\(old.historyStableHash.prefix(8)) -> \(new.historyStableHash.prefix(8)))")
    }
    let diff = max(0, prevTokens - cachedTokens)
    if changes.isEmpty {
        return "status: upstream cache variance suspected (Client structural prefix 100% stable, missed ~\(diff) tokens)"
    } else {
        let bustNote = changes.contains(where: { $0.contains("coreToolsHash") || $0.contains("systemHash") }) ? " [CLIENT CACHE BUST  

↪ DETECTED]" : ""
        return "Prefix miss source:\(bustNote) " + changes.joined(separator: ", ") + " (missed ~\(diff) tokens)"
    }
}

```

```

    }
}

/// 获取最近一次 Provider 推理返回的真实详细 Cache 记录
public func lastProviderCacheRecord(for sessionID: SessionID) -> SessionCacheRecord? {
    if let record = sessionCacheRecords[sessionID] {
        return record
    }
    if let hydrated = loadPersistedTelemetry(sessionID: sessionID) {
        sessionCacheRecords[sessionID] = hydrated
        return hydrated
    }
    return nil
}

private func telemetryFileURL(sessionID: SessionID) -> URL {
    let safeSessionID = sessionID.rawValue.filter { $0.isLetter || $0.isNumber || $0 == "_" || $0 == "-" }
    let dir = ecoreStore.baseDirectory.appendingPathComponent(safeSessionID, isDirectory: true)
    if !FileManager.default.fileExists(atPath: dir.path) {
        try? FileManager.default.createDirectory(at: dir, withIntermediateDirectories: true)
    }
    return dir.appendingPathComponent("telemetry.json", isDirectory: false)
}

private func savePersistedTelemetry(sessionID: SessionID, record: SessionCacheRecord, debt: Int) {
    struct DTO: Codable {
        let cachedTokens: Int
        let promptTokens: Int
        let previousPromptTokens: Int?
        let status: String
        let epoch: Int
        let epochReason: String?
        let stablePrefixHash: String?
        let missDiagnostics: String?
        let provider: String?
        let model: String?
        let cacheWriteTokens: Int?
        let cacheDebt: Int
    }
    let dto = DTO(
        cachedTokens: record.cachedTokens,
        promptTokens: record.promptTokens,
        previousPromptTokens: record.previousPromptTokens,
        status: record.status,
        epoch: record.epoch,
        epochReason: record.epochReason,
        stablePrefixHash: record.stablePrefixHash,
        missDiagnostics: record.missDiagnostics,
        provider: record.provider,
        model: record.model,
        cacheWriteTokens: record.cacheWriteTokens,
        cacheDebt: debt
    )
    if let data = try? JSONEncoder().encode(dto) {
        let url = telemetryFileURL(sessionID: sessionID)
        try? data.write(to: url, options: .atomic)
    }
}

private func loadPersistedTelemetry(sessionID: SessionID) -> SessionCacheRecord? {
    struct DTO: Codable {
        let cachedTokens: Int
        let promptTokens: Int
        let previousPromptTokens: Int?
        let status: String
    }

```

```

        let epoch: Int
        let epochReason: String?
        let stablePrefixHash: String?
        let missDiagnostics: String?
        let provider: String?
        let model: String?
        let cacheWriteTokens: Int?
        let cacheDebt: Int?
    }

    let url = telemetryFileURL(sessionID: sessionID)
    guard let data = try? Data(contentsOf: url),
        let dto = try? JSONDecoder().decode(DTO.self, from: data) else {
        return nil
    }

    let record = SessionCacheRecord(
        cachedTokens: dto.cachedTokens,
        promptTokens: dto.promptTokens,
        previousPromptTokens: dto.previousPromptTokens,
        status: dto.status,
        epoch: dto.epoch,
        epochReason: dto.epochReason,
        stablePrefixHash: dto.stablePrefixHash,
        missDiagnostics: dto.missDiagnostics,
        provider: dto.provider,
        model: dto.model,
        cacheWriteTokens: dto.cacheWriteTokens
    )

    if let debt = dto.cacheDebt, debt > 0 {
        Task { await scheduler.restoreDebtState(sessionID: sessionID, debt: debt) }
    }

    return record
}

/// 获取最近一次 Provider 推理返回的真实 Prefix Cache 命中情况 (兼容旧调用)
public func lastProviderCacheHit(for sessionID: SessionID) -> (cachedTokens: Int, promptTokens: Int)? {
    lastPromptCacheHitBySession[sessionID]
}

/// 最近一次 Provider 推理的 input token 数
public func lastProviderInputTokens(for sessionID: SessionID) -> Int? {
    lastProviderInputTokensBySession[sessionID]
}

/// L1 当前占用 token 数
public func l1UsageTokens(for sessionID: SessionID) -> Int {
    let dynamicTokens = residentPagesBySession[sessionID]?.values.reduce(0) { $0 + $1.tokens } ?? 0
    let baseTokens = sessionL1BaseTokens[sessionID] ?? 0
    return dynamicTokens + baseTokens
}

/// L1 条目数
public func l1Count(for sessionID: SessionID) -> Int {
    let dynamicCount = residentPagesBySession[sessionID]?.count ?? 0
    let baseCount = sessionL1BaseTokens[sessionID] ?? ((sessionL1BaseTokens[sessionID] ?? 0) > 0 ? 1 : 0)
    return dynamicCount + baseCount
}

/// P-Core 活跃上下文 Token 数
public func pCoreUsageTokens(for sessionID: SessionID) -> Int {
    lastProviderCacheRecord(for: sessionID)?.promptTokens ?? l1UsageTokens(for: sessionID)
}

/// E-Core 对象织物估算 Token 数 (按 4 字节约 1 Token 换算)
public func eCoreUsageTokens(for sessionID: SessionID) async -> Int {
    let objects = await ecoreStore.listObjects(sessionID: sessionID)

```

```

    return objects.reduce(0) { $0 + $1.totalBytes } / 4
}

/// E-Core 对象总数
public func eCoreObjectCount(for sessionID: SessionID) async -> Int {
    let objects = await ecoreStore.listObjects(sessionID: sessionID)
    return objects.count
}

/// E-Core 存储总字节数
public func eCoreTotalBytes(for sessionID: SessionID) async -> Int {
    let objects = await ecoreStore.listObjects(sessionID: sessionID)
    return objects.reduce(0) { $0 + $1.totalBytes }
}

/// [Legacy Compatibility] 旧 L2 工作集占用数
public func l2UsageTokens(for sessionID: SessionID) -> Int {
    warmL2EntriesBySession[sessionID]?.values.reduce(0) { $0 + $1.tokens } ?? 0
}

/// [Legacy Compatibility] 旧 L2 条目数
public func l2Count(for sessionID: SessionID) -> Int {
    warmL2EntriesBySession[sessionID]?.count ?? 0
}

/// [Legacy Compatibility] 旧 L3 占用数
public func l3UsageTokens(for sessionID: SessionID) async -> Int {
    0
}

/// [Legacy Compatibility] 旧 L3 条目数
public func l3Count(for sessionID: SessionID) async -> Int {
    0
}

/// 调度统计指标
public func pagingStats(for sessionID: SessionID) -> ContextPagingStats {
    ContextPagingStats(
        pageIns: pageInsBySession[sessionID] ?? 0,
        pageOuts: pageOutsBySession[sessionID] ?? 0,
        promotions: promotionsBySession[sessionID] ?? 0,
        demotions: demotionsBySession[sessionID] ?? 0
    )
}

/// 执行明确检索并将高权重条目调度到当前 Session 的 L1 Working Set
public func handleSearch(sessionID: SessionID, query: String, activeTask: String = "", activeFiles: [String] = [], limit: Int = 5) async
↳ throws -> String {
    clock &+= 1
    let currentClock = clock

    // 1. Search L2 Warm Cache
    var l2Candidates: [(WarmL2Entry, Double)] = []
    if let l2Entries = warmL2EntriesBySession[sessionID] {
        let normalizedQuery = query.lowercased()
        for entry in l2Entries.values {
            let content = entry.page?.content ?? entry.derivedPage?.content ?? ""
            let path = entry.page?.path ?? ""
            if content.localizedCaseInsensitiveContains(normalizedQuery) || path.localizedCaseInsensitiveContains(normalizedQuery) {
                let score = calculatePriority(
                    content: content,
                    path: path,
                    characterCount: content.count,
                    query: query,
                    activeTask: activeTask,

```



```

        activeFiles: activeFiles,
        clock: currentClock,
        lastUsed: entry.lastUsed,
        accessCount: entry.accessCount,
        isPinned: false
    )
    l2Candidates.append((entry, score + 2.0)) // Warm cache bonus
}
}

// 2. Search L3 Cold Cache
var l3Candidates: [(DerivedContextPage, Double)] = []
if let compactor {
    let derivedMatches = await compactor.derivedStore.search(sessionID: sessionID, query: query, limit: limit)
    for page in derivedMatches {
        let score = calculatePriority(
            content: page.content,
            path: "",
            characterCount: page.content.count,
            query: query,
            activeTask: activeTask,
            activeFiles: activeFiles,
            clock: currentClock,
            lastUsed: currentClock,
            accessCount: 1,
            isPinned: false
        )
        l3Candidates.append((page, score))
    }
}

// 3. Search Codebase Index
_ = try await contextPager.rebuildStaleFiles(using: scanner)
let contextQuery = ContextQuery(currentTask: query)
let searchResult = await contextPager.query(projectRoot: scanner.root, query: contextQuery, limit: limit * 2)

var codebaseCandidates: [(ContextPage, Double, String)] = []
for page in searchResult.pages {
    let priority = calculatePriority(
        content: page.content,
        path: page.path,
        characterCount: page.characterCount,
        query: query,
        activeTask: activeTask,
        activeFiles: activeFiles,
        clock: currentClock,
        lastUsed: currentClock,
        accessCount: 1,
        isPinned: false
    )
    codebaseCandidates.append((page, priority, "Explicit retrieval for query: \"(query)\""))
}

// Check if anything matched across all sources
guard !l2Candidates.isEmpty || !l3Candidates.isEmpty || !codebaseCandidates.isEmpty else {
    return "No matching context found for query: \"(query)\"."
}

var currentL1 = residentPagesBySession[sessionID] ?? []
var currentL2 = warmL2EntriesBySession[sessionID] ?? []
var pagedInCodebasePages: [ContextPage] = []
var pagedInDerivedPages: [DerivedContextPage] = []

// Promote L2 entries

```

```

for (l2Entry, _) in l2Candidates.prefix(limit) {
    currentL2.removeValue(forKey: l2Entry.id)
    promotionsBySession[sessionID, default: 0] += 1
    pageInsBySession[sessionID, default: 0] += 1

    if let page = l2Entry.page {
        currentL1[page.id] = L1ResidentPage(
            page: page,
            tokens: l2Entry.tokens,
            lastUsed: currentClock,
            accessCount: l2Entry.accessCount + 1,
            retrievalRelevance: 2.0,
            isPinned: false,
            inclusionReason: "Promoted from L2 warm cache"
        )
        pagedInCodebasePages.append(page)
    } else if let derived = l2Entry.derivedPage {
        pagedInDerivedPages.append(derived)
    }
}

// Admit L3 entries
for (l3Entry, _) in l3Candidates.prefix(limit) {
    pageInsBySession[sessionID, default: 0] += 1
    pagedInDerivedPages.append(l3Entry)
    residentDerivedPagesBySession[sessionID, default: [:]][l3Entry.id] = l3Entry
}

// Admit Codebase entries
codebaseCandidates.sort { $0.1 > $1.1 }
for (page, _, reason) in codebaseCandidates.prefix(limit) {
    currentL1[page.id] = L1ResidentPage(
        page: page,
        tokens: max(1, page.characterCount / 3),
        lastUsed: currentClock,
        accessCount: (currentL1[page.id]?.accessCount ?? 0) + 1,
        retrievalRelevance: 1.0,
        isPinned: false,
        inclusionReason: reason
    )
    pagedInCodebasePages.append(page)
    pageInsBySession[sessionID, default: 0] += 1
}

// Record admission into context pager
await contextPager.recordInjection(pagedInCodebasePages)

// Evict from L1 to L2 Warm Cache if over soft limit (demote lower priority entries)
let baseTokens = sessionL1BaseTokens[sessionID] ?? 0
var dynamicTokens = currentL1.values.reduce(0) { $0 + $1.tokens }

while (dynamicTokens + baseTokens) > policy.l1SoftLimit && currentL1.count > 1 {
    let lowest = currentL1.values.filter { !$0.isPinned }.min { lhs, rhs in
        let scoreL = calculatePriority(
            content: lhs.page.content,
            path: lhs.page.path,
            characterCount: lhs.page.characterCount,
            query: query,
            activeTask: activeTask,
            activeFiles: activeFiles,
            clock: currentClock,
            lastUsed: lhs.lastUsed,
            accessCount: lhs.accessCount,
            isPinned: lhs.isPinned
        )
    }
}

```

```

        let scoreR = calculatePriority(
            content: rhs.page.content,
            path: rhs.page.path,
            characterCount: rhs.page.characterCount,
            query: query,
            activeTask: activeTask,
            activeFiles: activeFiles,
            clock: currentClock,
            lastUsed: rhs.lastUsed,
            accessCount: rhs.accessCount,
            isPinned: rhs.isPinned
        )
        return scoreL < scoreR
    }
    guard let victim = lowest else { break }
    currentL1.removeValue(forKey: victim.page.id)
    dynamicTokens -= victim.tokens

    // Move to L2 warm cache
    currentL2[victim.page.id] = WarmL2Entry(
        id: victim.page.id,
        page: victim.page,
        tokens: victim.tokens,
        lastUsed: victim.lastUsed,
        accessCount: victim.accessCount,
        inclusionReason: "Paged-out to warm cache"
    )
    demotionsBySession[sessionID, default: 0] += 1
    pageOutsBySession[sessionID, default: 0] += 1
}

// Evict oldest from L2 if over L2 max
var l2Tokens = currentL2.values.reduce(0) { $0 + $1.tokens }
while l2Tokens > policy.l2Max && !currentL2.isEmpty {
    let oldest = currentL2.values.min { $0.lastUsed < $1.lastUsed }
    guard let victim = oldest else { break }
    currentL2.removeValue(forKey: victim.id)
    l2Tokens -= victim.tokens
}

residentPagesBySession[sessionID] = currentL1
warmL2EntriesBySession[sessionID] = currentL2

var outputSections: [String] = []
if !pagedInCodebasePages.isEmpty {
    let formatted = pagedInCodebasePages.map { page in
        "### \((page.path):\((page.startLine)-\((page.endLine)\n\`\`\`n\`\\(page.content)\n\`\`\`"
    }.joined(separator: "\n\n")
    outputSections.append("## Codebase Context (\(pagedInCodebasePages.count) pages paged into L1)\n" + formatted)
}

if !pagedInDerivedPages.isEmpty {
    let formatted = pagedInDerivedPages.map { page in
        let snippet = page.content.count > 500 ? String(page.content.prefix(500)) + "..." : page.content
        return "- \((page.sourceKind.rawValue)): \((snippet)"
    }.joined(separator: "\n")
    outputSections.append("## Historical Context\n" + formatted)
}

return outputSections.joined(separator: "\n\n")
}

private func calculatePriority(
    content: String,
    path: String,

```

```

characterCount: Int,
query: String,
activeTask: String,
activeFiles: [String],
clock: UInt64,
lastUsed: UInt64,
accessCount: Int,
isPinned: Bool
) -> Double {
    if isPinned { return weights.pin }

    let recencyScore = Double(lastUsed) / Double(max(1, clock)) * weights.recency
    let frequencyScore = Double(accessCount) * weights.frequency

    let lowerPath = path.lowercased()
    let activeFileMatch = (!lowerPath.isEmpty && activeFiles.contains { lowerPath.contains($0.lowercased()) }) ? weights.
↪ activeFileAffinity : 0.0
    let taskMatch = (!activeTask.isEmpty && content.localizedCaseInsensitiveContains(activeTask)) ? weights.taskAffinity : 0.0
    let explicitMatch = (!lowerPath.isEmpty && lowerPath.localizedCaseInsensitiveContains(query)) ? weights.explicitReuse : 0.0
    let relevanceScore = (content.localizedCaseInsensitiveContains(query) ? 1.0 : 0.2) * weights.relevance

    let tokenCost = Double(characterCount / 3) * weights.tokenCostPenalty

    return recencyScore + frequencyScore + relevanceScore + activeFileMatch + taskMatch + explicitMatch - tokenCost
}

/// L1 常驻 Working Set 中的动态页面（由 Cache Controller 严格管理）
public func residentPages(for sessionId: SessionID) -> [ContextPage] {
    guard let pages = residentPagesBySession[sessionId] else { return [] }
    return Array(pages.values.map(\.page))
}

/// L1 常驻 Working Set 中的衍生页面
public func residentDerivedPages(for sessionId: SessionID) -> [DerivedContextPage] {
    guard let pages = residentDerivedPagesBySession[sessionId] else { return [] }
    return Array(pages.values)
}

/// L1 常驻详细状态
public func residentEntries(for sessionId: SessionID) -> [L1ResidentPage] {
    guard let pages = residentPagesBySession[sessionId] else { return [] }
    return Array(pages.values)
}

/// 撤回（undo）操作后对齐 P-E 双核心架构状态与缓存基线
public func reconcileAfterRevert(sessionID: SessionID, remainingMessages: [Message]) async {
    // 1. 提取剩余消息中所有有效的 ToolCallID
    var validToolCallIDs = Set<ToolCallID>()
    for msg in remainingMessages {
        for part in msg.parts {
            if case let .toolCall(tc) = part {
                validToolCallIDs.insert(tc.callID)
            }
            if case let .toolResult(res) = part {
                validToolCallIDs.insert(res.callID)
            }
        }
    }

    // 2. E-Core 存储裁剪：清理已被撤回的 Tool 所生成的大对象文件
    await ecoreStore.prune(sessionID: sessionID, keepingToolCallIDs: validToolCallIDs)

    // 3. P-Core (L1) 状态重置：撤回导致上一次 Provider 调用的 Cache Record 失效
    lastProviderInputTokensBySession.removeValue(forKey: sessionID)
    previousPromptTokensBySession.removeValue(forKey: sessionID)
}

```

```

currentTurnFingerprintBySession.removeValue(forKey: sessionID)
lastTurnFingerprintBySession.removeValue(forKey: sessionID)
sessionCacheRecords.removeValue(forKey: sessionID)
residentDerivedPagesBySession.removeValue(forKey: sessionID)
sessionEpochs[sessionID] = (sessionEpochs[sessionID] ?? 1) + 1
sessionEpochReasons[sessionID] = "revert_turn"

// 4. 重新基于剩余有效消息精确估算 L1 / P-Core 常驻 Tokens
if remainingMessages.isEmpty {
    sessionL1BaseTokens.removeValue(forKey: sessionID)
    sessionL1BaseCount.removeValue(forKey: sessionID)
    residentPagesBySession.removeValue(forKey: sessionID)
} else {
    let estimator = ConservativeTokenEstimator()
    var entries: [ContextEntry] = []
    for msg in remainingMessages {
        let ctxRole: ContextRole
        let src: ContextSource
        switch msg.role {
        case .user: ctxRole = .user; src = .userMessage
        case .assistant: ctxRole = .assistant; src = .assistantMessage
        case .tool: ctxRole = .tool; src = .toolResult
        }
        for part in msg.parts {
            entries.append(ContextEntry(
                messageID: msg.id,
                role: ctxRole,
                source: src,
                part: part
            ))
        }
    }
    let tokens = estimator.estimate(entries: entries)
    sessionL1BaseTokens[sessionID] = tokens
    sessionL1BaseCount[sessionID] = entries.count
    previousPromptTokensBySession[sessionID] = tokens
    lastProviderInputTokensBySession[sessionID] = tokens

    // 建立撤回后的合成基线记录, 确保前缀复用与 P-Core 状态平滑衔接, 不发生归零或乱跳
    let revertEpoch = sessionEpochs[sessionID] ?? 1
    let revertRecord = SessionCacheRecord(
        cachedTokens: 0,
        promptTokens: tokens,
        previousPromptTokens: tokens,
        status: "coldNewEpoch",
        epoch: revertEpoch,
        epochReason: "revert_turn",
        stablePrefixHash: nil,
        missDiagnostics: nil,
        provider: nil,
        model: nil,
        cacheWriteTokens: nil
    )
    sessionCacheRecords[sessionID] = revertRecord
    savePersistedTelemetry(sessionID: sessionID, record: revertRecord, debt: 0)
}

// 5. 调度器经济学债务状态对齐
await scheduler.reset(sessionID: sessionID)
if remainingMessages.isEmpty {
    let url = telemetryFileURL(sessionID: sessionID)
    try? FileManager.default.removeItem(at: url)
}
}

```

```

/// 重置指定 Session 的所有级别缓存（用于 /new 或 session 清理）
public func resetSession(_ sessionID: SessionID) async {
    residentPagesBySession.removeValue(forKey: sessionID)
    residentDerivedPagesBySession.removeValue(forKey: sessionID)
    sessionL1BaseTokens.removeValue(forKey: sessionID)
    sessionL1BaseCount.removeValue(forKey: sessionID)
    lastProviderInputTokensBySession.removeValue(forKey: sessionID)
    warmL2EntriesBySession.removeValue(forKey: sessionID)
    pageInsBySession.removeValue(forKey: sessionID)
    pageOutsBySession.removeValue(forKey: sessionID)
    promotionsBySession.removeValue(forKey: sessionID)
    demotionsBySession.removeValue(forKey: sessionID)
    await scheduler.reset(sessionID: sessionID)
    await ecoreStore.cleanSession(sessionID: sessionID)
    let url = telemetryFileURL(sessionID: sessionID)
    try? FileManager.default.removeItem(at: url)
}
}

```

// File: Sources/LingXiCore/Modules/Context/CacheAwareContextScheduler.swift

```

import Foundation
import LingXiProtocol

/// 缓存经济学决策策略配置
public struct EconomicCompactPolicy: Sendable, Equatable, Codable {
    /// 缓存读取单价与写入单价的比值（通常为 0.1，即读比写便宜 90%）
    public var cacheReadWriteRatio: Double
    /// 最小评估的剩余轮次 Horizon
    public var minHorizon: Int
    /// 默认预估的会话剩余轮数
    public var defaultHorizon: Int
    /// 最大允许积累的缓存债务（达到上限后禁止主动破坏性压缩）
    public var maxCacheDebt: Int
    /// 偿还 1 点缓存债务所需的连续缓存命中轮次
    public var debtRecoveryTurns: Int

    public init(
        cacheReadWriteRatio: Double = 0.1,
        minHorizon: Int = 2,
        defaultHorizon: Int = 5,
        maxCacheDebt: Int = 2,
        debtRecoveryTurns: Int = 2
    ) {
        self.cacheReadWriteRatio = cacheReadWriteRatio
        self.minHorizon = minHorizon
        self.defaultHorizon = defaultHorizon
        self.maxCacheDebt = maxCacheDebt
        self.debtRecoveryTurns = debtRecoveryTurns
    }
}

/// 缓存调度器压缩决策枚举
public enum CompactDecision: Sendable, Equatable {
    case skip(reason: String)
    case economicCompact(reason: String)
    case emergencyWindowProtection(reason: String)

    public var shouldCompact: Bool {
        switch self {
            case .skip: return false
            case .economicCompact, .emergencyWindowProtection: return true
        }
    }
}

```

```

/// 会话缓存债务与稳定性状态
public struct CacheDebtState: Sendable, Equatable {
    public var cacheDebt: Int = 0
    public var consecutiveHits: Int = 0
    public var totalBusts: Int = 0
    public var lastCompactStep: Int = 0

    public init() {}
}

/// Cache-Aware Context Scheduler
/// 负责依据经济学 Break-even 计算、Cache Debt 门禁以及 Window Protection 硬限制，
/// 决定当前轮次是否应当执行上下文压缩，摆脱盲目高水位压缩对 Prompt Cache 的反复破坏。
public actor CacheAwareContextScheduler {
    public let policy: EconomicCompactPolicy
    private var debtStatesBySession: [SessionID: CacheDebtState] = [:]

    public init(policy: EconomicCompactPolicy = EconomicCompactPolicy()) {
        self.policy = policy
    }

    /// 获取或初始化会话的缓存债务状态
    public func debtState(for sessionID: SessionID) -> CacheDebtState {
        debtStatesBySession[sessionID] ?? CacheDebtState()
    }

    /// 恢复会话的缓存债务状态（用于持久化水合）
    public func restoreDebtState(sessionID: SessionID, debt: Int) {
        var state = debtStatesBySession[sessionID] ?? CacheDebtState()
        state.cacheDebt = min(policy.maxCacheDebt, max(0, debt))
        debtStatesBySession[sessionID] = state
    }

    /// 记录一次 Cache Hit（偿还债务）
    public func recordHit(sessionID: SessionID) {
        var state = debtStatesBySession[sessionID] ?? CacheDebtState()
        state.consecutiveHits += 1
        if state.consecutiveHits >= policy.debtRecoveryTurns {
            state.cacheDebt = max(0, state.cacheDebt - 1)
            state.consecutiveHits = 0
        }
        debtStatesBySession[sessionID] = state
    }

    /// 记录一次 Cache Bust（积累债务）
    public func recordBust(sessionID: SessionID) {
        var state = debtStatesBySession[sessionID] ?? CacheDebtState()
        state.cacheDebt = min(policy.maxCacheDebt, state.cacheDebt + 1)
        state.consecutiveHits = 0
        state.totalBusts += 1
        debtStatesBySession[sessionID] = state
    }

    /// 记录发生了一次压缩
    public func recordCompactionOccurred(sessionID: SessionID, step: Int) {
        var state = debtStatesBySession[sessionID] ?? CacheDebtState()
        state.lastCompactStep = step
        state.cacheDebt = min(policy.maxCacheDebt, state.cacheDebt + 1)
        state.consecutiveHits = 0
        debtStatesBySession[sessionID] = state
    }

    /// 核心决策函数：决定当前是否执行压缩
    /// - Parameters:

```

```

/// - currentTokens: 当前准备发送给 Provider 的总估计 Tokens
/// - hardLimit: 模型上下文窗口硬限制 (必须保证不超出)
/// - economicThreshold: 经济学起征点 (低于此门槛绝不主动压缩破坏缓存)
/// - estimatedEvictionTokens: 预估压缩能削减的 Tokens
/// - stablePrefixTokens: 被破坏的前缀中原本可复用的 Tokens
/// - remainingHorizon: 预估会话剩余请求轮次
public func evaluate(
    sessionID: SessionID,
    currentTokens: Int,
    hardLimit: Int,
    economicThreshold: Int,
    estimatedEvictionTokens: Int,
    stablePrefixTokens: Int,
    remainingHorizon: Int?
) -> CompactDecision {
    // 1. 最高优先级: Window Protection (硬窗口保护)
    // 哪怕在经济上严重亏损, 也必须保证绝对不能突破 Provider hardInputLimit 导致 400 失败!
    if currentTokens > hardLimit {
        return .emergencyWindowProtection(
            reason: "Context exceeds hard limit: current \(currentTokens) > hardLimit \(hardLimit)"
        )
    }

    // 2. 检查经济学起征点: 低于起征点绝不压缩, 保护前缀缓存
    if currentTokens <= economicThreshold {
        return .skip(
            reason: "Context below economic threshold: current \(currentTokens) <= threshold \(economicThreshold)"
        )
    }

    let state = debtStatesBySession[sessionID] ?? CacheDebtState()

    // 3. 检查 Cache Debt 门禁: 债务过高说明近期缓存击穿过于频繁, 禁止主动压缩
    if state.cacheDebt >= policy.maxCacheDebt {
        return .skip(
            reason: "Blocked by Cache Debt: current debt \(state.cacheDebt) >= max \(policy.maxCacheDebt)"
        )
    }

    // 4. Break-even 经济学公式计算
    // 设 C_write = 1.0, C_read = cacheReadWriteRatio (例如 0.1)
    // 成本 = stablePrefixTokens * 1.0
    // 收益 = horizon * estimatedEvictionTokens * cacheReadWriteRatio
    let horizon = max(policy.minHorizon, remainingHorizon ?? policy.defaultHorizon)
    guard estimatedEvictionTokens > 0 else {
        return .skip(reason: "Estimated eviction tokens is 0 or negative")
    }

    let neededHorizonDouble = Double(stablePrefixTokens) / (Double(estimatedEvictionTokens) * policy.cacheReadWriteRatio)
    let neededHorizon = max(1, Int(ceil(neededHorizonDouble)))

    if horizon < neededHorizon {
        return .skip(
            reason: "Economically unviable: need horizon >= \(neededHorizon) turns to break even, but actual horizon is \(horizon)"
        )
    }

    let expectedSavings = Int(Double(horizon * estimatedEvictionTokens) * policy.cacheReadWriteRatio) - stablePrefixTokens
    return .economicCompact(
        reason: "Break-even achieved: horizon \(horizon) >= \(neededHorizon), expected net savings ~\(expectedSavings) tokens"
    )
}

/// 重置会话状态
public func reset(sessionID: SessionID) {

```



```

        debtStatesBySession.removeValue(forKey: sessionID)
    }
}

```

// File: Sources/LingXiCore/Modules/Context/ContextCompaction.swift

```

import Foundation
import LingXiProtocol

public protocol TokenEstimator: Sendable {
    func estimate(text: String) -> Int
    func estimate(entries: [ContextEntry]) -> Int
    func estimate(tools: [ToolDefinition]) -> Int
}

public struct ConservativeTokenEstimator: TokenEstimator {
    public init() {}
    public func estimate(text: String) -> Int { max(1, (text.utf8.count + 2) / 3) }
    public func estimate(entries: [ContextEntry]) -> Int {
        entries.reduce(0) { $0 + estimate(text: ContextCompactor.content(of: $1.part)) + 4 }
    }
    public func estimate(tools: [ToolDefinition]) -> Int {
        tools.reduce(0) { total, tool in total + estimate(tool: tool) }
    }

    public func estimate(tool: ToolDefinition) -> Int {
        if let rawSchema = tool.rawInputSchema, let data = try? JSONEncoder().encode(rawSchema) {
            return estimate(text: tool.id.rawValue + " " + tool.description + " " + String(decoding: data, as: UTF8.self)) + 22
        }
        var text = "function: \(tool.name) \(tool.description) "
        for (name, prop) in tool.inputSchema.properties.sorted(by: { $0.key < $1.key }) {
            text += "\(name): \(prop.type.rawValue) \(prop.description) "
            if let enumValues = prop.enumValues {
                text += "enum: [\\(enumValues.joined(separator: ", "))]"
            }
        }
        if !tool.inputSchema.required.isEmpty {
            text += "required: [\\(tool.inputSchema.required.joined(separator: ", "))]"
        }
        return estimate(text: text) + 18
    }
}

public struct ModelContextProfile: Sendable, Equatable, Codable {
    public let contextWindowTokens: Int
    public let maxOutputTokens: Int?
    public let recommendedOutputReserveTokens: Int?
    public let source: String

    public init(contextWindowTokens: Int = 32_768, maxOutputTokens: Int? = nil, recommendedOutputReserveTokens: Int? = nil, source: String =
        ↩ "conservative fallback") {
        self.contextWindowTokens = max(1, contextWindowTokens)
        self.maxOutputTokens = maxOutputTokens
        self.recommendedOutputReserveTokens = recommendedOutputReserveTokens
        self.source = source
    }
}

public struct ContextBudgetPolicy: Sendable, Equatable, Codable {
    public let preferredActiveTokens: Int?
    public let preferredRatio: Double
    public let defaultActiveCeiling: Int
    public let safetyMarginTokens: Int
    public let fixedOverheadTokens: Int

```

```

public init(preferredActiveTokens: Int? = nil, preferredRatio: Double = 0.65, defaultActiveCeiling: Int = 64_000, safetyMarginTokens:
    ↳ Int = 1_024, fixedOverheadTokens: Int = 256) {
    self.preferredActiveTokens = preferredActiveTokens
    self.preferredRatio = preferredRatio
    self.defaultActiveCeiling = defaultActiveCeiling
    self.safetyMarginTokens = safetyMarginTokens
    self.fixedOverheadTokens = fixedOverheadTokens
}

public func with(preferredActiveTokens: Int?) -> ContextBudgetPolicy {
    ContextBudgetPolicy(
        preferredActiveTokens: preferredActiveTokens,
        preferredRatio: preferredRatio,
        defaultActiveCeiling: defaultActiveCeiling,
        safetyMarginTokens: safetyMarginTokens,
        fixedOverheadTokens: fixedOverheadTokens
    )
}

}

public struct ContextBudget: Sendable, Equatable {
    public let hardInputLimit: Int
    public let preferredActiveTokens: Int
    public let highWaterTokens: Int
    public let lowWaterTokens: Int
    public let reservedOutputTokens: Int
    public let fixedOverheadTokens: Int
    public let safetyMarginTokens: Int
}

}

public struct ContextBudgetPlanner: Sendable {
    public let policy: ContextBudgetPolicy
    public init(policy: ContextBudgetPolicy = ContextBudgetPolicy()) { self.policy = policy }

    public func with(preferredActiveTokens: Int?) -> ContextBudgetPlanner {
        ContextBudgetPlanner(policy: policy.with(preferredActiveTokens: preferredActiveTokens))
    }
}

public func plan(profile: ModelContextProfile, requestedMaxOutputTokens: Int? = nil, toolTokens: Int = 0) -> ContextBudget {
    let reserve = max(requestedMaxOutputTokens ?? 0, profile.recommendedOutputReserveTokens ?? profile.maxOutputTokens ?? 4_096)
    let hard = max(0, profile.contextWindowTokens - reserve - policy.fixedOverheadTokens - toolTokens - policy.safetyMarginTokens)
    let preferred = min(hard, policy.preferredActiveTokens ?? min(policy.defaultActiveCeiling, Int(Double(hard) * policy.preferredRatio)
    ↳ ))
    return ContextBudget(hardInputLimit: hard, preferredActiveTokens: preferred, highWaterTokens: min(hard, Int(Double(preferred) *
    ↳ 1.15)), lowWaterTokens: Int(Double(preferred) * 0.8), reservedOutputTokens: reserve, fixedOverheadTokens: policy.
    ↳ fixedOverheadTokens + toolTokens, safetyMarginTokens: policy.safetyMarginTokens)
}

}

public enum DerivedContextSourceKind: String, Sendable, Equatable { case user, assistant, historicalTool }
public enum ToolExchangeBatchState: String, Sendable, Equatable, Codable {
    case pending
    case settledAwaitingConsumption
    case consumed
    /// 崩溃时 pending batch 从自动重放，必须由上层显式恢复。
    case recoveryRequired
}

public enum DurableToolCallState: String, Sendable, Equatable, Codable {
    case requested
    case waitingForHuman
    case executing
    case completed
    case recoveryRequired
}

```

```

public struct ToolCallProvenance: Sendable, Equatable, Codable {
    public let batchID: String
    public let sessionID: SessionID
    public let agentRunID: AgentRunID?
    public let providerRequestID: ModelRequestID?
    public let providerStep: Int

    public init(batchID: String, sessionID: SessionID, agentRunID: AgentRunID?, providerRequestID: ModelRequestID?, providerStep: Int) {
        self.batchID = batchID
        self.sessionID = sessionID
        self.agentRunID = agentRunID
        self.providerRequestID = providerRequestID
        self.providerStep = providerStep
    }
}

public struct ToolExecutionClaim: Sendable, Equatable, Codable {
    public let claimID: String
    public let mutatesProject: Bool
    public let claimedAt: Date

    public init(claimID: String = UUID().uuidString, mutatesProject: Bool, claimedAt: Date = .now) {
        self.claimID = claimID
        self.mutatesProject = mutatesProject
        self.claimedAt = claimedAt
    }
}

public enum ToolCallHumanRequest: Sendable, Equatable, Codable {
    case permission(PermissionRequest)
    case question(QuestionRequest)
}

public enum ToolCallHumanReply: Sendable, Equatable, Codable {
    case permission(PermissionReply)
    case question(QuestionReply)
}

/// One durable state machine per provider ToolCall. A completed result is authoritative after restart.
public struct DurableToolCall: Sendable, Equatable, Codable {
    public let call: ToolCall
    public let state: DurableToolCallState
    public let request: ToolCallHumanRequest?
    public let reply: ToolCallHumanReply?
    public let executionClaim: ToolExecutionClaim?
    public let provenance: ToolCallProvenance
    public let result: ToolResult?

    public init(call: ToolCall, state: DurableToolCallState = .requested, request: ToolCallHumanRequest? = nil, reply: ToolCallHumanReply? =
        ↪ nil, executionClaim: ToolExecutionClaim? = nil, provenance: ToolCallProvenance, result: ToolResult? = nil) {
        self.call = call
        self.state = state
        self.request = request
        self.reply = reply
        self.executionClaim = executionClaim
        self.provenance = provenance
        self.result = result
    }

    public func with(state: DurableToolCallState? = nil, request: ToolCallHumanRequest? = nil, reply: ToolCallHumanReply? = nil,
        ↪ replaceHumanExchange: Bool = false, executionClaim: ToolExecutionClaim? = nil, result: ToolResult? = nil) -> DurableToolCall {
        DurableToolCall(call: call, state: state ?? self.state, request: replaceHumanExchange ? request : request ?? self.request, reply:
        ↪ replaceHumanExchange ? reply : reply ?? self.reply, executionClaim: executionClaim ?? self.executionClaim, provenance: provenance,
        ↪ result: result ?? self.result)
    }
}

```

```

    }
}

public struct ToolExchangeBatch: Sendable, Equatable {
    public let batchID: String
    public let sessionID: SessionID
    public let assistantMessageID: MessageID
    public let resultMessageID: MessageID?
    public let toolCalls: [ToolCall]
    public let toolResults: [ToolResult]
    public let toolCallStates: [DurableToolCall]
    public let continuationRequestID: ModelRequestID?
    public let providerStep: Int
    public let state: ToolExchangeBatchState
    public let estimatedTokens: Int
    public let revision: UInt64?
    public let turnID: TurnID?
    public var isComplete: Bool { Set(toolCalls.map(\.callID)) == Set(toolResults.map(\.callID)) && toolCalls.count == toolResults.count }

    public init(batchID: String, sessionID: SessionID, assistantMessageID: MessageID, resultMessageID: MessageID? = nil, toolCalls: [
        ↪ ToolCall], toolResults: [ToolResult] = [], toolCallStates: [DurableToolCall]? = nil, continuationRequestID: ModelRequestID? = nil,
        ↪ providerStep: Int, state: ToolExchangeBatchState, estimatedTokens: Int, revision: UInt64? = nil, turnID: TurnID? = nil) {
        self.batchID = batchID
        self.sessionID = sessionID
        self.assistantMessageID = assistantMessageID
        self.resultMessageID = resultMessageID
        self.toolCalls = toolCalls
        self.toolResults = toolResults
        self.toolCallStates = toolCallStates ?? toolCalls.map { DurableToolCall(call: $0, provenance: ToolCallProvenance(batchID: batchID,
        ↪ sessionID: sessionID, agentRunID: nil, providerRequestID: continuationRequestID, providerStep: providerStep)) }
        self.continuationRequestID = continuationRequestID
        self.providerStep = providerStep
        self.state = state
        self.estimatedTokens = estimatedTokens
        self.revision = revision
        self.turnID = turnID
    }

    public func with(state: ToolExchangeBatchState, resultMessageID: MessageID? = nil, toolResults: [ToolResult]? = nil, toolCallStates: [
        ↪ DurableToolCall]? = nil, estimatedTokens: Int? = nil, revision: UInt64? = nil, turnID: TurnID? = nil) -> ToolExchangeBatch {
        ToolExchangeBatch(batchID: batchID, sessionID: sessionID, assistantMessageID: assistantMessageID, resultMessageID: resultMessageID
        ↪ ?? self.resultMessageID, toolCalls: toolCalls, toolResults: toolResults ?? self.toolResults, toolCallStates: toolCallStates ?? self
        ↪ .toolCallStates, continuationRequestID: continuationRequestID, providerStep: providerStep, state: state, estimatedTokens:
        ↪ estimatedTokens ?? self.estimatedTokens, revision: revision ?? self.revision, turnID: turnID ?? self.turnID)
    }
}

public enum ProtocolSafeContextUnit: Sendable, Equatable {
    case userTurn([ContextEntry])
    case assistantText([ContextEntry])
    case toolExchangeBatch(ToolExchangeBatch, [ContextEntry])
    case projectContext(ContextEntry)
    case derivedContext(ContextEntry)
    public var entries: [ContextEntry] {
        switch self { case let .userTurn(entries), let .assistantText(entries), let .toolExchangeBatch(_, entries): entries; case let .
        ↪ projectContext(entry), let .derivedContext(entry): [entry] }
    }
}

public enum ModelRequestProtocolValidator {
    public static func validate(_ entries: [ContextEntry]) throws {
        var pending = Set<ToolCallID>()
        var expectingResults = false
        var activeAssistantMessageID: MessageID?
        var completedAssistantMessageIDs = Set<MessageID>()

```

```

    for entry in entries {
      switch entry.part {
        case let .toolCall(call):
          guard entry.role == .assistant, !expectingResults, !completedAssistantMessageIDs.contains(entry.messageID ?? MessageID("")),
↪ (pending.isEmpty || entry.messageID == activeAssistantMessageID), pending.insert(call.callID).inserted else {
            throw CoreError(code: .contextProtocolViolation, message: "畸形或重复 ToolCall: \(call.callID.rawValue)")
          }
          activeAssistantMessageID = entry.messageID
        case let .toolResult(result):
          expectingResults = true
          guard entry.role == .tool, pending.remove(result.callID) != nil else { throw CoreError(code: .contextProtocolViolation,
↪ message: "孤立、未知或重复 ToolResult: \(result.callID.rawValue)") }
          if pending.isEmpty {
            expectingResults = false
            if let activeAssistantMessageID { completedAssistantMessageIDs.insert(activeAssistantMessageID) }
            activeAssistantMessageID = nil
          }
        case .text:
          guard entry.role != .tool, pending.isEmpty else {
            throw CoreError(code: .contextProtocolViolation, message: "ToolCall / ToolResult 顺序错误")
          }
      }
    }
    guard pending.isEmpty else { throw CoreError(code: .contextProtocolViolation, message: "ToolCall 缺少 ToolResult") }
  }
}

public struct DerivedContextPage: Sendable, Equatable, Hashable {
  public let id: String
  public let sessionId: SessionID
  public let sourceKind: DerivedContextSourceKind
  public let content: String
  public let locator: String?
  public let contentHash: String
  public let messageId: MessageID?
  public let tokenEstimate: Int
  public let createdAt: Date
  public let version: Int
  public let provenanceIDs: [String]
  public let metadata: [String: String]
  public init(id: String? = nil, sessionId: SessionID, sourceKind: DerivedContextSourceKind, content: String, messageId: MessageID?,
↪ tokenEstimate: Int, locator: String? = nil, provenanceIDs: [String] = [], metadata: [String: String] = [:], createdAt: Date = .now,
↪ version: Int = 1) {
    self.sessionID = sessionId; self.sourceKind = sourceKind; self.content = content; self.locator = locator; self.messageID = messageId
↪ ; self.tokenEstimate = tokenEstimate; self.createdAt = createdAt; self.version = version; self.provenanceIDs = provenanceIDs; self.
↪ metadata = metadata
    contentHash = ContextPage.fingerprint(content.utf8)
    self.id = id ?? "derived:\(sessionId.rawValue):\((messageID?.rawValue ?? contentHash):\((contentHash))"
  }
}

public actor DerivedContextStore {
  private var pages: [SessionID: [DerivedContextPage]] = [:]
  private var l2: [SessionID: [DerivedContextPage]] = [:]
  private var pageOutCount = 0
  private var pageInCount = 0
  private var l3Hits = 0
  private var l2Hits = 0
  private var l2Promotions = 0
  private let persistence: SQLitePersistenceStore?
  public init(persistence: SQLitePersistenceStore? = nil) { self.persistence = persistence }
  public func restore() async throws {
    guard let persistence else { return }
    pages = Dictionary(grouping: try await persistence.loadDerived(), by: \.sessionId)
  }
  public func pageOut(_ page: DerivedContextPage) async throws {

```

```

    if !(pages[page.sessionID] ?? []).contains(page) {
        pages[page.sessionID, default: []].append(page)
        pageOutCount += 1
    }
}

public func search(sessionID: SessionID, query: String, limit: Int) -> [DerivedContextPage] {
    guard !query.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty else { return [] }
    let normalizedQuery = query.lowercased()
    let terms = Set(normalizedQuery.split { !$0.isLetter && !$0.isNumber }.map(String.init))
    let identifiers = normalizedQuery.split(whereSeparator: \.isWhitespace).filter { $0.contains("-") }
    let current = l2[sessionID] ?? []
    let candidates = pages[sessionID] ?? []
    let scored = candidates.enumerated().map { index, page in
        let content = page.content.lowercased()
        let lexical = terms.reduce(0) { $0 + (content.contains($1) ? 1 : 0) }
        let identifierMatch = identifiers.contains { content.contains($0) } ? 100 : 0
        let sourceWeight = page.sourceKind == .historicalTool ? 1 : 2
        let l2Bonus = current.contains(page) ? 2 : 0
        return (page, lexical, identifierMatch, lexical * 10 + sourceWeight + l2Bonus + index)
    }
    let lexicalMatches = scored.filter { !terms.isEmpty && $0.1 > 0 }
    let identifierMatches = scored.filter { $0.2 > 0 }
    let userIdentifierMatches = identifierMatches.filter { $0.0.sourceKind == .user }
    let candidatesForPageIn = !userIdentifierMatches.isEmpty ? userIdentifierMatches : (!identifierMatches.isEmpty ? identifierMatches :
    ↪ lexicalMatches)
    let matches = candidatesForPageIn.sorted { $0.3 > $1.3 }.prefix(limit).map(\.0)
    for page in matches {
        if current.contains(page) { l2Hits += 1 } else { l2Promotions += 1 }
    }
    l3Hits += matches.count
    l2[sessionID] = Array(matches)
    pageInCount += matches.count
    return Array(matches)
}

public func metrics(sessionID: SessionID) -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int, historicalToolPages: Int,
    ↪ l3Hits: Int, l2Hits: Int, l2Promotions: Int) { (l2[sessionID]?.count ?? 0, pages[sessionID]?.count ?? 0, pageOutCount, pageInCount
    ↪ , (pages[sessionID] ?? []).filter { $0.sourceKind == .historicalTool }.count, l3Hits, l2Hits, l2Promotions) }
public func allMetrics() -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int, historicalToolPages: Int, l3Hits: Int,
    ↪ l2Hits: Int, l2Promotions: Int) {
    (l2.values.reduce(0) { $0 + $1.count }, pages.values.reduce(0) { $0 + $1.count }, pageOutCount, pageInCount, pages.values.flatMap {
    ↪ $0 }.filter { $0.sourceKind == .historicalTool }.count, l3Hits, l2Hits, l2Promotions)
}

public func pages(sessionID: SessionID) -> [DerivedContextPage] { pages[sessionID] ?? [] }
public func clear(sessionID: SessionID) {
    pages.removeValue(forKey: sessionID)
    l2.removeValue(forKey: sessionID)
}
}

public enum CompactionTrigger: String, Sendable { case automaticHighWater, manual, emergencyHardLimit }

public struct CompactionResult: Sendable {
    public let entries: [ContextEntry]
    public let beforeTokens: Int
    public let afterTokens: Int
    public let pagedOut: Int
    public let derivedCreated: Int
    public let triggered: Bool
    public let triggerSource: CompactionTrigger
    public let mandatoryFloor: Int
    public let unitsKept: Int
    public let historicalToolBatchesPagedOut: Int
    public let projectBackedOffloads: Int
    public let redundantDrops: Int
    public let emergencyTrims: Int

```

```

    public let noEligibleReduction: Bool
}

public actor ContextCompactor {
    private let estimator: any TokenEstimator
    public nonisolated let derivedStore: DerivedContextStore
    private var unitResidencies: [SessionID: [MessageID: ContextUnitDebugSnapshot]] = [:]
    public init(estimator: any TokenEstimator = ConservativeTokenEstimator(), derivedStore: DerivedContextStore = DerivedContextStore()) {
        ↪ self.estimator = estimator; self.derivedStore = derivedStore
    }
    public func restoreDerived() async throws { try await derivedStore.restore() }
    public func restoreResidencies(sessionID: SessionID, values: [ContextUnitDebugSnapshot]) {
        unitResidencies[sessionID] = Dictionary(uniqueKeysWithValues: values.map { ($0.messageID, $0) })
    }
    public func reset(sessionID: SessionID) async {
        unitResidencies.removeValue(forKey: sessionID)
        await derivedStore.clear(sessionID: sessionID)
    }
    private struct Unit {
        let indices: [Int]
        let entries: [ContextEntry]
        let batch: ToolExchangeBatch?
        let priority: Int
    }

    public func compact(sessionID: SessionID, entries: [ContextEntry], budget: ContextBudget, batches: [ToolExchangeBatch] = [],
        ↪ projectBackedContents: Set<String> = [], trigger: CompactionTrigger = .automaticHighWater) async throws -> CompactionResult {
        let before = estimator.estimate(entries: entries)
        let units = makeUnits(entries: entries, batches: batches)
        let currentUser = entries.last { $0.source == .userMessage }?.messageID
        let mandatory = units.filter { unit in
            unit.entries.contains { $0.source == .system || $0.messageID == currentUser } ||
            unit.batch.map { $0.state != .consumed } == true
        }
        let mandatoryTokens = mandatory.reduce(0) { $0 + estimator.estimate(entries: $1.entries) }
        guard mandatoryTokens <= budget.hardInputLimit else { throw CoreError(code: .contextBudgetExceeded, message: "必需上下文超出模型输入
        ↪ 预算: estimated \(before), hardLimit \(budget.hardInputLimit), mandatory \(mandatoryTokens)") }
        guard trigger != .automaticHighWater || before > budget.highWaterTokens else {
            recordResidencies(sessionID: sessionID, kept: units, pagedOut: [])
            return CompactionResult(entries: entries, beforeTokens: before, afterTokens: before, pagedOut: 0, derivedCreated: 0, triggered:
        ↪ false, triggerSource: trigger, mandatoryFloor: mandatoryTokens, unitsKept: units.count, historicalToolBatchesPagedOut: 0,
        ↪ projectBackedOffloads: 0, redundantDrops: 0, emergencyTrims: 0, noEligibleReduction: true)
        }
        let target = trigger == .emergencyHardLimit ? budget.hardInputLimit : budget.lowWaterTokens
        var kept = mandatory
        var tokens = mandatoryTokens
        var pagedOut = 0, historicalBatches = 0, projectBacked = 0, derivedCreated = 0, redundant = 0
        var derivedUnits: [(unit: Unit, page: DerivedContextPage)] = []
        var nonDerivedPagedOut: [Unit] = []
        let mandatoryIndices = Set(mandatory.flatMap(\.indices))
        let optional = units.filter { !Set($0.indices).isSubset(of: mandatoryIndices) }.sorted {
            if $0.priority != $1.priority { return $0.priority > $1.priority }
            return ($0.indices.max() ?? 0) > ($1.indices.max() ?? 0)
        }
        for unit in optional {
            let cost = estimator.estimate(entries: unit.entries)
            if tokens + cost <= target {
                kept.append(unit)
                tokens += cost
            } else {
                pagedOut += 1
                if let batch = unit.batch {
                    historicalBatches += 1
                    let backed = batch.toolResults.allSatisfy { projectBackedContents.contains($0.content) }
                    if backed { projectBacked += 1 }
                    let evidence = batch.toolCalls.enumerated().map { offset, call in

```

```

        let result = batch.toolResults.indices.contains(offset) ? batch.toolResults[offset] : nil
        let resultSummary = backed ? "projectPage=available contentHash=\(result.map { ContextPage.fingerprint($0.content.
↪ utf8) } ?? "")" : "result=\(result?.content ?? "")"
        return "tool=\(call.toolID.rawValue) arguments=\(call.arguments) status=\(result?.success == true ? "ok" : "failed")
↪ \(resultSummary)"
        }.joined(separator: "\n")
        let page = DerivedContextPage(sessionID: sessionID, sourceKind: .historicalTool, content: "[Historical tool evidence]\n
↪ \(evidence)", messageID: batch.assistantMessageID, tokenEstimate: cost, provenanceIDs: [batch.batchID])
        try await derivedStore.pageOut(page)
        derivedUnits.append((unit, page))
        derivedCreated += 1
    } else if let first = unit.entries.first, first.source != .projectPage, first.source != .derivedPage {
        let content = unit.entries.map { Self.content(of: $0.part) }.joined(separator: "\n")
        if !content.isEmpty, !projectBackedContents.contains(content) {
            let kind: DerivedContextSourceKind = first.source == .userMessage ? .user : .assistant
            let page = DerivedContextPage(sessionID: sessionID, sourceKind: kind, content: content, messageID: first.messageID,
↪ tokenEstimate: cost, provenanceIDs: first.messageID.map { [$0.rawValue] } ?? [])
            try await derivedStore.pageOut(page)
            derivedUnits.append((unit, page))
            derivedCreated += 1
        } else { nonDerivedPagedOut.append(unit) }
    } else { redundant += 1; nonDerivedPagedOut.append(unit) }
}

let keptIndices = Set(kept.flatMap(\.indices))
let output = entries.enumerated().compactMap { keptIndices.contains($0.offset) ? $0.element : nil }
recordResidencies(sessionID: sessionID, kept: kept, pagedOut: nonDerivedPagedOut, derived: derivedUnits)
return CompactionResult(entries: output, beforeTokens: before, afterTokens: estimator.estimate(entries: output), pagedOut: pagedOut,
↪ derivedCreated: derivedCreated, triggered: pagedOut > 0, triggerSource: trigger, mandatoryFloor: mandatoryTokens, unitsKept: kept.
↪ count, historicalToolBatchesPagedOut: historicalBatches, projectBackedOffloads: projectBacked, redundantDrops: redundant,
↪ emergencyTrims: trigger == .emergencyHardLimit ? pagedOut : 0, noEligibleReduction: pagedOut == 0)
}

public func pageIn(sessionID: SessionID, query: String, remainingTokens: Int) async -> [ContextEntry] {
    var remaining = max(0, remainingTokens)
    var entries: [ContextEntry] = []
    for page in await derivedStore.search(sessionID: sessionID, query: query, limit: 4) {
        let entry = ContextEntry(messageID: MessageID(page.id), role: .system, source: .derivedPage, part: .text("[Session context]\n\
↪ page.content"))
        let cost = estimator.estimate(entries: [entry])
        guard cost <= remaining else { break }
        entries.append(entry)
        remaining -= cost
    }
    return entries
}

public func cacheMetrics(sessionID: SessionID) async -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int,
↪ historicalToolPages: Int, l3Hits: Int, l2Hits: Int, l2Promotions: Int) { await derivedStore.metrics(sessionID: sessionID) }
public func cacheMetrics() async -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int, historicalToolPages: Int, l3Hits:
↪ Int, l2Hits: Int, l2Promotions: Int) { await derivedStore.allMetrics() }
public func unitStates(sessionID: SessionID) -> [ContextUnitDebugSnapshot] { (unitResidencies[sessionID] ?? []).values.sorted { $0.
↪ messageID.rawValue < $1.messageID.rawValue } }
static func content(of part: SessionMessagePart) -> String { switch part { case let .text(text): text; case let .toolCall(call): call.
↪ arguments; case let .toolResult(result): result.content + (result.error?.message ?? "") } }

private func makeUnits(entries: [ContextEntry], batches: [ToolExchangeBatch]) -> [Unit] {
    let byMessageID = Dictionary(uniqueKeysWithValues: batches.flatMap { batch in
        [batch.assistantMessageID, batch.resultMessageID].compactMap { $0 }.map { ($0, batch) }
    })
    var seen = Set<String>()
    var result: [Unit] = []
    for (index, entry) in entries.enumerated() {
        if let id = entry.messageID, let batch = byMessageID[id] {
            guard seen.insert(batch.batchID).inserted else { continue }
            let indices = entries.indices.filter { candidate in
                guard let candidateID = entries[candidate].messageID else { return false }

```



```

        return candidateID == batch.assistantMessageID || candidateID == batch.resultMessageID
    }
    let unitEntries = indices.map { entries[$0] }
    result.append(Unit(indices: indices, entries: unitEntries, batch: batch, priority: batch.state == .consumed ? 1 : 100))
} else {
    let priority: Int
    switch entry.source {
    case .system: priority = 90
    case .userMessage: priority = 70
    case .assistantMessage: priority = 60
    case .projectPage, .derivedPage: priority = 40
    case .toolCall, .toolResult: priority = 1
    }
    result.append(Unit(indices: [index], entries: [entry], batch: nil, priority: priority))
}
}
return result
}

private func recordResidencies(sessionID: SessionID, kept: [Unit], pagedOut: [Unit], derived: [(unit: Unit, page: DerivedContextPage)] =
↳ [] ) {
    var values = unitResidencies[sessionID] ?? [:]
    for unit in kept {
        for id in unit.entries.compactMap(\.messageID) {
            values[id] = ContextUnitDebugSnapshot(messageID: id, residency: .active)
        }
    }
    for unit in pagedOut {
        for id in unit.entries.compactMap(\.messageID) {
            values[id] = ContextUnitDebugSnapshot(messageID: id, residency: .pagedOut)
        }
    }
    for pair in derived {
        for id in pair.unit.entries.compactMap(\.messageID) {
            values[id] = ContextUnitDebugSnapshot(messageID: id, residency: .derived, derivedPageID: pair.page.id, contentHash: pair.
↳ page.contentHash)
        }
    }
    unitResidencies[sessionID] = values
}
}

```

// File: Sources/LingXiCore/Modules/Context/ContextPager.swift

```

import Foundation
import LingXiProtocol

/// L2 是有字符上限的 LRU 工作集，页面超出上限时只保留在 L3。
public actor L2WorkingSet {
    private let characterBudget: Int
    private let policy: L2WorkingSetPolicy
    private let rankingPolicy: ContextPageRankingPolicy
    private var entries: [String: L2WorkingSetEntry] = [:]
    private var characterCount = 0
    private var clock: UInt64 = 0

    public init(
        characterBudget: Int = 48 * 1024,
        policy: L2WorkingSetPolicy = L2WorkingSetPolicy(),
        rankingPolicy: ContextPageRankingPolicy = ContextPageRankingPolicy()
    ) {
        self.characterBudget = max(0, characterBudget)
        self.policy = policy
        self.rankingPolicy = rankingPolicy
    }
}

```

```

public func page(id: String) -> ContextPage? {
    guard var entry = entries[id] else { return nil }
    entry.lastUsed = nextClock()
    entry.useCount += 1
    entries[id] = entry
    return entry.page
}

public func search(projectRoot: URL, query: String, limit: Int) -> [ContextPage] {
    search(projectRoot: projectRoot, query: ContextQuery(currentTask: query), limit: limit)
}

public func search(projectRoot: URL, query: ContextQuery, limit: Int) -> [ContextPage] {
    let projectID = ContextPage.projectIdentifier(for: projectRoot)
    let pages = rankingPolicy.rank(
        pages: entries.values.filter { $0.page.projectRoot == projectID }.map(\.page),
        query: query,
        symbolScoresByPageID: [:]
    ).prefix(max(0, limit)).map { $0.page }
    touch(pages.map(\.id))
    return pages
}

@discardableResult
public func promote(_ pages: [ContextPage], queryRelevance: Double = 0, taskAffinity: Double = 0, explicitPin: Bool = false) ->
↳ L2PromotionResult {
    var admitted: [ContextPage] = []
    var evicted: [ContextPage] = []
    var seen = Set<String>()
    for page in pages where seen.insert(page.id).inserted {
        if let old = entries.removeValue(forKey: page.id) {
            characterCount -= old.page.characterCount
        }
        guard page.characterCount <= characterBudget else { continue }
        while characterCount + page.characterCount > characterBudget, let victimID = policy.evictionCandidate(in: entries, clock: clock)
↳ , let victim = entries.removeValue(forKey: victimID) {
            characterCount -= victim.page.characterCount
            evicted.append(victim.page)
        }
        entries[page.id] = L2WorkingSetEntry(page: page, lastUsed: nextClock(), useCount: 1, queryRelevance: queryRelevance,
↳ taskAffinity: taskAffinity, explicitPin: explicitPin)
        characterCount += page.characterCount
        admitted.append(page)
    }
    return L2PromotionResult(admitted: admitted, evicted: evicted)
}

public func touch(_ pageIDs: [String]) {
    var seen = Set<String>()
    for id in pageIDs where seen.insert(id).inserted {
        guard var entry = entries[id] else { continue }
        entry.lastUsed = nextClock()
        entry.useCount += 1
        entries[id] = entry
    }
}

public func invalidate(_ pageIDs: [String]) {
    var seen = Set<String>()
    for id in pageIDs where seen.insert(id).inserted {
        if let removed = entries.removeValue(forKey: id) {
            characterCount -= removed.page.characterCount
        }
    }
}

```

```

}

public func invalidate(projectRoot: URL) {
    let projectID = ContextPage.projectIdentifier(for: projectRoot)
    invalidate(entries.values.filter { $0.page.projectRoot == projectID }.map { $0.page.id })
}

public func metrics() -> L2WorkingSetMetrics {
    L2WorkingSetMetrics(pageCount: entries.count, characterCount: characterCount)
}

private func nextClock() -> UInt64 {
    clock &+= 1
    return clock
}
}

```

/// 将确定性 L3 查询结果按项目预算装配为模型可用上下文，并维护 L2 命中统计。

```

public actor ContextPager {
    private let store: ProjectPageStore
    private let workingSet: L2WorkingSet
    private let projectCharacterBudget: Int
    private var hits = 0
    private var misses = 0
    private var pageFaults = 0
    private var l3Queries = 0
    private var l3Candidates = 0
    private var l3Materializations = 0
    private var staleRebuilds = 0
    private var initialIndexedFiles = 0
    private var promotions = 0
    private var evictions = 0
    private var selectedPages = 0
    private var injectedCharacters = 0
    private var retrievalMilliseconds = 0.0
    private var materializationMilliseconds = 0.0
    private var queryCharacters = 0
    private var queryTerms = 0
    private var candidatePages = 0
    private var candidateCharacters = 0
    private var latestSelectedPages = 0
    private var latestSelectedCharacters = 0
    private var latestInjectedPages = 0
    private var latestInjectedCharacters = 0
    private var filesChecked = 0
    private var filesRebuilt = 0
    private var scanMilliseconds = 0
    private var symbolHints = 0
    private var symbolExactMatches = 0
    private var symbolQualifiedExactMatches = 0
    private var symbolFallbackExactMatches = 0
    private var symbolPrefixMatches = 0
    private var symbolCandidatePages = 0
    private var symbolHintExtractionMilliseconds = 0.0
    private var symbolExactLookupMilliseconds = 0.0
    private var symbolPrefixLookupMilliseconds = 0.0
    private var symbolCandidateMergeMilliseconds = 0.0
    private var symbolRankingMilliseconds = 0.0
    private var symbolTotalMilliseconds = 0.0
    private var lexicalCandidatePages = 0
    private var currentSourceCandidates = 0
    private var documentationCandidates = 0
    private var referenceCandidates = 0
    private var relationHints = 0
}

```

```

private var directReferenceHits = 0
private var dependencyHits = 0
private var relatedPages = 0
private var referenceResolutionMilliseconds = 0.0
private var referenceExpansionMilliseconds = 0.0

public init(store: ProjectPageStore, workingSet: L2WorkingSet, projectCharacterBudget: Int = 48 * 1024) {
    self.store = store
    self.workingSet = workingSet
    self.projectCharacterBudget = max(0, projectCharacterBudget)
}

@discardableResult
public func rebuildStaleFiles(using scanner: ProjectScanner) async throws -> ProjectPageStoreUpdate {
    let update = try await store.rebuildStaleFiles(using: scanner)
    await workingSet.invalidate(update.invalidatedPageIDs)
    initialIndexedFiles += update.initialIndexedPaths.count
    staleRebuilds += update.rebuiltPaths.count
    filesChecked += update.filesChecked
    filesRebuilt += update.filesRebuilt
    scanMilliseconds += update.scanMilliseconds
    return update
}

public func query(projectRoot: URL, query: String, limit: Int = 20) async -> ContextPagerResult {
    await self.query(projectRoot: projectRoot, query: ContextQuery(currentTask: query), limit: limit)
}

public func query(projectRoot: URL, query: ContextQuery, limit: Int = 20) async -> ContextPagerResult {
    guard limit > 0, (!query.terms.isEmpty || !query.symbolHints.isEmpty) else { return ContextPagerResult(pages: [], metrics: metrics())
    ↵ } }
    let clock = ContinuousClock()
    let retrievalStart = clock.now
    let l2Candidates = await workingSet.search(projectRoot: projectRoot, query: query, limit: limit)
    let candidates: [ContextPage]
    let symbolMetrics: SymbolSearchMetrics
    let requiresStructuralCoverage = !query.symbolHints.isEmpty
    if l2Candidates.isEmpty || requiresStructuralCoverage {
        let result = await store.searchResult(projectRoot: projectRoot, query: query, limit: .max)
        candidates = result.pages
        symbolMetrics = result.symbolMetrics
        l3Queries += 1
        l3Candidates += candidates.count
    } else {
        candidates = l2Candidates
        symbolMetrics = .zero
    }
    retrievalMilliseconds += milliseconds(retrievalStart.duration(to: clock.now))
    queryCharacters = query.text.count
    queryTerms = query.terms.count
    candidatePages = candidates.count
    candidateCharacters = candidates.reduce(0) { $0 + $1.characterCount }
    symbolHints = symbolMetrics.hints
    symbolExactMatches = symbolMetrics.exactMatches
    symbolQualifiedExactMatches = symbolMetrics.qualifiedExactMatches
    symbolFallbackExactMatches = symbolMetrics.fallbackExactMatches
    symbolPrefixMatches = symbolMetrics.prefixMatches
    symbolCandidatePages = symbolMetrics.candidatePages
    symbolHintExtractionMilliseconds = query.symbolHintExtractionMilliseconds
    symbolExactLookupMilliseconds = symbolMetrics.exactLookupMilliseconds
    symbolPrefixLookupMilliseconds = symbolMetrics.prefixLookupMilliseconds
    symbolCandidateMergeMilliseconds = symbolMetrics.candidateMergeMilliseconds
    symbolRankingMilliseconds = symbolMetrics.rankingMilliseconds
    symbolTotalMilliseconds = query.symbolHintExtractionMilliseconds + symbolMetrics.totalMilliseconds
    lexicalCandidatePages = symbolMetrics.lexicalCandidatePages

```

```

currentSourceCandidates = symbolMetrics.currentSourceCandidates
documentationCandidates = symbolMetrics.documentationCandidates
referenceCandidates = symbolMetrics.referenceCandidates
relationHints = symbolMetrics.relationHints
directReferenceHits = symbolMetrics.directReferenceHits
dependencyHits = symbolMetrics.dependencyHits
relatedPages = symbolMetrics.relatedPages
referenceResolutionMilliseconds = symbolMetrics.resolutionMilliseconds
referenceExpansionMilliseconds = symbolMetrics.expansionMilliseconds
var selected: [ContextPage] = []
var seenIDs = Set<String>()
var seenContent = Set<String>()
var characters = 0
for page in candidates where selected.count < limit {
    guard seenIDs.insert(page.id).inserted, seenContent.insert(page.content).inserted,
        characters + page.characterCount <= projectCharacterBudget else {
        continue
    }
    selected.append(page)
    characters += page.characterCount
}
let materializationStart = clock.now
let before = (hits, misses, pageFaults, promotions, evictions)
let resolved = await resolve(selected)
let materialization = milliseconds(materializationStart.duration(to: clock.now))
materializationMilliseconds += materialization
l3Materializations += resolved.count
selectedPages += resolved.count
injectedCharacters += resolved.reduce(0) { $0 + $1.characterCount }
latestSelectedPages = selected.count
latestSelectedCharacters = characters
return ContextPagerResult(
    pages: resolved,
    metrics: metrics(),
    candidatePageCount: candidates.count,
    candidateCharacterCount: candidates.reduce(0) { $0 + $1.characterCount },
    selectedPageCount: selected.count,
    selectedCharacterCount: characters,
    turnMetrics: ContextPagingTurnMetrics(
        lookups: (hits - before.0) + (misses - before.1), hits: hits - before.0, misses: misses - before.1,
        pageFaults: pageFaults - before.2, promotions: promotions - before.3, evictions: evictions - before.4,
        candidatePages: candidates.count, candidateCharacters: candidates.reduce(0) { $0 + $1.characterCount },
        selectedPages: selected.count, selectedCharacters: characters,
        retrievalMilliseconds: milliseconds(retrievalStart.duration(to: materializationStart)), materializationMilliseconds:
↪ materialization,
        symbolHints: symbolMetrics.hints, symbolExactMatches: symbolMetrics.exactMatches,
        symbolQualifiedExactMatches: symbolMetrics.qualifiedExactMatches, symbolFallbackExactMatches: symbolMetrics.
↪ fallbackExactMatches,
        symbolPrefixMatches: symbolMetrics.prefixMatches, symbolCandidatePages: symbolMetrics.candidatePages,
        symbolHintExtractionMilliseconds: query.symbolHintExtractionMilliseconds,
        symbolExactLookupMilliseconds: symbolMetrics.exactLookupMilliseconds,
        symbolPrefixLookupMilliseconds: symbolMetrics.prefixLookupMilliseconds,
        symbolCandidateMergeMilliseconds: symbolMetrics.candidateMergeMilliseconds,
        symbolRankingMilliseconds: symbolMetrics.rankingMilliseconds,
        symbolTotalMilliseconds: query.symbolHintExtractionMilliseconds + symbolMetrics.totalMilliseconds,
        lexicalCandidatePages: symbolMetrics.lexicalCandidatePages,
        currentSourceCandidates: symbolMetrics.currentSourceCandidates,
        documentationCandidates: symbolMetrics.documentationCandidates,
        referenceCandidates: symbolMetrics.referenceCandidates,
        relationHints: symbolMetrics.relationHints, directReferenceHits: symbolMetrics.directReferenceHits,
        dependencyHits: symbolMetrics.dependencyHits, relatedPages: symbolMetrics.relatedPages,
        referenceResolutionMilliseconds: symbolMetrics.resolutionMilliseconds,
        referenceExpansionMilliseconds: symbolMetrics.expansionMilliseconds
    )
)

```

```

}

public func page(projectRoot: URL, id: String) async -> ContextPage? {
    guard let stored = await store.page(projectRoot: projectRoot, id: id) else {
        misses += 1
        return nil
    }
    return (await resolve([stored])).first
}

public func symbolLookup(projectRoot: URL, query: String, mode: String) async -> [Symbol] {
    await store.symbolLookup(projectRoot: projectRoot, query: query, mode: mode)
}

public func references(projectRoot: URL, symbolID: SymbolID) async -> [ProjectReference] {
    await store.references(projectRoot: projectRoot, symbolID: symbolID)
}

public func dependencies(projectRoot: URL, path: String, incoming: Bool) async -> [DependencyEdge] {
    await store.dependencies(projectRoot: projectRoot, path: path, incoming: incoming)
}

public func metrics() -> ContextPagerMetrics {
    ContextPagerMetrics(hits: hits, misses: misses, pageFaults: pageFaults)
}

public func invalidate(projectRoot: URL) async {
    await workingSet.invalidate(projectRoot: projectRoot)
}

public func debugMetrics(projectRoot: URL) async -> ContextPagingDebugMetrics {
    let l2 = await workingSet.metrics()
    let l3 = await store.statistics(projectRoot: projectRoot)
    return ContextPagingDebugMetrics(
        queryCharacters: queryCharacters,
        queryTerms: queryTerms,
        candidatePages: candidatePages,
        candidateCharacters: candidateCharacters,
        selectedPages: latestSelectedPages,
        selectedCharacters: latestSelectedCharacters,
        injectedPages: latestInjectedPages,
        injectedCharacters: latestInjectedCharacters,
        filesChecked: filesChecked,
        filesRebuilt: filesRebuilt,
        scanMilliseconds: scanMilliseconds,
        initialIndexedFiles: initialIndexedFiles,
        l2Lookups: hits + misses,
        l2Hits: hits,
        l2Misses: misses,
        l2Pages: l2.pageCount,
        l2Characters: l2.characterCount,
        l3Pages: l3.pages,
        l3Queries: l3.queries,
        l3Candidates: l3.candidates,
        l3Materializations: l3.materializations,
        staleRebuilds: staleRebuilds,
        pageFaults: pageFaults,
        promotions: promotions,
        evictions: evictions,
        retrievalMilliseconds: retrievalMilliseconds,
        materializationMilliseconds: materializationMilliseconds,
        symbolCount: l3.symbols,
        symbolIndexedFiles: l3.symbolFiles,
        symbolHints: symbolHints,
        symbolExactMatches: symbolExactMatches,
    )
}

```

```

symbolQualifiedExactMatches: symbolQualifiedExactMatches,
symbolFallbackExactMatches: symbolFallbackExactMatches,
symbolPrefixMatches: symbolPrefixMatches,
symbolCandidatePages: symbolCandidatePages,
symbolHintExtractionMilliseconds: symbolHintExtractionMilliseconds,
symbolExactLookupMilliseconds: symbolExactLookupMilliseconds,
symbolPrefixLookupMilliseconds: symbolPrefixLookupMilliseconds,
symbolCandidateMergeMilliseconds: symbolCandidateMergeMilliseconds,
symbolRankingMilliseconds: symbolRankingMilliseconds,
symbolTotalMilliseconds: symbolTotalMilliseconds,
lexicalCandidatePages: lexicalCandidatePages,
currentSourceCandidates: currentSourceCandidates,
documentationCandidates: documentationCandidates,
referenceCandidates: referenceCandidates
, referenceCount: l3.references.referenceCount, resolvedReferenceCount: l3.references.resolvedCount,
ambiguousReferenceCount: l3.references.ambiguousCount, unresolvedReferenceCount: l3.references.unresolvedCount,
dependencyCount: l3.references.dependencyCount, referenceIndexedFiles: l3.references.indexedFileCount,
relationHints: relationHints, directReferenceHits: directReferenceHits, dependencyHits: dependencyHits,
relatedPages: relatedPages, referenceResolutionMilliseconds: referenceResolutionMilliseconds,
referenceExpansionMilliseconds: referenceExpansionMilliseconds
)
}

public func recordInjection(_ pages: [ContextPage]) {
    latestInjectedPages = pages.count
    latestInjectedCharacters = pages.reduce(0) { $0 + $1.characterCount }
}

/// Session/Derived L3 与 Project L2/L3 共用本 pager 进入 L1, 绝不产生 Provider tool role。
public func pageInDerived(store: DerivedContextStore, sessionID: SessionID, query: String, remainingTokens: Int) async -> [ContextEntry]
↳ {
    var remaining = max(0, remainingTokens)
    var entries: [ContextEntry] = []
    let estimator = ConservativeTokenEstimator()
    for page in await store.search(sessionID: sessionID, query: query, limit: 4) {
        let entry = ContextEntry(messageID: MessageID(page.id), role: .system, source: .derivedPage, part: .text("[Session context]\n\(
↳ page.content)"))
        let cost = estimator.estimate(entries: [entry])
        guard cost <= remaining else { break }
        entries.append(entry)
        remaining -= cost
    }
    return entries
}

private func resolve(_ pages: [ContextPage]) async -> [ContextPage] {
    var resolved: [ContextPage] = []
    var faults: [ContextPage] = []
    for page in pages {
        let cached = await workingSet.page(id: page.id)
        if let cached, cached.hash == page.hash, cached.version == page.version {
            hits += 1
            resolved.append(cached)
        } else {
            if cached != nil { await workingSet.invalidate([page.id]) }
            misses += 1
            pageFaults += 1
            faults.append(page)
            resolved.append(page)
        }
    }
    let promotion = await workingSet.promote(faults)
    promotions += promotion.admitted.count
    evictions += promotion.evicted.count
    return resolved
}

```

```

}

private func milliseconds(_ duration: Duration) -> Double {
    let components = duration.components
    return Double(components.seconds) * 1_000 + Double(components.attoseconds) / 1_000_000_000_000_000
}

private func lexicalTerms(_ query: String) -> [String] {
    query.lowercased().split { !$0.isLetter && !$0.isNumber }.map(String.init).filter { $0.count > 1 }
}
}

```

// File: Sources/LingXiCore/Modules/Context/ContextPageRankingPolicy.swift

```

import Foundation

public struct PageCandidate: Sendable, Equatable {
    public let page: ContextPage
    public let score: Int
    public let symbolScore: Int
    public let referenceScore: Int
    public let textScore: Int
    public let authorityScore: Int

    public init(page: ContextPage, score: Int, symbolScore: Int = 0, referenceScore: Int = 0, textScore: Int = 0, authorityScore: Int = 0) {
        self.page = page
        self.score = score
        self.symbolScore = symbolScore
        self.referenceScore = referenceScore
        self.textScore = textScore
        self.authorityScore = authorityScore
    }
}

/// 符号与文本命中在同一候选序列中排序，Pager 无需区分检索来源。
public struct ContextPageRankingPolicy: Sendable {
    public let maximumSymbolContribution: Int
    public let maximumReferenceContribution: Int

    public init(maximumSymbolContribution: Int = 1_200, maximumReferenceContribution: Int = 800) {
        self.maximumSymbolContribution = max(0, maximumSymbolContribution)
        self.maximumReferenceContribution = max(0, maximumReferenceContribution)
    }

    public func rank(pages: [ContextPage], query: ContextQuery, symbolScoresByPageID: [String: Int], referenceScoresByPageID: [String: Int])
    ↪ => [Int]) -> [PageCandidate] {
        let lexicalTokens = query.terms
        guard !lexicalTokens.isEmpty || !query.symbolHints.isEmpty else { return [] }
        return pages.compactMap { page in
            let symbolScore = min(maximumSymbolContribution, symbolScoresByPageID[page.id, default: 0])
            let referenceScore = min(maximumReferenceContribution, referenceScoresByPageID[page.id, default: 0])
            let pathScore = pathScore(page, tokens: lexicalTokens)
            let headingScore = headingScore(page, tokens: lexicalTokens)
            let lexicalScore = lexicalScore(page, tokens: lexicalTokens)
            let textScore = pathScore + headingScore + lexicalScore

            // Relevance floor: require at least one match before applying authority
            guard symbolScore > 0 || referenceScore > 0 || textScore > 0 else { return nil }

            let authorityScore = sourceAuthority(for: page.sourceType)
            let score = symbolScore + referenceScore + textScore + authorityScore
            return PageCandidate(page: page, score: score, symbolScore: symbolScore, referenceScore: referenceScore, textScore: textScore,
            ↪ authorityScore: authorityScore)
        }.sorted {
            if $0.score != $1.score { return $0.score > $1.score }
        }
    }
}

```



```

        if $0.page.path != $1.page.path { return $0.page.path < $1.page.path }
        if $0.page.startLine != $1.page.startLine { return $0.page.startLine < $1.page.startLine }
        return $0.page.endLine < $1.page.endLine
    }
}

private func pathScore(_ page: ContextPage, tokens: [String]) -> Int {
    let path = page.path.lowercased()
    let filename = URL(fileURLWithPath: path).lastPathComponent.lowercased()
    return tokens.reduce(0) { score, token in
        if path == token || path.hasSuffix("/\(token)") { return score + 700 }
        if filename.contains(token) { return score + 600 }
        return score
    }
}

private func headingScore(_ page: ContextPage, tokens: [String]) -> Int {
    let heading = page.metadata.heading?.lowercased() ?? ""
    return tokens.reduce(0) { $0 + (heading.contains($1) ? 300 : 0) }
}

private func lexicalScore(_ page: ContextPage, tokens: [String]) -> Int {
    let content = page.content.lowercased()
    return tokens.reduce(0) { score, token in
        var occurrences = 0
        var range = content.startIndex..

```

// File: Sources/LingXiCore/Modules/Context/ECoreObjectFabric.swift

```

import Foundation
import LingXiProtocol

/// 强类型上下文对象唯一标识符
public struct ContextObjectID: Sendable, Equatable, Hashable, Codable, CustomStringConvertible {
    public let rawValue: String

    public var description: String { rawValue }

    public init(_ rawValue: String) throws {
        // 安全防护: 严格禁止路径穿越, 只允许字母数字短横线与下划线
        guard !rawValue.isEmpty,
              rawValue.count <= 128,
              rawValue.allSatisfy({ $0.isLetter || $0.isNumber || $0 == "_" || $0 == "-" }) else {
            throw CoreError(code: .toolArgumentInvalid, message: "Invalid ContextObjectID: \(rawValue)")
        }
    }
}

```

```

        self.rawValue = rawValue
    }

    public init(unchecked rawValue: String) {
        self.rawValue = rawValue
    }

    public init(from decoder: Decoder) throws {
        let container = try decoder.singleValueContainer()
        let value = try container.decode(String.self)
        try self.init(value)
    }

    public func encode(to encoder: Encoder) throws {
        var container = encoder.singleValueContainer()
        try container.encode(rawValue)
    }

    /// 依据工具调用与内容哈希确定性生成 ContextObjectID
    public static func generate(toolName: String, callID: ToolCallID, content: String) -> ContextObjectID {
        let sanitizedTool = toolName.filter { $0.isLetter || $0.isNumber || $0 == "_" }
        let cleanTool = sanitizedTool.isEmpty ? "obj" : sanitizedTool
        let cleanCall = callID.rawValue.filter { $0.isLetter || $0.isNumber || $0 == "_" || $0 == "-" }.prefix(12)

        var hash: UInt64 = 14_695_981_039_346_656_037
        for byte in content.utf8 {
            hash ^= UInt64(byte)
            hash &*= 1_099_511_628_211
        }
        let hashStr = String(format: "%08llx", hash)
        return ContextObjectID(unchecked: "obj_\(cleanTool)_\((cleanCall)_\((hashStr))")
    }
}

/// 外部权威观测对象元数据 (E-Core Context Object Metadata)
public struct ObservationMetadata: Sendable, Equatable, Codable {
    public let objectID: ContextObjectID
    public let toolCallID: ToolCallID
    public let toolName: String
    public let contentType: String
    public let totalLines: Int
    public let totalBytes: Int
    public let createdAt: Date
    public let contentHash: String

    public init(
        objectID: ContextObjectID,
        toolCallID: ToolCallID,
        toolName: String,
        contentType: String = "text/plain",
        totalLines: Int,
        totalBytes: Int,
        createdAt: Date = .now,
        contentHash: String
    ) {
        self.objectID = objectID
        self.toolCallID = toolCallID
        self.toolName = toolName
        self.contentType = contentType
        self.totalLines = totalLines
        self.totalBytes = totalBytes
        self.createdAt = createdAt
        self.contentHash = contentHash
    }
}

```

/// 召回切片数据传输对象

```
public struct RecallChunk: Sendable, Equatable, Codable {
    public let objectID: ContextObjectID
    public let offsetBytes: Int
    public let lengthBytes: Int
    public let content: String
    public let hasMore: Bool
    public let startLine: Int
    public let endLine: Int
    public let totalLines: Int
    public let totalBytes: Int

    public init(
        objectID: ContextObjectID,
        offsetBytes: Int,
        lengthBytes: Int,
        content: String,
        hasMore: Bool,
        startLine: Int,
        endLine: Int,
        totalLines: Int,
        totalBytes: Int
    ) {
        self.objectID = objectID
        self.offsetBytes = offsetBytes
        self.lengthBytes = lengthBytes
        self.content = content
        self.hasMore = hasMore
        self.startLine = startLine
        self.endLine = endLine
        self.totalLines = totalLines
        self.totalBytes = totalBytes
    }
}
```

/// E-Core 对象 Fabric 存储器：负责管理会话级外部持久化对象。

/// 遵循三大纪律八项注意：

/// 1. **Fail-Open**：所有写入与读取异常降级处理，绝不崩溃智能体主流程。

/// 2. 安全防路径穿越：严密过滤 **objectID** 字符。

/// 3. 原子写入与去重。

```
public actor ECoreObjectStore {
    public let baseDirectory: URL
    public let configuration: ContextObjectFabricConfiguration
    private var metadataCache: [SessionID: [ContextObjectID: ObservationMetadata]] = [:]

    public init(
        baseDirectory: URL? = nil,
        configuration: ContextObjectFabricConfiguration = ContextObjectFabricConfiguration()
    ) {
        self.configuration = configuration
        if let baseDirectory {
            self.baseDirectory = baseDirectory
        } else {
            let home = FileManager.default.homeDirectoryForCurrentUser
            self.baseDirectory = home.appendingPathComponent(".lingxiagent", isDirectory: true).appendingPathComponent("sessions",
↵ isDirectory: true)
        }
    }
}

/// 获取特定 Session 的对象存储根目录
private func sessionObjectsDirectory(sessionID: SessionID) -> URL {
    let safeSessionID = sessionID.rawValue.filter { $0.isLetter || $0.isNumber || $0 == "_" || $0 == "-" }
    return baseDirectory
        .appendingPathComponent(safeSessionID, isDirectory: true)
}
```

```

        .appendingPathComponent("objects", isDirectory: true)
    }

    /// 旁路存储对象：如果超过阈值且开启了 ecoreStorageEnabled，则持久化到磁盘
    @discardableResult
    public func store(
        sessionID: SessionID,
        toolCallID: ToolCallID,
        toolName: String,
        content: String,
        contentType: String = "text/plain",
        force: Bool = false
    ) async -> ObservationMetadata? {
        guard configuration.ecoreStorageEnabled else { return nil }
        let byteCount = content.utf8.count
        guard force || byteCount >= configuration.objectizationThreshold else {
            return nil
        }

        let objectID = ContextObjectID.generate(toolName: toolName, callID: toolCallID, content: content)

        // 计算行数
        var lineCount = 0
        for byte in content.utf8 {
            if byte == 10 { // '\n'
                lineCount += 1
            }
        }
        if !content.isEmpty && !content.hasSuffix("\n") {
            lineCount += 1
        }

        var hash: UInt64 = 14_695_981_039_346_656_037
        for byte in content.utf8 {
            hash ^= UInt64(byte)
            hash &*= 1_099_511_628_211
        }
        let contentHash = String(format: "%016llx", hash)

        let metadata = ObservationMetadata(
            objectID: objectID,
            toolCallID: toolCallID,
            toolName: toolName,
            contentType: contentType,
            totalLines: max(1, lineCount),
            totalBytes: byteCount,
            createdAt: .now,
            contentHash: contentHash
        )

        // Fail-Open 磁盘写入
        do {
            let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
            try FileManager.default.createDirectory(at: objectsDir, withIntermediateDirectories: true)

            let targetURL = objectsDir.appendingPathComponent("\(objectID.rawValue).txt", isDirectory: false)
            let tempURL = objectsDir.appendingPathComponent("\(objectID.rawValue).\(UUID().uuidString).tmp", isDirectory: false)
            let metaURL = objectsDir.appendingPathComponent("\(objectID.rawValue).meta.json", isDirectory: false)

            if !FileManager.default.fileExists(atPath: targetURL.path) {
                try content.write(to: tempURL, atomically: true, encoding: .utf8)
                _ = try? FileManager.default.removeItem(at: targetURL)
                try FileManager.default.moveItem(at: tempURL, to: targetURL)
            }
        }
    }

```

```

        let metaData = try JSONEncoder().encode(metadata)
        try metaData.write(to: metaURL, options: .atomic)

        if metadataCache[sessionID] == nil {
            metadataCache[sessionID] = [:]
        }
        metadataCache[sessionID]?[objectID] = metadata
        return metadata
    } catch {
        // Fail-open: 记录警告但不中断
        FileHandle.standardError.write(Data("[E-CORE WARNING] Failed to persist object \(objectID.rawValue): \(error)\n".utf8))
        return nil
    }
}

/// 获取完整对象内容
public func fetch(sessionID: SessionID, objectID: ContextObjectID) async throws -> String? {
    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    let fileURL = objectsDir.appendingPathComponent("\(objectID.rawValue).txt", isDirectory: false)
    guard FileManager.default.fileExists(atPath: fileURL.path) else {
        return nil
    }
    return try String(contentsOf: fileURL, encoding: .utf8)
}

/// 精确范围召回 (context_recall 核心调用)
public func recall(
    sessionID: SessionID,
    objectID: ContextObjectID,
    offsetBytes: Int = 0,
    limitBytes: Int? = nil,
    limitLines: Int? = nil
) async throws -> RecallChunk? {
    guard let content = try await fetch(sessionID: sessionID, objectID: objectID) else {
        return nil
    }

    let maxBytes = min(limitBytes ?? configuration.recallMaxBytes, configuration.recallMaxBytes)
    let maxLines = min(limitLines ?? configuration.recallMaxLines, configuration.recallMaxLines)

    let totalBytes = content.utf8.count
    guard offsetBytes >= 0, offsetBytes < totalBytes else {
        return RecallChunk(
            objectID: objectID,
            offsetBytes: offsetBytes,
            lengthBytes: 0,
            content: "",
            hasMore: false,
            startLine: 1,
            endLine: 1,
            totalLines: 1,
            totalBytes: totalBytes
        )
    }

    let utf8Data = Data(content.utf8)
    let startIdx = offsetBytes
    let endIdx = min(totalBytes, startIdx + maxBytes)
    let sliceData = utf8Data.subdata(in: startIdx..

```

```

let actualLinesUsed: Int
if lines.count > maxLines {
    var sliceWithNewline = lines.prefix(maxLines).joined(separator: "\n")
    if startIdx + sliceWithNewline.utf8.count < totalBytes && utf8Data[startIdx + sliceWithNewline.utf8.count] == 10 {
        sliceWithNewline.append("\n")
    }
    finalSlice = sliceWithNewline
    actualLinesUsed = maxLines
} else {
    finalSlice = sliceString
    actualLinesUsed = lines.count
}

let actualLength = finalSlice.utf8.count
let hasMore = (startIdx + actualLength) < totalBytes

// 计算当前 offset 处的行号
let prefixData = utf8Data.prefix(startIdx)
var startLine = 1
for b in prefixData {
    if b == 10 { startLine += 1 }
}
let endLine = startLine + max(0, actualLinesUsed - 1)

var totalLines = 0
for b in utf8Data {
    if b == 10 { totalLines += 1 }
}
if !content.isEmpty && !content.hasSuffix("\n") {
    totalLines += 1
}

return RecallChunk(
    objectID: objectID,
    offsetBytes: startIdx,
    lengthBytes: actualLength,
    content: finalSlice,
    hasMore: hasMore,
    startLine: startLine,
    endLine: endLine,
    totalLines: max(1, totalLines),
    totalBytes: totalBytes
)
}

/// 获取对象元数据
public func metadata(sessionID: SessionID, objectID: ContextObjectID) async -> ObservationMetadata? {
    if let cached = metadataCache[sessionID]?[objectID] {
        return cached
    }
    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    let metaURL = objectsDir.appendingPathComponent("\(objectID.rawValue).meta.json", isDirectory: false)
    guard let data = try? Data(contentsOf: metaURL),
        let meta = try? JSONDecoder().decode(ObservationMetadata.self, from: data) else {
        return nil
    }
    if metadataCache[sessionID] == nil {
        metadataCache[sessionID] = [:]
    }
    metadataCache[sessionID]?[objectID] = meta
    return meta
}

/// 检查对象是否存在
public func hasObject(sessionID: SessionID, objectID: ContextObjectID) async -> Bool {

```

```

    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    let fileURL = objectsDir.appendingPathComponent("\(objectID.rawValue).txt", isDirectory: false)
    return FileManager.default.fileExists(atPath: fileURL.path)
}

/// 列出指定会话下所有沉淀的 E-Core 观测对象元数据
public func listObjects(sessionID: SessionID) async -> [ObservationMetadata] {
    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    guard let fileURLs = try? FileManager.default.contentsOfDirectory(
        at: objectsDir,
        includingPropertiesForKeys: nil,
        options: [.skipsHiddenFiles]
    ) else {
        return Array(metadataCache[sessionID]?.values ?? [:].values)
    }

    var results: [ObservationMetadata] = []
    for url in fileURLs where url.pathExtension == "json" && url.lastPathComponent.contains(".meta.") {
        let baseName = url.deletingPathExtension().deletingPathExtension().lastPathComponent
        if let objID = try? ContextObjectID(baseName) {
            if let meta = await metadata(sessionID: sessionID, objectID: objID) {
                results.append(meta)
            }
        }
    }
    if results.isEmpty, let cached = metadataCache[sessionID] {
        return Array(cached.values)
    }
    return results
}

/// 依据保留的 ToolCallIDs 裁剪废弃的观测对象文件与缓存（用于撤回或会话状态协同）
public func prune(sessionID: SessionID, keepingToolCallIDs: Set<ToolCallID>) async {
    if keepingToolCallIDs.isEmpty {
        await cleanSession(sessionID: sessionID)
        return
    }
    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    guard let fileURLs = try? FileManager.default.contentsOfDirectory(
        at: objectsDir,
        includingPropertiesForKeys: nil,
        options: [.skipsHiddenFiles]
    ) else { return }

    for url in fileURLs where url.pathExtension == "json" && url.lastPathComponent.contains(".meta.") {
        let baseName = url.deletingPathExtension().deletingPathExtension().lastPathComponent
        guard let objID = try? ContextObjectID(baseName),
              let meta = await metadata(sessionID: sessionID, objectID: objID) else { continue }
        if !keepingToolCallIDs.contains(meta.toolCallID) {
            metadataCache[sessionID]?.removeValue(forKey: objID)
            let txtURL = objectsDir.appendingPathComponent("\(objID.rawValue).txt", isDirectory: false)
            try? FileManager.default.removeItem(at: url)
            try? FileManager.default.removeItem(at: txtURL)
        }
    }
}

/// 重置或清理 session 存储
public func cleanSession(sessionID: SessionID) async {
    metadataCache.removeValue(forKey: sessionID)
    let objectsDir = sessionObjectsDirectory(sessionID: sessionID)
    try? FileManager.default.removeItem(at: objectsDir)
}
}

```

// File: Sources/LingXiCore/Modules/Context/ProjectContextDomain.swift

```

import Foundation
import LingXiProtocol

public enum ContextPageSourceType: String, Sendable, Equatable, Hashable {
    case sourceFile
    case documentation
    case referenceDocumentation
    case researchArchive
    case configuration
    case test
    case projectMetadata
}

public struct ContextPageMetadata: Sendable, Equatable, Hashable {
    public let path: String
    public let language: String?
    public let heading: String?

    public init(path: String, language: String? = nil, heading: String? = nil) {
        self.path = path
        self.language = language
        self.heading = heading
    }
}

/// 可注入模型项目上下文的最小、不可变文本页。
public struct ContextPage: Sendable, Equatable, Hashable {
    public let id: String
    public let projectID: ProjectID?
    public let fileID: ProjectFileID?
    public let projectRoot: String
    public let path: String
    public let startLine: Int
    public let endLine: Int
    public let content: String
    public let hash: String
    public let version: String
    public let sourceType: ContextPageSourceType
    public let metadata: ContextPageMetadata

    public var characterCount: Int { content.count }
    public var byteCount: Int { content.utf8.count }

    public init(
        projectRoot: String,
        projectID: ProjectID? = nil,
        fileID: ProjectFileID? = nil,
        path: String,
        startLine: Int,
        endLine: Int,
        content: String,
        hash: String? = nil,
        version: String? = nil,
        sourceType: ContextPageSourceType = .sourceFile,
        metadata: ContextPageMetadata? = nil
    ) {
        self.projectRoot = projectRoot
        self.projectID = projectID
        self.fileID = fileID
        self.path = path
        self.startLine = startLine
        self.endLine = endLine
        self.content = content
        self.hash = hash ?? Self.fingerprint(content.utf8)
    }
}

```



```

        self.version = version ?? Self.fingerprint(content.utf8)
        self.sourceType = sourceType
        self.metadata = metadata ?? ContextPageMetadata(path: path)
        self.id = projectID.flatMap { project in fileID.map { "page:" + Self.fingerprint("\(project.rawValue)|\($0.rawValue)|\(startLine)|\(\(\(\endLine)".utf8) } } } ?? "\(projectRoot)|\(path):\(\startLine)-\(\endLine)"
    }

    public func binding(projectID: ProjectID, fileID: ProjectFileID) -> ContextPage {
        ContextPage(projectRoot: projectRoot, projectID: projectID, fileID: fileID, path: path, startLine: startLine, endLine: endLine,
        ↪ content: content, hash: hash, version: version, sourceType: sourceType, metadata: metadata)
    }

    public static func projectIdentifier(for root: URL) -> String {
        root.standardizedFileURL.resolvingSymlinksInPath().path
    }

    public static func fingerprint<C: Collection>(_ bytes: C) -> String where C.Element == UInt8 {
        var value: UInt64 = 0xcbf29ce484222325
        for byte in bytes {
            value ^= UInt64(byte)
            value &*= 0x100000001b3
        }
        return String(format: "%016llx", value)
    }
}

public struct ProjectScan: Sendable, Equatable {
    public let projectRoot: String
    public let files: [ScannedProjectFile]

    public var pages: [ContextPage] { files.flatMap(\.pages) }

    public init(projectRoot: String, files: [ScannedProjectFile]) {
        self.projectRoot = projectRoot
        self.files = files
    }
}

public struct ScannedProjectFile: Sendable, Equatable {
    public let fileID: ProjectFileID?
    public let path: String
    public let version: String
    public let pages: [ContextPage]
    public let fileSize: Int
    public let modificationDate: Date?

    public init(fileID: ProjectFileID? = nil, path: String, version: String, pages: [ContextPage], fileSize: Int = -1, modificationDate:
    ↪ Date? = nil) {
        self.fileID = fileID
        self.path = path
        self.version = version
        self.pages = pages
        self.fileSize = fileSize
        self.modificationDate = modificationDate
    }

    public func binding(projectID: ProjectID, fileID: ProjectFileID) -> ScannedProjectFile {
        ScannedProjectFile(fileID: fileID, path: path, version: version, pages: pages.map { $0.binding(projectID: projectID, fileID: fileID)
        ↪ }, fileSize: fileSize, modificationDate: modificationDate)
    }

    public static func == (lhs: Self, rhs: Self) -> Bool {
        lhs.path == rhs.path && lhs.version == rhs.version && lhs.pages == rhs.pages
    }
}

```

```

public struct ProjectPageStoreUpdate: Sendable, Equatable {
    public let initialIndexedPaths: [String]
    public let rebuiltPaths: [String]
    public let removedPaths: [String]
    public let invalidatedPageIDs: [String]
    public let filesChecked: Int
    public let filesRebuilt: Int
    public let initialIndexedPages: Int
    public let scanMilliseconds: Int

    public init(
        initialIndexedPaths: [String] = [],
        rebuiltPaths: [String] = [],
        removedPaths: [String] = [],
        invalidatedPageIDs: [String] = [],
        filesChecked: Int = 0,
        filesRebuilt: Int = 0,
        initialIndexedPages: Int = 0,
        scanMilliseconds: Int = 0
    ) {
        self.initialIndexedPaths = initialIndexedPaths
        self.rebuiltPaths = rebuiltPaths
        self.removedPaths = removedPaths
        self.invalidatedPageIDs = invalidatedPageIDs
        self.filesChecked = filesChecked
        self.filesRebuilt = filesRebuilt
        self.initialIndexedPages = initialIndexedPages
        self.scanMilliseconds = scanMilliseconds
    }
}

public struct L2PromotionResult: Sendable, Equatable {
    public let admitted: [ContextPage]
    public let evicted: [ContextPage]

    public init(admitted: [ContextPage], evicted: [ContextPage]) {
        self.admitted = admitted
        self.evicted = evicted
    }
}

public struct L2WorkingSetMetrics: Sendable, Equatable {
    public let pageCount: Int
    public let characterCount: Int

    public init(pageCount: Int, characterCount: Int) {
        self.pageCount = pageCount
        self.characterCount = characterCount
    }
}

public struct ContextPagerMetrics: Sendable, Equatable {
    public let hits: Int
    public let misses: Int
    public let pageFaults: Int

    public init(hits: Int = 0, misses: Int = 0, pageFaults: Int = 0) {
        self.hits = hits
        self.misses = misses
        self.pageFaults = pageFaults
    }
}

public struct ContextPagingTurnMetrics: Sendable, Equatable {

```

```

public let lookups: Int
public let hits: Int
public let misses: Int
public let pageFaults: Int
public let promotions: Int
public let evictions: Int
public let candidatePages: Int
public let candidateCharacters: Int
public let selectedPages: Int
public let selectedCharacters: Int
public let retrievalMilliseconds: Double
public let materializationMilliseconds: Double
public let symbolHints: Int
public let symbolExactMatches: Int
public let symbolQualifiedExactMatches: Int
public let symbolFallbackExactMatches: Int
public let symbolPrefixMatches: Int
public let symbolCandidatePages: Int
public let symbolHintExtractionMilliseconds: Double
public let symbolExactLookupMilliseconds: Double
public let symbolPrefixLookupMilliseconds: Double
public let symbolCandidateMergeMilliseconds: Double
public let symbolRankingMilliseconds: Double
public let symbolTotalMilliseconds: Double
public let lexicalCandidatePages: Int
public let currentSourceCandidates: Int
public let documentationCandidates: Int
public let referenceCandidates: Int
public let relationHints: Int
public let directReferenceHits: Int
public let dependencyHits: Int
public let relatedPages: Int
public let referenceResolutionMilliseconds: Double
public let referenceExpansionMilliseconds: Double

public static let zero = ContextPagingTurnMetrics(lookups: 0, hits: 0, misses: 0, pageFaults: 0, promotions: 0, evictions: 0,
    ↪ candidatePages: 0, candidateCharacters: 0, selectedPages: 0, selectedCharacters: 0, retrievalMilliseconds: 0,
    ↪ materializationMilliseconds: 0, symbolHints: 0, symbolExactMatches: 0, symbolQualifiedExactMatches: 0, symbolFallbackExactMatches:
    ↪ 0, symbolPrefixMatches: 0, symbolCandidatePages: 0, symbolHintExtractionMilliseconds: 0, symbolExactLookupMilliseconds: 0,
    ↪ symbolPrefixLookupMilliseconds: 0, symbolCandidateMergeMilliseconds: 0, symbolRankingMilliseconds: 0, symbolTotalMilliseconds: 0,
    ↪ lexicalCandidatePages: 0, currentSourceCandidates: 0, documentationCandidates: 0, referenceCandidates: 0, relationHints: 0,
    ↪ directReferenceHits: 0, dependencyHits: 0, relatedPages: 0, referenceResolutionMilliseconds: 0, referenceExpansionMilliseconds: 0)
}

public struct SymbolSearchMetrics: Sendable, Equatable {
    public let hints: Int
    public let exactMatches: Int
    public let qualifiedExactMatches: Int
    public let fallbackExactMatches: Int
    public let prefixMatches: Int
    public let candidatePages: Int
    public let exactLookupMilliseconds: Double
    public let prefixLookupMilliseconds: Double
    public let candidateMergeMilliseconds: Double
    public let rankingMilliseconds: Double
    public let totalMilliseconds: Double
    public let lexicalCandidatePages: Int
    public let currentSourceCandidates: Int
    public let documentationCandidates: Int
    public let referenceCandidates: Int
    public let relationHints: Int
    public let directReferenceHits: Int
    public let dependencyHits: Int
    public let relatedPages: Int
    public let resolutionMilliseconds: Double

```

```

    public let expansionMilliseconds: Double

    public static let zero = SymbolSearchMetrics(hints: 0, exactMatches: 0, qualifiedExactMatches: 0, fallbackExactMatches: 0, prefixMatches
    ↪ : 0, candidatePages: 0, exactLookupMilliseconds: 0, prefixLookupMilliseconds: 0, candidateMergeMilliseconds: 0, rankingMilliseconds
    ↪ : 0, totalMilliseconds: 0, lexicalCandidatePages: 0, currentSourceCandidates: 0, documentationCandidates: 0, referenceCandidates:
    ↪ 0, relationHints: 0, directReferenceHits: 0, dependencyHits: 0, relatedPages: 0, resolutionMilliseconds: 0, expansionMilliseconds:
    ↪ 0)
}

public struct ProjectPageSearchResult: Sendable, Equatable {
    public let pages: [ContextPage]
    public let symbolMetrics: SymbolSearchMetrics

    public init(pages: [ContextPage], symbolMetrics: SymbolSearchMetrics = .zero) {
        self.pages = pages
        self.symbolMetrics = symbolMetrics
    }
}

public struct ContextPagerResult: Sendable, Equatable {
    public let pages: [ContextPage]
    public let metrics: ContextPagerMetrics
    public let candidatePageCount: Int
    public let candidateCharacterCount: Int
    public let selectedPageCount: Int
    public let selectedCharacterCount: Int
    public let turnMetrics: ContextPagingTurnMetrics

    public var characterCount: Int { pages.reduce(0) { $0 + $1.characterCount } }

    public init(pages: [ContextPage], metrics: ContextPagerMetrics, candidatePageCount: Int = 0, candidateCharacterCount: Int = 0,
    ↪ selectedPageCount: Int? = nil, selectedCharacterCount: Int? = nil, turnMetrics: ContextPagingTurnMetrics = .zero) {
        self.pages = pages
        self.metrics = metrics
        self.candidatePageCount = candidatePageCount
        self.candidateCharacterCount = candidateCharacterCount
        self.selectedPageCount = selectedPageCount ?? pages.count
        self.selectedCharacterCount = selectedCharacterCount ?? pages.reduce(0) { $0 + $1.characterCount }
        self.turnMetrics = turnMetrics
    }
}

public struct ContextPagingDebugMetrics: Sendable, Equatable {
    public let queryCharacters: Int
    public let queryTerms: Int
    public let candidatePages: Int
    public let candidateCharacters: Int
    public let selectedPages: Int
    public let selectedCharacters: Int
    public let injectedPages: Int
    public let injectedCharacters: Int
    public let filesChecked: Int
    public let filesRebuilt: Int
    public let scanMilliseconds: Int
    public let initialIndexedFiles: Int
    public let l2Lookups: Int
    public let l2Hits: Int
    public let l2Misses: Int
    public let l2Pages: Int
    public let l2Characters: Int
    public let l3Pages: Int
    public let l3Queries: Int
    public let l3Candidates: Int
    public let l3Materializations: Int
    public let staleRebuilds: Int

```

```

public let pageFaults: Int
public let promotions: Int
public let evictions: Int
public let retrievalMilliseconds: Double
public let materializationMilliseconds: Double
public let symbolCount: Int
public let symbolIndexedFiles: Int
public let symbolHints: Int
public let symbolExactMatches: Int
public let symbolQualifiedExactMatches: Int
public let symbolFallbackExactMatches: Int
public let symbolPrefixMatches: Int
public let symbolCandidatePages: Int
public let symbolHintExtractionMilliseconds: Double
public let symbolExactLookupMilliseconds: Double
public let symbolPrefixLookupMilliseconds: Double
public let symbolCandidateMergeMilliseconds: Double
public let symbolRankingMilliseconds: Double
public let symbolTotalMilliseconds: Double
public let lexicalCandidatePages: Int
public let currentSourceCandidates: Int
public let documentationCandidates: Int
public let referenceCandidates: Int
public let referenceCount: Int
public let resolvedReferenceCount: Int
public let ambiguousReferenceCount: Int
public let unresolvedReferenceCount: Int
public let dependencyCount: Int
public let referenceIndexedFiles: Int
public let relationHints: Int
public let directReferenceHits: Int
public let dependencyHits: Int
public let relatedPages: Int
public let referenceResolutionMilliseconds: Double
public let referenceExpansionMilliseconds: Double
}

```

// File: Sources/LingXiCore/Modules/Context/ProjectPageStore.swift

```
import Foundation
```

/// L3 项目页存储。重建以文件版本为粒度，未变化文件的页面实例会被保留。

```

public actor ProjectPageStore {
    private var filesByProject: [String: [String: ScannedProjectFile]] = [:]
    private let symbolIndex: ProjectSymbolIndex
    private let symbolExtractor: SwiftSymbolExtractor
    private let referenceIndex: ProjectReferenceIndex
    private let referenceExtractor: SwiftReferenceExtractor
    private let rankingPolicy: ContextPageRankingPolicy
    private let persistence: SQLitePersistenceStore?

    public init(
        symbolIndex: ProjectSymbolIndex = ProjectSymbolIndex(),
        symbolExtractor: SwiftSymbolExtractor = SwiftSymbolExtractor(),
        referenceIndex: ProjectReferenceIndex = ProjectReferenceIndex(),
        referenceExtractor: SwiftReferenceExtractor = SwiftReferenceExtractor(),
        rankingPolicy: ContextPageRankingPolicy = ContextPageRankingPolicy(),
        persistence: SQLitePersistenceStore? = nil
    ) {
        self.symbolIndex = symbolIndex
        self.symbolExtractor = symbolExtractor
        self.referenceIndex = referenceIndex
        self.referenceExtractor = referenceExtractor
        self.rankingPolicy = rankingPolicy
        self.persistence = persistence
    }
}

```

```

}

@discardableResult
public func rebuildStaleFiles(using scanner: ProjectScanner) async throws -> ProjectPageStoreUpdate {
    let scanStartedAt = Date()
    _ = try await persistence?.invalidateCachesIfFormatMismatch()
    let isInitialIndex = filesByProject[scanner.projectRoot] == nil
    let oldFiles = filesByProject[scanner.projectRoot, default: [:]]
    let rawScan = try scanner.scanManifest(reusing: oldFiles)
    let scan = try await bindStableFileIdentities(rawScan)
    let newFiles = Dictionary(uniqueKeysWithValues: scan.files.map { ($0.path, $0) })
    let initialIndexedPaths = isInitialIndex ? newFiles.keys.sorted() : []
    var updatedFiles = isInitialIndex ? [:] : oldFiles
    var rebuiltPaths: [String] = []
    var removedPaths: [String] = []
    var invalidatedPageIDs: [String] = []

    for (path, file) in newFiles where isInitialIndex || oldFiles[path]?.version != file.version {
        if !isInitialIndex {
            rebuiltPaths.append(path)
            invalidatedPageIDs += oldFiles[path]?.pages.map(\.id) ?? []
        }
        updatedFiles[path] = file
        await updateSymbols(projectRoot: scan.projectRoot, file: file)
        await updateReferences(projectRoot: scan.projectRoot, file: file)
    }
    for (path, file) in oldFiles where !isInitialIndex && newFiles[path] == nil {
        removedPaths.append(path)
        invalidatedPageIDs += file.pages.map(\.id)
        updatedFiles.removeValue(forKey: path)
        await symbolIndex.remove(projectRoot: scan.projectRoot, path: path)
        await referenceIndex.removeReferences(projectRoot: scan.projectRoot, forPath: path)
    }
    filesByProject[scan.projectRoot] = updatedFiles
    await resolveReferences(projectRoot: scan.projectRoot)
    if let persistence {
        let symbols = await symbolIndex.allSymbols(projectRoot: scan.projectRoot)
        let references = await referenceIndex.references(projectRoot: scan.projectRoot)
        try await persistence.replaceProjectCache(pages: updatedFiles.values.flatMap(\.pages), symbols: symbols, references: references,
        ↪ dependencies: references.map(DependencyEdge.init(reference:)))
    }
    return ProjectPageStoreUpdate(
        initialIndexedPaths: initialIndexedPaths,
        rebuiltPaths: rebuiltPaths.sorted(),
        removedPaths: removedPaths.sorted(),
        invalidatedPageIDs: Array(Set(invalidatedPageIDs)).sorted(),
        filesChecked: scan.files.count,
        filesRebuilt: isInitialIndex ? 0 : rebuiltPaths.count,
        initialIndexedPages: isInitialIndex ? scan.pages.count : 0,
        scanMilliseconds: Int(Date().timeIntervalSince(scanStartedAt) * 1_000)
    )
}

public func page(projectRoot: URL, id: String) -> ContextPage? {
    pages(projectRoot: ContextPage.projectIdentifier(for: projectRoot)).first { $0.id == id }
}

public func pages(projectRoot: URL) -> [ContextPage] {
    pages(projectRoot: ContextPage.projectIdentifier(for: projectRoot))
}

public func statistics(projectRoot: URL) async -> (files: Int, pages: Int, symbols: Int, symbolFiles: Int, references:
    ↪ ReferenceIndexStats) {
    let files = filesByProject[ContextPage.projectIdentifier(for: projectRoot), default: [:]]
    let symbols = await symbolIndex.stats(projectRoot: ContextPage.projectIdentifier(for: projectRoot))

```

```

        return (files.count, files.values.reduce(0) { $0 + $1.pages.count }, symbols.symbolCount, symbols.fileCount, await referenceIndex.
↳ stats(projectRoot: ContextPage.projectIdentifier(for: projectRoot)))
    }

public func resources(projectRoot: URL) async -> (pages: [ContextPage], symbols: [Symbol], references: [ProjectReference]) {
    let id = ContextPage.projectIdentifier(for: projectRoot)
    return (pages(projectRoot: id), await symbolIndex.allSymbols(projectRoot: id), await referenceIndex.references(projectRoot: id))
}

public func symbolLookup(projectRoot: URL, query: String, mode: String) async -> [Symbol] {
    let id = ContextPage.projectIdentifier(for: projectRoot)
    switch mode {
    case "exact": return await symbolIndex.exact(projectRoot: id, name: query)
    case "qualified": return await symbolIndex.exactQualifiedName(projectRoot: id, name: query)
    default: return await symbolIndex.prefix(projectRoot: id, prefix: query)
    }
}

public func references(projectRoot: URL, symbolID: SymbolID) async -> [ProjectReference] {
    await referenceIndex.referencesTo(projectRoot: ContextPage.projectIdentifier(for: projectRoot), symbol: symbolID)
}

public func dependencies(projectRoot: URL, path: String, incoming: Bool) async -> [DependencyEdge] {
    let id = ContextPage.projectIdentifier(for: projectRoot)
    return incoming ? await referenceIndex.dependenciesTo(projectRoot: id, path: path) : await referenceIndex.dependenciesFrom(
↳ projectRoot: id, path: path)
}

public func search(projectRoot: URL, query: String, limit: Int = 20) async -> [ContextPage] {
    await searchResult(projectRoot: projectRoot, query: ContextQuery(currentTask: query), limit: limit).pages
}

public func search(projectRoot: URL, query: ContextQuery, limit: Int = 20) async -> [ContextPage] {
    await searchResult(projectRoot: projectRoot, query: query, limit: limit).pages
}

public func searchResult(projectRoot: URL, query: ContextQuery, limit: Int = 20) async -> ProjectPageSearchResult {
    let projectID = ContextPage.projectIdentifier(for: projectRoot)
    let clock = ContinuousClock()
    let totalStarted = clock.now
    var qualifiedMatches: [Symbol] = []
    var nameMatches: [Symbol] = []
    var prefixMatches: [Symbol] = []
    var relatedPageIDs = Set<String>()
    var referenceLookup = ReferenceLookupResult()
    var qualifiedExactIDs = Set<SymbolID>()
    var fallbackExactIDs = Set<SymbolID>()
    let exactStarted = clock.now
    for group in query.symbolHintGroups {
        for (offset, hint) in group.enumerated() {
            let qualified = await symbolIndex.exactQualifiedName(projectRoot: projectID, name: hint)
            let names = await symbolIndex.exact(projectRoot: projectID, name: hint)
            qualifiedMatches += qualified
            nameMatches += names
            if offset == 0, hint.contains(".") {
                qualifiedExactIDs.formUnion(qualified.map(\.id))
            } else {
                fallbackExactIDs.formUnion(qualified.map(\.id))
                fallbackExactIDs.formUnion(names.map(\.id))
            }
        }
    }
    let exactLookupMilliseconds = milliseconds(exactStarted.duration(to: clock.now))
    let prefixStarted = clock.now
    for hint in query.symbolHints {

```

```

    prefixMatches += await symbolIndex.prefix(projectRoot: projectID, prefix: hint)
  }
  let prefixLookupMilliseconds = milliseconds(prefixStarted.duration(to: clock.now))
  let mergeStarted = clock.now
  var symbolScoresByPageID: [String: Int] = [:]
  var exactIDs = Set<SymbolID>()
  var prefixIDs = Set<SymbolID>()
  let exactSymbols = Dictionary(
    (qualifiedMatches + nameMatches).map { ($0.id, $0) },
    uniquingKeysWith: { current, _ in current }
  )
  for symbol in exactSymbols.values {
    let score = qualifiedExactIDs.contains(symbol.id) ? 1_200 : 900
    symbolScoresByPageID[symbol.pageID] = max(symbolScoresByPageID[symbol.pageID, default: 0], score)
    exactIDs.insert(symbol.id)
  }
  for symbol in prefixMatches {
    symbolScoresByPageID[symbol.pageID] = max(symbolScoresByPageID[symbol.pageID, default: 0], 500)
    if !exactIDs.contains(symbol.id) { prefixIDs.insert(symbol.id) }
  }
  let resolutionStarted = clock.now
  let relatedSymbols = Set(exactSymbols.values.map(\.id) + prefixMatches.map(\.id))
  if !relatedSymbols.isEmpty || !query.relationHints.isEmpty {
    referenceLookup = await referenceIndex.lookupRelatedPages(
      projectRoot: projectID,
      symbolIDs: relatedSymbols,
      targetNames: Set(query.symbolHints),
      maximumPages: 8,
      maximumDepth: 1
    )
    relatedPageIDs = Set(referenceLookup.pageIDs)
  }
  let referenceResolutionMilliseconds = milliseconds(resolutionStarted.duration(to: clock.now))
  var referenceScoresByPageID: [String: Int] = [:]
  for pageID in relatedPageIDs { referenceScoresByPageID[pageID] = 800 }
  let expansionMilliseconds = milliseconds(resolutionStarted.duration(to: clock.now)) - referenceResolutionMilliseconds
  let candidateMergeMilliseconds = milliseconds(mergeStarted.duration(to: clock.now))
  let rankingStarted = clock.now
  let candidates = rankingPolicy.rank(
    pages: pages(projectRoot: projectID),
    query: query,
    symbolScoresByPageID: symbolScoresByPageID,
    referenceScoresByPageID: referenceScoresByPageID
  )
  let rankingMilliseconds = milliseconds(rankingStarted.duration(to: clock.now))
  return ProjectPageSearchResult(
    pages: candidates.prefix(max(0, limit)).map { $0.page },
    symbolMetrics: SymbolSearchMetrics(
      hints: query.symbolHints.count,
      exactMatches: exactIDs.count,
      qualifiedExactMatches: qualifiedExactIDs.count,
      fallbackExactMatches: fallbackExactIDs.count,
      prefixMatches: prefixIDs.count,
      candidatePages: symbolScoresByPageID.count,
      exactLookupMilliseconds: exactLookupMilliseconds,
      prefixLookupMilliseconds: prefixLookupMilliseconds,
      candidateMergeMilliseconds: candidateMergeMilliseconds,
      rankingMilliseconds: rankingMilliseconds,
      totalMilliseconds: milliseconds(totalStarted.duration(to: clock.now)),
      lexicalCandidatePages: candidates.filter { $0.textScore > 0 }.count,
      currentSourceCandidates: candidates.filter { $0.page.sourceType == .sourceFile }.count,
      documentationCandidates: candidates.filter { $0.page.sourceType == .documentation }.count,
      referenceCandidates: candidates.filter { $0.page.sourceType == .referenceDocumentation || $0.page.sourceType == .
↪ researchArchive }.count,
      relationHints: query.relationHints.count,

```



```

        directReferenceHits: referenceLookup.directReferenceHits,
        dependencyHits: referenceLookup.dependencyHits,
        relatedPages: relatedPageIDs.count,
        resolutionMilliseconds: referenceResolutionMilliseconds,
        expansionMilliseconds: expansionMilliseconds
    )
}

private func pages(projectRoot: String) -> [ContextPage] {
    filesByProject[projectRoot, default: [:]].values.flatMap(\.pages)
}

private func bindStableFileIdentities(_ scan: ProjectScan) async throws -> ProjectScan {
    guard let persistence else { return scan }
    let main = try await persistence.mainRootBinding()
    let existing = try await persistence.files().filter { $0.rootBindingID == main.id && $0.state == "active" }
    let byPath = Dictionary(uniqueKeysWithValues: existing.map { ($0.relativePath.rawValue, $0) })
    let incomingPaths = Set(scan.files.map(\.path))
    let missing = existing.filter { !incomingPaths.contains($0.relativePath.rawValue) }
    let newFiles = scan.files.filter { byPath[$0.path] == nil }
    let uniqueNewByHash = Dictionary(grouping: newFiles, by: \.version).filter { $0.value.count == 1 }
    let uniqueMissingByHash = Dictionary(grouping: missing, by: \.contentHash).filter { $0.value.count == 1 }
    var moved: [String: ProjectFileBinding] = [:]
    for (hash, candidates) in uniqueNewByHash {
        guard let old = uniqueMissingByHash[hash]?.first, let file = candidates.first else { continue }
        try await persistence.relocateFile(old.id, rootBindingID: main.id, relativePath: try ProjectRelativePath(file.path))
        moved[file.path] = try await persistence.file(old.id)
    }
    for old in missing where !moved.values.contains(where: { $0.id == old.id }) {
        try await persistence.markFileMissing(old.id)
    }
    var files: [ScannedProjectFile] = []
    for file in scan.files {
        let binding: ProjectFileBinding
        if let movedBinding = moved[file.path] { binding = movedBinding }
        else if let existingBinding = byPath[file.path] {
            binding = try await persistence.upsertFile(
                rootBindingID: main.id,
                relativePath: try ProjectRelativePath(file.path),
                contentHash: file.version,
                version: file.version
            )
            assert(binding.id == existingBinding.id)
        }
        else { binding = try await persistence.upsertFile(rootBindingID: main.id, relativePath: try ProjectRelativePath(file.path),
↪ contentHash: file.version, version: file.version) }
        files.append(file.binding(projectID: persistence.projectID, fileID: binding.id))
    }
    return ProjectScan(projectRoot: scan.projectRoot, files: files)
}

private func updateSymbols(projectRoot: String, file: ScannedProjectFile) async {
    guard URL(file.urlWithPath: file.path).pathExtension.lowercased() == "swift" else {
        await symbolIndex.remove(projectRoot: projectRoot, path: file.path)
        return
    }
    await symbolIndex.replace(
        projectRoot: projectRoot,
        path: file.path,
        symbols: symbolExtractor.extract(projectRoot: projectRoot, path: file.path, pages: file.pages)
    )
}

private func updateReferences(projectRoot: String, file: ScannedProjectFile) async {

```

```

guard URL(fileURLWithPath: file.path).pathExtension.lowercased() == "swift" else {
    await referenceIndex.removeReferences(projectRoot: projectRoot, forPath: file.path)
    return
}
let symbols = await symbolIndex.symbols(projectRoot: projectRoot, path: file.path)
await referenceIndex.replaceReferences(
    projectRoot: projectRoot,
    forPath: file.path,
    references: referenceExtractor.extract(projectRoot: projectRoot, path: file.path, pages: file.pages, symbols: symbols)
)
}

private func resolveReferences(projectRoot: String) async {
    let unresolved = await referenceIndex.references(projectRoot: projectRoot)
    let symbols = await symbolIndex.allSymbols(projectRoot: projectRoot)
    let byQualifiedNames = Dictionary(grouping: symbols, by: \.qualifiedName)
    let byName = Dictionary(grouping: symbols, by: \.name)
    var resolved: [ProjectReference] = []
    for reference in unresolved {
        guard reference.kind != .import else { resolved.append(reference); continue }
        var candidates = byQualifiedNames[reference.targetName, default: []]
        var quality: ReferenceResolutionQuality = .exactResolved
        if candidates.isEmpty {
            candidates = byName[reference.targetName, default: []]
            quality = .symbolNameResolved
        }
        if candidates.count > 1 {
            let sameFile = candidates.filter { $0.path == reference.sourcePath }
            if sameFile.count == 1 { candidates = sameFile }
            else { resolved.append(reference.resolving(to: candidates, quality: .ambiguous)); continue }
        }
        if candidates.isEmpty { resolved.append(reference.resolving(to: [], quality: reference.receiverHint == nil ? .unresolved : .
↪ receiverHint)) }
        else { resolved.append(reference.resolving(to: candidates, quality: quality)) }
    }
    await referenceIndex.replaceResolved(projectRoot: projectRoot, references: resolved)
}

private func milliseconds(_ duration: Duration) -> Double {
    let components = duration.components
    return Double(components.seconds) * 1_000 + Double(components.attoseconds) / 1_000_000_000_000_000
}
}

```

// File: Sources/LingXiCore/Modules/Context/ProjectScanner.swift

```

import Foundation

public enum ProjectScannerError: Error, Sendable, Equatable {
    case rootNotFound(String)
}

/// 仅扫描项目根目录以内的 UTF-8 常规文本文件。
public struct ProjectScanner: Sendable {
    public static let maximumFileBytes = 1_024 * 1_024
    public static let referenceDocumentationDirectories: Set<String> = ["references", "reference-docs"]
    public static let researchArchiveDirectories: Set<String> = ["opencode-extraction", "openchamber-extraction"]
    public let root: URL
    public let sensitivePathPolicy: SensitivePathPolicy
    public let minimumPageBytes: Int
    public let maximumPageBytes: Int

    public init(root: URL, sensitivePathPolicy: SensitivePathPolicy? = nil, minimumPageBytes: Int = 8 * 1024, maximumPageBytes: Int = 16 *
↪ 1024) {
        let resolvedRoot = root.standardizedFileURL.resolvingSymlinksInPath()
    }
}

```

```

self.root = resolvedRoot
self.sensitivePathPolicy = sensitivePathPolicy ?? SensitivePathPolicy(root: resolvedRoot)
self.minimumPageBytes = max(1, minimumPageBytes)
self.maximumPageBytes = max(max(1, minimumPageBytes), maximumPageBytes)
}

public var projectRoot: String { ContextPage.projectIdentifier(for: root) }

public func scan() throws -> [ContextPage] {
    try scanManifest().pages
}

public func scanManifest() throws -> ProjectScan {
    try scanManifest(reusing: [])
}

func scanManifest(reusing existingFiles: [String: ScannedProjectFile]) throws -> ProjectScan {
    let manager = FileManager.default
    guard manager.fileExists(atPath: root.path) else {
        throw ProjectScannerError.rootNotFound(root.path)
    }
    let keys: Set<URLResourceKey> = [.isDirectoryKey, .isRegularFileKey, .fileSizeKey, .contentModificationDateKey]
    guard let enumerator = manager.enumerator(
        at: root,
        includingPropertiesForKeys: Array(keys),
        options: [],
        errorHandler: { _, _ in true }
    ) else {
        return ProjectScan(projectRoot: projectRoot, files: [])
    }

    var files: [ScannedProjectFile] = []
    while let url = enumerator.nextObject() as? URL {
        let resolvedURL = url.standardizedFileURL.resolvingSymlinksInPath()
        guard resolvedURL.path.hasPrefix(root.path + "/"),
            let values = try? url.resourceValues(forKeys: keys),
            !isSymbolicLink(url, manager: manager) else {
            if (try? url.resourceValues(forKeys: keys))?.isDirectory == true { enumerator.skipDescendants() }
            continue
        }
        if values.isDirectory == true {
            if isExcludedPath(url) { enumerator.skipDescendants() }
            continue
        }
        let path = relativePath(for: url)
        let fileSize = values.fileSize ?? 0
        let modificationDate = values.contentModificationDate
        guard values.isRegularFile == true, !isExcludedPath(url), fileSize <= Self.maximumFileBytes else {
            continue
        }
        if let modificationDate,
            let existing = existingFiles[path],
            existing.fileSize == fileSize,
            existing.modificationDate == modificationDate {
            files.append(existing)
            continue
        }
        guard let data = try? Data(contentsOf: url), !isBinary(data), let text = String(data: data, encoding: .utf8) else {
            continue
        }
        let version = ContextPage.fingerprint(data)
        files.append(ScannedProjectFile(
            path: path,
            version: version,
            pages: makePages(path: path, text: text, version: version),

```

```

        fileSize: fileSize,
        modificationDate: modificationDate
    ))
}
return ProjectScan(projectRoot: projectRoot, files: files.sorted { $0.path < $1.path })
}

private func isExcludedPath(_ url: URL) -> Bool {
    if sensitivePathPolicy.isSensitive(url) { return true }
    let components = url.pathComponents.map { $0.lowercased() }
    return components.contains(where: { ["git", ".build", ".swiftpm", "deriveddata", "node_modules"].contains($0) })
}

private func isBinary(_ data: Data) -> Bool {
    data.prefix(8 * 1024).contains(0)
}

private func isSymbolicLink(_ url: URL, manager: FileManager) -> Bool {
    (try? manager.destinationOfSymbolicLink(atPath: url.path)) != nil
}

private func relativePath(for url: URL) -> String {
    String(url.standardizedFileURL.resolvingSymlinksInPath().path.dropFirst(root.path.count + 1))
}

private func makePages(path: String, text: String, version: String) -> [ContextPage] {
    guard !text.isEmpty else { return [] }
    var lines = text.components(separatedBy: "\n")
    let hasTrailingNewline = text.hasSuffix("\n")
    if hasTrailingNewline { lines.removeLast() }
    var pages: [ContextPage] = []
    var content = ""
    var byteCount = 0
    var startLine = 1

    for (index, segment) in lines.enumerated() {
        let line = segment + (index < lines.count - 1 || hasTrailingNewline ? "\n" : "")
        let lineBytes = line.utf8.count
        // Prefer 8-12 KB pages; retain complete lines and only exceed a limit for one long line.
        if !content.isEmpty && (
            byteCount + lineBytes > maximumPageBytes ||
            (byteCount >= minimumPageBytes && byteCount + lineBytes > (minimumPageBytes + maximumPageBytes) / 2)
        ) {
            pages.append(page(path: path, startLine: startLine, endLine: index, content: content, version: version))
            content = ""
            byteCount = 0
            startLine = index + 1
        }
        content += line
        byteCount += lineBytes
    }
    if !content.isEmpty {
        pages.append(page(path: path, startLine: startLine, endLine: lines.count, content: content, version: version))
    }
    return pages
}

private func page(path: String, startLine: Int, endLine: Int, content: String, version: String) -> ContextPage {
    let extensionName = URL(fileURLWithPath: path).pathExtension.lowercased()
    let sourceType: ContextPageSourceType
    let components = path.split(separator: "/").map { $0.lowercased() }
    if components.contains(where: { Self.researchArchiveDirectories.contains($0) }) { sourceType = .researchArchive }
    else if components.contains(where: { Self.referenceDocumentationDirectories.contains($0) }) { sourceType = .referenceDocumentation }
    else if path == "README.md" { sourceType = .projectMetadata }
    else if path.hasPrefix("Tests/") { sourceType = .test }
}

```

```

        else if ["md", "txt"].contains(extensionName) { sourceType = .documentation }
        else if ["json", "yaml", "yml", "toml", "xcconfig"].contains(extensionName) || URL(fileURLWithPath: path).lastPathComponent == "
↳ Package.swift" { sourceType = .configuration }
        else { sourceType = .sourceFile }
        let heading = content.split(separator: "\n").first(where: { $0.hasPrefix("#") }).map(String.init)
        return ContextPage(projectRoot: projectRoot, path: path, startLine: startLine, endLine: endLine, content: content, version: version,
↳ sourceType: sourceType, metadata: ContextPageMetadata(path: path, language: extensionName.isEmpty ? nil : extensionName, heading:
↳ heading))
    }
}

```

// File: Sources/LingXiCore/Modules/Context/ContextProjection.swift

```

import Foundation
import LingXiProtocol

/// P-Core 上下文投影器 (Context Projection)
/// 负责在构建发送给 Provider 的请求时，将超大或已归档的 ToolResult 转换为确定性、高复用率的稳定 Placeholder。
///
/// 核心原则：
/// 1. Canonical Truth 永不可变：SessionStore 始终保存完整真实的原始 ToolResult，P-Core 仅为向 Provider 暴露的短暂投影。
/// 2. FULL_SENDS 语义保证：每个 ToolResult 保证至少被完整发送 fullSendCount 次（默认 2 次），从第 3 次起转为 1KB 稳定首尾 Placeholder。
/// 3. Fail-Open 铁律：任何投影、读取或持久化异常均捕获并静默回退至透传原始 ToolResult，绝不阻断主流程。
public struct ContextProjection: Sendable {
    public let configuration: ContextObjectFabricConfiguration

    public init(configuration: ContextObjectFabricConfiguration = ContextObjectFabricConfiguration()) {
        self.configuration = configuration
    }

    /// 将 ContextEntry 数组投影为面向 Provider 的上下文条目列表。
    public func project(
        entries: [ContextEntry],
        session: Session,
        ecoreStore: ECoreObjectStore
    ) async -> [ContextEntry] {
        guard configuration.observationProjectionEnabled else {
            return entries
        }

        // 收集 Session 中全部 Assistant 消息的时间戳或索引，以便确定因果先后
        let messages = session.messages
        var assistantCountAfterMessageID: [MessageID: Int] = [:]

        for (idx, msg) in messages.enumerated() {
            if msg.role == .tool {
                // 统计该 Tool 结果消息之后，已经产生的 Assistant 响应消息数
                let laterAssistants = messages[(idx + 1)...].filter { $0.role == .assistant }.count
                assistantCountAfterMessageID[msg.id] = laterAssistants
            }
        }

        var projectedEntries: [ContextEntry] = []
        projectedEntries.reserveCapacity(entries.count)

        for entry in entries {
            guard entry.source == .toolResult,
                  case let .toolResult(result) = entry.part,
                  let messageID = entry.messageID else {
                projectedEntries.append(entry)
                continue
            }

            let assistantCount = assistantCountAfterMessageID[messageID] ?? 0

```

```

// 规则：前 fullSendCount 次请求，Provider 必须见到完整全文！
guard assistantCount >= configuration.fullSendCount else {
    projectedEntries.append(entry)
    continue
}

// 检查大小是否达到独立对象化阈值（10KB）
let byteCount = result.content.utf8.count
guard byteCount >= configuration.objectizationThreshold else {
    // 小于阈值的结果保持完整内联，避免无谓的对象碎片
    projectedEntries.append(entry)
    continue
}

// 满足条件：构建 1KB 稳定首尾 Placeholder
let toolName = result.toolName ?? "tool"
let objectID = ContextObjectID.generate(toolName: toolName, callID: result.callID, content: result.content)

// 确保该对象已在 E-Core 中安全归档（Fail-Open）
await ecoreStore.store(
    sessionID: session.id,
    toolCallID: result.callID,
    toolName: toolName,
    content: result.content,
    force: true
)

let placeholder = buildPlaceholder(
    objectID: objectID,
    toolName: toolName,
    content: result.content,
    excerptBytes: configuration.placeholderExcerpt
)

// 构造投影后的替代 ToolResult，外部 Canonical SessionStore 不受影响
let projectedResult = ToolResult(
    callID: result.callID,
    success: result.success,
    content: placeholder,
    toolName: result.toolName,
    metadata: result.metadata.merging(["context_object_id": objectID.rawValue, "projected": "true"]) { current, _ in current }
)

let projectedEntry = ContextEntry(
    messageID: entry.messageID,
    role: entry.role,
    source: entry.source,
    part: .toolResult(projectedResult)
)
projectedEntries.append(projectedEntry)
}

return projectedEntries
}

/// 构建符合规范的稳定首尾 Placeholder（总计约 1KB 摘要）
private func buildPlaceholder(
    objectID: ContextObjectID,
    toolName: String,
    content: String,
    excerptBytes: Int
) -> String {
    let totalBytes = content.utf8.count
    var lineCount = 0
    for byte in content.utf8 {

```

```

        if byte == 10 { lineCount += 1 }
    }
    if !content.isEmpty && !content.hasSuffix("\n") { lineCount += 1 }

    let halfExcerpt = max(64, excerptBytes / 2) // 默认 512 bytes
    let data = Data(content.utf8)

    let headData = data.prefix(halfExcerpt)
    let headExcerpt = String(decoding: headData, as: UTF8.self)

    let tailData = data.suffix(halfExcerpt)
    let tailExcerpt = String(decoding: tailData, as: UTF8.self)

    var hash: UInt64 = 14_695_981_039_346_656_037
    for byte in content.utf8 {
        hash ^= UInt64(byte)
        hash &*= 1_099_511_628_211
    }
    let contentHash = String(format: "%08llx", hash)

    return ""
    [Context Object: \(objectID.rawValue)]
    Tool: \(toolName)
    Size: \(totalBytes) bytes, \(max(1, lineCount)) lines
    Content Type: text/plain
    Hash: \(contentHash)
    --- First \(headData.count) bytes ---
    \(headExcerpt)
    --- Last \(tailData.count) bytes ---
    \(tailExcerpt)
    ---
    To retrieve additional lines or full content, use `context_recall(id: "\(objectID.rawValue)", offset: <bytes>, limit: <bytes>)\`.
    ""
}
}

```

// File: Sources/LingXiCore/Modules/Context/ContextQuery.swift

```
import Foundation
```

/// Pager 的 Provider 无关输入；只取当前任务与最近用户文本，不把完整 Session 无差别送入检索。

```

public struct ContextQuery: Sendable, Equatable {
    public let text: String
    public let terms: [String]
    public let symbolHints: [String]
    public let symbolHintGroups: [[String]]
    public let symbolHintExtractionMilliseconds: Double
    public let relationHints: [String]

    public init(currentTask: String, recentUserMessages: [String] = []) {
        let queryText = ([currentTask] + recentUserMessages.suffix(2)).joined(separator: "\n")
        text = queryText
        terms = Array(Set(queryText.lowercased().split { !$0.isLetter && !$0.isNumber }.map(String.init).filter { $0.count > 1 })).sorted()
        let clock = ContinuousClock()
        let started = clock.now
        let identifiers = queryText.split { character in
            !(character.isLetter || character.isNumber || character == "_" || character == "." || character == "(")
        }.map(String.init).compactMap(Self.symbolIdentifier)
        var groups: [[String]] = []
        for candidate in identifiers {
            let components = candidate.split(separator: ".").map(String.init)
            guard components.count > 1 else {
                groups.append([candidate])
                continue
            }
        }
    }
}

```

```

        var group = [candidate]
        for end in stride(from: components.count - 1, through: 1, by: -1) {
            group.append(components[0..

```

// File: Sources/LingXiCore/Modules/Context/SensitivePathPolicy.swift

```

import Foundation

public struct SensitivePathPolicy: Sendable {
    private let rootPath: String
    private let excludedRootPaths: [String]

    public init(root: URL, excluding excludedRoots: [URL] = []) {
        rootPath = Self.resolvedPath(root)
        excludedRootPaths = excludedRoots.map(Self.resolvedPath)
    }

    public func isSensitive(_ url: URL) -> Bool {
        let path = Self.resolvedPath(url)
        if excludedRootPaths.contains(where: { path == $0 || path.hasPrefix($0 + "/" ) }) {
            return true
        }

        let rootPrefix = rootPath + "/"
        let components = path == rootPath ? [] : path.hasPrefix(rootPrefix)
            ? String(path.dropFirst(rootPrefix.count)).split(separator: "/").map(String.init)
            : URL(fileURLWithPath: path).pathComponents
        return components.contains { component in
            let name = component.lowercased()
            return [".ssh", ".aws", ".gnupg", ".netrc", ".npmrc", "id_rsa"].contains(name)
                || name == ".env" || name.hasPrefix(".env.") || name.hasSuffix(".env")
                || name.hasSuffix(".pem") || name.hasSuffix(".key")
                || name.contains("credential") || name.contains("secret") || name.contains("token")
        }
    }
}

```



```

        private static func resolvedPath(_ url: URL) -> String {
            url.standardizedFileURL.resolvingSymlinksInPath().path
        }
    }
}

```

// File: Sources/LingXiCore/Modules/Session/SessionRuntime.swift

```

import Foundation
import LingXiProtocol

public actor SessionRestoreScheduler {
    private var ready = false
    private var completed = false
    private var readyWaiters: [CheckedContinuation<Void, Never>] = []
    private var completionWaiters: [CheckedContinuation<Void, Never>] = []

    public init() {}

    func markReady() {
        guard !ready else { return }
        ready = true
        readyWaiters.forEach { $0.resume() }
        readyWaiters.removeAll()
    }

    func markCompleted() {
        guard !completed else { return }
        completed = true
        completionWaiters.forEach { $0.resume() }
        completionWaiters.removeAll()
    }

    public func waitUntilReady() async {
        guard !ready else { return }
        await withCheckedContinuation { readyWaiters.append($0) }
    }

    public func waitUntilCompleted() async {
        guard !completed else { return }
        await withCheckedContinuation { completionWaiters.append($0) }
    }
}

/// 一个 Session 的串行 Agent Lane。Tool 结果以结构化 parts 回写 Session，再进入下一步模型输入。
public actor SessionRuntime {
    private struct ActiveExecution {
        let id: UUID
        let task: Task<Void, Never>
        let streamID: StreamID
        let isRestore: Bool
    }

    private let store: any SessionStore
    private let sessionID: SessionID
    private let modelBus: ModelBus
    private let dataPlane: DataPlane
    private let contextEngine: L1ContextEngine
    private let toolRuntime: ToolRuntime
    private let questions: QuestionRuntime
    private let permissions: PermissionEngine
    private let performanceStore: PerformanceStore
    private let contextPager: ContextPager
    private let projectScanner: ProjectScanner
    private let compactor: ContextCompactor
}

```

```

private let cacheController: ContextCacheController
private let budgetPlanner: ContextBudgetPlanner
private let persistence: SQLitePersistenceStore?
private let eventSink: @Sendable (CoreEvent) async -> Void
private let interactive: Bool
private let diagnosticsEnabled: Bool
private let runID: AgentRunID?
private let rootRunID: AgentRunID?
private let parentRunID: AgentRunID?
private let rootSessionID: SessionID
private let parentSessionID: SessionID?
private let runObserver: (@Sendable (AgentRunStatus, String?, ModelUsage?, CoreError?, AgentTerminalTrace?) async -> Void)?
private let executionProfile: SubagentExecutionProfile?
private let systemContext: String?
private let systemContextAtBeginning: Bool
private let maximumAgentSteps: Int
private let deadlinePolicy: ExecutionDeadlinePolicy
private let restoreScheduler: SessionRestoreScheduler?
private let diagnostics: RuntimeDiagnosticsStore?
private var turnRunning = false
private var activeExecution: ActiveExecution?
private var shuttingDown = false
private var toolBatches: [ToolExchangeBatch] = []
private var compactionGeneration = 0
private var latestModelRequestID: ModelRequestID?
public private(set) var latestContextManifest: ProviderContextManifest?
private var currentActiveEntries: [ContextEntry] = []
private var activeStreamContinuation: AsyncThrowingStream<ModelEvent, Error>.Continuation?
private var providerCacheEpoch: ProviderCacheEpoch?
private var epochProviderVisibleDynamicTools: [ToolDefinition] = []

private func abortActiveProviderStream() {
    activeStreamContinuation?.finish(throwing: CancellationError())
    activeStreamContinuation = nil
}

private func canonicalToolDefinition(_ tool: ToolDefinition) -> String {
    var properties: [String] = []
    for key in tool.inputSchema.properties.keys.sorted() {
        guard let property = tool.inputSchema.properties[key] else { continue }
        let enumValues = property.enumValues?.joined(separator: ",") ?? ""
        let minimum = property.minimum.map { String($0) } ?? ""
        let maximum = property.maximum.map { String($0) } ?? ""
        properties.append("\(key):\(property.type.rawValue):\(property.description):\(enumValues):\(minimum):\(maximum)")
    }
    let required = tool.inputSchema.required.sorted().joined(separator: ",")
    let capabilities = tool.capability.kinds.map{.rawValue}.sorted().joined(separator: ",")
    let rawSchema: String
    if let rawInputSchema = tool.rawInputSchema {
        let encoder = JSONEncoder()
        encoder.outputFormatting = [.sortedKeys]
        rawSchema = (try? String(decoding: encoder.encode(rawInputSchema), as: UTF8.self)) ?? ""
    } else {
        rawSchema = ""
    }
    return "\(tool.id.rawValue)|\(tool.name)|\(tool.description)|\(properties.joined(separator: ","))|\(required)|\(capabilities)|\(
↵ rawSchema)"
}

private func computePrefixFingerprint(
    systemContext: String?,
    coreTools: [ToolDefinition],
    dynamicTools: [ToolDefinition],
    model: ModelID,
    reasoning: String?,

```

```

    contextEntries: [ContextEntry],
    userTurnID: MessageID?
) -> PrefixFingerprint {
    func fnv1a(_ string: String) -> String {
        var hash: UInt64 = 14_695_981_039_346_656_037
        for byte in string.utf8 {
            hash ^= UInt64(byte)
            hash &*= 1_099_511_628_211
        }
        return String(format: "%016llx", hash)
    }

    let sys = systemContext ?? ""
    let systemHash = fnv1a("sys:" + sys)
    let coreToolsSchema = coreTools.map(canonicalToolDefinition).joined(separator: "\n")
    let coreToolsHash = fnv1a("core:" + coreToolsSchema)
    let dynamicToolsSchema = dynamicTools.map(canonicalToolDefinition).joined(separator: "\n")
    let leasedToolsHash = fnv1a("leased:" + dynamicToolsSchema)
    let profileStr = "\(model.rawValue)|\ (reasoning ?? "none")"
    let requestProfileHash = fnv1a("profile:" + profileStr)

    let historyTexts = contextEntries.filter { $0.messageID != userTurnID && $0.source != .system }.map {
        "\($0.role):\($0.part)"
    }.joined(separator: "||")
    let historyStableHash = fnv1a("history:" + historyTexts)

    let stablePrefixHash = fnv1a("\ (systemHash)|\ (coreToolsHash)|\ (requestProfileHash)")

    return PrefixFingerprint(
        systemHash: systemHash,
        developerHash: "",
        coreToolsHash: coreToolsHash,
        skillPrefixHash: "",
        leasedToolsHash: leasedToolsHash,
        historyStableHash: historyStableHash,
        requestProfileHash: requestProfileHash,
        stablePrefixHash: stablePrefixHash
    )
}

private func cacheEpoch(for coreTools: [ToolDefinition]) -> ProviderCacheEpoch {
    let coreSchema = coreTools.map(canonicalToolDefinition).joined(separator: "\n")
    let canonical = "system:\ (systemContext ?? "")\n coreTools:\ (coreSchema)"
    var hashValue: UInt64 = 14_695_981_039_346_656_037
    for byte in canonical.utf8 {
        hashValue ^= UInt64(byte)
        hashValue &*= 1_099_511_628_211
    }
    let hash = String(format: "%016llx", hashValue)
    if let providerCacheEpoch, providerCacheEpoch.hash == hash { return providerCacheEpoch }
    let epoch = ProviderCacheEpoch(epoch: (providerCacheEpoch?.epoch ?? 0) + 1, hash: hash)
    providerCacheEpoch = epoch
    epochProviderVisibleDynamicTools = [] // 新纪元起始, 重置动态工具追加缓冲
    return epoch
}

init(
    store: any SessionStore,
    sessionID: SessionID,
    modelBus: ModelBus,
    dataPlane: DataPlane,
    contextEngine: L1ContextEngine,
    toolRuntime: ToolRuntime,
    questions: QuestionRuntime,
    permissions: PermissionEngine,

```

```

performanceStore: PerformanceStore,
contextPager: ContextPager,
projectScanner: ProjectScanner,
eventSink: @escaping @Sendable (CoreEvent) async -> Void,
compactor: ContextCompactor,
budgetPlanner: ContextBudgetPlanner,
persistence: SQLitePersistenceStore? = nil,
cacheController: ContextCacheController? = nil,
interactive: Bool = false,
diagnosticsEnabled: Bool = false,
runID: AgentRunID? = nil,
rootRunID: AgentRunID? = nil,
parentRunID: AgentRunID? = nil,
rootSessionID: SessionID? = nil,
parentSessionID: SessionID? = nil,
executionProfile: SubagentExecutionProfile? = nil,
systemContext: String? = nil,
systemContextAtBeginning: Bool = true,
maxAgentLoopSteps: Int = 32,
runObserver: (@Sendable (AgentRunStatus, String?, ModelUsage?, CoreError?, AgentTerminalTrace?) async -> Void)? = nil,
deadlinePolicy: ExecutionDeadlinePolicy = ExecutionDeadlinePolicy(),
restoreScheduler: SessionRestoreScheduler? = nil,
diagnostics: RuntimeDiagnosticsStore? = nil
) {
    self.store = store
    self.sessionID = sessionID
    self.modelBus = modelBus
    self.dataPlane = dataPlane
    self.contextEngine = contextEngine
    self.toolRuntime = toolRuntime
    self.questions = questions
    self.permissions = permissions
    self.performanceStore = performanceStore
    self.contextPager = contextPager
    self.projectScanner = projectScanner
    self.compactor = compactor
    self.cacheController = cacheController ?? ContextCacheController(contextPager: contextPager, scanner: projectScanner, compactor:
↳ compactor)
    self.eventSink = eventSink
    self.budgetPlanner = budgetPlanner
    self.persistence = persistence
    self.interactive = interactive
    self.diagnosticsEnabled = diagnosticsEnabled
    self.runID = runID
    self.rootRunID = rootRunID
    self.parentRunID = parentRunID
    self.rootSessionID = rootSessionID ?? sessionID
    self.parentSessionID = parentSessionID
    self.executionProfile = executionProfile
    self.systemContext = systemContext
    self.systemContextAtBeginning = systemContextAtBeginning
    maximumAgentSteps = max(1, executionProfile?.maxSteps ?? maxAgentLoopSteps)
    self.runObserver = runObserver
    self.deadlinePolicy = deadlinePolicy
    self.restoreScheduler = restoreScheduler
    self.diagnostics = diagnostics
}

public func restore() async throws {
    guard let persistence else { return }
    toolBatches = try await persistence.toolBatches(sessionID: sessionID)
    if let compacted = try await persistence.compaction(sessionID: sessionID) {
        compactionGeneration = compacted.generation
        await compactor.restoreResidencies(sessionID: sessionID, values: compacted.residencies)
    }
}

```

```

for call in toolBatches.flatMap(\.toolCallStates) where call.reply == nil {
    switch call.request {
    case let .permission(request):
        await permissions.register(request, onAsk: { [eventSink] in
            await eventSink(.permissionAsked(request))
        }, onReply: { [weak self] reply in
            await self?.humanReply(batchID: call.provenance.batchID, callID: call.call.callID, request: .permission(request), reply:
↪ .permission(reply))
            await self?.resumeDurableTurn()
        })
    case let .question(request):
        await questions.register(request) { [weak self] _, reply in
            await self?.humanReply(batchID: call.provenance.batchID, callID: call.call.callID, request: .question(request), reply: .
↪ question(reply))
            await self?.resumeDurableTurn()
        }
    case nil:
        break
    }
}

if toolBatches.contains(where: { $0.state == .pending || $0.state == .recoveryRequired }) && !toolBatches.flatMap(\.toolCallStates).
↪ contains(where: { $0.request != nil && $0.reply == nil }) {
    await resumeDurableTurn()
}

}

/// Continue a persisted Tool batch before making the next provider request.
private func resumeDurableTurn() async {
    guard !shuttingDown, !turnRunning else { return }
    turnRunning = true
    let profiler = TurnProfiler(sessionID: sessionID, enabled: performanceStore.enabled)
    let opened = await dataPlane.openAgentStream()
    let handle = TurnHandle(sessionID: sessionID, streamID: opened.stream.id)
    let task = (try? await store.session(sessionID).messages.last { $0.role == .user }?.content) ?? ""
    let runID = self.runID
    let rootSessionID = self.rootSessionID
    let parentSessionID = self.parentSessionID
    let sessionID = self.sessionID
    let deadline = deadlinePolicy.deadline(for: runID == nil ? .agentRun : .subagent, requested: executionProfile?.timeoutSeconds.map {
↪ .seconds($0) })
    let executionID = UUID()
    let turnTask = Task {
        await AgentExecutionContext.$current.withValue(runID.map { (sessionID: sessionID, runID: $0, rootSessionID: rootSessionID,
↪ parentSessionID: parentSessionID) }) {
            await self.runTurn(handle: handle, sink: opened.sink, task: task, profiler: profiler, deadline: deadline, executionID:
↪ executionID, resume: true)
        }
    }
    activeExecution = ActiveExecution(id: executionID, task: turnTask, streamID: opened.stream.id, isRestore: true)
    await restoreScheduler?.markReady()
    await dataPlane.trackAgent(turnTask, streamID: opened.stream.id)
}

/// 启动一轮对话并立即返回 DMA 通道；同一 Session 只允许一个活动 turn。
public func startTurn(_ content: String) async throws -> OpenedStream {
    guard !shuttingDown, !turnRunning else {
        throw CoreError(code: .turnAlreadyRunning, message: "该 Session 已有进行中的对话轮次")
    }
    turnRunning = true
    let profiler = TurnProfiler(sessionID: sessionID, enabled: performanceStore.enabled)
    do {
        _ = try await store.session(sessionID)
        let userMessage = try await store.appendMessage(sessionID, role: .user, content: content)
        let userEntry = ContextEntry(messageID: userMessage.id, role: .user, source: .userMessage, part: .text(content))
        var updatedEntries = currentActiveEntries
    }
}

```

```

    if updatedEntries.isEmpty {
        if let snapshot = await contextEngine.latestSnapshot(for: sessionID) {
            updatedEntries = snapshot.entries
        } else if let session = try? await store.session(sessionID) {
            let residentPages = await cacheController.residentPages(for: sessionID)
            updatedEntries = await contextEngine.entries(for: session, projectPages: residentPages, systemContext: systemContext,
↪ systemContextAtBeginning: systemContextAtBeginning)
        }
    }

    if !updatedEntries.contains(where: { $0.messageID == userMessage.id }) {
        updatedEntries.append(userEntry)
    }

    await syncL1ResidentAccounting(with: updatedEntries)
    guard modelBus.gateway.modelID != nil else {
        throw CoreError(
            code: .provider,
            message: "未配置模型 Provider; 请检查 providers.json 与 CredentialStore: \"(modelBus.gateway.missingRequirements.joined(
↪ separator: ", ")\"
        )
    }

    let opened = await dataPlane.openAgentStream()
    let handle = TurnHandle(sessionID: sessionID, streamID: opened.stream.id)
    await eventSink(.turnStarted(handle))
    await runObserver?(.running, nil, nil, nil, nil)
    let runID = self.runID
    let rootSessionID = self.rootSessionID
    let parentSessionID = self.parentSessionID
    let sessionID = self.sessionID
    let deadline = deadlinePolicy.deadline(for: runID == nil ? .agentRun : .subagent, requested: executionProfile?.timeoutSeconds.
↪ map { .seconds($0) })
    let executionID = UUID()
    let turnTask = Task {
        await AgentExecutionContext.$current.withValue(runID.map { (sessionID: sessionID, runID: $0, rootSessionID: rootSessionID,
↪ parentSessionID: parentSessionID) }) {
            await self.runTurn(handle: handle, sink: opened.sink, task: content, profiler: profiler, deadline: deadline, executionID
↪ : executionID)
        }
    }

    activeExecution = ActiveExecution(id: executionID, task: turnTask, streamID: opened.stream.id, isRestore: false)
    await dataPlane.trackAgent(turnTask, streamID: opened.stream.id)
    return opened.stream
} catch {
    turnRunning = false
    throw error
}
}

private func runTurn(
    handle: TurnHandle,
    sink: AsyncThrowingStream<StreamChunk, Error>.Continuation,
    task: String,
    profiler: TurnProfiler,
    deadline: ExecutionDeadline,
    executionID: UUID,
    resume: Bool = false
) async {
    var index = 0
    var finalUsage: ModelUsage?
    var finalReason: ModelFinishReason?
    var lastSuccessfulRead: (signature: ToolRuntime.ReadOnlySignature, content: String)?
    var deterministicFailures: [String: String] = [:]
    let executionProfile = self.executionProfile

    do {
        try ensureExecuting(executionID)
    }

```

```

let currentSession = try await store.session(sessionID)
let userTurnID = currentSession.messages.last(where: { $0.role == .user })?.id
?? MessageID("recovery:\(sessionID.rawValue):\\(runID?.rawValue ?? "session")")
let runLease = RunLease(sessionID: sessionID, turnID: TurnID(userTurnID.rawValue), runID: runID.map { RunID($0.rawValue) },
↪ revision: currentSession.revision)
if resume {
    let session = try await store.session(sessionID)
    try await settleDurableBatches(session: session, deadline: deadline, profiler: profiler)
    try ensureExecuting(executionID)
}
var lastCallBatchSignature: String?
var consecutiveIdenticalBatches = 0
var consecutiveIdenticalFailures = 0
var consecutiveEmptyBatches = 0
var lastObservedFailure: String?
var lastExecutedCall: ToolCall?
var lastObservedContent: String?
var pendingLifecycleTraces: [ToolLifecycleTrace] = []

for step in 0..

```

```

        hardLimit: budget.hardInputLimit,
        economicThreshold: economicThreshold,
        estimatedEvictionTokens: max(1, rawTokens - budget.lowWaterTokens),
        stablePrefixTokens: max(1, budget.lowWaterTokens / 2),
        remainingHorizon: remainingHorizon
    )

let compacted: CompactionResult
switch decision {
case .skip:
    compacted = CompactionResult(
        entries: projectedEntries,
        beforeTokens: rawTokens,
        afterTokens: rawTokens,
        pagedOut: 0,
        derivedCreated: 0,
        triggered: false,
        triggerSource: .automaticHighWater,
        mandatoryFloor: 0,
        unitsKept: projectedEntries.count,
        historicalToolBatchesPagedOut: 0,
        projectBackedOffloads: 0,
        redundantDrops: 0,
        emergencyTrims: 0,
        noEligibleReduction: true
    )
case .economicCompact:
    compacted = try await compactor.compact(
        sessionID: sessionID,
        entries: projectedEntries,
        budget: budget,
        batches: toolBatches,
        projectBackedContents: Set(residentPages.map(\.content))
    )
    if compacted.triggered {
        await cacheController.scheduler.recordCompactionOccurred(sessionID: sessionID, step: step + 1)
    }
case .emergencyWindowProtection:
    compacted = try await compactor.compact(
        sessionID: sessionID,
        entries: projectedEntries,
        budget: budget,
        batches: toolBatches,
        projectBackedContents: Set(residentPages.map(\.content)),
        trigger: .automaticHighWater
    )
    if compacted.triggered {
        await cacheController.scheduler.recordCompactionOccurred(sessionID: sessionID, step: step + 1)
    }
}

profiler.recordCompaction(compacted, budget: budget)
if compacted.triggered { compactionGeneration += 1 }
if compacted.triggered { try await persistCompaction() }

let residentDerived = await cacheController.residentDerivedPages(for: sessionID)
let residentDerivedEntries: [ContextEntry] = residentDerived.map { page in
    ContextEntry(messageID: MessageID(page.id), role: .system, source: .derivedPage, part: .text("[Session context]\n\n(page.
↪ content)"))
}

var finalEntries = compacted.entries + residentDerivedEntries
var finalTokens = ConservativeTokenEstimator().estimate(entries: finalEntries)
if finalTokens > budget.hardInputLimit {
    let emergency = try await compactor.compact(sessionID: sessionID, entries: finalEntries, budget: budget, batches:
↪ toolBatches, projectBackedContents: Set(residentPages.map(\.content)), trigger: .emergencyHardLimit)
    profiler.recordCompaction(emergency, budget: budget)
}

```



```

        finalEntries = emergency.entries
        finalTokens = emergency.afterTokens
        if emergency.triggered { compactionGeneration += 1 }
        if emergency.triggered { try await persistCompaction() }
    }
    await cacheController.recordProviderInputTokens(sessionID: sessionID, tokens: finalTokens)
    await syncL1ResidentAccounting(with: finalEntries)
    let context = await contextEngine.snapshot(for: session, activeEntries: finalEntries, systemContext: systemContext,
    ↪ estimatedTokens: finalTokens, mandatoryTokens: compacted.mandatoryFloor, liveToolBatchCount: toolBatches.filter { $0.state != .
    ↪ consumed }.count, compactionGeneration: compactionGeneration)

    let manifestEntries: [ContextManifestEntry] = finalEntries.compactMap { entry in
        let tokenCount = ConservativeTokenEstimator().estimate(text: ContextCompactor.content(of: entry.part))
        let sourceKind: String
        let origin: String
        let inclusionReason: String
        let cacheProvenance: String
        let sourceID = entry.messageID?.rawValue ?? entry.page?.id ?? "transient"

        switch entry.source {
        case .system:
            sourceKind = "Pinned"
            origin = "systemContext"
            inclusionReason = "Pinned system prompt"
            cacheProvenance = "pinned"
        case .userMessage:
            if entry.messageID == session.messages.last?.id {
                sourceKind = "Current Turn"
                origin = "currentTurn"
                inclusionReason = "Active turn user message"
                cacheProvenance = "currentTurn"
            } else {
                sourceKind = "L1"
                origin = entry.messageID?.rawValue ?? "history"
                inclusionReason = "L1 historical user turn"
                cacheProvenance = "l1WorkingSet"
            }
        case .assistantMessage:
            sourceKind = "L1"
            origin = entry.messageID?.rawValue ?? "history"
            inclusionReason = "L1 historical assistant response"
            cacheProvenance = "l1WorkingSet"
        case .toolCall, .toolResult:
            sourceKind = "L1"
            origin = entry.messageID?.rawValue ?? "tool"
            inclusionReason = "L1 tool execution record"
            cacheProvenance = "l1WorkingSet"
        case .projectPage:
            sourceKind = "L1"
            origin = entry.page?.path ?? "projectPage"
            inclusionReason = "Dynamic page-in via Cache Controller"
            cacheProvenance = "cacheControllerL1"
        case .derivedPage:
            sourceKind = "L1"
            origin = entry.page?.path ?? "derivedSummary"
            inclusionReason = "Dynamic page-in via Cache Controller"
            cacheProvenance = "cacheControllerL1"
        }
    }
    return ContextManifestEntry(
        sourceKind: sourceKind,
        sourceID: sourceID,
        origin: origin,
        tokenCount: tokenCount,
        inclusionReason: inclusionReason,
        cacheProvenance: cacheProvenance
    )

```

```

    )
}
let manifest = ProviderContextManifest(
    sessionID: sessionID,
    step: step + 1,
    entries: manifestEntries,
    totalTokens: finalTokens
)
self.latestContextManifest = manifest
trace("context.manifest", step: step + 1)
trace("context.build.end", step: step + 1)
profiler.recordContext(context, build: contextStarted.duration(to: clock.now))
let dispatchStarted = clock.now
trace("model.next.begin", step: step + 1)
let effectiveContinuationID = toolBatches.last(where: { $0.state == .settledAwaitingConsumption })?.continuationRequestID ??
↪ latestModelRequestID
let supportsTools = modelBus.gateway.endpoint?.capabilities.toolCalling ?? true
let coreIDs = ToolRuntime.coreToolIDs
let coreTools = availableTools.filter { coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
let activeDynamic = availableTools.filter { !coreIDs.contains($0.id) }

// 检查是否有已有暴露工具发生了不兼容的 schema 改变
var schemaChanged = false
for tool in activeDynamic {
    if let existing = epochProviderVisibleDynamicTools.first(where: { $0.id == tool.id }) {
        if canonicalToolDefinition(existing) != canonicalToolDefinition(tool) {
            schemaChanged = true
            break
        }
    }
}
if schemaChanged {
    // 禁止原地篡改 schema! 显式推进 CacheEpoch 并重建 manifest
    await cacheController.advanceEpoch(sessionID: sessionID, reason: "mcpToolSchemaChanged")
    self.providerCacheEpoch = nil
    epochProviderVisibleDynamicTools = []
}

// 首次在当前 Epoch 暴露给模型的动态工具, 纯追加到 Provider-Visible Manifest
for tool in activeDynamic where !epochProviderVisibleDynamicTools.contains(where: { $0.id == tool.id }) {
    epochProviderVisibleDynamicTools.append(tool)
}

// 冻结 Epoch Tool Manifest 规范:
// Runtime 内部可根据租约 page-out / 释放资源,
// 但 Provider-Visible Tool Manifest 在当前 Epoch 内绝对冻结且单调追加, 绝不删除、缩水或洗牌!
let effectiveTools = supportsTools ? (coreTools + epochProviderVisibleDynamicTools) : []
let supportsReasoning = modelBus.gateway.endpoint?.capabilities.reasoning ?? true
let effectiveReasoning = supportsReasoning ? modelBus.gateway.reasoning : nil

let systemPinnedTokens = context.entries.filter { $0.source == .system }.reduce(0) { $0 + ConservativeTokenEstimator().
↪ estimate(entries: [$1]) }
let currentTurnTokens = context.entries.filter { $0.messageID == userTurnID }.reduce(0) { $0 + ConservativeTokenEstimator().
↪ estimate(entries: [$1]) }
let llTokens = max(0, finalTokens - systemPinnedTokens - currentTurnTokens)

let fingerprint = computePrefixFingerprint(
    systemContext: systemContext,
    coreTools: coreTools,
    dynamicTools: epochProviderVisibleDynamicTools,
    model: try modelID(),
    reasoning: effectiveReasoning,
    contextEntries: context.entries,
    userTurnID: userTurnID
)

```

```

    let approxPrefixBytes = (systemContext?.utf8.count ?? 0) + coreTools.reduce(0) { $0 + $1.name.utf8.count + $1.description.
↪ utf8.count + 120 }
    let approxVolatileBytes = currentTurnTokens * 4
    await cacheController.recordFingerprint(sessionID: sessionID, fingerprint: fingerprint, prefixBytes: approxPrefixBytes,
↪ volatileBytes: approxVolatileBytes)
    let clientHealth = await cacheController.lastClientHealth(for: sessionID) ?? ClientStructuralCacheHealth(stablePrefixHash:
↪ fingerprint.stablePrefixHash)

    let epochInfo = cacheEpoch(for: coreTools)
    let cachePlan = CanonicalCachePlan(
        epochIdentity: CanonicalCachePlan.EpochIdentity(epoch: Int(epochInfo.epoch), reason: "turn_\(step + 1)"),
        immutableBase: CanonicalCachePlan.ImmutableBase(
            systemPrompt: systemContext,
            developerPrompt: nil,
            coreTools: coreTools,
            stablePolicy: nil
        ),
        appendOnlyContext: CanonicalCachePlan.AppendOnlyContext(
            dynamicTools: epochProviderVisibleDynamicTools,
            messages: context.modelMessages(),
            skillActivations: []
        ),
        volatileTail: CanonicalCachePlan.VolatileTail(
            currentTurnState: nil,
            ephemeralNotes: nil
        ),
        structuralHealth: clientHealth
    )

    let request = ModelRequest(
        continuationOf: effectiveContinuationID,
        model: try modelID(),
        executionID: runID,
        messages: context.modelMessages(),
        tools: effectiveTools,
        reasoning: effectiveReasoning,
        debugStep: step + 1,
        overallTimeoutSeconds: deadline.remainingSeconds(),
        idleTimeoutSeconds: deadlinePolicy.idleTimeout(for: .provider).map { Self.durationSeconds($0) },
        cachePlan: cachePlan
    )
    if finalTokens > budget.hardInputLimit {
        throw CoreError(code: .contextBudgetExceeded, message: "最终模型请求超出输入预算")
    }
    try ModelRequestProtocolValidator.validate(context.entries)
    profiler.recordProtocolValidator(liveBatches: toolBatches.filter { $0.state != .consumed }.count)
    let submittedBatchIDs = Set(toolBatches.filter { $0.state == .settledAwaitingConsumption }.map { \.batchID })
    trace("provider.stream.begin", step: step + 1)
    latestModelRequestID = request.requestID

    let toolSchemaTokens = ConservativeTokenEstimator().estimate(tools: effectiveTools)
    let providerFramingTokens = budgetPlanner.policy.fixedOverheadTokens
    let estimatedPromptTokens = finalTokens + toolSchemaTokens + providerFramingTokens
    let cacheTelemetry = ProviderCacheTelemetry(
        stablePrefixTokens: systemPinnedTokens + toolSchemaTokens,
        reusableHistoryTokens: llTokens,
        volatileTailTokens: currentTurnTokens + providerFramingTokens,
        epoch: epochInfo
    )

    let triggerReason = step == 0 ? "initial_turn_prompt" : "tool_result_continuation"
    let callTrace = ProviderCallTrace(
        sessionID: sessionID,
        userTurnID: userTurnID,
        runID: runID,
        parentRunID: parentRunID,

```

```

        providerRequestID: "local:\(request.requestID.rawValue)",
        sequence: step + 1,
        reason: triggerReason,
        model: request.model.rawValue,
        estimatedPromptTokens: estimatedPromptTokens,
        actualUsage: nil,
        toolSchemaTokens: toolSchemaTokens,
        toolCount: effectiveTools.count,
        llTokens: llTokens,
        systemPinnedTokens: systemPinnedTokens,
        currentTurnTokens: currentTurnTokens,
        providerFramingTokens: providerFramingTokens,
        retryAttempt: 0,
        cacheTelemetry: cacheTelemetry
    )
    profiler.recordProviderCall(callTrace)

    let initialActivity = ProviderActivitySnapshot(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: "local:\(request.requestID.rawValue)",
        state: .requesting,
        model: request.model.rawValue,
        updatedAt: Date()
    )
    await eventSink(.providerActivityChanged(initialActivity))

    let events: AsyncThrowingStream<ModelEvent, Error>
    do {
        events = try await modelBus.stream(request) { [weak self] snapshot in
            await self?.eventSink(.providerActivityChanged(snapshot))
        }
    } catch {
        let failedActivity = ProviderActivitySnapshot(
            sessionID: sessionID,
            runID: runID,
            providerRequestID: "local:\(request.requestID.rawValue)",
            state: .failed,
            model: request.model.rawValue,
            detail: (error as? CoreError)?.message ?? error.localizedDescription,
            statusCode: (error as? ProviderRateLimitError)?.statusCode,
            updatedAt: Date()
        )
        await eventSink(.providerActivityChanged(failedActivity))
        profiler.updateLastProviderCallRateMetrics(await modelBus.gateway.rateMetrics(for: request.requestID))
        throw error
    }

    let dispatch = dispatchStarted.duration(to: clock.now)
    let streamStarted = clock.now
    var text = ""
    var visibleReasoning = false
    var calls: [ToolCall] = []
    var providerRequestID = "local:\(request.requestID.rawValue)"

    var streamContinuation: AsyncThrowingStream<ModelEvent, Error>.Continuation?
    let cancellableEvents = AsyncThrowingStream<ModelEvent, Error> { cont in
        streamContinuation = cont
    }
    activeStreamContinuation = streamContinuation
    let pumpTask = Task {
        do {
            for try await event in events {
                streamContinuation?.yield(event)
            }
            streamContinuation?.finish()
        }
    }

```

```

    } catch {
        streamContinuation?.finish(throwing: error)
    }
}
defer {
    activeStreamContinuation = nil
    pumpTask.cancel()
}

try await withTaskCancellationHandler {
    for try await event in cancellableEvents {
        if Task.isCancelled || shuttingDown { throw CancellationError() }
        if let currentRev = try? await store.currentRevision(sessionID), currentRev != runLease.revision {
            logDiagnostic("session.stale_event_dropped eventType=streaming_delta sessionID=\(sessionID.rawValue)
↪ eventRevision=\(runLease.revision) currentRevision=\(currentRev)")
            throw StaleRunError(sessionID: sessionID, expected: currentRev, actual: runLease.revision)
        }
        if await ProviderActivityRegistry.shared.isCancelled(providerRequestID: providerRequestID, runID: runID) {
            throw CancellationError()
        }
        switch event {
        case let .providerRequestID(value):
            providerRequestID = value
            profiler.updateLastProviderCallRequestID(value)
            let snapshot = await ProviderActivityRegistry.shared.record(
                sessionID: sessionID,
                runID: runID,
                providerRequestID: value,
                state: .streaming,
                model: request.model.rawValue
            )
            await eventSink(.providerActivityChanged(snapshot))
        case .started:
            trace("provider.stream.start", step: step + 1)
        case let .textDelta(delta):
            sink.yield(StreamChunk(streamID: handle.streamID, sessionID: sessionID, agentRunID: runID, modelStepID:
↪ currentModelStepID, stepNumber: currentStepNumber, index: index, text: delta, kind: .text))
            profiler.recordText(delta, streamElapsed: streamStarted.duration(to: clock.now))
            index += 1
            text += delta
        case let .reasoningDelta(delta):
            sink.yield(StreamChunk(streamID: handle.streamID, sessionID: sessionID, agentRunID: runID, modelStepID:
↪ currentModelStepID, stepNumber: currentStepNumber, index: index, text: delta, kind: .reasoning))
            profiler.recordReasoning(delta, streamElapsed: streamStarted.duration(to: clock.now))
            visibleReasoning = true
            index += 1
        case .toolCallStarted, .toolCallDelta:
            break // Tool arguments 只在完整聚合后进入控制面。
        case let .toolCallCompleted(call):
            calls.append(call.withProvenance(sessionID: sessionID, agentRunID: runID, modelStepID: currentModelStepID))
        case let .usage(usage):
            finalUsage = usage
            profiler.recordUsage(usage)
            profiler.updateLastProviderCallUsage(usage)
            if let input = usage.inputTokens {
                let hasCacheReport = (usage.cacheReadTokens != nil)
                let cached = usage.cacheReadTokens ?? 0
                let currentModel = (try? modelID())?.rawValue
                await cacheController.recordProviderCacheHit(
                    sessionID: sessionID,
                    cachedTokens: cached,
                    promptTokens: input,
                    cacheWriteTokens: usage.cacheWriteTokens,
                    provider: modelBus.gateway.endpoint?.providerID,
                    model: currentModel,

```

```

        isUnavailable: !hasCacheReport
    )
    if let record = await cacheController.lastProviderCacheRecord(for: sessionID),
        let diag = record.missDiagnostics {
        FileHandle.standardError.write(Data("[PREFIX_DIAG] \{(diag)\n".utf8))
    }
}
}
case let .completed(reason):
    finalReason = reason
case let .failed(error):
    throw error
}
}
} onCancel: {
    streamContinuation?.finish(throwing: CancellationError())
}
try ensureExecuting(executionID)
if modelBus.gateway.endpoint?.wireProtocol == .chatCompletions,
    (finalUsage?.reasoningTokens ?? 0) > 0,
    !visibleReasoning {
    throw CoreError(code: .modelStream, message: "Provider 返回 reasoning_tokens 但未产生 visible reasoning")
}
let rateMetrics = await modelBus.gateway.rateMetrics(for: request.requestID)
profiler.updateLastProviderCallRateMetrics(rateMetrics)
trace("provider.stream.end", step: step + 1)
await diagnostics?.record(
    kind: .provider,
    event: "provider.call.trace",
    sessionID: sessionID,
    runID: runID,
    parentRunID: parentRunID,
    providerRequestID: providerRequestID,
    metadata: [
        "sequence": String(step + 1),
        "reason": triggerReason,
        "model": request.model.rawValue,
        "estimatedPromptTokens": String(estimatedPromptTokens),
        "actualInputTokens": finalUsage?.inputTokens.map(String.init) ?? "none",
        "toolSchemaTokens": String(toolSchemaTokens),
        "toolCount": String(effectiveTools.count),
        "llTokens": String(llTokens),
        "systemPinnedTokens": String(systemPinnedTokens),
        "currentTurnTokens": String(currentTurnTokens),
        "providerFramingTokens": String(providerFramingTokens),
        "retryAttempt": String(rateMetrics.retryCount),
        "retryCount": String(rateMetrics.retryCount),
        "rateWaitMilliseconds": String(rateMetrics.rateWaitMilliseconds),
        "rateLimit429Count": String(rateMetrics.rateLimit429Count)
    ]
)

try await consumeSettledBatches(submittedBatchIDs)
profiler.recordModel(dispatch: dispatch, stream: streamStarted.duration(to: clock.now))

guard !calls.isEmpty else {
    await toolRuntime.finishMCPProviderStep(sessionID: sessionID)
    await completeTurn(
        handle: handle,
        sink: sink,
        content: text,
        finishReason: finalReason,
        usage: finalUsage,
        profiler: profiler,
        executionID: executionID,
        lease: runLease
    )
}

```

```

    )
    return
}

var assistantParts: [SessionMessagePart] = calls.map(SessionMessagePart.toolCall)
if !text.isEmpty { assistantParts.insert(.text(text), at: 0) }
trace("tool.batch.begin", step: step + 1, toolCount: calls.count)
await runObserver?.(waitingForTool, nil, finalUsage, nil, nil)
trace("tool.batch.count", step: step + 1, toolCount: calls.count)
guard Set(calls.map(\.callID)).count == calls.count else {
    throw CoreError(code: .modelStream, message: "Tool batch 含重复 toolCallID")
}
for call in calls {
    let rawName = call.toolID.rawValue.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !rawName.isEmpty && rawName != "Tool" && rawName != "{}" && rawName != "unknown" else {
        throw CoreError(code: .modelStream, message: "模型返回了无效的 Tool Name: '\(call.toolID.rawValue)'" )
    }
}
trace("session.parts.append.begin", step: step + 1, toolCount: calls.count)
let assistantMessage: Message
if persistence != nil { assistantMessage = Message(id: MessageID(UUID().uuidString), role: .assistant, parts: assistantParts
↪ , createdAt: .now) }
else { assistantMessage = try await store.appendMessage(sessionID, role: .assistant, parts: assistantParts, expectedRevision
↪ : runLease.revision) }
let batchID = UUID().uuidString
let batch = ToolExchangeBatch(batchID: batchID, sessionID: sessionID, assistantMessageID: assistantMessage.id, toolCalls:
↪ calls, toolCallStates: calls.map { DurableToolCall(call: $0, provenance: ToolCallProvenance(batchID: batchID, sessionID: sessionID,
↪ agentRunID: runID, providerRequestID: request.requestID, providerStep: step + 1)) }, continuationRequestID: request.requestID,
↪ providerStep: step + 1, state: .pending, estimatedTokens: ConservativeTokenEstimator().estimate(entries: assistantParts.map {
↪ ContextEntry(messageID: assistantMessage.id, role: .assistant, source: .toolCall, part: $0) }), revision: runLease.revision, turnID
↪ : runLease.turnID)
if let persistence { try await persistence.appendAssistantMessageAndBatch(sessionID: sessionID, message: assistantMessage,
↪ batch: batch, expectedRevision: runLease.revision) }
toolBatches.append(batch)
let assistantEntries = assistantParts.map { ContextEntry(messageID: assistantMessage.id, role: .assistant, source: .toolCall
↪ , part: $0) }
var updatedEntries = currentActiveEntries
updatedEntries.append(contentsOf: assistantEntries)
await syncLLResidentAccounting(with: updatedEntries)
trace("session.parts.append.end", step: step + 1, toolCount: calls.count)

trace("tool.batch.settle.begin", step: step + 1, toolCount: calls.count)
let signatures = calls.map { try? toolRuntime.readOnlySignature(for: $0) }
var outcomes = Array<ToolRuntime.ExecutionOutcome?>(repeating: nil, count: calls.count)
var publishedOutcomes = Set<Int>()
var primaryByIndex: [Int: Int] = [:]
var primaryBySignature: [ToolRuntime.ReadOnlySignature: Int] = [:]

for (offset, call) in calls.enumerated() {
    await eventSink(.toolCallCompleted(call.withProvenance(sessionID: sessionID, agentRunID: runID, modelStepID:
↪ currentModelStepID)))
    trace("tool.execute.begin", step: step + 1, toolCallID: call.callID)
    if let signature = signatures[offset], let previous = lastSuccessfulRead, previous.signature == signature {
        let outcome = duplicateOutcome(for: call, signature: signature, content: previous.content)
        outcomes[offset] = outcome
        await publishCompletedTool(outcome, batchID: batchID, modelStepID: currentModelStepID)
        publishedOutcomes.insert(offset)
    } else if let failureSignature = deterministicFailures[failureKey(for: call)] {
        let outcome = repeatedFailureOutcome(for: call, signature: failureSignature)
        outcomes[offset] = outcome
        await publishCompletedTool(outcome, batchID: batchID, modelStepID: currentModelStepID)
        publishedOutcomes.insert(offset)
    } else if let signature = signatures[offset], let primary = primaryBySignature[signature] {
        primaryByIndex[offset] = primary
    } else {

```

```

        if let signature = signatures[offset] { primaryBySignature[signature] = offset }
        primaryByIndex[offset] = offset
    }
}

await withTaskGroup(of: (Int, ToolRuntime.ExecutionOutcome).self) { group in
    for (offset, call) in calls.enumerated() where outcomes[offset] == nil && primaryByIndex[offset] == offset {
        let executionCall = call.withProvenance(sessionID: sessionID, agentRunID: runID, modelStepID: currentModelStepID)
        let observer = ToolExecutionObserver(
            permissionAsked: { request in await self.waitForHuman(batchID: batchID, callID: call.callID, request: .
↳ permission(request)) },
            permissionResolved: { request, reply in await self.humanReply(batchID: batchID, callID: call.callID, request: .
↳ permission(request), reply: .permission(reply)) },
            executionClaimed: { claim in
                await self.executionClaimed(batchID: batchID, callID: call.callID, claim: claim)
                await self.eventSink(.toolExecutionClaimed(executionCall))
            },
            questionAsked: { request in await self.waitForHuman(batchID: batchID, callID: call.callID, request: .question
↳ (request)) },
            questionResolved: { request, reply in await self.humanReply(batchID: batchID, callID: call.callID, request: .
↳ question(request), reply: .question(reply)) }
        )
        group.addTask { [toolRuntime, sessionID, eventSink] in
            let outcome = await toolRuntime.executeWithMetrics(executionCall, sessionID: sessionID, projectID: session.
↳ projectID ?? ProjectID("ephemeral"), executionProfile: executionProfile, onPermissionAsked: { request in
                await eventSink(.permissionAsked(request))
            }, observer: observer)
            return (offset, outcome)
        }
    }
    for await (offset, outcome) in group {
        outcomes[offset] = outcome
        guard !Task.isCancelled else { continue }
        trace("tool.execute.end", step: step + 1, toolCallID: calls[offset].callID)
        await publishCompletedTool(outcome, batchID: batchID, modelStepID: currentModelStepID)
        publishedOutcomes.insert(offset)

        // 关键: 当 primary tool 产出结果时, 同批次复用它的 secondary tool 立即发布完成, 杜绝幽灵旋转
        for sec in calls.indices where primaryByIndex[sec] == offset && sec != offset && outcomes[sec] == nil {
            let secondaryCall = calls[sec]
            let secOutcome = duplicateOutcome(
                for: secondaryCall,
                signature: signatures[sec]!,
                content: outcome.result.content,
                timing: outcome.result.timing,
                execution: outcome.execution
            )
            outcomes[sec] = secOutcome
            trace("tool.execute.end", step: step + 1, toolCallID: secondaryCall.callID)
            await publishCompletedTool(secOutcome, batchID: batchID, modelStepID: currentModelStepID)
            publishedOutcomes.insert(sec)
        }
    }
}

try Task.checkCancellation()

for (offset, call) in calls.enumerated() where outcomes[offset] == nil {
    guard let primary = primaryByIndex[offset], let previous = outcomes[primary] else {
        throw CoreError(code: .modelStream, message: "Tool batch settlement 缺少结果: \(call.callID.rawValue)")
    }
    let outcome = duplicateOutcome(
        for: call,
        signature: signatures[offset]!,
        content: previous.result.content,
        timing: previous.result.timing,

```



```

        execution: previous.execution
    )
    outcomes[offset] = outcome
    await publishCompletedTool(outcome, batchID: batchID, modelStepID: currentModelStepID)
    publishedOutcomes.insert(offset)
}

let settled = outcomes.compactMap { $0 }
guard settled.count == calls.count, zip(calls, settled).allSatisfy({ $0.0.callID == $0.1.result.callID }) else {
    throw CoreError(code: .modelStream, message: "ToolCall 与 ToolResult ID 不匹配")
}

for (offset, outcome) in settled.enumerated() {
    let call = calls[offset]
    profiler.recordTool(outcome)
    let result = outcome.result
    if !publishedOutcomes.contains(offset) {
        trace("tool.execute.end", step: step + 1, toolCallID: call.callID)
        await publishCompletedTool(outcome, batchID: batchID, modelStepID: currentModelStepID)
    }
    if let signature = signatures[offset], result.success || result.error?.code == "duplicateToolCall" {
        lastSuccessfulRead = (signature, result.content)
    } else {
        lastSuccessfulRead = nil
    }
    if let error = result.error, isDeterministicFailure(result) {
        deterministicFailures[failureKey(for: call)] = "\(error.code):\(error.message)"
    }
}

}

trace("session.parts.append.begin", step: step + 1, toolCount: settled.count)
// Phase 1A E-Core Sidecar Storage (Fail-Open):
for outcome in settled {
    let res = outcome.result
    if res.content.utf8.count >= cacheController.ecoreStore.configuration.objectizationThreshold {
        await cacheController.ecoreStore.store(
            sessionID: sessionID,
            toolCallID: res.callID,
            toolName: res.toolName ?? "unknown",
            content: res.content
        )
    }
}

let resultMessage: Message
if persistence != nil { resultMessage = Message(id: MessageID(UUID().uuidString), role: .tool, parts: settled.map { .
↪ toolResult($0.result) }, createdAt: .now) }
else { resultMessage = try await store.appendMessage(sessionID, role: .tool, parts: settled.map { .toolResult($0.result) },
↪ expectedRevision: runLease.revision) }
try await settleBatch(batchID: batchID, resultMessageID: resultMessage.id, results: settled.map{\.result}, resultMessage:
↪ resultMessage, lease: runLease)
await toolRuntime.finishMCPProviderStep(sessionID: sessionID)
let hasCancelledTool = settled.contains {
    $0.result.outcome == .cancelled ||
    $0.result.error?.code == CoreError.Code.toolCancelled.rawValue ||
    $0.result.error?.code == CoreError.Code.permissionCancelled.rawValue
}
if Task.isCancelled || shuttingDown || hasCancelledTool {
    throw CoreError(code: .toolCancelled, message: "AgentRun 已取消")
}
let allEmptyResults = settled.allSatisfy { outcome in
    let content = outcome.result.content.trimmingCharacters(in: .whitespacesAndNewlines)
    return content == "[]" || content == "[] (empty list)" || content.contains("0 tools found") || content.contains("No
↪ tools found") || (content.isEmpty && outcome.result.error == nil)
}
if allEmptyResults {
    consecutiveEmptyBatches += 1
} else {

```

```

        consecutiveEmptyBatches = 0
    }

    let toolResultEntries = settled.map { outcome -> ContextEntry in
        var res = outcome.result
        if consecutiveEmptyBatches >= 2 && outcome.result.callID == settled.last?.result.callID {
            let note = "\n[System note: The tool or resource query returned empty results. Do NOT repeatedly retry with slight
↪ keyword variations. If the capability or tool is unavailable, please skip this step or report directly to the user.]"
            res = res.withContent(res.content + note)
        }

        return ContextEntry(messageID: resultMessage.id, role: .tool, source: .toolResult, part: .toolResult(res))
    }

    var postToolEntries = currentActiveEntries
    postToolEntries.append(contentsOf: toolResultEntries)
    await syncL1ResidentAccounting(with: postToolEntries)
    trace("session.parts.append.end", step: step + 1, toolCount: settled.count)
    trace("tool.batch.settle.end", step: step + 1, toolCount: calls.count)
    pendingLifecycleTraces = settled.compactMap(\.lifecycleTrace)

    lastExecutedCall = calls.last
    lastObservedContent = settled.last?.result.content

    // No-progress detection:
    let batchSignature = calls.map { "\($0.toolID.rawValue):\($0.arguments)" }.joined(separator: ";")
    if batchSignature == lastCallBatchSignature {
        consecutiveIdenticalBatches += 1
    } else {
        lastCallBatchSignature = batchSignature
        consecutiveIdenticalBatches = 1
    }

    let batchFailures = settled.compactMap { $0.result.error?.message }.joined(separator: ";")
    if !batchFailures.isEmpty && batchFailures == lastObservedFailure {
        consecutiveIdenticalFailures += 1
    } else {
        lastObservedFailure = batchFailures.isEmpty ? nil : batchFailures
        consecutiveIdenticalFailures = batchFailures.isEmpty ? 0 : 1
    }

    if consecutiveIdenticalFailures >= 3 {
        let callDesc = lastExecutedCall.map { "\($0.toolID.rawValue) \($0.arguments.prefix(80))" } ?? "none"
        throw CoreError(
            code: .agentStepLimitReached,
            message: "Agent Tool Loop 检测到无进展死循环: 连续 \($consecutiveIdenticalFailures) 次遇到相同的 Tool 失败: \($
↪ lastObservedFailure ?? "") · 当前 step: \($step + 1) · 最后 ToolCall: \($callDesc)"
        )
    }

    if consecutiveEmptyBatches >= 4 {
        let callDesc = lastExecutedCall.map { "\($0.toolID.rawValue) \($0.arguments.prefix(80))" } ?? "none"
        throw CoreError(
            code: .agentStepLimitReached,
            message: "Agent 检测到连续 \($consecutiveEmptyBatches) 次工具或搜索返回空结果, 已主动停止盲目重试。建议跳过不可用能力或
↪ 向用户汇报。 · 当前 step: \($step + 1) · 最后 ToolCall: \($callDesc)"
        )
    }

    if consecutiveIdenticalBatches >= 8 {
        let callDesc = lastExecutedCall.map { "\($0.toolID.rawValue) \($0.arguments.prefix(80))" } ?? "none"
        throw CoreError(
            code: .agentStepLimitReached,
            message: "Agent Tool Loop 检测到无进展死循环: 连续 \($consecutiveIdenticalBatches) 次调用完全相同的 ToolCall · 当前
↪ step: \($step + 1) · 最后 ToolCall: \($callDesc) · 最后 observation: \($String((lastObservedContent ?? "").prefix(80)))"
        )
    }
}

```

```

    }
    let lastCallDesc = lastExecutedCall.map { "\($0.toolID.rawValue) \($0.arguments.prefix(80))" } ?? "none"
    let lastObsDesc = String((lastObservedContent ?? "").prefix(80))
    throw CoreError(
        code: .agentStepLimitReached,
        message: "Agent Tool Loop 超过上限 (\(maximumAgentSteps) steps) · 当前 step: \($0) · 最后 ToolCall: \($1)
↪ lastCallDesc) · 最后 observation: \($2)"
    )
} catch let error as StaleRunError {
    logDiagnostic("session.stale_run_dropped sessionID=\(error.sessionID.rawValue) expected=\(error.expected) actual=\(error.actual)
↪ ")
    await toolRuntime.abortMCPTurn(sessionID: sessionID)
    _ = await finishExecution(executionID)
    sink.finish()
} catch let error as CoreError {
    await toolRuntime.abortMCPTurn(sessionID: sessionID)
    await failTurn(handle: handle, sink: sink, error: error, profiler: profiler, executionID: executionID)
} catch is CancellationError {
    await toolRuntime.abortMCPTurn(sessionID: sessionID)
    await failTurn(handle: handle, sink: sink, error: CoreError(code: .toolCancelled, message: "AgentRun 已取消"), profiler:
↪ profiler, executionID: executionID)
} catch {
    await toolRuntime.abortMCPTurn(sessionID: sessionID)
    await failTurn(
        handle: handle,
        sink: sink,
        error: CoreError(code: .provider, message: String(describing: error)),
        profiler: profiler,
        executionID: executionID
    )
}
}

private func modelID() throws -> ModelID {
    guard let modelID = modelBus.gateway.modelID else {
        throw CoreError(code: .provider, message: "未配置模型 Provider")
    }
    return modelID
}

private func trace(_ event: String, step: Int, toolCallID: ToolCallID? = nil, toolCount: Int? = nil) {
    guard diagnosticsEnabled || diagnostics != nil else { return }
    var fields = ["[agent-trace]", "event=\(event)", "sessionID=\(sessionID.rawValue)", "step=\(step)"]
    if let toolCallID { fields.append("toolCallID=\(toolCallID.rawValue)") }
    if let toolCount { fields.append("toolCount=\(toolCount)") }
    let kind: RuntimeTraceKind = event.hasPrefix("provider.") ? .provider : event.hasPrefix("tool.") || event.hasPrefix("session.parts")
↪ ? .tool : event.hasPrefix("context.") ? .session : .agentRun
    let executionID = activeExecution?.id.uuidString
    let providerRequestID = latestModelRequestID?.rawValue
    Task { await diagnostics?.record(kind: kind, event: event, sessionID: sessionID, runID: runID, rootRunID: rootRunID, parentRunID:
↪ parentRunID, executionID: executionID, providerRequestID: providerRequestID, toolCallID: toolCallID, metadata: ["step": String(step)
↪ ], "toolCount": String(toolCount ?? 0)) }
}

private func syncL1ResidentAccounting(with entries: [ContextEntry]) async {
    currentActiveEntries = entries
    let tokens = ConservativeTokenEstimator().estimate(entries: entries)
    await cacheController.recordSessionL1Tokens(sessionID: sessionID, tokens: tokens, count: entries.count)
    if let session = try? await store.session(sessionID) {
        _ = await contextEngine.snapshot(for: session, activeEntries: entries, systemContext: systemContext, estimatedTokens: tokens,
↪ liveToolBatchCount: toolBatches.filter { $0.state != .consumed }.count, compactionGeneration: compactionGeneration)
    }
}

private func duplicateOutcome(

```

```

    for call: ToolCall,
    signature: ToolRuntime.ReadOnlySignature,
    content: String,
    timing: ToolTiming? = nil,
    execution: Duration = .zero
) -> ToolRuntime.ExecutionOutcome {
    let executionMs = max(1.0, timing?.executionMilliseconds ?? Double(execution.components.seconds * 1000))
    let queueMs = timing?.queueMilliseconds ?? 0
    let inheritedTiming = ToolTiming(milliseconds: executionMs, queueMilliseconds: queueMs, executionMilliseconds: executionMs)
    return ToolRuntime.ExecutionOutcome(
        result: ToolResult(
            callID: call.callID,
            success: false,
            content: content,
            error: ToolError(code: "duplicateToolCall", message: "连续重复调用已复用前一成功结果"),
            toolName: signature.toolName,
            timing: inheritedTiming
        ),
        permissionWait: .zero,
        permissionAsked: false,
        execution: execution == .zero ? .milliseconds(Int64(executionMs)) : execution,
        toolName: signature.toolName,
        resource: signature.resource
    )
}

private func failureKey(for call: ToolCall) -> String {
    "\(call.toolID.rawValue)|\(call.arguments)"
}

private func isDeterministicFailure(_ result: ToolResult) -> Bool {
    guard !result.success, let error = result.error else { return false }
    if result.outcome == .timedOut || result.outcome == .idleTimedOut || result.outcome == .cancelled { return false }
    return [
        CoreError.Code.toolNotFound.rawValue,
        CoreError.Code.toolArgumentInvalid.rawValue,
        CoreError.Code.toolValidationError.rawValue,
        CoreError.Code.permissionDenied.rawValue,
        CoreError.Code.workspaceViolation.rawValue,
        CoreError.Code.resourceNotFound.rawValue,
        CoreError.Code.resourceOutsideWorkspace.rawValue,
        CoreError.Code.symlinkEscape.rawValue,
        CoreError.Code.binaryFileUnsupported.rawValue,
        CoreError.Code.contentChanged.rawValue,
        CoreError.Code.ambiguousEdit.rawValue,
        CoreError.Code.invalidPatch.rawValue,
        CoreError.Code.patchConflict.rawValue,
        CoreError.Code.editTargetNotFound.rawValue
    ].contains(error.code)
}

private func repeatedFailureOutcome(for call: ToolCall, signature: String) -> ToolRuntime.ExecutionOutcome {
    let message = "相同 ToolID 和参数已再次产生确定性失败，已阻止原样重试；请更换策略或先修复原因。"
    return ToolRuntime.ExecutionOutcome(
        result: ToolResult(
            callID: call.callID,
            success: false,
            content: message,
            error: ToolError(code: "deterministicFailureRepeated", message: message),
            toolName: call.toolID.rawValue,
            outcome: .failure,
            summary: "repeat blocked",
            metadata: ["repeatBlocked": "true", "errorSignature": signature, "retryability": Retryability.none.rawValue]
        ),
        permissionWait: .zero,

```

```

        permissionAsked: false,
        execution: .zero,
        toolName: call.toolID.rawValue,
        resource: nil
    )
}

public func compactNow() async throws -> CompactSessionResponse {
    guard !turnRunning else { throw CoreError(code: .turnAlreadyRunning, message: "对话进行中，不能压缩当前 Session") }
    let session = try await store.session(sessionID)
    let residentPages = await cacheController.residentPages(for: sessionID)
    let toolTokens = ConservativeTokenEstimator().estimate(tools: await toolRuntime.availableDefinitions())
    let budget = budgetPlanner.plan(profile: modelBus.gateway.contextProfile, toolTokens: toolTokens)
    let entries = await contextEngine.entries(for: session, projectPages: residentPages, systemContext: systemContext,
    ↪ systemContextAtBeginning: systemContextAtBeginning)
    let result = try await compactor.compact(sessionID: sessionID, entries: entries, budget: budget, batches: toolBatches,
    ↪ projectBackedContents: Set(residentPages.map(\.content)), trigger: .manual)
    if result.triggered { compactionGeneration += 1 }
    await syncL1ResidentAccounting(with: result.entries)
    let response = CompactSessionResponse(triggerSource: result.triggerSource.rawValue, beforeEstimatedTokens: result.beforeTokens,
    ↪ afterEstimatedTokens: result.afterTokens, targetLowWater: budget.lowWaterTokens, mandatoryFloor: result.mandatoryFloor, unitsKept:
    ↪ result.unitsKept, unitsPagedOut: result.pagedOut, historicalToolBatchesPagedOut: result.historicalToolBatchesPagedOut,
    ↪ projectBackedOffloads: result.projectBackedOffloads, derivedPagesCreated: result.derivedCreated, redundantDrops: result.
    ↪ redundantDrops, emergencyTrims: result.emergencyTrims, compactionGeneration: compactionGeneration, noEligibleReduction: result.
    ↪ noEligibleReduction)
    try await persistCompaction()
    return response
}

public func cancelCurrentTurn() async {
    abortActiveProviderStream()
    guard let execution = activeExecution else { turnRunning = false; return }
    execution.task.cancel()
    lifecycle("cancellationRequested", waitingOn: "turnTask")
    await dataPlane.finishAgentStream(execution.streamID)
}

public func shutdown() async {
    shuttingDown = true
    abortActiveProviderStream()
    guard let execution = activeExecution else { turnRunning = false; return }
    execution.task.cancel()
    lifecycle("cancellationRequested", waitingOn: "turnTask")
    await dataPlane.finishAgentStream(execution.streamID)
    await execution.task.value
    _ = await finishExecution(execution.id)
    lifecycle("cleanupCompleted", waitingOn: "turnTask")
}

public func cacheMetrics() async -> (l2Pages: Int, l3Pages: Int, pageOutCount: Int, pageInCount: Int, historicalToolPages: Int, l3Hits:
    ↪ Int, l2Hits: Int, l2Promotions: Int) {
    await compactor.cacheMetrics(sessionID: sessionID)
}

public func contextUnitStates() async -> [ContextUnitDebugSnapshot] {
    await compactor.unitStates(sessionID: sessionID)
}

private func settleDurableBatches(session: Session, deadline: ExecutionDeadline, profiler: TurnProfiler) async throws {
    for batchID in toolBatches.filter({ $0.state == .pending || $0.state == .recoveryRequired }).map(\.batchID) {
        guard let batch = toolBatches.first(where: { $0.batchID == batchID }), batch.resultMessageID == nil else { continue }
        for call in batch.toolCallStates where call.state == .recoveryRequired {
            await completedToolCall(batchID: batchID, result: ToolResult(callID: call.call.callID, success: false, content: "", error:
            ↪ ToolError(code: CoreError.Code.executionStateUnknown.rawValue, message: "重启时 mutation Tool 的执行状态未知，需要验证"), toolName:
            ↪ call.call.toolID.rawValue, metadata: ["executionState": "unknown", "verificationRequired": "true"]))
        }
    }
}

```

```

    }
    for call in batch.toolCallStates {
        guard call.state == .requested, case let .question(request)? = call.request, case let .question(reply)? = call.reply else {
↳ continue }
        let selected = reply.selectedOptionIndices.map { request.options[$0] }
        let payload: [String: Any] = ["questionID": request.questionID.rawValue, "cancelled": reply.cancelled, "selectedOptions":
↳ selected, "text": reply.text ?? NSNull()]
        let content = String(decoding: try JSONSerialization.data(withJSONObject: payload, options: [.sortedKeys]), as: UTF8.self)
        await completedToolCall(batchID: batchID, result: ToolResult(callID: call.call.callID, success: true, content: content,
↳ toolName: call.call.toolID.rawValue))
    }

    let executable = toolBatches.first(where: { $0.batchID == batchID })?.toolCallStates.filter {
        guard $0.state == .requested else { return false }
        if case .question? = $0.request { return false }
        return true
    } ?? []
    await withTaskGroup(of: ToolRuntime.ExecutionOutcome.self) { group in
        for durable in executable {
            let call = durable.call
            let executionCall = call.withProvenance(sessionID: sessionID, agentRunID: runID)
            let observer = ToolExecutionObserver(
                permissionAsked: { request in await self.waitForHuman(batchID: batchID, callID: call.callID, request: .permission
↳ (request)) },
                permissionResolved: { request, reply in await self.humanReply(batchID: batchID, callID: call.callID, request: .
↳ permission(request), reply: .permission(reply)) },
                executionClaimed: { claim in
                    await self.executionClaimed(batchID: batchID, callID: call.callID, claim: claim)
                    await self.eventSink(.toolExecutionClaimed(executionCall))
                },
                questionAsked: { request in await self.waitForHuman(batchID: batchID, callID: call.callID, request: .question(
↳ request)) },
                questionResolved: { request, reply in await self.humanReply(batchID: batchID, callID: call.callID, request: .
↳ question(request), reply: .question(reply)) }
            )
            group.addTask { [toolRuntime, sessionID, eventSink, executionProfile] in
                await toolRuntime.executeWithMetrics(executionCall, sessionID: sessionID, projectID: session.projectID ?? ProjectID
↳ ("ephemeral"), executionProfile: executionProfile, onPermissionAsked: { request in
                    await eventSink(.permissionAsked(request))
                }, observer: observer)
            }
        }
        for await outcome in group {
            profiler.recordTool(outcome)
            await completedToolCall(batchID: batchID, result: outcome.result)
            outcome.lifecycleTrace?.record(.resultCommitted, exitCode: outcome.result.exitCode.map { Int32($0) })
            await eventSink(.toolResult(outcome.result.withProvenance(sessionID: sessionID, agentRunID: runID)))
            outcome.lifecycleTrace?.record(.applicationProjectionReceived, exitCode: outcome.result.exitCode.map { Int32($0) })
        }
    }

    guard let settled = toolBatches.first(where: { $0.batchID == batchID }), settled.toolCallStates.allSatisfy({ $0.state == .
↳ completed && $0.result != nil }) else { continue }
    let results = settled.toolCalls.compactMap { call in settled.toolCallStates.first { $0.call.callID == call.callID }?.result }
    guard results.count == settled.toolCalls.count else { continue }
    let message = Message(id: MessageID(UUID().uuidString), role: .tool, parts: results.map(SessionMessagePart.toolResult),
↳ createdAt: .now)
    try await settleBatch(batchID: batchID, resultMessageID: message.id, results: results, resultMessage: message)
    await toolRuntime.finishMCPProviderStep(sessionID: sessionID)
}

private func settleBatch(batchID: String, resultMessageID: MessageID, results: [ToolResult], resultMessage: Message, lease: RunLease? =
↳ nil) async throws {
    if let lease {

```

```

        let current = try await store.currentRevision(lease.sessionID)
        guard current == lease.revision else {
            logDiagnostic("session.stale_event_dropped eventType=tool_batch_settle sessionID=\(lease.sessionID.rawValue) eventRevision
↪ =\((lease.revision) currentRevision=\(current)")
            throw StaleRunError(sessionID: lease.sessionID, expected: current, actual: lease.revision)
        }
    }
    guard let index = toolBatches.firstIndex(where: { $0.batchID == batchID }), toolBatches[index].resultMessageID == nil, toolBatches[
↪ index].state == .pending || toolBatches[index].state == .recoveryRequired else { return }
    toolBatches[index] = toolBatches[index].with(state: .settledAwaitingConsumption, resultMessageID: resultMessageID, toolResults:
↪ results, revision: lease?.revision, turnID: lease?.turnID)
    if let persistence { try await persistence.appendToolResultMessageAndSettle(sessionID: sessionID, message: resultMessage, batch:
↪ toolBatches[index], expectedRevision: lease?.revision) }
}

private func waitingForHuman(batchID: String, callID: ToolCallID, request: ToolCallHumanRequest) async {
    await updateToolCall(batchID: batchID, callID: callID) { $0.with(state: .waitingForHuman, request: request, replaceHumanExchange:
↪ true) }
    await runObserver?().waitingForUser, nil, nil, nil, nil)
}

private func humanReply(batchID: String, callID: ToolCallID, request: ToolCallHumanRequest, reply: ToolCallHumanReply) async {
    await updateToolCall(batchID: batchID, callID: callID) { $0.with(state: .requested, request: request, reply: reply,
↪ replaceHumanExchange: true) }
    await runObserver?().running, nil, nil, nil, nil)
}

private func executionClaimed(batchID: String, callID: ToolCallID, claim: ToolExecutionClaim) async {
    await updateToolCall(batchID: batchID, callID: callID) { $0.with(state: .executing, executionClaim: claim) }
}

private func completedToolCall(batchID: String, result: ToolResult) async {
    await updateToolCall(batchID: batchID, callID: result.callID) { $0.with(state: .completed, result: result) }
}

private func publishCompletedTool(_ outcome: ToolRuntime.ExecutionOutcome, batchID: String, modelStepID: ModelStepID?) async {
    let result = outcome.result
    await completedToolCall(batchID: batchID, result: result)
    outcome.lifecycleTrace?.record(.resultCommitted, exitCode: result.exitCode.map { Int32($0) })
    await eventSink(.toolResult(result.withProvenance(sessionID: sessionID, agentRunID: runID, modelStepID: modelStepID)))
    outcome.lifecycleTrace?.record(.applicationProjectionReceived, exitCode: result.exitCode.map { Int32($0) })
}

private func updateToolCall(batchID: String, callID: ToolCallID, _ update: (DurableToolCall) -> DurableToolCall) async {
    guard let batchIndex = toolBatches.firstIndex(where: { $0.batchID == batchID }), let callIndex = toolBatches[batchIndex].
↪ toolCallStates.firstIndex(where: { $0.call.callID == callID }) else { return }
    var states = toolBatches[batchIndex].toolCallStates
    states[callIndex] = update(states[callIndex])
    toolBatches[batchIndex] = toolBatches[batchIndex].with(state: toolBatches[batchIndex].state, toolCallStates: states)
    try? await persistence?.saveToolBatch(toolBatches[batchIndex])
}

private func consumeSettledBatches(_ batchIDs: Set<String>) async throws {
    let updates = toolBatches.enumerated().compactMap { index, batch -> (Int, ToolExchangeBatch)? in
        guard batchIDs.contains(batch.batchID), batch.state == .settledAwaitingConsumption else { return nil }
        return (index, batch.with(state: .consumed))
    }
    if let persistence { try await persistence.saveToolBatches(updates.map{\.1}) }
    for (index, batch) in updates { toolBatches[index] = batch }
}

private func persistCompaction() async throws {
    guard let persistence else { return }
    try await persistence.saveCompaction(
        sessionID: sessionID,

```

```

        generation: compactionGeneration,
        residencies: await compactor.unitStates(sessionID: sessionID),
        derivedPages: await compactor.derivedStore.pages(sessionID: sessionID)
    )
}

private func completeTurn(
    handle: TurnHandle,
    sink: AsyncThrowingStream<StreamChunk, Error>.Continuation,
    content: String,
    finishReason: ModelFinishReason?,
    usage: ModelUsage?,
    profiler: TurnProfiler,
    executionID: UUID,
    lease: RunLease? = nil
) async {
    defer { Task { await dataPlane.finishAgentStream(handle.streamID) } }
    do {
        guard isExecuting(executionID) else { return }
        if let runID, await ProviderActivityRegistry.shared.isRunCancelled(runID) { return }
        if let lease {
            let current = try await store.currentRevision(lease.sessionID)
            guard current == lease.revision else {
                logDiagnostic("session.stale_event_dropped eventType=complete_turn sessionID=\(lease.sessionID.rawValue) eventRevision
↳ =\((lease.revision) currentRevision=\(current)")
                _ = await finishExecution(executionID)
                sink.finish()
                return
            }
        }
        let message = try await store.appendMessage(handle.sessionID, role: .assistant, content: content, expectedRevision: lease?.
↳ revision)
        let assistantEntry = ContextEntry(messageID: message.id, role: .assistant, source: .assistantMessage, part: .text(content))
        var updatedEntries = currentActiveEntries
        if !updatedEntries.contains(where: { $0.messageID == message.id }) {
            updatedEntries.append(assistantEntry)
        }
        await syncL1ResidentAccounting(with: updatedEntries)
        guard await finishExecution(executionID) else { return }
        await performanceStore.recordProviderCalls(sessionID: handle.sessionID, calls: profiler.recordedProviderCalls)
        if let report = profiler.report() { await performanceStore.save(report) }
        let reason: TerminalReason = content.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty ? .emptyCompletion : .completed
        let trace = makeTerminalTrace(
            reason: reason,
            transition: "running -> completed",
            source: "completeTurn",
            explanation: reason == .emptyCompletion ? "模型生成了空回复" : "模型完成了目标",
            finishReason: finishReason?.rawValue
        )
        await runObserver?(.completed, content, usage, nil, trace)
        await eventSink(.turnCompleted(TurnResult(
            sessionID: handle.sessionID,
            streamID: handle.streamID,
            assistantMessageID: message.id,
            finishReason: finishReason,
            usage: usage
        )))
        Task { [sessionID = handle.sessionID] in
            await self.summarizeSessionIfNeeded(sessionID: sessionID)
        }
    } catch let error as StaleRunError {
        logDiagnostic("session.stale_run_dropped sessionID=\(error.sessionID.rawValue) expected=\(error.expected) actual=\(error.actual)
↳ ")
        _ = await finishExecution(executionID)
        sink.finish()
    }
}

```



```

    } catch let error as CoreError {
        await failTurn(handle: handle, sink: sink, error: error, profiler: profiler, executionID: executionID)
    } catch {
        await failTurn(handle: handle, sink: sink, error: CoreError(code: .transport, message: String(describing: error)), profiler:
↳ profiler, executionID: executionID)
    }
}

private func makeTerminalTrace(
    reason: TerminalReason,
    transition: String,
    source: String,
    explanation: String,
    finishReason: String? = nil
) -> AgentTerminalTrace {
    AgentTerminalTrace(
        runID: runID ?? AgentRunID(UUID().uuidString),
        sessionID: sessionID,
        lastProviderRequestID: latestModelRequestID?.rawValue,
        finishReason: finishReason,
        lastToolCallID: toolBatches.last?.toolCalls.last?.callID,
        terminalTransition: transition,
        terminalReason: reason,
        transitionSource: source,
        explanation: explanation
    )
}

private func failTurn(
    handle: TurnHandle,
    sink: AsyncThrowingStream<StreamChunk, Error>.Continuation,
    error: CoreError,
    profiler: TurnProfiler,
    executionID: UUID
) async {
    guard activeExecution?.id == executionID else { return }
    await diagnostics?.record(kind: .error, event: "turn.failed", sessionID: sessionID, runID: runID, rootRunID: rootRunID, parentRunID:
↳ parentRunID, executionID: executionID.uuidString, providerRequestID: latestModelRequestID?.rawValue, errorCode: error.code.
↳ rawValue)
    let preservesDurableBatch = toolBatches.contains { ($0.state == .pending || $0.state == .recoveryRequired) && $0.resultMessageID ==
↳ nil }
    if Task.isCancelled && preservesDurableBatch {
        sink.finish()
        await dataPlane.finishAgentStream(handle.streamID)
        guard await finishExecution(executionID) else { return }
        await performanceStore.recordProviderCalls(sessionID: handle.sessionID, calls: profiler.recordedProviderCalls)
        if let report = profiler.report() { await performanceStore.save(report) }
        return
    }
    let isCancellation = Task.isCancelled || error.code == .toolCancelled || shuttingDown
    if let latestID = latestModelRequestID?.rawValue {
        let snapshot = await ProviderActivityRegistry.shared.record(
            sessionID: sessionID,
            runID: runID,
            providerRequestID: "local:\\(latestID)",
            state: isCancellation ? .cancelled : .failed
        )
        await eventSink(.providerActivityChanged(snapshot))
    }
    sink.finish(throwing: error)
    await dataPlane.finishAgentStream(handle.streamID)
    guard await finishExecution(executionID) else { return }
    await performanceStore.recordProviderCalls(sessionID: handle.sessionID, calls: profiler.recordedProviderCalls)
    if let report = profiler.report() { await performanceStore.save(report) }
    let reason: TerminalReason = {

```

```

        if isCancellation { return .userCancelled }
        if [.commandTimedOut, .idleTimedOut].contains(error.code) { return .deadlineExceeded }
        if error.code == .agentStepLimitReached { return .maxStepsReached }
        if error.code == .permissionDenied { return .blocked }
        if error.code == .provider { return .providerFailure }
        return .runtimeFailure
    }()
    let status: AgentRunStatus = isCancellation ? .cancelled : [.commandTimedOut, .idleTimedOut].contains(error.code) ? .timedOut : .
↪ failed
    let trace = makeTerminalTrace(
        reason: reason,
        transition: "running -> \(status.rawValue)",
        source: "failTurn",
        explanation: error.message
    )
    await runObserver?(status, nil, nil, error, trace)
    await eventSink(.turnFailed(TurnFailure(sessionID: handle.sessionID, streamID: handle.streamID, error: error)))
}

private func finishExecution(_ id: UUID) async -> Bool {
    guard let execution = activeExecution, execution.id == id else { return false }
    activeExecution = nil
    turnRunning = false
    if execution.isRestore { await restoreScheduler?.markCompleted() }
    return true
}

private func isExecuting(_ id: UUID) -> Bool {
    !shuttingDown && activeExecution?.id == id
}

private func ensureExecuting(_ id: UUID) throws {
    guard isExecuting(id) else { throw CancellationError() }
}

private static func durationSeconds(_ duration: Duration) -> Double {
    let components = duration.components
    return Double(components.seconds) + Double(components.attoseconds) / 1_000_000_000_000_000_000
}

private func lifecycle(_ event: String, waitingOn: String) {
    ExecutionLifecycleTrace.log(event, category: runID == nil ? .agentRun : .subagent, waitingOn: waitingOn)
}

private func logDiagnostic(_ message: String) {
    FileHandle.standardError.write(Data("[SESSION_RUNTIME] \(message)\n".utf8))
}

private var isRunningInTestEnvironment: Bool {
    NSClassFromString("XCTest") != nil ||
    ProcessInfo.processInfo.processName.contains("Testing") ||
    ProcessInfo.processInfo.processName.contains("xctest") ||
    CommandLine.arguments.contains(where: {
        $0.contains("Testing") || $0.contains("xctest") || $0.contains("swift-testing") || $0.contains("test")
    })
}

private func summarizeSessionIfNeeded(sessionID: SessionID) async {
    guard !isRunningInTestEnvironment else { return }
    guard runID == nil, persistence != nil else { return }
    guard let session = try? await store.session(sessionID) else { return }
    let userTurns = session.messages.filter { $0.role == .user }
    guard !userTurns.isEmpty else { return }

    // 当标题为空，或者当前仍为默认未命名/首句截断时，且对话在前 3 轮内，触发小模型智能摘要

```

```

    let isEmpty = session.title == nil || session.title?.isEmpty == true || session.title == "未命名会话" || session.title?.
    hasSuffix("...") == true
    guard isEmpty || userTurns.count <= 2 else { return }

    let relevantMessages = session.messages.filter { $0.role == .user || $0.role == .assistant }
    guard !relevantMessages.isEmpty else { return }
    let snippet = relevantMessages.prefix(4).map { msg in
        let role = msg.role == .user ? "用户" : "助手"
        let text = msg.content.prefix(160).replacingOccurrences(of: "\n", with: " ")
        return "\(role): \(text)"
    }.joined(separator: "\n")

    guard let model = modelBus.gateway.modelID else { return }
    let prompt = "请根据以下简短对话，用一句话（18个汉字以内）概括会话的核心任务，不要标点、引号或前缀，直接输出标题：\n\(snippet)"
    let request = ModelRequest(
        model: model,
        messages: [
            ModelMessage(role: .system, content: "你是会话摘要生成助手。请严格输出18个汉字以内的精准中文任务摘要，不包含标点符号、引号或解
            释。"),
            ModelMessage(role: .user, content: prompt)
        ],
        tools: [],
        reasoning: nil,
        overallTimeoutSeconds: 6.0
    )

    var summaryText = ""
    do {
        let stream = try await modelBus.stream(request)
        for try await event in stream {
            if case let .textDelta(delta) = event {
                summaryText += delta
            }
        }
    } catch {
        return
    }

    let clean = summaryText.trimmingCharacters(in: .whitespacesAndNewlines)
        .replacingOccurrences(of: "\"", with: "")
        .replacingOccurrences(of: " ", with: "")
        .replacingOccurrences(of: " ", with: "")
        .replacingOccurrences(of: "《", with: "")
        .replacingOccurrences(of: "》", with: "")
        .replacingOccurrences(of: "。", with: "")
        .replacingOccurrences(of: ":", with: "")
        .replacingOccurrences(of: ";", with: "")
        .replacingOccurrences(of: "\n", with: " ")

    if !clean.isEmpty && clean.count <= 35 {
        _ = try? await store.updateTitle(sessionID, title: clean)
    }
}
}

```

// File: Sources/LingXiCore/Modules/Session/SessionTurnCoordinator.swift

```

import Foundation
import LingXiProtocol

/// SessionTurnCoordinator: 负责 Session 内 Turn / Run 生命周期、并发约束、Event Log 与 Stream 屏障。
/// 硬规则：一个 Session 最多一个 active Root Run；额外 Turn 默认排队 (queue)。
public actor SessionTurnCoordinator {
    public let sessionID: SessionID
    public let eventLog: SessionEventLog

```

```

public private(set) var activeRootRunID: RunID?
private var queuedTurns: [TurnSnapshot] = []
private var turns: [TurnID: TurnSnapshot] = [:]
private var runs: [RunID: RunSnapshot] = [:]
private var interactions: [InteractionID: InteractionSnapshot] = [:]
private var modelSteps: [ModelStepID: ModelStepSnapshot] = [:]
private var toolInvocations: [ToolCallID: ToolInvocationSnapshot] = [:]

private var streamFrames: [StreamID: [StreamFrame]] = [:]
private var streamSubscribers: [StreamID: [UUID: AsyncStream<StreamFrame>.Continuation]] = [:]

public init(sessionID: SessionID, eventLog: SessionEventLog) {
    self.sessionID = sessionID
    self.eventLog = eventLog
}

/// 撤回 (undo) 操作后的全量状态重置与重新水合
public func resetForRevert(remainingMessages: [Message]) async {
    activeRootRunID = nil
    queuedTurns.removeAll()
    turns.removeAll()
    runs.removeAll()
    interactions.removeAll()
    modelSteps.removeAll()
    toolInvocations.removeAll()
    streamFrames.removeAll()

    await eventLog.resetToEvents([])
    await hydrateHistoricalMessages(remainingMessages)
}

/// 从持久化消息流中水合还原历史 turns 和 timeline events
public func hydrateHistoricalMessages(_ messages: [Message]) async {
    guard turns.isEmpty, !messages.isEmpty else { return }

    // 预先建立 callID 到 toolResult 的全局索引, 杜绝历史工具水合产生孤儿悬空 running 状态
    var toolResultsByCallID: [ToolCallID: ToolResult] = [:]
    for msg in messages {
        for part in msg.parts {
            if case let .toolResult(res) = part {
                toolResultsByCallID[res.callID] = res
            }
        }
    }

    var completedCallIDs: Set<ToolCallID> = []
    var currentTurnID: TurnID?
    for msg in messages {
        let causal = CausalContext(sessionID: sessionID, turnID: currentTurnID)
        switch msg.role {
        case .user:
            let tID = TurnID(msg.id.rawValue)
            currentTurnID = tID
            let snap = MessageSnapshot(
                messageID: msg.id,
                role: .user,
                text: msg.content,
                createdAt: msg.createdAt
            )
            let turnSnap = TurnSnapshot(
                turnID: tID,
                sessionID: sessionID,
                userMessage: snap,
                executionIntent: TurnExecutionIntent(),

```

```

        status: .completed,
        createdAt: msg.createdAt
    )
    turns[tID] = turnSnap

    let turnCausal = CausalContext(sessionID: sessionID, turnID: tID)
    await eventLog.append(causal: turnCausal, payload: .turnCreated(turnSnap))
    await eventLog.append(causal: turnCausal, payload: .userMessageCommitted(snap))

case .assistant:
    var textContent = ""
    for part in msg.parts {
        switch part {
        case let .text(txt):
            textContent += txt
        case let .toolCall(tc):
            let hasResult = toolResultsByCallID[tc.callID] != nil
            let invocation = ToolInvocationSnapshot(
                callID: tc.callID,
                toolID: tc.toolID,
                displayName: tc.toolName,
                argumentsSummary: tc.arguments,
                state: hasResult ? .completed : .cancelled
            )
            toolInvocations[tc.callID] = invocation
            await eventLog.append(causal: causal, payload: .toolRequested(invocation))
            if let res = toolResultsByCallID[tc.callID] {
                let summaryText = res.summary.isEmpty ? (res.content.count > 100 ? String(res.content.prefix(100)) + "...": res
↪ .content) : res.summary
                let resSnap = ToolResultSnapshot(
                    callID: res.callID,
                    toolName: res.toolName,
                    success: res.success,
                    summary: summaryText
                )
                await eventLog.append(causal: causal, payload: .toolCompleted(
                    callID: res.callID,
                    result: resSnap,
                    stdoutFinalIndex: nil,
                    stderrFinalIndex: nil
                ))
                completedCallIDs.insert(tc.callID)
            } else {
                await eventLog.append(causal: causal, payload: .toolCancelled(
                    callID: tc.callID,
                    stdoutFinalIndex: nil,
                    stderrFinalIndex: nil
                ))
                completedCallIDs.insert(tc.callID)
            }
        }
        case .toolResult:
            break
    }
}

if !textContent.isEmpty || !msg.parts.isEmpty {
    await eventLog.append(causal: causal, payload: .assistantMessageCommitted(
        messageID: msg.id,
        content: textContent,
        assistantFinalIndex: 0
    ))
}

if let tID = currentTurnID {
    await eventLog.append(causal: causal, payload: .turnCompleted(turnID: tID, terminalReason: .completed))
}

```

```

    case .tool:
        for part in msg.parts {
            if case let .toolResult(res) = part, !completedCallIDs.contains(res.callID) {
                let summaryText = res.summary.isEmpty ? (res.content.count > 100 ? String(res.content.prefix(100)) + "...": res.
↪ content) : res.summary
                let resSnap = ToolResultSnapshot(
                    callID: res.callID,
                    toolName: res.toolName,
                    success: res.success,
                    summary: summaryText
                )
                await eventLog.append(causal: causal, payload: .toolCompleted(
                    callID: res.callID,
                    result: resSnap,
                    stdoutFinalIndex: nil,
                    stderrFinalIndex: nil
                ))
                completedCallIDs.insert(res.callID)
            }
        }
    }
}

```

// MARK: - Turn 提交与调度

```

public struct SubmitTurnDecision: Sendable {
    public let turn: TurnSnapshot
    public let status: TurnStatus
    public let runID: RunID?
    public let shouldStartExecution: Bool
}

public func submitTurn(
    input: UserInput,
    intent: TurnExecutionIntent,
    userMessage: MessageSnapshot
) async -> SubmitTurnDecision {
    let turnID = TurnID()
    let causal = CausalContext(sessionID: sessionID, turnID: turnID)

    // 1. Commit user message and turn created events
    await eventLog.append(causal: causal, payload: .turnCreated(
        TurnSnapshot(turnID: turnID, sessionID: sessionID, userMessage: userMessage, executionIntent: intent, status: .queued)
    ))
    await eventLog.append(causal: causal, payload: .userMessageCommitted(userMessage))

    // 2. Check Root Run concurrency (max 1 active Root Run)
    if activeRootRunID != nil {
        let queuedTurn = TurnSnapshot(
            turnID: turnID,
            sessionID: sessionID,
            userMessage: userMessage,
            executionIntent: intent,
            status: .queued,
            rootRunID: nil,
            createdAt: Date()
        )
        queuedTurns.append(queuedTurn)
        turns[turnID] = queuedTurn
        await eventLog.append(causal: causal, payload: .runQueued(runID: RunID()))
        return SubmitTurnDecision(turn: queuedTurn, status: .queued, runID: nil, shouldStartExecution: false)
    } else {
        let runID = RunID()
        activeRootRunID = runID
    }
}

```

```

    let run = RunSnapshot(
        runID: runID,
        sessionID: sessionID,
        turnID: turnID,
        rootRunID: runID,
        status: .running,
        model: intent.modelSelection ?? "default",
        createdAt: Date()
    )
    runs[runID] = run

    let runningTurn = TurnSnapshot(
        turnID: turnID,
        sessionID: sessionID,
        userMessage: userMessage,
        executionIntent: intent,
        status: .running,
        rootRunID: runID,
        createdAt: Date()
    )
    turns[turnID] = runningTurn

    let runCausal = CausalContext(sessionID: sessionID, turnID: turnID, runID: runID, rootRunID: runID)
    await eventLog.append(causal: runCausal, payload: .runCreated(run))
    await eventLog.append(causal: runCausal, payload: .runStarted(runID: runID))

    return SubmitTurnDecision(turn: runningTurn, status: .running, runID: runID, shouldStartExecution: true)
}

public func cancelTurn(turnID: TurnID) async throws {
    guard let index = queuedTurns.firstIndex(where: { $0.turnID == turnID }) else {
        if let turn = turns[turnID], turn.status == .running {
            throw RuntimeError(category: .validation, code: "turnAlreadyRunning", message: "Turn 正在运行, 请使用 cancelRun 取消执行",
↵ retryability: .none, source: .client)
        }
        throw RuntimeError(category: .validation, code: "turnNotFound", message: "未找到处于排队状态的 Turn \(turnID.rawValue)",
↵ retryability: .none, source: .client)
    }
    let queued = queuedTurns.remove(at: index)
    let cancelledTurn = TurnSnapshot(
        turnID: turnID,
        sessionID: sessionID,
        userMessage: queued.userMessage,
        executionIntent: queued.executionIntent,
        status: .cancelled,
        rootRunID: nil,
        createdAt: queued.createdAt,
        completedAt: Date()
    )
    turns[turnID] = cancelledTurn
    let causal = CausalContext(sessionID: sessionID, turnID: turnID)
    await eventLog.append(causal: causal, payload: .turnCompleted(turnID: turnID, terminalReason: .userCancelled))
}

public func rollbackTurn(decision: SubmitTurnDecision) async {
    rollbackTurnID(decision.turn.turnID, runID: decision.runID)
}

public func rollbackTurnID(_ turnID: TurnID, runID: RunID?) {
    turns.removeValue(forKey: turnID)
    queuedTurns.removeAll(where: { $0.turnID == turnID })
    if let runID {
        runs.removeValue(forKey: runID)
        if activeRootRunID == runID {

```

```

        activeRootRunID = nil
    }
}

}

public struct NextTurnToRun: Sendable {
    public let turn: TurnSnapshot
    public let runID: RunID
}

public func finishRun(runID: RunID, reason: TerminalReason, error: RuntimeError? = nil) async -> NextTurnToRun? {
    guard let run = runs[runID] else { return nil }
    let turnID = run.turnID
    let causal = CausalContext(sessionID: sessionID, turnID: turnID, runID: runID, rootRunID: run.rootRunID)

    if let error {
        let failedRun = RunSnapshot(
            runID: runID,
            sessionID: sessionID,
            turnID: turnID,
            rootRunID: run.rootRunID,
            status: .failed,
            model: run.model,
            createdAt: run.createdAt,
            completedAt: Date(),
            terminalReason: reason
        )
        runs[runID] = failedRun
        if let turn = turns[turnID] {
            turns[turnID] = TurnSnapshot(
                turnID: turnID,
                sessionID: sessionID,
                userMessage: turn.userMessage,
                executionIntent: turn.executionIntent,
                status: .failed,
                rootRunID: run.rootRunID ?? runID,
                createdAt: turn.createdAt,
                completedAt: Date()
            )
        }
        await eventLog.append(causal: causal, payload: .runFailed(runID: runID, error: error))
        await eventLog.append(causal: causal, payload: .turnFailed(turnID: turnID, error: error))
    } else {
        let completedRun = RunSnapshot(
            runID: runID,
            sessionID: sessionID,
            turnID: turnID,
            rootRunID: run.rootRunID,
            status: .completed,
            model: run.model,
            createdAt: run.createdAt,
            completedAt: Date(),
            terminalReason: reason
        )
        runs[runID] = completedRun
        if let turn = turns[turnID] {
            turns[turnID] = TurnSnapshot(
                turnID: turnID,
                sessionID: sessionID,
                userMessage: turn.userMessage,
                executionIntent: turn.executionIntent,
                status: .completed,
                rootRunID: run.rootRunID ?? runID,
                createdAt: turn.createdAt,
                completedAt: Date()
            )
        }
    }
}

```



```

        )
    }
    await eventLog.append(causal: causal, payload: .runCompleted(runID: runID, terminalReason: reason))
    await eventLog.append(causal: causal, payload: .turnCompleted(turnID: turnID, terminalReason: reason))
}

if activeRootRunID == runID {
    activeRootRunID = nil
}

return await scheduleNextQueuedTurn()
}

public func cancelRun(runID: RunID, reason: String? = nil) async throws -> NextTurnToRun? {
    guard let run = runs[runID] else {
        throw RuntimeError(category: .validation, code: "runNotFound", message: "Run \(runID.rawValue) 不存在", retryability: .none,
↵ source: .client)
    }
    let causal = CausalContext(sessionID: sessionID, turnID: run.turnID, runID: runID, rootRunID: run.rootRunID)
    let cancelledRun = RunSnapshot(
        runID: runID,
        sessionID: sessionID,
        turnID: run.turnID,
        rootRunID: run.rootRunID,
        status: .cancelled,
        model: run.model,
        createdAt: run.createdAt,
        completedAt: Date(),
        terminalReason: .userCancelled
    )
    runs[runID] = cancelledRun
    if let turn = turns[run.turnID] {
        turns[run.turnID] = TurnSnapshot(
            turnID: run.turnID,
            sessionID: sessionID,
            userMessage: turn.userMessage,
            executionIntent: turn.executionIntent,
            status: .cancelled,
            rootRunID: run.rootRunID ?? runID,
            createdAt: turn.createdAt,
            completedAt: Date()
        )
    }
    await eventLog.append(causal: causal, payload: .runCancelled(runID: runID, reason: reason ?? "userCancelled"))
    await eventLog.append(causal: causal, payload: .turnCompleted(turnID: run.turnID, terminalReason: .userCancelled))

    if activeRootRunID == runID {
        activeRootRunID = nil
    }

    return await scheduleNextQueuedTurn()
}

private func scheduleNextQueuedTurn() async -> NextTurnToRun? {
    // Schedule next queued Turn if any
    if !queuedTurns.isEmpty {
        let next = queuedTurns.removeFirst()
        let nextRunID = RunID()
        activeRootRunID = nextRunID

        let nextRun = RunSnapshot(
            runID: nextRunID,
            sessionID: sessionID,
            turnID: next.turnID,
            rootRunID: nextRunID,

```

```

        status: .running,
        model: next.executionIntent.modelSelection ?? "default",
        createdAt: Date()
    )
    runs[nextRunID] = nextRun

    let runningTurn = TurnSnapshot(
        turnID: next.turnID,
        sessionID: sessionID,
        userMessage: next.userMessage,
        executionIntent: next.executionIntent,
        status: .running,
        rootRunID: nextRunID,
        createdAt: next.createdAt
    )
    turns[next.turnID] = runningTurn

    let nextCausal = CausalContext(sessionID: sessionID, turnID: next.turnID, runID: nextRunID, rootRunID: nextRunID)
    await eventLog.append(causal: nextCausal, payload: .runCreated(nextRun))
    await eventLog.append(causal: nextCausal, payload: .runStarted(runID: nextRunID))

    return NextTurnToRun(turn: runningTurn, runID: nextRunID)
}

return nil
}

public func currentAgentMode() -> AgentMode {
    if let activeID = activeRootRunID, let run = runs[activeID], let turn = turns[run.turnID] {
        return turn.executionIntent.mode
    }
    return turns.values.sorted(by: { $0.createdAt < $1.createdAt }).last?.executionIntent.mode ?? .build
}

private var streamTerminalIndices: [StreamID: UInt64] = [:]
private var knownStreamIDs: Set<StreamID> = []

// Assistant streaming 在首 frame 前必须已有稳定 MessageID
public func beginAssistantStream(stepID: ModelStepID, runID: RunID) async -> (messageID: MessageID, streamID: StreamID) {
    let messageID = MessageID()
    let streamID = StreamID()
    knownStreamIDs.insert(streamID)
    let run = runs[runID]
    let causal = CausalContext(sessionID: sessionID, turnID: run?.turnID, runID: runID, rootRunID: run?.rootRunID, modelStepID: stepID)

    // Assistant streaming 在首 frame 前必须已有稳定 MessageID
    await eventLog.append(causal: causal, payload: .assistantMessageStarted(messageID: messageID, assistantStreamID: streamID))
    return (messageID, streamID)
}

@discardableResult
public func emitStreamFrame(frame: StreamFrame) throws -> StreamFrame {
    knownStreamIDs.insert(frame.streamID)
    if let terminal = streamTerminalIndices[frame.streamID] {
        if frame.index > terminal {
            throw RuntimeError(category: .runtime, code: "streamAlreadyTerminated", message: "Stream \("\(frame.streamID.rawValue) 已在
↩ finalIndex \(\terminal) 结束", retryability: .none, source: .core)
        }
    }
}

streamFrames[frame.streamID, default: []].append(frame)
if let continuations = streamSubscribers[frame.streamID] {
    for continuation in continuations.values {
        continuation.yield(frame)
    }
}

```

```

    }
    return frame
}

public func closeStream(streamID: StreamID, finalIndex: UInt64) {
    streamTerminalIndices[streamID] = finalIndex
    if let continuations = streamSubscribers.removeValue(forKey: streamID) {
        for continuation in continuations.values {
            continuation.finish()
        }
    }
}

public func hasStream(_ streamID: StreamID) -> Bool {
    knownStreamIDs.contains(streamID) || streamFrames[streamID] != nil || modelSteps.values.contains { $0.visibleReasoningStreamID ==
    ↪ streamID || $0.assistantStreamID == streamID }
}

public func commitAssistantMessage(
    messageID: MessageID,
    streamID: StreamID? = nil,
    causal: CausalContext,
    content: String,
    finalIndex: UInt64
) async {
    if let streamID {
        closeStream(streamID: streamID, finalIndex: finalIndex)
    }
    await eventLog.append(causal: causal, payload: .assistantMessageCommitted(
        messageID: messageID,
        content: content,
        assistantFinalIndex: finalIndex
    ))
}

// MARK: - ModelStep Streaming & Barrier

public func beginModelStep(
    stepID: ModelStepID,
    runID: RunID,
    stepNumber: Int
) async -> (messageID: MessageID, reasoningStreamID: StreamID, assistantStreamID: StreamID) {
    let messageID = MessageID()
    let reasoningStreamID = StreamID()
    let assistantStreamID = StreamID()
    knownStreamIDs.insert(reasoningStreamID)
    knownStreamIDs.insert(assistantStreamID)
    let run = runs[runID]
    let causal = CausalContext(sessionID: sessionID, turnID: run?.turnID, runID: runID, rootRunID: run?.rootRunID, modelStepID: stepID)

    let snapshot = ModelStepSnapshot(
        stepID: stepID,
        runID: runID,
        stepNumber: stepNumber,
        status: "running",
        visibleReasoningStreamID: reasoningStreamID,
        assistantStreamID: assistantStreamID,
        startedAt: Date()
    )
    modelSteps[stepID] = snapshot

    await eventLog.append(causal: causal, payload: .modelStepStarted(
        stepID: stepID,
        visibleReasoningStreamID: reasoningStreamID,
        assistantStreamID: assistantStreamID
    ))
}

```

```

    ))
    await eventLog.append(causal: causal, payload: .assistantMessageStarted(messageID: messageID, assistantStreamID: assistantStreamID))
    return (messageID, reasoningStreamID, assistantStreamID)
}

public func completeModelStep(
    stepID: ModelStepID,
    causal: CausalContext,
    finalIndex: UInt64?,
    outputMetadata: ModelStepOutputMetadata?
) async {
    if let existing = modelSteps[stepID] {
        if let finalIndex, let streamID = existing.visibleReasoningStreamID {
            closeStream(streamID: streamID, finalIndex: finalIndex)
        }
        modelSteps[stepID] = ModelStepSnapshot(
            stepID: stepID,
            runID: existing.runID,
            stepNumber: existing.stepNumber,
            status: "completed",
            visibleReasoningStreamID: existing.visibleReasoningStreamID,
            assistantStreamID: existing.assistantStreamID,
            startedAt: existing.startedAt,
            completedAt: Date()
        )
    }
    await eventLog.append(causal: causal, payload: .modelStepCompleted(
        stepID: stepID,
        visibleReasoningFinalIndex: finalIndex,
        outputMetadata: outputMetadata
    ))
}

// MARK: - Tool Invocations & Barrier

private var toolStreams: [ToolCallID: (stdout: StreamID?, stderr: StreamID?)] = [:]

public func recordToolRequested(snapshot: ToolInvocationSnapshot, causal: CausalContext) async {
    toolInvocations[snapshot.callID] = snapshot
    await eventLog.append(causal: causal, payload: .toolRequested(snapshot))
}

public func recordToolScheduled(callID: ToolCallID, causal: CausalContext) async {
    if let inv = toolInvocations[callID] {
        toolInvocations[callID] = ToolInvocationSnapshot(
            callID: inv.callID,
            toolID: inv.toolID,
            displayName: inv.displayName,
            argumentsSummary: inv.argumentsSummary,
            state: .scheduled,
            resultPreview: inv.resultPreview,
            resultRef: inv.resultRef,
            durationMs: inv.durationMs,
            error: inv.error
        )
    }
    await eventLog.append(causal: causal, payload: .toolScheduled(callID: callID))
}

public func recordToolWaitingForPermission(callID: ToolCallID, permissionID: PermissionID, causal: CausalContext) async {
    if let inv = toolInvocations[callID] {
        toolInvocations[callID] = ToolInvocationSnapshot(
            callID: callID,
            toolID: inv.toolID,
            displayName: inv.displayName,

```

```

        argumentsSummary: inv.argumentsSummary,
        state: .waitingForPermission,
        resultPreview: inv.resultPreview,
        resultRef: inv.resultRef,
        durationMs: inv.durationMs,
        error: inv.error
    )
}
await eventLog.append(causal: causal, payload: .toolWaitingForPermission(callID: callID, permissionID: permissionID))
}

public func recordToolRunning(callID: ToolCallID, stdoutStreamID: StreamID?, stderrStreamID: StreamID?, causal: CausalContext) async {
    if let stdoutStreamID { knownStreamIDs.insert(stdoutStreamID) }
    if let stderrStreamID { knownStreamIDs.insert(stderrStreamID) }
    toolStreams[callID] = (stdout: stdoutStreamID, stderr: stderrStreamID)
    if let inv = toolInvocations[callID] {
        toolInvocations[callID] = ToolInvocationSnapshot(
            callID: callID,
            toolID: inv.toolID,
            displayName: inv.displayName,
            argumentsSummary: inv.argumentsSummary,
            state: .running,
            resultPreview: inv.resultPreview,
            resultRef: inv.resultRef,
            durationMs: inv.durationMs,
            error: inv.error
        )
    }
    await eventLog.append(causal: causal, payload: .toolRunning(callID: callID, stdoutStreamID: stdoutStreamID, stderrStreamID:
↔ stderrStreamID))
}

public func recordToolCompleted(
    callID: ToolCallID,
    result: ToolResultSnapshot,
    stdoutFinalIndex: UInt64?,
    stderrFinalIndex: UInt64?,
    causal: CausalContext
) async {
    if let streams = toolStreams.removeValue(forKey: callID) {
        if let stdoutFinalIndex, let stdout = streams.stdout {
            closeStream(streamID: stdout, finalIndex: stdoutFinalIndex)
        }
        if let stderrFinalIndex, let stderr = streams.stderr {
            closeStream(streamID: stderr, finalIndex: stderrFinalIndex)
        }
    }
    if let inv = toolInvocations[callID] {
        toolInvocations[callID] = ToolInvocationSnapshot(
            callID: callID,
            toolID: inv.toolID,
            displayName: inv.displayName,
            argumentsSummary: inv.argumentsSummary,
            state: .completed,
            resultPreview: result.preview,
            resultRef: result.contentRef,
            durationMs: result.timing.executionMilliseconds > 0 ? result.timing.executionMilliseconds : inv.durationMs,
            error: result.error
        )
    }
    await eventLog.append(causal: causal, payload: .toolCompleted(
        callID: callID,
        result: result,
        stdoutFinalIndex: stdoutFinalIndex,
        stderrFinalIndex: stderrFinalIndex
    ))
}

```

```

    ))
}

public func recordProviderRequestState(
    requestID: ProviderRequestID,
    state: ProviderRequestState,
    detail: String? = nil,
    statusCode: Int? = nil,
    causal: CausalContext
) async {
    await eventLog.append(causal: causal, payload: .providerRequestStateChanged(
        requestID: requestID,
        state: state,
        detail: detail,
        statusCode: statusCode
    ))
}

public func recordContextStateChanged(_ snapshot: ContextStateSnapshot, causal: CausalContext) async {
    await eventLog.append(causal: causal, payload: .contextStateChanged(snapshot))
}

// MARK: - Interactions

public func recordInteractionRequested(snapshot: InteractionSnapshot) async {
    interactions[snapshot.interactionID] = snapshot
    await eventLog.append(causal: snapshot.causal, payload: .interactionRequested(snapshot))
}

public func resolveInteraction(interactionID: InteractionID, resolution: InteractionResolution) async throws {
    guard let interaction = interactions[interactionID] else {
        throw RuntimeError(category: .permission, code: "interactionNotFound", message: "Interaction \("\(interactionID.rawValue) 不存在",
        ↪ retryability: .none, source: .client)
    }
    interactions.removeValue(forKey: interactionID)
    await eventLog.append(causal: interaction.causal, payload: .interactionResolved(interactionID: interactionID, resolution: resolution
    ↪ ))
}

public func listPendingInteractions() -> [InteractionSnapshot] {
    Array(interactions.values)
}

// MARK: - Stream Subscription

public func subscribeStream(streamID: StreamID, afterIndex: UInt64?) -> AsyncStream<StreamFrame> {
    let key = UUID()
    let cached = streamFrames[streamID] ?? []
    let replay = cached.filter { frame in
        if let afterIndex { return frame.index > afterIndex }
        return true
    }
    let terminalIndex = streamTerminalIndices[streamID]

    return AsyncStream { continuation in
        for frame in replay {
            continuation.yield(frame)
        }
        if terminalIndex != nil {
            // Once the stream has terminated, no future frames can ever be emitted.
            // Replaying available frames and finishing immediately prevents deadlocks.
            continuation.finish()
            return
        }
    }
    self.streamSubscribers[streamID, default: [:]][key] = continuation
}

```

```

        continuation.onTermination = { [weak self] _ in
            Task { [weak self] in
                await self?.removeStreamSubscriber(streamID: streamID, key: key)
            }
        }
    }
}

private func removeStreamSubscriber(streamID: StreamID, key: UUID) {
    streamSubscribers[streamID]?.removeValue(forKey: key)
}

// MARK: - Snapshot 组装

public func buildSnapshot(
    info: SessionSummary,
    contextState: ContextStateSnapshot,
    permissionConfiguration: PermissionConfiguration,
    agentMode: AgentMode,
    revision: UInt64
) async -> SessionSnapshot {
    let cursor = await eventLog.currentCursor()
    let recentEvents = await eventLog.recentEvents(count: 2000)
    let activeRun = activeRootRunID.flatMap { runs[$0] }

    return SessionSnapshot(
        sessionID: sessionID,
        info: info,
        recentTurns: Array(turns.values),
        activeRootRun: activeRun,
        activeChildRuns: [],
        pendingInteractions: Array(interactions.values),
        activeModelSteps: Array(modelSteps.values),
        recentToolInvocations: Array(toolInvocations.values),
        contextState: contextState,
        permissionConfiguration: permissionConfiguration,
        agentMode: agentMode,
        recentEvents: recentEvents,
        historyBeforeCursor: nil,
        eventCursor: cursor,
        revision: revision
    )
}

public func getTurn(turnID: TurnID) -> TurnSnapshot? {
    turns[turnID]
}

public func listTurns(page: PageRequest) -> Page<TurnSnapshot> {
    let all = Array(turns.values).sorted(by: { $0.createdAt < $1.createdAt })
    let items = Array(all.prefix(page.limit))
    return Page(items: items, nextCursor: nil, hasMore: all.count > items.count)
}

public func getRun(runID: RunID) -> RunSnapshot? {
    runs[runID]
}

public func listRuns(page: PageRequest) -> Page<RunSnapshot> {
    let all = Array(runs.values).sorted(by: { $0.createdAt < $1.createdAt })
    let items = Array(all.prefix(page.limit))
    return Page(items: items, nextCursor: nil, hasMore: all.count > items.count)
}
}

```

// File: Sources/LingXiCore/Modules/Session/SessionStore.swift

```

import Foundation
import LingXiProtocol

/// Session 存储契约。
/// 当前实现: InMemorySessionStore; 未来 SQLite 到来时替换实现,
/// AgentRuntime / SessionRuntime / TUI 的领域逻辑不变。
public protocol SessionStore: Actor, Sendable {
    func create(kind: SessionKind, parentSessionID: SessionID?, rootSessionID: SessionID?, spawnedByRunID: AgentRunID?, spawnedByToolCallID:
        ↳ ToolCallID?, title: String?) async throws -> Session
    func session(_ id: SessionID) async throws -> Session
    func listSessions() async throws -> [Session]
    func updateTitle(_ id: SessionID, title: String?) async throws -> Session
    func updateReasoningEffort(_ id: SessionID, effort: ReasoningEffort) async throws -> Session
    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, content: String) async throws -> Message
    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, parts: [SessionMessagePart]) async throws -> Message
    @discardableResult
    func appendMessage(_ sessionID: SessionID, message: Message) async throws -> Message
    @discardableResult
    func appendMessage(_ sessionID: SessionID, message: Message, expectedRevision: UInt64?) async throws -> Message
    @discardableResult
    func bumpRevision(_ sessionID: SessionID) async throws -> UInt64
    func currentRevision(_ sessionID: SessionID) async throws -> UInt64
    func deleteSession(_ id: SessionID) async throws
    @discardableResult
    func revertLastTurn(_ sessionID: SessionID, bumpRevision: Bool) async throws -> (revertedPrompt: String?, removedCount: Int)
}

public extension SessionStore {
    @discardableResult
    func revertLastTurn(_ sessionID: SessionID) async throws -> (revertedPrompt: String?, removedCount: Int) {
        try await revertLastTurn(sessionID, bumpRevision: true)
    }

    func create() async throws -> Session {
        try await create(kind: .primary, parentSessionID: nil, rootSessionID: nil, spawnedByRunID: nil, spawnedByToolCallID: nil, title: nil
            ↳ )
    }

    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, content: String) async throws -> Message {
        try await appendMessage(sessionID, role: role, content: content, expectedRevision: nil)
    }

    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, content: String, expectedRevision: UInt64?) async throws -> Message {
        try await appendMessage(sessionID, role: role, parts: [.text(content)], expectedRevision: expectedRevision)
    }

    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, parts: [SessionMessagePart]) async throws -> Message {
        try await appendMessage(sessionID, role: role, parts: parts, expectedRevision: nil)
    }

    @discardableResult
    func appendMessage(_ sessionID: SessionID, role: MessageRole, parts: [SessionMessagePart], expectedRevision: UInt64?) async throws ->
        ↳ Message {
        let message = Message(id: MessageID(UUID().uuidString), role: role, parts: parts, createdAt: Date())
        return try await appendMessage(sessionID, message: message, expectedRevision: expectedRevision)
    }
}

/// 内存实现: actor 保证并发安全。

```



```

public actor InMemorySessionStore: SessionStore {
    private var sessions: [SessionID: Session] = [:]
    private var order: [SessionID] = []

    public init() {}

    public func create(kind: SessionKind = .primary, parentSessionID: SessionID? = nil, rootSessionID: SessionID? = nil, spawnedByRunID:
        ↳ AgentRunID? = nil, spawnedByToolCallID: ToolCallID? = nil, title: String? = nil) async throws -> Session {
        guard parentSessionID == nil || sessions[parentSessionID!] != nil else { throw CoreError(code: .sessionNotFound, message: "Parent
        ↳ Session 不存在") }
        let id = SessionID(UUID().uuidString)
        let session = Session(id: id, createdAt: Date(), kind: kind, parentSessionID: parentSessionID, rootSessionID: rootSessionID ??
        ↳ parentSessionID.flatMap { sessions[$0]?.rootSessionID } ?? id, spawnedByRunID: spawnedByRunID, spawnedByToolCallID:
        ↳ spawnedByToolCallID, title: title)
        sessions[session.id] = session
        order.append(session.id)
        return session
    }

    public func session(_ id: SessionID) async throws -> Session {
        guard let session = sessions[id] else {
            throw CoreError(code: .sessionNotFound, message: "Session 不存在: \$(id.rawValue)")
        }
        return session
    }

    public func listSessions() async throws -> [Session] {
        order.compactMap { sessions[$0] }
    }

    public func updateTitle(_ id: SessionID, title: String?) async throws -> Session {
        guard let current = sessions[id] else { throw CoreError(code: .sessionNotFound, message: "Session 不存在: \$(id.rawValue)") }
        let updated = Session(id: current.id, createdAt: current.createdAt, kind: current.kind, parentSessionID: current.parentSessionID,
        ↳ rootSessionID: current.rootSessionID, spawnedByRunID: current.spawnedByRunID, spawnedByToolCallID: current.spawnedByToolCallID,
        ↳ title: title, reasoningEffort: current.reasoningEffort, projectID: current.projectID, cwdRootBindingID: current.cwdRootBindingID,
        ↳ cwdRelativePath: current.cwdRelativePath, updatedAt: current.updatedAt, messages: current.messages)
        sessions[id] = updated
        return updated
    }

    public func updateReasoningEffort(_ id: SessionID, effort: ReasoningEffort) async throws -> Session {
        guard let current = sessions[id] else { throw CoreError(code: .sessionNotFound, message: "Session 不存在: \$(id.rawValue)") }
        var updated = current
        updated.setReasoningEffort(effort)
        sessions[id] = updated
        return updated
    }

    @discardableResult
    public func appendMessage(_ sessionID: SessionID, message: Message) async throws -> Message {
        try await appendMessage(sessionID, message: message, expectedRevision: nil)
    }

    @discardableResult
    public func appendMessage(_ sessionID: SessionID, message: Message, expectedRevision: UInt64?) async throws -> Message {
        guard var session = sessions[sessionID] else {
            throw CoreError(code: .sessionNotFound, message: "Session 不存在: \$(sessionID.rawValue)")
        }
        if let expected = expectedRevision {
            guard session.revision == expected else {
                throw StaleRunError(sessionID: sessionID, expected: session.revision, actual: expected)
            }
        }
        session.append(message)
        sessions[sessionID] = session
    }

```

```

        return message
    }

@discardableResult
public fun bumpRevision(_ sessionId: SessionID) async throws -> UInt64 {
    guard var session = sessions[sessionId] else {
        throw CoreError(code: .sessionNotFound, message: "Session 不存在: \(sessionId.rawValue)")
    }
    let rev = session.bumpRevision()
    sessions[sessionId] = session
    return rev
}

public fun currentRevision(_ sessionId: SessionID) async throws -> UInt64 {
    guard let session = sessions[sessionId] else {
        throw CoreError(code: .sessionNotFound, message: "Session 不存在: \(sessionId.rawValue)")
    }
    return session.revision
}

public fun deleteSession(_ id: SessionID) async throws {
    sessions.removeValue(forKey: id)
    order.removeAll { $0 == id }
}

@discardableResult
public fun revertLastTurn(_ sessionId: SessionID, bumpRevision: Bool) async throws -> (revertedPrompt: String?, removedCount: Int) {
    guard var session = sessions[sessionId] else {
        throw CoreError(code: .sessionNotFound, message: "Session 不存在: \(sessionId.rawValue)")
    }
    let outcome = session.revertLastTurn(bumpRevision: bumpRevision)
    sessions[sessionId] = session
    return outcome
}
}

/// 每 project 一个 state.sqlite 的 canonical Session repository。
public actor PersistentSessionStore: SessionStore {
    private let persistence: SQLitePersistenceStore
    private var reasoningEfforts: [SessionID: ReasoningEffort] = [:]

    public init(persistence: SQLitePersistenceStore) { self.persistence = persistence }

    public fun create(kind: SessionKind = .primary, parentSessionID: SessionID? = nil, rootSessionID: SessionID? = nil, spawnedByRunID:
        ⇨ AgentRunID? = nil, spawnedByToolCallID: ToolCallID? = nil, title: String? = nil) async throws -> Session {
        let main = try await persistence.mainRootBinding()
        let parentRoot: SessionID?
        if let parentSessionID {
            parentRoot = try await session(parentSessionID).rootSessionID
        } else {
            parentRoot = nil
        }
        let id = SessionID(UUID().uuidString)
        let session = Session(
            id: id,
            createdAt: .now,
            kind: kind,
            parentSessionID: parentSessionID,
            rootSessionID: rootSessionID ?? parentRoot ?? id,
            spawnedByRunID: spawnedByRunID,
            spawnedByToolCallID: spawnedByToolCallID,
            title: title,
            projectID: persistence.projectID,
            cwdRootBindingID: main.id
        )
    }
}

```

```

    try await persistence.createSession(session)
    return session
}

public func session(_ id: SessionID) async throws -> Session {
    if var session = try await persistence.loadSessions().first(where: { $0.id == id }) {
        if let effort = reasoningEfforts[id] {
            session.setReasoningEffort(effort)
        }
        return session
    }
    if var global = try await persistence.loadGlobalSession(id) {
        if let effort = reasoningEfforts[id] {
            global.setReasoningEffort(effort)
        }
        return global
    }
    throw CoreError(code: .sessionNotFound, message: "Session 不存在: \$(id.rawValue)")
}

public func listSessions() async throws -> [Session] {
    var list = try await persistence.loadSessions()
    for i in list.indices {
        if let effort = reasoningEfforts[list[i].id] {
            list[i].setReasoningEffort(effort)
        }
    }
    return list
}

public func updateTitle(_ id: SessionID, title: String?) async throws -> Session {
    let current = try await session(id)
    try await persistence.updateSessionTitle(id, title: title)
    return Session(id: current.id, createdAt: current.createdAt, kind: current.kind, parentSessionID: current.parentSessionID,
        ↪ rootSessionID: current.rootSessionID, spawnedByRunID: current.spawnedByRunID, spawnedByToolCallID: current.spawnedByToolCallID,
        ↪ title: title, reasoningEffort: current.reasoningEffort, projectID: current.projectID, cwdRootBindingID: current.cwdRootBindingID,
        ↪ cwdRelativePath: current.cwdRelativePath, updatedAt: .now, revision: current.revision, messages: current.messages)
}

public func updateReasoningEffort(_ id: SessionID, effort: ReasoningEffort) async throws -> Session {
    reasoningEfforts[id] = effort
    var current = try await session(id)
    current.setReasoningEffort(effort)
    return current
}

@discardableResult
public func appendMessage(_ sessionID: SessionID, message: Message) async throws -> Message {
    try await appendMessage(sessionID, message: message, expectedRevision: nil)
}

@discardableResult
public func appendMessage(_ sessionID: SessionID, message: Message, expectedRevision: UInt64?) async throws -> Message {
    _ = try await session(sessionID)
    try await persistence.appendMessage(sessionID: sessionID, message: message, expectedRevision: expectedRevision)
    return message
}

@discardableResult
public func bumpRevision(_ sessionID: SessionID) async throws -> UInt64 {
    _ = try await session(sessionID)
    return try await persistence.bumpRevision(sessionID: sessionID)
}

public func currentRevision(_ sessionID: SessionID) async throws -> UInt64 {

```

```

        _ = try await session(sessionID)
        return try await persistence.currentRevision(sessionID: sessionID)
    }

    public func deleteSession(_ id: SessionID) async throws {
        try await persistence.deleteSession(id)
    }

    @discardableResult
    public func revertLastTurn(_ sessionID: SessionID, bumpRevision: Bool) async throws -> (revertedPrompt: String?, removedCount: Int) {
        _ = try await session(sessionID)
        return try await persistence.revertLastTurn(sessionID: sessionID, bumpRevision: bumpRevision)
    }
}

```

// File: Sources/LingXiCore/Modules/Session/SessionDomain.swift

```

import Foundation
import LingXiProtocol

// LingXi Session Domain: 产品/会话领域的消息。
// 刻意不与 Model.Message 复用——Session Message 表达会话语义，
// Model.Message 表达一次模型 inference 的输入，二者未来必然分化。
// 转换边界：SessionContextBuilder。

public enum MessageRole: String, Sendable {
    case user
    case assistant
    case tool
}

public struct Message: Sendable, Equatable {
    public let id: MessageID
    public let role: MessageRole
    public let parts: [SessionMessagePart]
    public let createdAt: Date

    public var content: String {
        parts.compactMap { if case let .text(text) = $0 { text } else { nil } }.joined()
    }

    public init(id: MessageID, role: MessageRole, content: String, createdAt: Date) {
        self.init(id: id, role: role, parts: [.text(content)], createdAt: createdAt)
    }

    public init(id: MessageID, role: MessageRole, parts: [SessionMessagePart], createdAt: Date) {
        self.id = id
        self.role = role
        self.parts = parts
        self.createdAt = createdAt
    }
}

public struct Session: Sendable, Equatable {
    public let id: SessionID
    public let kind: SessionKind
    public let parentSessionID: SessionID?
    public let rootSessionID: SessionID
    public let spawnedByRunID: AgentRunID?
    public let spawnedByToolCallID: ToolCallID?
    public let title: String?
    public var reasoningEffort: ReasoningEffort
    /// Durable cwd 只保存 binding 和相对路径，绝不复制 absolute path。
    public let projectID: ProjectID?
    public let cwdRootBindingID: RootBindingID?
}

```

```

public let cwdRelativePath: ProjectRelativePath
public let createdAt: Date
public private(set) var updatedAt: Date
public private(set) var revision: UInt64
public private(set) var messages: [Message]

public init(
    id: SessionID,
    createdAt: Date,
    kind: SessionKind = .primary,
    parentSessionID: SessionID? = nil,
    rootSessionID: SessionID? = nil,
    spawnedByRunID: AgentRunID? = nil,
    spawnedByToolCallID: ToolCallID? = nil,
    title: String? = nil,
    reasoningEffort: ReasoningEffort = .auto,
    projectID: ProjectID? = nil,
    cwdRootBindingID: RootBindingID? = nil,
    cwdRelativePath: ProjectRelativePath = .root,
    updatedAt: Date? = nil,
    revision: UInt64 = 0,
    messages: [Message] = []
) {
    self.id = id
    self.kind = kind
    self.parentSessionID = parentSessionID
    self.rootSessionID = rootSessionID ?? id
    self.spawnedByRunID = spawnedByRunID
    self.spawnedByToolCallID = spawnedByToolCallID
    self.title = title
    self.reasoningEffort = reasoningEffort
    self.projectID = projectID
    self.cwdRootBindingID = cwdRootBindingID
    self.cwdRelativePath = cwdRelativePath
    self.createdAt = createdAt
    self.updatedAt = updatedAt ?? createdAt
    self.revision = revision
    self.messages = messages
}

public mutating func append(_ message: Message) {
    messages.append(message)
    updatedAt = message.createdAt
}

@discardableResult
public mutating func bumpRevision() -> UInt64 {
    revision += 1
    updatedAt = Date()
    return revision
}

public mutating func revertLastTurn(bumpRevision: Bool = true) -> (revertedPrompt: String?, removedCount: Int) {
    guard let lastUserIdx = messages.lastIndex(where: { $0.role == .user }) else {
        return (nil, 0)
    }
    let userMsg = messages[lastUserIdx]
    let prompt = userMsg.content
    let removed = messages.count - lastUserIdx
    messages.removeSubrange(lastUserIdx...)
    if bumpRevision {
        revision += 1
    }
    updatedAt = Date()
    return (prompt, removed)
}

```

```

    }

    public mutating func setReasoningEffort(_ effort: ReasoningEffort) {
        self.reasoningEffort = effort
        self.updatedAt = Date()
    }
}

// MARK: - 协议层 DTO 转换 (Session Domain → Protocol DTO)

extension Session {
    public func toInfo() -> SessionInfo {
        SessionInfo(id: id, projectID: projectID, kind: kind, parentSessionID: parentSessionID, rootSessionID: rootSessionID, spawnedByRunID:
        ↩ spawnedByRunID, spawnedByToolCallID: spawnedByToolCallID, title: title, reasoningEffort: reasoningEffort, createdAt: createdAt,
        ↩ updatedAt: updatedAt, messageCount: messages.count)
    }

    public func toSnapshot() -> LegacySessionSnapshot {
        LegacySessionSnapshot(
            id: id, projectID: projectID, kind: kind, parentSessionID: parentSessionID, rootSessionID: rootSessionID, spawnedByRunID:
            ↩ spawnedByRunID, spawnedByToolCallID: spawnedByToolCallID, title: title,
            reasoningEffort: reasoningEffort,
            createdAt: createdAt,
            updatedAt: updatedAt,
            messages: messages.map(\.toSnapshot)
        )
    }
}

extension Message {
    public var toSnapshot: SessionMessageSnapshot {
        SessionMessageSnapshot(
            id: id,
            role: SessionMessageRole(rawValue: role.rawValue) ?? .user,
            parts: parts,
            createdAt: createdAt
        )
    }
}

```

// File: Sources/LingXiCore/Modules/Session/RunLease.swift

```

import Foundation
import LingXiProtocol

/// 统一运行凭证，用于在 Turn 的整个生命周期内标记并保护时间线版本。
public struct RunLease: Sendable, Equatable {
    public let sessionID: SessionID
    public let turnID: TurnID
    public let runID: RunID?
    public let revision: UInt64

    public init(sessionID: SessionID, turnID: TurnID, runID: RunID? = nil, revision: UInt64) {
        self.sessionID = sessionID
        self.turnID = turnID
        self.runID = runID
        self.revision = revision
    }
}

/// 当尝试向已经过期/已被撤回的 Session 版本写入时抛出的标准阻断错误。
public struct StaleRunError: Error, Sendable, Equatable, LocalizedError {
    public let sessionID: SessionID
    public let expected: UInt64
    public let actual: UInt64

```

```

    public init(sessionID: SessionID, expected: UInt64, actual: UInt64) {
        self.sessionID = sessionID
        self.expected = expected
        self.actual = actual
    }

    public var errorDescription: String? {
        "Session \(sessionID.rawValue) revision mismatch: expected \(expected), actual \(actual) (stale run aborted)"
    }
}

/// 统一的 Session Revision 校验守卫契约。
public protocol SessionRevisionGuard: Sendable {
    func validate(_ lease: RunLease) async throws
    func currentRevision(_ sessionID: SessionID) async throws -> UInt64
}

public extension SessionRevisionGuard {
    func validate(_ lease: RunLease) async throws {
        let current = try await currentRevision(lease.sessionID)
        guard current == lease.revision else {
            throw StaleRunError(
                sessionID: lease.sessionID,
                expected: current,
                actual: lease.revision
            )
        }
    }
}

/// 基于 SessionStore 的标准 RevisionGuard 实现。
public struct StoreSessionRevisionGuard: SessionRevisionGuard {
    private let store: any SessionStore

    public init(store: any SessionStore) {
        self.store = store
    }

    public func currentRevision(_ sessionID: SessionID) async throws -> UInt64 {
        try await store.currentRevision(sessionID)
    }
}

```

// File: Sources/LingXiCore/Modules/Session/SessionMutationLock.swift

```

import Foundation
import LingXiProtocol

/// Session-level mutation lock ensuring operations like submitTurn and revertLastTurn
/// on the same session execute in strict mutual exclusion.
public actor SessionMutationLock {
    public static let shared = SessionMutationLock()

    private var activeSessions: Set<SessionID> = []
    private var waiters: [SessionID: [CheckedContinuation<Void, Never>]] = [:]

    public init() {}

    public func withExclusiveMutation<T: Sendable>(_ sessionID: SessionID, _ operation: () async throws -> T) async throws -> T {
        while activeSessions.contains(sessionID) {
            await withCheckedContinuation { continuation in
                waiters[sessionID, default: []].append(continuation)
            }
        }
    }
}

```

```

        activeSessions.insert(sessionID)
    defer {
        activeSessions.remove(sessionID)
        if let next = waiters[sessionID]?.first {
            waiters[sessionID]?.removeFirst()
            if waiters[sessionID]?.isEmpty == true {
                waiters.removeValue(forKey: sessionID)
            }
            next.resume()
        }
    }
}
return try await operation()
}
}

```

// File: Sources/LingXiCore/Modules/Tool/ToolRuntime.swift

```

import CoreFoundation
import Foundation
import LingXiProtocol

public protocol ToolExecutor: Sendable {
    var definition: ToolDefinition { get }
    func resource(for arguments: String, profile: ExecutionProfile) throws -> String
    func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind>
    /// 外部目录审批必须引用 canonical path; Shell 的主要权限资源仍是命令文本。
    func externalResource(for arguments: String, profile: ExecutionProfile) throws -> String?
    func execute(arguments: String, profile: ExecutionProfile) async throws -> String
}

public extension ToolExecutor {
    func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> { definition.capability.kinds }
    func externalResource(for arguments: String, profile: ExecutionProfile) throws -> String? { nil }
}

public protocol ToolProvider: Sendable {
    func register(into registry: inout ToolRegistry) throws
}

public struct ToolExecutionObserver: Sendable {
    public let permissionAsked: @Sendable (PermissionRequest) async -> Void
    public let permissionResolved: @Sendable (PermissionRequest, PermissionReply) async -> Void
    public let executionClaimed: @Sendable (ToolExecutionClaim) async -> Void
    public let questionAsked: @Sendable (QuestionRequest) async -> Void
    public let questionResolved: @Sendable (QuestionRequest, QuestionReply) async -> Void

    public init(permissionAsked: @escaping @Sendable (PermissionRequest) async -> Void, permissionResolved: @escaping @Sendable (
        ↪ PermissionRequest, PermissionReply) async -> Void, executionClaimed: @escaping @Sendable (ToolExecutionClaim) async -> Void,
        ↪ questionAsked: @escaping @Sendable (QuestionRequest) async -> Void, questionResolved: @escaping @Sendable (QuestionRequest,
        ↪ QuestionReply) async -> Void) {
        self.permissionAsked = permissionAsked
        self.permissionResolved = permissionResolved
        self.executionClaimed = executionClaimed
        self.questionAsked = questionAsked
        self.questionResolved = questionResolved
    }
}

package enum ToolExecutionContext {
    @TaskLocal static var observer: ToolExecutionObserver?
    @TaskLocal static var sessionID: SessionID?
    @TaskLocal static var toolCallID: ToolCallID?
    @TaskLocal static var lifecycleTrace: ToolLifecycleTrace?
}

```



```

public struct BuiltInToolProvider: ToolProvider {
    public let tools: [any ToolExecutor]
    public init(tools: [any ToolExecutor]) { self.tools = tools }
    public func register(into registry: inout ToolRegistry) throws {
        for tool in tools { try registry.register(tool) }
    }
}

public enum ToolRegistryError: Error, Sendable, Equatable {
    case duplicateName(String)
}

/// 静态注册表。本阶段无动态插件或运行时注册。
public struct ToolRegistry: Sendable {
    private var tools: [ToolID: any ToolExecutor]

    public init(_ tools: [any ToolExecutor]) {
        self.tools = [:]
        for tool in tools {
            precondition(self.tools[tool.definition.id] == nil, "duplicate Tool: \(tool.definition.name)")
            self.tools[tool.definition.id] = tool
        }
    }

    public init(validating tools: [any ToolExecutor]) throws {
        self.tools = [:]
        for tool in tools { try register(tool) }
    }

    public mutating func register(_ tool: any ToolExecutor) throws {
        guard tools[tool.definition.id] == nil else { throw ToolRegistryError.duplicateName(tool.definition.name) }
        tools[tool.definition.id] = tool
    }

    public var definitions: [ToolDefinition] {
        tools.values.map(\.definition).sorted { $0.id.rawValue < $1.id.rawValue }
    }

    public func tool(for id: ToolID) -> (any ToolExecutor)? {
        if let direct = tools[id] {
            return direct
        }
        switch id.rawValue {
        case "write_to_file":
            return tools[ToolID("write_file")]
        case "replace_file_content":
            return tools[ToolID("edit_file")]
        case "patch_file":
            return tools[ToolID("apply_patch")]
        case "read_file", "view_file":
            return tools[ToolID("read_file")] ?? tools[ToolID("read")]
        default:
            return nil
        }
    }

    public func tool(named name: String) -> (any ToolExecutor)? {
        tool(for: ToolID(name))
    }
}

/// 会话级动态工具租借管理器：负责按需发现、租借与会话隔离。
public actor DynamicToolLeaseManager {
    private var leasedToolsBySession: [SessionID: Set<ToolID>] = [:]
    private var leasedToolsByRun: [AgentRunID: Set<ToolID>] = [:]

```

```

private var discoveredCandidatesBySession: [SessionID: Set<ToolID>] = [:]

public init() {}

public func lease(sessionID: SessionID, runID: AgentRunID?, toolID: ToolID) {
    if let runID { leasedToolsByRun[runID, default: []].insert(toolID) }
    else { leasedToolsBySession[sessionID, default: []].insert(toolID) }
}

public func unlease(sessionID: SessionID, toolID: ToolID) {
    leasedToolsBySession[sessionID]?.remove(toolID)
}

public func leasedTools(for sessionID: SessionID, runID: AgentRunID? = nil) -> Set<ToolID> {
    (leasedToolsBySession[sessionID] ?? []).union(runID.flatMap { leasedToolsByRun[$0] } ?? [])
}

public func recordCandidates(sessionID: SessionID, tools: Set<ToolID>) {
    discoveredCandidatesBySession[sessionID, default: []].formUnion(tools)
}

public func candidateTools(for sessionID: SessionID) -> Set<ToolID> {
    discoveredCandidatesBySession[sessionID] ?? []
}

public func resetSession(_ sessionID: SessionID) {
    leasedToolsBySession.removeValue(forKey: sessionID)
    discoveredCandidatesBySession.removeValue(forKey: sessionID)
}

public func resetRun(_ runID: AgentRunID) { leasedToolsByRun.removeValue(forKey: runID) }
}

/// Provider 无关的 Tool 执行入口：参数解析、路径预检、权限、执行和错误归一化。
public struct ToolRuntime: Sendable {
    public static let coreToolOrder: [ToolID] = [
        ToolID("shell"),
        ToolID("read_file"),
        ToolID("write_file"),
        ToolID("edit_file"),
        ToolID("apply_patch"),
        ToolID("list_directory"),
        ToolID("glob"),
        ToolID("grep"),
        ToolID("web_search"),
        ToolID("web_fetch"),
        ToolID("search_tools"),
        ToolID("load_tool"),
        ToolID("context_recall")
    ]

    public static let coreToolIDs: Set<ToolID> = Set(coreToolOrder)

    private let registry: ToolRegistry
    private let permissions: PermissionEngine
    private let mutations: ToolMutationCoordinator
    private let outputPolicy: ToolOutputPolicy
    private let outputArchive: ToolOutputArchive?
    private let outputSink: (@Sendable (ToolOutputChunk) async -> Void)?
    private let mcpPager: MCPToolPager?
    private let subagents: SubagentToolService?
    private let cacheController: ContextCacheController?
    private let deadlinePolicy: ExecutionDeadlinePolicy
    private let dynamicLeases: DynamicToolLeaseManager

    public init(

```

```

registry: ToolRegistry,
permissions: PermissionEngine,
mutations: ToolMutationCoordinator = ToolMutationCoordinator(),
outputPolicy: ToolOutputPolicy = ToolOutputPolicy(),
outputArchive: ToolOutputArchive? = nil,
outputSink: (@Sendable (ToolOutputChunk) async -> Void)? = nil,
mcpPager: MCPToolPager? = nil,
subagents: SubagentToolService? = nil,
cacheController: ContextCacheController? = nil,
deadlinePolicy: ExecutionDeadlinePolicy = ExecutionDeadlinePolicy(),
dynamicLeases: DynamicToolLeaseManager = DynamicToolLeaseManager()
) {
    self.registry = registry
    self.permissions = permissions
    self.mutations = mutations
    self.outputPolicy = outputPolicy
    self.outputArchive = outputArchive
    self.outputSink = outputSink
    self.mcpPager = mcpPager
    self.subagents = subagents
    self.cacheController = cacheController
    self.deadlinePolicy = deadlinePolicy
    self.dynamicLeases = dynamicLeases
}

public var definitions: [ToolDefinition] { registry.definitions }

public func resetSession(_ sessionID: SessionID) async {
    await dynamicLeases.resetSession(sessionID)
    await mcpPager?.discardSession(sessionID)
}

public func resetRun(_ runID: AgentRunID) async { await dynamicLeases.resetRun(runID) }

public func availableDefinitions(sessionID: SessionID? = nil, runID: AgentRunID? = nil, interactive: Bool = false, executionProfile:
↳ SubagentExecutionProfile? = nil) async -> [ToolDefinition] {
    let configuration = await permissions.currentConfiguration()
    let profile = Self.attenuatedProfile(requested: executionProfile?.permissionProfile.flatMap(ExecutionProfile.init(rawValue:)),
↳ parent: configuration.profile)
    let effectiveSessionID = sessionID ?? SessionID("ephemeral")
    let leasedIDs = await dynamicLeases.leasedTools(for: effectiveSessionID, runID: runID)

    var definitions = registry.definitions.filter { definition in
        if profile == .readOnly && !definition.capability.readOnly {
            return false
        }
        if let allowed = executionProfile?.toolProfile.map(Set.init) {
            return allowed.contains(definition.id.rawValue)
        }
        if definition.name == "question" {
            return interactive
        }
        if definition.id == ToolID("skill") {
            let values = definition.inputSchema.properties["name"]?.enumValues ?? []
            return !values.isEmpty
        }
        // Always-on core tools + dynamically leased tools
        return Self.coreToolIDs.contains(definition.id) || leasedIDs.contains(definition.id)
    }
    definitions += [MCPDiscoveryTools.search, MCPDiscoveryTools.load]
    if subagents != nil {
        let subagentDef = SubagentTool.definition
        if let allowed = executionProfile?.toolProfile.map(Set.init) {
            if allowed.contains(subagentDef.id.rawValue) { definitions.append(subagentDef) }
        } else if leasedIDs.contains(subagentDef.id) {

```

```

        definitions.append(subagentDef)
    }
}
if let sessionID, let mcpPager { definitions += await mcpPager.providerDefinitions(sessionID: sessionID) }
if let cacheController {
    let contextSearchDef = ContextRetrieveTool(id: "context_search", cacheController: cacheController).definition
    if let allowed = executionProfile?.toolProfile.map(Set.init) {
        if allowed.contains(contextSearchDef.id.rawValue) { definitions.append(contextSearchDef) }
    } else if leasedIDs.contains(contextSearchDef.id) {
        definitions.append(contextSearchDef)
    }
}
}
if let allowed = executionProfile?.toolProfile.map(Set.init) {
    definitions = definitions.filter { allowed.contains($0.id.rawValue) }
}
if profile == .readOnly { definitions = definitions.filter(\.capability.readOnly) }
if profile == .fullAccess {
    definitions = definitions.map { definition in
        ToolDefinition(
            id: definition.id,
            name: definition.name,
            description: "\((definition.description) Full Access: absolute paths and ordinary paths outside the workspace are allowed
↩ ; sensitive paths remain blocked.",
            inputSchema: definition.inputSchema,
            capability: definition.capability,
            rawInputSchema: definition.rawInputSchema
        )
    }
}
}
return definitions.sorted { lhs, rhs in
    let left = Self.coreToolOrder.firstIndex(of: lhs.id)
    let right = Self.coreToolOrder.firstIndex(of: rhs.id)
    switch (left, right) {
    case let (l?, r?): return l == r ? lhs.id.rawValue < rhs.id.rawValue : l < r
    case (_, nil): return true
    case (nil, _?): return false
    default: return lhs.id.rawValue < rhs.id.rawValue
    }
}
}
}

public struct ExecutionOutcome: Sendable {
    public let result: ToolResult
    public let permissionWait: Duration
    public let permissionAsked: Bool
    public let execution: Duration
    public let queueDuration: Duration
    public let executionDuration: Duration
    public let toolName: String
    public let resource: String?
    var lifecycleTrace: ToolLifecycleTrace?

    public init(result: ToolResult, permissionWait: Duration, permissionAsked: Bool, execution: Duration, queueDuration: Duration = .
↩ zero, toolName: String, resource: String?) {
        self.permissionWait = permissionWait
        self.permissionAsked = permissionAsked
        self.execution = execution
        self.queueDuration = queueDuration
        self.executionDuration = execution
        self.toolName = toolName
        self.resource = resource
        self.lifecycleTrace = nil
        let execMs = ToolRuntime.durationMilliseconds(execution)
        let queueMs = ToolRuntime.durationMilliseconds(queueDuration)
        let permMs = ToolRuntime.durationMilliseconds(permissionWait)

```

```

        self.result = result.withTiming(ToolTiming(
            milliseconds: execMs,
            queueMilliseconds: queueMs,
            executionMilliseconds: execMs,
            permissionMilliseconds: permMs
        ))
    }
}

private static func durationMilliseconds(_ duration: Duration) -> Double {
    let components = duration.components
    return Double(components.seconds) * 1_000 + Double(components.attoseconds) / 1_000_000_000_000_000
}

public struct ReadOnlySignature: Sendable, Equatable, Hashable {
    public let toolName: String
    public let canonicalArguments: String
    public let resource: String
}

public func readOnlySignature(for call: ToolCall) throws -> ReadOnlySignature? {
    guard let tool = registry.tool(for: call.toolID), tool.definition.capability.readOnly else { return nil }
    return ReadOnlySignature(
        toolName: tool.definition.name,
        canonicalArguments: Self.canonicalArguments(call.arguments),
        resource: try tool.resource(for: call.arguments, profile: .workspace)
    )
}

public func execute(
    _ call: ToolCall,
    sessionID: SessionID,
    projectID: ProjectID = ProjectID("ephemeral"),
    executionProfile: SubagentExecutionProfile? = nil,
    onPermissionAsked: (@Sendable (PermissionRequest) async -> Void)? = nil
) async -> ToolResult {
    await executeWithMetrics(call, sessionID: sessionID, projectID: projectID, executionProfile: executionProfile, onPermissionAsked:
↳ onPermissionAsked ?? { _ in }).result
}

public func executeWithMetrics(
    _ call: ToolCall,
    sessionID: SessionID,
    projectID: ProjectID = ProjectID("ephemeral"),
    executionProfile: SubagentExecutionProfile? = nil,
    onPermissionAsked: @escaping @Sendable (PermissionRequest) async -> Void,
    observer: ToolExecutionObserver? = nil,
    parentDeadline: ExecutionDeadline? = nil
) async -> ExecutionOutcome {
    let deadline = deadlinePolicy.deadline(for: category(for: call), requested: executionProfile?.timeoutSeconds.map { .seconds($0) },
↳ parent: parentDeadline)
    let lifecycle = ToolLifecycleTrace(toolCallID: call.callID)
    lifecycle.record(.requested)
    return await ToolExecutionContext.$lifecycleTrace.withValue(lifecycle) {
        await ToolExecutionContext.$toolCallID.withValue(call.callID) {
            do {
                var outcome = try await executeWithMetricsUnbounded(call, sessionID: sessionID, projectID: projectID, executionProfile:
↳ executionProfile, onPermissionAsked: onPermissionAsked, observer: observer, deadline: deadline)
                lifecycle.record(.toolResultBuilt, processPID: lifecycle.snapshot().last?.processPID, exitCode: outcome.result.exitCode.
↳ map { Int32($0) })
                outcome.lifecycleTrace = lifecycle
                return outcome
            } catch let error as CoreError {
                let outcome: ToolOutcome = error.code == .idleTimedOut ? .idleTimedOut : error.code == .toolCancelled || error.code == .
↳ permissionCancelled ? .cancelled : .timedOut

```

```

        let unknown = !isReadOnly(call)
        var metadata = ["deadlineCategory": deadline.category.rawValue, "effectiveTimeoutSeconds": String(format: "%.3f",
↪ deadline.timeoutSeconds)]
        if unknown { metadata["executionState"] = "unknown"; metadata["verificationRequired"] = "true" }
        let code = unknown && outcome != .cancelled ? CoreError.Code.executionStateUnknown.rawValue : error.code.rawValue
        var result = ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: "", error: ToolError(code
↪ : code, message: error.message), toolName: call.toolName, outcome: outcome, metadata: metadata), permissionWait: .zero,
↪ permissionAsked: false, execution: deadline.timeout, toolName: call.toolName, resource: nil)
        lifecycle.record(.toolResultBuilt, processPID: lifecycle.snapshot().last?.processPID, exitCode: result.result.exitCode.
↪ map { Int32($0) })
        result.lifecycleTrace = lifecycle
        return result
    } catch is CancellationError {
        var result = ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: "", error: ToolError(code
↪ : CoreError.Code.toolCancelled.rawValue, message: "Tool 执行已取消"), toolName: call.toolName, outcome: .cancelled), permissionWait:
↪ .zero, permissionAsked: false, execution: deadline.timeout, toolName: call.toolName, resource: nil)
        lifecycle.record(.toolResultBuilt)
        result.lifecycleTrace = lifecycle
        return result
    } catch {
        var result = ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: "", error: ToolError(code
↪ : CoreError.Code.toolExecutionFailed.rawValue, message: "Tool 执行失败"), toolName: call.toolName), permissionWait: .zero,
↪ permissionAsked: false, execution: deadline.timeout, toolName: call.toolName, resource: nil)
        lifecycle.record(.toolResultBuilt)
        result.lifecycleTrace = lifecycle
        return result
    }
}
}

private func executeWithMetricsUnbounded(
    _ call: ToolCall,
    sessionID: SessionID,
    projectID: ProjectID,
    executionProfile: SubagentExecutionProfile?,
    onPermissionAsked: @escaping @Sendable (PermissionRequest) async -> Void,
    observer: ToolExecutionObserver?,
    deadline: ExecutionDeadline
) async throws -> ExecutionOutcome {
    let clock = ContinuousClock()
    let lifecycleTrace = ToolExecutionContext.lifecycleTrace
    let admissionStart = clock.now
    var permissionWait: Duration = .zero
    var permissionAsked = false
    var execution: Duration = .zero
    var queueDuration: Duration = .zero
    var executionStartedAt: ContinuousClock.Instant?
    do {
        try Task.checkCancellation()
        if let allowed = executionProfile?.toolProfile, !allowed.contains(call.toolID.rawValue) {
            throw CoreError(code: .permissionDenied, message: "Execution Profile 不允许 \(call.toolID.rawValue)")
        }
        let baseConfiguration = await permissions.currentConfiguration()
        let effectiveProfile = Self.attenuatedProfile(requested: executionProfile?.permissionProfile.flatMap(ExecutionProfile.init(
↪ rawValue:)), parent: baseConfiguration.profile)
        if call.toolID == SubagentTool.definition.id, let subagents {
            try ToolSchemaValidator.validate(arguments: call.arguments, schema: SubagentTool.definition.inputSchema)
            let request = PermissionRequest(
                permissionID: PermissionID(UUID().uuidString),
                sessionID: sessionID,
                toolCallID: call.callID,
                toolID: call.toolID,
                capabilities: SubagentTool.definition.capability.kinds,
                resource: "child Agent session",

```

```

        description: "允许创建或控制 Child Agent"
    )
    let permissionStart = clock.now
    lifecycleTrace?.record(.permissionStart)
    let resolution = await permissions.resolve(request) {
        await observer?.permissionAsked(request)
        await onPermissionAsked(request)
    }
    permissionWait = permissionStart.duration(to: clock.now)
    lifecycleTrace?.record(.permissionEnd)
    permissionAsked = resolution.asked
    try Task.checkCancellation()
    await observer?.permissionResolved(request, PermissionReply(permissionID: request.permissionID, decision: resolution.
↪ decision))

    guard resolution.decision == .allow else { throw CoreError(code: .permissionDenied, message: "已拒绝 subagent") }
    try Task.checkCancellation()
    lifecycleTrace?.record(.admitted)
    let executionStart = clock.now
    executionStartedAt = executionStart
    queueDuration = admissionStart.duration(to: executionStart)
    await observer?.executionClaimed(ToolExecutionClaim(mutatesProject: true))
    lifecycleTrace?.record(.executorStart)
    let content = try await ExecutionWatchdog.run(freshExecutionDeadline(from: deadline)) {
        try await ToolExecutionContext.$observer.withValue(observer) {
            try await subagents.execute(arguments: call.arguments, sessionID: sessionID, callID: call.callID)
        }
    }
    execution = executionStart.duration(to: clock.now)
    return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: content, toolName: call.toolID.
↪ rawValue), permissionWait: permissionWait, permissionAsked: permissionAsked, execution: execution, toolName: call.toolID.rawValue,
↪ resource: request.resource)
    }

    if call.toolID.rawValue.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
        throw CoreError(code: .toolNotFound, message: "Tool call name 为空")
    }

    if call.toolID == MCPDiscoveryTools.search.id {
        struct SearchInput: Decodable {
            let query: String?
            let server: String?
            let capability: String?
            let maxResults: Int?
        }
        let decoder = JSONDecoder()
        decoder.keyDecodingStrategy = .convertFromSnakeCase
        let input = (try? decoder.decode(SearchInput.self, from: Data(call.arguments.utf8)))
            ?? SearchInput(query: call.arguments, server: nil, capability: nil, maxResults: nil)
        let query = (input.query ?? "").trimmingCharacters(in: .whitespacesAndNewlines)
        let terms = query.lowercased().split(whereSeparator: { !$0.isLetter && !$0.isNumber }).map(String.init)

        var candidateEntries: [[String: String]] = []
        var discoveredIDs: Set<ToolID> = []

        // 1. Search specialized builtin tools (if not filtering by external server)
        if input.server == nil || input.server?.isEmpty == true {
            var specialized: [ToolDefinition] = registry.definitions.filter { !Self.coreToolIDs.contains($0.id) && $0.id != ToolID("
↪ question") && $0.id != ToolID("skill") }
            if subagents != nil { specialized.append(SubagentTool.definition) }
            if let cacheController { specialized.append(ContextRetrieveTool(id: "context_search", cacheController: cacheController).
↪ definition) }

            for def in specialized {
                let text = "\(def.id.rawValue) \(def.name) \(def.description)".lowercased()
                let matches = terms.isEmpty || terms.contains(where: { text.contains($0) })
                if matches {
                    discoveredIDs.insert(def.id)
                }
            }
        }
    }

```

```

        candidateEntries.append([
            "tool_id": def.id.rawValue,
            "display_name": "builtin.\(def.name)",
            "short_description": def.description,
            "category": "specialized",
            "availability": "available"
        ])
    }
}

}

await dynamicLeases.recordCandidates(sessionID: sessionID, tools: discoveredIDs)

// 2. Search MCP tools via mcpPager (if available)
var mcpResultContent: String?
if let mcpPager {
    if let result = try? await ExecutionWatchdog.run(freshExecutionDeadline(from: deadline), operation: {
        try await mcpPager.searchToolResult(sessionID: sessionID, projectID: projectID, arguments: call.arguments)
    }) {
        if result.trimmingCharacters(in: .whitespacesAndNewlines) != "[]" {
            mcpResultContent = result
            if let data = result.data(using: .utf8),
                let mcpItems = try? JSONSerialization.jsonObject(with: data) as? [[String: Any]] {
                for item in mcpItems {
                    if let toolID = item["tool_id"] as? String {
                        discoveredIDs.insert(ToolID(toolID))
                        candidateEntries.append([
                            "tool_id": toolID,
                            "display_name": (item["displayName"] as? String) ?? toolID,
                            "short_description": (item["shortDescription"] as? String) ?? "",
                            "availability": (item["availability"] as? String) ?? "available"
                        ])
                    }
                }
            }
        }
    }
}

await dynamicLeases.recordCandidates(sessionID: sessionID, tools: discoveredIDs)

if let mcpResultContent, candidateEntries.allSatisfy({ $0["category"] != "specialized" }) {
    return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: mcpResultContent, toolName: call
↪ .toolID.rawValue), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: query
↪ )
}

let encoder = JSONEncoder()
encoder.outputFormatting = [.sortedKeys]
let content = (try? String(decoding: encoder.encode(candidateEntries), as: UTF8.self)) ?? "[]"
return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: content, toolName: call.toolID.
↪ rawValue), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: query)
}

if call.toolID == MCPDiscoveryTools.load.id {
    struct LoadInput: Decodable {
        let toolId: String?
        let tool_id: String?
        let id: String?
    }

    let decoder = JSONDecoder()
    let input = try? decoder.decode(LoadInput.self, from: Data(call.arguments.utf8))
    let candidateString = input?.tool_id ?? input?.toolId ?? input?.id ?? (call.arguments.trimmingCharacters(in: .
↪ whitespacesAndNewlines).hasPrefix("{") ? nil : call.arguments.trimmingCharacters(in: .whitespacesAndNewlines))
    guard let rawToolID = candidateString?.trimmingCharacters(in: .whitespacesAndNewlines), !rawToolID.isEmpty else {
        throw CoreError(code: .toolArgumentInvalid, message: "load_tool 需要 tool_id")
    }

    let normalized = rawToolID.hasPrefix("builtin.") ? String(rawToolID.dropFirst("builtin.".count)) : rawToolID

```



```

let targetID = ToolID(normalized)

// Check if targetID is a specialized builtin tool
let isSpecialized = registry.definitions.contains(where: { $0.id == targetID })
    || (subagents != nil && (targetID == SubagentTool.definition.id || normalized == "spawn_subagent" || normalized == "
↳ subagent"))
    || (cacheController != nil && (targetID == ToolID("context_search") || normalized == "context_retrieve"))

if isSpecialized {
    await dynamicLeases.lease(sessionID: sessionID, runID: call.agentRunID, toolID: targetID)
    let encoder = JSONEncoder()
    encoder.outputFormatting = [.sortedKeys]
    let content = (try? String(decoding: encoder.encode([
        "status": "leased",
        "tool": targetID.rawValue,
        "provider_name": targetID.rawValue,
        "message": "Tool '\(targetID.rawValue)' is now leased and will be available in your subsequent steps for this
↳ session."
    ]), as: UTF8.self)) ?? #"{"status":"leased"}"#
    return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: content, toolName: call.toolID.
↳ rawValue), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: targetID.
↳ rawValue)
    }

    if let mcpPager {
        let content = try await ExecutionWatchdog.run(freshExecutionDeadline(from: deadline)) {
            try await mcpPager.loadToolResult(sessionID: sessionID, arguments: call.arguments, schemaTokenBudget: 16_000)
        }
        return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: content, toolName: call.toolID.
↳ rawValue), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: nil)
        }

        throw CoreError(code: .toolNotFound, message: "Tool '\(rawToolID)' 未找到或不可租借")
    }

    if (call.toolID.rawValue == "context_search" || call.toolID.rawValue == "context_retrieve"), let cacheController {
        var query = call.arguments
        if let data = call.arguments.data(using: .utf8),
            let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
            let q = json["query"] as? String {
            query = q
        }
        let content = try await cacheController.handleSearch(sessionID: sessionID, query: query, activeTask: "", activeFiles: [],
↳ limit: 5)
        return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: content, toolName: call.toolID.
↳ rawValue), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: query)
        }

        if registry.tool(for: call.toolID) == nil, let mcpPager {
            if await mcpPager.canHandle(sessionID: sessionID, providerToolID: call.toolID) {
                return await executeMCP(call, sessionID: sessionID, projectID: projectID, profile: effectiveProfile, pager: mcpPager,
↳ onPermissionAsked: onPermissionAsked, observer: observer, deadline: deadline)
            }
        }

        guard let tool = registry.tool(for: call.toolID) else {
            throw CoreError(code: .toolNotFound, message: "未注册 Tool: '\(call.toolID.rawValue)")
        }

        try ToolSchemaValidator.validate(arguments: call.arguments, schema: tool.definition.inputSchema, additionalProperties: Self.
↳ codingProperties(for: tool.definition.id))
        let capabilities = try tool.capabilities(for: call.arguments, profile: effectiveProfile)
        guard effectiveProfile != .readOnly || ToolCapability(capabilities).readOnly else {
            throw CoreError(code: .permissionDenied, message: "readOnly Profile 不允许 '\(call.toolID.rawValue)")
        }

        let resource = try tool.resource(for: call.arguments, profile: effectiveProfile)
        let request = PermissionRequest(
            permissionID: PermissionID(UUID().uuidString),
            sessionID: sessionID,

```

```

        toolCallID: call.callID,
        toolID: call.toolID,
        capabilities: capabilities,
        resource: resource,
        description: "允许 \$(call.toolID.rawValue) 访问 \$(resource)"
    )
    if capabilities.contains(.externalFilesystem) {
        let externalRequest = PermissionRequest(
            permissionID: PermissionID(UUID().uuidString),
            sessionID: sessionID,
            toolCallID: call.callID,
            toolID: call.toolID,
            capabilities: [.externalFilesystem],
            resource: try tool.externalResource(for: call.arguments, profile: effectiveProfile) ?? resource,
            description: "允许 \$(call.toolID.rawValue) 访问 Workspace 外目录"
        )
        lifecycleTrace?.record(.permissionStart)
        let externalResolution = await permissions.resolve(externalRequest, action: .externalDirectory) {
            await observer?.permissionAsked(externalRequest)
            await onPermissionAsked(externalRequest)
        }
        permissionAsked = permissionAsked || externalResolution.asked
        lifecycleTrace?.record(.permissionEnd)
        try Task.checkCancellation()
        await observer?.permissionResolved(externalRequest, PermissionReply(permissionID: externalRequest.permissionID, decision:
↳ externalResolution.decision))
        guard externalResolution.decision == .allow else {
            throw CoreError(code: .permissionDenied, message: "已拒绝 Workspace 外目录: \$(resource)")
        }
    }
    let permissionStart = clock.now
    lifecycleTrace?.record(.permissionStart)
    let resolution = await permissions.resolve(request, action: Self.permissionAction(for: tool.definition.id)) {
        await observer?.permissionAsked(request)
        await onPermissionAsked(request)
    }
    permissionWait = permissionStart.duration(to: clock.now)
    permissionAsked = resolution.asked
    lifecycleTrace?.record(.permissionEnd)
    try Task.checkCancellation()
    await observer?.permissionResolved(request, PermissionReply(permissionID: request.permissionID, decision: resolution.decision))
    guard resolution.decision == .allow else {
        throw CoreError(code: .permissionDenied, message: "已拒绝 \$(call.toolID.rawValue): \$(resource)")
    }
    try Task.checkCancellation()
    lifecycleTrace?.record(.admitted)
    let executionStart = clock.now
    executionStartedAt = executionStart
    queueDuration = admissionStart.duration(to: executionStart)
    let mutates = capabilities.contains(.projectWrite) || capabilities.contains(.repositoryWrite) || capabilities.contains(.
↳ destructive)
    await observer?.executionClaimed(ToolExecutionClaim(mutatesProject: mutates))
    lifecycleTrace?.record(.executorStart)
    let operation: @Sendable () async throws -> String = {
        try await ToolExecutionContext.$observer.withValue(observer) {
            try await ToolExecutionContext.$sessionID.withValue(sessionID) {
                try await tool.execute(arguments: call.arguments, profile: effectiveProfile)
            }
        }
    }
    let rawContent: String
    if call.toolID == ToolID("question") {
        rawContent = try await operation()
    } else {
        rawContent = try await ExecutionWatchdog.run(freshExecutionDeadline(from: deadline)) {

```

```

        if mutates { return try await self.mutations.execute(operation) }
        return try await operation()
    }
}

if !rawContent.isEmpty {
    await outputSink?(ToolOutputChunk(toolCallID: call.callID, stream: .stdout, sequence: 0, payload: rawContent))
}

let bounded = outputPolicy.excerpt(rawContent)
let metadata = try await outputArchive?.archive(rawContent, metadata: bounded.metadata) ?? bounded.metadata
execution = executionStart.duration(to: clock.now)
let coding = Self.codingDetails(toolID: tool.definition.id, content: rawContent)
return ExecutionOutcome(
    result: ToolResult(callID: call.callID, success: true, content: bounded.content, toolName: tool.definition.name, summary:
↳ coding.summary, metadata: coding.metadata, output: metadata, exitCode: coding.exitCode, diagnostics: coding.diagnostics,
↳ changedFiles: coding.changedFiles, continuation: metadata.outputBlobRef),
    permissionWait: permissionWait,
    permissionAsked: permissionAsked,
    execution: execution,
    queueDuration: queueDuration,
    toolName: tool.definition.name,
    resource: resource
)
} catch let error as CoreError {
    if let command = Self.commandResult(from: error.message) {
        if let executionStartedAt { execution = executionStartedAt.duration(to: clock.now) }
        let rawContent = command.stdout + (command.stdout.isEmpty || command.stderr.isEmpty ? "" : "\n") + command.stderr
        let bounded = outputPolicy.excerpt(rawContent)
        let metadata = (try? await outputArchive?.archive(rawContent, metadata: bounded.metadata)) ?? bounded.metadata
        let timedOut = error.code == .commandTimedOut
        return ExecutionOutcome(
            result: ToolResult(callID: call.callID, success: false, content: bounded.content, error: ToolError(code: timedOut ?
↳ CoreError.Code.executionStateUnknown.rawValue : error.code.rawValue, message: timedOut ? "命令超时; 执行状态需要验证" : "命令以状态
↳ \((command.exitCode) 退出"), toolName: call.toolName, outcome: timedOut ? .timedOut : .failure, summary: "command failed", metadata:
↳ timedOut ? ["executionState": "unknown", "verificationRequired": "true"] : [:], output: metadata, exitCode: Int(command.exitCode),
↳ diagnostics: ToolDiagnostics(stdout: command.stdout, stderr: command.stderr), continuation: metadata.outputBlobRef),
            permissionWait: permissionWait,
            permissionAsked: permissionAsked,
            execution: execution,
            queueDuration: queueDuration,
            toolName: call.toolName,
            resource: nil
        )
    }
}

let timedOut = error.code == .commandTimedOut || error.code == .idleTimedOut
if let executionStartedAt { execution = executionStartedAt.duration(to: clock.now) }
var metadata: [String: String] = [:]
if timedOut {
    metadata["deadlineCategory"] = deadline.category.rawValue
    metadata["effectiveTimeoutSeconds"] = String(format: "%.3f", deadline.timeoutSeconds)
}
if timedOut && !isReadOnly(call) {
    metadata["executionState"] = "unknown"
    metadata["verificationRequired"] = "true"
}
return ExecutionOutcome(
    result: ToolResult(callID: call.callID, success: false, content: "", error: ToolError(code: timedOut && !isReadOnly(call) ?
↳ CoreError.Code.executionStateUnknown.rawValue : error.code.rawValue, message: error.message), toolName: call.toolName, outcome:
↳ error.code == .permissionDenied ? .denied : timedOut ? .timedOut : .failure, metadata: metadata),
    permissionWait: permissionWait,
    permissionAsked: permissionAsked,
    execution: execution,
    queueDuration: queueDuration,
    toolName: call.toolName,
    resource: nil
)
}

```

```

    } catch let error as MCPToolPagerError {
        if let executionStartedAt { execution = executionStartedAt.duration(to: clock.now) }
        let code: CoreError.Code = switch error { case .leaseMissing, .leaseExpired: .mcpToolLeaseMissing; case .schemaChanged: .
↪ mcpToolSchemaChanged; case .schemaTooLarge: .mcpToolSchemaTooLarge; case .schemaBudgetExceeded: .mcpToolSchemaBudgetExceeded;
↪ default: .mcpServerUnavailable }
        let serverID = await mcpPager?.markToolAndServerUnavailable(providerToolID: call.toolID)
        let serverName = serverID?.rawValue ?? "MCP"
        let message = "MCP tool unavailable (\(error)).\n[System Notice: MCP Server '\(serverName)' is unreachable or failed. All tools
↪ from '\(serverName)' have been disabled for this session. Do NOT attempt to load or call any tools from '\(serverName)' again;
↪ continue using built-in tools or report the status.]"
        return ExecutionOutcome(
            result: ToolResult(callID: call.callID, success: false, content: message, error: ToolError(code: code.rawValue, message:
↪ String(describing: error)), toolName: call.toolName, outcome: .failure, summary: "Tool failed: \(code.rawValue)"),
            permissionWait: permissionWait,
            permissionAsked: permissionAsked,
            execution: execution,
            queueDuration: queueDuration,
            toolName: call.toolName,
            resource: nil
        )
    } catch is CancellationError {
        throw CancellationError()
    } catch {
        if let executionStartedAt { execution = executionStartedAt.duration(to: clock.now) }
        return ExecutionOutcome(
            result: ToolResult(callID: call.callID, success: false, content: "Tool execution failed: \(error)", error: ToolError(code:
↪ CoreError.Code.toolExecutionFailed.rawValue, message: String(describing: error)), toolName: call.toolName),
            permissionWait: permissionWait,
            permissionAsked: permissionAsked,
            execution: execution,
            queueDuration: queueDuration,
            toolName: call.toolName,
            resource: nil
        )
    }
}

private func category(for call: ToolCall) -> ExecutionTimeoutCategory {
    switch call.toolID.rawValue {
    case "read_file", "list_directory", "skill", "context_recall": return .quickFilesystem
    case "glob", "grep", "symbol_lookup", "find_references", "dependency_query", "code_intelligence": return .search
    case "shell", "git", "process": return .foregroundShell
    default: return .foregroundShell
    }
}

private func isReadOnly(_ call: ToolCall) -> Bool {
    if call.toolID == MCPDiscoveryTools.search.id || call.toolID == MCPDiscoveryTools.load.id { return true }
    return registry.tool(for: call.toolID)?.definition.capability.readOnly ?? false
}

private func freshExecutionDeadline(from deadline: ExecutionDeadline) -> ExecutionDeadline {
    ExecutionDeadline(category: deadline.category, timeout: deadline.timeout, idleTimeout: deadline.idleTimeout)
}

public func finishMCPPProviderStep(sessionID: SessionID) async { await mcpPager?.finishProviderStep(sessionID: sessionID) }
public func abortMCPTurn(sessionID: SessionID) async { await mcpPager?.abortTurn(sessionID: sessionID) }

private func executeMCP(_ call: ToolCall, sessionID: SessionID, projectID: ProjectID, profile: ExecutionProfile, pager: MCPToolPager,
↪ onPermissionAsked: @escaping @Sendable (PermissionRequest) async -> Void, observer: ToolExecutionObserver?, deadline:
↪ ExecutionDeadline) async -> ExecutionOutcome {
    let clock = ContinuousClock()
    let lifecycleTrace = ToolExecutionContext.lifecycleTrace
    do {
        let lease = try await pager.resolve(sessionID: sessionID, providerToolID: call.toolID)

```

```

let annotations = await pager.annotations(for: lease.toolID)
let isDestructive = annotations?.destructiveHint == true || (annotations?.readOnlyHint != true && (call.toolID.rawValue.contains
↳ ("delete") || call.toolID.rawValue.contains("remove") || call.toolID.rawValue.contains("drop") || call.toolID.rawValue.contains("
↳ destroy") || call.toolID.rawValue.contains("kill"))))
var capabilities: Set<ToolCapabilityKind> = [.externalService, .networkAccess]
if isDestructive {
    capabilities.insert(.destructive)
}
let request = PermissionRequest(permissionID: PermissionID(UUID().uuidString), sessionID: sessionID, toolCallID: call.callID,
↳ toolID: lease.toolID, capabilities: capabilities, resource: lease.toolID.rawValue, description: "允许外部 MCP Tool \ \(lease.toolID.
↳ rawValue)")
guard profile != .readOnly || ToolCapability(request.capabilities).readOnly else { throw CoreError(code: .permissionDenied,
↳ message: "readOnly Profile 不允许 MCP Tool") }
let permissionStarted = clock.now
lifecycleTrace?.record(.permissionStart)
let resolution = await permissions.resolve(request) {
    await observer?.permissionAsked(request)
    await onPermissionAsked(request)
}
let permissionWait = permissionStarted.duration(to: clock.now)
lifecycleTrace?.record(.permissionEnd)
try Task.checkCancellation()
await observer?.permissionResolved(request, PermissionReply(permissionID: request.permissionID, decision: resolution.decision))
guard resolution.decision == .allow else { throw CoreError(code: .permissionDenied, message: "已拒绝 \ \(lease.toolID.rawValue)")
↳ }
let executionStarted = clock.now
lifecycleTrace?.record(.admitted)
await observer?.executionClaimed(ToolExecutionClaim(mutatesProject: true))
lifecycleTrace?.record(.executorStart)
let response = try await ExecutionWatchdog.run(freshExecutionDeadline(from: deadline)) {
    try await pager.execute(sessionID: sessionID, projectID: projectID, providerToolID: call.toolID, arguments: call.arguments)
}
if !response.content.isEmpty { await outputSink?(ToolOutputChunk(toolCallID: call.callID, stream: .stdout, sequence: 0, payload:
↳ response.content)) }
let bounded = outputPolicy.excerpt(response.content)
let metadata = try await outputArchive?.archive(response.content, metadata: bounded.metadata) ?? bounded.metadata
return ExecutionOutcome(result: ToolResult(callID: call.callID, success: true, content: bounded.content, toolName: response.
↳ lease.toolID.rawValue, metadata: ["mcpToolID": response.lease.toolID.rawValue, "schemaHash": response.lease.schemaHash], output:
↳ metadata), permissionWait: permissionWait, permissionAsked: resolution.needed, execution: executionStarted.duration(to: clock.now),
↳ toolName: response.lease.toolID.rawValue, resource: response.lease.toolID.rawValue)
} catch let error as MCPToolPagerError {
    let code: CoreError.Code = switch error { case .leaseMissing, .leaseExpired: .mcpToolLeaseMissing; case .schemaChanged: .
↳ mcpToolSchemaChanged; case .schemaToolLarge: .mcpToolSchemaToolLarge; case .schemaBudgetExceeded: .mcpToolSchemaBudgetExceeded;
↳ default: .mcpServerUnavailable }
    let serverID = await pager.markToolAndServerUnavailable(providerToolID: call.toolID)
    let serverName = serverID?.rawValue ?? "MCP"
    let explanation = "Tool '\(call.toolID.rawValue)' unavailable (\(error)).\n[System Notice: MCP Server '\(serverName)' is
↳ currently offline or unreachable. All tools from '\(serverName)' are disabled for this session. Do NOT attempt to call any tools
↳ from '\(serverName)' again; continue using other tools or report the server issue.]"
    return ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: explanation, error: ToolError(code:
↳ code.rawValue, message: String(describing: error)), toolName: call.toolID.rawValue, outcome: .failure, summary: "Tool failed: \
↳ code.rawValue")), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue, resource: nil)
} catch let error as CoreError {
    let timedOut = error.code == .commandTimedOut || error.code == .idleTimedOut
    var explanation = error.message
    if error.code == .mcpServerUnavailable {
        let serverID = await pager.markToolAndServerUnavailable(providerToolID: call.toolID)
        let serverName = serverID?.rawValue ?? "MCP"
        explanation = "MCP Server '\(serverName)' unavailable: \(error.message).\n[System Notice: MCP Server '\(serverName)' is
↳ offline or disconnected. All tools from this server have been automatically disabled. Do NOT attempt to retry or call other tools
↳ from '\(serverName)' in this session; please proceed using other available tools.]"
    }
    return ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: explanation, error: ToolError(code:
↳ error.code.rawValue, message: error.message), toolName: call.toolID.rawValue, outcome: error.code == .idleTimedOut ? .idleTimedOut
↳ : timedOut ? .timedOut : .failure, summary: "Tool failed: \(error.code.rawValue)"), permissionWait: .zero, permissionAsked: false,

```

```

↪ execution: .zero, toolName: call.toolID.rawValue, resource: nil)
    } catch {
        let serverID = await pager.markToolAndServerUnavailable(providerToolID: call.toolID)
        let serverName = serverID?.rawValue ?? "MCP"
        let explanation = "Tool '\(call.toolID.rawValue)' failed: \(error).\n[System Notice: Tool failed unexpectedly. Avoid repeated
↪ calls if server '\(serverName)' is malfunctioning.]"
        return ExecutionOutcome(result: ToolResult(callID: call.callID, success: false, content: explanation, error: ToolError(code:
↪ CoreError.Code.toolExecutionFailed.rawValue, message: String(describing: error)), toolName: call.toolID.rawValue, outcome: .failure
↪ , summary: "Tool failed"), permissionWait: .zero, permissionAsked: false, execution: .zero, toolName: call.toolID.rawValue,
↪ resource: nil)
    }
}

private static func canonicalArguments(_ arguments: String) -> String {
    guard let data = arguments.data(using: .utf8),
        let object = try? JSONSerialization.jsonObject(with: data),
        let normalized = try? JSONSerialization.data(withJSONObject: object, options: [.sortedKeys])
    else { return arguments }
    return String(decoding: normalized, as: UTF8.self)
}

private static func attenuatedProfile(requested: ExecutionProfile?, parent: ExecutionProfile) -> ExecutionProfile {
    let rank: [ExecutionProfile: Int] = [.readOnly: 0, .workspace: 1, .fullAccess: 2]
    guard let requested, rank[requested, default: 0] < rank[parent, default: 0] else { return parent }
    return requested
}

private static func permissionAction(for toolID: ToolID) -> PermissionAction? {
    switch toolID.rawValue {
    case "read_file", "list_directory": return .read
    case "edit_file", "write_file", "apply_patch": return .edit
    case "shell", "process", "git": return .shell
    default: return nil
    }
}

private static func commandResult(from message: String) -> CommandResult? {
    guard let data = message.data(using: .utf8) else { return nil }
    return try? JSONDecoder().decode(CommandResult.self, from: data)
}

private static func codingDetails(toolID: ToolID, content: String) -> (summary: String, metadata: [String: String], exitCode: Int?,
↪ diagnostics: ToolDiagnostics?, changedFiles: [String]) {
    guard let data = content.data(using: .utf8), let json = try? JSONSerialization.jsonObject(with: data) else { return ("", [:], nil,
↪ nil, []) }
    if ["apply_patch", "edit_file"].contains(toolID.rawValue), let operations = json as? [[String: Any]] {
        return ("applied \(operations.count) file operation(s)", [:], nil, nil, operations.compactMap { $0["path"] as? String }.sorted()
↪ )
    }
    guard let object = json as? [String: Any] else { return ("", [:], nil, nil, []) }
    let changedFiles = object["changed_files"] as? [String] ?? []
    let summary = object["summary"] as? String ?? ""
    let exitCode = (object["exit_code"] as? NSNumber)?.intValue ?? (object["exitCode"] as? NSNumber)?.intValue
    let stdout = object["stdout"] as? String
    let stderr = object["stderr"] as? String
    let diagnostics = stdout.map { ToolDiagnostics(command: object["command"] as? String, stdout: $0, stderr: stderr ?? "") }
    return (summary, [:], exitCode, diagnostics, changedFiles)
}

private static func codingProperties(for toolID: ToolID) -> [String: ToolInputProperty] {
    switch toolID.rawValue {
    case "read_file": return ["start_line": ToolInputProperty(type: .integer, description: "", minimum: 1), "end_line":
↪ ToolInputProperty(type: .integer, description: "", minimum: 1), "max_lines": ToolInputProperty(type: .integer, description: "",
↪ minimum: 1, maximum: 2_000), "line_numbers": ToolInputProperty(type: .boolean, description: "")]
    case "glob": return ["include_hidden": ToolInputProperty(type: .boolean, description: ""), "include_ignored": ToolInputProperty(type

```

```

    ↪ : .boolean, description: ""), "include_generated": ToolInputProperty(type: .boolean, description: "")]
    case "grep": return ["glob": ToolInputProperty(type: .string, description: ""), "include_hidden": ToolInputProperty(type: .boolean,
    ↪ description: ""), "include_ignored": ToolInputProperty(type: .boolean, description: ""), "include_generated": ToolInputProperty(
    ↪ type: .boolean, description: "")]
    default: return [:]
  }
}
}

/// JSON object is decoded once at the runtime boundary before permission or side effects.
enum ToolSchemaValidator {
  static func validate(arguments: String, schema: ToolInputSchema, additionalProperties: [String: ToolInputProperty] = [:]) throws {
    guard let data = arguments.data(using: .utf8),
      let object = try? JSONSerialization.jsonObject(with: data),
      let values = object as? [String: Any]
    else { throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数必须是 JSON object") }
    var allowedProperties = schema.properties
    allowedProperties.merge(additionalProperties) { _, replacement in replacement }
    if schema.properties["action"] != nil && schema.properties["task"] != nil {
      allowedProperties["permission_profile"] = ToolInputProperty(type: .string, description: "Optional permission profile")
      allowedProperties["budget_profile"] = ToolInputProperty(type: .string, description: "Optional budget profile")
      allowedProperties["context_profile"] = ToolInputProperty(type: .string, description: "Optional context profile")
      allowedProperties["max_steps"] = ToolInputProperty(type: .integer, description: "Optional maximum steps", minimum: 1)
      allowedProperties["timeout_seconds"] = ToolInputProperty(type: .integer, description: "Optional timeout in seconds", minimum: 1)
    }
    guard Set(values.keys).isSubset(of: Set(allowedProperties.keys)) else {
      throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数包含未知字段")
    }
    for name in schema.required where values[name] == nil {
      throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数缺少必填字段: \(name)")
    }
    for (name, value) in values {
      guard let property = allowedProperties[name], matches(value, property.type) else {
        throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数类型无效: \(name)")
      }
      if let values = property.enumValues, let value = value as? String, !values.contains(value) {
        throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数枚举无效: \(name)")
      }
      if let number = value as? NSNumber, property.type == .integer || property.type == .number {
        if let minimum = property.minimum, number.doubleValue < minimum { throw CoreError(code: .toolArgumentInvalid, message: "Tool
    ↪ 参数小于最小值: \(name)") }
        if let maximum = property.maximum, number.doubleValue > maximum { throw CoreError(code: .toolArgumentInvalid, message: "Tool
    ↪ 参数大于最大值: \(name)") }
      }
    }
  }

  private static func matches(_ value: Any, _ type: ToolInputType) -> Bool {
    switch type {
    case .string: return value is String
    case .boolean: return value is Bool
    case .integer:
      guard let number = value as? NSNumber else { return false }
      return CFGetTypeID(number) != CFBooleanGetTypeID() && floor(number.doubleValue) == number.doubleValue
    case .number:
      guard let number = value as? NSNumber else { return false }
      return CFGetTypeID(number) != CFBooleanGetTypeID()
    case .object: return value is [String: Any]
    case .array: return value is [Any]
    }
  }
}

```

// File: Sources/LingXiCore/Modules/Tool/BuiltinTools.swift

```

import CryptoKit
import Foundation
import LingXiProtocol

public struct WorkspaceRoot: Sendable {
    public let url: URL
    public let sensitivePathPolicy: SensitivePathPolicy

    public init(path: String, sensitivePathPolicy: SensitivePathPolicy? = nil) throws {
        let candidate = URL(fileURLWithPath: path).standardizedFileURL.resolvingSymlinksInPath()
        var isDirectory: ObjCBool = false
        guard FileManager.default.fileExists(atPath: candidate.path, isDirectory: &isDirectory), isDirectory.boolValue else {
            throw CoreError(code: .workspaceViolation, message: "Workspace Root 不存在或不是目录: \(candidate.path)")
        }
        url = candidate
        self.sensitivePathPolicy = sensitivePathPolicy ?? SensitivePathPolicy(root: candidate)
    }

    public func resolve(_ path: String, profile: ExecutionProfile = .workspace) throws -> URL {
        var cleanPath = path.trimmingCharacters(in: .whitespacesAndNewlines)
        if (cleanPath.hasPrefix("\") && cleanPath.hasSuffix("\")) || (cleanPath.hasPrefix("") && cleanPath.hasSuffix("")), cleanPath.
        ↪ count >= 2 {
            cleanPath = String(cleanPath.dropFirst().dropLast())
        }
        let home = FileManager.default.homeDirectoryForCurrentUser.path
        let expandedPath: String
        if cleanPath == "~" || cleanPath == "$HOME" || cleanPath == "${HOME}" {
            expandedPath = home
        } else if cleanPath.hasPrefix("~/") {
            expandedPath = home + "/" + String(cleanPath.dropFirst(2))
        } else if cleanPath.hasPrefix("$HOME/") {
            expandedPath = home + "/" + String(cleanPath.dropFirst(6))
        } else if cleanPath.hasPrefix("${HOME}/") {
            expandedPath = home + "/" + String(cleanPath.dropFirst(8))
        } else {
            expandedPath = cleanPath
        }
        let input = URL(fileURLWithPath: expandedPath, relativeTo: expandedPath.hasPrefix("/") ? nil : url)
        let candidate = input.standardizedFileURL.resolvingSymlinksInPath()
        let root = url.path.hasSuffix("/") ? url.path : url.path + "/"
        guard profile == .fullAccess || candidate.path == url.path || candidate.path.hasPrefix(root) else {
            throw CoreError(code: .workspaceViolation, message: "AccessScope=workspace 禁止访问 Workspace 外路径; 请先切换到 FullAccess/YOLO
            ↪ ")
        }
        guard !sensitivePathPolicy.isSensitive(candidate) else {
            throw CoreError(code: .workspaceViolation, message: "不允许访问敏感路径")
        }
        return candidate
    }
}

private func decodeArguments<T: Decodable>(_ arguments: String, as type: T.Type = T.self) throws -> T {
    guard let data = arguments.data(using: .utf8) else {
        throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数不是 UTF-8")
    }
    do {
        let decoder = JSONDecoder()
        decoder.keyDecodingStrategy = .convertFromSnakeCase
        return try decoder.decode(T.self, from: data)
    } catch {
        throw CoreError(code: .toolArgumentInvalid, message: "Tool 参数无效: \(error.localizedDescription)")
    }
}

private func json<T: Encodable>(_ value: T) throws -> String {

```



```

    let encoder = JSONEncoder()
    encoder.outputFormatting = [.sortedKeys, .withoutEscapingSlashes]
    return String(decoding: try encoder.encode(value), as: UTF8.self)
}

private func relativePath(_ url: URL, workspace: WorkspaceRoot) -> String {
    let root = workspace.url.path.hasSuffix("/") ? workspace.url.path : workspace.url.path + "/"
    return url.path.hasPrefix(root) ? String(url.path.dropFirst(root.count)) : url.path
}

private func filesystemCapabilities(_ url: URL, workspace: WorkspaceRoot, write: Bool) -> Set<ToolCapabilityKind> {
    var capabilities: Set<ToolCapabilityKind> = [write ? .projectWrite : .projectRead]
    let root = workspace.url.path.hasSuffix("/") ? workspace.url.path : workspace.url.path + "/"
    if url.path != workspace.url.path && !url.path.hasPrefix(root) {
        capabilities.insert(.externalFilesystem)
    }
    return capabilities
}

private func readText(_ file: URL, operation: String) throws -> String {
    var isDirectory: ObjCBool = false
    guard FileManager.default.fileExists(atPath: file.path, isDirectory: &isDirectory) else {
        throw CoreError(code: .toolExecutionFailed, message: "文件不存在: \(file.path)")
    }
    guard !isDirectory.boolValue else {
        throw CoreError(code: .toolExecutionFailed, message: "\(operation) 不能读取目录: \(file.path)")
    }
    let size = (try FileManager.default.attributesOfItem(atPath: file.path)[.size] as? NSNumber)?.intValue ?? 0
    guard size <= ReadFileTool.maximumBytes else {
        throw CoreError(code: .toolExecutionFailed, message: "文件超过 \(ReadFileTool.maximumBytes) bytes 限制: \(file.path)")
    }
    guard let content = String(data: try Data(contentsOf: file), encoding: .utf8) else {
        throw CoreError(code: .toolExecutionFailed, message: "文件不是 UTF-8 文本: \(file.path)")
    }
    return content
}

private func writableFile(_ file: URL) throws {
    var isDirectory: ObjCBool = false
    guard FileManager.default.fileExists(atPath: file.deletingLastPathComponent().path, isDirectory: &isDirectory), isDirectory.boolValue
    ↪ else {
        throw CoreError(code: .toolExecutionFailed, message: "父目录不存在: \(file.deletingLastPathComponent().path)")
    }
    if FileManager.default.fileExists(atPath: file.path, isDirectory: &isDirectory), isDirectory.boolValue {
        throw CoreError(code: .toolExecutionFailed, message: "不能写入目录: \(file.path)")
    }
}

private func writeText(_ content: String, to file: URL) throws {
    try writableFile(file)
    try Data(content.utf8).write(to: file, options: .atomic)
}

private func fileVersion(_ file: URL) throws -> String {
    let attributes = try FileManager.default.attributesOfItem(atPath: file.path)
    let bytes = (attributes[.size] as? NSNumber)?.intValue ?? 0
    let modified = (attributes[.modificationDate] as? Date)?.timeIntervalSinceReferenceDate ?? 0
    return "stat:\(bytes):\\(modified)"
}

private func checkExpectedContent(_ content: String?, hash: String?, version: String?, currentVersion: String? = nil, overwrite: Bool?)
    ↪ throws {
    guard overwrite != true else { return }
    guard hash != nil || version != nil else {
        throw CoreError(code: .contentChanged, message: "修改现有文件需要 expected_hash 或 expected_version")
    }
}

```

```

    }
    let actual = sha256Hex(content ?? "")
    let normalizedHash = hash?.lowercased().replacingOccurrences(of: "sha256:", with: "")
    let versionMatches = version.map { $0 == (currentVersion ?? actual) } ?? true
    guard (normalizedHash == nil || normalizedHash == actual) && versionMatches else {
        throw CoreError(code: .contentChanged, message: "文件内容已变更; expected_hash 或 expected_version 不匹配")
    }
}

private func fileWriteResult(for file: URL, workspace: WorkspaceRoot, content: String) -> FileWriteResult {
    let hash = sha256Hex(content)
    return FileWriteResult(path: relativePath(file, workspace: workspace), bytes: content.lengthOfBytes(using: .utf8), hash: hash, version:
        ↪ (try? fileVersion(file)) ?? hash)
}

private struct PathArguments: Decodable { let path: String }
private struct ReadArguments: Decodable {
    let path: String
    let startLine: Int?
    let endLine: Int?
    let maxLines: Int?
    let lineNumbers: Bool?

    enum CodingKeys: String, CodingKey {
        case path
        case startLine = "start_line"
        case startLineCamel = "startLine"
        case endLine = "end_line"
        case endLineCamel = "endLine"
        case maxLines = "max_lines"
        case maxLinesCamel = "maxLines"
        case lineNumbers = "line_numbers"
        case lineNumbersCamel = "lineNumbers"
    }
}

init(from decoder: Decoder) throws {
    let c = try decoder.container(keyedBy: CodingKeys.self)
    path = try c.decode(String.self, forKey: .path)
    startLine = try c.decodeIfPresent(Int.self, forKey: .startLine) ?? c.decodeIfPresent(Int.self, forKey: .startLineCamel)
    endLine = try c.decodeIfPresent(Int.self, forKey: .endLine) ?? c.decodeIfPresent(Int.self, forKey: .endLineCamel)
    maxLines = try c.decodeIfPresent(Int.self, forKey: .maxLines) ?? c.decodeIfPresent(Int.self, forKey: .maxLinesCamel)
    lineNumbers = try c.decodeIfPresent(Bool.self, forKey: .lineNumbers) ?? c.decodeIfPresent(Bool.self, forKey: .lineNumbersCamel)
}

private struct WriteArguments: Decodable {
    let path: String
    let content: String
    let expectedHash: String?
    let expectedVersion: String?
    let overwrite: Bool?

    enum CodingKeys: String, CodingKey {
        case path, content, expectedHash, expectedVersion, overwrite
        case targetFile = "target_file", targetFileCamel = "targetFile"
        case filePath = "file_path", filePathCamel = "filePath"
        case absolutePath = "absolute_path", absolutePathCamel = "AbsolutePath"
        case codeContent = "code_content", codeContentCamel = "codeContent"
        case targetContent = "target_content", targetContentCamel = "TargetContent"
    }
}

init(from decoder: Decoder) throws {
    let c = try decoder.container(keyedBy: CodingKeys.self)
    if let p = try c.decodeIfPresent(String.self, forKey: .path) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .targetFile) ?? c.decodeIfPresent(String.self, forKey: .targetFileCamel)

```

```

    ↪ ) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .filePath) ?? c.decodeIfPresent(String.self, forKey: .filePathCamel) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .absolutePath) ?? c.decodeIfPresent(String.self, forKey: .
    ↪ absolutePathCamel) {
        path = p
    } else {
        path = try c.decode(String.self, forKey: .path)
    }

    if let cnt = try c.decodeIfPresent(String.self, forKey: .content) {
        content = cnt
    } else if let cnt = try c.decodeIfPresent(String.self, forKey: .codeContent) ?? c.decodeIfPresent(String.self, forKey: .
    ↪ codeContentCamel) {
        content = cnt
    } else {
        content = try c.decode(String.self, forKey: .content)
    }

    expectedHash = try c.decodeIfPresent(String.self, forKey: .expectedHash)
    expectedVersion = try c.decodeIfPresent(String.self, forKey: .expectedVersion)
    overwrite = try c.decodeIfPresent(Bool.self, forKey: .overwrite)
}
}

private struct EditArguments: Decodable {
    let path: String
    let oldString: String
    let newString: String
    let replaceAll: Bool?
    let expectedHash: String?
    let expectedVersion: String?
    let overwrite: Bool?

    enum CodingKeys: String, CodingKey {
        case path, oldString, newString, replaceAll, expectedHash, expectedVersion, overwrite
        case targetFile = "target_file", targetFileCamel = "targetFile"
        case filePath = "file_path", filePathCamel = "filePath"
        case absolutePath = "absolute_path", absolutePathCamel = "AbsolutePath"
        case targetContent = "target_content", targetContentCamel = "TargetContent"
        case replacementContent = "replacement_content", replacementContentCamel = "ReplacementContent"
    }
}

init(from decoder: Decoder) throws {
    let c = try decoder.container(keyedBy: CodingKeys.self)
    if let p = try c.decodeIfPresent(String.self, forKey: .path) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .targetFile) ?? c.decodeIfPresent(String.self, forKey: .targetFileCamel)
    ↪ ) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .filePath) ?? c.decodeIfPresent(String.self, forKey: .filePathCamel) {
        path = p
    } else if let p = try c.decodeIfPresent(String.self, forKey: .absolutePath) ?? c.decodeIfPresent(String.self, forKey: .
    ↪ absolutePathCamel) {
        path = p
    } else {
        path = try c.decode(String.self, forKey: .path)
    }

    if let old = try c.decodeIfPresent(String.self, forKey: .oldString) {
        oldString = old
    } else if let old = try c.decodeIfPresent(String.self, forKey: .targetContent) ?? c.decodeIfPresent(String.self, forKey: .
    ↪ targetContentCamel) {
        oldString = old

```

```

    } else {
        oldString = try c.decode(String.self, forKey: .oldString)
    }

    if let nw = try c.decodeIfPresent(String.self, forKey: .newString) {
        newString = nw
    } else if let nw = try c.decodeIfPresent(String.self, forKey: .replacementContent) ?? c.decodeIfPresent(String.self, forKey: .
↪ replacementContentCamel) {
        newString = nw
    } else {
        newString = try c.decode(String.self, forKey: .newString)
    }

    replaceAll = try c.decodeIfPresent(Bool.self, forKey: .replaceAll)
    expectedHash = try c.decodeIfPresent(String.self, forKey: .expectedHash)
    expectedVersion = try c.decodeIfPresent(String.self, forKey: .expectedVersion)
    overwrite = try c.decodeIfPresent(Bool.self, forKey: .overwrite)
}
}

private struct GlobArguments: Decodable {
    let pattern: String
    let path: String?
    let maxResults: Int?
    let includeHidden: Bool?
    let includeIgnored: Bool?
    let includeGenerated: Bool?
}

private struct GrepArguments: Decodable {
    let pattern: String
    let path: String?
    let glob: String?
    let maxResults: Int?
    let includeHidden: Bool?
    let includeIgnored: Bool?
    let includeGenerated: Bool?
}

private struct PatchArguments: Decodable { let patch: String; let expectedHashes: [String: String]? }
private struct ShellArguments: Decodable {
    let command: String?
    let executable: String?
    let arguments: [String]?
    let cwd: String?
    let timeoutMs: Int?
}

private struct GitArguments: Decodable {
    let action: GitAction?
    let arguments: [String]?
    let cwd: String?
    let paths: [String]?
    let reference: String?
    let branch: String?
    let message: String?
    let limit: Int?
}

private struct ProcessArguments: Decodable {
    let action: String
    let executable: String?
    let arguments: [String]?
    let cwd: String?
    let id: String?
    let input: String?
    let stdoutCursor: Int?
    let stderrCursor: Int?
}

private struct QuestionArguments: Decodable { let question: String; let options: [String]?; let multiple: Bool? }

```

```

private struct SymbolArguments: Decodable { let symbol: String; let mode: String?; let direction: String? }
private struct CodeIntelligenceArguments: Decodable { let action: String; let query: String?; let path: String?; let line: Int?; let
    ↪ character: Int?; let maximumCharacters: Int? }
private struct SkillArguments: Decodable { let name: String }

private func pathArguments(_ arguments: String) throws -> PathArguments { try decodeArguments(arguments) }

private struct ReadLine: Codable { let number: Int; let content: String }
private struct ReadPage: Codable {
    let path: String
    let lines: [ReadLine]
    let startLine: Int
    let endLine: Int
    let nextLine: Int?
    let truncated: Bool
    let hash: String?
    let version: String
}

private func readPage(_ file: URL, workspace: WorkspaceRoot, input: ReadArguments) throws -> ReadPage {
    let start = max(1, input.startLine ?? 1)
    let end = input.endLine
    if let end, end < start { throw CoreError(code: .toolArgumentInvalid, message: "end_line 必须不小于 start_line") }
    let count = min(max(1, input.maxLines ?? end.map { $0 - start + 1 } ?? 200), 2_000)
    let handle = try FileHandle(forReadingFrom: file)
    defer { try? handle.close() }
    var carry = Data()
    var lineNumber = 1
    var lines: [ReadLine] = []
    var hasMore = false
    while let chunk = try handle.read(upToCount: 64 * 1_024), !chunk.isEmpty {
        if chunk.contains(0) { throw CoreError(code: .binaryFileUnsupported, message: "文件包含 NUL 字节, 不能作为文本读取") }
        carry.append(chunk)
        while let newline = carry.firstIndex(of: 10) {
            let lineData = carry.prefix(upTo: newline)
            carry.removeSubrange(...newline)
            guard let text = String(data: lineData, encoding: .utf8) else {
                throw CoreError(code: .binaryFileUnsupported, message: "文件不是 UTF-8 文本")
            }
            if lineNumber >= start && (end.map { lineNumber <= $0 } ?? true) {
                if lines.count < count { lines.append(ReadLine(number: lineNumber, content: text)) }
                else { hasMore = true; break }
            }
            lineNumber += 1
        }
        if hasMore { break }
    }
    if !carry.isEmpty && !hasMore {
        guard let text = String(data: carry, encoding: .utf8) else { throw CoreError(code: .binaryFileUnsupported, message: "文件不是 UTF-8
        ↪ 文本") }
        if lineNumber >= start && (end.map { lineNumber <= $0 } ?? true) {
            if lines.count < count { lines.append(ReadLine(number: lineNumber, content: text)) }
            else { hasMore = true }
        }
    }
    let nextLine = hasMore ? (lines.last?.number ?? start) + 1 : nil
    return ReadPage(path: relativePath(file, workspace: workspace), lines: lines, startLine: lines.first?.number ?? start, endLine: lines.
        ↪ last?.number ?? start - 1, nextLine: nextLine, truncated: hasMore, hash: nil, version: try fileVersion(file))
}

public struct ReadFileTool: ToolExecutor {
    public static let maximumBytes = 1_024 * 1_024
    private let workspace: WorkspaceRoot

    public init(workspace: WorkspaceRoot) {

```

```

        self.workspace = workspace
    }

    public let definition = ToolDefinition(
        id: ToolID("read_file"),
        description: "Read a UTF-8 text file inside the workspace.",
        inputSchema: ToolInputSchema(
            properties: [
                "path": ToolInputProperty(type: .string, description: "Workspace-relative file path"),
                "start_line": ToolInputProperty(type: .integer, description: "1-based start line"),
                "end_line": ToolInputProperty(type: .integer, description: "1-based end line")
            ],
            required: ["path"]
        ),
        capability: ToolCapability(readOnly: true)
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        try workspace.resolve(pathArguments(arguments).path, profile: profile).path
    }

    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        filesystemCapabilities(try workspace.resolve(pathArguments(arguments).path, profile: profile), workspace: workspace, write: false)
    }

    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: ReadArguments = try decodeArguments(arguments)
        let file = try workspace.resolve(input.path, profile: profile)
        if input.startLine != nil || input.endLine != nil || input.maxLines != nil || input.lineNumbers == true {
            return try json(readPage(file, workspace: workspace, input: input))
        }
        return try readText(file, operation: "read_file")
    }
}

public struct ContextRecallTool: ToolExecutor {
    private let ecoreStore: ECoreObjectStore?
    private let sessionId: SessionID?

    public init(ecoreStore: ECoreObjectStore? = nil, sessionId: SessionID? = nil) {
        self.ecoreStore = ecoreStore
        self.sessionID = sessionId
    }

    public let definition = ToolDefinition(
        id: ToolID("context_recall"),
        description: "Recall observation content by object ID and byte offset",
        inputSchema: ToolInputSchema(
            properties: [
                "id": ToolInputProperty(type: .string, description: "Context object ID"),
                "offset": ToolInputProperty(type: .integer, description: "Start byte offset"),
                "limit_bytes": ToolInputProperty(type: .integer, description: "Max bytes (default 16KB)"),
                "limit_lines": ToolInputProperty(type: .integer, description: "Max lines (default 400)"),
                "session_id": ToolInputProperty(type: .string, description: "Target session ID (optional)")
            ],
            required: ["id"]
        ),
        capability: ToolCapability(readOnly: true)
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        ""
    }

    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {

```

```

    [.projectRead]
}

public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    struct Input: Decodable {
        let id: String
        let offset: Int?
        let limitBytes: Int?
        let limitLines: Int?
        let sessionId: String?
    }
    let input: Input = try decodeArguments(arguments)
    let objectID: ContextObjectID
    do {
        objectID = try ContextObjectID(input.id)
    } catch {
        return "Error: Invalid ContextObjectID format '\(input.id)'"
    }

    guard let store = ecoreStore else {
        return "Error: E-Core object store is not configured."
    }

    let sID = input.sessionId.map(SessionID.init) ?? self.sessionID ?? ToolExecutionContext.sessionID ?? SessionID("default")
    let chunk = try await store.recall(
        sessionId: sID,
        objectID: objectID,
        offsetBytes: max(0, input.offset ?? 0),
        limitBytes: input.limitBytes,
        limitLines: input.limitLines
    )

    guard let chunk else {
        return "Context object '\(objectID.rawValue)' not found in session '\(sID.rawValue)'."
    }

    return ""
    [Context Object Slice: \(chunk.objectID.rawValue)]
    Lines: \(chunk.startLine) - \(chunk.endLine) of \(chunk.totalLines)
    Bytes: \(chunk.offsetBytes) - \(chunk.offsetBytes + chunk.lengthBytes) of \(chunk.totalBytes)
    Has More: \(chunk.hasMore ? "true" : "false")
    --- Content ---
    \(chunk.content)
    ""
}

}

public struct ListDirectoryTool: ToolExecutor {
    private let workspace: WorkspaceRoot

    public init(workspace: WorkspaceRoot) {
        self.workspace = workspace
    }

    public let definition = ToolDefinition(
        id: ToolID("list_directory"),
        description: "List direct entries of a directory inside the workspace.",
        inputSchema: ToolInputSchema(
            properties: ["path": ToolInputProperty(type: .string, description: "Workspace-relative directory path")],
            required: ["path"]
        ),
        capability: ToolCapability(readOnly: true)
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {

```

```

    try workspace.resolve(pathArguments(arguments).path, profile: profile).path
  }

public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
  filesystemCapabilities(try workspace.resolve(pathArguments(arguments).path, profile: profile), workspace: workspace, write: false)
}

public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
  let directory = try workspace.resolve(pathArguments(arguments).path, profile: profile)
  var isDirectory: ObjCBool = false
  guard FileManager.default.fileExists(atPath: directory.path, isDirectory: &isDirectory) else {
    throw CoreError(code: .toolExecutionFailed, message: "目录不存在: \(directory.path)")
  }
  guard isDirectory.boolValue else {
    throw CoreError(code: .toolExecutionFailed, message: "list_directory 只能读取目录: \(directory.path)")
  }
  let entries = try FileManager.default.contentsOfDirectory(
    at: directory,
    includingPropertiesForKeys: [.isDirectoryKey, .fileSizeKey],
    options: [.skipsHiddenFiles]
  )
  return try entries.filter { !workspace.sensitivePathPolicy.isSensitive($0) }.sorted { $0.lastPathComponent < $1.lastPathComponent }.
  ↪ map { entry in
    let values = try entry.resourceValues(forKeys: [.isDirectoryKey, .fileSizeKey])
    let kind = values.isDirectory == true ? "directory" : "file"
    let size = values.fileSize.map(String.init) ?? "-"
    return "\(entry.lastPathComponent)\t\(kind)\t\(size)"
  }.joined(separator: "\n")
}

private func regex(forGlob pattern: String) throws -> NSRegularExpression {
  let characters = Array(pattern)
  var source = ""
  var index = 0
  while index < characters.count {
    switch characters[index] {
    case "*":
      if index + 1 < characters.count, characters[index + 1] == "*" {
        index += 2
        if index < characters.count, characters[index] == "/" {
          source += "(?:.*/)?"
          index += 1
        } else {
          source += ".*"
        }
      }
      continue
    case "[^/]*":
      source += "[^/]*"
    case "?":
      source += "[^/]"
    case "\\\", \".\", \"^\", \"$\", \"|\", \"(\", \")\", \"[\", \"]\", \"{\", \"}\", \"+\":
      source += "\\\"
      source.append(characters[index])
    default:
      source.append(characters[index])
    }
    index += 1
  }
  return try NSRegularExpression(pattern: source + "$")
}

private func files(at root: URL, workspace: WorkspaceRoot, profile: ExecutionProfile) throws -> [URL] {
  var isDirectory: ObjCBool = false
  guard FileManager.default.fileExists(atPath: root.path, isDirectory: &isDirectory) else {
    throw CoreError(code: .toolExecutionFailed, message: "路径不存在: \(root.path)")
  }
}

```



```

guard isDirectory.boolValue else { return [root] }
guard let enumerator = FileManager.default.enumerator(at: root, includingPropertiesForKeys: [.isDirectoryKey, .isSymbolicLinkKey]) else
↳ {
    throw CoreError(code: .toolExecutionFailed, message: "无法枚举目录: \(root.path)")
}
var result: [URL] = []
while let entry = enumerator.nextObject() as? URL {
    let values = try? entry.resourceValues(forKeys: [.isDirectoryKey, .isSymbolicLinkKey])
    if values?.isSymbolicLink == true { enumerator.skipDescendants() }
    if workspace.sensitivePathPolicy.isSensitive(entry) {
        if values?.isDirectory == true { enumerator.skipDescendants() }
        continue
    }
    if let resolved = try? workspace.resolve(entry.path, profile: profile) { result.append(resolved) }
}
return result
}

public struct GlobTool: ToolExecutor {
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot) { self.workspace = workspace }
    public let definition = ToolDefinition(
        id: ToolID("glob"), description: "Find workspace files matching a glob pattern.",
        inputSchema: ToolInputSchema(properties: [
            "pattern": ToolInputProperty(type: .string, description: "Glob pattern"),
            "path": ToolInputProperty(type: .string, description: "Workspace-relative search root"),
            "max_results": ToolInputProperty(type: .integer, description: "Maximum matches", minimum: 1)
        ], required: ["pattern"]), capability: ToolCapability(readOnly: true)
    )
    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: GlobArguments = try decodeArguments(arguments)
        return try workspace.resolve(input.path ?? ".", profile: profile).path
    }
    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        filesystemCapabilities(URL(fileURLWithPath: try resource(for: arguments, profile: profile)), workspace: workspace, write: false)
    }
    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: GlobArguments = try decodeArguments(arguments)
        let root = try workspace.resolve(input.path ?? ".", profile: profile)
        var isDirectory: ObjCBool = false
        guard FileManager.default.fileExists(atPath: root.path, isDirectory: &isDirectory) else {
            throw CoreError(code: .toolExecutionFailed, message: "路径不存在: \(root.path)")
        }
        guard isDirectory.boolValue else {
            throw CoreError(code: .toolExecutionFailed, message: "glob 搜索根路径必须是目录: \(root.path)")
        }
        let limit = min(max(1, input.maxResults ?? 1_000), 10_000)
        let result = try await runRipgrep(arguments: ripgrepArguments(input, root: root), root: root, workspace: workspace, profile: profile
↳ )
        let paths = ignoredByWorkspaceGitignore(result.stdout.split(separator: "\n", omittingEmptySubsequences: true).map(String.init), root
↳ : root, includeIgnored: input.includeIgnored)
            .map { workspaceRelativeSearchPath($0, root: root, workspace: workspace) }
            .sorted()
        return try json(Array(paths.prefix(limit)))
    }
}

private struct GrepMatch: Codable { let path: String; let line: Int; let content: String }
private struct SearchResult<T: Codable>: Codable { let matches: [T]; let truncated: Bool }

private func ripgrepExecutable() throws -> String {
    guard let executable = LingXiPlatform.process.resolveExecutable(named: "rg", customSearchPaths: ["/opt/homebrew/bin", "/usr/local/bin",
↳ "/usr/bin"]) else {
        throw CoreError(code: .toolExecutionFailed, message: "未找到受支持的 ripgrep (rg) 可执行文件")
    }
}

```

```

    return executable
}

private func generatedExcludes(_ includeGenerated: Bool?) -> [String] {
    guard includeGenerated != true else { return [] }
    return ["!*/.build/*", "!*/build/*", "!*/dist/*", "!*/node_modules/*", "!*/vendor/*", "!*/coverage/*"]
}

private let sensitiveSearchExcludes = [
    "!*/.ssh/*", "!*/.aws/*", "!*/.gnupg/*", "!*/.env", "!*/.env.*", "!*/*.pem", "!*/*.key",
    "!*/*credential*", "!*/*secret*", "!*/*token*"
]

private func workspaceIgnoreArguments(root: URL, includeIgnored: Bool?) -> [String] {
    guard includeIgnored != true else { return [] }
    let ignore = root.appendingPathComponent(".gitignore")
    return FileManager.default.fileExists(atPath: ignore.path) ? ["--ignore-file", ignore.path] : []
}

private func ignoredByWorkspaceGitignore(_ paths: [String], root: URL, includeIgnored: Bool?) -> [String] {
    guard includeIgnored != true,
        let text = try? String(contentsOf: root.appendingPathComponent(".gitignore"), encoding: .utf8)
    else { return paths }
    let patterns = text.split(separator: "\n").map(String.init).filter { !$0.isEmpty && !$0.hasPrefix("#") }
    return paths.filter { original in
        let path = original.hasPrefix("./") ? String(original.dropFirst(2)) : original
        var ignored = false
        for raw in patterns {
            let negated = raw.hasPrefix("!")
            let pattern = negated ? String(raw.dropFirst()) : raw
            let matches: Bool
            if !pattern.contains("/") { matches = path.split(separator: "/").contains(pattern[...]) }
            else { matches = (try? regex(forGlob: pattern).firstMatch(in: path, range: NSRange(path.startIndex..., in: path))) != nil }
            if matches { ignored = !negated }
        }
        return !ignored
    }
}

private func ripgrepArguments(_ input: GlobArguments, root: URL) -> [String] {
    var arguments = ["--files", "--no-require-git", "--color", "never", "--glob", input.pattern]
    if input.includeHidden == true { arguments.append("--hidden") }
    if input.includeIgnored == true { arguments.append("--no-ignore") }
    arguments += workspaceIgnoreArguments(root: root, includeIgnored: input.includeIgnored)
    for exclude in generatedExcludes(input.includeGenerated) + sensitiveSearchExcludes { arguments += ["--glob", exclude] }
    return arguments
}

private func ripgrepArguments(_ input: GrepArguments, root: URL, targetIsFile: Bool = false, filePath: String? = nil) -> [String] {
    var arguments = ["--json", "--no-require-git", "--line-number", "--color", "never", "--regex", input.pattern]
    if !targetIsFile, let glob = input.glob { arguments += ["--glob", glob] }
    if input.includeHidden == true { arguments.append("--hidden") }
    if input.includeIgnored == true { arguments.append("--no-ignore") }
    arguments += workspaceIgnoreArguments(root: root, includeIgnored: input.includeIgnored)
    for exclude in generatedExcludes(input.includeGenerated) + sensitiveSearchExcludes { arguments += ["--glob", exclude] }
    if targetIsFile, let filePath {
        arguments.append(filePath)
    } else {
        arguments.append(".")
    }
    return arguments
}

private func runRipgrep(arguments: [String], root: URL, workspace: WorkspaceRoot, profile: ExecutionProfile) async throws -> CommandResult {
    let executable = try ripgrepExecutable()

```

```

let setup = try processSetup(executable: executable, arguments: arguments, workspace: workspace, cwd: root, profile: profile)
let result = try await runToolProcess(invocation: setup.0, cwd: root, environment: setup.1, timeoutMilliseconds: 30_000, lifecycleTrace:
  ↪ ToolExecutionContext.lifecycleTrace)
guard result.exitCode == 0 || result.exitCode == 1 else {
  throw CoreError(code: .commandFailed, message: try json(result))
}
return result
}

private func workspaceRelativeSearchPath(_ path: String, root: URL, workspace: WorkspaceRoot) -> String {
  if path.hasPrefix("/") {
    let wsPath = workspace.url.path
    if path == wsPath { return "." }
    if path.hasPrefix(wsPath + "/") {
      return String(path.dropFirst(wsPath.count + 1))
    }
  }
  let local = path.hasPrefix("./") ? String(path.dropFirst(2)) : path
  if root.path == workspace.url.path { return local }
  let prefix = relativePath(root, workspace: workspace)
  return prefix == "." || prefix.isEmpty ? local : prefix + "/" + local
}

public struct GrepTool: ToolExecutor {
  private let workspace: WorkspaceRoot
  public init(workspace: WorkspaceRoot) { self.workspace = workspace }
  public let definition = ToolDefinition(
    id: ToolID("grep"), description: "Search UTF-8 workspace files with a regular expression.",
    inputSchema: ToolInputSchema(properties: [
      "pattern": ToolInputProperty(type: .string, description: "Regular expression"),
      "path": ToolInputProperty(type: .string, description: "Workspace-relative search root"),
      "max_results": ToolInputProperty(type: .integer, description: "Maximum matches", minimum: 1)
    ], required: ["pattern"]), capability: ToolCapability(readOnly: true)
  )
  public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
    let input: GrepArguments = try decodeArguments(arguments)
    return try workspace.resolve(input.path ?? ".", profile: profile).path
  }
  public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
    filesystemCapabilities(URL(fileURLWithPath: try resource(for: arguments, profile: profile)), workspace: workspace, write: false)
  }
  public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input: GrepArguments = try decodeArguments(arguments)
    let target = try workspace.resolve(input.path ?? ".", profile: profile)
    var isDirectory: ObjCBool = false
    guard FileManager.default.fileExists(atPath: target.path, isDirectory: &isDirectory) else {
      throw CoreError(code: .toolExecutionFailed, message: "搜索路径不存在: \(target.path)")
    }
    let limit = min(max(1, input.maxResults ?? 1_000), 10_000)
    let targetIsFile = !isDirectory.boolValue
    if targetIsFile, workspace.sensitivePathPolicy.isSensitive(target) {
      return try json([GrepMatch]())
    }
    let runDir = targetIsFile ? (FileManager.default.fileExists(atPath: target.deletingLastPathComponent().path) ? target.
  ↪ deletingLastPathComponent() : workspace.url) : target
    let rgArgs = ripgrepArguments(input, root: runDir, targetIsFile: targetIsFile, filePath: targetIsFile ? target.path : nil)
    let result = try await runRipgrep(arguments: rgArgs, root: runDir, workspace: workspace, profile: profile)
    var matches: [GrepMatch] = []
    for row in result.stdout.split(separator: "\n", omittingEmptySubsequences: true) {
      guard let data = row.data(using: .utf8),
        let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
        object["type"] as? String == "match",
        let payload = object["data"] as? [String: Any],
        let pathObject = payload["path"] as? [String: Any],
        let path = pathObject["text"] as? String,

```

```

        let line = (payload["line_number"] as? NSNumber)?.intValue,
        let lineObject = payload["lines"] as? [String: Any],
        let text = lineObject["text"] as? String
    else { continue }
    let relPath = workspaceRelativeSearchPath(path, root: runDir, workspace: workspace)
    matches.append(GrepMatch(path: relPath, line: line, content: text.hasSuffix("\n") ? String(text.dropLast()) : text))
}
let ordered = matches.filter { !ignoredByWorkspaceGitignore([$0.path], root: runDir, includeIgnored: input.includeIgnored).isEmpty }
↪ }.sorted { $0.path == $1.path ? $0.line < $1.line : $0.path < $1.path }
return try json(Array(ordered.prefix(limit)))
}
}

private struct FileWriteResult: Codable {
    let path: String
    let bytes: Int
    let hash: String
    let version: String
}

private struct CommandToolResult: Codable {
    let command: String
    let stdout: String
    let stderr: String
    let exitCode: Int32
    let summary: String

    enum CodingKeys: String, CodingKey { case command, stdout, stderr, exitCode = "exit_code", summary }
}

public struct WriteFileTool: ToolExecutor {
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot) { self.workspace = workspace }
    public let definition = ToolDefinition(
        id: ToolID("write_file"), description: "Write UTF-8 text to a file. Supports workspace-relative paths, absolute paths, ~, and $HOME
↪ (e.g. ~/Desktop/file.txt).",
        inputSchema: ToolInputSchema(properties: [
            "path": ToolInputProperty(type: .string, description: "File path (supports workspace-relative, absolute, ~, and $HOME)"),
            "content": ToolInputProperty(type: .string, description: "Replacement file content"),
            "expected_hash": ToolInputProperty(type: .string, description: "Current SHA-256 required before writing"),
            "expected_version": ToolInputProperty(type: .string, description: "Current version required before writing"),
            "overwrite": ToolInputProperty(type: .boolean, description: "Explicitly bypass stale-content checks")
        ], required: ["path", "content"]), capability: ToolCapability(readOnly: false)
    )
    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: WriteArguments = try decodeArguments(arguments)
        return try workspace.resolve(input.path, profile: profile).path
    }
    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        filesystemCapabilities(URL(fileURLWithPath: try resource(for: arguments, profile: profile)), workspace: workspace, write: true)
    }
    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: WriteArguments = try decodeArguments(arguments)
        let file = try workspace.resolve(input.path, profile: profile)
        if FileManager.default.fileExists(atPath: file.path) {
            let existing = try readText(file, operation: "write_file")
            try checkExpectedContent(existing, hash: input.expectedHash, version: input.expectedVersion, currentVersion: try fileVersion(
↪ file), overwrite: input.overwrite)
        }
        try writeText(input.content, to: file)
        return try json(fileWriteResult(for: file, workspace: workspace, content: input.content))
    }
}

public struct EditFileTool: ToolExecutor {

```

```

private let workspace: WorkspaceRoot
public init(workspace: WorkspaceRoot) { self.workspace = workspace }
public let definition = ToolDefinition(
    id: ToolID("edit_file"), description: "Replace exact text in a UTF-8 file. Supports workspace-relative paths, absolute paths, ~, and
    ↪ $HOME.",
    inputSchema: ToolInputSchema(properties: [
        "path": ToolInputProperty(type: .string, description: "File path (supports workspace-relative, absolute, ~, and $HOME)"),
        "old_string": ToolInputProperty(type: .string, description: "Text to replace"),
        "new_string": ToolInputProperty(type: .string, description: "Replacement text"),
        "replace_all": ToolInputProperty(type: .boolean, description: "Replace every occurrence"),
        "expected_hash": ToolInputProperty(type: .string, description: "Current SHA-256 required before editing"),
        "expected_version": ToolInputProperty(type: .string, description: "Current version required before editing"),
        "overwrite": ToolInputProperty(type: .boolean, description: "Explicitly bypass stale-content checks")
    ], required: ["path", "old_string", "new_string"]), capability: ToolCapability(readOnly: false)
)
public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
    let input: EditArguments = try decodeArguments(arguments)
    return try workspace.resolve(input.path, profile: profile).path
}
public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
    filesystemCapabilities(URL(fileURLWithPath: try resource(for: arguments, profile: profile)), workspace: workspace, write: true)
}
public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input: EditArguments = try decodeArguments(arguments)
    guard !input.oldString.isEmpty else { throw CoreError(code: .toolArgumentInvalid, message: "old_string 不能为空") }
    let file = try workspace.resolve(input.path, profile: profile)
    let original = try readText(file, operation: "edit_file")
    try checkExpectedContent(original, hash: input.expectedHash, version: input.expectedVersion, currentVersion: try fileVersion(file),
    ↪ overwrite: input.overwrite)
    let count = original.components(separatedBy: input.oldString).count - 1
    guard count > 0 else { throw CoreError(code: .contentChanged, message: "未找到要替换的文本: \{(file.path)\}") }
    guard input.replaceAll == true || count == 1 else { throw CoreError(code: .ambiguousEdit, message: "匹配到 \{count\} 处文本; 请使用
    ↪ replace_all") }
    let updated = input.replaceAll == true ? original.replacingOccurrences(of: input.oldString, with: input.newString) : original.
    ↪ replacingOccurrences(of: input.oldString, with: input.newString, options: [], range: original.range(of: input.oldString))
    try writeText(updated, to: file)
    return try json(fileWriteResult(for: file, workspace: workspace, content: updated))
}
}

private enum PatchKind: Equatable { case add, update, delete }
private struct PatchSpec { var kind: PatchKind; var path: String; var moveTo: String?; var lines: [String] }
private struct PatchWrite { let url: URL; let content: String }
private struct PatchPlan { let writes: [PatchWrite]; let deletes: [URL] }
private struct PatchResult: Codable { let operation: String; let path: String }

/// 测试可通过此钩子在第 N 次落盘前制造失败, 验证补丁事务的回滚。
public enum ApplyPatchFailpoint {
    private static let state = ApplyPatchFailpointState()

    public static func fail(after operation: Int?) { state.set(after: operation) }

    fileprivate static func shouldFail(at operation: Int) -> Bool {
        state.shouldFail(at: operation)
    }
}

private final class ApplyPatchFailpointState: @unchecked Sendable {
    private let lock = NSLock()
    private var operation: Int?

    func set(after operation: Int?) { lock.lock(); self.operation = operation; lock.unlock() }

    func shouldFail(at operation: Int) -> Bool { lock.lock(); defer { lock.unlock() }; return self.operation == operation }
}

private struct PatchSnapshot { let url: URL; let data: Data? }

```

```

private func patchSnapshots(for plan: PatchPlan) throws -> [PatchSnapshot] {
    try Dictionary(grouping: plan.writes.map(\.url) + plan.deletes, by: \.path).values.map { urls in
        let url = urls[0]
        return PatchSnapshot(url: url, data: FileManager.default.fileExists(atPath: url.path) ? try Data(contentsOf: url) : nil)
    }
}

private func restorePatchSnapshots(_ snapshots: [PatchSnapshot]) throws {
    for snapshot in snapshots {
        if let data = snapshot.data {
            try data.write(to: snapshot.url, options: .atomic)
        } else if FileManager.default.fileExists(atPath: snapshot.url.path) {
            try FileManager.default.removeItem(at: snapshot.url)
        }
    }
}

private func parsePatch(_ patch: String) throws -> [PatchSpec] {
    var lines = patch.components(separatedBy: "\n").map { $0.hasSuffix("\r") ? String($0.dropLast()) : $0 }
    while lines.last == "" { lines.removeLast() }
    guard lines.first == "*** Begin Patch", lines.last == "*** End Patch" else {
        throw CoreError(code: .invalidPatch, message: "Patch 必须以 *** Begin Patch 和 *** End Patch 包裹")
    }
    var specs: [PatchSpec] = []
    var current: PatchSpec?
    func flush() { if let current { specs.append(current) } }
    for line in lines.dropFirst().dropLast() {
        if line.hasPrefix("*** Add File: ") || line.hasPrefix("*** Update File: ") || line.hasPrefix("*** Delete File: ") {
            flush()
            let add = line.hasPrefix("*** Add File: ")
            let update = line.hasPrefix("*** Update File: ")
            let prefix = add ? "*** Add File: " : update ? "*** Update File: " : "*** Delete File: "
            let path = String(line.dropFirst(prefix.count))
            guard !path.isEmpty else { throw CoreError(code: .invalidPatch, message: "Patch 路径不能为空") }
            current = PatchSpec(kind: add ? .add : update ? .update : .delete, path: path, moveTo: nil, lines: [])
        } else if line.hasPrefix("*** Move to: ") {
            guard var spec = current, spec.kind == .update else {
                throw CoreError(code: .invalidPatch, message: "Move to 必须紧随 Update File")
            }
            let path = String(line.dropFirst("*** Move to: ".count))
            guard !path.isEmpty else { throw CoreError(code: .invalidPatch, message: "移动目标不能为空") }
            spec.moveTo = path
            current = spec
        } else {
            guard var spec = current else { throw CoreError(code: .invalidPatch, message: "Patch 缺少文件操作") }
            spec.lines.append(line)
            current = spec
        }
    }
    flush()
    guard !specs.isEmpty else { throw CoreError(code: .invalidPatch, message: "Patch 不包含文件操作") }
    return specs
}

private func applyHunks(_ rows: [String], to content: String) throws -> String {
    var result = content.components(separatedBy: "\n")
    var hunks: [[String]] = [[]]
    for row in rows {
        if row.hasPrefix("@@") { hunks.append([]) }
        else if row == "\\ No newline at end of file" { continue }
        else { hunks[hunks.count - 1].append(row) }
    }
    for hunk in hunks where !hunk.isEmpty {
        var old: [String] = []

```

```

var new: [String] = []
for row in hunk {
    guard let marker = row.first, marker == " " || marker == "+" || marker == "-" else {
        throw CoreError(code: .invalidPatch, message: "Update 行必须以空格、+ 或 - 开头")
    }
    let line = String(row.dropFirst())
    if marker != "+" { old.append(line) }
    if marker != "-" { new.append(line) }
}
guard !old.isEmpty, result.count >= old.count else {
    throw CoreError(code: .patchConflict, message: "Patch 缺少可定位的上下文")
}
let positions = (0...(result.count - old.count)).filter { Array(result[$0.. $\$0 + old.count$ ]) == old }
guard positions.count == 1, let position = positions.first else {
    throw CoreError(code: .patchConflict, message: positions.isEmpty ? "Patch 上下文不匹配" : "Patch 上下文不唯一")
}
result.replaceSubrange(position.. $(position + old.count)$ , with: new)
}
return result.joined(separator: "\n")
}

private func patchPlan(_ specs: [PatchSpec], expectedHashes: [String: String]?, workspace: WorkspaceRoot, profile: ExecutionProfile) throws
    ↪ -> PatchPlan {
    var writes: [PatchWrite] = []
    var deletes: [URL] = []
    var touched = Set<String>()
    for spec in specs {
        let source = try workspace.resolve(spec.path, profile: profile)
        let target = try spec.moveTo.map { try workspace.resolve( $\$0$ , profile: profile) } ?? source
        let paths = target == source ? [source.path] : [source.path, target.path]
        guard touched.isDisjoint(with: paths) else { throw CoreError(code: .invalidPatch, message: "Patch 重复操作文件") }
        touched.formUnion(paths)
        switch spec.kind {
        case .add:
            guard spec.lines.allSatisfy({  $\$0$ .hasPrefix("+") }) else { throw CoreError(code: .invalidPatch, message: "Add File 内容必须以 +
            ↪ 开头") }
            guard !FileManager.default.fileExists(atPath: source.path) else { throw CoreError(code: .patchConflict, message: "文件已存在: \(\(
            ↪ source.path)") }
            try writableFile(source)
            let content = spec.lines.map { String( $\$0$ .dropFirst()) }.joined(separator: "\n") + (spec.lines.isEmpty ? "" : "\n")
            writes.append(PatchWrite(url: source, content: content))
        case .update:
            let original = try readText(source, operation: "apply_patch")
            if let expected = expectedHashes?[spec.path], expected.lowercased().replacingOccurrences(of: "sha256:", with: "") != sha256Hex(
            ↪ original) {
                throw CoreError(code: .contentChanged, message: "Patch 前置版本不匹配: \((spec.path)")
            }
            try writableFile(target)
            if target != source {
                guard !FileManager.default.fileExists(atPath: target.path) else { throw CoreError(code: .patchConflict, message: "移动目标已
                ↪ 存在: \((target.path)") }
                try writableFile(target)
                deletes.append(source)
            }
            writes.append(PatchWrite(url: target, content: try applyHunks(spec.lines, to: original)))
        case .delete:
            guard spec.lines.isEmpty else { throw CoreError(code: .invalidPatch, message: "Delete File 不接受内容") }
            let original = try readText(source, operation: "apply_patch")
            if let expected = expectedHashes?[spec.path], expected.lowercased().replacingOccurrences(of: "sha256:", with: "") != sha256Hex(
            ↪ original) {
                throw CoreError(code: .contentChanged, message: "Patch 前置版本不匹配: \((spec.path)")
            }
            deletes.append(source)
        }
    }
}

```

```

    return PatchPlan(writes: writes, deletes: deletes)
}

public struct ApplyPatchTool: ToolExecutor {
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot) { self.workspace = workspace }
    public let definition = ToolDefinition(
        id: ToolID("apply_patch"), description: "Apply an Add, Update, Delete, or Move patch in the workspace.",
        inputSchema: ToolInputSchema(properties: [
            "patch": ToolInputProperty(type: .string, description: "Patch in *** Begin Patch syntax")
        ], required: ["patch"]),
        capability: ToolCapability([.projectWrite, .destructive])
    )
    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: PatchArguments = try decodeArguments(arguments)
        return try parsePatch(input.patch).map { try workspace.resolve($0.path, profile: profile).path }.sorted().joined(separator: ",")
    }
    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        let input: PatchArguments = try decodeArguments(arguments)
        let specs = try parsePatch(input.patch)
        var result: Set<ToolCapabilityKind> = [.projectWrite, .destructive]
        for spec in specs {
            for path in [spec.path, spec.moveTo].compactMap({ $0 }) {
                result.formUnion(filesystemCapabilities(try workspace.resolve(path, profile: profile), workspace: workspace, write: true))
            }
        }
        return result
    }
    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: PatchArguments = try decodeArguments(arguments)
        let specs = try parsePatch(input.patch)
        let plan = try patchPlan(specs, expectedHashes: input.expectedHashes, workspace: workspace, profile: profile)
        let snapshots = try patchSnapshots(for: plan)
        do {
            var operation = 0
            for write in plan.writes {
                operation += 1
                if ApplyPatchFailpoint.shouldFail(at: operation) { throw CoreError(code: .toolExecutionFailed, message: "apply_patch
↪ failpoint \$(operation)") }
                try writeText(write.content, to: write.url)
            }
            for file in plan.deletes {
                operation += 1
                if ApplyPatchFailpoint.shouldFail(at: operation) { throw CoreError(code: .toolExecutionFailed, message: "apply_patch
↪ failpoint \$(operation)") }
                try FileManager.default.removeItem(at: file)
            }
        } catch {
            do {
                try restorePatchSnapshots(snapshots)
            } catch {
                throw CoreError(code: .patchConflict, message: "Patch 失败且回滚失败: \$(error.localizedDescription)")
            }
            throw error
        }
        let results = specs.map { PatchResult(operation: $0.kind == .add ? "add" : $0.kind == .delete ? "delete" : $0.moveTo == nil ? "
↪ update" : "move", path: $0.moveTo ?? $0.path) }
        return try json(results)
    }
}

private func cwd(_ value: String?, workspace: WorkspaceRoot, profile: ExecutionProfile) throws -> URL {
    let url = try workspace.resolve(value ?? ".", profile: profile)
    var isDirectory: ObjCBool = false
    guard FileManager.default.fileExists(atPath: url.path, isDirectory: &isDirectory) else {

```



```

        throw CoreError(code: .toolExecutionFailed, message: "工作目录不存在: \(url.path)")
    }
    guard isDirectory.boolValue else {
        throw CoreError(code: .toolExecutionFailed, message: "工作目录必须是目录而不是文件: \(url.path)")
    }
    return url
}

private func processSetup(executable: String, arguments: [String], workspace: WorkspaceRoot, cwd: URL, profile: ExecutionProfile) throws ->
    ⇨ (ToolProcessInvocation, [String: String]) {
    guard executable.hasPrefix("/") , FileManager.default.isExecutableFile(atPath: executable) else {
        throw CoreError(code: .toolArgumentInvalid, message: "executable 必须是可执行的绝对路径")
    }
    var environment = EnvironmentSanitizer.sanitized()
    let temporary = workspace.url.appendingPathComponent(".lingxi-tmp", isDirectory: true)
    try FileManager.default.createDirectory(at: temporary, withIntermediateDirectories: true)
    environment["TMPDIR"] = temporary.path
    // /usr/bin/git is a system tool; inherited Xcode selection can force xcrun to write host-global caches outside policy.
    if executable.hasSuffix("/git") {
        environment.removeValue(forKey: "DEVELOPER_DIR")
        environment.removeValue(forKey: "SDKROOT")
        environment.removeValue(forKey: "TOOLCHAINS")
        environment["GIT_CONFIG_GLOBAL"] = "/dev/null"
        environment["GIT_CONFIG_NOSYSTEM"] = "1"
    }
    if profile == .workspace {
        let developerDirectory = environment["DEVELOPER_DIR"].map { URL(fileURLWithPath: $0, isDirectory: true) }
        let policy = SandboxPolicy(workspace: workspace.url, readOnlyPaths: developerDirectory.map { [$0] } ?? [])
        return (try ShellSandboxBackend.workspace().invocation(executable: executable, arguments: arguments, policy: policy), environment)
    }
    return (ToolProcessInvocation(executable: executable, arguments: arguments), environment)
}

private func isReadOnlyShellCommand(_ cmd: String) -> Bool {
    var trimmed = cmd.trimmingCharacters(in: .whitespacesAndNewlines)
    guard !trimmed.isEmpty else { return false }
    trimmed = trimmed.replacingOccurrences(of: "2>/dev/null", with: "")
        .replacingOccurrences(of: ">/dev/null", with: "")
        .replacingOccurrences(of: "1>/dev/null", with: "")
        .replacingOccurrences(of: "2>&1", with: "")
        .replacingOccurrences(of: "&>/dev/null", with: "")
    if trimmed.contains(">") || trimmed.contains("rm ") || trimmed.contains("mv ") || trimmed.contains("cp ") || trimmed.contains("mkdir ")
        ⇨ || trimmed.contains("touch ") || trimmed.contains("chmod ") || trimmed.contains("chown ") || trimmed.contains("tee ") {
        return false
    }
    let readOnlyTokens: Set<String> = ["ls", "pwd", "which", "echo", "cat", "head", "tail", "grep", "find", "diff", "file", "stat", "true",
        ⇨ "false", "wc", "readlink", "whoami", "uname", "id", "hostname", "date"]
    let firstToken = trimmed.components(separatedBy: CharacterSet.whitespaces.union(CharacterSet(charactersIn: "|;&"))).first { !$0.isEmpty
        ⇨ } ?? ""
    let base = URL(fileURLWithPath: firstToken).lastPathComponent
    return readOnlyTokens.contains(base)
}

public struct ShellTool: ToolExecutor {
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot) { self.workspace = workspace }
    public let definition = ToolDefinition(
        id: ToolID("shell"), description: "Run a shell command or absolute executable argv.",
        inputSchema: ToolInputSchema(properties: [
            "command": ToolInputProperty(type: .string, description: "Shell command"),
            "executable": ToolInputProperty(type: .string, description: "Absolute executable path"),
            "arguments": ToolInputProperty(type: .array, description: "Executable argv"),
            "cwd": ToolInputProperty(type: .string, description: "Workspace-relative working directory"),
            "timeout_ms": ToolInputProperty(type: .integer, description: "Timeout in milliseconds", minimum: 1, maximum: 300_000)
        ], required: [], capability: ToolCapability([.processExecute])
    )
}

```

```

)
public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
    let input: ShellArguments = try decodeArguments(arguments)
    _ = try cwd(input.cwd, workspace: workspace, profile: profile)
    return input.command ?? ([input.executable ?? "" ] + (input.arguments ?? [])).joined(separator: " ")
}

public func externalResource(for arguments: String, profile: ExecutionProfile) throws -> String? {
    let input: ShellArguments = try decodeArguments(arguments)
    return try cwd(input.cwd, workspace: workspace, profile: profile).path
}

public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
    let input: ShellArguments = try decodeArguments(arguments)
    let isWrite = !isReadOnlyShellCommand(input.command ?? input.executable ?? "")
    return Set([.processExecute]).union(filesystemCapabilities(try cwd(input.cwd, workspace: workspace, profile: profile), workspace:
↳ workspace, write: isWrite))
}

public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input: ShellArguments = try decodeArguments(arguments)
    let command: (String, [String])
    if let shell = input.command {
        #if os(Windows)
            let shellExe = LingXiPlatform.process.resolveExecutable(named: "powershell.exe", customSearchPaths: nil) ?? "C:\\Windows\\
↳ System32\\cmd.exe"
            let shellArgs = shellExe.lowercased().contains("powershell") ? ["-NoProfile", "-NonInteractive", "-Command", shell] : ["/c",
↳ shell]
            command = (shellExe, shellArgs)
        #else
            let shellExe = LingXiPlatform.process.resolveExecutable(named: "sh", customSearchPaths: ["/bin", "/usr/bin"]) ?? "/bin/sh"
            command = (shellExe, ["-c", shell])
        #endif
    } else if let executable = input.executable {
        command = (executable, input.arguments ?? [])
    } else {
        throw CoreError(code: .toolArgumentInvalid, message: "shell 需要 command 或 executable")
    }
    let directory = try cwd(input.cwd, workspace: workspace, profile: profile)
    let setup = try processSetup(executable: command.0, arguments: command.1, workspace: workspace, cwd: directory, profile: profile)
    let result = try await runToolProcess(invocation: setup.0, cwd: directory, environment: setup.1, timeoutMilliseconds: input.
↳ timeoutMs ?? 60_000, lifecycleTrace: ToolExecutionContext.lifecycleTrace)
    guard result.exitCode == 0 else { throw CoreError(code: .commandFailed, message: try json(result)) }
    return try json(result)
}

}

public enum GitAction: String, Codable, Sendable, CaseIterable {
    case status, diff, log, show, branch, add, restore, checkout, `switch`, commit
}

private func gitCommand(_ input: GitArguments) throws -> (GitAction, [String]) {
    if let action = input.action {
        switch action {
            case .status: return (action, ["status", "--short"])
            case .diff: return (action, ["diff", "--"] + (input.paths ?? []))
            case .log: return (action, ["log", "--oneline", "-n", String(min(max(input.limit ?? 10, 1), 100))])
            case .show: return (action, ["show", input.reference ?? "HEAD"])
            case .branch: return (action, input.branch.map { ["branch", $0] } ?? ["branch"])
            case .add: return (action, ["add", "--"] + (input.paths ?? []))
            case .restore: return (action, ["restore", "--"] + (input.paths ?? []))
            case .checkout: guard let reference = input.reference else { throw CoreError(code: .toolArgumentInvalid, message: "checkout 需要
↳ reference") }; return (action, ["checkout", reference])
            case .switch: guard let branch = input.branch else { throw CoreError(code: .toolArgumentInvalid, message: "switch 需要 branch") };
↳ return (action, ["switch", branch])
            case .commit: guard let message = input.message, !message.isEmpty else { throw CoreError(code: .toolArgumentInvalid, message: "
↳ commit 需要 message") }; return (action, ["commit", "-m", message])
        }
    }
}

```

```

    }
    guard let legacy = input.arguments, let command = legacy.first, let action = GitAction(rawValue: command) else {
        throw CoreError(code: .toolArgumentInvalid, message: "git 需要受支持的 action")
    }
    // Compatibility is intentionally limited to read-only legacy invocations.
    guard [.status, .diff, .log, .show].contains(action), !legacy.dropFirst().contains(where: { $0 == "-C" || $0 == "--git-dir" || $0 == "--"
↳ work-tree" }) else {
        throw CoreError(code: .toolArgumentInvalid, message: "git 仅支持结构化 action")
    }
    return (action, legacy)
}

public struct GitTool: ToolExecutor {
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot) { self.workspace = workspace }
    public let definition = ToolDefinition(
        id: ToolID("git"), description: "Run an allow-listed structured git action in the workspace.",
        inputSchema: ToolInputSchema(properties: [
            "action": ToolInputProperty(type: .string, description: "Git action", enumValues: GitAction.allCases.map(\.rawValue)),
            "arguments": ToolInputProperty(type: .array, description: "Legacy read-only git argv"),
            "cwd": ToolInputProperty(type: .string, description: "Workspace-relative working directory"),
            "paths": ToolInputProperty(type: .array, description: "Pathspecs for supported actions"),
            "reference": ToolInputProperty(type: .string, description: "Git revision"),
            "branch": ToolInputProperty(type: .string, description: "Branch name"),
            "message": ToolInputProperty(type: .string, description: "Commit message"),
            "limit": ToolInputProperty(type: .integer, description: "Log entry limit", minimum: 1, maximum: 100)
        ], required: []), capability: ToolCapability([.repositoryRead])
    )
    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: GitArguments = try decodeArguments(arguments)
        return try cwd(input.cwd, workspace: workspace, profile: profile).path
    }
    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        let input: GitArguments = try decodeArguments(arguments)
        let (action, _) = try gitCommand(input)
        var result: Set<ToolCapabilityKind> = [.status, .diff, .log, .show].contains(action) || action == .branch && input.branch == nil ?
↳ [.repositoryRead] : [.repositoryWrite]
        if action == .restore || action == .checkout || action == .switch { result.insert(.destructive) }
        return result
    }
    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: GitArguments = try decodeArguments(arguments)
        _ = try capabilities(for: arguments, profile: profile)
        let (_, command) = try gitCommand(input)
        let directory = try cwd(input.cwd, workspace: workspace, profile: profile)
        guard let gitExecutable = LingXiPlatform.process.resolveExecutable(named: "git", customSearchPaths: ["/Library/Developer/
↳ CommandLineTools/usr/bin", "/usr/bin", "/usr/local/bin"]) else {
            throw CoreError(code: .gitError, message: "未找到可执行的 git 命令")
        }
        let setup = try processSetup(executable: gitExecutable, arguments: command, workspace: workspace, cwd: directory, profile: profile)
        let result = try await runToolProcess(invocation: setup.0, cwd: directory, environment: setup.1, timeoutMilliseconds: 60_000,
↳ lifecycleTrace: ToolExecutionContext.lifecycleTrace)
        guard result.exitCode == 0 else { throw CoreError(code: .gitError, message: try json(result)) }
        return try json(result)
    }
}

public actor ToolProcessStore {
    private var processes: [String: ManagedToolProcess] = [:]

    public init() {}

    func start(id: String, invocation: ToolProcessInvocation, cwd: URL, environment: [String: String], lifecycleTrace: ToolLifecycleTrace? =
↳ nil) throws -> ProcessStatus {
        guard processes[id] == nil else { throw CoreError(code: .toolArgumentInvalid, message: "进程 ID 已存在: \(id)") }

```

```

    let process = ManagedToolProcess(invocation: invocation, cwd: cwd, environment: environment, lifecycleTrace: lifecycleTrace)
    try process.launch()
    processes[id] = process
    return process.snapshot(id: id, stdoutCursor: nil, stderrCursor: nil)
}

func poll(id: String, stdoutCursor: Int?, stderrCursor: Int?) throws -> ProcessStatus {
    guard let process = processes[id] else { throw CoreError(code: .processNotFound, message: "进程不存在: \(id)") }
    return process.snapshot(id: id, stdoutCursor: stdoutCursor, stderrCursor: stderrCursor)
}

func input(id: String, text: String, stdoutCursor: Int?, stderrCursor: Int?) throws -> ProcessStatus {
    guard let process = processes[id] else { throw CoreError(code: .processNotFound, message: "进程不存在: \(id)") }
    try process.write(text)
    return process.snapshot(id: id, stdoutCursor: stdoutCursor, stderrCursor: stderrCursor)
}

func stop(id: String, stdoutCursor: Int?, stderrCursor: Int?) throws -> ProcessStatus {
    guard let process = processes[id] else { throw CoreError(code: .processNotFound, message: "进程不存在: \(id)") }
    process.terminate()
    return process.snapshot(id: id, stdoutCursor: stdoutCursor, stderrCursor: stderrCursor)
}

func stopAll() async {
    for process in processes.values { process.terminate() }
    for process in processes.values { await process.waitForExit() }
    processes.removeAll()
}
}

public struct ProcessTool: ToolExecutor {
    private let store: ToolProcessStore
    private let workspace: WorkspaceRoot
    public init(workspace: WorkspaceRoot, store: ToolProcessStore = ToolProcessStore()) {
        self.workspace = workspace
        self.store = store
    }

    public let definition = ToolDefinition(
        id: ToolID("process"), description: "Start, poll, provide input to, or stop a long-running process.",
        inputSchema: ToolInputSchema(properties: [
            "action": ToolInputProperty(type: .string, description: "start, poll, input, or stop", enumValues: ["start", "poll", "input", "
↪ stop", "status", "terminate"]),
            "executable": ToolInputProperty(type: .string, description: "Absolute executable path for start"),
            "arguments": ToolInputProperty(type: .array, description: "Executable argv for start"),
            "cwd": ToolInputProperty(type: .string, description: "Workspace-relative working directory"),
            "id": ToolInputProperty(type: .string, description: "Process ID"),
            "input": ToolInputProperty(type: .string, description: "UTF-8 stdin data"),
            "stdout_cursor": ToolInputProperty(type: .integer, description: "stdout cursor", minimum: 0),
            "stderr_cursor": ToolInputProperty(type: .integer, description: "stderr cursor", minimum: 0)
        ], required: ["action"]), capability: ToolCapability([.processExecute])
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: ProcessArguments = try decodeArguments(arguments)
        return try cwd(input.cwd, workspace: workspace, profile: profile).path
    }

    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: ProcessArguments = try decodeArguments(arguments)
        let id = input.id ?? UUID().uuidString
        let status: ProcessStatus
        switch input.action {
        case "start":
            guard let executable = input.executable else { throw CoreError(code: .toolArgumentInvalid, message: "start 需要 executable") }
            let directory = try cwd(input.cwd, workspace: workspace, profile: profile)
            let setup = try processSetup(executable: executable, arguments: input.arguments ?? [], workspace: workspace, cwd: directory,
↪ profile: profile)

```

```

        status = try await store.start(id: id, invocation: setup.0, cwd: directory, environment: setup.1, lifecycleTrace:
↳ ToolExecutionContext.lifecycleTrace)
    case "poll", "status": status = try await store.poll(id: id, stdoutCursor: input.stdoutCursor, stderrCursor: input.stderrCursor)
    case "input":
        guard let text = input.input else { throw CoreError(code: .toolArgumentInvalid, message: "input 需要 input") }
        status = try await store.input(id: id, text: text, stdoutCursor: input.stdoutCursor, stderrCursor: input.stderrCursor)
    case "stop", "terminate": status = try await store.stop(id: id, stdoutCursor: input.stdoutCursor, stderrCursor: input.stderrCursor)
    default: throw CoreError(code: .toolArgumentInvalid, message: "未知 process action: \$(input.action)")
    }
    return try json(status)
}
}

public struct QuestionTool: ToolExecutor {
    private let questions: QuestionRuntime?

    public init(questions: QuestionRuntime? = nil) {
        self.questions = questions
    }

    public let definition = ToolDefinition(
        id: ToolID("question"), description: "Ask the user to choose from provided options.",
        inputSchema: ToolInputSchema(properties: [
            "question": ToolInputProperty(type: .string, description: "Question for the user"),
            "options": ToolInputProperty(type: .array, description: "Optional choices"),
            "multiple": ToolInputProperty(type: .boolean, description: "Whether multiple choices are allowed")
        ], required: ["question"]), capability: ToolCapability([.userInteraction])
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: QuestionArguments = try decodeArguments(arguments)
        return input.question
    }

    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        guard let questions else {
            throw CoreError(code: .questionUnavailable, message: "当前 transport 不支持交互式问题")
        }
        let input: QuestionArguments = try decodeArguments(arguments)
        let request = QuestionRequest(
            questionID: QuestionID(UUID().uuidString),
            question: input.question,
            options: input.options ?? [],
            allowsMultiple: input.multiple ?? false
        )
        let reply = try await questions.ask(request)
        let selectedOptions = reply.selectedOptionIndices.map { request.options[$0] }
        return try json(QuestionToolResult(
            questionID: request.questionID.rawValue,
            cancelled: reply.cancelled,
            selectedOptions: selectedOptions,
            text: reply.text
        ))
    }
}

private struct QuestionToolResult: Codable {
    let questionID: String
    let cancelled: Bool
    let selectedOptions: [String]
    let text: String?
}

private struct SkillTool: ToolExecutor {
    let workspace: WorkspaceRoot

    var definition: ToolDefinition {
        let names = Self.names(in: workspace)
    }
}

```

```

        return ToolDefinition(
            id: ToolID("skill"),
            description: names.isEmpty ? "Load a workspace skill by name. No skills are currently available." : "Load a workspace skill by
↪ name. Available: \"(names.join(separator: ", ")).\"",
            inputSchema: ToolInputSchema(properties: ["name": ToolInputProperty(type: .string, description: "Available skill name",
↪ enumValues: names)], required: ["name"]),
            capability: ToolCapability(readOnly: true)
        )
    }

    init(workspace: WorkspaceRoot) {
        self.workspace = workspace
    }

    func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: SkillArguments = try decodeArguments(arguments)
        return try skillFile(input.name, profile: profile).path
    }

    func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: SkillArguments = try decodeArguments(arguments)
        return try readText(skillFile(input.name, profile: profile), operation: "skill")
    }

    private func skillFile(_ name: String, profile: ExecutionProfile) throws -> URL {
        guard !name.isEmpty, name.allSatisfy({ $0.isLetter || $0.isNumber || "-_".contains($0) }) else {
            throw CoreError(code: .toolArgumentInvalid, message: "Skill 名称无效")
        }
        let file = try workspace.resolve(".lingxi/skills/\(name)/SKILL.md", profile: profile)
        guard FileManager.default.fileExists(atPath: file.path) else {
            throw CoreError(code: .resourceNotFound, message: "Skill 不存在: \"(name)\"")
        }
        return file
    }

    private static func names(in workspace: WorkspaceRoot) -> [String] {
        let root = workspace.url.appendingPathComponent(".lingxi/skills", isDirectory: true)
        return ((try? FileManager.default.contentsOfDirectory(at: root, includingPropertiesForKeys: [.isDirectoryKey], options: [.
↪ skipsHiddenFiles])) ?? [])
            .filter { FileManager.default.fileExists(atPath: $0.appendingPathComponent("SKILL.md").path) }
            .map{\.lastPathComponent}
            .sorted()
    }

    }

    private struct IndexResult: Codable {
        let id: String
        let path: String
        let name: String
        let kind: String
        let line: Int
    }

    private struct ProjectIndexTool: ToolExecutor {
        enum Kind { case symbols, references, dependencies }

        let definition: ToolDefinition
        let kind: Kind
        let pager: ContextPager
        let scanner: ProjectScanner

        func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
            let input: SymbolArguments = try decodeArguments(arguments)
            return input.symbol
        }
    }

```

```

func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input: SymbolArguments = try decodeArguments(arguments)
    _ = try await pager.rebuildStaleFiles(using: scanner)
    switch kind {
    case .symbols:
        let symbols = await pager.symbolLookup(projectRoot: scanner.root, query: input.symbol, mode: input.mode ?? "prefix")
        return try json(symbols.map { IndexResult(id: $0.id.rawValue, path: $0.path, name: $0.qualifiedName, kind: $0.kind.rawValue,
↵ line: $0.line) })
    case .references:
        let symbols = await pager.symbolLookup(projectRoot: scanner.root, query: input.symbol, mode: input.mode ?? "qualified")
        guard symbols.count == 1 else {
            throw CoreError(code: .resourceNotFound, message: symbols.isEmpty ? "未找到 Symbol: \(input.symbol)" : "Symbol 不唯一: \(
↵ input.symbol)")
        }
        let references = await pager.references(projectRoot: scanner.root, symbolID: symbols[0].id)
        return try json(references.map { IndexResult(id: $0.id.rawValue, path: $0.sourcePath, name: $0.targetName, kind: $0.
↵ resolutionQuality.rawValue, line: $0.sourceLine) })
    case .dependencies:
        let edges = await pager.dependencies(projectRoot: scanner.root, path: input.symbol, incoming: input.direction == "incoming")
        return try json(edges.map { IndexResult(id: $0.evidence.rawValue, path: $0.sourcePath, name: $0.targetPath ?? $0.targetModule ??
↵ "", kind: $0.kind.rawValue, line: 0) })
    }
}

private struct CodeIntelligenceTool: ToolExecutor {
    let definition: ToolDefinition
    let intelligence: CodeIntelligence
    let workspace: WorkspaceRoot

    func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: CodeIntelligenceArguments = try decodeArguments(arguments)
        return input.path ?? input.query ?? workspace.url.path
    }

    func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: CodeIntelligenceArguments = try decodeArguments(arguments)
        switch input.action {
        case "symbols": return try json(await intelligence.symbols(input.query ?? ""))
        case "definition": return try json(await intelligence.definition(path: try required(input.path, "path"), line: try required(input.
↵ line, "line"), character: input.character ?? 0))
        case "references": return try json(await intelligence.references(path: try required(input.path, "path"), line: try required(input.
↵ line, "line"), character: input.character ?? 0))
        case "document_symbols": return try json(await intelligence.documentSymbols(path: try required(input.path, "path")))
        case "diagnostics": return try json(await intelligence.diagnostics(path: try required(input.path, "path")))
        case "repo_map": return await intelligence.repoMap()
        case "context": return try json(await intelligence.context(input.query ?? "", maximumCharacters: min(max(0, input.maximumCharacters
↵ ?? 16_000), 32_768)))
        default: throw CoreError(code: .toolArgumentInvalid, message: "未知 code_intelligence action")
        }
    }

    private func required<T>(_ value: T?, _ name: String) throws -> T {
        guard let value else { throw CoreError(code: .toolArgumentInvalid, message: "code_intelligence 缺少 \(name)") }
        return value
    }
}

private struct ContextRetrieveArguments: Codable {
    let query: String
    let limit: Int?
}

package struct ContextRetrieveTool: ToolExecutor {

```

```

package let definition: ToolDefinition
let cacheController: ContextCacheController

package init(id: String = "context_search", cacheController: ContextCacheController) {
    self.cacheController = cacheController
    self.definition = ToolDefinition(
        id: ToolID(id),
        description: "Search and retrieve relevant context from the project codebase index or compacted session history. The runtime
↪ cache controller evaluates weighted priority and pages the highest relevance entries into the L1 working set.",
        inputSchema: ToolInputSchema(
            properties: [
                "query": ToolInputProperty(type: .string, description: "Search query describing what context to retrieve"),
                "limit": ToolInputProperty(type: .integer, description: "Maximum items to retrieve (default: 5)", minimum: 1, maximum:
↪ 20)
            ],
            required: ["query"]
        ),
        capability: ToolCapability(readOnly: true)
    )
}

package func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
    let input: ContextRetrieveArguments = try decodeArguments(arguments)
    return input.query
}

package func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input: ContextRetrieveArguments = try decodeArguments(arguments)
    let limit = min(max(1, input.limit ?? 5), 20)
    let sessionId = ToolExecutionContext.sessionID ?? SessionID("default")
    return try await cacheController.handleSearch(
        sessionId: sessionId,
        query: input.query,
        activeTask: "",
        activeFiles: [],
        limit: limit
    )
}

private struct TodoArguments: Codable {
    let action: String
    let id: String?
    let title: String?
    let status: String?
}

private struct TodoMutationResponse: Codable {
    let status: String
    let task: TodoItemData?
    let id: String?
    let newStatus: String?
    let message: String?
    init(status: String, task: TodoItemData? = nil, id: String? = nil, newStatus: String? = nil, message: String? = nil) {
        self.status = status
        self.task = task
        self.id = id
        self.newStatus = newStatus
        self.message = message
    }
}

private struct TodoListResponse: Codable {
    let status: String
    let tasks: [TodoItemData]
}

```



```

}

public struct TodoTool: ToolExecutor {
    public let definition = ToolDefinition(
        id: ToolID("todo"),
        description: "Manage task and to-do items for the current session. For multi-step tasks, comprehensive health checks, environment
↪ diagnostics, or refactoring, ALWAYS use this tool first (action: 'add') to establish a checklist, then update status ('in_progress
↪ ', 'completed', 'failed') as you proceed to maintain real-time visibility on the sidebar.",
        inputSchema: ToolInputSchema(
            properties: [
                "action": ToolInputProperty(type: .string, description: "Action to perform: add, update, list, clear", enumValues: ["add", "
↪ update", "list", "clear"]),
                "id": ToolInputProperty(type: .string, description: "Unique task ID, e.g. task-1"),
                "title": ToolInputProperty(type: .string, description: "Task title or description"),
                "status": ToolInputProperty(type: .string, description: "Task status", enumValues: ["pending", "in_progress", "completed", "
↪ failed"])
            ],
            required: ["action"]
        ),
        capability: ToolCapability(readOnly: false)
    )

    public init() {}

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input: TodoArguments = try decodeArguments(arguments)
        return "\(input.action): \(input.title ?? input.id ?? "")"
    }

    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input: TodoArguments = try decodeArguments(arguments)
        let sessionKey = ToolExecutionContext.sessionID?.rawValue ?? "default"
        switch input.action {
        case "add":
            guard let title = input.title, !title.isEmpty else {
                throw CoreError(code: .toolArgumentInvalid, message: "add 需要 title")
            }
            let taskID = input.id ?? UUID().uuidString.prefix(6).lowercased()
            let status = input.status ?? "pending"
            let item = TodoItemData(id: String(taskID), title: title, status: status)
            TodoStore.shared.addTo(item, for: sessionKey)
            return try json(TodoMutationResponse(status: "ok", task: item))
        case "update":
            guard let id = input.id, !id.isEmpty else {
                throw CoreError(code: .toolArgumentInvalid, message: "update 需要 id")
            }
            let status = input.status ?? "in_progress"
            let ok = TodoStore.shared.updateTodo(id: id, status: status, title: input.title, for: sessionKey)
            return try json(TodoMutationResponse(status: ok ? "ok" : "not_found", id: id, newStatus: status))
        case "list":
            let items = TodoStore.shared.getTodos(for: sessionKey)
            return try json(TodoListResponse(status: "ok", tasks: items))
        case "clear":
            TodoStore.shared.clear(for: sessionKey)
            return try json(TodoMutationResponse(status: "ok", message: "Todos cleared"))
        default:
            throw CoreError(code: .toolArgumentInvalid, message: "不支持的 action: \(input.action)")
        }
    }
}

public extension BuiltInToolProvider {
    init(workspace: WorkspaceRoot, contextPager: ContextPager? = nil, scanner: ProjectScanner? = nil, questions: QuestionRuntime? = nil,
↪ processes: ToolProcessStore? = nil, codeIntelligence: CodeIntelligence? = nil, cacheController: ContextCacheController? = nil,
↪ webSearchEndpoint: URL? = nil) {

```

```

let indexTools: [any ToolExecutor]
if let contextPager, let scanner {
    indexTools = [
        ProjectIndexTool(definition: ToolDefinition(id: ToolID("symbol_lookup"), description: "Look up a symbol in the configured
↪ code index.", inputSchema: ToolInputSchema(properties: ["symbol": ToolInputProperty(type: .string, description: "Symbol name"), "
↪ mode": ToolInputProperty(type: .string, description: "exact, qualified, or prefix", enumValues: ["exact", "qualified", "prefix"])),
↪ required: ["symbol"]), capability: ToolCapability(readOnly: true)), kind: .symbols, pager: contextPager, scanner: scanner),
        ProjectIndexTool(definition: ToolDefinition(id: ToolID("find_references"), description: "Find incoming references for a
↪ unique indexed symbol.", inputSchema: ToolInputSchema(properties: ["symbol": ToolInputProperty(type: .string, description: "Symbol
↪ query"), "mode": ToolInputProperty(type: .string, description: "exact or qualified", enumValues: ["exact", "qualified"])), required
↪ : ["symbol"]), capability: ToolCapability(readOnly: true)), kind: .references, pager: contextPager, scanner: scanner),
        ProjectIndexTool(definition: ToolDefinition(id: ToolID("dependency_query"), description: "Query incoming or outgoing indexed
↪ file dependencies.", inputSchema: ToolInputSchema(properties: ["symbol": ToolInputProperty(type: .string, description: "Project-
↪ relative file path"), "direction": ToolInputProperty(type: .string, description: "incoming or outgoing", enumValues: ["incoming", "
↪ outgoing"])), required: ["symbol"]), capability: ToolCapability(readOnly: true)), kind: .dependencies, pager: contextPager, scanner
↪ : scanner),
    ]
} else {
    indexTools = []
}

let intelligenceTools: [any ToolExecutor] = codeIntelligence.map { intelligence in
    [CodeIntelligenceTool(definition: ToolDefinition(id: ToolID("code_intelligence"), description: "Structured code intelligence.
↪ Prefer this for symbols, definitions, references, diagnostics, repository map, and bounded context; it falls back safely when LSP
↪ is unavailable.", inputSchema: ToolInputSchema(properties: ["action": ToolInputProperty(type: .string, description: "symbols,
↪ definition, references, document_symbols, diagnostics, repo_map, or context", enumValues: ["symbols", "definition", "references", "
↪ document_symbols", "diagnostics", "repo_map", "context"]), "query": ToolInputProperty(type: .string, description: "Symbol or task
↪ query"), "path": ToolInputProperty(type: .string, description: "Workspace-relative source path"), "line": ToolInputProperty(type: .
↪ integer, description: "One-based line", minimum: 1), "character": ToolInputProperty(type: .integer, description: "Zero-based
↪ character", minimum: 0), "maximum_characters": ToolInputProperty(type: .integer, description: "Bounded context character limit",
↪ minimum: 0, maximum: 32_768)), required: ["action"]), capability: ToolCapability(readOnly: true)), intelligence: intelligence,
↪ workspace: workspace)]
} ?? []

self.init(tools: [
    ReadFileTool(workspace: workspace),
    ContextRecallTool(ecoreStore: cacheController?.ecoreStore),
    ListDirectoryTool(workspace: workspace),
    GlobTool(workspace: workspace),
    GrepTool(workspace: workspace),
    WriteFileTool(workspace: workspace),
    EditFileTool(workspace: workspace),
    ApplyPatchTool(workspace: workspace),
    ShellTool(workspace: workspace),
    WebSearchTool(endpoint: webSearchEndpoint),
    WebFetchTool(),
    ProcessTool(workspace: workspace, store: processes ?? ToolProcessStore()),
    GitTool(workspace: workspace),
    SkillTool(workspace: workspace),
    QuestionTool(questions: questions),
    TodoTool()
] + indexTools + intelligenceTools)
}
}

public extension ToolRegistry {
    static func builtin(workspace: WorkspaceRoot, contextPager: ContextPager? = nil, scanner: ProjectScanner? = nil, questions:
↪ QuestionRuntime? = nil, processes: ToolProcessStore? = nil, codeIntelligence: CodeIntelligence? = nil, cacheController:
↪ ContextCacheController? = nil, webSearchEndpoint: URL? = nil) -> ToolRegistry {
        ToolRegistry(BuiltInToolProvider(workspace: workspace, contextPager: contextPager, scanner: scanner, questions: questions, processes
↪ : processes, codeIntelligence: codeIntelligence, cacheController: cacheController, webSearchEndpoint: webSearchEndpoint).tools)
    }
}

```

// File: Sources/LingXiCore/Modules/Tool/WebTools.swift

import Foundation

```

#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

private struct WebSearchInput: Decodable {
    let query: String
    let maxResults: Int?
}

private struct WebFetchInput: Decodable {
    let url: String
    let maxBytes: Int?
}

private func webArguments<T: Decodable>(_ arguments: String, as type: T.Type) throws -> T {
    guard let data = arguments.data(using: .utf8) else {
        throw CoreError(code: .toolArgumentInvalid, message: "Web Tool 参数不是 UTF-8")
    }
    do {
        let decoder = JSONDecoder()
        decoder.keyDecodingStrategy = .convertFromSnakeCase
        return try decoder.decode(T.self, from: data)
    } catch {
        throw CoreError(code: .toolArgumentInvalid, message: "Web Tool 参数无效: \(error.localizedDescription)")
    }
}

private func webURL(_ raw: String) throws -> URL {
    guard let url = URL(string: raw), let scheme = url.scheme?.lowercased(), scheme == "https" || scheme == "http" else {
        throw CoreError(code: .toolArgumentInvalid, message: "Web URL 必须使用 HTTPS 或 HTTP")
    }
    if scheme == "http" && !(url.host == "127.0.0.1" || url.host == "localhost" || url.host == "::1") {
        throw CoreError(code: .toolArgumentInvalid, message: "非 loopback Web URL 必须使用 HTTPS")
    }
    return url
}

public struct WebFetchTool: ToolExecutor {
    private let session: URLSession

    public init(session: URLSession = .shared) {
        self.session = session
    }

    public let definition = ToolDefinition(
        id: ToolID("web_fetch"),
        description: "Fetch a bounded web page projection from an HTTPS URL.",
        inputSchema: ToolInputSchema(
            properties: [
                "url": ToolInputProperty(type: .string, description: "HTTPS URL"),
                "max_bytes": ToolInputProperty(type: .integer, description: "Maximum response bytes", minimum: 1, maximum: 32_768)
            ],
            required: ["url"]
        ),
        capability: ToolCapability(readOnly: true)
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        try webURL(webArguments(arguments, as: WebFetchInput.self).url).absoluteString
    }

    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        [.networkAccess]
    }
}

```

```

public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
    let input = try webArguments(arguments, as: WebFetchInput.self)
    let url = try webURL(input.url)
    let request = URLRequest(url: url, cachePolicy: .reloadIgnoringLocalCacheData, timeoutInterval: 20)
    let (data, response) = try await session.data(for: request)
    guard let http = response as? HTTPURLResponse else {
        throw CoreError(code: .toolExecutionFailed, message: "Web fetch 没有 HTTP 响应")
    }
    let limit = min(max(1, input.maxBytes ?? 32_768), 32_768)
    let bounded = data.prefix(limit)
    let content = String(decoding: bounded, as: UTF8.self)
    let result: [String: Any] = [
        "url": url.absoluteString,
        "status": http.statusCode,
        "content": content,
        "truncated": data.count > limit
    ]
    guard JSONSerialization.isValidJSONObject(result), let encoded = try? JSONSerialization.data(withJSONObject: result, options: [.
↵ sortedKeys]) else {
        throw CoreError(code: .toolExecutionFailed, message: "Web fetch 结果无法编码")
    }
    return String(decoding: encoded, as: UTF8.self)
}
}

public struct WebSearchTool: ToolExecutor {
    private let session: URLSession
    private let endpoint: URL?

    public init(session: URLSession = .shared, endpoint: URL? = nil) {
        self.session = session
        self.endpoint = endpoint
    }

    public let definition = ToolDefinition(
        id: ToolID("web_search"),
        description: "Search the configured web backend and return bounded structured snippets.",
        inputSchema: ToolInputSchema(
            properties: [
                "query": ToolInputProperty(type: .string, description: "Search query"),
                "max_results": ToolInputProperty(type: .integer, description: "Maximum snippets", minimum: 1, maximum: 10)
            ],
            required: ["query"]
        ),
        capability: ToolCapability(readOnly: true)
    )

    public func resource(for arguments: String, profile: ExecutionProfile) throws -> String {
        let input = try webArguments(arguments, as: WebSearchInput.self)
        return "web_search:\(input.query)"
    }

    public func capabilities(for arguments: String, profile: ExecutionProfile) throws -> Set<ToolCapabilityKind> {
        [.networkAccess]
    }

    public func execute(arguments: String, profile: ExecutionProfile) async throws -> String {
        let input = try webArguments(arguments, as: WebSearchInput.self)
        let query = input.query.trimmingCharacters(in: .whitespacesAndNewlines)
        guard !query.isEmpty else { throw CoreError(code: .toolArgumentInvalid, message: "web_search query 不能为空") }
        guard let configured = endpoint else {
            throw CoreError(code: .toolExecutionFailed, message: "web_search 未配置 backend")
        }
        let base = try webURL(configured.absoluteString)

```

```

var components = URLComponents(url: base, resolvingAgainstBaseURL: false)
let existingQueryItems = components?.queryItems ?? []
components?.queryItems = existingQueryItems + [
    URLQueryItem(name: "q", value: query),
    URLQueryItem(name: "limit", value: String(min(max(1, input.maxResults ?? 5), 10)))
]
guard let url = components?.url else { throw CoreError(code: .toolExecutionFailed, message: "web_search backend URL 无效") }
let (data, response) = try await session.data(for: URLRequest(url: url, timeoutInterval: 20))
guard let http = response as? HTTPURLResponse, (200..<300).contains(http.statusCode) else {
    throw CoreError(code: .toolExecutionFailed, message: "web_search backend 请求失败")
}
let object = try JSONSerialization.jsonObject(with: data)
let rawItems: [[String: Any]]
if let items = object as? [[String: Any]] { rawItems = items }
else if let dict = object as? [String: Any], let items = (dict["results"] ?? dict["items"]) as? [[String: Any]] { rawItems = items }
else { throw CoreError(code: .toolExecutionFailed, message: "web_search backend 返回格式无效") }
let limit = min(max(1, input.maxResults ?? 5), 10)
let results: [[String: String]] = rawItems.prefix(limit).map { item in
    let title = String((item["title"] as? String ?? "").prefix(240))
    let snippet = String((item["snippet"] as? String ?? item["description"] as? String ?? "").prefix(600))
    let url = item["url"] as? String ?? item["link"] as? String ?? ""
    return ["title": title, "snippet": snippet, "url": url]
}
let output: [String: Any] = ["query": query, "total": results.count, "results": results]
guard let encoded = try? JSONSerialization.data(withJSONObject: output, options: [.sortedKeys]) else {
    throw CoreError(code: .toolExecutionFailed, message: "web_search 结果无法编码")
}
return String(decoding: encoded, as: UTF8.self)
}
}

```

// File: Sources/LingXiCore/Modules/Tool/ToolMutationCoordinator.swift

```

import Foundation

private actor MutationGate {
    private var acquired = false
    private var waiters: [CheckedContinuation<Void, Never>] = []

    func execute(_ operation: @escaping @Sendable () async throws -> String) async throws -> String {
        if acquired {
            await withCheckedContinuation { waiters.append($0) }
        } else {
            acquired = true
        }
        do {
            let result = try await operation()
            release()
            return result
        } catch {
            release()
            throw error
        }
    }

    private func release() {
        if waiters.isEmpty { acquired = false }
        else { waiters.removeFirst().resume() }
    }
}

/// Serializes workspace mutations and immediately refreshes project-backed indexes when configured.
public actor ToolMutationCoordinator {
    // ponytail: global lock, use per-workspace gates only if independent workspaces become throughput-bound.
    private static let gate = MutationGate()

```

```

private let pager: ContextPager?
private let scanner: ProjectScanner?

public init(pager: ContextPager? = nil, scanner: ProjectScanner? = nil) {
    self.pager = pager
    self.scanner = scanner
}

public func reconcile() async throws {
    if let pager, let scanner { _ = try await pager.rebuildStaleFiles(using: scanner) }
}

public func execute(_ operation: @escaping @Sendable () async throws -> String) async throws -> String {
    try await Self.gate.execute { [self] in
        let result = try await operation()
        Task { [weak self] in
            try? await self?.reconcile()
        }
        return result
    }
}
}

```

// File: Sources/LingXiCore/Modules/Tool/ToolOutputArchive.swift

```

import Foundation
import LingXiProtocol

public actor ToolOutputArchive {
    private let persistence: SQLitePersistenceStore?

    public init(persistence: SQLitePersistenceStore?) {
        self.persistence = persistence
    }

    public func archive(_ content: String, metadata: ToolOutputMetadata) async throws -> ToolOutputMetadata {
        guard metadata.truncated, let persistence else { return metadata }
        return ToolOutputMetadata(
            truncated: true,
            totalCharacters: metadata.totalCharacters,
            totalBytes: metadata.totalBytes,
            visibleCharacters: metadata.visibleCharacters,
            visibleBytes: metadata.visibleBytes,
            outputBlobRef: try await persistence.storeToolOutput(content)
        )
    }
}

```

// File: Sources/LingXiCore/Modules/Tool/ToolOutputPolicy.swift

```

import Foundation
import LingXiProtocol

/// Bounds model-visible evidence while preserving complete output in BlobStore when available.
public struct ToolOutputPolicy: Sendable {
    public let maximumCharacters: Int
    public let maximumLines: Int

    public init(maximumCharacters: Int = 16 * 1024, maximumLines: Int = 400) {
        self.maximumCharacters = max(1, maximumCharacters)
        self.maximumLines = max(1, maximumLines)
    }

    public func excerpt(_ text: String) -> (content: String, metadata: ToolOutputMetadata) {
        let lines = text.split(separator: "\n", omittingEmptySubsequences: false)
    }
}

```

```

        let kept = lines.prefix(maximumLines).joined(separator: "\n")
        let content = String(kept.prefix(maximumCharacters))
        return (content, ToolOutputMetadata(truncated: content.utf8.count < text.utf8.count, totalCharacters: text.count, totalBytes: text.
↵ utf8.count, visibleCharacters: content.count, visibleBytes: content.utf8.count))
    }
}

```

// File: Sources/LingXiCore/Modules/MCP/MCPRuntime.swift

```

import Foundation
import LingXiProtocol

/// Phase 12 architectural invariant: MCP schemas are only held by active leases.
public let MCP_FULL_SCHEMA_PERMANENT_RESIDENCY = "FORBIDDEN"

public struct MCPServerID: RawRepresentable, Sendable, Hashable, Codable { public let rawValue: String; public init(_ rawValue: String) {
    ↵ self.rawValue = rawValue }; public init(rawValue: String) { self.init(rawValue) } }
public struct AuthContextID: RawRepresentable, Sendable, Hashable, Codable { public let rawValue: String; public init(_ rawValue: String) {
    ↵ self.rawValue = rawValue }; public init(rawValue: String) { self.init(rawValue) } }
public enum MCPProtocolEra: String, Sendable, Codable { case modern, legacy }
public enum MCPCacheScope: String, Sendable, Codable { case `public`, `private` }
public enum MCPLeaseState: String, Sendable, Codable { case armed, used, expired, revoked }

public enum MCPServerRuntimeStatus: Sendable, Equatable, Codable {
    case ready(toolCount: Int)
    case empty
    case error(reason: String)
    case disabled
}

public struct MCPToolAnnotations: Sendable, Equatable, Codable {
    public let readOnlyHint: Bool?
    public let destructiveHint: Bool?
    public let idempotentHint: Bool?
    public let openWorldHint: Bool?
    public init(readOnlyHint: Bool? = nil, destructiveHint: Bool? = nil, idempotentHint: Bool? = nil, openWorldHint: Bool? = nil) { self.
    ↵ readOnlyHint = readOnlyHint; self.destructiveHint = destructiveHint; self.idempotentHint = idempotentHint; self.openWorldHint =
    ↵ openWorldHint }
}

/// L3 entry: metadata only. The full JSON Schema lives exclusively in MCPToolSchemaStore.
public struct MCPToolCatalogEntry: Sendable, Equatable, Codable {
    public let toolID: ToolID
    public let serverID: MCPServerID
    public let serverAlias: String
    public let upstreamName: String
    public let title: String
    public let shortDescription: String
    public let tags: [String]
    public let annotations: MCPToolAnnotations
    public let schemaHash: String
    public let era: MCPProtocolEra
    public var available: Bool
    public var stale: Bool
    public let cacheScope: MCPCacheScope
    public let authContextID: AuthContextID?
    public var lastSeen: Date
}

public struct MCPDiscoveredTool: Sendable {
    public let entry: MCPToolCatalogEntry
    public let inputSchema: JSONValue
    public init(entry: MCPToolCatalogEntry, inputSchema: JSONValue) { self.entry = entry; self.inputSchema = inputSchema }
}

```

```

public struct MCPToolSearchCandidate: Sendable, Equatable, Codable {
    public let toolID: ToolID
    public let displayName: String
    public let serverAlias: String
    public let shortDescription: String
    public let riskHint: String
    public let availability: String
    public let temperature: String
    fileprivate enum CodingKeys: String, CodingKey { case toolID = "tool_id", displayName, serverAlias, shortDescription, riskHint,
        ↪ availability, temperature }
}

extension MCPToolSearchCandidate {
    /// Wire contract: `load_tool` takes a plain-string `tool_id`; a synthesized
    /// `{"rawValue": ...}` encoding is unusable for the model.
    public init(from decoder: Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)
        toolID = ToolID(try container.decode(String.self, forKey: .toolID))
        displayName = try container.decode(String.self, forKey: .displayName)
        serverAlias = try container.decode(String.self, forKey: .serverAlias)
        shortDescription = try container.decode(String.self, forKey: .shortDescription)
        riskHint = try container.decode(String.self, forKey: .riskHint)
        availability = try container.decode(String.self, forKey: .availability)
        temperature = try container.decode(String.self, forKey: .temperature)
    }

    public func encode(to encoder: Encoder) throws {
        var container = encoder.container(keyedBy: CodingKeys.self)
        try container.encode(toolID.rawValue, forKey: .toolID)
        try container.encode(displayName, forKey: .displayName)
        try container.encode(serverAlias, forKey: .serverAlias)
        try container.encode(shortDescription, forKey: .shortDescription)
        try container.encode(riskHint, forKey: .riskHint)
        try container.encode(availability, forKey: .availability)
        try container.encode(temperature, forKey: .temperature)
    }
}

public struct MCPToolSchemaLease: Sendable, Equatable, Codable {
    public let leaseID: String
    public let sessionID: SessionID
    public let toolID: ToolID
    public let schemaHash: String
    public let providerName: String
    public let createdAt: Date
    public let expiresAt: Date
    public var state: MCPLeaseState
}

public enum MCPToolPagerError: Error, Sendable, Equatable {
    case missingTool, unavailable, schemaMissing, schemaChanged, schemaTooLarge, schemaBudgetExceeded, leaseMissing, leaseExpired,
        ↪ taskUnsupported, catalogTooLarge
}

public protocol MCPToolInvoker: Sendable {
    func call(serverID: MCPServerID, toolName: String, arguments: String) async throws -> String
}

public actor MCPToolSchemaStore {
    private let directory: URL
    public init(directory: URL? = nil) {
        self.directory = directory ?? FileManager.default.temporaryDirectory.appendingPathComponent("lingxi-mcp-schemas-\(UUID().uuidString)
        ↪ ", isDirectory: true)
        try? FileManager.default.createDirectory(at: self.directory, withIntermediateDirectories: true)
        try? FileManager.default.setAttributes([.posixPermissions: 0o700], ofItemAtPath: self.directory.path)
    }
}

```



```

}

public func put(_ schema: JSONValue, toolID: ToolID, hash: String) throws {
    let encoder = JSONEncoder(); encoder.outputFormatting = [.sortedKeys]
    let url = file(toolID, hash)
    try encoder.encode(schema).write(to: url, options: .atomic)
    try FileManager.default.setAttributes([.posixPermissions: 0o600], ofItemAtPath: url.path)
}

public func schema(toolID: ToolID, hash: String) -> JSONValue? {
    guard let data = try? Data(contentsOf: file(toolID, hash)) else { return nil }
    return try? JSONDecoder().decode(JSONValue.self, from: data)
}

public func count() -> Int { files().count }

public func bytes() -> Int { files().reduce(0) { $0 + ((try? $1.resourceValues(forKeys: [.fileSizeKey])).fileSize) ?? 0 } }

public func remove(toolID: ToolID, keepingHash: String? = nil) {
    let keep = keepingHash.map { file(toolID, $0).lastPathComponent }
    for url in files() where url.lastPathComponent.hasPrefix(prefix(toolID)) && url.lastPathComponent != keep { try? FileManager.default
        ↪ .removeItem(at: url) }
}

private func files() -> [URL] { (try? FileManager.default.contentsOfDirectory(at: directory, includingPropertiesForKeys: [.fileSizeKey])
    ↪ ) ?? [] }

private func prefix(_ id: ToolID) -> String { sha256Hex(id.rawValue) + "-" }

private func file(_ id: ToolID, _ hash: String) -> URL { directory.appendingPathComponent(prefix(id) + sha256Hex(hash) + ".json") }
}

public struct ProviderToolNameCodec: Sendable {
    public init() {}

    public func encode(serverAlias: String, upstreamName: String, toolID: ToolID) -> String {
        let base = (serverAlias + "_" + upstreamName).lowercased().map { $0.isLetter || $0.isNumber || $0 == "_" ? $0 : "-" }
        return String(base.prefix(48)) + "_" + String(sha256Hex(toolID.rawValue).prefix(10))
    }
}

public actor MCPToolPager {
    private struct SessionState { var l1: [ToolID: Int] = [:]; var candidates: Set<ToolID> = []; var leases: [String: MCPToolSchemaLease] =
        ↪ [:]; var presented: Set<String> = [] }

    private let schemas: MCPToolSchemaStore
    private let codec = ProviderToolNameCodec()
    private let maxSchemaBytes: Int
    private let maxSchemaDepth: Int
    private let maxCatalogTools: Int
    private let invoker: (any MCPToolInvoker)?
    private var catalog: [ToolID: MCPToolCatalogEntry] = [:]
    private var sessions: [SessionID: SessionState] = [:]
    private var l2: [ProjectID: [ToolID: Int]] = [:]
    private var providerSchemaCounts: [SessionID: [Int]] = [:]
    private var serverStatuses: [MCPServerID: MCPServerRuntimeStatus] = [:]
    private var serverAliases: [MCPServerID: String] = [:]
    public private(set) var pageFaults = 0

    public init(schemaStore: MCPToolSchemaStore = MCPToolSchemaStore(), invoker: (any MCPToolInvoker)? = nil, maxSchemaBytes: Int = 128 *
        ↪ 1024, maxSchemaDepth: Int = 64, maxCatalogTools: Int = 1_000) {
        self.schemas = schemaStore; self.invoker = invoker; self.maxSchemaBytes = maxSchemaBytes; self.maxSchemaDepth = maxSchemaDepth; self
        ↪ .maxCatalogTools = maxCatalogTools
    }

    public func recordServerStatus(_ status: MCPServerRuntimeStatus, for serverID: MCPServerID, alias: String? = nil) {
        serverStatuses[serverID] = status
        if let alias { serverAliases[serverID] = alias }
    }

    public func serverStatus(for serverID: MCPServerID) -> MCPServerRuntimeStatus? {
        serverStatuses[serverID]
    }

    public func allServerStatuses() -> [MCPServerID: MCPServerRuntimeStatus] {

```

```

    serverStatuses
}

public var hasAvailableTools: Bool { !catalog.isEmpty }

public func canHandle(sessionID: SessionID, providerToolID: ToolID) -> Bool {
    guard !providerToolID.rawValue.isEmpty else { return false }
    if let session = sessions[sessionID], session.leases.values.contains(where: { $0.providerName == providerToolID.rawValue }) {
        return true
    }
    return catalog.keys.contains(providerToolID)
}

/// Catalog update is all-or-nothing at the caller boundary: callers pass only a completed tools/list generation.
public func replaceCatalog(serverID: MCPServerID, tools: [MCPDiscoveredTool]) async throws {
    let old = catalog.values.filter { $0.serverID == serverID }.map(\.toolID)
    guard Set(tools.map { $0.entry.toolID }).count == tools.count else { throw MCPToolPagerError.missingTool }
    guard tools.allSatisfy({ catalog[$0.entry.toolID]?.serverID == nil || catalog[$0.entry.toolID]?.serverID == serverID }) else { throw
    ↵ MCPToolPagerError.missingTool }
    guard catalog.count - old.count + tools.count <= maxCatalogTools else { throw MCPToolPagerError.catalogTooLarge }
    for tool in tools {
        let encoded = try JSONEncoder().encode(tool.inputSchema)
        guard encoded.count <= maxSchemaBytes, depth(tool.inputSchema) <= maxSchemaDepth else { throw MCPToolPagerError.schemaTooLarge }
    }
    for tool in tools {
        try await schemas.put(tool.inputSchema, toolID: tool.entry.toolID, hash: tool.entry.schemaHash)
    }
    let incoming = Dictionary(uniqueKeysWithValues: tools.map { ($0.entry.toolID, $0.entry.schemaHash) })
    for id in old { catalog.removeValue(forKey: id); removeReferences(id); await schemas.remove(toolID: id, keepingHash: incoming[id]) }
    for tool in tools { catalog[tool.entry.toolID] = tool.entry }
    if tools.isEmpty {
        serverStatuses[serverID] = .empty
    } else {
        serverStatuses[serverID] = .ready(toolCount: tools.count)
    }
}

public func markServerUnavailable(serverID: MCPServerID) {
    serverStatuses[serverID] = .error(reason: "Server unavailable")
    for (id, entry) in catalog where entry.serverID == serverID {
        var updated = entry
        updated.available = false
        updated.stale = true
        catalog[id] = updated
    }
    for sessionID in sessions.keys {
        guard var state = sessions[sessionID] else { continue }
        state.leases = state.leases.filter { lease in
            if let entry = catalog[lease.value.toolID], entry.serverID == serverID {
                return false
            }
            return true
        }
        state.candidates = state.candidates.filter { toolID in
            guard let entry = catalog[toolID] else { return true }
            return entry.serverID != serverID
        }
        sessions[sessionID] = state
    }
}

@discardableResult
public func markToolAndServerUnavailable(providerToolID: ToolID) -> MCPServerID? {
    let matched = catalog.values.first(where: {
        $0.toolID == providerToolID ||

```

```

        codec.encode(serverAlias: $0.serverAlias, upstreamName: $0.upstreamName, toolID: $0.toolID) == providerToolID.rawValue
    })
    guard let entry = matched else { return nil }
    markServerUnavailable(serverID: entry.serverID)
    return entry.serverID
}

public func serverForTool(providerToolID: ToolID) -> MCPServerID? {
    catalog.values.first(where: {
        $0.toolID == providerToolID ||
        codec.encode(serverAlias: $0.serverAlias, upstreamName: $0.upstreamName, toolID: $0.toolID) == providerToolID.rawValue
    })?.serverID
}

public func annotations(for toolID: ToolID) -> MCPToolAnnotations? {
    if let entry = catalog[toolID] {
        return entry.annotations
    }
    return catalog.values.first(where: {
        codec.encode(serverAlias: $0.serverAlias, upstreamName: $0.upstreamName, toolID: $0.toolID) == toolID.rawValue
    })?.annotations
}

public func search(sessionID: SessionID, projectID: ProjectID, query: String, server: String? = nil, capability: String? = nil,
    ↪ maxResults: Int = 6) -> [MCPToolSearchCandidate] {
    // Models commonly pass "" for optional filters; an empty filter must mean "unfiltered", not "matches empty alias".
    let server = server?.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty == true ? nil : server?.trimmingCharacters(in: .
    ↪ whitespacesAndNewlines)
    let capability = capability?.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty == true ? nil : capability?.trimmingCharacters(
    ↪ in: .whitespacesAndNewlines)
    let terms = query.lowercased().split(whereSeparator: { !$0.isLetter && !$0.isNumber }).map(String.init)
    let session = sessions[sessionID] ?? SessionState()
    let project = l2[projectID] ?? []
    var matched: [(entry: MCPToolCatalogEntry, score: Int)] = []

    for entry in catalog.values {
        guard entry.available else {
            // 排除已下线/不可用的 MCP 服务工具，严禁再次暴露给模型引发反复试探
            continue
        }
        if let server = server {
            let s = server.lowercased()
            let a = entry.serverAlias.lowercased()
            let id = entry.serverID.rawValue.lowercased()
            guard a == s || a.contains(s) || s.contains(a) || id == s || id.contains(s) else {
                continue
            }
        }
        if let capability = capability, !entry.tags.contains(capability) {
            continue
        }

        let text = "\(entry.serverAlias) \(entry.serverID.rawValue) \(entry.upstreamName) \(entry.title) \(entry.shortDescription) \("
        ↪ entry.tags.joined(separator: " ").lowercased()
        var lexical = 0
        for term in terms {
            if text.contains(term) {
                lexical += 100
            }
        }
        let a = entry.serverAlias.lowercased()
        if a == term || a.contains(term) || term.contains(a) {
            lexical += 250
        }
        let name = entry.upstreamName.lowercased()
        if name == term || name.contains(term) {

```

```

        lexical += 150
    }
}

let boost = (session.l1[entry.toolID] ?? 0) * 4 + (project[entry.toolID] ?? 0) * 2
if terms.isEmpty || lexical >= 100 {
    matched.append((entry, lexical + boost))
}
}

matched.sort { $0.score == $1.score ? $0.entry.toolID.rawValue < $1.entry.toolID.rawValue : $0.score > $1.score }
let limit = max(1, min(maxResults, 24))
var result = Array(matched.prefix(limit)).map { item in
    let entry = item.entry
    return MCPToolSearchCandidate(toolID: entry.toolID, displayName: "\(entry.serverAlias).\(entry.upstreamName)", serverAlias:
↳ entry.serverAlias, shortDescription: entry.shortDescription, riskHint: entry.annotations.readOnlyHint == true ? "external/read-only
↳ hint" : "external/untrusted", availability: entry.available ? (entry.stale ? "stale" : "available") : "unavailable", temperature:
↳ session.l1[entry.toolID] != nil ? "hot" : project[entry.toolID] != nil ? "warm" : "cold")
}

// 探测诊断：若检索目标涉及空列表或错误状态的 MCP 服务，暴露明确的诊断条目，切断模型无休止的重试
for (sID, sStatus) in serverStatuses {
    let alias = serverAliases[sID] ?? sID.rawValue
    let matchesServerFilter: Bool
    if let server {
        let s = server.lowercased()
        matchesServerFilter = (sID.rawValue.lowercased().contains(s) || alias.lowercased().contains(s) || s.contains(sID.rawValue.
↳ lowercased()))
    } else {
        matchesServerFilter = terms.contains(where: { sID.rawValue.lowercased().contains($0) || alias.lowercased().contains($0) })
    }
    if matchesServerFilter || (result.isEmpty && terms.isEmpty) {
        switch sStatus {
        case .empty:
            result.append(MCPToolSearchCandidate(
                toolID: ToolID("diagnostic::\(alias)::empty"),
                displayName: "\(alias) (无工具: 0 discovered)",
                serverAlias: alias,
                shortDescription: "MCP server '\(alias)' is connected and active, but registered 0 tools (empty list). Do NOT query
↳ or attempt to use tools from this server; proceed with builtin tools.",
                riskHint: "mcp/empty",
                availability: "empty",
                temperature: "cold"
            ))
        case let .error(reason):
            result.append(MCPToolSearchCandidate(
                toolID: ToolID("diagnostic::\(alias)::error"),
                displayName: "\(alias) (服务错误)",
                serverAlias: alias,
                shortDescription: "MCP server '\(alias)' is unavailable or errored: \(reason). Do NOT query or attempt to use tools
↳ from this server; proceed with builtin tools.",
                riskHint: "mcp/error",
                availability: "unavailable",
                temperature: "cold"
            ))
        default:
            break
    }
}
}

var updated = session; updated.candidates = Set(result.map(\.toolID)); sessions[sessionID] = updated
return result
}

```

```

public func load(sessionID: SessionID, toolID: ToolID, schemaTokenBudget: Int) async throws -> MCPToolSchemaLease {
    if toolID.rawValue.hasPrefix("diagnostic:") {
        throw CoreError(code: .toolNotFound, message: "This entry is a diagnostic notification indicating the MCP server is empty or
↪ unavailable. Please do not attempt to load it. Proceed with builtin tools.")
    }
    purgeExpired(sessionID)
    let effectiveID: ToolID
    if catalog[toolID] != nil {
        effectiveID = toolID
    } else if let found = catalog.first(where: {
        "\($0.value.serverAlias).\($0.value.upstreamName)" == toolID.rawValue ||
        "\($0.value.serverAlias)::\($0.value.upstreamName)" == toolID.rawValue ||
        "\($0.value.serverID.rawValue)::\($0.value.upstreamName)" == toolID.rawValue
    }) {
        effectiveID = found.key
    } else if let found = catalog.first(where: { $0.value.upstreamName == toolID.rawValue }) {
        effectiveID = found.key
    } else {
        effectiveID = toolID
    }
    guard sessions[sessionID]?.candidates.contains(effectiveID) == true || catalog[effectiveID] != nil else { throw MCPToolPagerError.
↪ missingTool }
    if let current = sessions[sessionID]?.leases.values.first(where: { $0.state == .armed && $0.expiresAt > .now }) {
        guard current.toolID == effectiveID else { throw MCPToolPagerError.taskUnsupported }
        return current
    }
    guard let entry = catalog[effectiveID], entry.available else { throw MCPToolPagerError.unavailable }
    guard let schema = await schemas.schema(toolID: effectiveID, hash: entry.schemaHash) else { throw MCPToolPagerError.schemaMissing }
    let bytes = try JSONEncoder().encode(schema).count
    guard bytes <= maxSchemaBytes, depth(schema) <= maxSchemaDepth else { throw MCPToolPagerError.schemaTooLarge }
    guard (bytes + 3) / 4 <= schemaTokenBudget else { throw MCPToolPagerError.schemaBudgetExceeded }
    pageFaults += 1
    let providerName = codec.encode(serverAlias: entry.serverAlias, upstreamName: entry.upstreamName, toolID: effectiveID)
    let lease = MCPToolSchemaLease(leaseID: UUID().uuidString, sessionID: sessionID, toolID: effectiveID, schemaHash: entry.schemaHash,
↪ providerName: providerName, createdAt: .now, expiresAt: .now.addingTimeInterval(300), state: .armed)
    var state = sessions[sessionID] ?? SessionState(); state.leases[lease.leaseID] = lease; sessions[sessionID] = state
    return lease
}

public func providerDefinitions(sessionID: SessionID) async -> [ToolDefinition] {
    purgeExpired(sessionID)
    let leases = sessions[sessionID]?.leases.values.filter { $0.state == .armed && $0.expiresAt > .now } ?? []
    providerSchemaCounts[sessionID, default: []].append(leases.count)
    if providerSchemaCounts[sessionID]!.count > 64 { providerSchemaCounts[sessionID]!.removeFirst(providerSchemaCounts[sessionID]!.count
↪ - 64) }
    if !leases.isEmpty { var state = sessions[sessionID] ?? SessionState(); state.presented.formUnion(leases.map(\.leaseID)); sessions[
↪ sessionID] = state }
    var result: [ToolDefinition] = []
    for lease in leases {
        guard let entry = catalog[lease.toolID], let schema = await schemas.schema(toolID: lease.toolID, hash: lease.schemaHash) else {
↪ continue }
        result.append(ToolDefinition(id: ToolID(lease.providerName), name: lease.providerName, description: entry.shortDescription,
↪ inputSchema: ToolInputSchema(properties: [:], required: []), capability: ToolCapability([.networkAccess, .destructive]),
↪ rawInputSchema: schema))
    }
    return result.sorted { $0.id.rawValue < $1.id.rawValue }
}

public func resolve(sessionID: SessionID, providerToolID: ToolID, now: Date = .now) throws -> MCPToolSchemaLease {
    guard let lease = sessions[sessionID]?.leases.values.first(where: { $0.providerName == providerToolID.rawValue && $0.state == .armed
↪ }) else { throw MCPToolPagerError.leaseMissing }
    guard lease.expiresAt > now else {
        sessions[sessionID]?.leases.removeValue(forKey: lease.leaseID)
        throw MCPToolPagerError.leaseExpired
    }
}

```

```

    guard let entry = catalog[lease.toolID], entry.schemaHash == lease.schemaHash else { throw MCPToolPagerError.schemaChanged }
    return lease
}

public func markUsed(sessionID: SessionID, providerToolID: ToolID, projectID: ProjectID) throws -> MCPToolSchemaLease {
    let lease = try resolve(sessionID: sessionID, providerToolID: providerToolID)
    var state = sessions[sessionID] ?? SessionState(); state.l1[lease.toolID, default: 0] += 1
    trim(&state.l1)
    if var stored = state.leases[lease.leaseID] { stored.state = .used; state.leases[lease.leaseID] = stored }; sessions[sessionID] =
↪ state
    l2[projectID, default: [:]][lease.toolID, default: 0] += 1
    trim(&l2[projectID]!)
    return lease
}

public func lazyReacquireLease(sessionID: SessionID, providerToolID: ToolID, projectID: ProjectID) async throws -> MCPToolSchemaLease {
    guard let entry = catalog.values.first(where: { codec.encode(serverAlias: $0.serverAlias, upstreamName: $0.upstreamName, toolID: $0.
↪ toolID) == providerToolID.rawValue }) else {
        throw MCPToolPagerError.missingTool
    }
    guard entry.available else { throw MCPToolPagerError.unavailable }
    let lease = MCPToolSchemaLease(
        leaseID: UUID().uuidString,
        sessionID: sessionID,
        toolID: entry.toolID,
        schemaHash: entry.schemaHash,
        providerName: providerToolID.rawValue,
        createdAt: .now,
        expiresAt: .now.addingTimeInterval(300),
        state: .used
    )
    var state = sessions[sessionID] ?? SessionState()
    state.leases[lease.leaseID] = lease
    state.l1[entry.toolID, default: 0] += 1
    trim(&state.l1)
    sessions[sessionID] = state
    l2[projectID, default: [:]][entry.toolID, default: 0] += 1
    trim(&l2[projectID]!)
    return lease
}

public func execute(sessionID: SessionID, projectID: ProjectID, providerToolID: ToolID, arguments: String) async throws -> (lease:
↪ MCPToolSchemaLease, content: String) {
    let lease: MCPToolSchemaLease
    do {
        lease = try markUsed(sessionID: sessionID, providerToolID: providerToolID, projectID: projectID)
    } catch let error as MCPToolPagerError where error == .leaseMissing || error == .leaseExpired {
        lease = try await lazyReacquireLease(sessionID: sessionID, providerToolID: providerToolID, projectID: projectID)
    }
    guard let entry = catalog[lease.toolID], let invoker else { throw MCPToolPagerError.unavailable }
    return (lease, try await invoker.call(serverID: entry.serverID, toolName: entry.upstreamName, arguments: arguments))
}

public func searchToolResult(sessionID: SessionID, projectID: ProjectID, arguments: String) throws -> String {
    struct Input: Decodable { let query: String?; let server: String?; let capability: String?; let maxResults: Int? }
    let decoder = JSONDecoder(); decoder.keyDecodingStrategy = .convertFromSnakeCase
    let input = try? decoder.decode(Input.self, from: Data(arguments.utf8))
    let query = input?.query ?? (arguments.trimmingCharacters(in: .whitespacesAndNewlines).hasPrefix("{") ? "" : arguments)
    let encoder = JSONEncoder(); encoder.outputFormatting = [.sortedKeys]
    return String(decoding: try encoder.encode(search(sessionID: sessionID, projectID: projectID, query: query, server: input?.server,
↪ capability: input?.capability, maxResults: input?.maxResults ?? 12)), as: UTF8.self)
}

public func loadToolResult(sessionID: SessionID, arguments: String, schemaTokenBudget: Int) async throws -> String {
    struct Input: Decodable { let toolId: String? }

```

```

let decoder = JSONDecoder(); decoder.keyDecodingStrategy = .convertFromSnakeCase
let input = try? decoder.decode(Input.self, from: Data(arguments.utf8))
let rawID = input?.toolID?.trimmingCharacters(in: .whitespacesAndNewLines) ?? (arguments.trimmingCharacters(in: .
↳ whitespacesAndNewLines).hasPrefix("{") ? "" : arguments.trimmingCharacters(in: .whitespacesAndNewLines))
guard !rawID.isEmpty else {
    throw CoreError(code: .toolArgumentInvalid, message: "load_tool 需要有效 tool_id")
}
let lease = try await load(sessionID: sessionID, toolID: ToolID(rawID), schemaTokenBudget: schemaTokenBudget)
let encoder = JSONEncoder(); encoder.outputFormatting = [.sortedKeys]
return String(decoding: try encoder.encode(["status": "leased", "tool": lease.toolID.rawValue, "provider_name": lease.providerName])
↳ , as: UTF8.self)
}

/// Called after every provider response; this is the sole schema page-out path.
public func finishProviderStep(sessionID: SessionID) {
    guard var state = sessions[sessionID] else { return }
    for id in state.presented {
        guard var lease = state.leases[id] else { continue }
        lease.state = lease.expiresAt <= .now ? .expired : .revoked
        state.leases.removeValue(forKey: id)
    }
    state.presented.removeAll()
    sessions[sessionID] = state
}

public func abortTurn(sessionID: SessionID) { sessions[sessionID]?.leases.removeAll(); sessions[sessionID]?.presented.removeAll() }
public func discardSession(_ sessionID: SessionID) { sessions.removeValue(forKey: sessionID); providerSchemaCounts.removeValue(forKey:
↳ sessionID) }

public func leaseCount(sessionID: SessionID) -> Int { purgeExpired(sessionID); return sessions[sessionID]?.leases.count ?? 0 }
/// Request audit only: counts, never schema bodies.
public func requestSchemaCounts(sessionID: SessionID) -> [Int] { providerSchemaCounts[sessionID] ?? [] }
public func catalogCount() -> Int { catalog.count }
public func schemaStoreMetrics() async -> (count: Int, bytes: Int) { (await schemas.count(), await schemas.bytes()) }
public func fullSchemaResidencyCount() -> Int { sessions.values.reduce(0) { $0 + $1.presented.count } }
public func activeLeaseCount() -> Int { sessions.values.reduce(0) { $0 + $1.leases.values.filter { $0.state == .armed && $0.expiresAt >
↳ .now }.count } }

private func removeReferences(_ toolID: ToolID) {
    for sessionID in sessions.keys { sessions[sessionID]?.l1.removeValue(forKey: toolID); sessions[sessionID]?.candidates.remove(toolID)
↳ ; sessions[sessionID]?.leases = sessions[sessionID]?.leases.filter { $0.value.toolID != toolID } }
    for projectID in l2.keys { l2[projectID]?.removeValue(forKey: toolID) }
}
private func purgeExpired(_ sessionID: SessionID, now: Date = .now) {
    guard var state = sessions[sessionID] else { return }
    state.leases = state.leases.filter { $0.value.expiresAt > now }
    sessions[sessionID] = state
}
private func trim(_ values: inout [ToolID: Int]) { while values.count > 32, let key = values.min(by: { $0.value < $1.value })?.key {
↳ values.removeValue(forKey: key) } }
private func depth(_ value: JSONValue) -> Int { switch value { case let .array(values): return 1 + (values.map(depth).max() ?? 0); case
↳ let .object(values): return 1 + (values.values.map(depth).max() ?? 0); default: return 1 } }
}

public enum MCPDiscoveryTools {
    public static let search = ToolDefinition(id: ToolID("search_tools"), description: "Search external MCP tool capabilities. Returns
↳ metadata only, never schemas.", inputSchema: ToolInputSchema(properties: ["query": ToolInputProperty(type: .string, description: "
↳ Capability query"), "server": ToolInputProperty(type: .string, description: "Optional server alias"), "capability":
↳ ToolInputProperty(type: .string, description: "Optional capability tag"), "max_results": ToolInputProperty(type: .integer,
↳ description: "Maximum candidates", minimum: 1, maximum: 8)], required: ["query"], capability: ToolCapability(readOnly: true))
    public static let load = ToolDefinition(id: ToolID("load_tool"), description: "Lease one external MCP tool for the next inference only
↳ .", inputSchema: ToolInputSchema(properties: ["tool_id": ToolInputProperty(type: .string, description: "ToolID returned by
↳ search_tools"), required: ["tool_id"], capability: ToolCapability(readOnly: true))
}

```

// File: Sources/LingXiCore/Modules/MCP/MCPTransport.swift

```

import Foundation
import LingXiProtocol
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif

public enum MCPTransportKind: String, Sendable, Codable { case stdio, streamableHTTP }
public enum MCPProtocolPreference: String, Sendable, Codable { case auto, modern, legacy }

public struct SecretRef: Sendable, Equatable, Codable { public let identifier: String; public init(_ identifier: String) { self.identifier =
    ↪ identifier } }

public protocol SecretResolver: Sendable { func resolve(_ ref: SecretRef) throws -> String? }
public struct EnvironmentSecretResolver: SecretResolver {
    private let values: [String: String]
    public init(_ values: [String: String]) { self.values = values }
    public func resolve(_ ref: SecretRef) throws -> String? { values[ref.identifier] }
}

public struct InMemorySecretResolver: SecretResolver {
    private let values: [String: String]
    public init(_ values: [String: String] = [:]) { self.values = values }
    public func resolve(_ ref: SecretRef) throws -> String? { values[ref.identifier] }
}

public enum MCPAuthentication: Sendable, Equatable, Codable {
    case none
    case bearer(SecretRef)
    case header(name: String, value: SecretRef)
}

public struct MCPServerConfiguration: Sendable, Equatable, Codable {
    public let serverID: MCPServerID
    public let alias: String
    public let transport: MCPTransportKind
    public let command: String?
    public let arguments: [String]
    public let endpoint: URL?
    public let protocolPreference: MCPProtocolPreference
    public let enabled: Bool
    public let auth: MCPAuthentication
    /// Target names only. Values must originate from SecretResolver, never persistence.
    public let environment: [String: SecretRef]
    public let timeoutSeconds: Double
    public init(serverID: MCPServerID, alias: String, transport: MCPTransportKind, command: String? = nil, arguments: [String] = [],
        ↪ endpoint: URL? = nil, protocolPreference: MCPProtocolPreference = .auto, enabled: Bool = true, auth: MCPAuthentication = .none,
        ↪ environment: [String: SecretRef] = [:], timeoutSeconds: Double = 60) { self.serverID = serverID; self.alias = alias; self.transport
        ↪ = transport; self.command = command; self.arguments = arguments; self.endpoint = endpoint; self.protocolPreference =
        ↪ protocolPreference; self.enabled = enabled; self.auth = auth; self.environment = environment; self.timeoutSeconds = timeoutSeconds
        ↪ }
}

public enum MCPProtocolVersionNegotiator {
    public static let modern = "2024-11-05"
    public static let legacy = "2024-10-07"
    public static func select(preference: MCPProtocolPreference, supportsModern: Bool, supportsLegacy: Bool) throws -> MCPProtocolEra {
        switch preference {
            case .modern where supportsModern: return .modern
            case .legacy where supportsLegacy: return .legacy
            case .auto where supportsModern: return .modern
            case .auto where supportsLegacy: return .legacy
            default: throw CoreError(code: .mcpProtocolUnsupported, message: "MCP protocol version unsupported")
        }
    }
}

public actor MCPServerRegistry {

```



```

private var servers: [MCPServerID: MCPServerConfiguration] = [:]
public init(_ configurations: [MCPServerConfiguration] = []) { servers = Dictionary(uniqueKeysWithValues: configurations.map { ($0.
    ↪ serverID, $0) }) }
public func register(_ configuration: MCPServerConfiguration) { servers[configuration.serverID] = configuration }
public func list() -> [MCPServerConfiguration] { servers.values.sorted { $0.alias < $1.alias } }
public func server(_ id: MCPServerID) -> MCPServerConfiguration? { servers[id] }
public func setEnabled(_ id: MCPServerID, enabled: Bool) { guard let value = servers[id] else { return }; servers[id] =
    ↪ MCPServerConfiguration(serverID: value.serverID, alias: value.alias, transport: value.transport, command: value.command, arguments:
    ↪ value.arguments, endpoint: value.endpoint, protocolPreference: value.protocolPreference, enabled: enabled, auth: value.auth,
    ↪ environment: value.environment, timeoutSeconds: value.timeoutSeconds) }
public func replace(_ configurations: [MCPServerConfiguration]) { servers = Dictionary(uniqueKeysWithValues: configurations.map { ($0.
    ↪ serverID, $0) }) }
}

public actor MCPConnectionManager: MCPToolInvoker {
    public enum Health: String, Sendable, Codable { case healthy, degraded, unavailable }
    private var invokers: [MCPServerID: any MCPToolInvoker] = [:]
    private var health: [MCPServerID: Health] = [:]
    public init() {}
    public func register(_ invoker: any MCPToolInvoker, for serverID: MCPServerID) { invokers[serverID] = invoker; health[serverID] = .
        ↪ healthy }
    public func unregister(_ serverID: MCPServerID) { invokers.removeValue(forKey: serverID); health.removeValue(forKey: serverID) }
    public func health(for serverID: MCPServerID) -> Health { health[serverID] ?? .unavailable }
    public func call(serverID: MCPServerID, toolName: String, arguments: String) async throws -> String {
        guard let invoker = invokers[serverID] else { throw CoreError(code: .mcpServerUnavailable, message: "MCP server unavailable: \(
            ↪ serverID.rawValue)") }
        do {
            let result = try await invoker.call(serverID: serverID, toolName: toolName, arguments: arguments)
            health[serverID] = .healthy
            return result
        } catch is CancellationError {
            throw CancellationError()
        } catch {
            health[serverID] = .degraded
            throw error
        }
    }
}

public func shutdown() { invokers.removeAll(); health.removeAll() }
}

/// Minimal Streamable HTTP JSON-RPC transport. It deliberately does not implement deprecated HTTP+SSE.
public struct MCPStreamableHTTPTransport: MCPToolInvoker {
    public let configuration: MCPServerConfiguration
    private let resolver: any SecretResolver
    private let session: URLSession
    public init(configuration: MCPServerConfiguration, resolver: any SecretResolver = InMemorySecretResolver(), session: URLSession = .
        ↪ shared) { self.configuration = configuration; self.resolver = resolver; self.session = session }
    public func call(serverID: MCPServerID, toolName: String, arguments: String) async throws -> String {
        let parameters = try JSONSerialization.jsonObject(with: Data(arguments.utf8))
        let response = try await post(method: "tools/call", name: toolName, parameters: ["name": toolName, "arguments": parameters])
        return try MCPWire.resultText(response.data, contentType: response.contentType)
    }
}

/// tools/list stays outside the provider tool set; callers atomically install this completed generation into L3.
public func listTools() async throws -> [MCPDiscoveredTool] {
    var sessionID: String?
    let initParams: [String: Any] = [
        "protocolVersion": protocolVersion,
        "capabilities": [String: Any](),
        "clientInfo": ["name": "lingxiagent", "version": "0.1.1"]
    ]
    if let initResp = try? await post(method: "initialize", parameters: initParams) {
        sessionID = initResp.sessionID
        _ = try? await post(method: "notifications/initialized", parameters: [:], sessionID: sessionID)
    }
}

```

```

var cursor: String?
var seen = Set<String>()
var result: [MCPDiscoveredTool] = []
for _ in 0..<100 {
    var parameters: [String: Any] = [:]
    if let cursor { parameters["cursor"] = cursor }
    let response = try await post(method: "tools/list", parameters: parameters, sessionID: sessionID)
    let page = try MCPToolDiscovery.decode(response.data, contentType: response.contentType, configuration: configuration)
    for tool in page.tools {
        guard seen.insert(tool.entry.upstreamName).inserted else { throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "
↳ Duplicate MCP tool name: \"(tool.entry.upstreamName)\") }
        result.append(tool)
    }
    guard let next = page.nextCursor, !next.isEmpty else { return result }
    guard next != cursor else { throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "MCP tools/list cursor loop") }
    cursor = next
}
throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "MCP tools/list exceeded page limit")
}

public func get() async throws -> Int {
    guard configuration.enabled else { throw CoreError(code: .mcpServerUnavailable, message: "MCP server disabled") }
    guard let endpoint = configuration.endpoint else { throw CoreError(code: .mcpServerUnavailable, message: "MCP HTTP endpoint missing
↳ ") }
    var request = URLRequest(url: endpoint); request.httpMethod = "GET"; request.timeoutInterval = configuration.timeoutSeconds
    request.setValue(protocolVersion, forHTTPHeaderField: "MCP-Protocol-Version")
    try applyAuth(to: &request)
    let (_, response) = try await session.data(for: request)
    guard let http = response as? HTTPURLResponse else { throw CoreError(code: .mcpServerUnavailable, message: "MCP GET failed") }
    return http.statusCode
}

private func post(method: String, name: String? = nil, parameters: [String: Any], sessionID: String? = nil) async throws -> (data: Data,
↳ contentType: String, sessionID: String?) {
    guard configuration.enabled else { throw CoreError(code: .mcpServerUnavailable, message: "MCP server disabled") }
    guard let endpoint = configuration.endpoint else { throw CoreError(code: .mcpServerUnavailable, message: "MCP HTTP endpoint missing
↳ ") }
    var request = URLRequest(url: endpoint); request.httpMethod = "POST"; request.timeoutInterval = configuration.timeoutSeconds
    request.setValue("application/json", forHTTPHeaderField: "Content-Type")
    request.setValue("application/json, text/event-stream", forHTTPHeaderField: "Accept")
    request.setValue(protocolVersion, forHTTPHeaderField: "MCP-Protocol-Version")
    request.setValue(method, forHTTPHeaderField: "Mcp-Method")
    if let sessionID { request.setValue(sessionID, forHTTPHeaderField: "Mcp-Session-Id") }
    if let name { request.setValue(name, forHTTPHeaderField: "Mcp-Name") }
    try applyAuth(to: &request)
    request.httpBody = try JSONSerialization.data(withJSONObject: ["jsonrpc": "2.0", "id": UUID().uuidString, "method": method, "params
↳ ": parameters], options: [.sortedKeys])
    let (data, response): (Data, URLResponse)
    do { (data, response) = try await session.data(for: request) }
    catch is CancellationError { throw CancellationError() }
    catch let error as URLError where error.code == .timedOut { throw CoreError(code: .commandTimedOut, message: "MCP HTTP \"(method)
↳ timed out\") }
    catch { throw CoreError(code: .mcpServerUnavailable, message: "MCP HTTP \"(method) transport failed\") }
    guard let http = response as? HTTPURLResponse else {
        throw CoreError(code: .mcpServerUnavailable, message: "MCP HTTP \"(method) failed")
    }
    if !(200..<300).contains(http.statusCode) {
        // debug
        // print("MCP HTTP \"(method) debug: status=\"(http.statusCode), body=\"(String(decoding: data, as: UTF8.self))")
    }
    if http.statusCode == 401 || http.statusCode == 403 {
        let auth = http.value(forHTTPHeaderField: "WWW-Authenticate") ?? ""
        if auth.lowercased().contains("oauth") || auth.lowercased().contains("bearer") {
            throw CoreError(code: .permissionDenied, message: "OAuth authorization required (HTTP \"(http.statusCode)\")")
        }
    }
}

```

```

    }
    throw CoreError(code: .permissionDenied, message: "Authentication required (HTTP \$(http.statusCode))")
}
guard (200..<300).contains(http.statusCode) else {
    let bodyStr = String(decoding: data, as: UTF8.self)
    throw CoreError(code: .mcpServerUnavailable, message: "MCP HTTP \$(method) failed (HTTP \$(http.statusCode)): \$(bodyStr.prefix
↵ (200))")
}
let respSessionID = http.value(forHTTPHeaderField: "Mcp-Session-Id") ?? sessionID
return (data, http.value(forHTTPHeaderField: "Content-Type") ?? "application/json", respSessionID)
}
private func applyAuth(to request: inout URLRequest) throws {
    switch configuration.auth {
    case .none: break
    case let .bearer(ref): if let secret = try resolver.resolve(ref) { request.setValue("Bearer \$(secret)", forHTTPHeaderField: "
↵ Authorization") }
    case let .header(name, ref): if let secret = try resolver.resolve(ref) { request.setValue(secret, forHTTPHeaderField: name) }
    }
}
private var protocolVersion: String { configuration.protocolPreference == .legacy ? MCPProtocolVersionNegotiator.legacy :
↵ MCPProtocolVersionNegotiator.modern }
}

/// stdio is command + argv only and receives a sanitized explicit environment.
public struct MCPStdioTransport: MCPToolInvoker {
    public let configuration: MCPServerConfiguration
    private let resolver: any SecretResolver
    public init(configuration: MCPServerConfiguration, resolver: any SecretResolver = InMemorySecretResolver()) { self.configuration =
↵ configuration; self.resolver = resolver }
    public func call(serverID: MCPServerID, toolName: String, arguments: String) async throws -> String {
        let parameters = try JSONSerialization.jsonObject(with: Data(arguments.utf8))
        return try MCPWire.resultText(await request(method: "tools/call", parameters: ["name": toolName, "arguments": parameters]))
    }
    public func listTools() async throws -> [MCPDiscoveredTool] {
        var cursor: String?
        var seen = Set<String>()
        var result: [MCPDiscoveredTool] = []
        for _ in 0..<100 {
            var parameters: [String: Any] = [:]
            if let cursor { parameters["cursor"] = cursor }
            let page = try MCPToolDiscovery.decode(await request(method: "tools/list", parameters: parameters), configuration: configuration
↵ )
            for tool in page.tools {
                guard seen.insert(tool.entry.upstreamName).inserted else { throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "
↵ Duplicate MCP tool name: \$(tool.entry.upstreamName)") }
                result.append(tool)
            }
            guard let next = page.nextCursor, !next.isEmpty else { return result }
            guard next != cursor else { throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "MCP tools/list cursor loop") }
            cursor = next
        }
        throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "MCP tools/list exceeded page limit")
    }
}
private func request(method: String, parameters: [String: Any]) async throws -> Data {
    try Task.checkCancellation()
    guard configuration.enabled else { throw CoreError(code: .mcpServerUnavailable, message: "MCP server disabled") }
    guard let command = configuration.command, command.hasPrefix("/") , FileManager.default.isExecutableFile(atPath: command) else {
↵ throw CoreError(code: .mcpServerUnavailable, message: "MCP stdio executable unavailable") }
    var environment = EnvironmentSanitizer.sanitized()
    for (name, ref) in configuration.environment { if let value = try resolver.resolve(ref) { environment[name] = value } }

    let process = Process()
    process.executableURL = URL(fileURLWithPath: command)
    process.arguments = configuration.arguments

```

```

process.environment = environment
process.currentDirectoryURL = URL(fileURLWithPath: FileManager.default.currentDirectoryPath, isDirectory: true)

let stdinPipe = Pipe()
let stdoutPipe = Pipe()
process.standardInput = stdinPipe
process.standardOutput = stdoutPipe
// Server diagnostics must never fill an unread pipe or enter the JSON-RPC stream.
process.standardError = FileHandle.nullDevice

do {
    try process.run()
} catch {
    throw CoreError(code: .mcpServerUnavailable, message: "Failed to launch MCP stdio process: \(error.localizedDescription)")
}

let stdinHandle = stdinPipe.fileHandleForWriting
let stdoutHandle = stdoutPipe.fileHandleForReading
try stdoutPipe.fileHandleForWriting.close()
let timeoutSeconds = configuration.timeoutSeconds > 0 ? configuration.timeoutSeconds : 30.0
let reqId = UUID().uuidString

let initReq: [String: Any] = [
    "jsonrpc": "2.0",
    "id": "init-1",
    "method": "initialize",
    "params": [
        "protocolVersion": "2024-11-05",
        "capabilities": [String: Any](),
        "clientInfo": ["name": "lingxiagent", "version": "0.1.1"]
    ]
]
let initData = try JSONSerialization.data(withJSONObject: initReq)
let notifyReq: [String: Any] = [
    "jsonrpc": "2.0",
    "method": "notifications/initialized",
    "params": [String: Any]()
]
let notifyData = try JSONSerialization.data(withJSONObject: notifyReq)
let targetReq: [String: Any] = [
    "jsonrpc": "2.0",
    "id": reqId,
    "method": method,
    "params": parameters
]
let targetReqData = try JSONSerialization.data(withJSONObject: targetReq)

let startTime = ContinuousClock().now
let (chunks, continuation) = AsyncStream<Data>.makeStream()
stdoutHandle.readabilityHandler = { handle in
    let data = handle.availableData
    if data.isEmpty {
        handle.readabilityHandler = nil
        continuation.finish()
    } else {
        continuation.yield(data)
    }
}

let watchdog = Task {
    do { try await Task.sleep(for: .seconds(timeoutSeconds)) }
    catch { return }
    continuation.finish()
}
defer {

```

```

        watchdog.cancel()
        stdoutHandle.readabilityHandler = nil
        continuation.finish()
        try? stdinHandle.close()
        if process.isRunning {
            process.terminate()
        }
    }
}

try stdinHandle.write(contentsOf: initData)
try stdinHandle.write(contentsOf: Data("\n".utf8))

var buffer = Data()
var initCompleted = false
var targetResultData: Data?

for await chunk in chunks {
    try Task.checkCancellation()
    buffer.append(chunk)
    guard buffer.count <= 8 * 1_024 * 1_024 else {
        throw CoreError(code: .mcpDiscoveryLimitExceeded, message: "MCP stdio response exceeded 8 MiB")
    }

    while let newlineRange = buffer.range(of: Data("\n".utf8)) {
        let lineData = buffer.subdata(in: 0..

```

```

        throw CoreError(code: .mcpServerUnavailable, message: "MCP stdio did not return a response for \(method)")
    }
    return finalData
}
}

private enum MCPToolDiscovery {
    struct Page { let tools: [MCPDiscoveredTool]; let nextCursor: String? }
    private struct Response: Decodable {
        struct Result: Decodable { let tools: [Tool]; let nextCursor: String? }
        struct Tool: Decodable { let name: String; let title: String?; let description: String?; let inputSchema: JSONValue; let annotations
        ↪ : MCPToolAnnotations? }
        let result: Result
    }

    static func decode(_ data: Data, contentType: String = "application/json", configuration: MCPServerConfiguration) throws -> Page {
        let decoded: Response
        do {
            decoded = try JSONDecoder().decode(Response.self, from: try MCPWire.jsonData(data, contentType: contentType))
        } catch {
            let sample = String(decoding: data.prefix(500), as: UTF8.self)
            throw CoreError(code: .mcpServerUnavailable, message: "MCP decode error: \(error), sample: \sample)")
        }
        return try Page(tools: decoded.result.tools.map { tool in
            let encoded = try JSONEncoder().encode(tool.inputSchema)
            let toolID = ToolID("\(configuration.serverID.rawValue)::\tool.name)")
            return MCPDiscoveredTool(
                entry: MCPToolCatalogEntry(
                    toolID: toolID,
                    serverID: configuration.serverID,
                    serverAlias: configuration.alias,
                    upstreamName: tool.name,
                    title: tool.title ?? tool.name,
                    shortDescription: String((tool.description ?? tool.name).prefix(512)),
                    tags: [],
                    annotations: tool.annotations ?? MCPToolAnnotations(),
                    schemaHash: sha256Hex(String(decoding: encoded, as: UTF8.self)),
                    era: .modern,
                    available: true,
                    stale: false,
                    cacheScope: .public,
                    authContextID: nil,
                    lastSeen: .now
                ),
                inputSchema: tool.inputSchema
            )
        }, nextCursor: decoded.result.nextCursor)
    }
}

private enum MCPWire {
    static func resultText(_ data: Data, contentType: String = "application/json") throws -> String {
        guard let object = try JSONSerialization.jsonObject(with: jsonData(data, contentType: contentType)) as? [String: Any] else { throw
        ↪ CoreError(code: .mcpServerUnavailable, message: "Malformed MCP response") }
        if object["error"] != nil { throw CoreError(code: .toolExecutionFailed, message: "MCP tool returned an error") }
        let result = object["result"] as? [String: Any] ?? [:]
        if let structured = result["structuredContent"] { return String(decoding: try JSONSerialization.data(withJSONObject: structured,
        ↪ options: [.sortedKeys]), as: UTF8.self) }
        if let content = result["content"] as? [[String: Any]] { return content.compactMap { $0["text"] as? String }.joined(separator: "\n")
        ↪ }
        return String(decoding: try JSONSerialization.data(withJSONObject: result, options: [.sortedKeys]), as: UTF8.self)
    }

    static func jsonData(_ data: Data, contentType: String) throws -> Data {
        guard contentType.lowercased().contains("text/event-stream") else { return data }
    }
}

```

```

        let events = String(decoding: data, as: UTF8.self).split(separator: "\n").compactMap { line -> String? in
            let text = line.trimmingCharacters(in: .whitespaces)
            return text.hasPrefix("data:") ? String(text.dropFirst(5)).trimmingCharacters(in: .whitespaces) : nil
        }
        guard let final = events.last(where: { $0 != "[DONE]" }) else { throw CoreError(code: .mcpServerUnavailable, message: "Malformed MCP
↩ SSE response") }
        return Data(final.utf8)
    }
}

```

// File: Sources/LingXiCore/Modules/MCP/MCPOAuthClient.swift

```

import Foundation
import CryptoKit
import LingXiProtocol
import LingXiPlatform
#if canImport(Darwin)
import Darwin
#elseif canImport(Glibc)
import Glibc
#endif

// MARK: - Loopback Callback Server

public final class LoopbackOAuthServer: @unchecked Sendable {
    private var serverSock: Int32 = -1
    public private(set) var port: UInt16 = 0
    private var isClosed = false
    private let lock = NSLock()

    public init(preferredPort: UInt16 = 54321) throws {
        let sock = socket(AF_INET, SOCK_STREAM, 0)
        guard sock >= 0 else {
            throw CoreError(code: .transport, message: "Failed to allocate socket for OAuth callback")
        }

        var reuse: Int32 = 1
        setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &reuse, socklen_t(MemoryLayout<Int32>.size))

        var addr = sockaddr_in()
        addr.sin_family = sa_family_t(AF_INET)
        addr.sin_addr.s_addr = inet_addr("127.0.0.1")
        addr.sin_port = preferredPort.bigEndian

        var bindResult = withUnsafePointer(to: &addr) {
            $0.withMemoryRebound(to: sockaddr.self, capacity: 1) {
                bind(sock, $0, socklen_t(MemoryLayout<sockaddr_in>.size))
            }
        }

        if bindResult != 0 {
            // Port in use, bind to ephemeral port (0)
            addr.sin_port = 0
            bindResult = withUnsafePointer(to: &addr) {
                $0.withMemoryRebound(to: sockaddr.self, capacity: 1) {
                    bind(sock, $0, socklen_t(MemoryLayout<sockaddr_in>.size))
                }
            }
        }

        guard bindResult == 0 else {
            close(sock)
            throw CoreError(code: .transport, message: "Failed to bind loopback socket for OAuth callback")
        }
    }
}

```

```

var len = socklen_t(MemoryLayout<sockaddr_in>.size)
var actualAddr = sockaddr_in()
withUnsafeMutablePointer(to: &actualAddr) {
    $0.withMemoryRebound(to: sockaddr.self, capacity: 1) {
        _ = getsockname(sock, $0, &len)
    }
}
self.port = UInt16(bigEndian: actualAddr.sin_port)
self.serverSock = sock

listen(sock, 1)
}

deinit {
    closeServer()
}

public func closeServer() {
    lock.lock()
    defer { lock.unlock() }
    if !isClosed && serverSock >= 0 {
        close(serverSock)
        serverSock = -1
        isClosed = true
    }
}

public func waitForCallback(expectedState: String, timeoutSeconds: Double = 180.0) async throws -> String {
    return try await withCheckedThrowingContinuation { continuation in
        DispatchQueue.global(qos: .userInitiated).async { [weak self] in
            guard let self else {
                continuation.resume(throwing: CoreError(code: .commandTimedOut, message: "Callback server deallocated"))
                return
            }

            self.lock.lock()
            let sock = self.serverSock
            self.lock.unlock()

            guard sock >= 0 else {
                continuation.resume(throwing: CoreError(code: .commandTimedOut, message: "Socket already closed"))
                return
            }

            var clientAddr = sockaddr_in()
            var clientLen = socklen_t(MemoryLayout<sockaddr_in>.size)
            let clientSock = withUnsafeMutablePointer(to: &clientAddr) {
                $0.withMemoryRebound(to: sockaddr.self, capacity: 1) {
                    accept(sock, $0, &clientLen)
                }
            }

            guard clientSock >= 0 else {
                continuation.resume(throwing: CoreError(code: .transport, message: "Failed to accept OAuth callback connection"))
                return
            }

            var buf = [CChar](repeating: 0, count: 4096)
            let bytesRead = read(clientSock, &buf, 4095)
            guard bytesRead > 0 else {
                close(clientSock)
                continuation.resume(throwing: CoreError(code: .transport, message: "Empty OAuth callback request"))
                return
            }

```



```
public struct MCP0AuthProtectedResourceMetadata: Codable, Sendable {
```

```

    public let resource: String?
    public let authorizationServers: [String]?
    public let scopesSupported: [String]?
    public let resourceName: String?

    enum CodingKeys: String, CodingKey {
        case resource
        case authorizationServers = "authorization_servers"
        case scopesSupported = "scopes_supported"
        case resourceName = "resource_name"
    }
}

public struct MCP0AuthServerMetadata: Codable, Sendable {
    public let issuer: String?
    public let authorizationEndpoint: String
    public let tokenEndpoint: String
    public let registrationEndpoint: String?
    public let scopesSupported: [String]?
    public let codeChallengeMethodsSupported: [String]?
    public let tokenEndpointAuthMethodsSupported: [String]?

    enum CodingKeys: String, CodingKey {
        case issuer
        case authorizationEndpoint = "authorization_endpoint"
        case tokenEndpoint = "token_endpoint"
        case registrationEndpoint = "registration_endpoint"
        case scopesSupported = "scopes_supported"
        case codeChallengeMethodsSupported = "code_challenge_methods_supported"
        case tokenEndpointAuthMethodsSupported = "token_endpoint_auth_methods_supported"
    }
}

public struct MCP0AuthRegistrationResponse: Codable, Sendable {
    public let clientID: String
    public let clientSecret: String?
    public let tokenEndpointAuthMethod: String?

    enum CodingKeys: String, CodingKey {
        case clientID = "client_id"
        case clientSecret = "client_secret"
        case tokenEndpointAuthMethod = "token_endpoint_auth_method"
    }
}

// MARK: - MCP0AuthClient Engine

public enum MCP0AuthClient {
    public struct DiscoveredEndpoints: Sendable {
        public let authorizationEndpoint: URL
        public let tokenEndpoint: URL
        public let registrationEndpoint: URL?
        public let scopes: [String]
        public let resourceName: String?
    }

    /// Discovers OAuth endpoints according to RFC 9728 and RFC 8414
    public static func discoverEndpoints(for endpointURL: URL) async throws -> DiscoveredEndpoints {
        guard let host = endpointURL.host else {
            throw CoreError(code: .toolArgumentInvalid, message: "Invalid MCP server URL: \(endpointURL)")
        }

        let scheme = endpointURL.scheme ?? "https"
        var candidates: [URL] = []

```

```

// Path-specific protected resource metadata (RFC 9728)
let path = endpointURL.path
if !path.isEmpty && path != "/" {
    if let u = URL(string: "\(\scheme):/\(host)/.well-known/oauth-protected-resource\(path)") {
        candidates.append(u)
    }
}
if let u = URL(string: "\(\scheme):/\(host)/.well-known/oauth-protected-resource") {
    candidates.append(u)
}

var protectedResource: MCP0AuthProtectedResourceMetadata?
for candidate in candidates {
    if let (data, resp) = try? await URLSession.shared.data(from: candidate),
        let http = resp as? HTTPURLResponse, (200..<300).contains(http.statusCode),
        let meta = try? JSONDecoder().decode(MCP0AuthProtectedResourceMetadata.self, from: data) {
        protectedResource = meta
        break
    }
}

let authServerBase: String
if let servers = protectedResource?.authorizationServers, let first = servers.first, !first.isEmpty {
    authServerBase = first
} else {
    authServerBase = "\(\scheme):/\(host)"
}

var serverMetaCandidates: [URL] = []
if let u = URL(string: "\(\authServerBase)/.well-known/oauth-authorization-server") {
    serverMetaCandidates.append(u)
}
if let u = URL(string: "\(\authServerBase)/.well-known/openid-configuration") {
    serverMetaCandidates.append(u)
}

var authServerMeta: MCP0AuthServerMetadata?
for candidate in serverMetaCandidates {
    if let (data, resp) = try? await URLSession.shared.data(from: candidate),
        let http = resp as? HTTPURLResponse, (200..<300).contains(http.statusCode),
        let meta = try? JSONDecoder().decode(MCP0AuthServerMetadata.self, from: data) {
        authServerMeta = meta
        break
    }
}

guard let authMeta = authServerMeta,
    let authURL = URL(string: authMeta.authorizationEndpoint),
    let tokenURL = URL(string: authMeta.tokenEndpoint) else {
    throw CoreError(code: .mcpServerUnavailable, message: "Could not discover OAuth authorization server metadata for \(endpointURL)
↳ ")
}

let regURL = authMeta.registrationEndpoint.flatMap { URL(string: $0) }
let scopes = protectedResource?.scopesSupported ?? authMeta.scopesSupported ?? ["default"]

return DiscoveredEndpoints(
    authorizationEndpoint: authURL,
    tokenEndpoint: tokenURL,
    registrationEndpoint: regURL,
    scopes: scopes,
    resourceName: protectedResource?.resourceName
)
}

```

```

/// Performs Dynamic Client Registration (RFC 7591)
public static func registerClient(
    registrationEndpoint: URL,
    redirectURI: URL,
    clientName: String = "LingXiAgent"
) async throws -> MCP0AuthRegistrationResponse {
    var req = URLRequest(url: registrationEndpoint)
    req.httpMethod = "POST"
    req.setValue("application/json", forHTTPHeaderField: "Content-Type")

    let body: [String: Any] = [
        "client_name": clientName,
        "redirect_uris": [redirectURI.absoluteString],
        "grant_types": ["authorization_code", "refresh_token"],
        "response_types": ["code"]
    ]
    req.httpBody = try JSONSerialization.data(withJSONObject: body)

    let (data, resp) = try await URLSession.shared.data(for: req)
    guard let http = resp as? HTTPURLResponse, (200..<300).contains(http.statusCode) else {
        let bodyStr = String(decoding: data, as: UTF8.self)
        throw CoreError(code: .permissionDenied, message: "Dynamic client registration failed \(http.statusCode
↵ ?? -1)): \(bodyStr)")
    }

    return try JSONDecoder().decode(MCP0AuthRegistrationResponse.self, from: data)
}

/// Exchanges authorization code for OAuth access tokens
public static func exchangeCodeForToken(
    tokenEndpoint: URL,
    clientID: String,
    clientSecret: String?,
    authMethod: String?,
    redirectURI: URL,
    code: String,
    codeVerifier: String
) async throws -> String {
    var req = URLRequest(url: tokenEndpoint)
    req.httpMethod = "POST"
    req.setValue("application/x-www-form-urlencoded", forHTTPHeaderField: "Content-Type")
    req.setValue("application/json", forHTTPHeaderField: "Accept")

    var bodyParams: [String: String] = [
        "grant_type": "authorization_code",
        "code": code,
        "redirect_uri": redirectURI.absoluteString,
        "code_verifier": codeVerifier
    ]

    var allowed = CharacterSet.alphanumerics
    allowed.insert(charactersIn: "-._~")

    if let secret = clientSecret, !secret.isEmpty {
        if authMethod == nil || authMethod == "client_secret_basic" {
            let encID = clientID.addingPercentEncoding(withAllowedCharacters: allowed) ?? clientID
            let encSecret = secret.addingPercentEncoding(withAllowedCharacters: allowed) ?? secret
            let cred = "\(encID):\(encSecret)"
            let base64 = Data(cred.utf8).base64EncodedString()
            req.setValue("Basic \(base64)", forHTTPHeaderField: "Authorization")
        } else {
            bodyParams["client_id"] = clientID
            bodyParams["client_secret"] = secret
        }
    } else {

```

```

        bodyParams["client_id"] = clientID
    }
    let bodyStr = bodyParams.map { key, value in
        let encKey = key.addingPercentEncoding(withAllowedCharacters: allowed) ?? key
        let encValue = value.addingPercentEncoding(withAllowedCharacters: allowed) ?? value
        return "\\(encKey)=\\(encValue)"
    }.joined(separator: "&")
    req.httpBody = Data(bodyStr.utf8)

    let (data, resp) = try await URLSession.shared.data(for: req)
    guard let http = resp as? HTTPURLResponse, (200..<300).contains(http.statusCode) else {
        let bodyErr = String(decoding: data, as: UTF8.self)
        throw CoreError(code: .permissionDenied, message: "Token exchange failed \\((resp as? HTTPURLResponse)?.statusCode ?? -1)): \\(
↳ bodyErr)")
    }

    guard let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
        let token = json["access_token"] as? String else {
        throw CoreError(code: .provider, message: "No access_token found in token response")
    }

    return token
}

/// 自动在系统默认浏览器中打开授权 URL
public static func openURLInBrowser(_ url: URL) {
    LingXiPlatform.system.openBrowser(at: url)
}
}

```

// File: Sources/LingXiCore/Infrastructure/Persistence/SQLitePersistenceStore.swift

```

import Foundation
import SQLite3
import LingXiProtocol

private let SQLITE_TRANSIENT = unsafeBitCast(-1, to: sqlite3_destructor_type.self)

public enum PersistenceError: Error, Sendable, Equatable {
    case sqlite(String)
    case migration(Int)
    case missingMainRoot(ProjectID)
}

public enum PersistenceFailpoint: Sendable, Equatable {
    case beforeCompactionCommit
    case beforeSaveAgentRun(SessionKind? = nil)
}

public struct StructuredPathAuditViolation: Sendable, Equatable {
    public let location: String
    public let value: String
}

/// 单 actor 持有两个 SQLite handle; 所有写入均经过此序列化事务边界。
public actor SQLitePersistenceStore {
    public static let databaseSchemaVersion = 6
    public static let contextFormatVersion = 1
    public static let indexFormatVersion = 1

    public let dataRoot: URL
    public nonisolated let projectID: ProjectID
    private nonisolated(unsafe) let catalog: OpaquePointer
    private nonisolated(unsafe) let state: OpaquePointer
    private let blobs: FileBlobStore
}

```

```

private var failpoint: PersistenceFailpoint?

public init(dataRoot: URL, mainRoot: URL, projectID: ProjectID? = nil) throws {
    self.dataRoot = dataRoot.standardizedFileURL
    try FileManager.default.createDirectory(at: dataRoot, withIntermediateDirectories: true)
    let catalogDB = try Self.open(dataRoot.appendingPathComponent("catalog.sqlite"))
    catalog = catalogDB
    try Self.configure(catalogDB)
    try Self.migrate(catalogDB, create: { try Self.createCatalogSchema(catalogDB) }, upgrade: { try Self.execute(catalogDB, "PRAGMA
↪ user_version = 2", []) }, upgradeV3: { try Self.execute(catalogDB, "PRAGMA user_version = 3", []) }, upgradeV4: { try Self.execute(
↪ catalogDB, "PRAGMA user_version = 4", []) }, upgradeV5: { try Self.execute(catalogDB, "PRAGMA user_version = 5", []) }, upgradeV6:
↪ { try Self.execute(catalogDB, "PRAGMA user_version = 6", []) })
    let canonicalRoot = mainRoot.standardizedFileURL.resolvingSymlinksInPath()
    if let projectID {
        self.projectID = projectID
    } else if let existing = try Self.scalar(catalogDB, "SELECT project_id FROM root_bindings WHERE absolute_root = ? AND kind = 'main'
↪ AND lifecycle_state = 'active' LIMIT 1", [canonicalRoot.path]) {
        self.projectID = ProjectID(existing)
    } else {
        self.projectID = ProjectID(UUID().uuidString)
    }
    let projectDirectory = dataRoot.appendingPathComponent("projects", isDirectory: true).appendingPathComponent(self.projectID.rawValue
↪ , isDirectory: true)
    try FileManager.default.createDirectory(at: projectDirectory, withIntermediateDirectories: true)
    let stateDB = try Self.open(projectDirectory.appendingPathComponent("state.sqlite"))
    state = stateDB
    try Self.configure(stateDB)
    try Self.migrate(stateDB, create: { try Self.createStateSchema(stateDB) }, upgrade: { try Self.upgradeStateSchemaV2(stateDB) },
↪ upgradeV3: { try Self.upgradeStateSchemaV3(stateDB) }, upgradeV4: { try Self.upgradeStateSchemaV4(stateDB) }, upgradeV5: { try Self
↪ .upgradeStateSchemaV5(stateDB) }, upgradeV6: { try Self.upgradeStateSchemaV6(stateDB) })
    _ = try? Self.script(stateDB, "ALTER TABLE sessions ADD COLUMN revision INTEGER NOT NULL DEFAULT 0")
    Self.ensureAllExistingProjectsHaveSessionRevision(dataRoot: dataRoot)
    try Self.execute(stateDB, "CREATE TABLE IF NOT EXISTS persistence_metadata(key TEXT PRIMARY KEY, value TEXT NOT NULL)", [])
    try Self.execute(stateDB, "CREATE TABLE IF NOT EXISTS file_mutation_journal(id INTEGER PRIMARY KEY AUTOINCREMENT, session_id TEXT
↪ NOT NULL, turn_id TEXT NOT NULL, revision INTEGER NOT NULL, tool_call_id TEXT NOT NULL, path TEXT NOT NULL, before_hash TEXT,
↪ before_content TEXT, after_hash TEXT, after_content TEXT, created_at REAL NOT NULL)", [])
    try Self.execute(stateDB, "CREATE INDEX IF NOT EXISTS idx_fmj_session ON file_mutation_journal(session_id)", [])
    blobs = try FileBlobStore(directory: projectDirectory.appendingPathComponent("blobs", isDirectory: true))
    try Self.transaction(catalogDB) {
        try Self.execute(catalogDB, "INSERT OR IGNORE INTO projects(project_id, created_at, updated_at) VALUES(?, ?, ?)", [self.
↪ projectID.rawValue, Self.now, Self.now])
        let count = try Self.scalar(catalogDB, "SELECT COUNT(*) FROM root_bindings WHERE project_id = ? AND kind = 'main' AND
↪ lifecycle_state = 'active'", [self.projectID.rawValue]).flatMap(Int.init) ?? 0
        if count == 0 {
            try Self.execute(catalogDB, "INSERT INTO root_bindings(binding_id, project_id, kind, absolute_root, parent_binding_id,
↪ binding_revision, lifecycle_state, time_created, time_updated, time_last_seen) VALUES(?, ?, 'main', ?, NULL, 1, 'active', ?, ?, ?)
↪ ", ["RB-" + UUID().uuidString, self.projectID.rawValue, canonicalRoot.path, Self.now, Self.now, Self.now])
        }
    }
}

deinit { sqlite3_close(catalog); sqlite3_close(state) }

public func mainRootBinding() throws -> RootBinding {
    guard let binding = try rootBindings(projectID: projectID).first(where: { $0.kind == .main && $0.lifecycleState == .active }) else {
        ↪ throw PersistenceError.missingMainRoot(projectID)
    }
    return binding
}

public func rootBindings(projectID: ProjectID? = nil) throws -> [RootBinding] {
    let id = (projectID ?? self.projectID).rawValue
    return try Self.rows(catalog, "SELECT binding_id, project_id, kind, absolute_root, parent_binding_id, binding_revision,
↪ lifecycle_state, time_created, time_updated, time_last_seen FROM root_bindings WHERE project_id = ? ORDER BY kind, binding_id", [id
↪ ]).compactMap(Self.decodeRoot)
}

```

```

public func rootBinding(_ id: RootBindingID) throws -> RootBinding? {
    try Self.rows(catalog, "SELECT binding_id, project_id, kind, absolute_root, parent_binding_id, binding_revision, lifecycle_state,
↳ time_created, time_updated, time_last_seen FROM root_bindings WHERE binding_id = ?", [id.rawValue]).compactMap(Self.decodeRoot).
↳ first
}

@discardableResult
public func addChildRoot(kind: RootBindingKind, absoluteRoot: URL) throws -> RootBinding {
    precondition(kind != .main)
    let main = try mainRootBinding()
    let id = RootBindingID("RB-" + UUID().uuidString)
    let root = absoluteRoot.standardizedFileURL.resolvingSymlinksInPath().path
    try Self.transaction(catalog) {
        try Self.execute(catalog, "INSERT INTO root_bindings(binding_id, project_id, kind, absolute_root, parent_binding_id,
↳ binding_revision, lifecycle_state, time_created, time_updated, time_last_seen) VALUES(?, ?, ?, ?, ?, 1, 'active', ?, ?, ?)", [id.
↳ rawValue, projectID.rawValue, kind.rawValue, root, main.id.rawValue, Self.now, Self.now, Self.now])
    }
    return try rootBinding(id)!
}

/// 根迁移只更新 catalog 中的一行，绝不遍历 project state。
public func rebindRoot(projectID: ProjectID, rootBindingID: RootBindingID, newAbsoluteRoot: URL) throws {
    let canonical = newAbsoluteRoot.standardizedFileURL.resolvingSymlinksInPath().path
    try Self.transaction(catalog) {
        try Self.execute(catalog, "UPDATE root_bindings SET absolute_root = ?, binding_revision = binding_revision + 1, lifecycle_state
↳ = 'active', time_updated = ?, time_last_seen = ? WHERE binding_id = ? AND project_id = ?", [canonical, Self.now, Self.now,
↳ rootBindingID.rawValue, projectID.rawValue])
    }
}

public func markRootMissing(_ id: RootBindingID) throws {
    try Self.execute(catalog, "UPDATE root_bindings SET lifecycle_state = 'missing', time_updated = ? WHERE binding_id = ?", [Self.now,
↳ id.rawValue])
}

public func upsertFile(rootBindingID: RootBindingID, relativePath: ProjectRelativePath, contentHash: String, version: String, state
↳ fileState: String = "active") throws -> ProjectFileBinding {
    if let existing = try file(rootBindingID: rootBindingID, relativePath: relativePath) {
        try Self.execute(state, "UPDATE project_files SET content_hash = ?, version = ?, state = ?, time_updated = ?, time_last_seen = ?
↳ WHERE file_id = ?", [contentHash, version, fileState, Self.now, Self.now, existing.id.rawValue])
        return try file(existing.id)!
    }
    let id = ProjectFileID("F-" + UUID().uuidString)
    try Self.execute(state, "INSERT INTO project_files(file_id, project_id, root_binding_id, relative_path, content_hash, version, state
↳ , time_created, time_updated, time_last_seen) VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", [id.rawValue, projectID.rawValue,
↳ rootBindingID.rawValue, relativePath.rawValue, contentHash, version, fileState, Self.now, Self.now, Self.now])
    return try file(id)!
}

public func file(_ id: ProjectFileID) throws -> ProjectFileBinding? {
    try Self.rows(state, "SELECT file_id, project_id, root_binding_id, relative_path, content_hash, version, state, time_created,
↳ time_updated, time_last_seen FROM project_files WHERE file_id = ?", [id.rawValue]).compactMap(Self.decodeFile).first
}

public func file(rootBindingID: RootBindingID, relativePath: ProjectRelativePath) throws -> ProjectFileBinding? {
    try Self.rows(state, "SELECT file_id, project_id, root_binding_id, relative_path, content_hash, version, state, time_created,
↳ time_updated, time_last_seen FROM project_files WHERE root_binding_id = ? AND relative_path = ?", [rootBindingID.rawValue,
↳ relativePath.rawValue]).compactMap(Self.decodeFile).first
}

public func files() throws -> [ProjectFileBinding] {
    try Self.rows(state, "SELECT file_id, project_id, root_binding_id, relative_path, content_hash, version, state, time_created,
↳ time_updated, time_last_seen FROM project_files WHERE project_id = ?", [projectID.rawValue]).compactMap(Self.decodeFile)
}

```

```

public func relocateFile(_ id: ProjectFileID, rootBindingID: RootBindingID, relativePath: ProjectRelativePath) throws {
    try Self.execute(state, "UPDATE project_files SET root_binding_id = ?, relative_path = ?, time_updated = ?, time_last_seen = ? WHERE
↪ file_id = ?", [rootBindingID.rawValue, relativePath.rawValue, Self.now, Self.now, id.rawValue])
}

public func markFileMissing(_ id: ProjectFileID) throws {
    try Self.execute(state, "UPDATE project_files SET state = 'missing', time_updated = ? WHERE file_id = ?", [Self.now, id.rawValue])
}

public func createSession(_ session: Session) throws {
    try writeSession(session)
}

public func appendMessage(sessionID: SessionID, message: Message, expectedRevision: UInt64? = nil) throws {
    try Self.transaction(state) {
        if let expected = expectedRevision {
            let actual = try Self.scalar(state, "SELECT revision FROM sessions WHERE session_id = ?", [sessionID.rawValue]).flatMap(
↪ UInt64.init) ?? 0
            guard actual == expected else {
                throw StaleRunError(sessionID: sessionID, expected: actual, actual: expected)
            }
        }
        let ordinal = try Self.scalar(state, "SELECT COALESCE(MAX(ordinal), -1) + 1 FROM messages WHERE session_id = ?", [sessionID.
↪ rawValue]).flatMap(Int.init) ?? 0
        try Self.insertMessage(state, sessionID: sessionID, message: message, ordinal: ordinal)
    }
}

public func appendAssistantMessageAndBatch(sessionID: SessionID, message: Message, batch: ToolExchangeBatch, expectedRevision: UInt64? =
↪ nil) throws {
    let ordinal = try Self.nextMessageOrdinal(state, sessionID)
    try Self.transaction(state) {
        if let expected = expectedRevision {
            let actual = try Self.scalar(state, "SELECT revision FROM sessions WHERE session_id = ?", [sessionID.rawValue]).flatMap(
↪ UInt64.init) ?? 0
            guard actual == expected else {
                throw StaleRunError(sessionID: sessionID, expected: actual, actual: expected)
            }
        }
        try Self.insertMessage(state, sessionID: sessionID, message: message, ordinal: ordinal)
        try Self.writeBatch(state, batch)
    }
}

public func appendToolResultMessageAndSettle(sessionID: SessionID, message: Message, batch: ToolExchangeBatch, expectedRevision: UInt64?
↪ = nil) throws {
    let ordinal = try Self.nextMessageOrdinal(state, sessionID)
    try Self.transaction(state) {
        if let expected = expectedRevision {
            let actual = try Self.scalar(state, "SELECT revision FROM sessions WHERE session_id = ?", [sessionID.rawValue]).flatMap(
↪ UInt64.init) ?? 0
            guard actual == expected else {
                throw StaleRunError(sessionID: sessionID, expected: actual, actual: expected)
            }
        }
        try Self.insertMessage(state, sessionID: sessionID, message: message, ordinal: ordinal)
        try Self.writeBatch(state, batch)
    }
}

public func bumpRevision(sessionID: SessionID) throws -> UInt64 {
    try Self.transaction(state) {
        try Self.execute(state, "UPDATE sessions SET revision = revision + 1, updated_at = ? WHERE session_id = ?", [Self.now, sessionID
↪ .rawValue])
    }
}

```



```

    }
    return try currentRevision(sessionID: sessionID)
}

public func currentRevision(sessionID: SessionID) throws -> UInt64 {
    try Self.scalar(state, "SELECT revision FROM sessions WHERE session_id = ?", [sessionID.rawValue]).flatMap(UInt64.init) ?? 0
}

public func revertLastTurn(sessionID: SessionID, bumpRevision: Bool = true) throws -> (revertedPrompt: String?, removedCount: Int) {
    let rows = try Self.rows(state, "SELECT message_id, ordinal, role FROM messages WHERE session_id = ? ORDER BY ordinal DESC", [
    ↪ sessionID.rawValue])
    guard let lastUserRow = rows.first(where: { $0[2] == MessageRole.user.rawValue }),
        let userOrdinal = Int(lastUserRow[1]) else {
        return (nil, 0)
    }
    let userMessageID = lastUserRow[0]
    let partRows = try Self.rows(state, "SELECT payload FROM message_parts WHERE message_id = ? ORDER BY ordinal LIMIT 1", [
    ↪ userMessageID])
    var revertedPrompt: String?
    if let partJSON = partRows.first?.first,
        let part = try? JSONDecoder().decode(SessionMessagePart.self, from: Data(partJSON.utf8)),
        case let .text(text) = part {
        revertedPrompt = text
    }

    let toDelete = rows.filter { (Int($0[1]) ?? -1) >= userOrdinal }.map { $0[0] }
    try Self.transaction(state) {
        for mID in toDelete {
            try Self.execute(state, "DELETE FROM message_parts WHERE message_id = ?", [mID])
            try Self.execute(state, "DELETE FROM tool_exchange_batches WHERE assistant_message_id = ? OR result_message_id = ?", [mID,
    ↪ mID])
            try Self.execute(state, "DELETE FROM messages WHERE message_id = ?", [mID])
        }
        try Self.execute(state, "DELETE FROM compaction_state WHERE session_id = ?", [sessionID.rawValue])
        try Self.execute(state, "DELETE FROM derived_context WHERE session_id = ?", [sessionID.rawValue])
        try Self.execute(state, "DELETE FROM file_mutation_journal WHERE session_id = ?", [sessionID.rawValue])
        if bumpRevision {
            try Self.execute(state, "UPDATE sessions SET revision = revision + 1, updated_at = ? WHERE session_id = ?", [Self.now,
    ↪ sessionID.rawValue])
        } else {
            try Self.execute(state, "UPDATE sessions SET updated_at = ? WHERE session_id = ?", [Self.now, sessionID.rawValue])
        }
    }
    return (revertedPrompt, toDelete.count)
}

public func clearCompactionAndDerived(sessionID: SessionID) throws {
    try Self.transaction(state) {
        try Self.execute(state, "DELETE FROM compaction_state WHERE session_id = ?", [sessionID.rawValue])
        try Self.execute(state, "DELETE FROM derived_context WHERE session_id = ?", [sessionID.rawValue])
    }
}

public func recordFileMutation(_ mutation: FileMutation) throws {
    let beforeB64 = mutation.beforeContent?.base64EncodedString() ?? ""
    let afterB64 = mutation.afterContent?.base64EncodedString() ?? ""
    try Self.execute(
        state,
        "INSERT INTO file_mutation_journal(session_id, turn_id, revision, tool_call_id, path, before_hash, before_content, after_hash,
    ↪ after_content, created_at) VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?)",
        [
            mutation.sessionID.rawValue,
            mutation.turnID.rawValue,
            mutation.revision,
            mutation.toolCallID.rawValue,

```

```

        mutation.path,
        mutation.beforeHash ?? "",
        beforeB64,
        mutation.afterHash ?? "",
        afterB64,
        mutation.createdAt.timeIntervalSince1970
    ]
}
)
}

public func loadFileMutations(sessionID: SessionID) throws -> [FileMutation] {
    let rows = try Self.rows(
        state,
        "SELECT session_id, turn_id, revision, tool_call_id, path, before_hash, before_content, after_hash, after_content, created_at
        ↪ FROM file_mutation_journal WHERE session_id = ? ORDER BY id ASC",
        [sessionID.rawValue]
    )
    return rows.compactMap { row in
        guard row.count >= 10 else { return nil }
        let sID = SessionID(row[0])
        let tID = TurnID(row[1])
        let rev = UInt64(row[2]) ?? 0
        let cID = ToolCallID(row[3])
        let path = row[4]
        let beforeHash = row[5].isEmpty ? nil : row[5]
        let beforeContent = row[6].isEmpty ? nil : Data(base64Encoded: row[6])
        let afterHash = row[7].isEmpty ? nil : row[7]
        let afterContent = row[8].isEmpty ? nil : Data(base64Encoded: row[8])
        let created = Double(row[9]).map { Date(timeIntervalSince1970: $0) } ?? Date()
        return FileMutation(
            sessionID: sID,
            turnID: tID,
            revision: rev,
            toolCallID: cID,
            path: path,
            beforeHash: beforeHash,
            beforeContent: beforeContent,
            afterHash: afterHash,
            afterContent: afterContent,
            createdAt: created
        )
    }
}

public func deleteFileMutations(sessionID: SessionID) throws {
    try Self.execute(state, "DELETE FROM file_mutation_journal WHERE session_id = ?", [sessionID.rawValue])
}

public func loadSessions() throws -> [Session] {
    let sessions = try Self.rows(state, "SELECT session_id, project_id, kind, parent_session_id, root_session_id, spawned_by_run_id,
    ↪ spawned_by_tool_call_id, title, cwd_root_binding_id, cwd_relative_path, created_at, updated_at, revision FROM sessions WHERE
    ↪ project_id = ? ORDER BY created_at", [projectID.rawValue])
    return try sessions.map { row in
        let id = SessionID(row[0]); let messages = try loadMessages(sessionID: id)
        let rev = row.count >= 13 ? (UInt64(row[12]) ?? 0) : 0
        return Session(id: id, createdAt: Self.parseDate(row[10]), kind: SessionKind(rawValue: row[2]) ?? .primary, parentSessionID: row
        ↪ [3].isEmpty ? nil : SessionID(row[3]), rootSessionID: SessionID(row[4]), spawnedByRunID: row[5].isEmpty ? nil : AgentRunID(row[5]),
        ↪ spawnedByToolCallID: row[6].isEmpty ? nil : ToolCallID(row[6]), title: row[7].isEmpty ? nil : row[7], projectID: ProjectID(row[1])
        ↪ , cwdRootBindingID: RootBindingID(row[8]), cwdRelativePath: ProjectRelativePath(rawValue: row[9]), updatedAt: Self.parseDate(row
        ↪ [11]), revision: rev, messages: messages)
    }
}

public func loadAllGlobalSessions() throws -> [SessionSummary] {
    try Self.loadAllGlobalSessions(dataRoot: dataRoot)
}

```

```

}

public static func loadAllGlobalSessions(dataRoot: URL) throws -> [SessionSummary] {
    let catalogPath = dataRoot.appendingPathComponent("catalog.sqlite")
    guard FileManager.default.fileExists(atPath: catalogPath.path) else { return [] }
    var catalogDB: OpaquePointer?
    guard sqlite3_open_v2(catalogPath.path, &catalogDB, SQLITE_OPEN_READONLY | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK, let catalogDB
↪ else {
        return []
    }
    defer { sqlite3_close(catalogDB) }

    let roots = try rows(catalogDB, "SELECT project_id, absolute_root FROM root_bindings WHERE kind = 'main' AND lifecycle_state = '
↪ active'", [])
    var summaries: [SessionSummary] = []

    for row in roots {
        guard row.count >= 2 else { continue }
        let pID = row[0]
        let absRoot = row[1]
        let stateURL = dataRoot.appendingPathComponent("projects/\(pID)/state.sqlite")
        guard FileManager.default.fileExists(atPath: stateURL.path) else { continue }
        var stateDB: OpaquePointer?
        guard sqlite3_open_v2(stateURL.path, &stateDB, SQLITE_OPEN_READONLY | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK, let stateDB else
↪ {
            continue
        }
        defer { sqlite3_close(stateDB) }

        let sessionRows = (try? rows(stateDB, "SELECT session_id, title, created_at, updated_at FROM sessions ORDER BY updated_at DESC",
↪ [])) ?? []
        for sRow in sessionRows {
            guard sRow.count >= 4 else { continue }
            let sID = sRow[0]
            var title = sRow[1].trimmingCharacters(in: .whitespacesAndNewlines)
            let cDate = parseDate(sRow[2])
            let uDate = parseDate(sRow[3])

            let msgCount = (try? scalar(stateDB, "SELECT COUNT(*) FROM messages WHERE session_id = ?", [sID])).flatMap(Int.init) ?? 0

            if title.isEmpty {
                let firstUserPayload = try? scalar(stateDB, "SELECT payload FROM message_parts WHERE message_id IN (SELECT message_id
↪ FROM messages WHERE session_id = ? AND role = 'user' ORDER BY ordinal LIMIT 1) LIMIT 1", [sID])
                if let firstUserPayload,
                    let data = firstUserPayload.data(using: .utf8),
                    let part = try? JSONDecoder().decode(SessionMessagePart.self, from: data) {
                    switch part {
                    case let .text(txt):
                        let clean = txt.trimmingCharacters(in: .whitespacesAndNewlines)
                            .replacingOccurrences(of: "\n", with: " ")
                        title = clean.count > 50 ? String(clean.prefix(50)) + "...": clean
                    default:
                        break
                    }
                }
            }
            if title.isEmpty {
                title = "未命名会话"
            }

            summaries.append(
                SessionSummary(
                    sessionID: SessionID(sID),
                    title: title,
                    createdAt: cDate,

```

```

        updatedAt: uDate,
        turnCount: msgCount,
        mode: .build,
        reasoningEffort: .auto,
        workingDirectory: absRoot,
        messageCount: msgCount
    )
    )
}

summaries.sort(by: { $0.updatedAt > $1.updatedAt })
return summaries
}

public static func findProjectDirectory(for sessionID: SessionID, dataRoot: URL) throws -> (projectID: ProjectID, absoluteRoot: String)?
↳ {
    let catalogPath = dataRoot.appendingPathComponent("catalog.sqlite")
    guard FileManager.default.fileExists(atPath: catalogPath.path) else { return nil }
    var catalogDB: OpaquePointer?
    guard sqlite3_open_v2(catalogPath.path, &catalogDB, SQLITE_OPEN_READONLY | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK, let catalogDB
↳ else { return nil }
    defer { sqlite3_close(catalogDB) }

    let roots = try rows(catalogDB, "SELECT project_id, absolute_root FROM root_bindings WHERE kind = 'main' AND lifecycle_state = '
↳ active'", [])
    for row in roots {
        guard row.count >= 2 else { continue }
        let pID = row[0]
        let absRoot = row[1]
        let stateURL = dataRoot.appendingPathComponent("projects/\(pID)/state.sqlite")
        guard FileManager.default.fileExists(atPath: stateURL.path) else { continue }
        var stateDB: OpaquePointer?
        guard sqlite3_open_v2(stateURL.path, &stateDB, SQLITE_OPEN_READONLY | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK, let stateDB else
↳ { continue }
        defer { sqlite3_close(stateDB) }
        let exists = (try? scalar(stateDB, "SELECT 1 FROM sessions WHERE session_id = ? LIMIT 1", [sessionID.rawValue])) != nil
        if exists {
            return (ProjectID(pID), absRoot)
        }
    }
    return nil
}

public func loadGlobalSession(_ id: SessionID) throws -> Session? {
    if let local = try loadSessions().first(where: { $0.id == id }) {
        return local
    }
    guard let info = try Self.findProjectDirectory(for: id, dataRoot: dataRoot) else {
        return nil
    }
    let otherStateURL = dataRoot.appendingPathComponent("projects/\(info.projectID.rawValue)/state.sqlite")
    var otherDB: OpaquePointer?
    guard sqlite3_open_v2(otherStateURL.path, &otherDB, SQLITE_OPEN_READONLY | SQLITE_OPEN_FULLMUTEX, nil) == SQLITE_OK, let otherDB
↳ else { return nil }
    defer { sqlite3_close(otherDB) }

    let rows = try Self.rows(otherDB, "SELECT session_id, project_id, kind, parent_session_id, root_session_id, spawned_by_run_id,
↳ spawned_by_tool_call_id, title, cwd_root_binding_id, cwd_relative_path, created_at, updated_at FROM sessions WHERE session_id = ?
↳ LIMIT 1", [id.rawValue])
    guard let row = rows.first, row.count >= 12 else { return nil }
    let messages = try Self.rows(otherDB, "SELECT message_id, role, created_at FROM messages WHERE session_id = ? ORDER BY ordinal", [id
↳ .rawValue]).map { mRow -> Message in
        let parts = try Self.rows(otherDB, "SELECT payload FROM message_parts WHERE message_id = ? ORDER BY ordinal", [mRow[0]]).map {
↳ try JSONDecoder().decode(SessionMessagePart.self, from: Data($0[0].utf8)) }
        guard let role = MessageRole(rawValue: mRow[1]) else { throw PersistenceError.sqlite("invalid message role") }

```

```

        return Message(id: MessageID(mRow[0]), role: role, parts: parts, createdAt: Self.parseDate(mRow[2]))
    }
}

return Session(
    id: id,
    createdAt: Self.parseDate(row[10]),
    kind: SessionKind(rawValue: row[2]) ?? .primary,
    parentSessionID: row[3].isEmpty ? nil : SessionID(row[3]),
    rootSessionID: SessionID(row[4]),
    spawnedByRunID: row[5].isEmpty ? nil : AgentRunID(row[5]),
    spawnedByToolCallID: row[6].isEmpty ? nil : ToolCallID(row[6]),
    title: row[7].isEmpty ? nil : row[7],
    projectID: ProjectID(row[1]),
    cwdRootBindingID: RootBindingID(row[8]),
    cwdRelativePath: ProjectRelativePath(rawValue: row[9]),
    updatedAt: Self.parseDate(row[11]),
    messages: messages
)
}

}

public func loadMessages(sessionID: SessionID) throws -> [Message] {
    try Self.rows(state, "SELECT message_id, role, created_at FROM messages WHERE session_id = ? ORDER BY ordinal", [sessionID.rawValue
↪ ]).map { row in
        let parts = try Self.rows(state, "SELECT payload FROM message_parts WHERE message_id = ? ORDER BY ordinal", [row[0]]).map { try
↪ JSONDecoder().decode(SessionMessagePart.self, from: Data($0[0].utf8)) }
        guard let role = MessageRole(rawValue: row[1]) else { throw PersistenceError.sqlite("invalid message role") }
        return Message(id: MessageID(row[0]), role: role, parts: parts, createdAt: Self.parseDate(row[2]))
    }
}

}

public func deleteSession(_ id: SessionID) throws {
    try Self.transaction(state) {
        try Self.execute(state, "DELETE FROM agent_run_results WHERE run_id IN (SELECT run_id FROM agent_runs WHERE session_id = ?)", [
↪ id.rawValue])
        try Self.execute(state, "DELETE FROM agent_runs WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM compaction_state WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM derived_context WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM session_l2 WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM tool_exchange_batches WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM message_parts WHERE message_id IN (SELECT message_id FROM messages WHERE session_id = ?)",
↪ [id.rawValue])
        try Self.execute(state, "DELETE FROM messages WHERE session_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM sessions WHERE session_id = ?", [id.rawValue])
    }
}

}

public func updateSessionTitle(_ id: SessionID, title: String?) throws {
    try Self.execute(state, "UPDATE sessions SET title = ?, updated_at = ? WHERE session_id = ?", [title ?? NSNull(), Self.date(.now),
↪ id.rawValue])
}

}

public func deleteAgentRun(_ id: AgentRunID) throws {
    try Self.transaction(state) {
        try Self.execute(state, "DELETE FROM agent_run_results WHERE run_id = ?", [id.rawValue])
        try Self.execute(state, "DELETE FROM agent_runs WHERE run_id = ?", [id.rawValue])
    }
}

}

public func saveAgentRun(_ run: AgentRunInfo, profile: SubagentExecutionProfile? = nil) throws {
    try consumeAgentRunFailpoint(run)
    try writeAgentRun(run, profile: profile)
}

}

/// A persistent child is visible only once its first AgentRun is durable too.
public func createChildSessionAndRun(_ session: Session, run: AgentRunInfo, profile: SubagentExecutionProfile? = nil) throws {
    guard session.kind == .subagent, run.sessionID == session.id, run.agentKind == .subagent else {

```

```

        throw PersistenceError.sqlite("child session and AgentRun must agree")
    }
    try consumeAgentRunFailpoint(run)
    try Self.transaction(state) {
        try writeSession(session)
        try writeAgentRun(run, profile: profile)
    }
}

public func loadAgentRuns(sessionID: SessionID? = nil) throws -> [AgentRunInfo] {
    let query = sessionID == nil ? "SELECT run_id, session_id, project_id, parent_run_id, root_run_id, agent_kind, status, provider_id,
    ⇨ account_id, profile_id, model_id, reasoning, context_profile, started_at, finished_at, latest_activity_at, usage_json, error_json,
    ⇨ title FROM agent_runs ORDER BY latest_activity_at" : "SELECT run_id, session_id, project_id, parent_run_id, root_run_id, agent_kind
    ⇨ , status, provider_id, account_id, profile_id, model_id, reasoning, context_profile, started_at, finished_at, latest_activity_at,
    ⇨ usage_json, error_json, title FROM agent_runs WHERE session_id = ? ORDER BY latest_activity_at"
    return try Self.rows(state, query, sessionID.map { [$0.rawValue] } ?? []).compactMap { row in
        guard let kind = SessionKind(rawValue: row[5]), let status = AgentRunStatus(rawValue: row[6]) else { return nil }
        return AgentRunInfo(runID: AgentRunID(row[0]), sessionID: SessionID(row[1]), projectID: ProjectID(row[2]), parentRunID: row[3].
    ⇨ isEmpty ? nil : AgentRunID(row[3]), rootRunID: AgentRunID(row[4]), agentKind: kind, status: status, modelSelection: ModelSelection(
    ⇨ providerID: row[7], accountID: row[8].isEmpty ? nil : row[8], profileID: row[9].isEmpty ? nil : row[9], modelID: row[10], reasoning
    ⇨ : row[11].isEmpty ? nil : row[11], contextProfile: row[12].isEmpty ? nil : row[12]), startedAt: row[13].isEmpty ? nil : Self.
    ⇨ parseDate(row[13]), finishedAt: row[14].isEmpty ? nil : Self.parseDate(row[14]), latestActivityAt: Self.parseDate(row[15]), error:
    ⇨ row[17].isEmpty ? nil : try? JSONDecoder().decode(CoreError.self, from: Data(row[17].utf8)), usage: (try? JSONDecoder().decode(
    ⇨ AgentRunUsage.self, from: Data(row[16].utf8))) ?? AgentRunUsage(), title: row[18].isEmpty ? nil : row[18])
    }
}

public func agentRunProfile(_ runID: AgentRunID) throws -> SubagentExecutionProfile? {
    guard let row = try Self.rows(state, "SELECT permission_profile, tool_profile, budget_profile, context_profile, profile_json FROM
    ⇨ agent_runs WHERE run_id = ?", [runID.rawValue]).first else { return nil }
    if !row[4].isEmpty { return try JSONDecoder().decode(SubagentExecutionProfile.self, from: Data(row[4].utf8)) }
    let tools = row[1].isEmpty ? nil : try? JSONDecoder().decode([String].self, from: Data(row[1].utf8))
    guard !row[0].isEmpty || tools != nil || !row[2].isEmpty || !row[3].isEmpty else { return nil }
    return SubagentExecutionProfile(permissionProfile: row[0].isEmpty ? nil : row[0], toolProfile: tools, budgetProfile: row[2].isEmpty
    ⇨ ? nil : row[2], contextProfile: row[3].isEmpty ? nil : row[3])
}

public func saveAgentRunResult(_ result: SubagentResult) throws {
    try writeAgentRunResult(result)
}

/// Terminal state and its result form one durable fact; callers must not publish either first.
public func saveTerminalAgentRun(_ run: AgentRunInfo, result: SubagentResult, profile: SubagentExecutionProfile? = nil) throws {
    guard run.status.isTerminal, result.runID == run.runID, result.status == run.status else {
        throw PersistenceError.sqlite("terminal AgentRun and result must agree")
    }
    try consumeAgentRunFailpoint(run)
    try Self.transaction(state) {
        try writeAgentRun(run, profile: profile)
        try writeAgentRunResult(result)
    }
}

private func writeSession(_ session: Session) throws {
    guard let root = session.cwdRootBindingID else { throw PersistenceError.missingMainRoot(projectID) }
    try Self.execute(state, "INSERT INTO sessions(session_id, project_id, kind, parent_session_id, root_session_id, spawned_by_run_id,
    ⇨ spawned_by_tool_call_id, title, cwd_root_binding_id, cwd_relative_path, created_at, updated_at, revision, metadata) VALUES(?, ?, ?,
    ⇨ ?, ?, ?, ?, ?, ?, ?, ?, '{')", [session.id.rawValue, projectID.rawValue, session.kind.rawValue, session.parentSessionID?.
    ⇨ rawValue ?? NSNull(), session.rootSessionID.rawValue, session.spawnedByRunID?.rawValue ?? NSNull(), session.spawnedByToolCallID?.
    ⇨ rawValue ?? NSNull(), session.title ?? NSNull(), root.rawValue, session.cwdRelativePath.rawValue, Self.date(session.createdAt),
    ⇨ Self.date(session.createdAt), String(session.revision)])
}

private func consumeAgentRunFailpoint(_ run: AgentRunInfo) throws {
    if case let .beforeSaveAgentRun(targetKind) = failpoint, targetKind == nil || targetKind == run.agentKind {

```

```

        failpoint = nil
        throw PersistenceError.sqlite("injected saveAgentRun failure")
    }
}

private func writeAgentRun(_ run: AgentRunInfo, profile: SubagentExecutionProfile?) throws {
    let selection = run.modelSelection
    let usage = String(decoding: try JSONEncoder().encode(run.usage), as: UTF8.self)
    let error = try run.error.map { try String(decoding: JSONEncoder().encode($0), as: UTF8.self) }
    let tools = try profile?.toolProfile.map { try String(decoding: JSONEncoder().encode($0), as: UTF8.self) }
    let profileJSON = try profile.map { String(decoding: try JSONEncoder().encode($0), as: UTF8.self) }
    try Self.execute(state, "INSERT OR REPLACE INTO agent_runs(run_id, session_id, project_id, parent_run_id, root_run_id, agent_kind,
    ↪ status, provider_id, account_id, profile_id, model_id, reasoning, context_profile, permission_profile, tool_profile, budget_profile
    ↪ , profile_json, started_at, finished_at, latest_activity_at, usage_json, error_json, title) VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,
    ↪ ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", [run.runID.rawValue, run.sessionID.rawValue, run.projectID.rawValue ?? projectID.rawValue,
    ↪ run.parentRunID?.rawValue ?? NSNull(), run.rootRunID.rawValue, run.agentKind.rawValue, run.status.rawValue, selection.providerID,
    ↪ selection.accountID ?? NSNull(), selection.profileID ?? NSNull(), selection.modelID, selection.reasoning ?? NSNull(), selection.
    ↪ contextProfile ?? NSNull(), profile?.permissionProfile ?? NSNull(), tools ?? NSNull(), profile?.budgetProfile ?? NSNull(),
    ↪ profileJSON ?? NSNull(), run.startedAt.map(Self.date) ?? NSNull(), run.finishedAt.map(Self.date) ?? NSNull(), Self.date(run.
    ↪ latestActivityAt), usage, error ?? NSNull(), run.title ?? NSNull()])
}

private func writeAgentRunResult(_ result: SubagentResult) throws {
    let error = try result.error.map { try String(decoding: JSONEncoder().encode($0), as: UTF8.self) }
    try Self.execute(state, "INSERT OR REPLACE INTO agent_run_results(run_id, status, final_text, touched_resources_json,
    ↪ artifact_refs_json, usage_json, error_json, timestamp) VALUES(?, ?, ?, ?, ?, ?, ?, ?)", [result.runID.rawValue, result.status.
    ↪ rawValue, result.finalText ?? NSNull(), String(decoding: try JSONEncoder().encode(result.touchedResources), as: UTF8.self), String(
    ↪ decoding: try JSONEncoder().encode(result.artifactReferences), as: UTF8.self), String(decoding: try JSONEncoder().encode(result.
    ↪ usage), as: UTF8.self), error ?? NSNull(), Self.date(result.timestamp)])
}

public func agentRunResult(_ runID: AgentRunID) throws -> SubagentResult? {
    guard let row = try Self.rows(state, "SELECT r.session_id, x.status, x.final_text, x.touched_resources_json, x.artifact_refs_json, x
    ↪ .usage_json, x.error_json, x.timestamp FROM agent_runs r JOIN agent_run_results x ON x.run_id = r.run_id WHERE r.run_id = ?", [
    ↪ runID.rawValue]).first, let status = AgentRunStatus(rawValue: row[1]) else { return nil }
    return SubagentResult(childSessionID: SessionID(row[0]), runID: runID, status: status, finalText: row[2].isEmpty ? nil : row[2],
    ↪ touchedResources: (try? JSONDecoder().decode([ToolTouchedResource].self, from: Data(row[3].utf8))) ?? [], artifactReferences: (try?
    ↪ JSONDecoder().decode([String].self, from: Data(row[4].utf8))) ?? [], usage: (try? JSONDecoder().decode(AgentRunUsage.self, from:
    ↪ Data(row[5].utf8))) ?? AgentRunUsage(), error: row[6].isEmpty ? nil : try? JSONDecoder().decode(CoreError.self, from: Data(row[6].
    ↪ utf8)), timestamp: Self.parseDate(row[7]))
}

/// Snapshot and normalized task rows are committed together; recovery reads the snapshot only.
public func saveWorkflow(_ workflow: WorkflowSnapshot) throws {
    let encoder = JSONEncoder()
    let snapshot = String(decoding: try encoder.encode(workflow), as: UTF8.self)
    try Self.transaction(state) {
        try Self.execute(state, "INSERT OR REPLACE INTO workflows(workflow_id, project_id, root_session_id, root_run_id, status,
    ↪ checkpoint_json, snapshot_json, created_at, updated_at) VALUES(?, ?, ?, ?, ?, ?, ?, ?)", [workflow.id.rawValue, projectID.
    ↪ rawValue, workflow.rootSessionID.rawValue, workflow.rootRunID.rawValue, workflow.status.rawValue, String(decoding: try encoder.
    ↪ encode(workflow.checkpoint), as: UTF8.self), snapshot, Self.date(workflow.createdAt), Self.date(workflow.updatedAt)])
        try Self.execute(state, "DELETE FROM workflow_pending_inputs WHERE workflow_id = ?", [workflow.id.rawValue])
        try Self.execute(state, "DELETE FROM workflow_dependencies WHERE workflow_id = ?", [workflow.id.rawValue])
        try Self.execute(state, "DELETE FROM workflow_tasks WHERE workflow_id = ?", [workflow.id.rawValue])
        for task in workflow.tasks {
            let definition = String(decoding: try encoder.encode(task.definition), as: UTF8.self)
            let provenance = try task.provenance.map { String(decoding: try encoder.encode($0), as: UTF8.self) }
            let result = try task.result.map { String(decoding: try encoder.encode($0), as: UTF8.self) }
            let error = try task.error.map { String(decoding: try encoder.encode($0), as: UTF8.self) }
            try Self.execute(state, "INSERT INTO workflow_tasks(workflow_id, task_id, status, definition_json, provenance_json,
    ↪ result_json, error_json) VALUES(?, ?, ?, ?, ?, ?, ?)", [workflow.id.rawValue, task.definition.id.rawValue, task.status.rawValue,
    ↪ definition, provenance ?? NSNull(), result ?? NSNull(), error ?? NSNull()])
            for dependency in task.definition.dependencies {
                try Self.execute(state, "INSERT INTO workflow_dependencies(workflow_id, task_id, dependency_task_id) VALUES(?, ?, ?)", [
    ↪ workflow.id.rawValue, task.definition.id.rawValue, dependency.rawValue])
            }
        }
    }
}

```

```

    }
    if let pending = task.pendingInput {
        let kind: String
        switch pending { case .question: kind = "question"; case .permission: kind = "permission"; case .decision: kind = "
↪ decision" }
        try Self.execute(state, "INSERT INTO workflow_pending_inputs(workflow_id, task_id, kind, payload_json) VALUES(?, ?, ?,
↪ ?)", [workflow.id.rawValue, task.definition.id.rawValue, kind, String(decoding: try encoder.encode(pending), as: UTF8.self)])
    }
}
}

public func loadWorkflows() throws -> [WorkflowSnapshot] {
    try Self.rows(state, "SELECT snapshot_json FROM workflows WHERE project_id = ? ORDER BY created_at", [projectID.rawValue]).
↪ compactMap { row in
        try? JSONDecoder().decode(WorkflowSnapshot.self, from: Data(row[0].utf8))
    }
}

public func saveDerived(_ page: DerivedContextPage) throws {
    try writeDerived(page)
}

public func loadDerived() throws -> [DerivedContextPage] {
    try Self.rows(state, "SELECT derived_page_id, session_id, source_kind, inline_content, blob_ref, message_id, token_estimate,
↪ created_at, version, provenance_json, metadata_json FROM derived_context WHERE project_id = ? ORDER BY created_at", [projectID.
↪ rawValue]).compactMap { row in
        guard let kind = DerivedContextSourceKind(rawValue: row[2]) else { return nil }
        let content = row[3].isEmpty ? ((try? blobs.get(row[4])).flatMap { String(data: $0, encoding: .utf8) } ?? "") : row[3]
        return DerivedContextPage(id: row[0], sessionId: SessionID(row[1]), sourceKind: kind, content: content, messageId: row[5],
↪ isEmpty ? nil : MessageID(row[5]), tokenEstimate: Int(row[6]) ?? 0, provenanceIDs: (try? JSONDecoder().decode([String].self, from:
↪ Data(row[9].utf8))) ?? [], metadata: (try? JSONDecoder().decode([String: String].self, from: Data(row[10].utf8))) ?? [:], createdAt
↪ : Self.parseDate(row[7]), version: Int(row[8]) ?? 1)
    }
}

public func saveCompaction(sessionID: SessionID, generation: Int, residencies: [ContextUnitDebugSnapshot], derivedPages: [
↪ DerivedContextPage] = []) throws {
    let fail = failpoint == .beforeCompactionCommit
    failpoint = nil
    try Self.transaction(state) {
        for page in derivedPages { try writeDerived(page, database: state) }
        if fail { throw PersistenceError.sqlite("injected compaction failure") }
        try Self.execute(state, "INSERT OR REPLACE INTO compaction_state(session_id, generation, residency_json, updated_at) VALUES(?,
↪ ?, ?, ?)", [sessionID.rawValue, String(generation), String(decoding: try JSONEncoder().encode(residencies), as: UTF8.self), Self.
↪ now])
    }
}

public func armFailpoint(_ value: PersistenceFailpoint) { failpoint = value }

public func compaction(sessionID: SessionID) throws -> (generation: Int, residencies: [ContextUnitDebugSnapshot])? {
    guard let row = try Self.rows(state, "SELECT generation, residency_json FROM compaction_state WHERE session_id = ?", [sessionID.
↪ rawValue]).first else { return nil }
    return (Int(row[0]) ?? 0, (try? JSONDecoder().decode([ContextUnitDebugSnapshot].self, from: Data(row[1].utf8))) ?? [])
}

public func saveToolBatch(_ batch: ToolExchangeBatch) throws {
    try Self.writeBatch(state, batch)
}

public func saveToolBatches(_ batches: [ToolExchangeBatch]) throws {
    guard !batches.isEmpty else { return }
    try Self.transaction(state) {
        for batch in batches { try Self.writeBatch(state, batch) }
    }
}

```



```

    }
}

public func storeToolOutput(_ output: String) throws -> String {
    try blobs.put(Data(output.utf8))
}

/// Rebuildable project cache. Bodies remain in the filesystem (or blob store), never here.
public func replaceProjectCache(pages: [ContextPage], symbols: [Symbol], references: [ProjectReference], dependencies: [DependencyEdge])
    ↪ throws {
    try Self.transaction(state) {
        for table in ["project_pages", "cached_symbols", "cached_references", "cached_dependencies"] {
            try Self.execute(state, "DELETE FROM \(table) WHERE project_id = ?", [projectID.rawValue])
        }
        for page in pages {
            guard let fileID = page.fileID else { continue }
            try Self.execute(state, "INSERT INTO project_pages(page_id, project_id, file_id, start_line, end_line, content_hash, version
            ↪ , source_type, characters, metadata) VALUES(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", [page.id, projectID.rawValue, fileID.rawValue, String(
            ↪ page.startLine), String(page.endLine), page.hash, page.version, page.sourceType.rawValue, String(page.characterCount), page.
            ↪ metadata.heading ?? ""])
        }
        for symbol in symbols {
            guard let fileID = symbol.fileID else { continue }
            try Self.execute(state, "INSERT INTO cached_symbols(symbol_id, project_id, file_id, name, qualified_name, kind, line,
            ↪ page_id) VALUES(?, ?, ?, ?, ?, ?, ?, ?)", [symbol.id.rawValue, projectID.rawValue, fileID.rawValue, symbol.name, symbol.
            ↪ qualifiedName, symbol.kind.rawValue, String(symbol.line), symbol.pageID])
        }
        for reference in references {
            guard let fileID = reference.sourceFileID else { continue }
            try Self.execute(state, "INSERT INTO cached_references(reference_id, project_id, source_file_id, target_file_id, source_line
            ↪ , target_name, kind, resolution) VALUES(?, ?, ?, ?, ?, ?, ?, ?)", [reference.id.rawValue, projectID.rawValue, fileID.rawValue,
            ↪ reference.targetFileID?.rawValue ?? NSNull(), String(reference.sourceLine), reference.targetName, reference.kind.rawValue,
            ↪ reference.resolutionQuality.rawValue])
        }
        for dependency in dependencies {
            guard let source = dependency.sourceFileID else { continue }
            try Self.execute(state, "INSERT INTO cached_dependencies(project_id, source_file_id, target_file_id, kind, evidence_id)
            ↪ VALUES(?, ?, ?, ?, ?)", [projectID.rawValue, source.rawValue, dependency.targetFileID?.rawValue ?? NSNull(), dependency.kind.
            ↪ rawValue, dependency.evidence.rawValue])
        }
        try Self.execute(state, "INSERT OR REPLACE INTO persistence_metadata(key, value) VALUES('index_format_version', ?)", [String(
            ↪ Self.indexFormatVersion)])
    }
}

/// 索引是可重建缓存; 版本变化时只清缓存, 绝不触碰 Session、Derived 或 File identity。
@discardableResult
public func invalidateCachesIfFormatMismatch() throws -> Bool {
    let current = Int(try Self.scalar(state, "SELECT value FROM persistence_metadata WHERE key = 'index_format_version'", []) ?? "0") ??
    ↪ 0
    guard current != Self.indexFormatVersion else { return false }
    try Self.transaction(state) {
        for table in ["project_pages", "cached_symbols", "cached_references", "cached_dependencies"] {
            try Self.execute(state, "DELETE FROM \(table) WHERE project_id = ?", [projectID.rawValue])
        }
        try Self.execute(state, "INSERT OR REPLACE INTO persistence_metadata(key, value) VALUES('index_format_version', ?)", [String(
            ↪ Self.indexFormatVersion)])
    }
    return true
}

public func setCacheFormatVersionForTesting(_ version: Int) throws {
    try Self.execute(state, "INSERT OR REPLACE INTO persistence_metadata(key, value) VALUES('index_format_version', ?)", [String(version
    ↪ )])
}

```

```

public func cacheCounts() throws -> (pages: Int, symbols: Int, references: Int, dependencies: Int) {
    func count(_ table: String) throws -> Int { try Self.scalar(state, "SELECT COUNT(*) FROM \ \(table) WHERE project_id = ?", [projectID,
    ↪ rawValue]).flatMap(Int.init) ?? 0 }
    return (try count("project_pages"), try count("cached_symbols"), try count("cached_references"), try count("cached_dependencies"))
}

public func toolBatches(sessionID: SessionID) throws -> [ToolExchangeBatch] {
    try Self.rows(state, "SELECT batch_id, assistant_message_id, result_message_id, provider_step, state, estimated_tokens,
    ↪ tool_calls_json, tool_results_json, continuation_request_id, tool_call_states_json FROM tool_exchange_batches WHERE session_id = ?
    ↪ ORDER BY provider_step", [sessionID.rawValue]).compactMap { row -> ToolExchangeBatch? in
        guard let persisted = ToolExchangeBatchState(rawValue: row[4]) else { return nil }
        let calls = (try? JSONDecoder().decode([ToolCall].self, from: Data(row[6].utf8))) ?? []
        let results = (try? JSONDecoder().decode([ToolResult].self, from: Data(row[7].utf8))) ?? []
        let states = try? JSONDecoder().decode([DurableToolCall].self, from: Data(row[9].utf8))
        let recoveredStates = ((states?.isEmpty == false ? states : nil) ?? calls.map { call in
            let result = results.first { $0.callID == call.callID }
            return DurableToolCall(call: call, state: result == nil ? .recoveryRequired : .completed, provenance: ToolCallProvenance(
    ↪ batchID: row[0], sessionID: sessionID, agentRunID: nil, providerRequestID: row[8].isEmpty ? nil : ModelRequestID(row[8]),
    ↪ providerStep: Int(row[3]) ?? 0), result: result)
        }).map(Self.recoverToolCall)
        let recovered: ToolExchangeBatchState = recoveredStates.contains(where: { $0.state == .recoveryRequired }) ? .recoveryRequired :
    ↪ persisted
        return ToolExchangeBatch(batchID: row[0], sessionID: sessionID, assistantMessageID: MessageID(row[1]), resultMessageID: row[2].
    ↪ isEmpty ? nil : MessageID(row[2]), toolCalls: calls, toolResults: results, toolCallStates: recoveredStates, continuationRequestID:
    ↪ row[8].isEmpty ? nil : ModelRequestID(row[8]), providerStep: Int(row[3]) ?? 0, state: recovered, estimatedTokens: Int(row[5]) ?? 0)
    }
}

public func statistics() throws -> (stateBytes: Int64, walBytes: Int64, blobBytes: Int64, sessions: Int, messages: Int, derived: Int,
    ↪ files: Int) {
    let stateURL = dataRoot.appendingPathComponent("projects").appendingPathComponent(projectID.rawValue).appendingPathComponent("state.
    ↪ sqlite")
    let size = Int64((try? stateURL.resourceValues(forKeys: [.fileSizeKey]).fileSize) ?? 0)
    let wal = Int64((try? URL(fileURLWithPath: stateURL.path + "-wal").resourceValues(forKeys: [.fileSizeKey]).fileSize) ?? 0)
    func count(_ table: String) throws -> Int { try Self.scalar(state, "SELECT COUNT(*) FROM \ \(table)", []).flatMap(Int.init) ?? 0 }
    return (size, wal, try blobs.byteCount(), try count("sessions"), try count("messages"), try count("derived_context"), try count("
    ↪ project_files"))
}

public func integrityCheck() throws -> Bool {
    try Self.scalar(catalog, "PRAGMA quick_check", []) == "ok" && Self.scalar(catalog, "PRAGMA foreign_key_check", []) == nil && Self.
    ↪ scalar(state, "PRAGMA quick_check", []) == "ok" && Self.scalar(state, "PRAGMA foreign_key_check", []) == nil
}

/// 只检查 durable locator 列；正文、JSON payload 与 Tool 输出均不是路径事实。
public func structuredAbsolutePathViolations(containing prefixes: [String]) throws -> [StructuredPathAuditViolation] {
    let checks = [
        ("sessions", "cwd_relative_path"),
        ("project_files", "relative_path"),
        ("project_files", "root_binding_id"),
        ("tool_exchange_batches", "assistant_message_id"),
        ("tool_exchange_batches", "result_message_id"),
        ("derived_context", "message_id"),
        ("derived_context", "blob_ref")
    ]
    var violations: [StructuredPathAuditViolation] = []
    for (table, column) in checks {
        for row in try Self.rows(state, "SELECT \ \(column) FROM \ \(table)", []) {
            guard let value = row.first, prefixes.contains(where: { value.contains($0) }) else { continue }
            violations.append(StructuredPathAuditViolation(location: "state.\ \(table).\ \(column)", value: value))
        }
    }
    // catalog 中唯一允许的 absolute filesystem locator 是 root_bindings.absolute_root。
    return violations
}

```

```

}

private static var now: String { date(.now) }
private static func date(_ value: Date) -> String { String(value.timeIntervalSince1970) }
private static func parseDate(_ value: String) -> Date { Date(timeIntervalSince1970: Double(value) ?? 0) }
private static func open(_ url: URL) throws -> OpaquePointer {
    var db: OpaquePointer?; guard sqlite3_open_v2(url.path, &db, SQLITE_OPEN_CREATE | SQLITE_OPEN_READWRITE | SQLITE_OPEN_FULLMUTEX, nil
    ↪ ) == SQLITE_OK, let db else { throw PersistenceError.sqlite("open \(url.lastPathComponent)") }; return db
}
private static func configure(_ db: OpaquePointer) throws { try script(db, "PRAGMA foreign_keys = ON; PRAGMA journal_mode = WAL; PRAGMA
    ↪ synchronous = NORMAL; PRAGMA busy_timeout = 5000") }
private static func migrate(_ db: OpaquePointer, create: () throws -> Void, upgrade: () throws -> Void, upgradeV3: () throws -> Void,
    ↪ upgradeV4: () throws -> Void, upgradeV5: () throws -> Void, upgradeV6: () throws -> Void) throws {
    let version = Int(try scalar(db, "PRAGMA user_version", []) ?? "0") ?? 0
    try transaction(db) { try MigrationRunner.migrate(from: version, applyV0ToV1: create, applyV1ToV2: upgrade, applyV2ToV3: upgradeV3,
    ↪ applyV3ToV4: upgradeV4, applyV4ToV5: upgradeV5, applyV5ToV6: upgradeV6) }
}
private static func transaction(_ db: OpaquePointer, _ body: () throws -> Void) throws { try execute(db, "BEGIN IMMEDIATE", []); do {
    ↪ try body(); try execute(db, "COMMIT", []) } catch { try? execute(db, "ROLLBACK", []); throw error } }
private static func nextMessageOrdinal(_ db: OpaquePointer, _ sessionID: SessionID) throws -> Int { try scalar(db, "SELECT COALESCE(MAX(
    ↪ ordinal), -1) + 1 FROM messages WHERE session_id = ?", [sessionID.rawValue]).flatMap(Int.init) ?? 0 }
private static func insertMessage(_ db: OpaquePointer, sessionID: SessionID, message: Message, ordinal: Int) throws {
    try execute(db, "INSERT INTO messages(message_id, session_id, ordinal, role, created_at) VALUES(?, ?, ?, ?, ?)", [message.id.
    ↪ rawValue, sessionID.rawValue, String(ordinal), message.role.rawValue, date(message.createdAt)])
    for (partOrdinal, part) in message.parts.enumerated() { try execute(db, "INSERT INTO message_parts(message_id, ordinal, payload)
    ↪ VALUES(?, ?, ?)", [message.id.rawValue, String(partOrdinal), String(decoding: try JSONEncoder().encode(part), as: UTF8.self)]) }
    try execute(db, "UPDATE sessions SET updated_at = ? WHERE session_id = ?", [date(message.createdAt), sessionID.rawValue])
}
private static func writeBatch(_ db: OpaquePointer, _ batch: ToolExchangeBatch) throws {
    try execute(db, "INSERT OR REPLACE INTO tool_exchange_batches(batch_id, session_id, assistant_message_id, result_message_id,
    ↪ provider_step, state, estimated_tokens, tool_calls_json, tool_results_json, continuation_request_id, tool_call_states_json) VALUES
    ↪ (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", [batch.batchID, batch.sessionID.rawValue, batch.assistantMessageID.rawValue, batch.
    ↪ resultMessageID?.rawValue ?? NSNull(), String(batch.providerStep), batch.state.rawValue, String(batch.estimatedTokens), String(
    ↪ decoding: try JSONEncoder().encode(batch.toolCalls), as: UTF8.self), String(decoding: try JSONEncoder().encode(batch.toolResults),
    ↪ as: UTF8.self), batch.continuationRequestID?.rawValue ?? NSNull(), String(decoding: try JSONEncoder().encode(batch.toolCallStates),
    ↪ as: UTF8.self)])
}
private static func recoverToolCall(_ call: DurableToolCall) -> DurableToolCall {
    switch call.state {
    case .completed, .waitingForHuman, .requested, .recoveryRequired:
        return call
    case .executing:
        return call.executionClaim?.mutatesProject == true ? call.with(state: .recoveryRequired) : call.with(state: .requested)
    }
}
private func writeDerived(_ page: DerivedContextPage, database: OpaquePointer? = nil) throws {
    let contentRef: String?
    let inline: String?
    if page.content.utf8.count > 8 * 1024 {
        contentRef = try blobs.put(Data(page.content.utf8))
        inline = nil
    } else {
        contentRef = nil
        inline = page.content
    }
    try Self.execute(database ?? state, "INSERT OR REPLACE INTO derived_context(derived_page_id, project_id, session_id, source_kind,
    ↪ content_hash, inline_content, blob_ref, message_id, token_estimate, created_at, version, provenance_json, metadata_json) VALUES(?,
    ↪ ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)", [page.id, projectID.rawValue, page.sessionID.rawValue, page.sourceKind.rawValue, page.
    ↪ contentHash, inline ?? NSNull(), contentRef ?? NSNull(), page.messageID.rawValue ?? NSNull(), String(page.tokenEstimate), Self.
    ↪ date(page.createdAt), String(page.version), String(decoding: try JSONEncoder().encode(page.provenanceIDs), as: UTF8.self), String(
    ↪ decoding: try JSONEncoder().encode(page.metadata), as: UTF8.self)])
}
private static func execute(_ db: OpaquePointer, _ sql: String, _ values: [Any]) throws { var statement: OpaquePointer?; guard
    ↪ sqlite3_prepare_v2(db, sql, -1, &statement, nil) == SQLITE_OK, let statement else { throw PersistenceError.sqlite(String(cString:
    ↪ sqlite3_errmsg(db))) }; defer { sqlite3_finalize(statement) }; try bind(statement, values); guard sqlite3_step(statement) ==

```

```

    ↪ SQLITE_DONE else { throw PersistenceError.sqlite(String(cString: sqlite3_errmsg(db))) } }
private static func script(_ db: OpaquePointer, _ sql: String) throws { var error: UnsafeMutablePointer<CChar?>; guard sqlite3_exec(db,
    ↪ sql, nil, nil, &error) == SQLITE_OK else { defer { sqlite3_free(error) }; throw PersistenceError.sqlite(error.map { String(cString:
    ↪ $0) } ?? String(cString: sqlite3_errmsg(db))) } }
private static func scalar(_ db: OpaquePointer, _ sql: String, _ values: [Any]) throws -> String? { try rows(db, sql, values).first?.
    ↪ first }
private static func rows(_ db: OpaquePointer, _ sql: String, _ values: [Any]) throws -> [[String]] { var statement: OpaquePointer?;
    ↪ guard sqlite3_prepare_v2(db, sql, -1, &statement, nil) == SQLITE_OK, let statement else { throw PersistenceError.sqlite(String(
    ↪ cString: sqlite3_errmsg(db))) }; defer { sqlite3_finalize(statement) }; try bind(statement, values); var result: [[String]] = [];
    ↪ while sqlite3_step(statement) == SQLITE_ROW { result.append((0..<Int(sqlite3_column_count(statement))).map { sqlite3_column_text(
    ↪ statement, Int32($0)).map { String(cString: $0) } ?? "" })); return result }
private static func bind(_ statement: OpaquePointer, _ values: [Any]) throws { for (index, value) in values.enumerated() { let i = Int32
    ↪ (index + 1); let rc: Int32; if value is NSNull { rc = sqlite3_bind_null(statement, i) } else { rc = sqlite3_bind_text(statement, i,
    ↪ String(describing: value), -1, SQLITE_TRANSIENT) }; guard rc == SQLITE_OK else { throw PersistenceError.sqlite("bind") } } }
private static func createCatalogSchema(_ db: OpaquePointer) throws { try script(db, "CREATE TABLE IF NOT EXISTS projects(project_id
    ↪ TEXT PRIMARY KEY, created_at TEXT NOT NULL, updated_at TEXT NOT NULL); CREATE TABLE IF NOT EXISTS root_bindings(binding_id TEXT
    ↪ PRIMARY KEY, project_id TEXT NOT NULL REFERENCES projects(project_id), kind TEXT NOT NULL, absolute_root TEXT NOT NULL,
    ↪ parent_binding_id TEXT REFERENCES root_bindings(binding_id), binding_revision INTEGER NOT NULL, lifecycle_state TEXT NOT NULL,
    ↪ time_created TEXT NOT NULL, time_updated TEXT NOT NULL, time_last_seen TEXT); CREATE UNIQUE INDEX IF NOT EXISTS
    ↪ one_active_main_root ON root_bindings(project_id) WHERE kind = 'main' AND lifecycle_state = 'active'; PRAGMA user_version = 1") }
private static func upgradeStateSchemaV2(_ db: OpaquePointer) throws {
    try script(db, "ALTER TABLE sessions ADD COLUMN kind TEXT NOT NULL DEFAULT 'primary'; ALTER TABLE sessions ADD COLUMN
    ↪ parent_session_id TEXT; ALTER TABLE sessions ADD COLUMN root_session_id TEXT; ALTER TABLE sessions ADD COLUMN spawned_by_run_id
    ↪ TEXT; ALTER TABLE sessions ADD COLUMN spawned_by_tool_call_id TEXT; ALTER TABLE sessions ADD COLUMN title TEXT; UPDATE sessions SET
    ↪ root_session_id = session_id WHERE root_session_id IS NULL; CREATE INDEX IF NOT EXISTS sessions_parent_idx ON sessions(
    ↪ parent_session_id); CREATE TABLE IF NOT EXISTS agent_runs(run_id TEXT PRIMARY KEY, session_id TEXT NOT NULL REFERENCES sessions(
    ↪ session_id), project_id TEXT NOT NULL, parent_run_id TEXT, root_run_id TEXT NOT NULL, agent_kind TEXT NOT NULL, status TEXT NOT
    ↪ NULL, provider_id TEXT NOT NULL, model_id TEXT NOT NULL, reasoning TEXT, context_profile TEXT, permission_profile TEXT,
    ↪ tool_profile TEXT, budget_profile TEXT, started_at TEXT, finished_at TEXT, latest_activity_at TEXT NOT NULL, usage_json TEXT NOT
    ↪ NULL, error_json TEXT, title TEXT); CREATE INDEX IF NOT EXISTS agent_runs_session_idx ON agent_runs(session_id, latest_activity_at)
    ↪ ; CREATE TABLE IF NOT EXISTS agent_run_results(run_id TEXT PRIMARY KEY REFERENCES agent_runs(run_id), status TEXT NOT NULL,
    ↪ final_text TEXT, touched_resources_json TEXT NOT NULL, artifact_refs_json TEXT NOT NULL, usage_json TEXT NOT NULL, error_json TEXT,
    ↪ timestamp TEXT NOT NULL); PRAGMA user_version = 2")
}
private static func upgradeStateSchemaV3(_ db: OpaquePointer) throws {
    try script(db, "ALTER TABLE agent_runs ADD COLUMN account_id TEXT; ALTER TABLE agent_runs ADD COLUMN profile_id TEXT; PRAGMA
    ↪ user_version = 3")
}
private static func upgradeStateSchemaV4(_ db: OpaquePointer) throws { try script(db, "ALTER TABLE agent_runs ADD COLUMN profile_json
    ↪ TEXT; ALTER TABLE tool_exchange_batches ADD COLUMN continuation_request_id TEXT; PRAGMA user_version = 4") }
private static func upgradeStateSchemaV5(_ db: OpaquePointer) throws { try script(db, "CREATE TABLE IF NOT EXISTS workflows(workflow_id
    ↪ TEXT PRIMARY KEY, project_id TEXT NOT NULL, root_session_id TEXT NOT NULL REFERENCES sessions(session_id), root_run_id TEXT NOT
    ↪ NULL REFERENCES agent_runs(run_id), status TEXT NOT NULL, checkpoint_json TEXT NOT NULL, snapshot_json TEXT NOT NULL, created_at
    ↪ TEXT NOT NULL, updated_at TEXT NOT NULL); CREATE TABLE IF NOT EXISTS workflow_tasks(workflow_id TEXT NOT NULL REFERENCES workflows(
    ↪ workflow_id), task_id TEXT NOT NULL, status TEXT NOT NULL, definition_json TEXT NOT NULL, provenance_json TEXT, result_json TEXT,
    ↪ error_json TEXT, PRIMARY KEY(workflow_id, task_id)); CREATE TABLE IF NOT EXISTS workflow_dependencies(workflow_id TEXT NOT NULL
    ↪ REFERENCES workflows(workflow_id), task_id TEXT NOT NULL, dependency_task_id TEXT NOT NULL, PRIMARY KEY(workflow_id, task_id,
    ↪ dependency_task_id)); CREATE TABLE IF NOT EXISTS workflow_pending_inputs(workflow_id TEXT NOT NULL REFERENCES workflows(workflow_id
    ↪ ), task_id TEXT NOT NULL, kind TEXT NOT NULL, payload_json TEXT NOT NULL, PRIMARY KEY(workflow_id, task_id)); CREATE INDEX IF NOT
    ↪ EXISTS workflow_status_idx ON workflows(project_id, status); PRAGMA user_version = 5") }
private static func upgradeStateSchemaV6(_ db: OpaquePointer) throws { try script(db, "ALTER TABLE tool_exchange_batches ADD COLUMN
    ↪ tool_call_states_json TEXT NOT NULL DEFAULT '[]'; PRAGMA user_version = 6") }
private static func createStateSchema(_ db: OpaquePointer) throws {
    try script(db, "CREATE TABLE IF NOT EXISTS sessions(session_id TEXT PRIMARY KEY, project_id TEXT NOT NULL, cwd_root_binding_id TEXT
    ↪ NOT NULL, cwd_relative_path TEXT NOT NULL, created_at TEXT NOT NULL, updated_at TEXT NOT NULL, revision INTEGER NOT NULL DEFAULT 0,
    ↪ metadata TEXT NOT NULL); CREATE TABLE IF NOT EXISTS messages(message_id TEXT PRIMARY KEY, session_id TEXT NOT NULL REFERENCES
    ↪ sessions(session_id), ordinal INTEGER NOT NULL, role TEXT NOT NULL, created_at TEXT NOT NULL, UNIQUE(session_id, ordinal)); CREATE
    ↪ TABLE IF NOT EXISTS message_parts(message_id TEXT NOT NULL REFERENCES messages(message_id), ordinal INTEGER NOT NULL, payload TEXT
    ↪ NOT NULL, PRIMARY KEY(message_id, ordinal)); CREATE TABLE IF NOT EXISTS tool_exchange_batches(batch_id TEXT PRIMARY KEY, session_id
    ↪ TEXT NOT NULL REFERENCES sessions(session_id), assistant_message_id TEXT NOT NULL, result_message_id TEXT, provider_step INTEGER
    ↪ NOT NULL, state TEXT NOT NULL, estimated_tokens INTEGER NOT NULL, tool_calls_json TEXT NOT NULL, tool_results_json TEXT NOT NULL);
    ↪ CREATE TABLE IF NOT EXISTS derived_context(derived_page_id TEXT PRIMARY KEY, project_id TEXT NOT NULL, session_id TEXT NOT NULL
    ↪ REFERENCES sessions(session_id), source_kind TEXT NOT NULL, content_hash TEXT NOT NULL, inline_content TEXT, blob_ref TEXT,
    ↪ message_id TEXT, token_estimate INTEGER NOT NULL, created_at TEXT NOT NULL, version INTEGER NOT NULL, provenance_json TEXT NOT NULL
    ↪ , metadata_json TEXT NOT NULL); CREATE TABLE IF NOT EXISTS compaction_state(session_id TEXT PRIMARY KEY REFERENCES sessions(

```

```

    ↪ session_id), generation INTEGER NOT NULL, residency_json TEXT NOT NULL, updated_at TEXT NOT NULL); CREATE TABLE IF NOT EXISTS
    ↪ project_files(file_id TEXT PRIMARY KEY, project_id TEXT NOT NULL, root_binding_id TEXT NOT NULL, relative_path TEXT NOT NULL,
    ↪ content_hash TEXT NOT NULL, version TEXT NOT NULL, state TEXT NOT NULL, time_created TEXT NOT NULL, time_updated TEXT NOT NULL,
    ↪ time_last_seen TEXT, UNIQUE(root_binding_id, relative_path)); CREATE TABLE IF NOT EXISTS project_pages(page_id TEXT PRIMARY KEY,
    ↪ project_id TEXT NOT NULL, file_id TEXT NOT NULL, start_line INTEGER NOT NULL, end_line INTEGER NOT NULL, content_hash TEXT NOT NULL
    ↪ , version TEXT NOT NULL, source_type TEXT NOT NULL, characters INTEGER NOT NULL, metadata TEXT NOT NULL); CREATE TABLE IF NOT
    ↪ EXISTS cached_symbols(symbol_id TEXT PRIMARY KEY, project_id TEXT NOT NULL, file_id TEXT NOT NULL, name TEXT NOT NULL,
    ↪ qualified_name TEXT NOT NULL, kind TEXT NOT NULL, line INTEGER NOT NULL, page_id TEXT NOT NULL); CREATE TABLE IF NOT EXISTS
    ↪ cached_references(reference_id TEXT PRIMARY KEY, project_id TEXT NOT NULL, source_file_id TEXT NOT NULL, target_file_id TEXT,
    ↪ source_line INTEGER NOT NULL, target_name TEXT NOT NULL, kind TEXT NOT NULL, resolution TEXT NOT NULL); CREATE TABLE IF NOT EXISTS
    ↪ cached_dependencies(project_id TEXT NOT NULL, source_file_id TEXT NOT NULL, target_file_id TEXT, kind TEXT NOT NULL, evidence_id
    ↪ TEXT NOT NULL, PRIMARY KEY(project_id, source_file_id, evidence_id)); CREATE TABLE IF NOT EXISTS project_l2(page_id TEXT PRIMARY
    ↪ KEY, project_id TEXT NOT NULL, score REAL NOT NULL, use_count INTEGER NOT NULL, last_used INTEGER NOT NULL, version TEXT NOT NULL);
    ↪ CREATE TABLE IF NOT EXISTS session_l2(derived_page_id TEXT PRIMARY KEY, session_id TEXT NOT NULL, use_count INTEGER NOT NULL,
    ↪ last_used INTEGER NOT NULL, version INTEGER NOT NULL); PRAGMA user_version = 1")
    _ = try? script(db, "ALTER TABLE sessions ADD COLUMN revision INTEGER NOT NULL DEFAULT 0")
}

private static func decodeRoot(_ row: [String]) -> RootBinding? { guard row.count == 10, let kind = RootBindingKind(rawValue: row[2]),
    ↪ let state = RootBindingLifecycleState(rawValue: row[6]) else { return nil }; return RootBinding(id: RootBindingID(row[0]),
    ↪ projectID: ProjectID(row[1]), kind: kind, absoluteRoot: URL(fileURLWithPath: row[3]), parentBindingID: row[4].isEmpty ? nil :
    ↪ RootBindingID(row[4]), bindingRevision: Int(row[5]) ?? 0, lifecycleState: state, createdAt: parseDate(row[7]), updatedAt: parseDate
    ↪ (row[8]), lastSeenAt: row[9].isEmpty ? nil : parseDate(row[9])) }

private static func decodeFile(_ row: [String]) -> ProjectFileBinding? { guard row.count == 10 else { return nil }; return
    ↪ ProjectFileBinding(id: ProjectFileID(row[0]), projectID: ProjectID(row[1]), rootBindingID: RootBindingID(row[2]), relativePath:
    ↪ ProjectRelativePath(rawValue: row[3]), contentHash: row[4], version: row[5], state: row[6], createdAt: parseDate(row[7]), updatedAt
    ↪ : parseDate(row[8]), lastSeenAt: row[9].isEmpty ? nil : parseDate(row[9])) }

private static func ensureAllExistingProjectsHaveSessionRevision(dataRoot: URL) {
    let projectsDir = dataRoot.appendingPathComponent("projects", isDirectory: true)
    guard let projectDirs = try? FileManager.default.contentsOfDirectory(at: projectsDir, includingPropertiesForKeys: [.isDirectoryKey],
    ↪ options: [.skipsHiddenFiles]) else { return }
    for dir in projectDirs {
        let stateURL = dir.appendingPathComponent("state.sqlite")
        guard FileManager.default.fileExists(atPath: stateURL.path) else { continue }
        if let db = try? open(stateURL) {
            _ = try? script(db, "ALTER TABLE sessions ADD COLUMN revision INTEGER NOT NULL DEFAULT 0")
            sqlite3_close(db)
        }
    }
}

}

}

```

// File: Sources/LingXiCore/Infrastructure/Persistence/ProjectPersistenceDomain.swift

```

import Foundation
import LingXiProtocol

public struct RootBindingID: RawRepresentable, Sendable, Equatable, Hashable, Codable {
    public let rawValue: String
    public init(_ rawValue: String) { self.rawValue = rawValue }
    public init(rawValue: String) { self.rawValue = rawValue }
}

public struct ProjectFileID: RawRepresentable, Sendable, Equatable, Hashable, Codable {
    public let rawValue: String
    public init(_ rawValue: String) { self.rawValue = rawValue }
    public init(rawValue: String) { self.rawValue = rawValue }
}

public struct ProjectRelativePath: RawRepresentable, Sendable, Equatable, Hashable, Codable {
    public let rawValue: String

    public init(_ rawValue: String) throws {
        guard !rawValue.hasPrefix("/") , !rawValue.hasPrefix("~"),
            !rawValue.split(separator: "/").contains(".."),
            !rawValue.contains("//"), !rawValue.contains("\\")
    }
}

```

```

        else { throw WorkspaceResolutionError.invalidRelativePath(rawValue) }
        self.rawValue = rawValue == "." ? "" : rawValue
    }

    public init(rawValue: String) { self.rawValue = rawValue }
    public static let root = ProjectRelativePath(rawValue: "")
}

public enum RootBindingKind: String, Sendable, Codable, CaseIterable {
    case main
    case temporary
    case sandbox
    case worktree
    case external
}

public enum RootBindingLifecycleState: String, Sendable, Codable {
    case active
    case missing
    case inactive
}

public struct RootBinding: Sendable, Equatable, Codable {
    public let id: RootBindingID
    public let projectID: ProjectID
    public let kind: RootBindingKind
    public let absoluteRoot: URL
    public let parentBindingID: RootBindingID?
    public let bindingRevision: Int
    public let lifecycleState: RootBindingLifecycleState
    public let createdAt: Date
    public let updatedAt: Date
    public let lastSeenAt: Date?
}

public struct ProjectFileBinding: Sendable, Equatable, Codable {
    public let id: ProjectFileID
    public let projectID: ProjectID
    public let rootBindingID: RootBindingID
    public let relativePath: ProjectRelativePath
    public let contentHash: String
    public let version: String
    public let state: String
    public let createdAt: Date
    public let updatedAt: Date
    public let lastSeenAt: Date?
}

public enum WorkspaceResolutionError: Error, Sendable, Equatable {
    case unknownRootBinding(RootBindingID)
    case invalidRelativePath(String)
    case escapedRoot(RootBindingID)
}

/// 唯一的持久路径解析入口。所有 durable locator 均为 RootBindingID + RelativePath。
public actor WorkspaceResolver {
    private let store: SQLitePersistenceStore

    public init(store: SQLitePersistenceStore) { self.store = store }

    public func resolve(rootBindingID: RootBindingID, relativePath: ProjectRelativePath) async throws -> URL {
        guard let binding = try await store.rootBinding(rootBindingID) else {
            throw WorkspaceResolutionError.unknownRootBinding(rootBindingID)
        }
        let root = binding.absoluteRoot.standardizedFileURL.resolvingSymlinksInPath()
    }
}

```

```

        let target = root.appendingPathComponent(relativePath.rawValue).standardizedFileURL.resolvingSymlinksInPath()
        let rootPath = root.path
        guard target.path == rootPath || target.path.hasPrefix(rootPath + "/") else {
            throw WorkspaceResolutionError.escapedRoot(rootBindingID)
        }
        return target
    }
}

public protocol BlobStore: Sendable {
    func put(_ data: Data) throws -> String
    func get(_ reference: String) throws -> Data?
    func byteCount() throws -> Int64
}

public struct FileBlobStore: BlobStore {
    private let directory: URL

    public init(directory: URL) throws {
        self.directory = directory
        try FileManager.default.createDirectory(at: directory, withIntermediateDirectories: true)
    }

    public func put(_ data: Data) throws -> String {
        let reference = ContextPage.fingerprint(data)
        let url = directory.appendingPathComponent(reference)
        if !FileManager.default.fileExists(atPath: url.path) {
            try data.write(to: url, options: .atomic)
        }
        return reference
    }

    public func get(_ reference: String) throws -> Data? {
        let url = directory.appendingPathComponent(reference)
        return FileManager.default.fileExists(atPath: url.path) ? try Data(contentsOf: url) : nil
    }

    public func byteCount() throws -> Int64 {
        let urls = try FileManager.default.contentsOfDirectory(at: directory, includingPropertiesForKeys: [.fileSizeKey])
        return Int64(urls.reduce(0) { $0 + ((try? $1.resourceValues(forKeys: [.fileSizeKey])).fileSize) ?? 0 })
    }
}

```

// File: Sources/LingXiCore/Infrastructure/Persistence/MigrationRunner.swift

```

import Foundation

public struct PersistenceMigrationError: Error, Sendable, Equatable {
    public let version: Int

    public init(version: Int) { self.version = version }
}

/// SQLite migration 的唯一入口。调用者负责提供 SQLite transaction; 失败时绝不允许以空库替代旧数据。
public enum MigrationRunner {
    public static func migrate(from version: Int, applyV0ToV1: () throws -> Void) throws {
        try migrate(from: version, applyV0ToV1: applyV0ToV1, applyV1ToV2: {}, applyV2ToV3: {}, applyV3ToV4: {}, applyV4ToV5: {}, applyV5ToV6
        ↪ : {})
    }

    public static func migrate(from version: Int, applyV0ToV1: () throws -> Void, applyV1ToV2: () throws -> Void) throws {
        try migrate(from: version, applyV0ToV1: applyV0ToV1, applyV1ToV2: applyV1ToV2, applyV2ToV3: {}, applyV3ToV4: {}, applyV4ToV5: {},
        ↪ applyV5ToV6: {})
    }
}

```

```

public static func migrate(from version: Int, applyV0ToV1: () throws -> Void, applyV1ToV2: () throws -> Void, applyV2ToV3: () throws ->
↳ Void) throws {
    try migrate(from: version, applyV0ToV1: applyV0ToV1, applyV1ToV2: applyV1ToV2, applyV2ToV3: applyV2ToV3, applyV3ToV4: {},
↳ applyV4ToV5: {}, applyV5ToV6: {})
}

public static func migrate(from version: Int, applyV0ToV1: () throws -> Void, applyV1ToV2: () throws -> Void, applyV2ToV3: () throws ->
↳ Void, applyV3ToV4: () throws -> Void, applyV4ToV5: () throws -> Void, applyV5ToV6: () throws -> Void) throws {
    guard version <= SQLitePersistenceStore.databaseSchemaVersion else {
        throw PersistenceMigrationError(version: version)
    }
    if version == 0 {
        try applyV0ToV1()
    }
    if version <= 1 {
        try applyV1ToV2()
    }
    if version <= 2 {
        try applyV2ToV3()
    }
    if version <= 3 {
        try applyV3ToV4()
    }
    if version <= 4 {
        try applyV4ToV5()
    }
    if version <= 5 {
        try applyV5ToV6()
    }
}
}

```

// File: Sources/LingXiCore/Infrastructure/EventLog/EventLogStore.swift

```

import Foundation
import LingXiProtocol

/// SessionEventLog: 管理单个 Session 的因果事件日志与 Replay 边界。
public actor SessionEventLog {
    private struct PersistedMeta: Codable {
        let generationID: String
        var sequence: UInt64
    }

    public let sessionID: SessionID
    public let generationID: EventLogGenerationID
    public private(set) var sequence: UInt64
    private var events: [SessionEventEnvelope]
    private var subscribers: [UUID: AsyncStream<SessionEventEnvelope>.Continuation]
    private let maxRetainedEvents: Int
    private let storageDirectory: URL?

    public init(
        sessionID: SessionID,
        generationID: EventLogGenerationID? = nil,
        initialSequence: UInt64 = 0,
        maxRetainedEvents: Int = 10_000,
        storageDirectory: URL? = nil
    ) {
        self.sessionID = sessionID
        self.maxRetainedEvents = maxRetainedEvents
        self.storageDirectory = storageDirectory
        self.subscribers = [:]

        var resolvedGen = generationID ?? EventLogGenerationID("gen-\(sessionID.rawValue)")

```



```

var resolvedSeq = initialSequence
var loadedEvents: [SessionEventEnvelope] = []

if let dir = storageDirectory {
    let sessionDir = dir.appendingPathComponent("sessions/\(sessionID.rawValue)", isDirectory: true)
    try? FileManager.default.createDirectory(at: sessionDir, withIntermediateDirectories: true)
    let metaURL = sessionDir.appendingPathComponent("meta.json")
    if let metaData = try? Data(contentsOf: metaURL),
        let meta = try? JSONDecoder().decode(PersistedMeta.self, from: metaData) {
        resolvedGen = generationID ?? EventLogGenerationID(meta.generationID)
        resolvedSeq = meta.sequence
    } else {
        let meta = PersistedMeta(generationID: resolvedGen.rawValue, sequence: resolvedSeq)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
    }

    let eventsURL = sessionDir.appendingPathComponent("events.jsonl")
    if let linesData = try? Data(contentsOf: eventsURL),
        let linesStr = String(data: linesData, encoding: .utf8) {
        let lines = linesStr.split(separator: "\n")
        let decoder = JSONDecoder()
        for line in lines {
            if let lineData = line.data(using: .utf8),
                let envelope = try? decoder.decode(SessionEventEnvelope.self, from: lineData) {
                loadedEvents.append(envelope)
            }
        }
    }
}

self.generationID = resolvedGen
self.sequence = resolvedSeq
self.events = loadedEvents
}

public func currentCursor() -> EventCursor {
    EventCursor(generationID: generationID, sequence: sequence)
}

public func currentSequence() -> UInt64 {
    sequence
}

public func currentWatermark() -> EventWatermark {
    EventWatermark(scope: .session(sessionID), cursor: currentCursor())
}

@discardableResult
public func append(causal: CausalContext, payload: SessionEventPayload) -> SessionEventEnvelope {
    sequence += 1
    let cursor = EventCursor(generationID: generationID, sequence: sequence)
    let envelope = SessionEventEnvelope(cursor: cursor, timestamp: Date(), causal: causal, payload: payload)
    events.append(envelope)
    if events.count > maxRetainedEvents {
        events.removeFirst(events.count - maxRetainedEvents)
    }

    if let dir = storageDirectory {
        let sessionDir = dir.appendingPathComponent("sessions/\(sessionID.rawValue)", isDirectory: true)
        let metaURL = sessionDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
    }
}

```

```

    }
    let eventsURL = sessionDir.appendingPathComponent("events.jsonl")
    if let envelopeData = try? JSONEncoder().encode(envelope),
        let lineStr = String(data: envelopeData, encoding: .utf8) {
        let lineToAppend = lineStr + "\n"
        if FileManager.default.fileExists(atPath: eventsURL.path) {
            if let fileHandle = try? FileHandle(forWritingTo: eventsURL) {
                defer { try? fileHandle.close() }
                _ = try? fileHandle.seekToEnd()
                try? fileHandle.write(contentsOf: Data(lineToAppend.utf8))
            }
        } else {
            try? Data(lineToAppend.utf8).write(to: eventsURL)
        }
    }
}

for subscriber in subscribers.values {
    subscriber.yield(envelope)
}
return envelope
}

public func rollbackLastAppended() {
    guard !events.isEmpty else { return }
    events.removeLast()
    sequence = max(0, sequence - 1)
    if let dir = storageDirectory {
        let sessionDir = dir.appendingPathComponent("sessions/\(sessionID.rawValue)", isDirectory: true)
        let metaURL = sessionDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = sessionDir.appendingPathComponent("events.jsonl")
        var newContent = ""
        for env in events {
            if let envData = try? JSONEncoder().encode(env),
                let s = String(data: envData, encoding: .utf8) {
                newContent += s + "\n"
            }
        }
        try? Data(newContent.utf8).write(to: eventsURL)
    }
}

public func truncateEvents(afterSequence targetSeq: UInt64) {
    events.removeAll { $0.cursor.sequence > targetSeq }
    sequence = targetSeq
    if let dir = storageDirectory {
        let sessionDir = dir.appendingPathComponent("sessions/\(sessionID.rawValue)", isDirectory: true)
        let metaURL = sessionDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = sessionDir.appendingPathComponent("events.jsonl")
        var newContent = ""
        for env in events {
            if let envData = try? JSONEncoder().encode(env),
                let s = String(data: envData, encoding: .utf8) {
                newContent += s + "\n"
            }
        }
        try? Data(newContent.utf8).write(to: eventsURL)
    }
}

```

```

    }
}

public func resetToEvents(_ newEvents: [SessionEventEnvelope]) {
    self.events = newEvents
    self.sequence = newEvents.last?.cursor.sequence ?? 0
    if let dir = storageDirectory {
        let sessionDir = dir.appendingPathComponent("sessions/\(sessionID.rawValue)", isDirectory: true)
        let metaURL = sessionDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = sessionDir.appendingPathComponent("events.jsonl")
        var newContent = ""
        for env in events {
            if let envData = try? JSONEncoder().encode(env),
                let s = String(data: envData, encoding: .utf8) {
                newContent += s + "\n"
            }
        }
        try? Data(newContent.utf8).write(to: eventsURL)
    }
}

public func subscribe(after: EventCursor?) throws -> AsyncStream<SessionEventEnvelope> {
    var replayEvents: [SessionEventEnvelope] = []
    if let after {
        guard after.generationID == generationID else {
            throw RuntimeError(
                category: .runtime,
                code: "replayUnavailable",
                message: "Generation mismatch: requested=\(after.generationID.rawValue) current=\(generationID.rawValue)",
                retryability: .afterUserAction,
                source: .core
            )
        }
        if let firstRetained = events.first, after.sequence < firstRetained.cursor.sequence - 1 {
            throw RuntimeError(
                category: .runtime,
                code: "replayUnavailable",
                message: "Event log retention gap: requested sequence \((after.sequence) is older than earliest retained \((firstRetained.
↵ cursor.sequence))",
                retryability: .afterUserAction,
                source: .core
            )
        }
        replayEvents = events.filter { $0.cursor.sequence > after.sequence }
    }

    let key = UUID()
    return AsyncStream { continuation in
        for event in replayEvents {
            continuation.yield(event)
        }
        self.subscribers[key] = continuation
        continuation.onTermination = { [weak self] _ in
            Task { [weak self] in
                await self?.removeSubscriber(key)
            }
        }
    }
}

public func listEvents(before: EventCursor?, after: EventCursor?, limit: Int) -> [SessionEventEnvelope] {

```

```

    var filtered = events
    if let after {
        filtered = filtered.filter { $0.cursor.sequence > after.sequence }
    }
    if let before {
        filtered = filtered.filter { $0.cursor.sequence < before.sequence }
    }
    if filtered.count > limit {
        return Array(filtered.prefix(limit))
    }
    return filtered
}

public func recentEvents(count: Int = 50) -> [SessionEventEnvelope] {
    if events.count > count {
        return Array(events.suffix(count))
    }
    return events
}

private func removeSubscriber(_ key: UUID) {
    subscribers.removeValue(forKey: key)
}
}

/// RuntimeEventLog: 管理全局 Runtime 级别的因果事件日志与 Replay。
public actor RuntimeEventLog {
    private struct PersistedMeta: Codable {
        let generationID: String
        var sequence: UInt64
    }

    public let generationID: EventLogGenerationID
    public private(set) var sequence: UInt64
    private var events: [RuntimeEventEnvelope]
    private var subscribers: [UUID: AsyncStream<RuntimeEventEnvelope>.Continuation]
    private let maxRetainedEvents: Int
    private let storageDirectory: URL?

    public init(
        generationID: EventLogGenerationID? = nil,
        initialSequence: UInt64 = 0,
        maxRetainedEvents: Int = 5_000,
        storageDirectory: URL? = nil
    ) {
        self.maxRetainedEvents = maxRetainedEvents
        self.storageDirectory = storageDirectory
        self.subscribers = [:]

        var resolvedGen = generationID ?? EventLogGenerationID("gen-runtime-global")
        var resolvedSeq = initialSequence
        var loadedEvents: [RuntimeEventEnvelope] = []

        if let dir = storageDirectory {
            let runtimeDir = dir.appendingPathComponent("runtime", isDirectory: true)
            try? FileManager.default.createDirectory(at: runtimeDir, withIntermediateDirectories: true)
            let metaURL = runtimeDir.appendingPathComponent("meta.json")
            if let metaData = try? Data(contentsOf: metaURL),
                let meta = try? JSONDecoder().decode(PersistedMeta.self, from: metaData) {
                resolvedGen = generationID ?? EventLogGenerationID(meta.generationID)
                resolvedSeq = meta.sequence
            } else {
                let meta = PersistedMeta(generationID: resolvedGen.rawValue, sequence: resolvedSeq)
                if let data = try? JSONEncoder().encode(meta) {
                    try? data.write(to: metaURL)
                }
            }
        }
    }
}

```

```

    }
}

let eventsURL = runtimeDir.appendingPathComponent("events.jsonl")
if let linesData = try? Data(contentsOf: eventsURL),
let linesStr = String(data: linesData, encoding: .utf8) {
    let lines = linesStr.split(separator: "\n")
    let decoder = JSONDecoder()
    for line in lines {
        if let lineData = line.data(using: .utf8),
        let envelope = try? decoder.decode(RuntimeEventEnvelope.self, from: lineData) {
            loadedEvents.append(envelope)
        }
    }
}

self.generationID = resolvedGen
self.sequence = resolvedSeq
self.events = loadedEvents
}

public func currentCursor() -> EventCursor {
    EventCursor(generationID: generationID, sequence: sequence)
}

public func currentSequence() -> UInt64 {
    sequence
}

public func currentWatermark() -> EventWatermark {
    EventWatermark(scope: .runtime, cursor: currentCursor())
}

@discardableResult
public func append(payload: RuntimeEventPayload) -> RuntimeEventEnvelope {
    sequence += 1
    let cursor = EventCursor(generationID: generationID, sequence: sequence)
    let envelope = RuntimeEventEnvelope(cursor: cursor, timestamp: Date(), payload: payload)
    events.append(envelope)
    if events.count > maxRetainedEvents {
        events.removeFirst(events.count - maxRetainedEvents)
    }

    if let dir = storageDirectory {
        let runtimeDir = dir.appendingPathComponent("runtime", isDirectory: true)
        let metaURL = runtimeDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = runtimeDir.appendingPathComponent("events.jsonl")
        if let envelopeData = try? JSONEncoder().encode(envelope),
        let lineStr = String(data: envelopeData, encoding: .utf8) {
            let lineToAppend = lineStr + "\n"
            if FileManager.default.fileExists(atPath: eventsURL.path) {
                if let fileHandle = try? FileHandle(forWritingTo: eventsURL) {
                    defer { try? fileHandle.close() }
                    _ = try? fileHandle.seekToEnd()
                    try? fileHandle.write(contentsOf: Data(lineToAppend.utf8))
                }
            } else {
                try? Data(lineToAppend.utf8).write(to: eventsURL)
            }
        }
    }
}

```

```

    }

    for subscriber in subscribers.values {
        subscriber.yield(envelope)
    }
    return envelope
}

public func rollbackLastAppended() {
    guard !events.isEmpty else { return }
    events.removeLast()
    sequence = max(0, sequence - 1)
    if let dir = storageDirectory {
        let runtimeDir = dir.appendingPathComponent("runtime", isDirectory: true)
        let metaURL = runtimeDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = runtimeDir.appendingPathComponent("events.jsonl")
        var newContent = ""
        for env in events {
            if let envData = try? JSONEncoder().encode(env),
                let s = String(data: envData, encoding: .utf8) {
                newContent += s + "\n"
            }
        }
        try? Data(newContent.utf8).write(to: eventsURL)
    }
}

public func truncateEvents(afterSequence targetSeq: UInt64) {
    events.removeAll { $0.cursor.sequence > targetSeq }
    sequence = targetSeq
    if let dir = storageDirectory {
        let runtimeDir = dir.appendingPathComponent("runtime", isDirectory: true)
        let metaURL = runtimeDir.appendingPathComponent("meta.json")
        let meta = PersistedMeta(generationID: generationID.rawValue, sequence: sequence)
        if let data = try? JSONEncoder().encode(meta) {
            try? data.write(to: metaURL)
        }
        let eventsURL = runtimeDir.appendingPathComponent("events.jsonl")
        var newContent = ""
        for env in events {
            if let envData = try? JSONEncoder().encode(env),
                let s = String(data: envData, encoding: .utf8) {
                newContent += s + "\n"
            }
        }
        try? Data(newContent.utf8).write(to: eventsURL)
    }
}

public func subscribe(after: EventCursor?) -> AsyncStream<RuntimeEventEnvelope> {
    var replayEvents: [RuntimeEventEnvelope] = []
    if let after, after.generationID == generationID {
        replayEvents = events.filter { $0.cursor.sequence > after.sequence }
    }

    let key = UUID()
    return AsyncStream { continuation in
        for event in replayEvents {
            continuation.yield(event)
        }
        self.subscribers[key] = continuation
    }
}

```

```

        continuation.onTermination = { [weak self] _ in
            Task { [weak self] in
                await self?.removeSubscriber(key)
            }
        }
    }
}

private func removeSubscriber(_ key: UUID) {
    subscribers.removeValue(forKey: key)
}
}

/// IdempotencyJournal: 持久化命令去重日志, 保证相同 commandID 不重复副作用。
public actor IdempotencyJournal {
    private struct Entry {
        let receiptData: Data
    }

    private var journal: [CommandID: Entry] = [:]
    private let storageDirectory: URL?

    public init(storageDirectory: URL? = nil) {
        self.storageDirectory = storageDirectory
        if let dir = storageDirectory {
            let journalDir = dir.appendingPathComponent("idempotency", isDirectory: true)
            try? FileManager.default.createDirectory(at: journalDir, withIntermediateDirectories: true)
            if let fileURLs = try? FileManager.default.contentsOfDirectory(at: journalDir, includingPropertiesForKeys: nil) {
                for fileURL in fileURLs where fileURL.pathExtension == "json" {
                    let cmdStr = fileURL.deletingPathExtension().lastPathComponent
                    if let data = try? Data(contentsOf: fileURL) {
                        journal[CommandID(cmdStr)] = Entry(receiptData: data)
                    }
                }
            }
        }
    }

    public func get<R: Codable & Sendable>(commandID: CommandID, as type: R.Type) -> CommandReceipt<R>? {
        guard let entry = journal[commandID] else { return nil }
        return try? JSONDecoder().decode(CommandReceipt<R>.self, from: entry.receiptData)
    }

    public func record<R: Codable & Sendable>(commandID: CommandID, receipt: CommandReceipt<R>) {
        guard let data = try? JSONEncoder().encode(receipt) else { return }
        journal[commandID] = Entry(receiptData: data)
        if let dir = storageDirectory {
            let journalDir = dir.appendingPathComponent("idempotency", isDirectory: true)
            let fileURL = journalDir.appendingPathComponent("\(commandID.rawValue).json")
            try? data.write(to: fileURL)
        }
    }

    public func rollback(commandID: CommandID) {
        journal.removeValue(forKey: commandID)
        if let dir = storageDirectory {
            let journalDir = dir.appendingPathComponent("idempotency", isDirectory: true)
            let fileURL = journalDir.appendingPathComponent("\(commandID.rawValue).json")
            try? FileManager.default.removeItem(at: fileURL)
        }
    }
}

```

// File: Sources/LingXiCore/Infrastructure/EventLog/DurableCommandWAL.swift

```

import Foundation
import LingXiProtocol

/// StagedWALRecord: 描述单个 Command 事务的在途/已提交状态。
public struct StagedWALRecord: Codable, Sendable, Equatable {
    public let commandID: String
    public let commandName: String
    public var stage: String // "staged", "eventsAppended", "committed"
    public var createdSessionID: String?
    public var sessionID: String?
    public var turnID: String?
    public var runID: String?
    public var initialRuntimeSequence: UInt64?
    public var initialSessionSequence: UInt64?
    public var receiptData: Data?
    public let createdAt: Date
    public var updatedAt: Date

    public init(
        commandID: String,
        commandName: String,
        stage: String = "staged",
        createdSessionID: String? = nil,
        sessionID: String? = nil,
        turnID: String? = nil,
        runID: String? = nil,
        initialRuntimeSequence: UInt64? = nil,
        initialSessionSequence: UInt64? = nil,
        receiptData: Data? = nil,
        createdAt: Date = Date(),
        updatedAt: Date = Date()
    ) {
        self.commandID = commandID
        self.commandName = commandName
        self.stage = stage
        self.createdSessionID = createdSessionID
        self.sessionID = sessionID
        self.turnID = turnID
        self.runID = runID
        self.initialRuntimeSequence = initialRuntimeSequence
        self.initialSessionSequence = initialSessionSequence
        self.receiptData = receiptData
        self.createdAt = createdAt
        self.updatedAt = updatedAt
    }
}

/// DurableCommandWAL: 管理磁盘 Write-Ahead Log 与进程崩溃恢复。
/// 保证在 state mutation、event append、receipt 任何阶段被 SIGKILL / exit 终止后，
/// 重新启动时能自动消除未提交的孤儿状态与孤儿事件，恢复原子性。
public actor DurableCommandWAL {
    public let storageDirectory: URL?
    private let walDir: URL?
    private let committedDir: URL?

    public init(storageDirectory: URL? = nil) {
        self.storageDirectory = storageDirectory
        if let dir = storageDirectory {
            let base = dir.appendingPathComponent("wal", isDirectory: true)
            let committed = dir.appendingPathComponent("committed_tx", isDirectory: true)
            try? FileManager.default.createDirectory(at: base, withIntermediateDirectories: true)
            try? FileManager.default.createDirectory(at: committed, withIntermediateDirectories: true)
            self.walDir = base
            self.committedDir = committed
        } else {

```



```

        self.walDir = nil
        self.committedDir = nil
    }
}

/// 开始事务: 写入 .wal 文件
public func beginTransaction(commandID: CommandID, commandName: String) {
    guard let walDir else { return }
    let record = StagedWALRecord(commandID: commandID.rawValue, commandName: commandName, stage: "staged")
    writeWAL(record, to: walDir)
}

/// 记录状态机变更 (Phase 2)
public func recordState(
    commandID: CommandID,
    createdSessionID: SessionID? = nil,
    sessionID: SessionID? = nil,
    turnID: TurnID? = nil,
    runID: RunID? = nil,
    initialRuntimeSequence: UInt64? = nil,
    initialSessionSequence: UInt64? = nil
) {
    guard let walDir else { return }
    var record = readWAL(commandID: commandID) ?? StagedWALRecord(commandID: commandID.rawValue, commandName: "unknown")
    if let createdSessionID { record.createdSessionID = createdSessionID.rawValue }
    if let sessionID { record.sessionID = sessionID.rawValue }
    if let turnID { record.turnID = turnID.rawValue }
    if let runID { record.runID = runID.rawValue }
    if let initialRuntimeSequence { record.initialRuntimeSequence = initialRuntimeSequence }
    if let initialSessionSequence { record.initialSessionSequence = initialSessionSequence }
    record.stage = "stateMutated"
    record.updatedAt = Date()
    writeWAL(record, to: walDir)
}

/// 记录语义事件已追加 (Phase 3)
public func recordEventsAppended(commandID: CommandID) {
    guard let walDir else { return }
    guard var record = readWAL(commandID: commandID) else { return }
    record.stage = "eventsAppended"
    record.updatedAt = Date()
    writeWAL(record, to: walDir)
}

/// 提交事务: 将 receipt 写入 committed_tx 并原子删除 .wal (Phase 4)
public func commitTransaction<R: Codable & Sendable>(commandID: CommandID, receipt: CommandReceipt<R>) {
    guard let data = try? JSONEncoder().encode(receipt) else { return }
    if let committedDir {
        let committedURL = committedDir.appendingPathComponent("\(commandID.rawValue).json")
        try? data.write(to: committedURL)
    }
    if let walDir {
        let walURL = walDir.appendingPathComponent("\(commandID.rawValue).wal")
        try? FileManager.default.removeItem(at: walURL)
    }
}

/// 检查并获取已提交的 receipt
public func getCommittedReceipt<R: Codable & Sendable>(commandID: CommandID, as type: R.Type) -> CommandReceipt<R>? {
    guard let committedDir else { return nil }
    let fileURL = committedDir.appendingPathComponent("\(commandID.rawValue).json")
    guard let data = try? Data(contentsOf: fileURL) else { return nil }
    return try? JSONDecoder().decode(CommandReceipt<R>.self, from: data)
}

```

```

/// 进程重启恢复：回滚所有未达到 committed 阶段的孤儿状态与孤儿事件
public func recover(
    sessionStore: any SessionStore,
    runtimeEventLog: RuntimeEventLog,
    coordinatorProvider: (SessionID) async throws -> SessionTurnCoordinator
) async {
    guard let walDir else { return }
    guard let fileURLs = try? FileManager.default.contentsOfDirectory(at: walDir, includingPropertiesForKeys: nil) else { return }

    for fileURL in fileURLs where fileURL.pathExtension == "wal" {
        guard let data = try? Data(contentsOf: fileURL),
            let record = try? JSONDecoder().decode(StagedWALRecord.self, from: data) else {
            try? FileManager.default.removeItem(at: fileURL)
            continue
        }

        // 发现未提交的崩溃事务，必须自动消除未提交副作用！
        // 1. 如果该事务曾创建会话，删除未提交会话
        if let createdSession = record.createdSessionID {
            try? await sessionStore.deleteSession(SessionID(createdSession))
        }

        // 2. 如果该事务曾追加 Runtime 事件，截断回退至 initialRuntimeSequence
        if let initialRuntimeSeq = record.initialRuntimeSequence {
            await runtimeEventLog.truncateEvents(afterSequence: initialRuntimeSeq)
        }

        // 3. 如果该事务曾追加 Session 事件或创建 Turn/Run，回滚 Session 状态
        if let sessionIDStr = record.sessionID ?? record.createdSessionID {
            let sessionID = SessionID(sessionIDStr)
            if let coord = try? await coordinatorProvider(sessionID) {
                if let initialSessionSeq = record.initialSessionSequence {
                    await coord.eventLog.truncateEvents(afterSequence: initialSessionSeq)
                }
                if let turnIDStr = record.turnID {
                    await coord.rollbackTurnID(TurnID(turnIDStr), runID: record.runID.flatMap { RunID($0) })
                }
            }
        }

        // 4. 清除 .wal
        try? FileManager.default.removeItem(at: fileURL)
    }
}

private func readWAL(commandID: CommandID) -> StagedWALRecord? {
    guard let walDir else { return nil }
    let url = walDir.appendingPathComponent("\(commandID.rawValue).wal")
    guard let data = try? Data(contentsOf: url) else { return nil }
    return try? JSONDecoder().decode(StagedWALRecord.self, from: data)
}

private func writeWAL(_ record: StagedWALRecord, to dir: URL) {
    let url = dir.appendingPathComponent("\(record.commandID).wal")
    if let data = try? JSONEncoder().encode(record) {
        try? data.write(to: url)
    }
}
}

```

// File: Sources/LingXiCore/Modules/Model/ModelGateway.swift

```

import Foundation
import LingXiProtocol

```

```

/// 模型网关: Core 内所有模型推理的正式入口。
/// 持有当前 Provider, 屏蔽 Provider 的构建与选择细节。
public struct ModelGateway: Sendable {
    public struct Unavailable: Sendable, Equatable {
        public let missingRequirements: [String]
    }

    private let provider: (any ModelProvider)?
    public let endpoint: ResolvedModelEndpoint?
    public let missingRequirements: [String]
    public let reasoning: String?
    public let deadlinePolicy: ExecutionDeadlinePolicy
    private let rateScheduler: ProviderRateScheduler
    public var modelID: ModelID? { endpoint?.modelID }
    public var contextProfile: ModelContextProfile { endpoint?.contextProfile ?? ModelContextProfile() }

    public init(provider: (any ModelProvider)?, modelID: ModelID?, missingRequirements: [String] = [], contextProfile: ModelContextProfile =
        ↪ ModelContextProfile(), reasoning: String? = nil, deadlinePolicy: ExecutionDeadlinePolicy = ExecutionDeadlinePolicy()) {
        self.provider = provider
        endpoint = modelID.map { ResolvedModelEndpoint(providerID: "default", modelID: $0, baseURL: nil, wireProtocol: .chatCompletions,
        ↪ contextProfile: contextProfile) }
        self.missingRequirements = missingRequirements
        self.reasoning = reasoning
        self.deadlinePolicy = deadlinePolicy
        rateScheduler = .shared
    }

    public init(assembly: ModelRuntimeAssembly?, missingRequirements: [String] = [], reasoning: String? = nil, deadlinePolicy:
        ↪ ExecutionDeadlinePolicy = ExecutionDeadlinePolicy()) {
        provider = assembly?.provider
        endpoint = assembly?.endpoint
        self.missingRequirements = missingRequirements
        self.reasoning = reasoning
        self.deadlinePolicy = deadlinePolicy
        rateScheduler = .shared
    }

    public var isConfigured: Bool { provider != nil && modelID != nil }

    public var status: ProviderStatus {
        ProviderStatus(
            configured: isConfigured,
            model: modelID?.rawValue,
            baseURL: endpoint?.baseURL?.absoluteString,
            missingRequirements: missingRequirements
        )
    }
}

/// Fast Path: 进入模型高速总线。
public func stream(
    _ request: ModelRequest,
    onActivityChanged: (@Sendable (ProviderActivitySnapshot) async -> Void)? = nil
) async throws -> AsyncThrowingStream<ModelEvent, Error> {
    guard let provider else {
        throw CoreError(
            code: .provider,
            message: "未配置模型 Provider; 请检查 providers.json 与 CredentialStore: \"\((missingRequirements.joined(separator: ", "))\""
        )
    }
    guard let endpoint else { throw CoreError(code: .provider, message: "未配置模型 Endpoint") }

    let currentContext = AgentExecutionContext.current
    let sessionID = currentContext?.sessionID ?? SessionID("ephemeral")
    let runID = request.executionID ?? currentContext?.runID
    let providerRequestID = "local:\(request.requestID.rawValue)"

```

```

if await ProviderActivityRegistry.shared.isCancelled(providerRequestID: providerRequestID, runID: runID) {
    throw CancellationError()
}

let scheduledSnapshot = await ProviderActivityRegistry.shared.record(
    sessionID: sessionID,
    runID: runID,
    providerRequestID: providerRequestID,
    state: .scheduled,
    model: request.model.rawValue
)
await onActivityChanged?(scheduledSnapshot)

let estimate = ConservativeTokenEstimator().estimate(text: request.messages.map(\.content).joined(separator: "\n")) +
↳ ConservativeTokenEstimator().estimate(tools: request.tools)
let policy = endpoint.rateLimits.retryPolicy
var retries = 0

while true {
    try Task.checkCancellation()
    if await ProviderActivityRegistry.shared.isCancelled(providerRequestID: providerRequestID, runID: runID) {
        throw CancellationError()
    }
    let waitingSnapshot = await ProviderActivityRegistry.shared.record(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: .waitingForRateBudget,
        model: request.model.rawValue
    )
    await onActivityChanged?(waitingSnapshot)
    try await rateScheduler.admit(endpoint: endpoint, requestID: request.requestID, estimatedTokens: estimate)
    let requestingSnapshot = await ProviderActivityRegistry.shared.record(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: .requesting,
        model: request.model.rawValue,
        detail: "等待模型首字响应 [\"(request.model.rawValue)\"]..."
    )
    await onActivityChanged?(requestingSnapshot)
    do {
        // Admission intentionally precedes the provider watchdog: queueing must not consume inference, tool, or HITL timeouts.
        let deadline = deadlinePolicy.deadline(
            for: .provider,
            requested: request.overallTimeoutSeconds.map { .milliseconds(Int($0 * 1_000)) }
        )
        let source = try await ExecutionWatchdog.run(deadline) {
            try await provider.stream(request)
        }
        let effective = ExecutionDeadline(category: .provider, startedAt: deadline.startedAt, timeout: deadline.timeout, idleTimeout
↳ : request.idleTimeoutSeconds.map { .milliseconds(Int($0 * 1_000)) } ?? deadline.idleTimeout)
        return releaseRatePermit(
            ExecutionWatchdog.stream(source, deadline: effective),
            endpoint: endpoint,
            requestID: request.requestID,
            sessionID: sessionID,
            runID: runID,
            providerRequestID: providerRequestID,
            onActivityChanged: onActivityChanged
        )
    } catch let error as ProviderRateLimitError {
        await rateScheduler.release(requestID: request.requestID, endpoint: endpoint)
        let delay = retryDelay(error.retryAfter, policy: policy, retry: retries + 1)
    }
}

```

```

await rateScheduler.recordRateLimit(endpoint: endpoint, requestID: request.requestID, cooldown: delay)
guard retries < policy.maxRetries else {
    let failSnap = await ProviderActivityRegistry.shared.record(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: .failed,
        model: request.model.rawValue,
        detail: "已达重试上限 (\(policy.maxRetries) 次) · \(\error.classified.userFacingSummary)",
        statusCode: error.statusCode
    )
    await onActivityChanged?(failSnap)
    throw error.underlying
}
retries += 1
await rateScheduler.recordRetry(requestID: request.requestID)

let delaySeconds = Double(delay.components.seconds) + Double(delay.components.attoseconds) / 1e18
let delayText = delaySeconds >= 1 ? String(format: "%.1fs", delaySeconds) : "\(\Int(delaySeconds * 1000))ms"

let state: ProviderActivityState
let detail: String
switch error.classified.category {
case .rateLimited:
    state = .rateLimited
    detail = "上游限流 (429), 等待配额恢复 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries) 次重试"
case .gatewayTimeout:
    state = .retryScheduled
    detail = "上游网关超时 (\(error.statusCode)), 等待重试 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries) 次重试"
case .serverOverloaded:
    state = .retryScheduled
    detail = "上游服务过载 (\(error.statusCode)), 退避重试 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries) 次重试"
case .badGateway:
    state = .retryScheduled
    detail = "上游网关异常 (\(error.statusCode)), 等待重试 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries) 次重试"
case .internalServerError:
    state = .retryScheduled
    detail = "上游服务内部错误 (500), 等待重试 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries) 次重试"
default:
    state = .retryScheduled
    detail = "\(\error.classified.category.userDescription), 等待重试 (\(delayText)) · 正在第 \(\retries)/\(\policy.maxRetries)
↩ ) 次重试"
}

let retrySnap = await ProviderActivityRegistry.shared.record(
    sessionID: sessionID,
    runID: runID,
    providerRequestID: providerRequestID,
    state: state,
    model: request.model.rawValue,
    detail: detail,
    statusCode: error.statusCode
)
await onActivityChanged?(retrySnap)
try await Task.sleep(for: delay)
await rateScheduler.recordWait(requestID: request.requestID, duration: delay)
} catch {
    let isCancelled = (error is CancellationError)
    if isCancelled {
        await rateScheduler.cancel(requestID: request.requestID, endpoint: endpoint)
    } else {
        await rateScheduler.release(requestID: request.requestID, endpoint: endpoint)
    }
}
let coreErr = error as? CoreError
let isTransient = (coreErr?.code == .transportLost || coreErr?.code == .commandTimedOut)

```

```

if isTransient && !isCancelled && retries < policy.maxRetries {
    retries += 1
    let delay = retryDelay(nil, policy: policy, retry: retries)
    await rateScheduler.recordRetry(requestID: request.requestID)

    let delaySeconds = Double(delay.components.seconds) + Double(delay.components.attoseconds) / 1e18
    let delayText = delaySeconds >= 1 ? String(format: "%.1fs", delaySeconds) : "\(Int(delaySeconds * 1000))ms"
    let detail = coreErr?.code == .commandTimedOut
        ? "请求超时, 等待重试 (\(delayText)) · 正在第 \(retries)/\(policy.maxRetries) 次重试"
        : "网络传输中断, 等待重试 (\(delayText)) · 正在第 \(retries)/\(policy.maxRetries) 次重试"

    let retrySnap = await ProviderActivityRegistry.shared.record(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: .retryScheduled,
        model: request.model.rawValue,
        detail: detail,
        statusCode: coreErr?.code == .commandTimedOut ? 408 : nil
    )
    await onActivityChanged?(retrySnap)
    try? await Task.sleep(for: delay)
    await rateScheduler.recordWait(requestID: request.requestID, duration: delay)
    continue
}

let terminalState: ProviderActivityState = isCancelled ? .cancelled : .failed
let failSnap = await ProviderActivityRegistry.shared.record(
    sessionID: sessionID,
    runID: runID,
    providerRequestID: providerRequestID,
    state: terminalState,
    model: request.model.rawValue,
    detail: (error as? CoreError)?.message ?? error.localizedDescription
)
await onActivityChanged?(failSnap)
throw error
}
}

public func rateMetrics(for requestID: ModelRequestID) async -> ProviderRateMetrics {
    await rateScheduler.metrics(for: requestID)
}

public func resetRateLimiter(endpoint: ResolvedModelEndpoint? = nil) async {
    await rateScheduler.reset(endpoint: endpoint)
}

private func releaseRatePermit(
    _ source: AsyncThrowingStream<ModelEvent, Error>,
    endpoint: ResolvedModelEndpoint,
    requestID: ModelRequestID,
    sessionID: SessionID,
    runID: AgentRunID?,
    providerRequestID: String,
    onActivityChanged: (@Sendable (ProviderActivitySnapshot) async -> Void)? = nil
) -> AsyncThrowingStream<ModelEvent, Error> {
    let doneFlag = StreamDoneFlag()
    return AsyncThrowingStream { continuation in
        let pump = Task {
            var released = false
            func releasePermit() async {
                guard !released else { return }
                released = true
                await rateScheduler.release(requestID: requestID, endpoint: endpoint)
            }
        }
    }
}

```

```

}
do {
    var streamedFirstChunk = false
    for try await event in source {
        if Task.isCancelled {
            doneFlag.markDone()
            released = true
            await rateScheduler.cancel(requestID: requestID, endpoint: endpoint)
            let snap = await ProviderActivityRegistry.shared.record(
                sessionID: sessionID,
                runID: runID,
                providerRequestID: providerRequestID,
                state: .cancelled,
                model: endpoint.modelID.rawValue
            )
            await onActivityChanged?(snap)
            continuation.finish(throwing: CancellationError())
            return
        }
        if await ProviderActivityRegistry.shared.isCancelled(providerRequestID: providerRequestID, runID: runID) {
            doneFlag.markDone()
            released = true
            await rateScheduler.cancel(requestID: requestID, endpoint: endpoint)
            continuation.finish(throwing: CancellationError())
            return
        }
        if !streamedFirstChunk {
            streamedFirstChunk = true
            let snap = await ProviderActivityRegistry.shared.record(
                sessionID: sessionID,
                runID: runID,
                providerRequestID: providerRequestID,
                state: .streaming,
                model: endpoint.modelID.rawValue
            )
            await onActivityChanged?(snap)
        }
        if case let .usage(usage) = event {
            await rateScheduler.recordUsage(endpoint: endpoint, requestID: requestID, usage: usage)
        }
        continuation.yield(event)
    }
    doneFlag.markDone()
    await releasePermit()
    let snap = await ProviderActivityRegistry.shared.record(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: .completed,
        model: endpoint.modelID.rawValue
    )
    await onActivityChanged?(snap)
    continuation.finish()
} catch {
    doneFlag.markDone()
    let isCancelled = (error is CancellationError)
    if isCancelled {
        released = true
        await rateScheduler.cancel(requestID: requestID, endpoint: endpoint)
    } else {
        await releasePermit()
    }
}
let terminalState: ProviderActivityState = isCancelled ? .cancelled : .failed
let snap = await ProviderActivityRegistry.shared.record(
    sessionID: sessionID,

```

```

        runID: runID,
        providerRequestID: providerRequestID,
        state: terminalState,
        model: endpoint.modelID.rawValue,
        detail: (error as? CoreError)?.message ?? error.localizedDescription
    )
    await onActivityChanged?(snap)
    continuation.finish(throwing: error)
}

continuation.onTermination = { @Sendable termination in
    guard !doneFlag.isDone else { return }
    guard case .cancelled = termination else { return }
    pump.cancel()
    Task {
        await rateScheduler.cancel(requestID: requestID, endpoint: endpoint)
        let snap = await ProviderActivityRegistry.shared.cancel(providerRequestID: providerRequestID)
        if let snap { await onActivityChanged?(snap) }
    }
}
}

private func retryDelay(_ retryAfter: Duration?, policy: ProviderRetryPolicy, retry: Int) -> Duration {
    if let retryAfter { return retryAfter }
    let base = min(policy.maxDelayMilliseconds, policy.initialDelayMilliseconds * (1 << min(retry - 1, 20)))
    let jitter = Int(Double(base) * policy.jitterRatio * Double.random(in: 0...1))
    return .milliseconds(base + jitter)
}

private final class StreamDoneFlag: @unchecked Sendable {
    private let lock = NSLock()
    private var done = false

    func markDone() {
        lock.lock()
        done = true
        lock.unlock()
    }

    var isDone: Bool {
        lock.lock()
        defer { lock.unlock() }
        return done
    }
}

/// 模型高速总线：所有 Adapter 在这里收口为“恰好一个合法 terminal”。
public struct ModelBus: Sendable {
    public let gateway: ModelGateway

    public init(gateway: ModelGateway) {
        self.gateway = gateway
    }

    public func stream(
        _ request: ModelRequest,
        onActivityChanged: (@Sendable (ProviderActivitySnapshot) async -> Void)? = nil
    ) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        let source = try await gateway.stream(request, onActivityChanged: onActivityChanged)
        return AsyncThrowingStream { continuation in
            let pump = Task {
                var terminal: ModelFinishReason?
                var completedCalls = 0
            }

```



```

do {
    for try await event in source {
        if terminal != nil {
            continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider terminal 后仍返回事件")))
            continuation.finish()
            return
        }
        switch event {
        case .toolCallCompleted:
            completedCalls += 1
            continuation.yield(event)
        case let .completed(reason):
            terminal = reason
        case .failed:
            continuation.yield(event)
            continuation.finish()
            return
        default:
            continuation.yield(event)
        }
    }
    if Task.isCancelled { continuation.finish(); return }
    guard let terminal else {
        continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider stream 缺少 terminal")))
        continuation.finish()
        return
    }
    guard (completedCalls > 0) == (terminal == .toolCalls) else {
        continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider terminal 与 ToolCall 不一致")))
        continuation.finish()
        return
    }
    continuation.yield(.completed(terminal))
    continuation.finish()
} catch let error as CoreError {
    continuation.yield(.failed(error))
    continuation.finish()
} catch {
    continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider stream 连接中断")))
    continuation.finish()
}
}
continuation.onTermination = { @Sendable _ in pump.cancel() }
}
}
}

```

// File: Sources/LingXiCore/Modules/Model/ModelDomain.swift

```

import Foundation
import LingXiProtocol

// LingXi 自己的模型领域模型。
// 这里没有任何 Provider 原生类型 (choices / delta / finish_reason 等
// 只存在于 OpenAICompatibleProvider Adapter 内部)。

public struct ModelID: Sendable, Equatable, Hashable {
    public let rawValue: String

    public init(_ rawValue: String) {
        self.rawValue = rawValue
    }
}

```

/// 一次模型调用的 Core 身份。它不同于 AgentRun, 也不同于 Provider request/response ID。

```

public struct ModelRequestID: Sendable, Equatable, Hashable, Codable {
    public let rawValue: String

    public init(_ rawValue: String = UUID().uuidString) {
        self.rawValue = rawValue
    }
}

public enum ModelRole: String, Sendable {
    case system
    case user
    case assistant
    case tool
}

/// 模型输入的结构化部分；Provider Adapter 再转换为厂商消息格式。
public enum ModelContentPart: Sendable, Equatable {
    case text(String)
    case toolCall(ToolCall)
    case toolResult(ToolResult)
}

/// Provider wire 只能消费此稳定投影；ToolResult 的内部诊断与持久化字段不会自动外泄。
public enum ResultMaterializationTier: String, Sendable, Codable, Equatable {
    case inlineSmall      // < 1KB: 完全内联
    case structuredDigest // 1KB..<8KB: 结构化摘要 + ContentRef
    case previewLarge     // 8KB..<32KB: 精简预览 (Head 3/4 + Tail 1/4)
    case archiveHuge      // >= 32KB: 纯 ContentRef 引用归档
}

public struct ModelToolResultProjection: Sendable, Equatable {
    public let callID: ToolCallID
    public let toolName: String?
    public let success: Bool
    public let content: String
    public let summary: String?
    public let totalCount: Int?
    public let shownCount: Int?
    public let truncated: Bool?
    public let page: Int?
    public let cursor: String?
    public let items: [String]?

    public init(
        callID: ToolCallID,
        toolName: String?,
        success: Bool,
        content: String,
        summary: String? = nil,
        totalCount: Int? = nil,
        shownCount: Int? = nil,
        truncated: Bool? = nil,
        page: Int? = nil,
        cursor: String? = nil,
        items: [String]? = nil
    ) {
        self.callID = callID
        self.toolName = toolName
        self.success = success
        self.content = content
        self.summary = summary
        self.totalCount = totalCount
        self.shownCount = shownCount
        self.truncated = truncated
        self.page = page
    }
}

```

```

self.cursor = cursor
self.items = items
}

public static func project(_ result: ToolResult, budget: ToolResultBudget = .default) -> Self {
    guard result.success else {
        let error = result.error ?? ToolError(code: "toolExecutionFailed", message: "Tool 执行失败")
        let retryability: Retryability = {
            if result.outcome == .denied { return .afterUserAction }
            if result.outcome == .timedOut || result.outcome == .idleTimedOut { return .transient }
            if result.metadata["retryability"] == Retryability.transient.rawValue { return .transient }
            if result.metadata["retryability"] == Retryability.afterDelay.rawValue { return .afterDelay }
            return .none
        }()
        let errorKind = result.metadata["errorKind"] ?? error.code
        let summary: (String) -> String = { String($0.replacingOccurrences(of: "\n", with: " ").prefix(240)) }
        let projection: [String: Any] = [
            "error": ["code": error.code, "message": error.message],
            "errorKind": errorKind,
            "retryability": retryability.rawValue,
            "durationMilliseconds": result.timing.milliseconds,
            "exitCode": result.exitCode.map { $0 as Any } ?? NSNull(),
            "stdoutSummary": summary(result.diagnostics?.stdout ?? ""),
            "stderrSummary": summary(result.diagnostics?.stderr ?? ""),
            "permissionDenied": result.outcome == .denied,
            "scopeDenied": result.metadata["scopeDenied"] == "true"
        ]
        let content = (try? String(decoding: JSONSerialization.data(withJSONObject: projection, options: [.sortedKeys]), as: UTF8.self))
        ↪ ?? error.message
        return Self(callID: result.callID, toolName: result.toolName, success: false, content: content, summary: error.message)
    }

    let tool = (result.toolName ?? "").lowercased()

    // 1. Glob
    if tool == "glob" {
        var paths: [String] = []
        if let data = result.content.data(using: .utf8),
            let list = try? JSONDecoder().decode([String].self, from: data) {
            paths = list
        } else {
            paths = result.content.split(separator: "\n", omittingEmptySubsequences: true).map(String.init)
        }
        let total = paths.count
        let shown = min(total, budget.maxShown)
        let truncated = total > shown || result.content.count > budget.maxCharacters
        let summary = "Glob · \ \(total) matches\ \(truncated ? " · showing \ \(shown)" : "")"
        let shownPaths = Array(paths.prefix(shown))
        let projectedJSON: String
        if truncated {
            let dict: [String: Any] = [
                "summary": summary,
                "totalCount": total,
                "shownCount": shown,
                "truncated": true,
                "page": 1,
                "matches": shownPaths
            ]
            if let data = try? JSONSerialization.data(withJSONObject: dict, options: [.sortedKeys]),
                let str = String(data: data, encoding: .utf8) {
                projectedJSON = str
            } else {
                projectedJSON = summary + "\n" + shownPaths.joined(separator: "\n")
            }
        } else {

```

```

        projectedJSON = result.content
    }
    return Self(
        callID: result.callID,
        toolName: result.toolName,
        success: true,
        content: projectedJSON,
        summary: summary,
        totalCount: total,
        shownCount: shown,
        truncated: truncated,
        page: 1,
        cursor: truncated ? String(shown) : nil,
        items: shownPaths
    )
}

// 2. ListDirectory
if tool == "list_directory" {
    let lines = result.content.split(separator: "\n", omittingEmptySubsequences: true).map(String.init)
    let total = lines.count
    let shown = min(total, budget.maxShown)
    let truncated = total > shown || result.content.count > budget.maxCharacters
    let summary = "ListDirectory · \$(total) entries\$(truncated ? " · showing \$(shown)" : "")"
    let shownEntries = Array(lines.prefix(shown))
    let projectedJSON: String
    if truncated {
        let dict: [String: Any] = [
            "summary": summary,
            "totalCount": total,
            "shownCount": shown,
            "truncated": true,
            "page": 1,
            "entries": shownEntries
        ]
        if let data = try? JSONSerialization.data(withJSONObject: dict, options: [.sortedKeys]),
            let str = String(data: data, encoding: .utf8) {
            projectedJSON = str
        } else {
            projectedJSON = summary + "\n" + shownEntries.joined(separator: "\n")
        }
    } else {
        projectedJSON = result.content
    }
    return Self(
        callID: result.callID,
        toolName: result.toolName,
        success: true,
        content: projectedJSON,
        summary: summary,
        totalCount: total,
        shownCount: shown,
        truncated: truncated,
        page: 1,
        cursor: truncated ? String(shown) : nil,
        items: shownEntries
    )
}

// 3. Grep
if tool == "grep" {
    if let data = result.content.data(using: .utf8),
        let list = try? JSONSerialization.jsonObject(with: data) as? [[String: Any]] {
        let total = list.count
        let shown = min(total, budget.maxShown)

```

```

let truncated = total > shown || result.content.count > budget.maxCharacters
let summary = "Grep · \((total) matches\((truncated ? " · showing \((shown)" : "")"
let shownMatches = Array(list.prefix(shown))
let projectedJSON: String
if truncated {
  let dict: [String: Any] = [
    "summary": summary,
    "totalCount": total,
    "shownCount": shown,
    "truncated": true,
    "page": 1,
    "matches": shownMatches
  ]
  if let data = try? JSONSerialization.data(withJSONObject: dict, options: [.sortedKeys]),
    let str = String(data: data, encoding: .utf8) {
    projectedJSON = str
  } else {
    projectedJSON = summary
  }
} else {
  projectedJSON = result.content
}
return Self(
  callID: result.callID,
  toolName: result.toolName,
  success: true,
  content: projectedJSON,
  summary: summary,
  totalCount: total,
  shownCount: shown,
  truncated: truncated,
  page: 1,
  cursor: truncated ? String(shown) : nil
)
}
}

// 4. Read (read_file / read)
if tool == "read_file" || tool == "read" {
  // 4.1 如果已经是结构化 ReadPage JSON, 尊重其已有分页元数据, 不要二次破坏
  if let data = result.content.data(using: .utf8),
    let pageDict = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
    let rawLines = pageDict["lines"] as? [[String: Any]] {
    let startLine = pageDict["startLine"] as? Int ?? 1
    let endLine = pageDict["endLine"] as? Int ?? rawLines.count
    let nextLine = pageDict["nextLine"] as? Int
    let truncated = pageDict["truncated"] as? Bool ?? false
    let lineTexts = rawLines.compactMap { dict -> String? in
      if let num = dict["number"] as? Int, let c = dict["content"] as? String {
        return "\(num)\t\(c)"
      }
    }
    return dict["content"] as? String
  }
}
let summary = "ReadFile · lines \((startLine)-\((endLine) (truncated: \((truncated)))"
return Self(
  callID: result.callID,
  toolName: result.toolName,
  success: true,
  content: result.content,
  summary: summary,
  totalCount: rawLines.count,
  shownCount: rawLines.count,
  truncated: truncated,
  page: max(1, startLine / max(1, rawLines.count)),
  cursor: nextLine.map(String.init),

```

```

        items: lineTexts
    )
}

// 4.2 普通全文读取：提供合理的单次阅读视窗（默认 60 行）
let lines = result.content.split(separator: "\n", omittingEmptySubsequences: false).map(String.init)
let total = lines.count
let maxLines = max(budget.maxShown, 60)
let shown = min(total, maxLines)
let truncated = total > shown || result.content.count > budget.maxCharacters
let summary = truncated ? "ReadFile · \$(total) lines · showing \$(shown)" : result.summary
let shownLines = Array(lines.prefix(shown))
let projectedContent: String
if truncated {
    let dict: [String: Any] = [
        "summary": summary,
        "totalCount": total,
        "shownCount": shown,
        "truncated": true,
        "page": 1,
        "nextLine": shown + 1,
        "tip": "File truncated at line \$(shown). Use read_file with start_line=\$(shown + 1) to read further.",
        "lines": shownLines
    ]
    if let data = try? JSONSerialization.data(withJSONObject: dict, options: [.sortedKeys]),
        let str = String(data: data, encoding: .utf8) {
        projectedContent = str
    } else {
        projectedContent = summary + "\n" + shownLines.joined(separator: "\n")
    }
} else {
    projectedContent = result.content
}
return Self(
    callID: result.callID,
    toolName: result.toolName,
    success: true,
    content: projectedContent,
    summary: summary,
    totalCount: total,
    shownCount: shown,
    truncated: truncated,
    page: 1,
    cursor: truncated ? String(shown + 1) : nil,
    items: shownLines
)
}

// 5. Search (search_tools / search)
if tool == "search_tools" || tool == "search" {
    if let data = result.content.data(using: .utf8),
        let list = try? JSONSerialization.jsonObject(with: data) as? [[String: Any]] {
        let total = list.count
        let shown = min(total, budget.maxShown)
        let truncated = total > shown || result.content.count > budget.maxCharacters
        let summary = truncated ? "Search · \$(total) results · showing \$(shown)" : result.summary
        let shownItems = Array(list.prefix(shown))
        let projectedJSON: String
        if truncated {
            let dict: [String: Any] = [
                "summary": summary,
                "totalCount": total,
                "shownCount": shown,
                "truncated": true,
                "page": 1,

```

```

        "results": shownItems
    ]
    if let data = try? JSONSerialization.data(withJSONObject: dict, options: [.sortedKeys]),
        let str = String(data: data, encoding: .utf8) {
        projectedJSON = str
    } else {
        projectedJSON = summary
    }
} else {
    projectedJSON = result.content
}
return Self(
    callID: result.callID,
    toolName: result.toolName,
    success: true,
    content: projectedJSON,
    summary: summary,
    totalCount: total,
    shownCount: shown,
    truncated: truncated,
    page: 1,
    cursor: truncated ? String(shown) : nil
)
}
}

// 6. Generic bounded character budget & Multi-Tier Materialization
if result.content.count >= 32 * 1024 {
    let head = result.content.prefix(512)
    let tail = result.content.suffix(256)
    let dropped = result.content.count - head.count - tail.count
    let truncatedContent = "\n(head)\n... [(\ndropped) characters truncated for prefix-cache efficiency · tier: archiveHuge] ...\n(\n(
↔ tail)"
    let summary = "ToolResult · \n(result.toolName ?? "output") · Archived · \n(result.content.count) chars"
    return Self(
        callID: result.callID,
        toolName: result.toolName,
        success: true,
        content: truncatedContent,
        summary: summary,
        truncated: true
    )
} else if result.content.count > budget.maxCharacters {
    let maxKeep = budget.maxCharacters
    let headSize = min(result.content.count, maxKeep * 3 / 4)
    let tailSize = min(result.content.count - headSize, maxKeep / 4)
    let head = result.content.prefix(headSize)
    let tail = result.content.suffix(tailSize)
    let dropped = result.content.count - headSize - tailSize
    let truncatedContent = "\n(head)\n... [(\ndropped) characters truncated for prefix-cache efficiency] ...\n(tail)"
    return Self(
        callID: result.callID,
        toolName: result.toolName,
        success: true,
        content: truncatedContent,
        summary: result.summary,
        truncated: true
    )
}

return Self(callID: result.callID, toolName: result.toolName, success: true, content: result.content, summary: result.summary)
}

public static func projectToolResult(_ result: ToolResult, budget: ToolResultBudget = .default) -> ToolResult {
    guard result.success else { return result }

```

```

        let projected = project(result, budget: budget)
        let outMeta = ToolOutputMetadata(
            truncated: projected.truncated ?? false,
            totalCharacters: result.content.count,
            visibleCharacters: projected.content.count
        )
        return result.withContent(projected.content, summary: projected.summary, output: outMeta)
    }
}

public struct ModelMessage: Sendable, Equatable {
    public let role: ModelRole
    public let parts: [ModelContentPart]

    public var content: String {
        parts.compactMap { if case let .text(text) = $0 { text } else { nil } }.joined()
    }

    public init(role: ModelRole, content: String) {
        self.init(role: role, parts: [.text(content)])
    }

    public init(role: ModelRole, parts: [ModelContentPart]) {
        self.role = role
        self.parts = parts
    }
}

public struct ModelRequest: Sendable, Equatable {
    public let requestID: ModelRequestID
    public let continuationOf: ModelRequestID?
    public let model: ModelID
    public let executionID: AgentRunID?
    public let system: String?
    public let messages: [ModelMessage]
    public let tools: [ToolDefinition]
    public let reasoning: String?
    public let debugStep: Int?
    public let overallTimeoutSeconds: Double?
    public let idleTimeoutSeconds: Double?
    public let cachePlan: CanonicalCachePlan?

    public init(
        requestID: ModelRequestID = ModelRequestID(),
        continuationOf: ModelRequestID? = nil,
        model: ModelID,
        executionID: AgentRunID? = nil,
        system: String? = nil,
        messages: [ModelMessage],
        tools: [ToolDefinition] = [],
        reasoning: String? = nil,
        debugStep: Int? = nil,
        overallTimeoutSeconds: Double? = nil,
        idleTimeoutSeconds: Double? = nil,
        cachePlan: CanonicalCachePlan? = nil
    ) {
        self.requestID = requestID
        self.continuationOf = continuationOf
        self.model = model
        self.executionID = executionID
        self.system = system
        self.messages = messages
        self.tools = tools
        self.reasoning = reasoning
        self.debugStep = debugStep
    }
}

```



```

        self.overallTimeoutSeconds = overallTimeoutSeconds
        self.idleTimeoutSeconds = idleTimeoutSeconds
        self.cachePlan = cachePlan
    }
}

public enum ProviderTraceSanitizer {
    public static func requestID(_ value: String?) -> String? {
        guard let value else { return nil }
        let trimmed = String(value.prefix(128))
        guard !trimmed.isEmpty,
            trimmed.allSatisfy({ $0.isLetter || $0.isNumber || "-_.".contains($0) })
        else { return nil }
        return trimmed
    }
}

/// 模型推理事件流。高频 delta 走 DMA, started/usage/completed/failed 由 Agent 分流到控制面。
public enum ModelEvent: Sendable, Equatable {
    /// Provider HTTP 响应提供的可审计请求 ID, 已在 adapter 边界完成清洗。
    case providerRequestID(String)
    /// Provider 连接建立、推理即将开始。
    case started
    case textDelta(String)
    /// 推理内容 delta; Provider 不支持时不会出现。
    case reasoningDelta(String)
    case toolCallStarted(callID: ToolCallID, toolID: ToolID)
    case toolCallDelta(callID: ToolCallID, arguments: String)
    case toolCallCompleted(ToolCall)
    case usage(ModelUsage)
    case completed(ModelFinishReason)
    /// 流中途失败 (Model Stream Error) 。
    case failed(CoreError)
}

```

// File: Sources/LingXiCore/Modules/Model/ModelProvider.swift

```

import Foundation
import LingXiProtocol

/// Provider 接口: LingXi Domain 正式边界。
/// 任何 Provider (OpenAI 原生 / Anthropic / Gemini / 兼容端点) 实现此接口,
/// 上层 (Gateway / Bus / Agent) 永远只看到 LingXi 领域类型。
public protocol ModelProvider: Sendable {
    /// 建立推理流。连接 / HTTP 失败直接 throw (Provider Error) ;
    /// 成功后返回 ModelEvent 流 (流中途失败以 .failed 事件交付)。
    func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error>
    func endExecution(_ executionID: AgentRunID) async
}

public extension ModelProvider {
    func endExecution(_ executionID: AgentRunID) async {}
}

```

// File: Sources/LingXiCore/Modules/Model/OpenAICompatibleProvider.swift

```

import Foundation
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

/// OpenAI-compatible Provider Adapter。
/// 本类型是 Core 中唯一允许出现 OpenAI Chat Completions JSON 结构的地方:
/// choices / delta / finish_reason / prompt_tokens 等只存在于下方私有 DTO,

```

```

/// 出口一律转换为 LingXi ModelEvent。
public struct OpenAICompatibleProvider: ModelProvider {
    private let config: ProviderConfig
    private let transport: any ProviderHTTPTransport
    private let provenance: ProviderProvenanceStore

    public init(config: ProviderConfig, session: URLSession = .shared, provenance: ProviderProvenanceStore = ProviderProvenanceStore()) {
        self.config = config
        transport = URLSessionProviderHTTPTransport(session: session)
        self.provenance = provenance
    }

    public init(config: ProviderConfig, transport: any ProviderHTTPTransport, provenance: ProviderProvenanceStore = ProviderProvenanceStore
↳ ()) {
        self.config = config
        self.transport = transport
        self.provenance = provenance
    }

    // MARK: - ModelProvider

    public func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        let prior = try await provenance.resolveContinuation(for: request.continuationOf, wire: .chatCompletions)
        let urlRequest = try makeURLRequest(request, continuation: prior)
        Self.log("request step=\(request.debugStep ?? 0) bytes=\(urlRequest.httpBody?.count ?? 0)", enabled: config.diagnosticsEnabled)

        // 连接阶段: 失败直接 throw (Provider Error), 由调用方转换为控制面结果。
        let response = try await transport.send(urlRequest, context: ProviderHTTPRequestContext(wireProtocol: .chatCompletions, model:
↳ request.model.rawValue, requestID: request.requestID, executionID: request.executionID, step: request.debugStep ?? 0))
        let requestID = response.header("x-request-id")
        ?? response.header("openai-request-id")
        ?? response.header("cf-ray")
        guard (200..<300).contains(response.statusCode) else {
            let errorBody = (try? await Self.collectText(response.body)) ?? ""
            let requestFields: [String]
            if let body = urlRequest.httpBody, let json = (try? JSONSerialization.jsonObject(with: body)) as? [String: Any] {
                requestFields = json.keys.sorted()
            } else {
                requestFields = []
            }
            let toolSchemas = request.tools.map { "\($0.name)\($0.inputSchema.required.joined(separator: ", "))" }
            let capabilityProjection = "toolsRequested=\(!request.tools.isEmpty) (count=\(request.tools.count)), reasoningRequested=\(
↳ request.reasoning != nil)"
            let diagnostics = Self.makeWireDiagnostics(
                provider: config.baseURL.host ?? "custom",
                model: request.model.rawValue,
                wireAdapter: config.wireProtocol.rawValue,
                url: urlRequest.url?.absoluteString ?? config.chatCompletionsURL.absoluteString,
                httpMethod: urlRequest.httpMethod ?? "POST",
                requestFields: requestFields,
                toolSchemas: toolSchemas,
                capabilityProjection: capabilityProjection,
                responseBody: errorBody
            )
            Self.log("http error step=\(request.debugStep ?? 0) status=\(response.statusCode) diagnostics:\n\((diagnostics)", enabled: config
↳ .diagnosticsEnabled)
            let error = Self.httpError(status: response.statusCode, requestID: requestID, diagnostics: diagnostics)
            throw ProviderRateLimitError.from(statusCode: response.statusCode, headers: response.headers, body: errorBody, underlying: error
↳ )
        }

        // 数据面 pump: 连接已建立, 事件从独立任务流出。
        let events = AsyncThrowingStream<ModelEvent, Error> { continuation in
            if let requestID = ProviderTraceSanitizer.requestID(requestID) {
                continuation.yield(.providerRequestID(requestID))
            }
        }
    }
}

```

```

    }
    let pump = Task {
        await Pump(
            source: response.body,
            continuation: continuation,
            request: request,
            prior: prior,
            provenance: provenance,
            debugStep: request.debugStep,
            diagnosticsEnabled: config.diagnosticsEnabled,
            performanceDiagnosticsEnabled: config.performanceDiagnosticsEnabled
        ).run()
    }
    continuation.onTermination = { @Sendable _ in
        pump.cancel()
    }
}
return events
}

// MARK: - Request 转换 (Domain → OpenAI-compatible JSON)

public func makeURLRequest(_ request: ModelRequest) throws -> URLRequest {
    try makeURLRequest(request, continuation: nil)
}

private func makeURLRequest(_ request: ModelRequest, continuation: ProviderContinuation?) throws -> URLRequest {
    var urlRequest = URLRequest(url: config.chatCompletionsURL)
    urlRequest.httpMethod = "POST"
    if let timeout = request.overallTimeoutSeconds { urlRequest.timeoutInterval = timeout }
    urlRequest.setValue("application/json", forHTTPHeaderField: "Content-Type")
    urlRequest.setValue("application/json", forHTTPHeaderField: "Accept")
    urlRequest.setValue("LingXiAgent/1.0", forHTTPHeaderField: "User-Agent")
    switch config.authentication {
    case .none: break
    case let .bearer(secret): urlRequest.setValue("Bearer \(secret)", forHTTPHeaderField: "Authorization")
    case let .header(name, value): urlRequest.setValue(value, forHTTPHeaderField: name)
    }
    for (name, value) in config.requiredHeaders {
        urlRequest.setValue(value, forHTTPHeaderField: name)
    }
    urlRequest.httpBody = try Self.makeRequestBody(request, continuation: continuation, parallelToolCalls: config.parallelToolCalling ??
↪ true)
    return urlRequest
}

/// 可测试: Domain 请求 → wire JSON。
public static func makeRequestBody(_ request: ModelRequest, parallelToolCalls: Bool? = nil) throws -> Data {
    try makeRequestBody(request, continuation: nil, parallelToolCalls: parallelToolCalls)
}

private static func makeRequestBody(_ request: ModelRequest, continuation: ProviderContinuation?, parallelToolCalls: Bool? = nil) throws
↪ -> Data {
    var messages: [ChatRequestBody.Message] = []
    let orderedTools: [ToolDefinition]
    if let plan = request.cachePlan {
        if let system = plan.immutableBase.systemPrompt, !system.isEmpty {
            messages.append(Message(role: "system", content: system))
        }
        let contextMsgs = plan.appendOnlyContext.messages.flatMap { providerMessages($0, continuation: continuation) }
        if messages.contains(where: { $0.role == "system" }) {
            // 已有系统提示词，严格删除后续重复的 system 消息，确保 KV Cache 前缀严格命中
            messages.append(contentsOf: contextMsgs.filter { $0.role != "system" })
        } else {
            messages.append(contentsOf: contextMsgs)
        }
    }
}

```

```

    }
    orderedTools = plan.immutableBase.coreTools + plan.appendOnlyContext.dynamicTools
  } else {
    if let system = request.system, !system.isEmpty {
      messages.append(Message(role: "system", content: system))
    }
    let contextMsgs = request.messages.flatMap { providerMessages($0, continuation: continuation) }
    if messages.contains(where: { $0.role == "system" }) {
      messages.append(contentsOf: contextMsgs.filter { $0.role != "system" })
    } else {
      messages.append(contentsOf: contextMsgs)
    }
    let coreIDs = ToolRuntime.coreToolIDs
    let core = request.tools.filter { coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
    let dynamic = request.tools.filter { !coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
    orderedTools = core + dynamic
  }
  let enableParallel = orderedTools.isEmpty ? nil : ((parallelToolCalls ?? true) ? true : nil)
  let sanitizedMessages = sanitizeMessagesForContract(messages)
  let body = ChatRequestBody(
    model: request.model.rawValue,
    stream: true,
    messages: sanitizedMessages,
    tools: orderedTools.isEmpty ? nil : orderedTools.map(ProviderTool.init),
    parallelToolCalls: enableParallel,
    streamOptions: ChatRequestBody.StreamOptions(includeUsage: true)
  )
  let encoder = JSONEncoder()
  encoder.outputFormatting = [.sortedKeys]
  return try encoder.encode(body)
}

```

/// 保证送往上游模型的历史消息严格符合 OpenAI API 规范：

/// 1. 任何带有 tool_calls 的 assistant 消息，必须紧随其声明的所有 tool 结果消息；

/// 若后续 tool 结果丢失或被截断，自动删除孤儿 tool_call；

/// 2. 任何 tool 角色消息如果前面缺少声明其 call_id 的 assistant 消息，坚决删除；

/// 3. 若 assistant 的所有 tool_calls 均被修剪且无内容，整条丢弃，杜绝发送空消息引发 400 或模型死循环。

```

static func sanitizeMessagesForContract(_ rawMessages: [ChatRequestBody.Message]) -> [ChatRequestBody.Message] {
  var result: [ChatRequestBody.Message] = []
  var i = 0
  while i < rawMessages.count {
    let msg = rawMessages[i]
    if msg.role == "assistant", let toolCalls = msg.toolCalls, !toolCalls.isEmpty {
      var toolResults: [ChatRequestBody.Message] = []
      var j = i + 1
      while j < rawMessages.count && rawMessages[j].role == "tool" {
        toolResults.append(rawMessages[j])
        j += 1
      }
      let answeredIDs = Set(toolResults.compactMap(\.toolCallID))
      let validCalls = toolCalls.filter { answeredIDs.contains($0.id) }
      let validCallIDs = Set(validCalls.map(\.id))
      let validToolResults = toolResults.filter { m in
        guard let id = m.toolCallID else { return false }
        return validCallIDs.contains(id)
      }
      if !validCalls.isEmpty {
        result.append(Message(
          role: "assistant",
          content: (msg.content?.isEmpty ?? true) ? nil : msg.content,
          toolCalls: validCalls
        ))
        result.append(contentsOf: validToolResults)
      } else if let content = msg.content, !content.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {

```

```

        result.append(Message(role: "assistant", content: content))
    }
    i = j
} else if msg.role == "tool" {
    // 孤儿 tool 消息, 跳过
    i += 1
} else {
    result.append(msg)
    i += 1
}
}
return result
}

private static func providerMessages(_ message: ModelMessage, continuation: ProviderContinuation?) -> [ChatRequestBody.Message] {
    let calls = message.parts.compactMap { part -> ToolCall? in
        guard case let .toolCall(call) = part else { return nil }
        return call
    }
    let results = message.parts.compactMap { part -> ToolResult? in
        guard case let .toolResult(result) = part else { return nil }
        return result
    }
    switch message.role {
    case .tool:
        return results.compactMap { result in
            let projected = ModelToolResultProjection.project(result)
            let callID = continuation?.externalCallID(for: projected.callID) ?? projected.callID.rawValue
            guard !callID.isEmpty else { return nil }
            return Message(role: "tool", content: projected.content, toolCallID: callID)
        }
    case .assistant:
        let validCalls = calls.compactMap { call -> ProviderToolCall? in
            let name = call.toolID.rawValue.trimmingCharacters(in: .whitespacesAndNewlines)
            guard !name.isEmpty else { return nil }
            let id = continuation?.externalCallID(for: call.callID) ?? call.callID.rawValue
            return ProviderToolCall(id: id, function: ProviderToolCall.Function(name: name, arguments: call.arguments))
        }
        return [Message(
            role: "assistant",
            content: message.content.isEmpty && !validCalls.isEmpty ? nil : message.content,
            toolCalls: validCalls.isEmpty ? nil : validCalls
        )]
    case .system, .user:
        return [Message(role: message.role.rawValue, content: message.content)]
    }
}

// MARK: - 错误转换

public static func httpError(status: Int, requestId: String? = nil, diagnostics: String? = nil) -> CoreError {
    let safeRequestID = requestId.map { String($0.prefix(128)) }.flatMap { value in
        value.isEmpty || !value.allSatisfy({ $0.isLetter || $0.isNumber || "-_.".contains($0) }) ? nil : value
    }
    let suffix = safeRequestID.map { " requestID=\($0)" } ?? ""
    var msg = "Provider HTTP 请求失败: status=\(status)\(suffix)"
    if let diagnostics, !diagnostics.isEmpty {
        msg += "\n[Wire Diagnostics]\n" + diagnostics
    }
    return CoreError(code: .provider, message: msg)
}

public static func makeWireDiagnostics(
    provider: String,
    model: String,

```

```

        wireAdapter: String,
        url: String,
        httpMethod: String,
        requestFields: [String],
        toolSchemas: [String],
        capabilityProjection: String,
        responseBody: String
    ) -> String {
        var lines: [String] = []
        lines.append("resolved provider: \(provider)")
        lines.append("resolved model: \(model)")
        lines.append("wire adapter: \(wireAdapter)")
        lines.append("final URL/path: \(url)")
        lines.append("HTTP method: \(httpMethod)")
        lines.append("request fields: [\(requestFields.joined(separator: " ", ""))]")
        lines.append("tool schemas: [\(toolSchemas.joined(separator: " ", ""))]")
        lines.append("reasoning/tool capability projection: \(capabilityProjection)")
        lines.append("response/error body: \(\sanitizeResponseBody(responseBody))")
        return lines.joined(separator: "\n")
    }

    public static func sanitizeResponseBody(_ raw: String) -> String {
        var cleaned = raw.replacingOccurrences(of: "Bearer [A-Za-z0-9_\\-\\.]+", with: "Bearer [redacted]", options: .regularExpression)
        cleaned = cleaned.replacingOccurrences(of: "\"([a-zA-Z0-9_-]*(?:api[_]?key|token|secret|password)[a-zA-Z0-9_-]*)\"\\s*:\\s"
        ↪ *\"[^\"]+\"\\s*", with: "\"$1\": \"[redacted]\"", options: .regularExpression)
        cleaned = cleaned.replacingOccurrences(of: "[A-Za-z0-9_]*(?:SECRET|API_KEY|APIKEY|ACCESS_KEY)[A-Za-z0-9_]*", with: "[redacted]",
        ↪ options: .regularExpression)
        if cleaned.count > 2048 {
            cleaned = String(cleaned.prefix(2048)) + "... (truncated)"
        }
        return cleaned.isEmpty ? "(empty)" : cleaned
    }

    /// 可测试: SSE data 行 payload → 0..n 个 ModelEvent。
    /// malformed JSON 抛出明确错误, 绝不 crash Core。
    public static func events(forSSEPayload payload: String) throws -> [ModelEvent] {
        let trimmed = payload.trimmingCharacters(in: .whitespaces)
        guard !trimmed.isEmpty else { return [] }
        return events(from: try decodeSSEChunk(trimmed))
    }

    private static func decodeSSEChunk(_ payload: String) throws -> SSEChunk {
        guard let data = payload.data(using: .utf8),
              let chunk = try? JSONDecoder().decode(SSEChunk.self, from: data)
        else {
            throw CoreError(code: .modelStream, message: "SSE chunk JSON 解析失败")
        }
        return chunk
    }

    private static func events(from chunk: SSEChunk) -> [ModelEvent] {
        var events: [ModelEvent] = []
        if let usage = chunk.usage.map(Self.usage(from:)) {
            events.append(.usage(usage))
        }
        if let choice = chunk.choices?.first {
            if let reason = choice.delta?.reasoning, !reason.isEmpty {
                events.append(.reasoningDelta(reason))
            }
            if let text = choice.delta?.content, !text.isEmpty {
                events.append(.textDelta(text))
            }
            if let reason = choice.finishReason {
                events.append(.completed(reason))
            }
        }
    }

```

```

    }
    if chunk.choices?.first?.finishReason == nil, let reason = chunk.finishReason {
        events.append(.completed(reason))
    }
    return events
}

public static func finishReason(fromOpenAI raw: String?) -> ModelFinishReason? {
    switch raw {
    case "stop": .stop
    case "length": .maxTokens
    case "content_filter": .contentFilter
    case "tool_calls": .toolCalls
    case nil, .some(""): nil
    default: .unknown
    }
}

private static func extractCachedTokens(from raw: SSEUsage) -> Int? {
    let detailsList = [
        raw.promptTokensDetails,
        raw.promptTokenDetails,
        raw.inputTokensDetails,
        raw.inputTokenDetails
    ]
    for d in detailsList {
        if let t = d?.cachedTokens ?? d?.cachedPromptTokens ?? d?.cacheReadTokens {
            return t
        }
    }
    if let direct = raw.cachedTokens ?? raw.cachedPromptTokens ?? raw.cacheReadTokens {
        return direct
    }
    return raw.promptCacheHitTokens ?? raw.cacheReadInputTokens
}

private static func usage(from raw: SSEUsage) -> ModelUsage {
    let cached = extractCachedTokens(from: raw)
    return ModelUsage(
        inputTokens: raw.promptTokens,
        outputTokens: raw.completionTokens,
        reasoningTokens: raw.completionTokensDetails?.reasoningTokens,
        cacheReadTokens: cached,
        cacheWriteTokens: raw.cacheCreationInputTokens
    )
}

static func collectText(_ bytes: URLSession.AsyncBytes) async throws -> String {
    var data = Data()
    var count = 0
    for try await byte in bytes {
        data.append(byte)
        count += 1
        if count > 64 * 1024 { break }
    }
    return String(decoding: data, as: UTF8.self)
}

static func collectText(_ stream: AsyncThrowingStream<Data, Error>) async throws -> String {
    var data = Data()
    var count = 0
    for try await chunk in stream {
        let remaining = 64 * 1024 - count
        guard remaining > 0 else { break }
        data.append(chunk.prefix(remaining))
    }
}

```

```

        count += min(chunk.count, remaining)
    }
    return String(decoding: data, as: UTF8.self)
}

private static func log(_ message: String, enabled: Bool) {
    guard enabled else { return }
    FileHandle.standardError.write(Data("[chat-diagnostic] \(message)\n").utf8)
}

// MARK: - Pump (数据面)

/// 消费网络字节: bytes → SSE buffer → 行 → JSON → ModelEvent。
private final class Pump: Sendable {
    private let source: AsyncThrowingStream<Data, Error>
    private let continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation
    private let request: ModelRequest
    private let prior: ProviderContinuation?
    private let provenance: ProviderProvenanceStore
    private let debugStep: Int?
    private let diagnosticsEnabled: Bool
    private let performanceDiagnosticsEnabled: Bool

    init(source: AsyncThrowingStream<Data, Error>, continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation, request:
↳ ModelRequest, prior: ProviderContinuation?, provenance: ProviderProvenanceStore, debugStep: Int?, diagnosticsEnabled: Bool,
↳ performanceDiagnosticsEnabled: Bool) {
        self.source = source
        self.continuation = continuation
        self.request = request
        self.prior = prior
        self.provenance = provenance
        self.debugStep = debugStep
        self.diagnosticsEnabled = diagnosticsEnabled
        self.performanceDiagnosticsEnabled = performanceDiagnosticsEnabled
    }

    func run() async {
        var decoder = SSEDecoder()
        var completed: ModelFinishReason?
        var toolCalls = ToolCallBuffer(requestID: request.requestID, debugStep: debugStep, diagnosticsEnabled:
↳ performanceDiagnosticsEnabled)
        var sawDone = false
        var textChunks = 0
        var reasoningChunks = 0
        var toolChunks = 0
        continuation.yield(.started)

        do {
            outer: for try await chunk in source {
                for line in decoder.feed(chunk) {
                    if try handle(line: line, into: &completed, toolCalls: &toolCalls, textChunks: &textChunks, reasoningChunks: &
↳ reasoningChunks, toolChunks: &toolChunks) {
                        sawDone = true
                        break outer
                    }
                }
            }
        }
        if !sawDone {
            for line in decoder.flushPending() {
                if try handle(line: line, into: &completed, toolCalls: &toolCalls, textChunks: &textChunks, reasoningChunks: &
↳ reasoningChunks, toolChunks: &toolChunks) {
                    sawDone = true
                    break
                }
            }
        }
    }
}

```



```

    }
} catch let error as CoreError {
    if Task.isCancelled { continuation.finish(); return }
    continuation.yield(.failed(error))
    continuation.finish()
    return
} catch {
    if Task.isCancelled { continuation.finish(); return }
    continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider 流连接中断")))
    continuation.finish()
    return
}

if Task.isCancelled { continuation.finish(); return }
guard sawDone || completed != nil else {
    continuation.yield(.failed(CoreError(code: .modelStream, message: "Provider SSE 意外结束")))
    continuation.finish()
    return
}

do {
    for event in try toolCalls.complete() {
        continuation.yield(event)
    }
    let combinedReferences = (prior?.references ?? []).filter { priorRef in !toolCalls.references.contains { $0.domainCallID ==
↳ priorRef.domainCallID } } + toolCalls.references
    if !combinedReferences.isEmpty {
        try await provenance.record(ProviderContinuation(requestID: request.requestID, executionID: request.executionID, wire: .
↳ chatCompletions, references: combinedReferences))
    }
} catch let error as CoreError {
    continuation.yield(.failed(error))
    continuation.finish()
    return
} catch {
    continuation.yield(.failed(CoreError(code: .modelStream, message: "Tool Call 聚合失败")))
    continuation.finish()
    return
}

// [DONE] 或 EOF: 有 finish_reason 用之, 否则宽容收尾。
OpenAICompatibleProvider.log("stream step=\(debugStep ?? 0) textChunks=\(textChunks) reasoningChunks=\(reasoningChunks)
↳ toolChunks=\(toolChunks) finish=\(String(describing: completed))", enabled: diagnosticsEnabled)
continuation.yield(.completed(completed ?? .unknown))
continuation.finish()
}

/// 处理一行 SSE; 返回 true 表示遇到 [DONE], 应停止读取。
/// malformed JSON 抛错, 由 run 统一转为 .failed 并终止。
private func handle(
    line: String,
    into completed: inout ModelFinishReason?,
    toolCalls: inout ToolCallBuffer,
    textChunks: inout Int,
    reasoningChunks: inout Int,
    toolChunks: inout Int
) throws -> Bool {
    let trimmed = line.trimmingCharacters(in: .whitespaces)
    if trimmed.isEmpty || trimmed.hasPrefix(":") { return false }
    guard trimmed.hasPrefix("data:") else { return false } // event:/retry:/id: 忽略
    let payload = trimmed.dropFirst("data:".count).trimmingCharacters(in: .whitespaces)

    let unquotedPayload = payload.trimmingCharacters(in: CharacterSet(charactersIn: "\"'\\t "))
    if unquotedPayload.caseInsensitiveCompare("[done]") == .orderedSame || unquotedPayload.caseInsensitiveCompare("done") == .
↳ orderedSame { return true }

```

```

    let chunk = try OpenAICompatibleProvider.decodeSSEChunk(String(payload))
    if let delta = chunk.choices?.first?.delta {
        if !(delta.content ?? "").isEmpty { textChunks += 1 }
        if !(delta.reasoning ?? "").isEmpty { reasoningChunks += 1 }
        toolChunks += delta.toolCalls?.count ?? 0
    }
    for event in OpenAICompatibleProvider.events(from: chunk) {
        if case let .completed(reason) = event {
            completed = reason
        } else {
            continuation.yield(event)
        }
    }
    if let calls = chunk.choices?.first?.delta?.toolCalls {
        for event in try toolCalls.consume(calls) {
            continuation.yield(event)
        }
    }
    return false
}
}

extension OpenAICompatibleProvider {
    struct ChatRequestBody: Encodable {
        struct Message: Encodable {
            let role: String
            let content: String?
            let toolCalls: [ProviderToolCall]?
            let toolCallID: String?

            init(role: String, content: String?, toolCalls: [ProviderToolCall]? = nil, toolCallID: String? = nil) {
                self.role = role
                self.content = content
                self.toolCalls = toolCalls
                self.toolCallID = toolCallID
            }
        }

        enum CodingKeys: String, CodingKey {
            case role, content
            case toolCalls = "tool_calls"
            case toolCallID = "tool_call_id"
        }
    }

    struct StreamOptions: Encodable {
        let includeUsage: Bool

        enum CodingKeys: String, CodingKey {
            case includeUsage = "include_usage"
        }
    }

    let model: String
    let stream: Bool
    let messages: [Message]
    let tools: [ProviderTool]?
    let parallelToolCalls: Bool?
    let streamOptions: StreamOptions?

    enum CodingKeys: String, CodingKey {
        case model, stream, messages, tools
        case parallelToolCalls = "parallel_tool_calls"
        case streamOptions = "stream_options"
    }
}

```

```

}

typealias Message = ChatRequestBody.Message

struct ProviderTool: Encodable {
    struct Function: Encodable {
        let name: String
        let description: String
        let parameters: JSONValue
    }

    let type = "function"
    let function: Function

    init(_ definition: ToolDefinition) {
        function = Function(
            name: definition.name,
            description: definition.description,
            parameters: definition.rawInputSchema ?? .object([
                "type": .string("object"),
                "properties": .object(definition.inputSchema.properties.mapValues { property in
                    var value: [String: JSONValue] = ["type": .string(property.type.rawValue), "description": .string(property.
↵ description)]
                    if let enumValues = property.enumValues { value["enum"] = .array(enumValues.map(JSONValue.string)) }
                    if let minimum = property.minimum { value["minimum"] = .number(minimum) }
                    if let maximum = property.maximum { value["maximum"] = .number(maximum) }
                    return .object(value)
                }
            )),
            "required": .array(definition.inputSchema.required.map(JSONValue.string)),
            "additionalProperties": .bool(false),
        ))
    }
}

struct ProviderToolCall: Encodable {
    struct Function: Encodable {
        let name: String
        let arguments: String
    }

    let id: String
    let type = "function"
    let function: Function

    init(id: String, function: Function) {
        self.id = id
        self.function = function
    }

    init(_ call: ToolCall) {
        id = call.callID.rawValue
        function = Function(name: call.toolID.rawValue, arguments: call.arguments)
    }
}

struct SSEChunk: Decodable {
    struct Choice: Decodable {
        struct Delta: Decodable {
            let content: String?
            // Compatible providers use both names for streamed reasoning.
            let reasoning: String?
            let toolCalls: [SSEToolCall]?

            init(from decoder: Decoder) throws {

```

```

        let container = try decoder.container(keyedBy: CodingKeys.self)
        content = try container.decodeIfPresent(String.self, forKey: .content)
        reasoning = try container.decodeIfPresent(String.self, forKey: .reasoning)
        ?? container.decodeIfPresent(String.self, forKey: .reasoningContent)
        toolCalls = try container.decodeIfPresent([SSEToolCall].self, forKey: .toolCalls)
    }

    enum CodingKeys: String, CodingKey {
        case content
        case reasoning
        case reasoningContent = "reasoning_content"
        case toolCalls = "tool_calls"
    }
}

let delta: Delta?
let finishReason: ModelFinishReason?

init(from decoder: Decoder) throws {
    let container = try decoder.container(keyedBy: CodingKeys.self)
    delta = try container.decodeIfPresent(Delta.self, forKey: .delta)
    let raw = try container.decodeIfPresent(String.self, forKey: .finishReason)
    ?? container.decodeIfPresent(String.self, forKey: .camelFinishReason)
    ?? container.decodeIfPresent(String.self, forKey: .stopReason)
    finishReason = OpenAICompatibleProvider.finishReason(
        fromOpenAI: raw
    )
}

enum CodingKeys: String, CodingKey {
    case delta
    case finishReason = "finish_reason"
    case camelFinishReason = "finishReason"
    case stopReason = "stop_reason"
}

let choices: [Choice]?
let usage: SSEUsage?
let finishReason: ModelFinishReason?

enum CodingKeys: String, CodingKey {
    case choices, usage
    case finishReason = "finish_reason"
    case camelFinishReason = "finishReason"
    case stopReason = "stop_reason"
}

init(from decoder: Decoder) throws {
    let container = try decoder.container(keyedBy: CodingKeys.self)
    choices = try container.decodeIfPresent([Choice].self, forKey: .choices)
    usage = try container.decodeIfPresent(SSEUsage.self, forKey: .usage)
    let raw = try container.decodeIfPresent(String.self, forKey: .finishReason)
    ?? container.decodeIfPresent(String.self, forKey: .camelFinishReason)
    ?? container.decodeIfPresent(String.self, forKey: .stopReason)
    finishReason = OpenAICompatibleProvider.finishReason(fromOpenAI: raw)
}
}

struct SSEUsage: Decodable {
    struct CompletionDetails: Decodable {
        let reasoningTokens: Int?
        enum CodingKeys: String, CodingKey {
            case reasoningTokens = "reasoning_tokens"
        }
    }
}

```

```

}

struct PromptDetails: Decodable {
    let cachedTokens: Int?
    let cachedPromptTokens: Int?
    let cacheReadTokens: Int?
    enum CodingKeys: String, CodingKey {
        case cachedTokens = "cached_tokens"
        case cachedPromptTokens = "cached_prompt_tokens"
        case cacheReadTokens = "cache_read_tokens"
    }
}

let promptTokens: Int?
let completionTokens: Int?
let completionTokensDetails: CompletionDetails?
let promptTokensDetails: PromptDetails?
let promptTokenDetails: PromptDetails?
let inputTokensDetails: PromptDetails?
let inputTokenDetails: PromptDetails?
let cachedTokens: Int?
let cachedPromptTokens: Int?
let cacheReadTokens: Int?
let promptCacheHitTokens: Int?
let promptCacheMissTokens: Int?
let cacheReadInputTokens: Int?
let cacheCreationInputTokens: Int?

enum CodingKeys: String, CodingKey {
    case promptTokens = "prompt_tokens"
    case completionTokens = "completion_tokens"
    case completionTokensDetails = "completion_tokens_details"
    case promptTokensDetails = "prompt_tokens_details"
    case promptTokenDetails = "prompt_token_details"
    case inputTokensDetails = "input_tokens_details"
    case inputTokenDetails = "input_token_details"
    case cachedTokens = "cached_tokens"
    case cachedPromptTokens = "cached_prompt_tokens"
    case cacheReadTokens = "cache_read_tokens"
    case promptCacheHitTokens = "prompt_cache_hit_tokens"
    case promptCacheMissTokens = "prompt_cache_miss_tokens"
    case cacheReadInputTokens = "cache_read_input_tokens"
    case cacheCreationInputTokens = "cache_creation_input_tokens"
}

}

struct SSEToolCall: Decodable {
    struct Function: Decodable {
        let name: String?
        let arguments: String?

        init(name: String? = nil, arguments: String? = nil) {
            self.name = name
            self.arguments = arguments
        }
    }
}

let index: Int
let id: String?
let function: Function?

init(index: Int, id: String? = nil, function: Function? = nil) {
    self.index = index
    self.id = id
    self.function = function
}

```

```

    }
}

struct ToolCallBuffer {
    private struct Partial {
        var id: String?
        var name: String?
        var arguments = ""
        var started = false
        var emittedArgumentCount = 0
    }

    private var calls: [Int: Partial] = [:]
    private var domainIDs: [Int: ToolCallID] = [:]
    private let requestID: ModelRequestID
    private let debugStep: Int?
    private let diagnosticsEnabled: Bool
    private(set) var references: [ProviderToolCallReference] = []

    init(requestID: ModelRequestID, debugStep: Int?, diagnosticsEnabled: Bool) {
        self.requestID = requestID
        self.debugStep = debugStep
        self.diagnosticsEnabled = diagnosticsEnabled
    }

    mutating func consume(_ deltas: [SSEToolCall]) throws -> [ModelEvent] {
        var events: [ModelEvent] = []
        for delta in deltas {
            var partial = calls[delta.index] ?? Partial()
            if let id = delta.id, !id.isEmpty {
                partial.id = id
            }
            if let name = delta.function?.name, !name.isEmpty {
                if partial.name == nil {
                    partial.name = name
                } else {
                    partial.name? += name
                }
            }
            if let arguments = delta.function?.arguments, !arguments.isEmpty {
                partial.arguments += arguments
            }
            if let id = partial.id, !id.isEmpty, let rawName = partial.name?.trimmingCharacters(in: .whitespacesAndNewlines), !rawName.isEmpty, !partial.started {
                let domainID = ToolCallID("lingxi:\(requestID.rawValue):\(\(delta.index))")
                domainIDs[delta.index] = domainID
                references.append(ProviderToolCallReference(wire: .chatCompletions, domainCallID: domainID, externalCallID: id))
                partial.started = true
                events.append(.toolCallStarted(callID: domainID, toolID: ToolID(rawName)))
            }
            if partial.started, let domainID = domainIDs[delta.index], partial.emittedArgumentCount < partial.arguments.count {
                let arguments = String(partial.arguments.dropFirst(partial.emittedArgumentCount))
                partial.emittedArgumentCount = partial.arguments.count
                events.append(.toolCallDelta(callID: domainID, arguments: arguments))
            }
            calls[delta.index] = partial
        }
        return events
    }

    mutating func complete() throws -> [ModelEvent] {
        defer { calls.removeAll() }
        return try calls.keys.sorted().map { index in
            var call = calls[index] ?? Partial()
            if let id = call.id, !id.isEmpty, let rawName = call.name?.trimmingCharacters(in: .whitespacesAndNewlines), !rawName.isEmpty

```

```

↪ , !call.started {
    let domainID = ToolCallID("lingxi:\(requestID.rawValue):\((index)")
    domainIDs[index] = domainID
    references.append(ProviderToolCallReference(wire: .chatCompletions, domainCallID: domainID, externalCallID: id))
    call.started = true
}
guard let domainID = domainIDs[index], let name = call.name?.trimmingCharacters(in: .whitespacesAndNewlines), !name.isEmpty,
↪ call.started else {
    throw CoreError(code: .modelStream, message: "Tool Call 信息不完整: tool name 为空")
}
guard name != "Tool" && name != "{}" && name != "unknown" else {
    throw CoreError(code: .modelStream, message: "模型返回了无效的 Tool Name: '\(name)'" )
}
guard let data = call.arguments.data(using: .utf8),
    (try? JSONSerialization.jsonObject(with: data)) is [String: Any]
else {
    throw CoreError(code: .modelStream, message: "Tool Call 参数不是 JSON object")
}
if diagnosticsEnabled {
    let hash = call.arguments.utf8.reduce(UInt64(1469598103934665603)) { ($0 ^ UInt64($1)) &* 1099511628211 }
    FileHandle.standardError.write(Data("[tool-debug] step=\(debugStep ?? 0) index=\(index) argsHash=\(String(hash, radix:
↪ 16))\n".utf8))
}
return .toolCallCompleted(ToolCall(callID: domainID, toolID: ToolID(name), arguments: call.arguments))
}
}
}
}
}

```

// File: Sources/LingXiCore/Modules/Model/OpenAIResponsesProvider.swift

```

import Foundation
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

/// OpenAI Responses API adapter. Its DTOs never leave this file.
public struct OpenAIResponsesProvider: ModelProvider {
    private let config: ProviderConfig
    private let transport: any ProviderHTTPTransport
    private let provenance: ProviderProvenanceStore

    public init(config: ProviderConfig, session: URLSession = .shared, provenance: ProviderProvenanceStore = ProviderProvenanceStore()) {
        self.config = config
        transport = URLSessionProviderHTTPTransport(session: session)
        self.provenance = provenance
    }

    public init(config: ProviderConfig, transport: any ProviderHTTPTransport, provenance: ProviderProvenanceStore = ProviderProvenanceStore
↪ ()) {
        self.config = config
        self.transport = transport
        self.provenance = provenance
    }

    public func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        let prior = try await provenance.resolveContinuation(for: request.continuationOf, wire: .responses)
        let previousResponseID = config.remoteStateEnabled ? prior?.responseID : nil
        let urlRequest = try makeURLRequest(request, continuation: prior, previousResponseID: previousResponseID)
        let startedAt = Date()
        Self.logRequest(urlRequest, step: request.debugStep, localContinuation: prior != nil, enabled: config.diagnosticsEnabled)
        let response = try await transport.send(urlRequest, context: ProviderHTTPRequestContext(wireProtocol: .responses, model: request.
↪ model.rawValue, requestID: request.requestID, executionID: request.executionID, step: request.debugStep ?? 0))
        let requestID = response.header("x-request-id")
    }
}

```

```

        ?? response.header("openai-request-id")
        ?? response.header("cf-ray")
    Self.log("http step=\(request.debugStep ?? 0) status=\(response.statusCode) requestId=\(requestID ?? "none") elapsedMs=\(Self.
    ↪ elapsed(since: startedAt))", enabled: config.diagnosticsEnabled)
    guard (200..<300).contains(response.statusCode) else {
        let body = (try? await OpenAICompatibleProvider.collectText(response.body)) ?? ""
        Self.logHTTPError(body, step: request.debugStep, status: response.statusCode, requestId: requestID, enabled: config.
    ↪ diagnosticsEnabled)
        let requestFields: [String]
        if let reqBody = urlRequest.httpBody, let json = (try? JSONSerialization.jsonObject(with: reqBody)) as? [String: Any] {
            requestFields = json.keys.sorted()
        } else {
            requestFields = []
        }
        let toolSchemas = request.tools.map { "\($0.name)\($0.inputSchema.required.joined(separator: ", "))" }
        let capabilityProjection = "toolsRequested=\(!request.tools.isEmpty) (count=\(request.tools.count)), reasoningRequested=\(
    ↪ request.reasoning != nil)"
        let diagnostics = OpenAICompatibleProvider.makeWireDiagnostics(
            provider: config.baseURL.host ?? "openai",
            model: request.model.rawValue,
            wireAdapter: config.wireProtocol.rawValue,
            url: urlRequest.url?.absoluteString ?? config.responsesURL.absoluteString,
            httpMethod: urlRequest.httpMethod ?? "POST",
            requestFields: requestFields,
            toolSchemas: toolSchemas,
            capabilityProjection: capabilityProjection,
            responseBody: body
        )
        let error = OpenAICompatibleProvider.httpError(status: response.statusCode, requestId: requestID, diagnostics: diagnostics)
        throw ProviderRateLimitError.from(statusCode: response.statusCode, headers: response.headers, body: body, underlying: error)
    }

    let events = AsyncThrowingStream<ModelEvent, Error> { continuation in
        if let requestID = ProviderTraceSanitizer.requestID(requestID) {
            continuation.yield(.providerRequestID(requestID))
        }
        let pump = Task {
            await Pump(source: response.body, continuation: continuation, request: request, prior: prior, step: request.debugStep,
    ↪ providerRequestID: requestID, startedAt: startedAt, provenance: provenance, sensitiveValues: Self.sensitiveValues(config.
    ↪ authentication), diagnosticsEnabled: config.diagnosticsEnabled).run()
        }
        continuation.onTermination = { @Sendable _ in
            pump.cancel()
        }
    }
    return events
}

public func endExecution(_ executionID: AgentRunID) async {
    await provenance.remove(executionID)
}

public func makeURLRequest(_ request: ModelRequest, previousResponseID: String? = nil) throws -> URLRequest {
    try makeURLRequest(request, continuation: nil, previousResponseID: previousResponseID)
}

private func makeURLRequest(_ request: ModelRequest, continuation: ProviderContinuation?, previousResponseID: String?) throws ->
    ↪ URLRequest {
    var urlRequest = URLRequest(url: config.responsesURL)
    urlRequest.httpMethod = "POST"
    if let timeout = request.overallTimeoutSeconds { urlRequest.timeoutInterval = timeout }
    urlRequest.setValue("application/json", forHTTPHeaderField: "Content-Type")
    switch config.authentication {
    case .none: break
    case let .bearer(secret): urlRequest.setValue("Bearer \(secret)", forHTTPHeaderField: "Authorization")
    }
}

```



```

    case let .header(name, value): urlRequest.setValue(value, forHTTPHeaderField: name)
  }
  for (name, value) in config.requiredHeaders {
    urlRequest.setValue(value, forHTTPHeaderField: name)
  }
  let isCodexBackend = config.baseURL.host?.contains("chatgpt.com") == true
  let effectiveStore = isCodexBackend ? false : config.remoteStateEnabled
  urlRequest.httpBody = try Self.makeRequestBody(
    request,
    continuation: continuation,
    previousResponseID: effectiveStore ? previousResponseID : nil,
    store: effectiveStore
  )
  return urlRequest
}

/// Domain request -> Responses API JSON.
public static func makeRequestBody(_ request: ModelRequest) throws -> Data {
  try makeRequestBody(request, continuation: nil, previousResponseID: nil, store: false)
}

private static func makeRequestBody(_ request: ModelRequest, continuation: ProviderContinuation?, previousResponseID: String?, store:
  ⇨ Bool) throws -> Data {
  let messages = previousResponseID == nil ? request.messages : continuationMessages(request)
  let input = messages.flatMap { message -> [ResponseRequestBody.Input] in
    let calls = message.parts.compactMap { if case let .toolCall(call) = $0 { call } else { nil } }
    let results = message.parts.compactMap { if case let .toolResult(result) = $0 { result } else { nil } }
    switch message.role {
    case .tool:
      return results.map {
        let projected = ModelToolResultProjection.project($0)
        return .functionOutput(callID: continuation?.externalCallID(for: projected.callID) ?? projected.callID.rawValue, output:
  ⇨ projected.content)
      }
    case .assistant:
      var items: [ResponseRequestBody.Input] = []
      if !message.content.isEmpty { items.append(.message(role: "assistant", content: message.content)) }
      let byID = Dictionary(uniqueKeysWithValues: calls.map { ($0.callID, $0) })
      let ordered = continuation?.orderedItems ?? []
      if request.reasoning != nil, !ordered.isEmpty, ordered.contains(where: { if case let .toolCall(id) = $0 { byID[id] != nil }
  ⇨ else { false } }) {
        for item in ordered {
          switch item {
          case let .opaque(data): items.append(.opaque(data))
          case let .toolCall(id):
            guard let call = byID[id] else { continue }
            items.append(.functionCall(callID: continuation?.externalCallID(for: id) ?? id.rawValue, name: call.toolID.
  ⇨ rawValue, arguments: call.arguments))
          }
        }
      } else {
        items.append(contentsOf: calls.map { .functionCall(callID: continuation?.externalCallID(for: $0.callID) ?? $0.callID.
  ⇨ rawValue, name: $0.toolID.rawValue, arguments: $0.arguments) })
      }
      return items
    case .system:
      return [.message(role: "developer", content: message.content)]
    case .user:
      return [.message(role: "user", content: message.content)]
    }
  }
  let orderedTools: [ToolDefinition]
  let instructions: String?
  if let plan = request.cachePlan {
    instructions = plan.immutableBase.systemPrompt ?? request.system
  }

```

```

        orderedTools = plan.immutableBase.coreTools + plan.appendOnlyContext.dynamicTools
    } else {
        instructions = request.system
        let coreIDs = ToolRuntime.coreToolIDs
        let core = request.tools.filter { coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        let dynamic = request.tools.filter { !coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        orderedTools = core + dynamic
    }
    let encoder = JSONEncoder()
    encoder.outputFormatting = [.sortedKeys]
    return try encoder.encode(ResponseRequestBody(
        model: request.model.rawValue,
        stream: true,
        store: store,
        instructions: instructions,
        input: input,
        tools: orderedTools.isEmpty ? nil : orderedTools.map(ResponseRequestBody.Tool.init),
        reasoning: request.reasoning.map { ResponseRequestBody.Reasoning(effort: $0) },
        include: !store && request.reasoning != nil ? ["reasoning.encrypted_content"] : nil,
        previousResponseID: previousResponseID
    ))
}

public static func events(forSSEPayload payload: String, decoder: inout ResponsesSSEDecoder) throws -> [ModelEvent] {
    try decoder.consume(payload)
}

private static func continuationMessages(_ request: ModelRequest) -> [ModelMessage] {
    if let callIndex = request.messages.lastIndex(where: { message in
        message.role == .assistant && message.parts.contains { if case .toolCall = $0 { true } else { false } }
    }) {
        return Array(request.messages.suffix(from: request.messages.index(after: callIndex)))
    }
    let lastUser = request.messages.lastIndex { $0.role == .user }
    guard let toolIndex = request.messages.lastIndex(where: { $0.role == .tool }), lastUser == nil || toolIndex > lastUser! else {
        ↪ return [] }
    return [request.messages[toolIndex]]
}

private static func logRequest(_ request: URLRequest, step: Int?, localContinuation: Bool, enabled: Bool) {
    guard enabled, let body = request.httpBody,
        let object = try? JSONSerialization.jsonObject(with: body) as? [String: Any]
    else { return }
    let input = object["input"] as? [[String: Any]] ?? []
    let outputCount = input.filter { $0["type"] as? String == "function_call_output" }.count
    let callCount = input.filter { $0["type"] as? String == "function_call" }.count
    let toolCount = (object["tools"] as? [[String: Any]])?.count ?? 0
    let remoteContinuation = object["previous_response_id"] != nil
    log("request step=\(step ?? 0) wire=responses localContinuation=\(localContinuation) remoteContinuation=\(remoteContinuation)
    ↪ inputItems=\(input.count) functionCalls=\(callCount) functionOutputs=\(outputCount) tools=\(toolCount) bytes=\(body.count)
    ↪ timeoutSec=\(Int(request.timeoutInterval))", enabled: enabled)
}

private static func logHTTPError(_ body: String, step: Int?, status: Int, requestID: String?, enabled: Bool) {
    log("httpError step=\(step ?? 0) status=\(status) requestID=\(requestID ?? "none") body=\(sanitizedProviderToken(body, limit: 300)
    ↪ ?? "none")", enabled: enabled)
}

private static func elapsed(since date: Date) -> Int {
    Int(Date().timeIntervalSince(date) * 1000)
}

private static func sensitiveValues(_ authentication: ProviderAuthentication) -> [String] {
    switch authentication {
    case .none: []

```

```

        case let .bearer(value), let .header(_, value): [value]
    }
}

private static func log(_ message: String, enabled: Bool) {
    guard enabled else { return }
    FileHandle.standardError.write(Data(["responses-diagnostic"] \ (message)\n").utf8))
}

private final class Pump: Sendable {
    private let source: AsyncThrowingStream<Data, Error>
    private let continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation
    private let request: ModelRequest
    private let prior: ProviderContinuation?
    private let step: Int?
    private let providerRequestID: String?
    private let startedAt: Date
    private let provenance: ProviderProvenanceStore
    private let sensitiveValues: [String]
    private let diagnosticsEnabled: Bool

    init(source: AsyncThrowingStream<Data, Error>, continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation, request:
↳ ModelRequest, prior: ProviderContinuation?, step: Int?, providerRequestID: String?, startedAt: Date, provenance:
↳ ProviderProvenanceStore, sensitiveValues: [String], diagnosticsEnabled: Bool) {
        self.source = source
        self.continuation = continuation
        self.request = request
        self.prior = prior
        self.step = step
        self.providerRequestID = providerRequestID
        self.startedAt = startedAt
        self.provenance = provenance
        self.sensitiveValues = sensitiveValues
        self.diagnosticsEnabled = diagnosticsEnabled
    }

    func run() async {
        var lines = SSEDecoder()
        var decoder = ResponsesSSEDecoder(requestID: request.requestID, sensitiveValues: sensitiveValues)
        var completed: ModelFinishReason?
        var lastEventType = "none"
        var terminalObserved = false
        var failureObserved = false
        var responseID: String?
        continuation.yield(.started)
        do {
            outer: for try await chunk in source {
                for line in lines.feed(chunk) {
                    if try emit(line, decoder: &decoder, completed: &completed, lastEventType: &lastEventType, terminalObserved: &
↳ terminalObserved, failureObserved: &failureObserved, responseID: &responseID) { break outer }
                }
            }
            if !terminalObserved {
                for line in lines.flushPending() {
                    _ = try emit(line, decoder: &decoder, completed: &completed, lastEventType: &lastEventType, terminalObserved: &
↳ terminalObserved, failureObserved: &failureObserved, responseID: &responseID)
                }
            }
            if Task.isCancelled { continuation.finish(); return }
            if failureObserved { continuation.finish(); return }
            guard terminalObserved else { throw CoreError(code: .modelStream, message: "Responses SSE 意外结束") }
            let combinedReferences = (prior?.references ?? []).filter { priorRef in !decoder.references.contains { $0.domainCallID ==
↳ priorRef.domainCallID } } + decoder.references
            if responseID != nil || !combinedReferences.isEmpty || !decoder.orderedItems.isEmpty {
                try await provenance.record(ProviderContinuation(requestID: request.requestID, executionID: request.executionID, wire: .
↳ responses, responseID: responseID, references: combinedReferences, orderedItems: decoder.orderedItems))
            }
        }
    }
}

```

```

    }
    OpenAIResponsesProvider.log("eof step=\(step ?? 0) requestId=\(providerRequestID ?? "none") responseIDPresent=\(responseID
↪ != nil) lastEvent=\(lastEventType) terminal=\(terminalObserved) elapsedMs=\(OpenAIResponsesProvider.elapsed(since: startedAt))",
↪ enabled: diagnosticsEnabled)
    continuation.yield(.completed(completed ?? .unknown))
    continuation.finish()
  } catch let error as CoreError {
    if Task.isCancelled { continuation.finish(); return }
    OpenAIResponsesProvider.log("failed step=\(step ?? 0) requestId=\(providerRequestID ?? "none") responseIDPresent=\(
↪ responseID != nil) lastEvent=\(lastEventType) terminal=\(terminalObserved) elapsedMs=\(OpenAIResponsesProvider.elapsed(since:
↪ startedAt)) code=\(error.code.rawValue)", enabled: diagnosticsEnabled)
    continuation.yield(.failed(error))
    continuation.finish()
  } catch {
    if Task.isCancelled { continuation.finish(); return }
    OpenAIResponsesProvider.log("failed step=\(step ?? 0) requestId=\(providerRequestID ?? "none") responseIDPresent=\(
↪ responseID != nil) lastEvent=\(lastEventType) terminal=\(terminalObserved) elapsedMs=\(OpenAIResponsesProvider.elapsed(since:
↪ startedAt)) code=transport", enabled: diagnosticsEnabled)
    continuation.yield(.failed(CoreError(code: .modelStream, message: "Responses SSE 连接中断")))
    continuation.finish()
  }
}

private func emit(_ line: String, decoder: inout ResponsesSSEDecoder, completed: inout ModelFinishReason?, lastEventType: inout
↪ String, terminalObserved: inout Bool, failureObserved: inout Bool, responseID: inout String?) throws -> Bool {
  let trimmed = line.trimmingCharacters(in: .whitespaces)
  guard trimmed.hasPrefix("data:") else { return false }
  let payload = String(trimmed.dropFirst("data:".count)).trimmingCharacters(in: .whitespaces)
  guard payload != "[DONE]" else { return true }
  if let data = payload.data(using: .utf8), let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any], let type
↪ = object["type"] as? String {
    lastEventType = sanitizedProviderToken(type, limit: 120) ?? "unknown"
    terminalObserved = terminalObserved || ["response.completed", "response.incomplete", "response.failed", "error"].contains(
↪ type)
    if let response = object["response"] as? [String: Any], let id = response["id"] as? String { responseID = id }
    OpenAIResponsesProvider.log("event step=\(step ?? 0) requestId=\(providerRequestID ?? "none") responseIDPresent=\(responseID
↪ != nil) type=\(lastEventType) elapsedMs=\(OpenAIResponsesProvider.elapsed(since: startedAt))", enabled: diagnosticsEnabled)
    if let failure = ResponsesProviderFailure(eventType: type, object: object, sensitiveValues: sensitiveValues) {
      OpenAIResponsesProvider.log("providerError step=\(step ?? 0) requestId=\(providerRequestID ?? "none") event=\(failure.
↪ eventType) code=\(failure.code ?? "none") param=\(failure.param ?? "none") message=\(failure.diagnosticMessage)", enabled:
↪ diagnosticsEnabled)
    }
  }
}

for event in try decoder.consume(payload) {
  if case let .completed(reason) = event { completed = reason }
  else if case .failed = event { failureObserved = true; continuation.yield(event) }
  else { continuation.yield(event) }
}
return terminalObserved
}

}

}

/// Explicit stream state machine for OpenAI Responses protocol.
public enum ResponsesStreamState: Sendable, Equatable {
  case idle
  case inProgress(responseID: String?)
  case inOutputItem(outputIndex: Int, itemType: String, itemID: String?)
  case streamingText(outputIndex: Int, contentIndex: Int)
  case streamingToolArguments(outputIndex: Int, callID: String)
  case completed(status: String)
  case failed(error: String)
}

```

```

public struct ResponsesStreamStateMachine: Sendable {
    public private(set) var state: ResponsesStreamState = .idle
    public private(set) var responseID: String?
    public private(set) var completedCallIDs: [String] = []
    public private(set) var references: [ProviderToolCallReference] = []

    private struct PartialCall {
        var name: String?
        var arguments = ""
        var started = false
        var emitted = 0
        var completed = false
    }

    private var calls: [String: PartialCall] = [:]
    private var itemToCallID: [String: String] = [:]
    private var callToItemID: [String: String] = [:]
    private var orderedKeys: [String] = []
    private var opaqueItems: [String: Data] = [:]
    private var domainIDs: [String: ToolCallID] = [:]
    private let requestID: ModelRequestID
    private let sensitiveValues: [String]

    public var orderedItems: [ProviderContinuationItem] {
        orderedKeys.compactMap { key in
            if key.hasPrefix("opaque:"), let data = opaqueItems[key] { return .opaque(data) }
            if key.hasPrefix("call:"), let id = domainIDs[String(key.dropFirst("call:".count))] { return .toolCall(id) }
            return nil
        }
    }

    public init(requestID: ModelRequestID = ModelRequestID(), sensitiveValues: [String] = []) {
        self.requestID = requestID
        self.sensitiveValues = sensitiveValues
    }

    public mutating func feedEvent(type: String, object: [String: Any]) throws -> [ModelEvent] {
        switch (state, type) {
            // 1. response.created & response.in_progress
            case (_, "response.created"), (_, "response.in_progress"):
                if let response = object["response"] as? [String: Any], let id = response["id"] as? String {
                    self.responseID = id
                }
                state = .inProgress(responseID: self.responseID)
                return []

            // 2. output_item.added
            case (_, "response.output_item.added"):
                let outputIndex = object["output_index"] as? Int ?? 0
                guard let item = object["item"] as? [String: Any], let itemType = item["type"] as? String else { return [] }
                let itemID = item["id"] as? String
                state = .inOutputItem(outputIndex: outputIndex, itemType: itemType, itemID: itemID)
                if itemType == "reasoning", let itemID {
                    let key = "opaque:\(itemID)"
                    if !orderedKeys.contains(key) { orderedKeys.append(key) }
                    opaqueItems[key] = try JSONSerialization.data(withJSONObject: item, options: [.sortedKeys])
                    return []
                }
                if itemType == "function_call" {
                    if let itemID, let callID = item["call_id"] as? String {
                        itemToCallID[itemID] = callID
                        callToItemID[callID] = itemID
                    }
                    return try updateCall(callID: extractCallID(in: item), name: item["name"] as? String, arguments: item["arguments"] as?
↪ String, finish: false)

```

```

    }
    return []

// 3. content_part.added
case (_, "response.content_part.added"):
    let outputIndex = object["output_index"] as? Int ?? 0
    let contentIndex = object["content_index"] as? Int ?? 0
    state = .streamingText(outputIndex: outputIndex, contentIndex: contentIndex)
    return []

// 4. output_text.delta
case (_, "response.output_text.delta"):
    let outputIndex = object["output_index"] as? Int ?? 0
    let contentIndex = object["content_index"] as? Int ?? 0
    state = .streamingText(outputIndex: outputIndex, contentIndex: contentIndex)
    if let delta = object["delta"] as? String {
        return [.textDelta(delta)]
    }
    return []

// 5. output_text.done & content_part.done
case (_, "response.output_text.done"), (_, "response.content_part.done"):
    state = .inProgress(responseID: self.responseID)
    return []

// 6. reasoning / reasoning_summary delta
case (_, "response.reasoning_summary_text.delta"), (_, "response.reasoning.delta"):
    if let delta = object["delta"] as? String {
        return [.reasoningDelta(delta)]
    }
    return []

// 7. function_call_arguments.delta
case (_, "response.function_call_arguments.delta"):
    let outputIndex = object["output_index"] as? Int ?? 0
    let callID = extractCallID(in: object)
    if let callID {
        state = .streamingToolArguments(outputIndex: outputIndex, callID: callID)
    }
    return try updateCall(callID: callID, name: object["name"] as? String, arguments: object["delta"] as? String, finish: false)

// 8. function_call_arguments.done
case (_, "response.function_call_arguments.done"):
    let callID = extractCallID(in: object)
    return try updateCall(callID: callID, name: object["name"] as? String, arguments: object["arguments"] as? String, finish: true)

// 9. output_item.done
case (_, "response.output_item.done"):
    state = .inProgress(responseID: self.responseID)
    guard let item = object["item"] as? [String: Any], let itemType = item["type"] as? String else { return [] }
    if itemType == "function_call" {
        if let itemID = item["id"] as? String, let callID = item["call_id"] as? String {
            itemToCallID[itemID] = callID
            callToItemID[callID] = itemID
        }
        return try updateCall(callID: extractCallID(in: item), name: item["name"] as? String, arguments: item["arguments"] as?
↵ String, finish: true)
    }
    if itemType == "reasoning", let itemID = item["id"] as? String {
        let key = "opaque:\(itemID)"
        if !orderedKeys.contains(key) { orderedKeys.append(key) }
        opaqueItems[key] = try JSONSerialization.data(withJSONObject: item, options: [.sortedKeys])
        return []
    }
    return []

```

```

// 10. response.completed
case (_, "response.completed"):
    state = .completed(status: "completed")
    var events: [ModelEvent] = []
    if let response = object["response"] as? [String: Any] {
        if let usage = response["usage"] as? [String: Any] {
            events.append(.usage(Self.parseUsage(usage)))
        }
        let finishReason: ModelFinishReason = completedCallIDs.isEmpty ? .stop : .toolCalls
        events.append(.completed(finishReason))
    } else {
        events.append(.completed(.unknown))
    }
    return events

// 11. response.incomplete
case (_, "response.incomplete"):
    state = .completed(status: "incomplete")
    var events: [ModelEvent] = []
    if let response = object["response"] as? [String: Any], let usage = response["usage"] as? [String: Any] {
        events.append(.usage(Self.parseUsage(usage)))
    }
    events.append(.completed(.maxTokens))
    return events

// 12. response.failed / error
case (_, "response.failed"), (_, "error"):
    let failure = ResponsesProviderFailure(eventType: type, object: object, sensitiveValues: sensitiveValues)
    let errorMsg = failure?.diagnosticMessage ?? "Responses error event=\\(type)"
    state = .failed(error: errorMsg)
    return [.failed(failure?.coreError ?? CoreError(code: .modelStream, message: errorMsg))]

default:
    return []
}
}

private mutating func updateCall(callID: String?, name: String?, arguments: String?, finish: Bool) throws -> [ModelEvent] {
    guard let callID, !callID.isEmpty else { throw CoreError(code: .modelStream, message: "Responses function call 缺少 call_id") }
    let domainID = domainIDs[callID] ?? ToolCallID("lingxi:\\(requestID.rawValue):\\(domainIDs.count)")
    domainIDs[callID] = domainID
    let orderKey = "call:\\(callID)"
    if !orderedKeys.contains(orderKey) { orderedKeys.append(orderKey) }
    var call = calls[callID] ?? PartialCall()
    call.name = name ?? call.name
    if let arguments, !arguments.isEmpty {
        if finish || arguments.hasPrefix(call.arguments) { call.arguments = arguments }
        else { call.arguments += arguments }
    }
    var events: [ModelEvent] = []
    if let name = call.name, !call.started {
        call.started = true
        events.append(.toolCallStarted(callID: domainID, toolID: ToolID(name)))
    }
    if call.started, call.emitted < call.arguments.count {
        events.append(.toolCallDelta(callID: domainID, arguments: String(call.arguments.dropFirst(call.emitted))))
        call.emitted = call.arguments.count
    }
    if finish, !call.completed {
        guard let name = call.name else { throw CoreError(code: .modelStream, message: "Responses function call 缺少 name") }
        call.completed = true
        completedCallIDs.append(callID)
        references.append(ProviderToolCallReference(wire: .responses, domainCallID: domainID, externalCallID: callID, externalItemID:
↵ callToItemID[callID]))
    }
}

```

```

        events.append(.toolCallCompleted(ToolCall(callID: domainID, toolID: ToolID(name), arguments: call.arguments)))
    }
    calls[callID] = call
    return events
}

private func extractCallID(in object: [String: Any]) -> String? {
    (object["call_id"] as? String) ?? (object["item_id"] as? String).flatMap { itemToCallID[$0] }
}

private static func parseUsage(_ value: [String: Any]) -> ModelUsage {
    let outputDetails = (value["output_tokens_details"] as? [String: Any]) ?? (value["output_token_details"] as? [String: Any])
    let inputDetails = (value["input_tokens_details"] as? [String: Any])
        ?? (value["input_token_details"] as? [String: Any])
        ?? (value["prompt_tokens_details"] as? [String: Any])
        ?? (value["prompt_token_details"] as? [String: Any])
    let cached = (inputDetails?["cached_tokens"] as? Int)
        ?? (inputDetails?["cached_prompt_tokens"] as? Int)
        ?? (inputDetails?["cache_read_tokens"] as? Int)
        ?? (value["cached_tokens"] as? Int)
        ?? (value["cached_prompt_tokens"] as? Int)
        ?? (value["prompt_cache_hit_tokens"] as? Int)
        ?? (value["cache_read_input_tokens"] as? Int)
    return ModelUsage(
        inputTokens: (value["input_tokens"] as? Int) ?? (value["prompt_tokens"] as? Int),
        outputTokens: (value["output_tokens"] as? Int) ?? (value["completion_tokens"] as? Int),
        reasoningTokens: outputDetails?["reasoning_tokens"] as? Int,
        cacheReadTokens: cached,
        cacheWriteTokens: (value["cache_creation_input_tokens"] as? Int)
    )
}
}

/// Stateful Responses API decoder wrapper. Backed by ResponsesStreamStateMachine.
public struct ResponsesSSEDecoder {
    public var stateMachine: ResponsesStreamStateMachine

    public var completedCallIDs: [String] { stateMachine.completedCallIDs }
    public var references: [ProviderToolCallReference] { stateMachine.references }
    public var orderedItems: [ProviderContinuationItem] { stateMachine.orderedItems }
    public var responseID: String? { stateMachine.responseID }

    public init(requestID: ModelRequestID = ModelRequestID(), sensitiveValues: [String] = []) {
        stateMachine = ResponsesStreamStateMachine(requestID: requestID, sensitiveValues: sensitiveValues)
    }

    public mutating func consume(_ payload: String) throws -> [ModelEvent] {
        guard let data = payload.data(using: .utf8),
            let object = try JSONSerialization.jsonObject(with: data) as? [String: Any],
            let type = object["type"] as? String
        else { throw CoreError(code: .modelStream, message: "Responses SSE JSON 解析失败") }
        return try stateMachine.feedEvent(type: type, object: object)
    }

    public mutating func feedSSEText(_ text: String) throws -> [ModelEvent] {
        var events: [ModelEvent] = []
        var lines = SSEDecoder()
        if let data = text.data(using: .utf8) {
            for line in lines.feed(data) {
                let trimmed = line.trimmingCharacters(in: .whitespaces)
                guard trimmed.hasPrefix("data:") else { continue }
                let payload = String(trimmed.dropFirst("data:".count)).trimmingCharacters(in: .whitespaces)
                guard payload != "[DONE]" else { break }
                events.append(contentsOf: try consume(payload))
            }
        }
    }
}

```



```

        for line in lines.flushPending() {
            let trimmed = line.trimmingCharacters(in: .whitespaces)
            guard trimmed.hasPrefix("data:") else { continue }
            let payload = String(trimmed.dropFirst("data:".count)).trimmingCharacters(in: .whitespaces)
            guard payload != "[DONE]" else { break }
            events.append(contentsOf: try consume(payload))
        }
    }
    return events
}

private struct ResponsesProviderFailure {
    let eventType: String
    let code: String?
    let message: String
    let param: String?

    init?(eventType: String, object: [String: Any], sensitiveValues: [String]) {
        guard eventType == "response.failed" || eventType == "error" else { return nil }
        let details: [String: Any]
        if eventType == "response.failed" {
            details = ((object["response"] as? [String: Any])?["error"] as? [String: Any]) ?? (object["error"] as? [String: Any]) ?? [:]
        } else {
            details = (object["error"] as? [String: Any]) ?? object
        }
        self.eventType = sanitizedProviderToken(eventType, limit: 120) ?? "error"
        code = (details["code"] as? String).flatMap { sanitizedProviderToken($0, sensitiveValues: sensitiveValues, limit: 120) }
        param = (details["param"] as? String).flatMap { sanitizedProviderToken($0, sensitiveValues: sensitiveValues, limit: 160) }
        message = sanitizedProviderMessage(details["message"] as? String ?? "Provider returned an error without a message", sensitiveValues:
↳ sensitiveValues, limit: 1_024)
    }

    var coreError: CoreError {
        var fields = ["event=\(eventType)"]
        if let code { fields.append("code=\(code)") }
        if let param { fields.append("param=\(param)") }
        return CoreError(code: .modelStream, message: "Responses provider error \(fields.joined(separator: " ")): \(message)")
    }

    var diagnosticMessage: String { sanitizedProviderMessage(message, sensitiveValues: [], limit: 240) }
}

private func sanitizedProviderToken(_ value: String, sensitiveValues: [String] = [], limit: Int) -> String? {
    let sanitized = sanitizedProviderMessage(value, sensitiveValues: sensitiveValues, limit: limit)
        .map { $0.isLetter || $0.isNumber || "._:/[!\".contains($0) ? $0 : "_" }
    let result = String(sanitized)
    return result.isEmpty ? nil : result
}

private func sanitizedProviderMessage(_ value: String, sensitiveValues: [String], limit: Int) -> String {
    var result = value.replacingOccurrences(of: "\r", with: " ").replacingOccurrences(of: "\n", with: " ")
    for secret in sensitiveValues where !secret.isEmpty { result = result.replacingOccurrences(of: secret, with: "<redacted>") }
    let patterns = [
        #"(?i)bearer\s+[A-Za-z0-9._~+/-=]+"#,
        #"(?i)\(\"(?:authorization|api[_]?key|access[_]?token|client[_]?secret|encrypted_content|prompt|input)\"\\s*:\\s*\")\"(?:\\.|\"\\")"
↳ *\"\"#,
        #"(?i)\\b[A-Za-z0-9._-]*(?:secret|api[_]?key)[A-Za-z0-9._-]*\\b"#,
        #"\\bsk-[A-Za-z0-9_-]+\\b"#,
    ]
    for pattern in patterns {
        guard let regex = try? NSRegularExpression(pattern: pattern) else { continue }
        let range = NSRange(result.startIndex..., in: result)
        result = regex.stringByReplacingMatches(in: result, range: range, withTemplate: pattern.contains("authorization") ? "$1<redacted>"
↳ ">\"\" : "<redacted>")
    }
}

```

```

    }
    result = result.split(whereSeparator: { $0.isWhitespace }).joined(separator: " ")
    guard result.count > limit else { return result }
    return String(result.prefix(limit)) + "..."
}

private struct ResponseRequestBody: Encodable {
    struct Tool: Encodable {
        let type = "function"
        let name: String
        let description: String
        let parameters: JSONValue

        init(_ definition: ToolDefinition) {
            name = definition.name
            description = definition.description
            parameters = definition.rawInputSchema ?? .object([
                "type": .string("object"),
                "properties": .object(definition.inputSchema.properties.mapValues { property in
                    var value: [String: JSONValue] = ["type": .string(property.type.rawValue), "description": .string(property.description)]
                    if let values = property.enumValues { value["enum"] = .array(values.map(JSONValue.string)) }
                    if let minimum = property.minimum { value["minimum"] = .number(minimum) }
                    if let maximum = property.maximum { value["maximum"] = .number(maximum) }
                    return .object(value)
                }
            ]),
            "required": .array(definition.inputSchema.required.map(JSONValue.string)),
            "additionalProperties": .bool(false),
        )
    }
}

struct Reasoning: Encodable {
    let effort: String?
    let summary: String?
    init(effort: String? = nil, summary: String? = nil) {
        self.effort = effort
        self.summary = summary
    }
}

enum Input: Encodable {
    case message(role: String, content: String)
    case functionCall(callID: String, name: String, arguments: String, itemID: String? = nil)
    case functionOutput(callID: String, output: String)
    case opaque(Data)

    enum Keys: String, CodingKey { case type, role, content, callID = "call_id", name, arguments, output, id, status }

    func encode(to encoder: Encoder) throws {
        var values = encoder.container(keyedBy: Keys.self)
        switch self {
        case let .message(role, content):
            try values.encode(role, forKey: .role); try values.encode(content, forKey: .content)
        case let .functionCall(callID, name, arguments, itemID):
            try values.encode("function_call", forKey: .type)
            try values.encode(callID, forKey: .callID)
            try values.encode(name, forKey: .name)
            try values.encode(arguments, forKey: .arguments)
            if let itemID { try values.encode(itemID, forKey: .id) }
        case let .functionOutput(callID, output):
            try values.encode("function_call_output", forKey: .type); try values.encode(callID, forKey: .callID); try values.encode(
↳ output, forKey: .output)
        case let .opaque(data):
            try JSONDecoder().decode(JSONValue.self, from: data).encode(to: encoder)
        }
    }
}

```

```

    }
}

let model: String
let stream: Bool
let store: Bool
let instructions: String?
let input: [Input]
let tools: [Tool]?
let reasoning: Reasoning?
let include: [String]?
let previousResponseID: String?

enum CodingKeys: String, CodingKey {
    case model, stream, store, instructions, input, tools, reasoning, include
    case previousResponseID = "previous_response_id"
}
}

```

// File: Sources/LingXiCore/Modules/Model/AnthropicMessagesProvider.swift

```

import Foundation
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

/// Anthropic Messages adapter. Native wire identities and DTOs remain local to this file.
public struct AnthropicMessagesProvider: ModelProvider {
    private let config: ProviderConfig
    private let transport: any ProviderHTTPTransport
    private let provenance: ProviderProvenanceStore

    public init(config: ProviderConfig, session: URLSession = .shared, provenance: ProviderProvenanceStore = ProviderProvenanceStore()) {
        self.config = config
        transport = URLSessionProviderHTTPTransport(session: session)
        self.provenance = provenance
    }

    public init(config: ProviderConfig, transport: any ProviderHTTPTransport, provenance: ProviderProvenanceStore = ProviderProvenanceStore)
        ↪ () {
        self.config = config
        self.transport = transport
        self.provenance = provenance
    }

    public func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        let prior = try await provenance.resolveContinuation(for: request.continuationOf, wire: .anthropicMessages)
        let urlRequest = try makeURLRequest(request, continuation: prior)
        let response = try await transport.send(urlRequest, context: ProviderHTTPRequestContext(wireProtocol: .anthropicMessages, model:
        ↪ request.model.rawValue, requestID: request.requestID, executionID: request.executionID, step: request.debugStep ?? 0))
        let requestID = response.header("request-id")
        guard (200..<300).contains(response.statusCode) else {
            let body = (try? await OpenAICompatibleProvider.collectText(response.body)) ?? ""
            let requestFields: [String]
            if let reqBody = urlRequest.httpBody, let json = (try? JSONSerialization.jsonObject(with: reqBody)) as? [String: Any] {
                requestFields = json.keys.sorted()
            } else {
                requestFields = []
            }
            let toolSchemas = request.tools.map { "\($0.name)\($0.inputSchema.required.joined(separator: ", "))" }
            let capabilityProjection = "toolsRequested=\(!request.tools.isEmpty) (count=\(request.tools.count)), reasoningRequested=(
        ↪ request.reasoning != nil)"
            let diagnostics = OpenAICompatibleProvider.makeWireDiagnostics(
                provider: config.baseURL.host ?? "anthropic",

```

```

        model: request.model.rawValue,
        wireAdapter: config.wireProtocol.rawValue,
        url: urlRequest.url?.absoluteString ?? config.anthropicMessagesURL.absoluteString,
        httpMethod: urlRequest.httpMethod ?? "POST",
        requestFields: requestFields,
        toolSchemas: toolSchemas,
        capabilityProjection: capabilityProjection,
        responseBody: body
    )

    let error = OpenAICompatibleProvider.httpError(status: response.statusCode, requestID: requestID, diagnostics: diagnostics)
    throw ProviderRateLimitError.from(statusCode: response.statusCode, headers: response.headers, body: body, underlying: error)
}

return AsyncThrowingStream { continuation in
    if let requestID = ProviderTraceSanitizer.requestID(requestID) {
        continuation.yield(.providerRequestID(requestID))
    }

    let pump = Task { await Pump(source: response.body, continuation: continuation, request: request, prior: prior, provenance:
↪ provenance).run() }
    continuation.onTermination = { @Sendable _ in
        pump.cancel()
    }
}
}

public func endExecution(_ executionID: AgentRunID) async {
    await provenance.remove(executionID)
}

public func makeURLRequest(_ request: ModelRequest) throws -> URLRequest {
    try makeURLRequest(request, continuation: nil)
}

private func makeURLRequest(_ request: ModelRequest, continuation: ProviderContinuation?) throws -> URLRequest {
    guard request.reasoning == nil else {
        throw CoreError(code: .provider, message: "Anthropic reasoning configuration is not implemented")
    }

    var result = URLRequest(url: config.anthropicMessagesURL)
    result.httpMethod = "POST"
    if let timeout = request.overallTimeoutSeconds { result.timeoutInterval = timeout }
    result.setValue("application/json", forHTTPHeaderField: "Content-Type")
    result.setValue("2023-06-01", forHTTPHeaderField: "anthropic-version")
    switch config.authentication {
    case .none: break
    case let .bearer(secret): result.setValue("Bearer \(secret)", forHTTPHeaderField: "Authorization")
    case let .header(name, value): result.setValue(value, forHTTPHeaderField: name)
    }

    for (name, value) in config.requiredHeaders {
        result.setValue(value, forHTTPHeaderField: name)
    }

    result.httpBody = try Self.makeRequestBody(request, maxOutputTokens: config.maxOutputTokens ?? 4_096, continuation: continuation)
    return result
}

public static func makeRequestBody(_ request: ModelRequest, maxOutputTokens: Int = 4_096) throws -> Data {
    try makeRequestBody(request, maxOutputTokens: maxOutputTokens, continuation: nil)
}

private static func makeRequestBody(_ request: ModelRequest, maxOutputTokens: Int, continuation: ProviderContinuation?) throws -> Data {
    let orderedTools: [ToolDefinition]
    let system: String?
    if let plan = request.cachePlan {
        system = plan.immutableBase.systemPrompt
        orderedTools = plan.immutableBase.coreTools + plan.appendOnlyContext.dynamicTools
    } else {
        let messageSystem = request.messages.filter { $0.role == .system }.map(\.content).filter { !$0.isEmpty }

```

```

        system = ([request.system].compactMap { $0 } + messageSystem).joined(separator: "\n\n")
        let coreIDs = ToolRuntime.coreToolIDs
        let core = request.tools.filter { coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        let dynamic = request.tools.filter { !coreIDs.contains($0.id) }.sorted(by: { $0.id.rawValue < $1.id.rawValue })
        orderedTools = core + dynamic
    }

    let messages = try request.messages.compactMap { message -> RequestBody.Message? in
        switch message.role {
        case .system:
            return nil
        case .user:
            return RequestBody.Message(role: "user", content: message.parts.compactMap { part in
                if case let .text(text) = part { return .text(text) }
                return nil
            })
        case .assistant:
            return RequestBody.Message(role: "assistant", content: try message.parts.map { part in
                switch part {
                case let .text(text): return .text(text)
                case let .toolCall(call):
                    guard let data = call.arguments.data(using: .utf8),
                          let input = try? JSONDecoder().decode(JSONValue.self, from: data),
                          case .object = input
                    else { throw CoreError(code: .provider, message: "Anthropic tool input must be a JSON object") }
                    return .toolUse(id: continuation?.externalCallID(for: call.callID) ?? call.callID.rawValue, name: call.toolID,
↳ rawValue, input: input)
                case .toolResult:
                    throw CoreError(code: .provider, message: "Anthropic assistant message cannot contain tool results")
                }
            })
        case .tool:
            return RequestBody.Message(role: "user", content: message.parts.compactMap { part in
                guard case let .toolResult(result) = part else { return nil }
                let projected = ModelToolResultProjection.project(result)
                return .toolResult(id: continuation?.externalCallID(for: projected.callID) ?? projected.callID.rawValue, content:
↳ projected.content, isError: !projected.success)
            })
        }
    }

    let enableExplicitCache = request.cachePlan?.capabilities?.explicitCacheControlSupported ?? false
    let coreCount = request.cachePlan?.immutableBase.coreTools.count ?? 0
    let tools: [RequestBody.Tool]? = orderedTools.isEmpty ? nil : orderedTools.enumerated().map { index, def in
        let isBreakpoint = enableExplicitCache && index == (coreCount > 0 ? coreCount - 1 : 0)
        return RequestBody.Tool(def, cacheControl: isBreakpoint ? RequestBody.CacheControl() : nil)
    }

    let encoder = JSONEncoder()
    encoder.outputFormatting = [.sortedKeys]
    return try encoder.encode(RequestBody(
        model: request.model.rawValue,
        maxTokens: max(1, maxOutputTokens),
        stream: true,
        system: (system?.isEmpty ?? true) ? nil : system,
        messages: messages,
        tools: tools
    ))
}

private final class Pump: Sendable {
    private let source: AsyncThrowingStream<Data, Error>
    private let continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation
    private let request: ModelRequest
    private let prior: ProviderContinuation?

```

```

private let provenance: ProviderProvenanceStore

init(source: AsyncThrowingStream<Data, Error>, continuation: AsyncThrowingStream<ModelEvent, Error>.Continuation, request:
↳ ModelRequest, prior: ProviderContinuation?, provenance: ProviderProvenanceStore) {
    self.source = source
    self.continuation = continuation
    self.request = request
    self.prior = prior
    self.provenance = provenance
}

func run() async {
    var lines = SSEDecoder()
    var decoder = AnthropicSSEDecoder(requestID: request.requestID)
    var terminal = false
    var failed = false
    continuation.yield(.started)
    do {
        outer: for try await chunk in source {
            for line in lines.feed(chunk) {
                if try emit(line, decoder: &decoder, terminal: &terminal, failed: &failed) { break outer }
            }
        }
        if !terminal {
            for line in lines.flushPending() {
                _ = try emit(line, decoder: &decoder, terminal: &terminal, failed: &failed)
            }
        }
        if Task.isCancelled || failed { continuation.finish(); return }
        guard terminal else { throw CoreError(code: .modelStream, message: "Anthropic SSE ended unexpectedly") }
        let combinedReferences = (prior?.references ?? []).filter { priorRef in !decoder.references.contains { $0.domainCallID ==
↳ priorRef.domainCallID } } + decoder.references
        if !combinedReferences.isEmpty {
            try await provenance.record(ProviderContinuation(requestID: request.requestID, executionID: request.executionID, wire: .
↳ anthropicMessages, references: combinedReferences))
        }
        continuation.finish()
    } catch let error as CoreError {
        if Task.isCancelled { continuation.finish(); return }
        continuation.yield(.failed(error))
        continuation.finish()
    } catch {
        if Task.isCancelled { continuation.finish(); return }
        continuation.yield(.failed(CoreError(code: .modelStream, message: "Anthropic SSE connection interrupted")))
        continuation.finish()
    }
}

private func emit(_ line: String, decoder: inout AnthropicSSEDecoder, terminal: inout Bool, failed: inout Bool) throws -> Bool {
    let trimmed = line.trimmingCharacters(in: .whitespaces)
    guard trimmed.hasPrefix("data:") else { return false }
    let payload = String(trimmed.dropFirst("data:".count)).trimmingCharacters(in: .whitespaces)
    if payload == "[DONE]" { return false }
    if let data = payload.data(using: .utf8),
        let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
        let type = object["type"] as? String {
        terminal = terminal || type == "message_stop" || type == "error"
    }
    for event in try decoder.consume(payload) {
        if case .failed = event { failed = true }
        continuation.yield(event)
    }
    return terminal
}
}

```

```

}

public struct AnthropicSSEDecoder {
    private struct PartialTool {
        var externalID: String
        var domainID: ToolCallID
        var name: String
        var arguments = ""
    }

    private var tools: [Int: PartialTool] = [:]
    private let requestID: ModelRequestID
    private var pendingFinishReason: ModelFinishReason?
    public private(set) var references: [ProviderToolCallReference] = []

    public init(requestID: ModelRequestID = ModelRequestID()) { self.requestID = requestID }

    public mutating func consume(_ payload: String) throws -> [ModelEvent] {
        guard let data = payload.data(using: .utf8),
              let object = try JSONSerialization.jsonObject(with: data) as? [String: Any],
              let type = object["type"] as? String
        else { throw CoreError(code: .modelStream, message: "Anthropic SSE JSON parsing failed") }

        switch type {
        case "message_start":
            let message = object["message"] as? [String: Any]
            return usage(message?["usage"] as? [String: Any]).map { [.usage($0)] } ?? []
        case "content_block_start":
            guard let index = object["index"] as? Int, let block = object["content_block"] as? [String: Any] else { return [] }
            switch block["type"] as? String {
            case "text": return (block["text"] as? String).flatMap { $0.isEmpty ? nil : [.textDelta($0)] } ?? []
            case "thinking": return []
            case "tool_use":
                guard let id = block["id"] as? String, let name = block["name"] as? String else {
                    throw CoreError(code: .modelStream, message: "Anthropic tool use is missing identity")
                }
                let domainID = ToolCallID("lingxi:\(requestID.rawValue):\(\index)")
                tools[index] = PartialTool(externalID: id, domainID: domainID, name: name)
                return [.toolCallStarted(callID: domainID, toolID: ToolID(name))]
            default: return []
            }
        case "content_block_delta":
            guard let index = object["index"] as? Int, let delta = object["delta"] as? [String: Any] else { return [] }
            switch delta["type"] as? String {
            case "text_delta": return (delta["text"] as? String).map { [.textDelta($0)] } ?? []
            case "thinking_delta": return []
            case "input_json_delta":
                guard var tool = tools[index], let fragment = delta["partial_json"] as? String else { return [] }
                tool.arguments += fragment
                tools[index] = tool
                return [.toolCallDelta(callID: tool.domainID, arguments: fragment)]
            default: return []
            }
        case "content_block_stop":
            guard let index = object["index"] as? Int, let tool = tools.removeValue(forKey: index) else { return [] }
            guard let data = tool.arguments.data(using: .utf8),
                  (try? JSONSerialization.jsonObject(with: data)) is [String: Any]
            else { throw CoreError(code: .modelStream, message: "Anthropic tool input is not a JSON object") }
            references.append(ProviderToolCallReference(wire: .anthropicMessages, domainCallID: tool.domainID, externalCallID: tool.
↩ externalID))
            return [.toolCallCompleted(ToolCall(callID: tool.domainID, toolID: ToolID(tool.name), arguments: tool.arguments))]
        case "message_delta":
            var events: [ModelEvent] = []
            if let value = usage(object["usage"] as? [String: Any]) { events.append(.usage(value)) }
            let delta = object["delta"] as? [String: Any]

```

```

        if let reason = finishReason(delta?["stop_reason"] as? String) { pendingFinishReason = reason }
        return events
    case "message_stop":
        return [.completed(pendingFinishReason ?? .unknown)]
    case "error":
        return [.failed(CoreError(code: .modelStream, message: "Anthropic SSE returned an error"))]
    default:
        return []
    }
}

private func usage(_ raw: [String: Any]?) -> ModelUsage? {
    guard let raw else { return nil }
    return ModelUsage(
        inputTokens: raw["input_tokens"] as? Int,
        outputTokens: raw["output_tokens"] as? Int,
        reasoningTokens: nil,
        cacheReadTokens: raw["cache_read_input_tokens"] as? Int,
        cacheWriteTokens: raw["cache_creation_input_tokens"] as? Int
    )
}

private func finishReason(_ raw: String?) -> ModelFinishReason? {
    switch raw {
    case "end_turn", "stop_sequence": .stop
    case "max_tokens": .maxTokens
    case "tool_use": .toolCalls
    case "refusal": .contentFilter
    case nil, "": nil
    default: .unknown
    }
}

private struct RequestBody: Encodable {
    struct CacheControl: Encodable {
        let type: String
        init(type: String = "ephemeral") { self.type = type }
    }

    struct Message: Encodable { let role: String; let content: [Content] }
    struct Tool: Encodable {
        let name: String
        let description: String
        let inputSchema: JSONValue
        let cacheControl: CacheControl?

        init(_ definition: ToolDefinition, cacheControl: CacheControl? = nil) {
            name = definition.name
            description = definition.description
            inputSchema = definition.rawInputSchema ?? .object([
                "type": .string("object"),
                "properties": .object(definition.inputSchema.properties.mapValues { .object(["type": .string($0.type.rawValue), "description"
↪ ": .string($0.description)]) })),
                "required": .array(definition.inputSchema.required.map(JSONValue.string)),
                "additionalProperties": .bool(false),
            ])
            self.cacheControl = cacheControl
        }
    }

    enum CodingKeys: String, CodingKey {
        case name, description
        case inputSchema = "input_schema"
        case cacheControl = "cache_control"
    }
}

```



```

}

enum Content: Encodable {
    case text(String)
    case toolUse(id: String, name: String, input: JSONValue)
    case toolResult(id: String, content: String, isError: Bool)

    enum Keys: String, CodingKey { case type, text, id, name, input, toolUseID = "tool_use_id", content, isError = "is_error" }
    func encode(to encoder: Encoder) throws {
        var values = encoder.container(keyedBy: Keys.self)
        switch self {
        case let .text(text):
            try values.encode("text", forKey: .type); try values.encode(text, forKey: .text)
        case let .toolUse(id, name, input):
            try values.encode("tool_use", forKey: .type); try values.encode(id, forKey: .id); try values.encode(name, forKey: .name);
        ↪ try values.encode(input, forKey: .input)
        case let .toolResult(id, content, isError):
            try values.encode("tool_result", forKey: .type); try values.encode(id, forKey: .toolUseID); try values.encode(content,
        ↪ forKey: .content); try values.encode(isError, forKey: .isError)
        }
    }
}

let model: String
let maxTokens: Int
let stream: Bool
let system: String?
let messages: [Message]
let tools: [Tool]?

enum CodingKeys: String, CodingKey { case model, stream, system, messages, tools; case maxTokens = "max_tokens" }
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderRateScheduler.swift

```

import Foundation
import LingXiProtocol

public struct ProviderRetryPolicy: Sendable, Equatable, Codable {
    public let maxRetries: Int
    public let initialDelayMilliseconds: Int
    public let maxDelayMilliseconds: Int
    public let jitterRatio: Double

    public init(maxRetries: Int = 5, initialDelayMilliseconds: Int = 2_000, maxDelayMilliseconds: Int = 30_000, jitterRatio: Double = 0.25)
    ↪ {
        self.maxRetries = max(0, maxRetries)
        self.initialDelayMilliseconds = max(0, initialDelayMilliseconds)
        self.maxDelayMilliseconds = max(initialDelayMilliseconds, maxDelayMilliseconds)
        self.jitterRatio = min(1, max(0, jitterRatio))
    }

    private enum CodingKeys: String, CodingKey { case maxRetries, initialDelayMilliseconds, maxDelayMilliseconds, jitterRatio }

    public init(from decoder: Decoder) throws {
        let values = try decoder.container(keyedBy: CodingKeys.self)
        self.init(
            maxRetries: try values.decodeIfPresent(Int.self, forKey: .maxRetries) ?? 5,
            initialDelayMilliseconds: try values.decodeIfPresent(Int.self, forKey: .initialDelayMilliseconds) ?? 2_000,
            maxDelayMilliseconds: try values.decodeIfPresent(Int.self, forKey: .maxDelayMilliseconds) ?? 30_000,
            jitterRatio: try values.decodeIfPresent(Double.self, forKey: .jitterRatio) ?? 0.25
        )
    }
}

public struct ProviderRateLimits: Sendable, Equatable, Codable {

```

```

public let tpm: Int?
public let rpm: Int?
public let maxConcurrentRequests: Int?
public let retryPolicy: ProviderRetryPolicy

public init(tpm: Int? = nil, rpm: Int? = nil, maxConcurrentRequests: Int? = nil, retryPolicy: ProviderRetryPolicy = ProviderRetryPolicy
↳ ()) {
    self.tpm = tpm.flatMap { $0 > 0 ? $0 : nil }
    self.rpm = rpm.flatMap { $0 > 0 ? $0 : nil }
    self.maxConcurrentRequests = maxConcurrentRequests.flatMap { $0 > 0 ? $0 : nil }
    self.retryPolicy = retryPolicy
}

private enum CodingKeys: String, CodingKey { case tpm, rpm, maxConcurrentRequests, retryPolicy }

public init(from decoder: Decoder) throws {
    let values = try decoder.container(keyedBy: CodingKeys.self)
    self.init(
        tpm: try values.decodeIfPresent(Int.self, forKey: .tpm),
        rpm: try values.decodeIfPresent(Int.self, forKey: .rpm),
        maxConcurrentRequests: try values.decodeIfPresent(Int.self, forKey: .maxConcurrentRequests),
        retryPolicy: try values.decodeIfPresent(ProviderRetryPolicy.self, forKey: .retryPolicy) ?? ProviderRetryPolicy()
    )
}

public struct ProviderRateLimitError: Error, Sendable {
    public let statusCode: Int
    public let retryAfter: Duration?
    public let underlying: CoreError
    public let classified: ClassifiedProviderError

    public init(statusCode: Int, retryAfter: Duration? = nil, underlying: CoreError, classified: ClassifiedProviderError? = nil) {
        self.statusCode = statusCode
        self.retryAfter = retryAfter
        self.underlying = underlying
        self.classified = classified ?? ProviderErrorClassifier.classify(statusCode: statusCode, headers: [:], body: nil, underlying:
↳ underlying)
    }

    public static func from(statusCode: Int, headers: [String: String], body: String, underlying: CoreError) -> Error {
        let classified = ProviderErrorClassifier.classify(statusCode: statusCode, headers: headers, body: body, underlying: underlying)
        if classified.category.isRetryable {
            return ProviderRateLimitError(
                statusCode: statusCode,
                retryAfter: classified.retryAfter,
                underlying: underlying,
                classified: classified
            )
        } else {
            // 不可重试的永久错误，立即熔断并给出结构化诊断与排障建议
            var msg = classified.userFacingSummary
            let diag = classified.diagnostics ?? underlying.message
            if !diag.isEmpty && !diag.contains(classified.userFacingSummary) {
                msg += "\n[Wire Diagnostics]\n" + diag
            }
            return CoreError(code: .provider, message: msg)
        }
    }
}

public struct ProviderRateMetrics: Sendable, Equatable {
    public let retryCount: Int
    public let rateWaitMilliseconds: Int
    public let rateLimit429Count: Int

```

```

}

actor ProviderRateScheduler {
    static let shared = ProviderRateScheduler()

    private struct Key: Hashable {
        let provider: String
        let model: String
        let account: String?
    }

    private struct Workload {
        let requestID: ModelRequestID
        let timestamp: Date
        let tokens: Int
    }

    private var workloads: [Key: [Workload]] = [:]
    private var activeRequestIDs: [Key: Set<ModelRequestID>] = [:]
    private var blockedUntil: [Key: Date] = [:]
    private var metrics: [ModelRequestID: ProviderRateMetrics] = [:]

    func admit(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, estimatedTokens: Int) async throws {
        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        let limits = endpoint.rateLimits
        let clock = ContinuousClock()
        let started = clock.now
        var iterations = 0
        while true {
            try Task.checkCancellation()
            prune(key)
            if let blocked = blockedUntil[key], blocked > .now {
                try await Task.sleep(for: .milliseconds(max(1, Int(blocked.timeIntervalSinceNow * 1_000))))
                continue
            }
            let current = workloads[key] ?? []
            let concurrent = activeRequestIDs[key]?.count ?? 0
            let hasConcurrencyCapacity = concurrent < (limits.maxConcurrentRequests ?? .max)
            let hasRpmCapacity = current.count < (limits.rpm ?? .max)
            let hasTpmCapacity = canAdmitTokens(current, estimate: estimatedTokens, limit: limits.tpm)

            if hasConcurrencyCapacity && hasRpmCapacity && hasTpmCapacity {
                workloads[key, default: []].append(Workload(requestID: requestID, timestamp: .now, tokens: estimatedTokens))
                activeRequestIDs[key, default: []].insert(requestID)
                let elapsed = started.duration(to: clock.now).components
                let waited = elapsed.seconds * 1_000 + elapsed.attoseconds / 1_000_000_000_000_000
                if waited > 0 { addWait(requestID, milliseconds: Int(waited)) }
                return
            }

            iterations += 1
            // 积极放行与自愈: 当有空余并发槽位且无服务端 429 锁定, 但仅因本地 TPM 累加虚高被阻挡时, 在轻量交错退避 (<=40ms) 后积极放行, 绝不因
            ↪ 本地累加虚高阻塞多轮推理
            if hasConcurrencyCapacity && hasRpmCapacity && iterations > 2 && blockedUntil[key] == nil {
                workloads[key, default: []].append(Workload(requestID: requestID, timestamp: .now, tokens: estimatedTokens))
                activeRequestIDs[key, default: []].insert(requestID)
                let elapsed = started.duration(to: clock.now).components
                let waited = elapsed.seconds * 1_000 + elapsed.attoseconds / 1_000_000_000_000_000
                if waited > 0 { addWait(requestID, milliseconds: Int(waited)) }
                return
            }
        }

        if iterations > 30 && concurrent > 0 {
            activeRequestIDs[key]?.removeAll()
            continue
        }

        try await Task.sleep(for: waitDuration(current, concurrent: concurrent, limits: limits, estimate: estimatedTokens))
    }

```

```

    }
}

public func activeRequests(for endpoint: ResolvedModelEndpoint) -> Int {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    return activeRequestIDs[key]?.count ?? 0
}

public func release(endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    if var set = activeRequestIDs[key], !set.isEmpty {
        set.remove(set.first!)
        activeRequestIDs[key] = set
    }
}

public func release(requestID: ModelRequestID, endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    activeRequestIDs[key]?.remove(requestID)
}

public func cancel(requestID: ModelRequestID, endpoint: ResolvedModelEndpoint) {
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    activeRequestIDs[key]?.remove(requestID)
    workloads[key]?.removeAll { $0.requestID == requestID }
}

public func reset(endpoint: ResolvedModelEndpoint? = nil) {
    if let endpoint {
        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        activeRequestIDs.removeValue(forKey: key)
        workloads.removeValue(forKey: key)
        blockedUntil.removeValue(forKey: key)
    } else {
        activeRequestIDs.removeAll()
        workloads.removeAll()
        blockedUntil.removeAll()
    }
}

func recordRetry(requestID: ModelRequestID) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount + 1, rateWaitMilliseconds: current.rateWaitMilliseconds,
    ↪ rateLimit429Count: current.rateLimit429Count)
}

func recordRateLimit(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, cooldown: Duration) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount, rateWaitMilliseconds: current.rateWaitMilliseconds,
    ↪ rateLimit429Count: current.rateLimit429Count + 1)
    let components = cooldown.components
    let seconds = Double(components.seconds) + Double(components.attoseconds) / 1_000_000_000_000_000
    let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
    blockedUntil[key] = max(blockedUntil[key] ?? .distantPast, .now.addingTimeInterval(seconds))
}

func recordWait(requestID: ModelRequestID, duration: Duration) {
    let components = duration.components
    let milliseconds = Int(components.seconds * 1_000 + components.attoseconds / 1_000_000_000_000_000)
    if milliseconds > 0 { addWait(requestID, milliseconds: milliseconds) }
}

func recordUsage(endpoint: ResolvedModelEndpoint, requestID: ModelRequestID, usage: ModelUsage) {
    let actualTokens = [usage.inputTokens, usage.outputTokens, usage.reasoningTokens].compactMap { $0 }.reduce(0, +)
    guard actualTokens > 0 else { return }

```

```

        let key = Key(provider: endpoint.providerID, model: endpoint.modelID.rawValue, account: endpoint.accountID)
        workloads[key] = workloads[key, default: []].map {
            $0.requestID == requestID ? Workload(requestID: $0.requestID, timestamp: $0.timestamp, tokens: actualTokens) : $0
        }
    }

func metrics(for requestID: ModelRequestID) -> ProviderRateMetrics {
    metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
}

private func addWait(_ requestID: ModelRequestID, milliseconds: Int) {
    let current = metrics[requestID] ?? ProviderRateMetrics(retryCount: 0, rateWaitMilliseconds: 0, rateLimit429Count: 0)
    metrics[requestID] = ProviderRateMetrics(retryCount: current.retryCount, rateWaitMilliseconds: current.rateWaitMilliseconds +
        ↪ milliseconds, rateLimit429Count: current.rateLimit429Count)
}

private func prune(_ key: Key) {
    let cutoff = Date.now.addingTimeInterval(-60)
    workloads[key] = workloads[key, default: []].filter { $0.timestamp > cutoff }
    if (blockedUntil[key] ?? .distantPast) <= .now { blockedUntil[key] = nil }
}

private func canAdmitTokens(_ workloads: [Workload], estimate: Int, limit: Int?) -> Bool {
    guard let limit else { return true }
    return estimate > limit || workloads.reduce(0) { $0 + $1.tokens } + estimate <= limit
}

private func waitDuration(_ workloads: [Workload], concurrent: Int, limits: ProviderRateLimits, estimate: Int) -> Duration {
    if let maximum = limits.maxConcurrentRequests, concurrent >= maximum { return .milliseconds(10) }
    if let rpm = limits.rpm, workloads.count >= rpm, let oldest = workloads.map(\.timestamp).min() {
        let delayMs = min(20, max(1, Int(oldest.addingTimeInterval(60).timeIntervalSinceNow * 1_000)))
        return .milliseconds(delayMs)
    }
    if let tpm = limits.tpm, estimate <= tpm {
        var total = workloads.reduce(0) { $0 + $1.tokens }
        for workload in workloads.sorted(by: { $0.timestamp < $1.timestamp }) where total + estimate > tpm {
            total -= workload.tokens
            if total + estimate <= tpm {
                let delayMs = min(20, max(1, Int(workload.timestamp.addingTimeInterval(60).timeIntervalSinceNow * 1_000)))
                return .milliseconds(delayMs)
            }
        }
    }
    return .milliseconds(10)
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderErrorClassifier.swift

```

import Foundation
import LingXiProtocol

/// 标准化的模型提供商与上游网关错误分类。
public enum ProviderErrorCategory: String, Sendable, Codable, Equatable, CaseIterable {
    // MARK: - 瞬态错误（可自动指数退避重试）
    /// HTTP 429 或 TPM/RPM 配额耗尽
    case rateLimited
    /// HTTP 503 / 529 提供商服务暂时过载
    case serverOverloaded
    /// HTTP 408 / 504 / 522 / 524 或首字等待超时
    case gatewayTimeout
    /// HTTP 502 / 520 / 521 / 523 网关或反向代理异常
    case badGateway
    /// HTTP 500 提供商服务内部错误
    case internalServerError
}

```

```

/// 底层传输中断、连接重置、DNS 失败
case networkFailure

// MARK: - 永久性错误（不可重试，立即熔断）
/// HTTP 401: API Key / Access Token 无效或已过期
case authFailure
/// HTTP 402 / insufficient_quota: 余额不足或额度耗尽
case quotaExhausted
/// HTTP 403: 访问被拒绝、区域限制或账号风控
case accessForbidden
/// HTTP 404 / model_not_found: 模型、接口或资源不存在
case modelNotFound
/// HTTP 405: 请求方法不支持
case methodNotAllowed
/// HTTP 409: 资源状态冲突
case conflict
/// HTTP 413: 请求体或上下文过大超出单次限制
case payloadTooLarge
/// HTTP 415: 媒体格式或 Content-Type 不支持
case unsupportedMediaType
/// HTTP 400 (context_length_exceeded): 模型上下文长度超限
case contextExceeded
/// HTTP 400 / 422: 参数错误或不符合接口 Schema
case invalidRequest
/// HTTP 423: 账号或资源被临时锁定
case locked
/// content_filter / safety: 触发安全审查与敏感词拦截
case contentFiltered
/// HTTP 501: 功能未实现
case notImplemented
/// 未知异常
case unknown

/// 是否属于瞬态可重试错误。
public var isRetryable: Bool {
    switch self {
        case .rateLimited, .serverOverloaded, .gatewayTimeout, .badGateway, .internalServerError, .networkFailure:
            return true
        case .authFailure, .quotaExhausted, .accessForbidden, .modelNotFound, .methodNotAllowed,
            .conflict, .payloadTooLarge, .unsupportedMediaType, .contextExceeded, .invalidRequest,
            .locked, .contentFiltered, .notImplemented, .unknown:
            return false
    }
}

/// 用户可理解的中文诊断描述。
public var userDescription: String {
    switch self {
        case .rateLimited:
            return "上游请求速率超限 (429)"
        case .serverOverloaded:
            return "上游服务暂时过载 (529/503)"
        case .gatewayTimeout:
            return "上游网关或响应超时 (504/408)"
        case .badGateway:
            return "上游网关异常 (502)"
        case .internalServerError:
            return "上游服务内部错误 (500)"
        case .networkFailure:
            return "网络连接中断或握手失败"
        case .authFailure:
            return "API Key / Access Token 无效或已过期 (401)"
        case .quotaExhausted:
            return "账户余额不足或额度耗尽 (402)"
        case .accessForbidden:

```

```

        return "访问被拒绝 (403), 可能是账号风控或区域限制"
    case .modelNotFound:
        return "模型或接口资源不存在 (404)"
    case .methodNotAllowed:
        return "HTTP 请求方法不支持 (405)"
    case .conflict:
        return "资源状态冲突 (409)"
    case .payloadTooLarge:
        return "请求体或上下文过大超出单次限制 (413)"
    case .unsupportedMediaType:
        return "媒体格式不支持 (415)"
    case .contextExceeded:
        return "模型上下文长度超限 (400)"
    case .invalidRequest:
        return "请求参数错误或不符合接口规范 (400/422)"
    case .locked:
        return "账号或资源被临时锁定 (423)"
    case .contentFiltered:
        return "触发提供商内容安全审查风控拦截"
    case .notImplemented:
        return "上游接口未实现此功能 (501)"
    case .unknown:
        return "上游模型服务异常"
    }
}

```

/// 针对永久熔断错误给用户的行动建议。

```

public var suggestedAction: String? {
    switch self {
    case .authFailure:
        return "请检查提供商配置中的 API Key 是否有效或过期"
    case .quotaExhausted:
        return "请检查账户余额并前往服务商后台充值, 或切换至其他可用模型"
    case .accessForbidden:
        return "请检查当前网络地区或账号权限设置"
    case .modelNotFound:
        return "请检查模型名称配置是否正确, 或当前 API Key 是否有该模型权限"
    case .contextExceeded:
        return "上下文已达模型上限, 请执行 /compact 压缩会话或新建会话"
    case .payloadTooLarge:
        return "单次输入内容过大, 请精简输入或附件后重试"
    case .contentFiltered:
        return "请调整输入内容避免触碰提供商合规策略"
    case .invalidRequest:
        return "请求参数或工具调用定义不符合上游规范, 请检查配置"
    case .locked:
        return "账号或资源被锁定, 请登录服务商平台解锁"
    default:
        return nil
    }
}
}

```

/// 标准化分类后的模型提供商错误。

```

public struct ClassifiedProviderError: Error, Sendable {
    public let category: ProviderErrorCategory
    public let statusCode: Int?
    public let errorType: String?
    public let serverMessage: String?
    public let retryAfter: Duration?
    public let underlying: Error?
    public let diagnostics: String?

    public init(
        category: ProviderErrorCategory,

```

```

        statusCode: Int? = nil,
        errorType: String? = nil,
        serverMessage: String? = nil,
        retryAfter: Duration? = nil,
        underlying: Error? = nil,
        diagnostics: String? = nil
    ) {
        self.category = category
        self.statusCode = statusCode
        self.errorType = errorType
        self.serverMessage = serverMessage
        self.retryAfter = retryAfter
        self.underlying = underlying
        self.diagnostics = diagnostics
    }

    /// 面向用户的清晰诊断概要。
    public var userFacingSummary: String {
        var parts: [String] = []
        if let code = statusCode {
            parts.append("[\($code)]")
        }
        parts.append(category.userDescription)
        if let msg = serverMessage, !msg.isEmpty {
            parts.append(" · \($msg)")
        }
        if let action = category.suggestedAction {
            parts.append("(\\($action))")
        }
        return parts.joined(separator: " ")
    }
}

/// 模型错误分类器。
public enum ProviderErrorClassifier {
    /// 对 HTTP 状态码、响应体及底层错误进行全面分类。
    public static func classify(
        statusCode: Int?,
        headers: [String: String] = [:],
        body: String? = nil,
        underlying: Error? = nil,
        diagnostics: String? = nil
    ) -> ClassifiedProviderError {
        let (extractedType, extractedMsg) = parseJSONError(body)
        let normalizedBody = (body ?? "").lowercased()
        let normalizedMsg = (extractedMsg ?? "").lowercased()
        let normalizedType = (extractedType ?? "").lowercased()
        let retry = parseRetryAfter(headers: headers, body: body)

        // 1. 优先根据明确的业务错误特征识别（即使 HTTP 状态码不标准也能精准识别）
        if isRateLimited(statusCode: statusCode, type: normalizedType, message: normalizedMsg, body: normalizedBody) {
            return ClassifiedProviderError(
                category: .rateLimited,
                statusCode: statusCode ?? 429,
                errorType: extractedType ?? "rate_limit_exceeded",
                serverMessage: extractedMsg,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }

        if isContextLengthExceeded(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
            return ClassifiedProviderError(
                category: .contextExceeded,

```



```

        statusCode: statusCode ?? 400,
        errorType: extractedType ?? "context_length_exceeded",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isQuotaExceeded(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .quotaExhausted,
        statusCode: statusCode ?? 402,
        errorType: extractedType ?? "insufficient_quota",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isContentFiltered(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .contentFiltered,
        statusCode: statusCode ?? 400,
        errorType: extractedType ?? "content_filter",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

if isModelNotFound(type: normalizedType, message: normalizedMsg, body: normalizedBody) {
    return ClassifiedProviderError(
        category: .modelNotFound,
        statusCode: statusCode ?? 404,
        errorType: extractedType ?? "model_not_found",
        serverMessage: extractedMsg,
        retryAfter: nil,
        underlying: underlying,
        diagnostics: diagnostics
    )
}

// 2. 根据标准 HTTP 状态码分类
if let code = statusCode {
    switch code {
    case 400:
        return ClassifiedProviderError(
            category: .invalidRequest,
            statusCode: code,
            errorType: extractedType,
            serverMessage: extractedMsg,
            underlying: underlying,
            diagnostics: diagnostics
        )
    case 401:
        return ClassifiedProviderError(
            category: .authFailure,
            statusCode: code,
            errorType: extractedType ?? "unauthorized",
            serverMessage: extractedMsg,
            underlying: underlying,

```

```
        diagnostics: diagnostics
    )
case 402:
    return ClassifiedProviderError(
        category: .quotaExhausted,
        statusCode: code,
        errorType: extractedType ?? "payment_required",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 403:
    return ClassifiedProviderError(
        category: .accessForbidden,
        statusCode: code,
        errorType: extractedType ?? "forbidden",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 404:
    return ClassifiedProviderError(
        category: .modelNotFound,
        statusCode: code,
        errorType: extractedType ?? "not_found",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 405:
    return ClassifiedProviderError(
        category: .methodNotAllowed,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 408:
    return ClassifiedProviderError(
        category: .gatewayTimeout,
        statusCode: code,
        errorType: extractedType ?? "request_timeout",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 409:
    return ClassifiedProviderError(
        category: .conflict,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 413:
    return ClassifiedProviderError(
        category: .payloadTooLarge,
        statusCode: code,
        errorType: extractedType ?? "payload_too_large",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
```

```
)
case 415:
    return ClassifiedProviderError(
        category: .unsupportedMediaType,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 422:
    return ClassifiedProviderError(
        category: .invalidRequest,
        statusCode: code,
        errorType: extractedType ?? "unprocessable_entity",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 423:
    return ClassifiedProviderError(
        category: .locked,
        statusCode: code,
        errorType: extractedType ?? "locked",
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 429:
    return ClassifiedProviderError(
        category: .rateLimited,
        statusCode: code,
        errorType: extractedType ?? "rate_limit_exceeded",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 500:
    return ClassifiedProviderError(
        category: .internalServerError,
        statusCode: code,
        errorType: extractedType ?? "internal_server_error",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 501:
    return ClassifiedProviderError(
        category: .notImplemented,
        statusCode: code,
        errorType: extractedType,
        serverMessage: extractedMsg,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 502, 520, 521, 523:
    return ClassifiedProviderError(
        category: .badGateway,
        statusCode: code,
        errorType: extractedType ?? "bad_gateway",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
```

```

        diagnostics: diagnostics
    )
}
case 503, 529:
    return ClassifiedProviderError(
        category: .serverOverloaded,
        statusCode: code,
        errorType: extractedType ?? "service_unavailable",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
case 504, 522, 524:
    return ClassifiedProviderError(
        category: .gatewayTimeout,
        statusCode: code,
        errorType: extractedType ?? "gateway_timeout",
        serverMessage: extractedMsg,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
default:
    if code >= 500 {
        return ClassifiedProviderError(
            category: .badGateway,
            statusCode: code,
            errorType: extractedType,
            serverMessage: extractedMsg,
            retryAfter: retry,
            underlying: underlying,
            diagnostics: diagnostics
        )
    }
}
}

// 3. 检查底层网络/传输错误
if let underlying {
    let desc = underlying.localizedDescription.lowercased()
    if let coreErr = underlying as? CoreError {
        if coreErr.code == .commandTimedOut {
            return ClassifiedProviderError(
                category: .gatewayTimeout,
                statusCode: 408,
                errorType: "command_timed_out",
                serverMessage: coreErr.message,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }
        if coreErr.code == .transportLost {
            return ClassifiedProviderError(
                category: .networkFailure,
                statusCode: nil,
                errorType: "transport_lost",
                serverMessage: coreErr.message,
                retryAfter: retry,
                underlying: underlying,
                diagnostics: diagnostics
            )
        }
    }
}
if desc.contains("timed out") || desc.contains("timeout") {

```

```

        return ClassifiedProviderError(
            category: .gatewayTimeout,
            statusCode: 408,
            errorType: "timeout",
            serverMessage: underlying.localizedDescription,
            retryAfter: retry,
            underlying: underlying,
            diagnostics: diagnostics
        )
    }
}

if desc.contains("connection reset") || desc.contains("network connection lost") || desc.contains("cannot connect") {
    return ClassifiedProviderError(
        category: .networkFailure,
        statusCode: nil,
        errorType: "network_error",
        serverMessage: underlying.localizedDescription,
        retryAfter: retry,
        underlying: underlying,
        diagnostics: diagnostics
    )
}
}

// 4. 兜底
return ClassifiedProviderError(
    category: .unknown,
    statusCode: statusCode,
    errorType: extractedType,
    serverMessage: extractedMsg,
    retryAfter: retry,
    underlying: underlying,
    diagnostics: diagnostics
)
}

// MARK: - 辅助特征识别

private static func isContextLengthExceeded(type: String, message: String, body: String) -> Bool {
    type.contains("context_length_exceeded")
    || type.contains("context_window_exceeded")
    || message.contains("context length")
    || message.contains("maximum context length")
    || message.contains("context_length_exceeded")
    || message.contains("prompt is too long")
    || message.contains("tokens exceed")
    || body.contains("context_length_exceeded")
    || (body.contains("maximum context length") && body.contains("tokens"))
}

private static func isQuotaExceeded(type: String, message: String, body: String) -> Bool {
    guard !message.contains("rpm") && !message.contains("tpm") && !body.contains("rpm exhausted") && !body.contains("tpm exhausted")
    ↪ else {
        return false
    }
    return type.contains("insufficient_quota")
        || type.contains("quota_exceeded")
        || type.contains("credit_expired")
        || type.contains("billing_not_active")
        || message.contains("insufficient quota")
        || message.contains("exceeded your current quota")
        || message.contains("quota exceeded")
        || message.contains("balance insufficient")
        || message.contains("额度不足")
        || message.contains("余额不足")
        || message.contains("欠费")
}

```

```

        || body.contains("insufficient_quota")
    }

private static func isContentFiltered(type: String, message: String, body: String) -> Bool {
    type.contains("content_filter")
    || type.contains("sensitive_content")
    || message.contains("content filter")
    || message.contains("safety system")
    || message.contains("moderation")
    || message.contains("合规")
    || message.contains("违规")
    || body.contains("content_filter")
}

private static func isModelNotFound(type: String, message: String, body: String) -> Bool {
    type.contains("model_not_found")
    || type.contains("model_not_exists")
    || message.contains("model does not exist")
    || message.contains("model not found")
    || message.contains("no such model")
    || (type.contains("invalid_request_error") && message.contains("model"))
}

private static func isRateLimited(statusCode: Int?, type: String, message: String, body: String) -> Bool {
    statusCode == 429
    || type.contains("rate_limit")
    || type.contains("rate_limit_exceeded")
    || type.contains("tpm_exhausted")
    || type.contains("rpm_exhausted")
    || message.contains("rate limit")
    || message.contains("too many requests")
    || message.contains("tpm exhausted")
    || message.contains("rpm exhausted")
    || message.contains("rpm limit")
    || message.contains("tpm limit")
    || body.contains("429001")
    || (body.contains("inference") && body.contains("tpm") && body.contains("exhaust"))
    || (body.contains("inference") && body.contains("rpm") && body.contains("exhaust"))
}

private static func parseJSONError(_ body: String?) -> (type: String?, message: String?) {
    guard let body, !body.isEmpty, let data = body.data(using: .utf8) else { return (nil, nil) }
    guard let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any] else { return (nil, nil) }

    if let errorObj = json["error"] as? [String: Any] {
        let type = errorObj["type"] as? String ?? errorObj["code"] as? String
        let message = errorObj["message"] as? String
        return (type, message)
    } else if let errorMsg = json["error"] as? String {
        return (nil, errorMsg)
    } else if let message = json["message"] as? String {
        let code = json["code"] as? String ?? (json["code"] as? Int).map(String.init)
        return (code, message)
    }
    return (nil, nil)
}

private static func parseRetryAfter(headers: [String: String], body: String?) -> Duration? {
    if let value = headers.first(where: { $0.key.caseInsensitiveCompare("Retry-After-Ms") == .orderedSame })?.value.trimmingCharacters(
        ↪ in: .whitespacesAndNewlines), let milliseconds = Double(value), milliseconds >= 0 {
        return .milliseconds(Int(milliseconds))
    }
    if let value = headers.first(where: { $0.key.caseInsensitiveCompare("Retry-After") == .orderedSame })?.value.trimmingCharacters(in:
        ↪ .whitespacesAndNewlines) {
        if let seconds = Double(value), seconds >= 0 { return .milliseconds(Int(seconds * 1_000)) }
    }
}

```

```

        let formatter = DateFormatter()
        formatter.locale = Locale(identifier: "en_US_POSIX")
        formatter.timeZone = TimeZone(secondsFromGMT: 0)
        formatter.dateFormat = "EEE', ' dd MMM yyyy HH':'mm':'ss z"
        if let date = formatter.date(from: value) {
            return .milliseconds(max(0, Int(date.timeIntervalSinceNow * 1_000)))
        }
    }
}
return nil
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderConfig.swift

```

import Foundation
import LingXiProtocol

/// Provider 实例与已解析端点必须原子组装，endpoint 是运行时唯一真相。
public struct ModelRuntimeAssembly: Sendable {
    public let provider: any ModelProvider
    public let endpoint: ResolvedModelEndpoint
    public var modelID: ModelID { endpoint.modelID }
    public var contextProfile: ModelContextProfile { endpoint.contextProfile }

    public init(provider: any ModelProvider, modelID: ModelID, contextProfile: ModelContextProfile = ModelContextProfile(), endpoint:
        ↳ ResolvedModelEndpoint? = nil) {
        self.provider = provider
        self.endpoint = endpoint ?? ResolvedModelEndpoint(providerID: "default", modelID: modelID, baseURL: nil, wireProtocol: .
        ↳ chatCompletions, contextProfile: contextProfile)
    }

    public static let unavailable = ModelRuntimeAssembly(provider: UnavailableProvider(), modelID: ModelID(""))
}

private struct UnavailableProvider: ModelProvider {
    func stream(_ request: ModelRequest) async throws -> AsyncThrowingStream<ModelEvent, Error> {
        throw CoreError(code: .provider, message: "未配置模型 Provider")
    }
}

/// Provider 连接配置（仅存在于 Core 内部，不进入协议层）。
public struct ProviderConfig: Sendable {
    public let baseURL: URL
    public let authentication: ProviderAuthentication
    public let model: String
    public let wireProtocol: ModelWireProtocol
    public let diagnosticsEnabled: Bool
    public let performanceDiagnosticsEnabled: Bool
    public let remoteStateEnabled: Bool
    public let maxOutputTokens: Int?
    public let requiredHeaders: [String: String]
    public let parallelToolCalling: Bool?

    public init(
        baseURL: URL,
        apiKey: String?,
        model: String,
        wireProtocol: ModelWireProtocol = .chatCompletions,
        diagnosticsEnabled: Bool = false,
        performanceDiagnosticsEnabled: Bool = false,
        remoteStateEnabled: Bool = false,
        maxOutputTokens: Int? = nil,
        requiredHeaders: [String: String] = [:],
        parallelToolCalling: Bool? = true
    ) {

```

```

        self.baseURL = baseURL
        authentication = apiKey.map(ProviderAuthentication.bearer) ?? .none
        self.model = model
        self.wireProtocol = wireProtocol
        self.diagnosticsEnabled = diagnosticsEnabled
        self.performanceDiagnosticsEnabled = performanceDiagnosticsEnabled
        self.remoteStateEnabled = remoteStateEnabled
        self.maxOutputTokens = maxOutputTokens
        self.requiredHeaders = requiredHeaders
        self.parallelToolCalling = parallelToolCalling
    }

    public init(
        baseURL: URL,
        authentication: ProviderAuthentication,
        model: String,
        wireProtocol: ModelWireProtocol,
        diagnosticsEnabled: Bool = false,
        performanceDiagnosticsEnabled: Bool = false,
        remoteStateEnabled: Bool = false,
        maxOutputTokens: Int? = nil,
        requiredHeaders: [String: String] = [:],
        parallelToolCalling: Bool? = true
    ) {
        self.baseURL = baseURL
        self.authentication = authentication
        self.model = model
        self.wireProtocol = wireProtocol
        self.diagnosticsEnabled = diagnosticsEnabled
        self.performanceDiagnosticsEnabled = performanceDiagnosticsEnabled
        self.remoteStateEnabled = remoteStateEnabled
        self.maxOutputTokens = maxOutputTokens
        self.requiredHeaders = requiredHeaders
        self.parallelToolCalling = parallelToolCalling
    }

    /// OpenAI-compatible chat completions 端点。
    /// Base URL 已包含最终兼容层级（如 .../chat/completions）时不再追加，避免 /v1/v1。
    public var chatCompletionsURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/chat/completions") {
            return baseURL
        }
        return URL(string: trimmedString + "/chat/completions") ?? baseURL
    }

    public var responsesURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/responses") { return baseURL }
        return URL(string: trimmedString + "/responses") ?? baseURL
    }

    public var anthropicMessagesURL: URL {
        let trimmedString = baseURL.absoluteString.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
        if trimmedString.hasSuffix("/v1/messages") || trimmedString.hasSuffix("/messages") { return baseURL }
        return URL(string: trimmedString + "/v1/messages") ?? baseURL
    }
}

public enum ProviderAuthentication: Sendable, Equatable {
    case none
    case bearer(String)
    case header(name: String, value: String)
}

```



```

/// Wire protocol is selected by the resolved endpoint, not by a Provider brand.
public enum ModelWireProtocol: String, Sendable, Equatable, Codable {
    case chatCompletions
    case responses
    case anthropicMessages
}

public struct ResolvedModelEndpoint: Sendable, Equatable {
    public let providerID: String
    public let productID: String
    public let endpointID: String?
    public let accountID: String?
    public let profileID: String?
    public let modelID: ModelID
    public let baseURL: URL?
    public let wireProtocol: ModelWireProtocol
    public let contextProfile: ModelContextProfile
    public let capabilities: ModelCapabilities
    public let rateLimits: ProviderRateLimits

    public init(providerID: String, productID: String? = nil, endpointID: String? = nil, accountID: String? = nil, profileID: String? = nil,
        ↪ modelID: ModelID, baseURL: URL?, wireProtocol: ModelWireProtocol, contextProfile: ModelContextProfile = ModelContextProfile(),
        ↪ capabilities: ModelCapabilities = ModelCapabilities(), rateLimits: ProviderRateLimits = ProviderRateLimits()) {
        self.providerID = providerID
        self.productID = productID ?? providerID
        self.endpointID = endpointID
        self.accountID = accountID
        self.profileID = profileID
        self.modelID = modelID
        self.baseURL = baseURL
        self.wireProtocol = wireProtocol
        self.contextProfile = contextProfile
        self.capabilities = capabilities
        self.rateLimits = rateLimits
    }
}

public struct ModelCapabilities: Codable, Sendable, Equatable {
    public let toolCalling: Bool?
    public let parallelToolCalling: Bool?
    public let reasoning: Bool?
    public let vision: Bool?
    public let structuredOutput: Bool?
    public let reasoningCapability: ReasoningCapability?

    public init(toolCalling: Bool? = nil, parallelToolCalling: Bool? = nil, reasoning: Bool? = nil, vision: Bool? = nil, structuredOutput:
        ↪ Bool? = nil, reasoningCapability: ReasoningCapability? = nil) {
        self.toolCalling = toolCalling
        self.parallelToolCalling = parallelToolCalling
        self.reasoning = reasoning
        self.vision = vision
        self.structuredOutput = structuredOutput
        self.reasoningCapability = reasoningCapability
    }
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderHTTPTransport.swift

```

import Foundation
#if canImport(FoundationNetworking)
import FoundationNetworking
#endif
import LingXiProtocol

public struct ProviderHTTPRequestContext: Sendable, Equatable {

```

```

public let wireProtocol: ModelWireProtocol
public let model: String
public let requestID: ModelRequestID
public let executionID: AgentRunID?
public let step: Int

public init(wireProtocol: ModelWireProtocol, model: String, requestID: ModelRequestID = ModelRequestID(), executionID: AgentRunID?, step
↵ : Int) {
    self.wireProtocol = wireProtocol
    self.model = model
    self.requestID = requestID
    self.executionID = executionID
    self.step = step
}
}

public struct ProviderHTTPResponse: Sendable {
    public let statusCode: Int
    public let headers: [String: String]
    public let body: AsyncThrowingStream<Data, Error>

    public init(statusCode: Int, headers: [String: String] = [:], body: AsyncThrowingStream<Data, Error>) {
        self.statusCode = statusCode
        self.headers = headers
        self.body = body
    }

    public func header(_ name: String) -> String? {
        headers.first { $0.key.caseInsensitiveCompare(name) == .orderedSame }?.value
    }
}

public protocol ProviderHTTPTransport: Sendable {
    func send(_ request: URLRequest, context: ProviderHTTPRequestContext) async throws -> ProviderHTTPResponse
}

public struct URLSessionProviderHTTPTransport: ProviderHTTPTransport {
    public static let sharedSession: URLSession = {
        let config = URLSessionConfiguration.default
        config.httpMaximumConnectionsPerHost = 16
        config.timeoutIntervalForRequest = 45
        config.timeoutIntervalForResource = 300
        config.waitsForConnectivity = false
        return URLSession(configuration: config)
    }()

    private let session: URLSession

    public init(session: URLSession = URLSessionProviderHTTPTransport.sharedSession) {
        self.session = session
    }

    public func send(_ request: URLRequest, context: ProviderHTTPRequestContext) async throws -> ProviderHTTPResponse {
        let bytes: URLSession.AsyncBytes
        let response: URLResponse
        do {
            (bytes, response) = try await session.bytes(for: request)
        } catch is CancellationError {
            throw CancellationError()
        } catch let error as URLError where error.code == .timedOut {
            throw CoreError(code: .commandTimedOut, message: "Provider request timed out")
        } catch let error as URLError where error.code == .cancelled {
            throw CancellationError()
        } catch {
            if Task.isCancelled { throw CancellationError() }
        }
    }
}

```

```

        let detail = (error as? URLError)?.localizedDescription ?? error.localizedDescription
        throw CoreError(code: .transportLost, message: "Provider transport failed: \(detail)")
    }
    guard let http = response as? HTTPURLResponse else {
        throw CoreError(code: .provider, message: "Provider 返回非 HTTP 响应")
    }
    let body = AsyncThrowingStream<Data, Error> { continuation in
        let pump = Task {
            do {
                var chunk = Data()
                for try await byte in bytes {
                    if Task.isCancelled { throw CancellationError() }
                    chunk.append(byte)
                    if byte == 0x0A || chunk.count >= 4_096 {
                        continuation.yield(chunk)
                        chunk.removeAll(keepingCapacity: true)
                    }
                }
                if !chunk.isEmpty {
                    continuation.yield(chunk)
                }
                continuation.finish()
            } catch {
                continuation.finish(throwing: error)
            }
        }
        continuation.onTermination = { @Sendable _ in
            pump.cancel()
            bytes.task.cancel()
        }
    }
    return ProviderHTTPResponse(
        statusCode: http.statusCode,
        headers: Dictionary(uniqueKeysWithValues: http.allHeaderFields.compactMap { key, value in
            guard let key = key as? String else { return nil }
            return (key, String(describing: value))
        }),
        body: body
    )
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderActivityRegistry.swift

```

import Foundation
import LingXiProtocol

public actor ProviderActivityRegistry {
    public static let shared = ProviderActivityRegistry()

    private var activities: [String: ProviderActivitySnapshot] = [:] // providerRequestID -> snapshot
    private var cancelledRequestIDs: Set<String> = []
    private var cancelledRunIDs: Set<AgentRunID> = []

    public init() {}

    @discardableResult
    public func record(
        sessionID: SessionID,
        runID: AgentRunID?,
        providerRequestID: String,
        state: ProviderActivityState,
        model: String? = nil,
        detail: String? = nil,
        statusCode: Int? = nil
    )

```

```

) -> ProviderActivitySnapshot {
    let effectiveState: ProviderActivityState
    if let runID, cancelledRunIDs.contains(runID) {
        effectiveState = .cancelled
    } else if cancelledRequestIDs.contains(providerRequestID) && !state.isTerminal {
        effectiveState = .cancelled
    } else {
        effectiveState = state
    }
    let snapshot = ProviderActivitySnapshot(
        sessionID: sessionID,
        runID: runID,
        providerRequestID: providerRequestID,
        state: effectiveState,
        model: model,
        detail: detail,
        statusCode: statusCode,
        updatedAt: .now
    )
    activities[providerRequestID] = snapshot
    return snapshot
}

@discardableResult
public func cancel(providerRequestID: String) -> ProviderActivitySnapshot? {
    guard !providerRequestID.isEmpty else { return nil }
    cancelledRequestIDs.insert(providerRequestID)
    guard var snapshot = activities[providerRequestID] else { return nil }
    snapshot = ProviderActivitySnapshot(
        sessionID: snapshot.sessionID,
        runID: snapshot.runID,
        providerRequestID: snapshot.providerRequestID,
        state: .cancelled,
        model: snapshot.model,
        detail: snapshot.detail,
        statusCode: snapshot.statusCode,
        updatedAt: .now
    )
    activities[providerRequestID] = snapshot
    return snapshot
}

@discardableResult
public func cancelRun(_ runID: AgentRunID) -> [ProviderActivitySnapshot] {
    cancelledRunIDs.insert(runID)
    var updated: [ProviderActivitySnapshot] = []
    for (id, snapshot) in activities where snapshot.runID == runID && !snapshot.state.isTerminal {
        if !id.isEmpty {
            cancelledRequestIDs.insert(id)
        }
        let newSnapshot = ProviderActivitySnapshot(
            sessionID: snapshot.sessionID,
            runID: snapshot.runID,
            providerRequestID: snapshot.providerRequestID,
            state: .cancelled,
            model: snapshot.model,
            detail: snapshot.detail,
            statusCode: snapshot.statusCode,
            updatedAt: .now
        )
        activities[id] = newSnapshot
        updated.append(newSnapshot)
    }
    return updated
}

```

```

public func isCancelled(providerRequestID: String, runID: AgentRunID?) -> Bool {
    if !providerRequestID.isEmpty && cancelledRequestIDs.contains(providerRequestID) { return true }
    if let runID, cancelledRunIDs.contains(runID) { return true }
    return false
}

public func isRunCancelled(_ runID: AgentRunID) -> Bool {
    cancelledRunIDs.contains(runID)
}

public func activeActivities(for sessionID: SessionID) -> [ProviderActivitySnapshot] {
    activities.values.filter { $0.sessionID == sessionID && !$0.state.isTerminal }
}

public func latestActivity(for runID: AgentRunID) -> ProviderActivitySnapshot? {
    activities.values.filter { $0.runID == runID }.max(by: { $0.updatedAt < $1.updatedAt })
}

public func reset() {
    activities.removeAll()
    cancelledRequestIDs.removeAll()
    cancelledRunIDs.removeAll()
}
}

```

// File: Sources/LingXiCore/Modules/Model/ProviderProvenance.swift

```

import Foundation
import LingXiProtocol

/// Provider 原生调用身份只存在于 Model Adapter 边界，不能成为 LingXi ToolCallID。
public struct ProviderToolCallReference: Sendable, Equatable, Codable {
    public let wire: ModelWireProtocol
    public let domainCallID: ToolCallID
    public let externalCallID: String
    public let externalItemID: String?

    public init(wire: ModelWireProtocol, domainCallID: ToolCallID, externalCallID: String, externalItemID: String? = nil) {
        self.wire = wire
        self.domainCallID = domainCallID
        self.externalCallID = externalCallID
        self.externalItemID = externalItemID
    }
}

public enum ProviderContinuationItem: Sendable, Equatable, Codable {
    /// 原始 Provider item JSON; Core 不解释或改写其中的 opaque content。
    case opaque(Data)
    case toolCall(ToolCallID)
}

public struct ProviderContinuation: Sendable, Equatable, Codable {
    public let requestID: ModelRequestID
    public let executionID: AgentRunID?
    public let wire: ModelWireProtocol
    public let responseID: String?
    public let references: [ProviderToolCallReference]
    public let orderedItems: [ProviderContinuationItem]

    public init(requestID: ModelRequestID, executionID: AgentRunID?, wire: ModelWireProtocol, responseID: String? = nil, references: [
        ⇐ ProviderToolCallReference], orderedItems: [ProviderContinuationItem] = []) {
        self.requestID = requestID
        self.executionID = executionID
        self.wire = wire
    }
}

```

```

        self.responseID = responseID
        self.references = references
        self.orderedItems = orderedItems
    }

    public func externalCallID(for domainCallID: ToolCallID) -> String? {
        references.first { $0.domainCallID == domainCallID }?.externalCallID
    }
}

/// Entries are keyed by one model request, so concurrent Main/Child decoders cannot share correlation state.
public actor ProviderProvenanceStore {
    private var entries: [ModelRequestID: ProviderContinuation] = [:]
    private var referencesByDomainID: [ToolCallID: ProviderToolCallReference] = [:]
    private let directory: URL

    public init(directory: URL? = nil) {
        self.directory = directory ?? FileManager.default.temporaryDirectory.appendingPathComponent("lingxi-provider-provenance-\(UUID().
        ↪ uuidString)", isDirectory: true)
        try? FileManager.default.createDirectory(at: self.directory, withIntermediateDirectories: true)
        try? FileManager.default.setAttributes([.posixPermissions: 0o700], ofItemAtPath: self.directory.path)
    }

    public func continuation(for requestID: ModelRequestID?, wire: ModelWireProtocol) throws -> ProviderContinuation? {
        guard let requestID else { return nil }
        let url = file(requestID)
        let value: ProviderContinuation?
        if let cached = entries[requestID] { value = cached }
        else if FileManager.default.fileExists(atPath: url.path) { value = try JSONDecoder().decode(ProviderContinuation.self, from: Data(
        ↪ contentsOf: url)) }
        else { value = nil }
        guard let value, value.wire == wire else { return nil }
        entries[requestID] = value
        for ref in value.references { referencesByDomainID[ref.domainCallID] = ref }
        return value
    }

    public func resolveContinuation(for requestID: ModelRequestID?, wire: ModelWireProtocol) throws -> ProviderContinuation? {
        let base = try continuation(for: requestID, wire: wire)
        var combined = base?.references ?? []
        for (domainID, ref) in referencesByDomainID where ref.wire == wire {
            if !combined.contains(where: { $0.domainCallID == domainID }) {
                combined.append(ref)
            }
        }
        if let base {
            return ProviderContinuation(requestID: base.requestID, executionID: base.executionID, wire: base.wire, responseID: base.
            ↪ responseID, references: combined, orderedItems: base.orderedItems)
        } else if !combined.isEmpty {
            return ProviderContinuation(requestID: requestID ?? ModelRequestID(), executionID: nil, wire: wire, responseID: nil, references:
            ↪ combined, orderedItems: [])
        }
        return nil
    }

    public func record(_ continuation: ProviderContinuation) throws {
        let url = file(continuation.requestID)
        try JSONEncoder().encode(continuation).write(to: url, options: .atomic)
        try FileManager.default.setAttributes([.posixPermissions: 0o600], ofItemAtPath: url.path)
        entries[continuation.requestID] = continuation
        for ref in continuation.references {
            referencesByDomainID[ref.domainCallID] = ref
        }
    }
}

```

```

public func remove(_ executionID: AgentRunID) {
    entries = entries.filter { $0.value.executionID != executionID }
    for url in files() {
        guard let data = try? Data(contentsOf: url), let value = try? JSONDecoder().decode(ProviderContinuation.self, from: data), value
        ↪ .executionID == executionID else { continue }
        try? FileManager.default.removeItem(at: url)
    }
}

private func files() -> [URL] { (try? FileManager.default.contentsOfDirectory(at: directory, includingPropertiesForKeys: nil)) ?? [] }
private func file(_ requestID: ModelRequestID) -> URL { directory.appendingPathComponent(sha256Hex(requestID.rawValue) + ".json") }
}

```

// File: Sources/LingXiCore/Modules/Model/CanonicalCachePlan.swift

```

import Foundation
import LingXiProtocol

```

/// 跨 Provider、跨 Model、跨 Protocol 的统一缓存结构计划。
 /// 在任何 Provider wire encoding 之前建立，各 Provider Adapter 只能将该 Plan 序列化为对应协议，
 /// 严禁 Adapter 自行决定哪些内容是 Stable 还是 Volatile。

```

public struct CanonicalCachePlan: Sendable, Equatable {
    public struct EpochIdentity: Sendable, Equatable {
        public let epoch: Int
        public let reason: String

        public init(epoch: Int, reason: String = "initial") {
            self.epoch = epoch
            self.reason = reason
        }
    }
}

```

```

public struct ImmutableBase: Sendable, Equatable {
    public let systemPrompt: String?
    public let developerPrompt: String?
    public let coreTools: [ToolDefinition]
    public let stablePolicy: String?

    public init(
        systemPrompt: String? = nil,
        developerPrompt: String? = nil,
        coreTools: [ToolDefinition] = [],
        stablePolicy: String? = nil
    ) {
        self.systemPrompt = systemPrompt
        self.developerPrompt = developerPrompt
        self.coreTools = coreTools
        self.stablePolicy = stablePolicy
    }
}

```

```

public struct AppendOnlyContext: Sendable, Equatable {
    public let dynamicTools: [ToolDefinition]
    public let messages: [ModelMessage]
    public let skillActivations: [String]

    public init(
        dynamicTools: [ToolDefinition] = [],
        messages: [ModelMessage] = [],
        skillActivations: [String] = []
    ) {
        self.dynamicTools = dynamicTools
        self.messages = messages
        self.skillActivations = skillActivations
    }
}

```

```

    }

    public struct VolatileTail: Sendable, Equatable {
        public let currentTurnState: String?
        public let ephemeralNotes: String?

        public init(currentTurnState: String? = nil, ephemeralNotes: String? = nil) {
            self.currentTurnState = currentTurnState
            self.ephemeralNotes = ephemeralNotes
        }
    }

    public let epochIdentity: EpochIdentity
    public let immutableBase: ImmutableBase
    public let appendOnlyContext: AppendOnlyContext
    public let volatileTail: VolatileTail
    public let structuralHealth: ClientStructuralCacheHealth
    public let capabilities: ProviderCacheCapabilities?

    public init(
        epochIdentity: EpochIdentity,
        immutableBase: ImmutableBase,
        appendOnlyContext: AppendOnlyContext,
        volatileTail: VolatileTail = VolatileTail(),
        structuralHealth: ClientStructuralCacheHealth,
        capabilities: ProviderCacheCapabilities? = nil
    ) {
        self.epochIdentity = epochIdentity
        self.immutableBase = immutableBase
        self.appendOnlyContext = appendOnlyContext
        self.volatileTail = volatileTail
        self.structuralHealth = structuralHealth
        self.capabilities = capabilities
    }
}

```

// File: Sources/LingXiCore/Modules/Model/SSEDecoder.swift

```

import Foundation

/// SSE 行缓冲解码器。
/// 网络字节按任意大小的块喂入，按完整行输出；
/// 不假设一个 TCP chunk 对应一个 SSE event，处理 \n 与 \r\n。
public struct SSEDecoder {
    private var buffer: [UInt8] = []

    public init() {}

    /// 喂入任意网络块，返回其中已完整到达的行（不含行终止符）。
    public mutating func feed(_ data: Data) -> [String] {
        data.withUnsafeBytes { raw in
            buffer.append(contentsOf: raw.bindMemory(to: UInt8.self))
        }
        return extractLines()
    }

    /// 流结束时调用：输出残留的最后一行（SSE 允许末行无换行）。
    public mutating func flushPending() -> [String] {
        guard !buffer.isEmpty else { return [] }
        let line = String(decoding: buffer, as: UTF8.self)
        buffer.removeAll()
        return [line]
    }

    private mutating func extractLines() -> [String] {

```



```
var lines: [String] = []
var lineStart = buffer.startIndex
var index = buffer.startIndex

while index < buffer.endIndex {
    guard buffer[index] == UInt8(ascii: "\n") else {
        index += 1
        continue
    }
    var end = index
    if end > lineStart, buffer[end - 1] == UInt8(ascii: "\r") {
        end -= 1
    }
    lines.append(String(decoding: buffer[lineStart..
```